User Guide

Corezoid Actor Engine

2025 © Corezoid Inc.
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved

Visit corezoid.com for details

Table of contents

Table of contents **2**
1. Introduction to Corezoid **6**
1.1 Overview of Corezoid 6
1.2 Advantages of the Corezoid Platform 7
1.3 Typical Use Cases 7
2. Work Start and Core Concepts **8**
2.1 Registration and Account Setup 8
2.2 Interface Overview 9
2.2.1 Workspace tab 10
2.2.2 Activity Monitor 12
2.2.3 Users & Groups Tab 14
2.2.4 Docs Tab 17
2.2.5 User Profile 18
View and edit your profile information 19
View and change your license parameters 20
2.2.6 Process Editor 25
2.2.7 Sharing Settings Menu 31
2.3 Core Concepts **33**
2.3.1 Nodes 33
2.3.2 Tasks 38
REF parameter 39
2.3.2.1 Task Adding 40
Send Task Manually 41
2.3.2.2 Task Parameters 46
2.3.2.3 Task Management 55
2.3.3 Processes 61
2.3.4 State Diagrams 62
2.3.5 Parameters 63
Task parameters **63**
Local parameters 64
Getting task parameters from State Diagram 65
Node parameter examples 67
2.3.6 Functions 70
2.3.7 Dashboards 78
2.3.8 Projects 85
2.3.8.1 Stages 91
2.3.8.2 Aliases 100
2.3.8.3 Variables 106
2.3.8.4 Versions 117

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

2.3.8.5 Merge 121
2.3.9 Notation Schema for Corezoid Node Parameters 128
3. Nodes Description 129
3.1 Start 129
3.2 End 140
3.3 Condition 145
3.4 Delay 159
3.5 Set Parameters 167
3.6 Code 178
3.7 Git Call 201
3.8 API Call 231
3.9 Database Call 251
3.10 Queue 265
3.11 Get from Queue 274
3.12 Copy Task 281
3.13 Call a Process 296
3.14 Reply to Process 309
3.15 Set State 323
3.16 Waiting for Callback 333
3.17 Modify Task 342
3.18 Sum 354
3.19 Parallel Processing Node (Coming Soon) 362
4. Deployment Options 363
4.1 VMware 364
4.2 VMware Fusion 379
4.3 Deploy Corezoid on Google Cloud Engine 391
4.4 Deploy Corezoid on Azure Marketplace 402
4.5 Deploy Corezoid on AWS Marketplace 409
5. Corezoid Processes Guide 421
5.1 Building Processes 421
5.2 Debugging a Process 426
6. Advanced Features 432
7. Communications Orchestrator 434
Creating Bots in Messengers and Connecting Them to Your Communications Orchestrator 436
8. Tutorials 504
Google OAuth 504
1. Create project on the Google Cloud Console 504
2. Enable Google OAuth API 506
3. Learn requirements and flowchart for queries to Google APIs 508
4. Create OAuth 2.0 project folder in Corezoid 509

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

5. Create Google credentials 510
6. Obtain Google Code 517
7. Create Process for Access Token Generation 518
8. Create and configure API Call node in token generation process 520
9. Generate Google Access Token 523
10. Create Refresh token process 523
11. Configure token refresh 523
12. Create and configure state diagram for access token storage 524
9. References 539
9.1 Simulator documentation 539
9.2 Customer Agreement 539
A. Definitions 539
B. Rights granted 541
C. Ownership and restrictions 542
Ownership 542
Restrictions 542
D. Fees, taxes, and payment 542
E. Disclaimers 543
F. Indemnification 543
G. Limitation of liability 544
H. Term, Termination, and Suspension 545
I. Miscellaneous 546
9.3 Privacy Policy 550
Information collected 550
Geolocation 552
Tracking technologies 552
Choice 552
Use and disclosure of information 553
Sharing your personal information 553
Security of information 554
Data subject rights 555
Your California privacy rights 556
Coppa and access by minors 556
Changes to this privacy policy 556
User consent 556
Contact us 557
9.4 Cookie Policy 557
What are cookies and does Corezoid use them? 557
How to control cookies 559
Changes to this privacy policy 560
Contact us 560

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

9.5 Send Notification Emails 560
List of Corezoid events for which notification emails are sent 560
Process example 564
Adding Process ID to settings 568
9.6 Notifying users on process blockin/deblocking 570
Overview 570
Process example 571
Adding Process ID to settings 573
10. Additional resources **574**
11. Release Notes

12. Appendices **575**
12.1 Corezoid Glossary of Terms 575
12.2 Configuration Examples 578
12.3 API Reference 579
API methods use examples 582
12.4 Error Codes and Troubleshooting 587
12.5 Integration Guides 596
12.6 Security Best Practices 596
12.7 Configuration Notes 597
12.8 Performance Tips 609
12.9 User Agreements and Policies 609
12.10 Code Samples 609
12.11 Corezoid SDK 609

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

1. Introduction to Corezoid

1.1 Overview of Corezoid

Corezoid is a cloud-based actor engine designed to facilitate the creation, hosting, and execution of algorithms of varying complexity. Its versatile capabilities allow users to build solutions such as:

- chatbots
- communication workflows
- CRM functionalities
- omnichannel marketing strategies
- hardware monitoring
- anti-fraud systems
- financial management tools.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

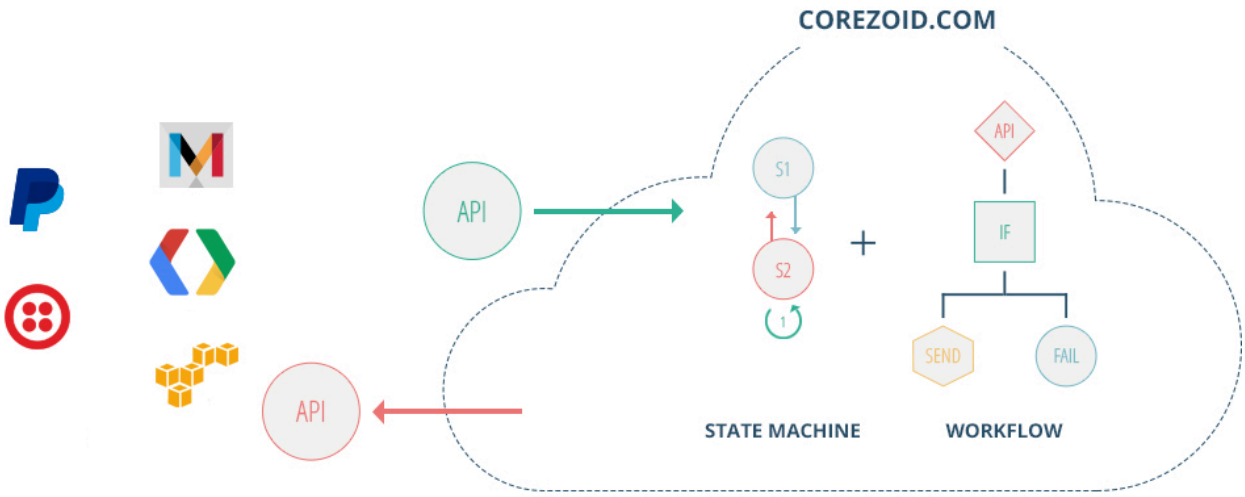


Diagram 1: **Corezoid general operation scheme**

The platform can be seen as a "meta-programming language" or "glue for APIs," enabling seamless integration of systems regardless of their original programming language (e.g., PHP, Python, Java, C++). Corezoid simplifies the orchestration of Processes by connecting data flows from multiple APIs into a unified logic structure.

Notable features of Corezoid include:

- Rapid deployment and scalability in the cloud.
- Support for API-driven interactions with systems.
- A shift from code-intensive development to Process assembly by non-technical users.
- An intuitive interface that bridges the gap between business analysts, managers, and developers.

1.2 Advantages of the Corezoid Platform

Corezoid offers a range of benefits to users, including:

1. **Speed and Efficiency**: The platform accelerates time-to-market by simplifying the development process, enabling non-technical users to configure and manage workflows without extensive coding expertise.
2. **Cost Reduction**: By reducing reliance on software engineers for routine process adjustments, Corezoid minimizes maintenance and development costs.
3. **Scalability**: Corezoid seamlessly scales with the complexity of workflows, from simple automations to enterprise-level orchestration involving hundreds of APIs.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

4. **Real-Time Monitoring**: Built-in dashboards and alerts provide actionable insights into ongoing Processes.
5. **Ease of Integration**: Compatible with a wide range of programming languages and systems, Corezoid simplifies interconnectivity within diverse IT ecosystems.

1.3 Typical Use Cases

Corezoid has been employed successfully across various industries and scenarios, including but not limited to:

- **Customer Support**: Building intelligent chatbots and automated communication systems for real-time customer engagement.
- **Marketing Automation**: Designing and managing omnichannel marketing campaigns with dynamic triggers and responses.
- **Operational Efficiency**: Developing workflow automation for internal processes, reducing manual tasks, and enhancing productivity.
- **Fraud Detection**: Implementing systems to monitor, flag, and respond to suspicious activities in real time.
- **Financial Applications**: Creating and managing financial monitoring and reporting tools.
- **Hardware Monitoring**: Establishing robust systems to track, analyze, and maintain hardware performance across distributed networks.

With its extensive capabilities, Corezoid serves as a powerful platform for businesses seeking to optimize operations, improve agility, and innovate without the constraints of traditional software development.

Corezoid on FinancesOnline.

Corezoid on SourceForge.

2. Work Start and Core Concepts

2.1 Registration and Account Setup

To begin using Corezoid, follow these steps to register and set up your account:

1. **Sign Up**: Visit the Corezoid website and create an account using your email address.
2. **Account Verification**: Check your email for a verification link and confirm your registration.
3. **Initial Configuration**:
    - **Workspace**: Familiarize yourself with the **Workspace**, **Activity Monitor**, and **Users & Groups** tabs, **Create** button and toolbars.
    - **API Keys**: Generate API keys to enable system integrations and secure access.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- **Profile Settings**: Customize your profile, language preferences, and notifications.

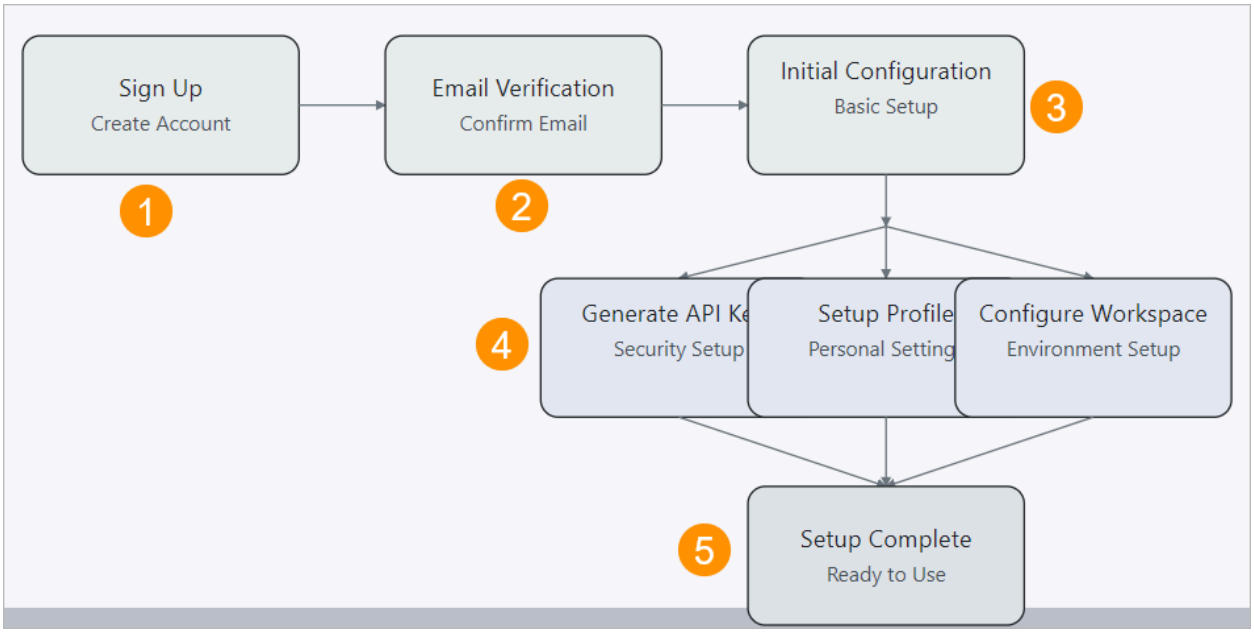
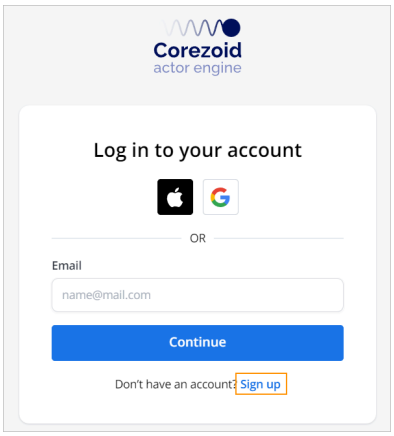
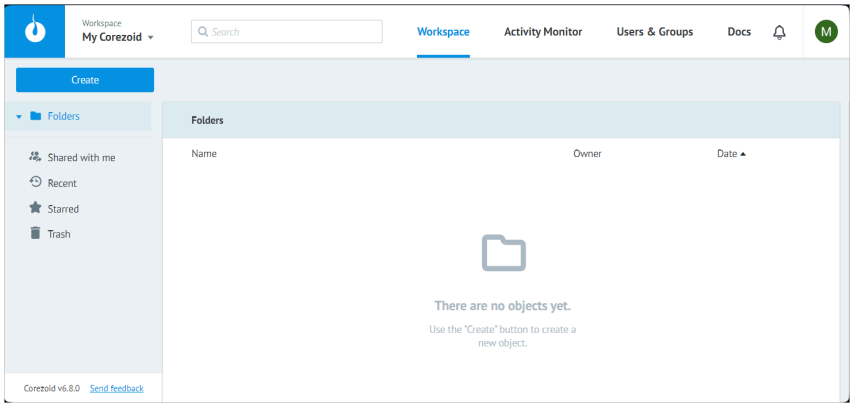


Diagram 2: **Registration and configuration flow**.



Sign up form



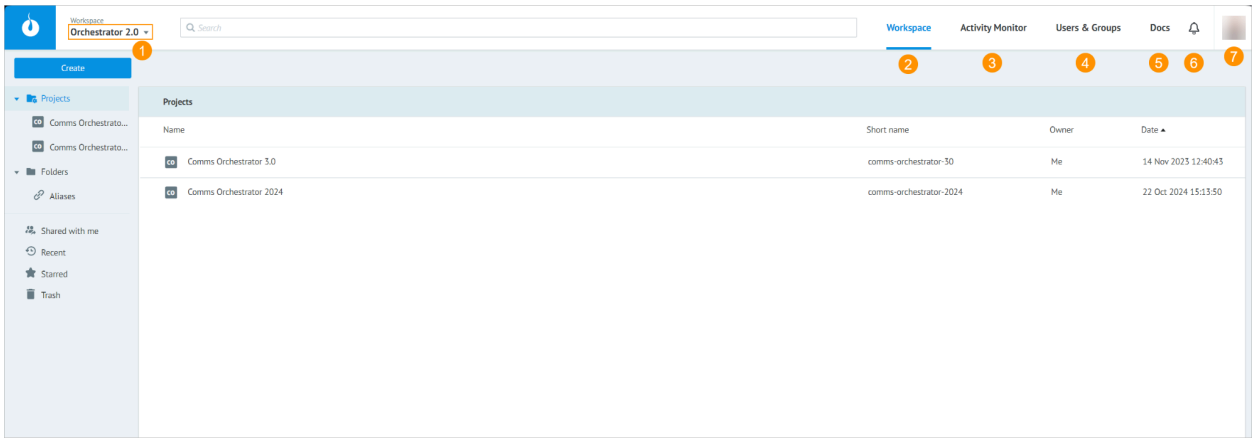
Your Corezoid workspace after logging in

Screenshot 1: **Illustration of the registration procedure and main dashboard view after logging in**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

2.2 Interface Overview

Use Corezoid UI to manage workflows and monitor Processes through key features available in tabs and panel:



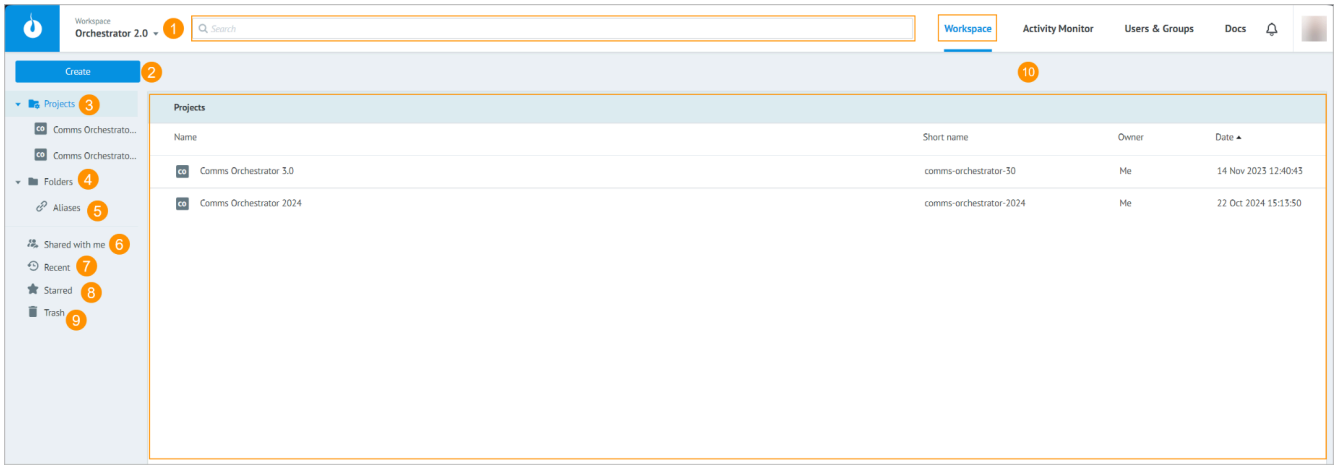
Screenshot 2: **Corezoid main menu**.

1. **Workspace selection list:** select a Corezoid Workspace.
2. **Workspace tab:** create and manage projects, Processes, dashboards, and State Diagrams.
3. **Activity Monitor tab:** get real-time info on task execution for the specified period broken down by Processes.
4. **Users & Groups tab:** view and manage users and user groups.
5. **Docs:** view the Corezoid documentation.
6. **Notifications:** view system notifications.
7. **Profile:** manage your profile.

2.2.1 Workspace tab

The **Workspace** is the tab for creating and managing your Corezoid objects, including Projects, Processes, and Dashboards. Here, you have the following items:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

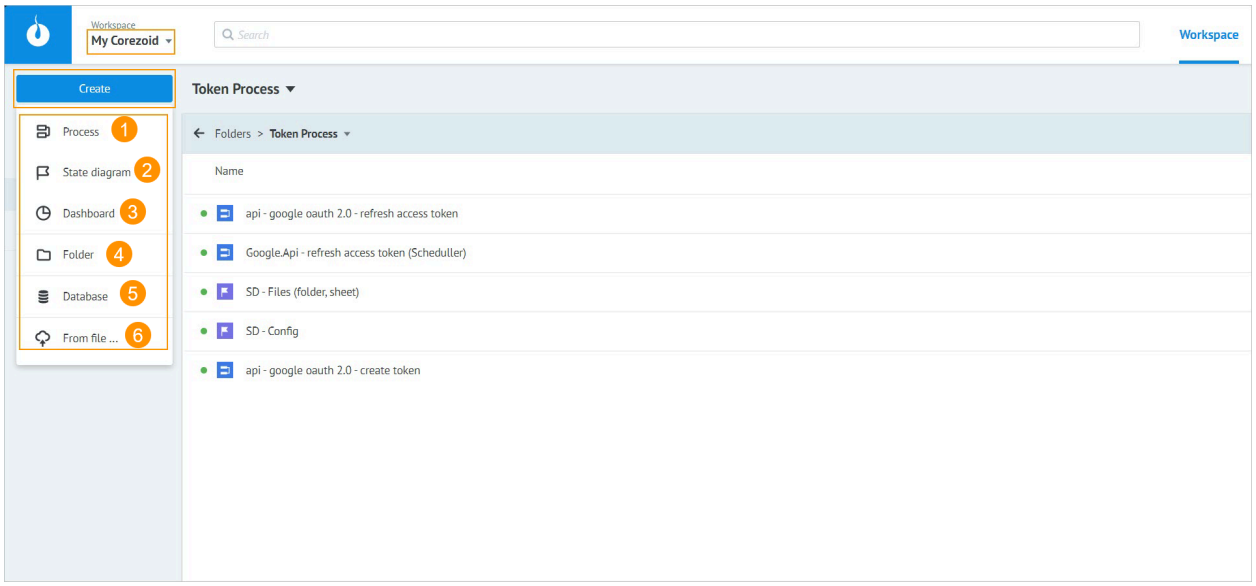


Screenshot 3: **Workspace tab**.

1. **Search field**: Enter a Process, Dashboard, or State Diagram name to find it in the current Workspace.
2. **Create**: Click to create objects depending on the currently selected Workspace. With the **My Corezoid** Workspace selected, you can select one of the items in the list to create the corresponding object.
3. **Projects**: Click to view all the Projects in the Workspace.
4. **Folders**: Click to view all the folders in the Workspace.
5. **Aliases**: Click to view all the Workspace-level aliases.
6. **Shared with me**: Open the list of objects shared with you by other users.
7. **Recent**: Open the list of the objects you've recently opened.
8. **Starred**: Open the list of favorite objects.
9. **Trash**: Open the recycle bin, where you can select objects to restore or delete permanently.
10. Workspace main window: View and manage your workspace objects here.

For more information on editing Processes and State Diagrams on the Workspace tab, go to Process Editor.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

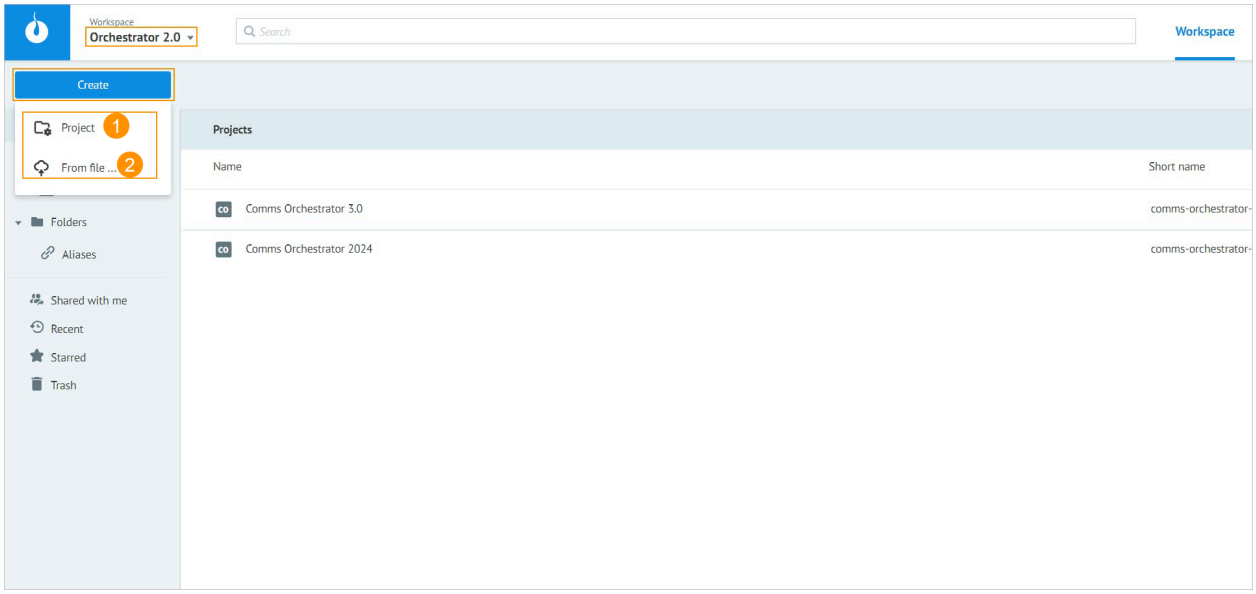


Screenshot 4: **Create button menu for My Corezoid**.

1. **Process:** Create a Process.
2. **State Diagram:** Create a State Diagram.
3. **Dashboard:** Create a Dashboard.
4. **Folder:** Create a folder to store Processes, State Diagrams, Dashboards, aliases, API Keys, Projects.
5. **Database:** Create a database to use in the Database Call node.
6. **From file**: Select this item to upload a Process, State Diagram, Dashboard, or a folder with Corezoid objects stored in a JSON or a ZIP file to the currently selected Workspace.

With a user-created Workspace selected, you can select one of the items in the list to create the corresponding object:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 5: **Create button menu for user Workspace**.

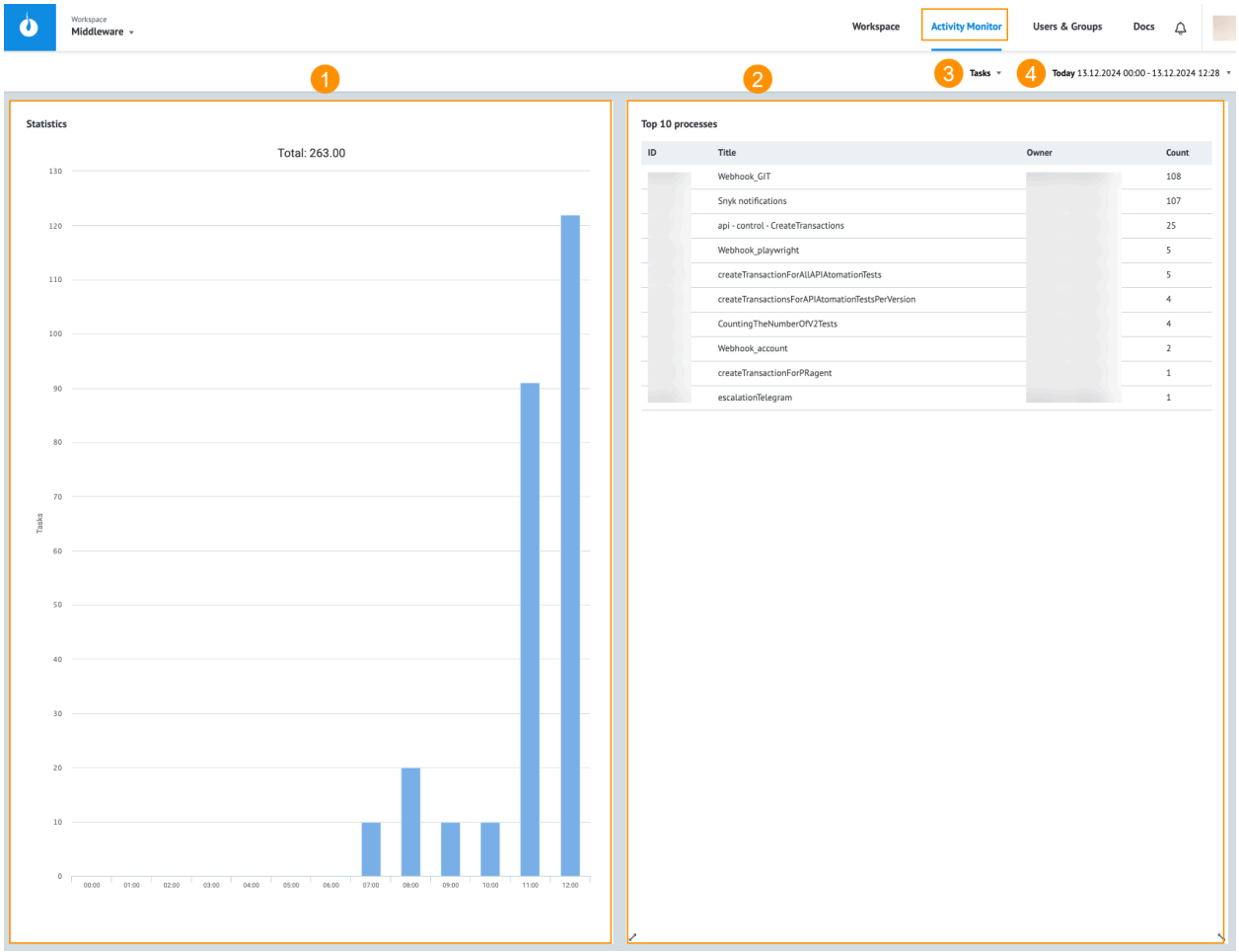
1. **Project:** Create a Project in the workspace.
2. **From file**: select this item to upload a Project stored in a ZIP file to the currently selected Workspace.

2.2.2 Activity Monitor

The **Activity Monitor** provides real-time visibility into system operations:

- **Task Execution**: Monitor the status of active tasks.
- **Performance Metrics**: Review throughput, latency, and other key performance indicators (KPIs) to optimize Processes.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 6: **Activity monitor tab.**

On the **Activity Monitor** tab, you have the following:

1. Statistics: the window that shows the task processing statistics graph for the selected period.
2. Top 10 Processes: the window that shows ID, title, owner, and processed tasks count for the Processes with the highest task processing rate.
3. Filter: select the statistics sorting by
    - State changes
    - Tasks
    - Traffic (MB).

4. Statistics display range: select one of the following ranges for the statistics display on the tab:
    - Custom range: set the custom range.
    - Today.
    - Last 10 minutes.
    - Last hour.
    - Last 24 hours.

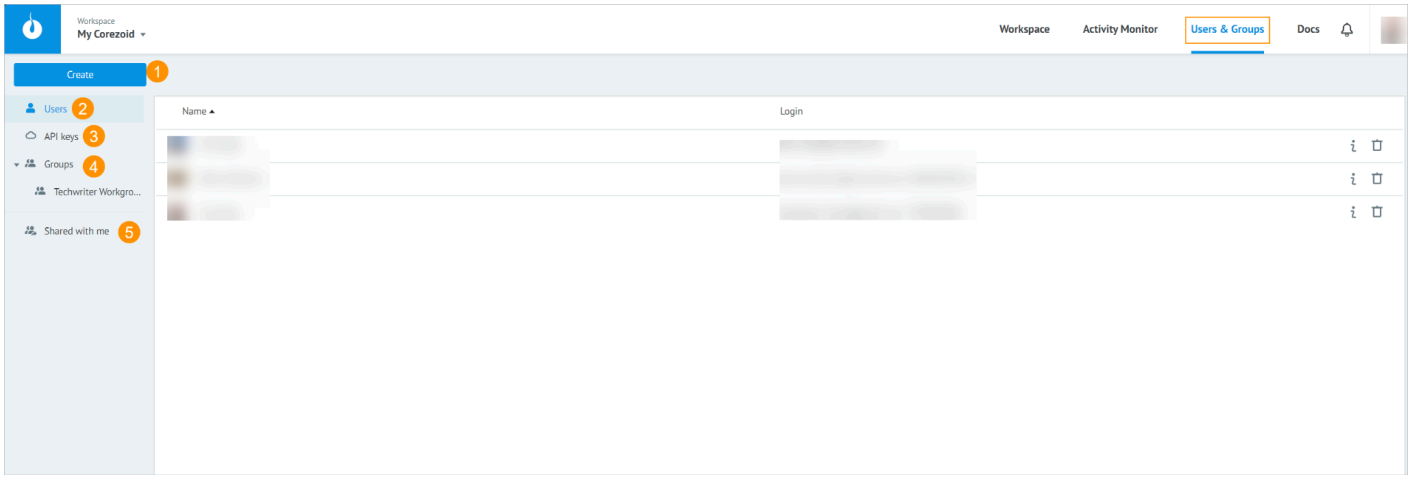
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- Last 7 days.
- Last 30 days.
- Previous hour.
- Previous day.
- Previous week.
- Previous month.

2.2.3 Users & Groups Tab

On the **Users & Groups** tab, you can:

- View info and manage users, user groups, and API keys in the Workspace.
- View and manage users and user groups shared with you in the Workspace by other users.



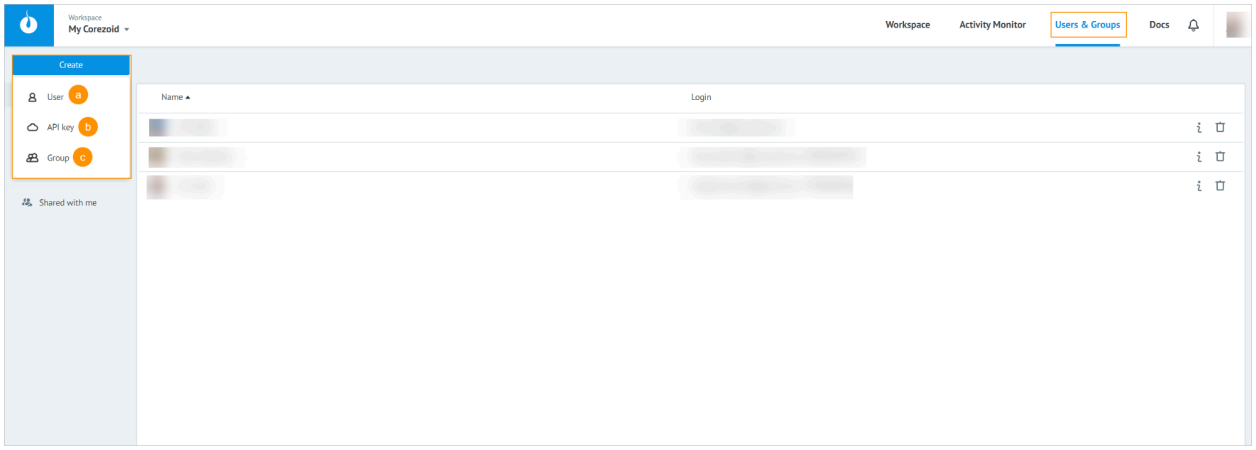
Screenshot 7: **Users & Groups tab.**

On the **Users & Groups** tab, you have the following controls and tabs:

1. **Create**: click to create the following objects in the current Workspace

   a. User: a profile used to log in and work in Corezoid.
   b. API Key: a unique identifier used to authenticate requests made to an API
   c. Group: users united in a group for an easier management.

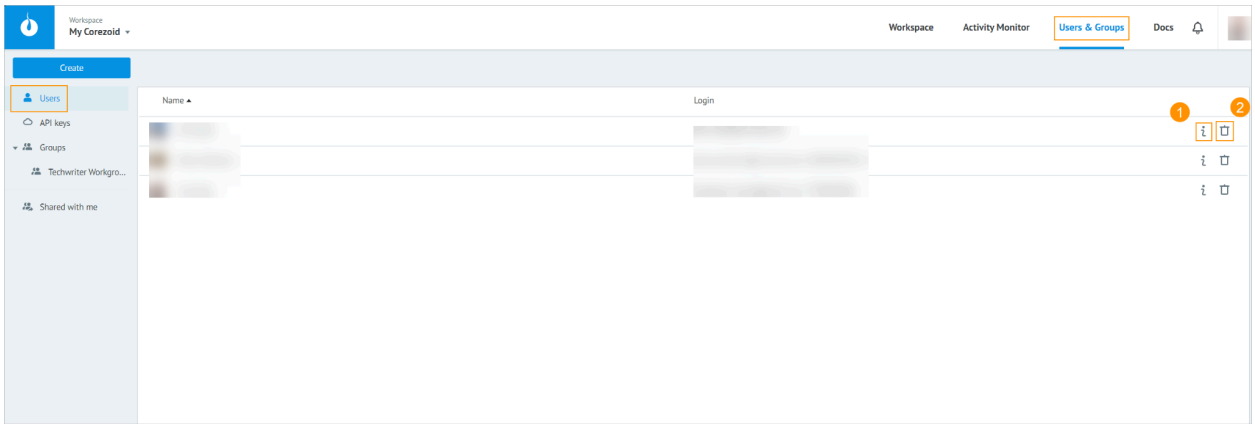
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 8: **Users & Groups Create menu.**

2. **Users:** Open the Workspace users list, where you can view each user's avatar, name, and login. You can:

    1. View info for each user
    2. Delete a user in the list.

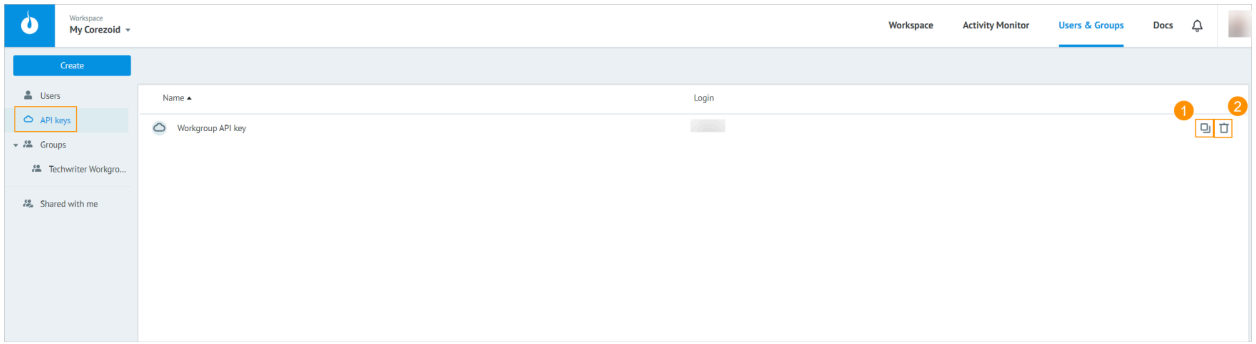


Screenshot 9: **Users & Groups tab - Users list.**

3. **API Keys**: Open the Workspace API keys list, where you can:

    1. Copy a key's secret.
    2. Delete a key in the list.

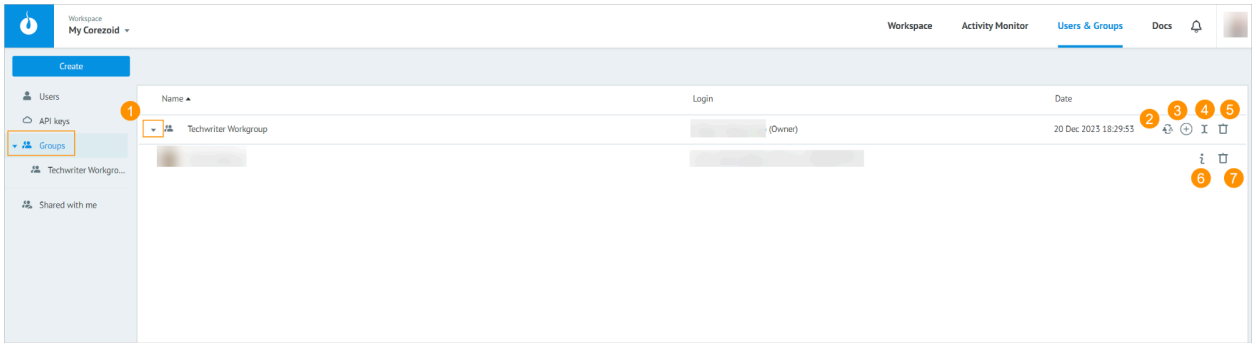
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 10: **Users & Groups tab - API keys list.**

4. **Groups**: Open the user groups list, where you can:

    1. View the list of users included in the group by clicking the arrow near a group's name.
    2. Specify the new owner for the group by clicking the change ownership icon.
    3. Add users to the group by clicking the add user icon.
    4. Rename the group by clicking the rename icon.
    5. Delete the group by clicking the delete icon.
    6. View the group user's info by clicking the info icon.
    7. Delete a user by clicking the delete icon.



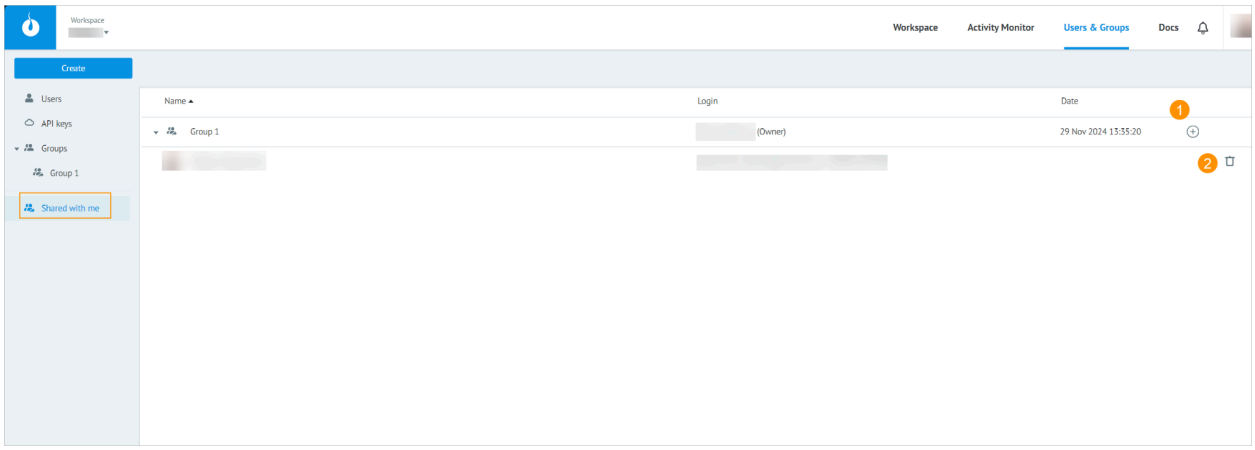
Screenshot 11: **Users & Groups tab - Groups list.**

5. **Shared with me**: Open the list of user groups shared with you by other users. Here, you can:

    1. Add new users to shared groups by clicking the plus icon.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

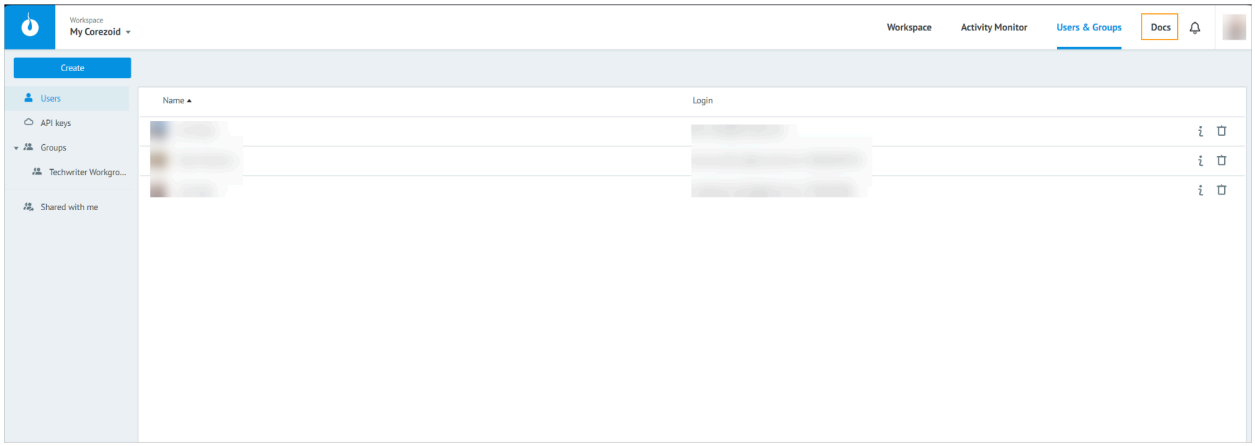
2. Delete users from groups by clicking the delete icon.



Screenshot 12: **Users & Groups tab - Shared with me.**

2.2.4 Docs Tab

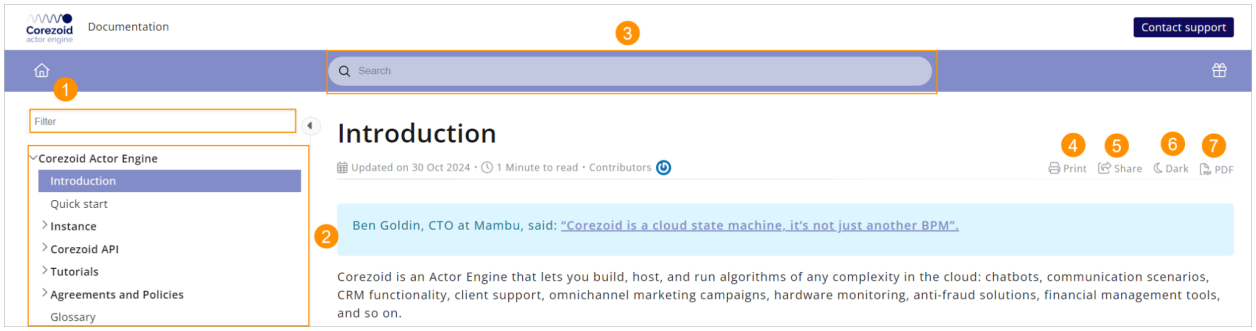
You can access Corezoid documentation, including description of Processes, nodes and API using the **Docs tab:**



Screenshot 13: **Docs tab.**

Clicking the **Docs** tab redirects you to the Corezoid documentation portal, where you can:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 14: **Documentation portal menu.**

1. **Filter:** enter a word or phrase to display only the documentation sections and subsections that match your search criteria. The filter instantly narrows down content as you type, helping you quickly locate relevant information.
2. **Documentation structure:** browse the documentation by clicking arrows to expand/collapse sections and access specific articles within sections and subsections.
3. **Search:** enter a phrase to view all articles and sections containing that phrase, then click to open the desired result.
4. **Print:** print the currently displayed article.
5. **Share:** share a link to the current article.
6. **Theme:** switch between dark and light color schemes.
7. **Export:** download the current article as a PDF file.

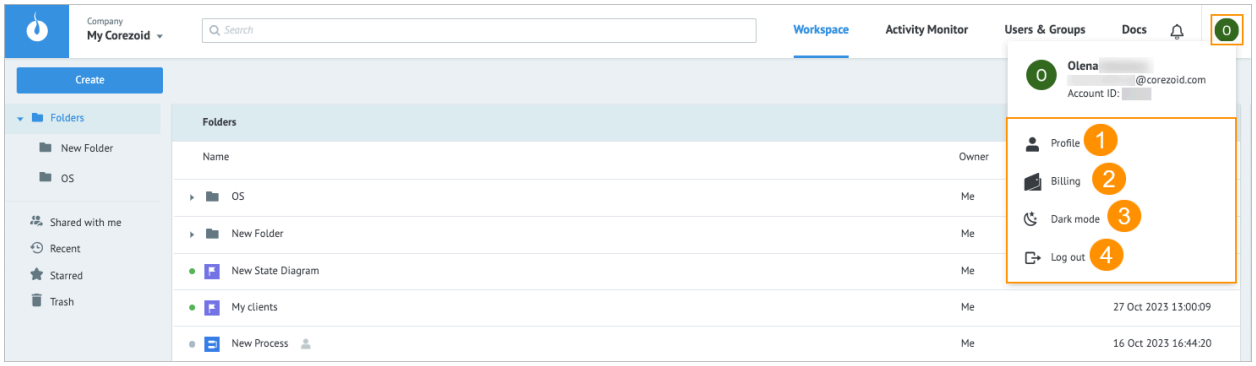
2.2.5 User Profile

The Profile page shows the user's email, account ID, personal information, and billing details.

On the tabs of the Profile page, you can perform the following actions:

1. **Profile:** View and edit your profile information.
2. **Billing:** Check billing details and upgrade your license plan.
3. **Dark mode:** Change the platform UI display Light mode to the Dark mode by clicking the avatar and in the dropdown list, selecting Dark mode.
4. **Log out:** Log out from the platform by clicking the avatar and in the dropdown list, selecting Log out.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

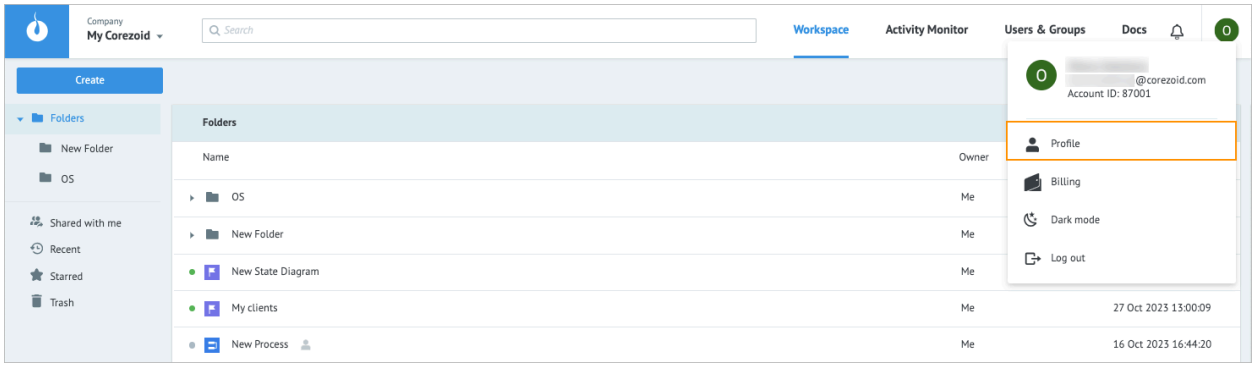


Screenshot 15: **Profile menu.**

View and edit your profile information

To view and edit your profile information:

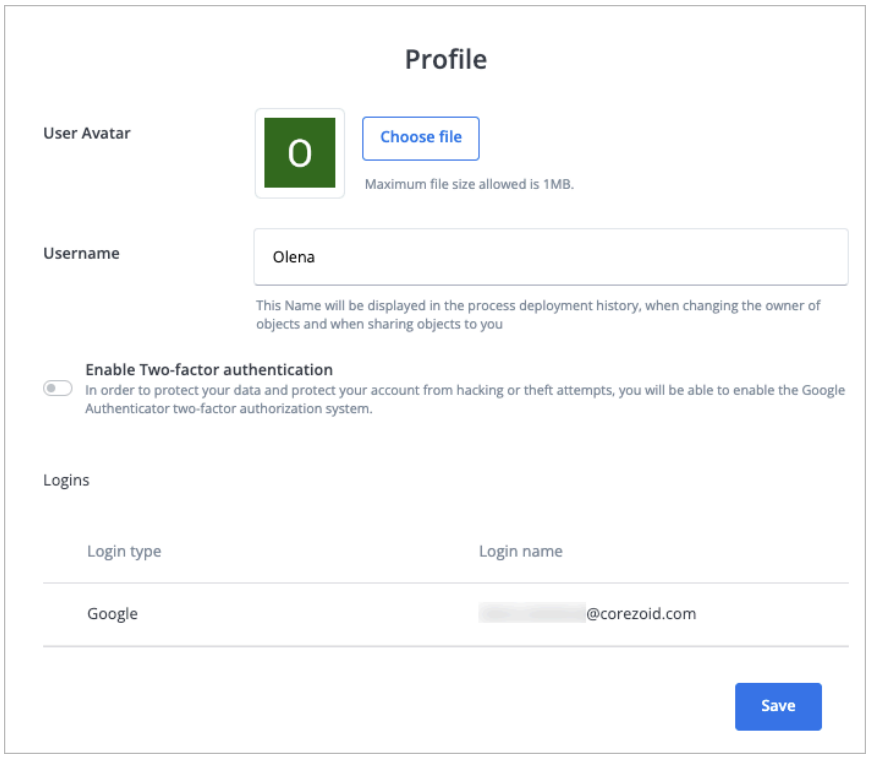
1. In the upper-right corner of the page, click your avatar, and then in the dropdown list, select Profile.



Screenshot 16: **Edit your Profile info.**

2. On the Profile tab:
    a. Click Choose file and upload a photo.
    b. Enter a username in the Username field.
    c. (Optional) Turn on the Enable Two-factor authentication toggle and follow the prompts in the dialog that appear to set the two-factor authentication.
    d. Click Save.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



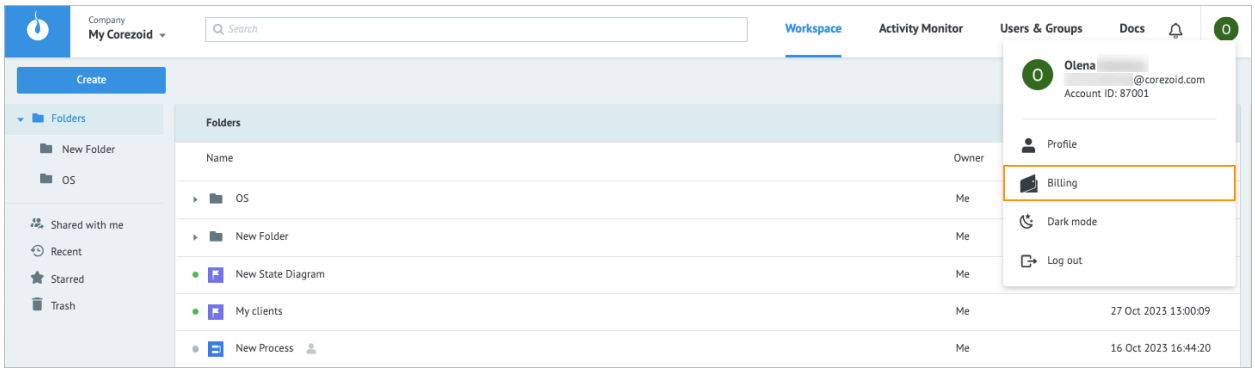
Screenshot 17: **Profile editing page.**

Your profile information has been updated, and you can edit it anytime.

View and change your license parameters

To check your license details and change them:

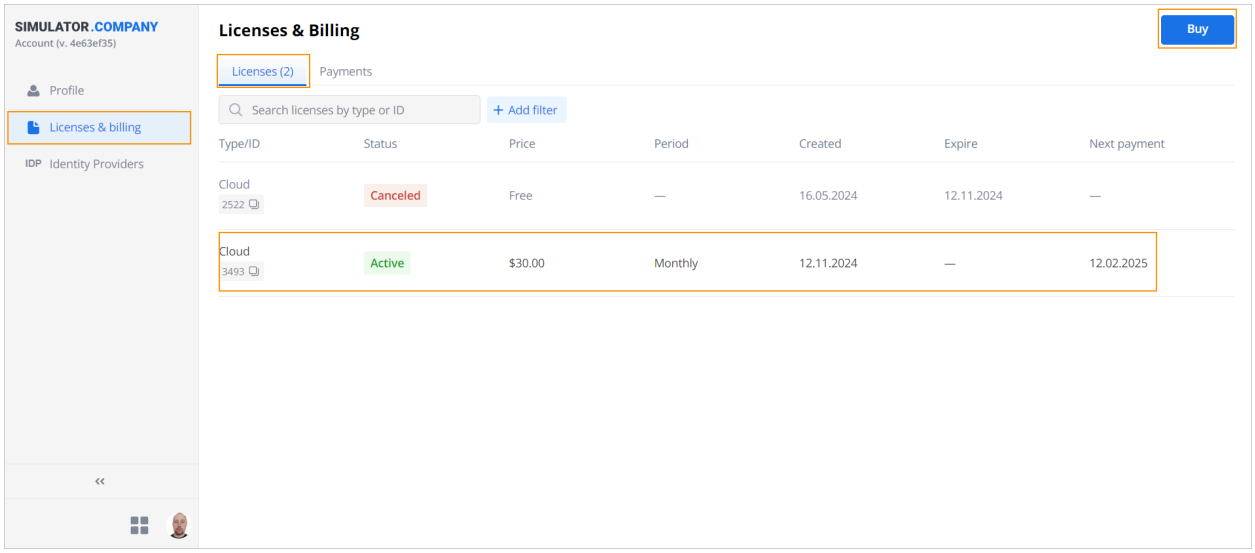
1. In the upper-right corner of the page, click your avatar, and then in the dropdown list, select Billing.



Screenshot 18: **Edit your Billing info.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

2. On the Account **Licenses & billing** page, on the **Licenses** tab that opens, click an active license you want to upgrade. You can also buy a new license by clicking the **Buy** button in the right upper corner:

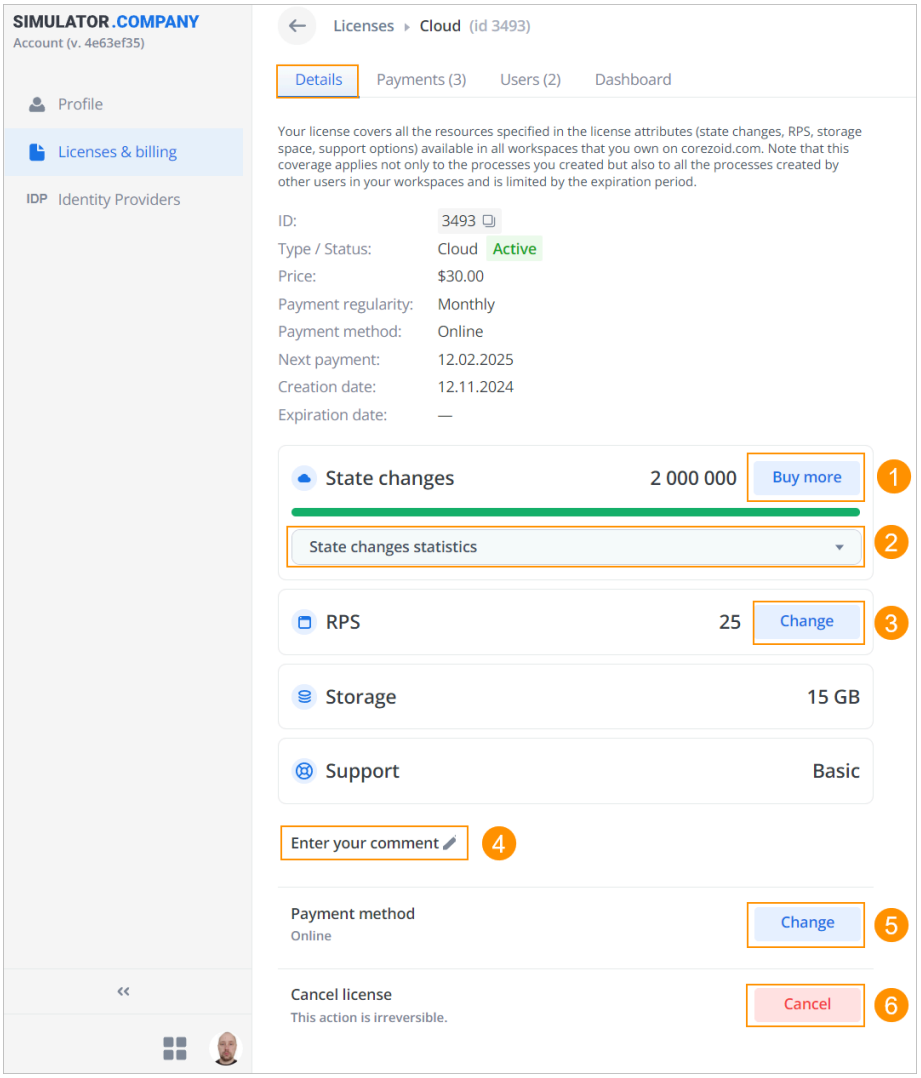


Screenshot 19: **Upgrade your license or buy a new one.**

3. On the **Details** tab of the open license, you can view the license parameters:
   - ID,
   - Type/Status
   - Price
   - Payment regularity
   - Payment method
   - Next payment date
   - Creation date
   - Expiration date
   - Storage space
   - Support level.

   Also, you can change and view the following license parameters:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 20: **Change your license parameters.**

3.1 **State changes**: Increase your state changes by clicking **Buy more**.

A state change occurs when a task transitions between nodes. For example, a task passing through four nodes consumes three state changes. The formula for calculating state changes in a Process: **SC = (N – 1) x T**, where:

- **SC**: number of state changes.
- **N**: number of nodes in a Process.
- **T**: total of tasks received in a Process.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3.2 **State change statistics**: View state change statistics on your license.

3.3 **RPS**: Change your current RPS by clicking Change.

RPS is the maximum number of tasks that can be processed across all Processes per second.

With 10 RPS license:

- ✓ 10 tasks/second: Successful processing
- ✗ 11 tasks/second: `RPS limit reached` error

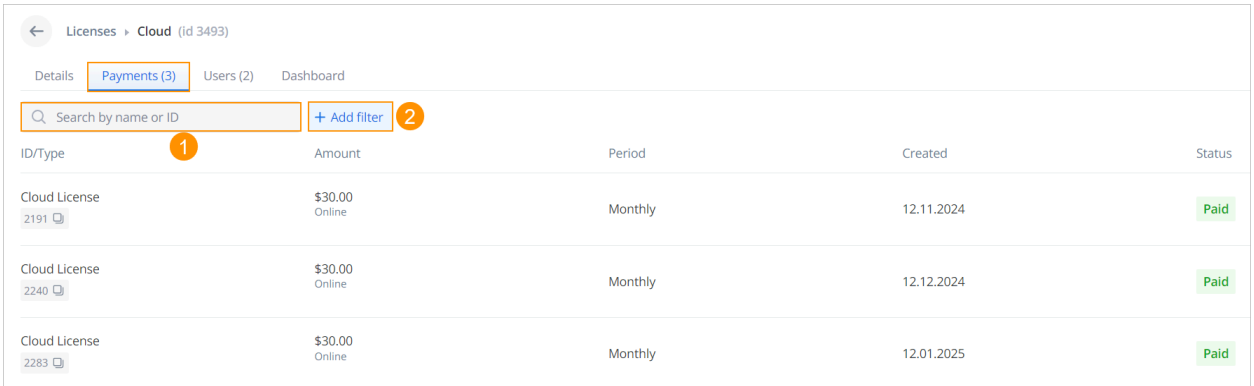
3.4 Enter a comment.

3.5 **Payment method**: Change the payment method for the license by clicking **Change**.

3.6 **Cancel license**: Cancel the license by clicking **Cancel**.

Note: This action is irreversible.

4. To view the license payment history, open the **Payments** tab:



Screenshot 21: **Payments tab.**

4.1 **Search field**: Search the payment by name or ID in the list.

4.2 **Add filter**: Click to add a search filter.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

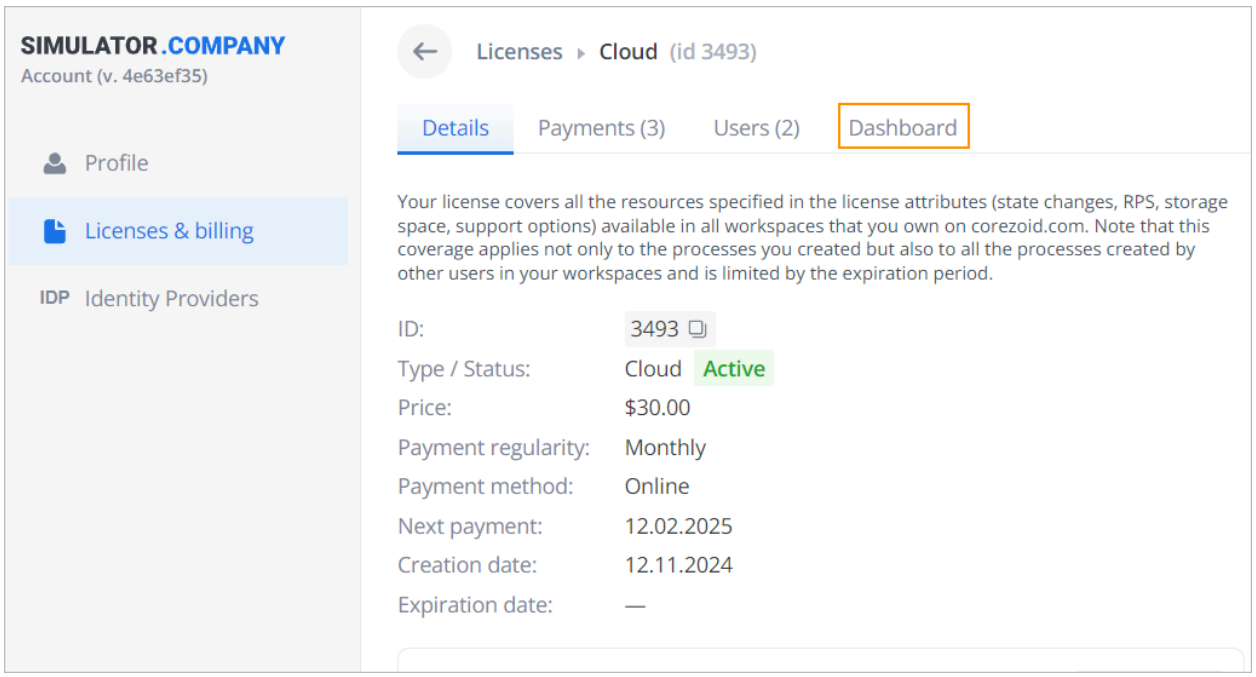
5. To view and change the users granted the license, open the **Users** tab:

Screenshot 22: **Users tab.**

5.1 **Search field**: Search the user by name or ID in the list.

5.2 **Blocked/Active**: Switch between active and blocked statuses of users granted the license by the license holder user. Blocking a user makes it impossible for the user to use the license. You can switch back to active to grant the user the license again.

6. To view the **Dashboard** on your license, open the **Dashboard** tab:



Screenshot 23: **Dashboard tab.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

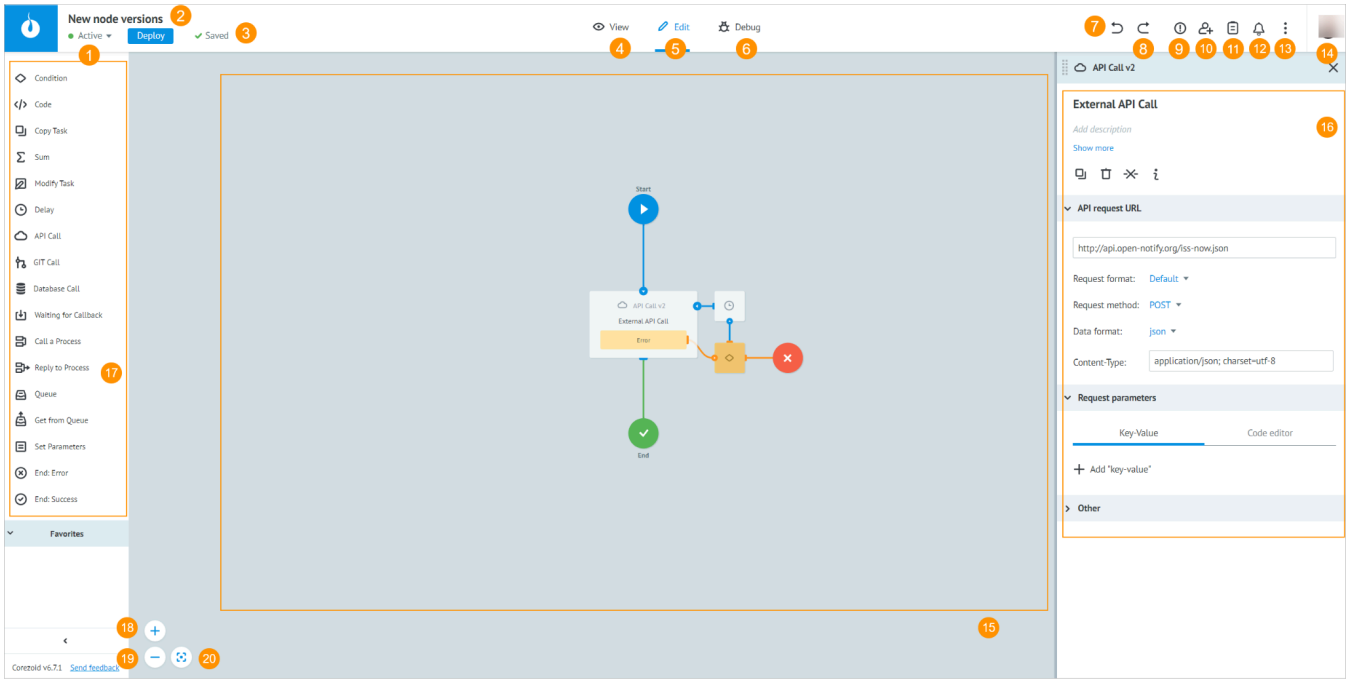
2.2.6 Process Editor

To edit a Process or a State Diagram in the Process editor:

1. Open the **Workspace** Tab: Click the Process or State Diagram you want to edit.
2. Go to the Edit tab in the Process editor, you can use the following:

Edit Tab

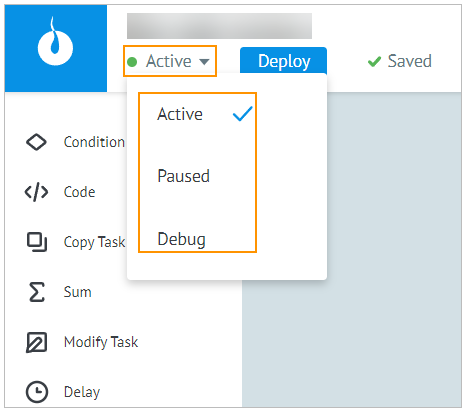
On the Edit tab in the Process editor, you can use:



Screenshot 24: **Process editor general panels**

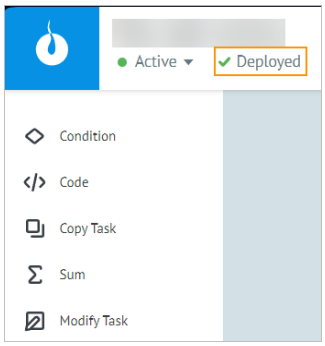
1. **Process/State Diagram State Selection**: Use the dropdown to toggle between Active, Paused, and Debug states:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 25: **Process state selection.**

2. **Deploy**: Click to apply saved changes to the Process:



Screenshot 26: **Process deployed indication.**

3. **Saved Indicator**: Confirms all changes are auto-saved and ready for deployment.

4. **View Tab**: Switch to View mode to observe tasks.

5. **Edit Tab (default)**: Switch to Edit mode to add or modify nodes.

6. **Debug Tab**: Use to track task movement step by step.

7. **Undo**: Undo recent actions.

8. **Redo**: Redo recent actions.

9. **Errors**: View Process or State Diagram errors.

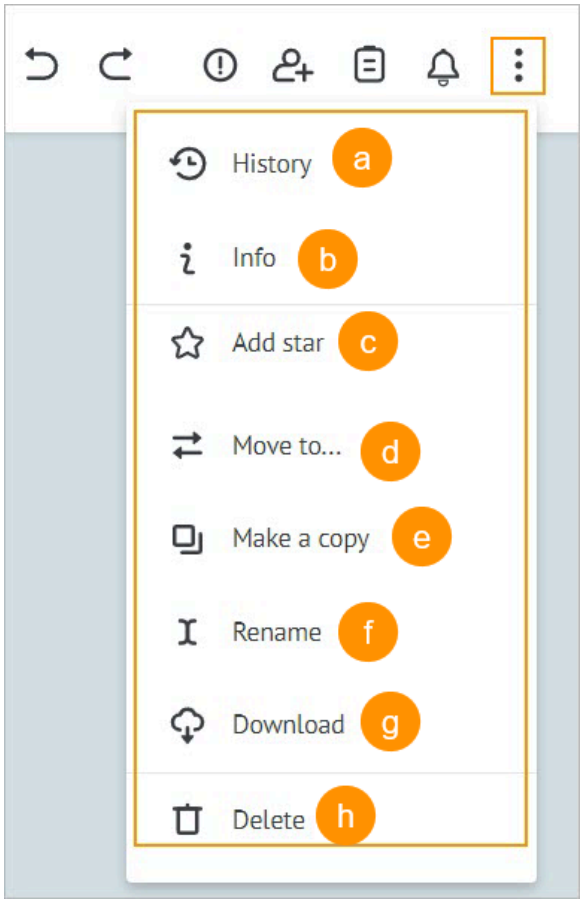
10. **Share**: Manage sharing settings.

11. **Task Parameters**: Open the task parameters window.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

12. **Notifications**: View notifications.

13. **Three-Dot Menu**: Click for additional options:

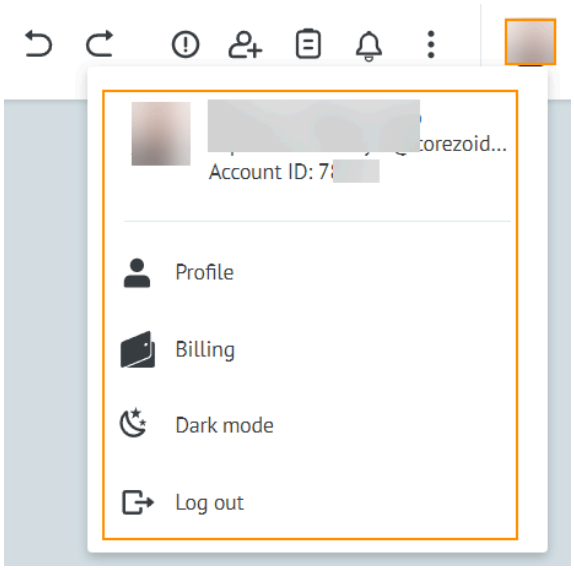


Screenshot 27: **Three-dot menu.**

a. **History**: View version history.
b. **Info**: View Process/State Diagram details.
c. **Add Star**: Mark as a favorite in the Workspace.
d. **Move To**: Relocate to a folder.
e. **Make a Copy**: Duplicate the Process/State Diagram.
f. **Rename**: Rename the Process/State Diagram.
g. **Download**: Save as a zip or JSON file.
h. **Delete**: Remove the Process/State Diagram.

14. **Profile Menu**:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



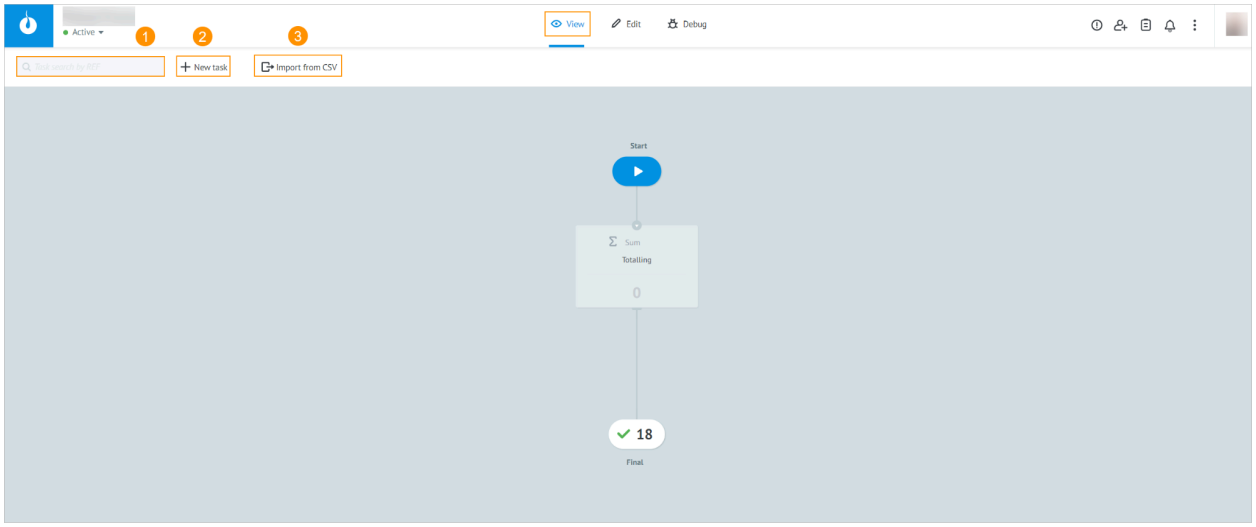
Screenshot 28: **Profile menu**

15. **Canvas**: The main workspace to add, connect, and manage nodes.
16. **Node Configuration Panel**: Configure the selected node.
17. **Nodes Panel**: Add nodes to the canvas from the list.
18. **Zoom in**: zoom in on the canvas.
19. **Zoom out**: zoom out on the canvas.
20. **Center**: Center all nodes on the canvas.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

View Tab

On the **View** tab, you have the following:



Screenshot 29: **View tab controls.**

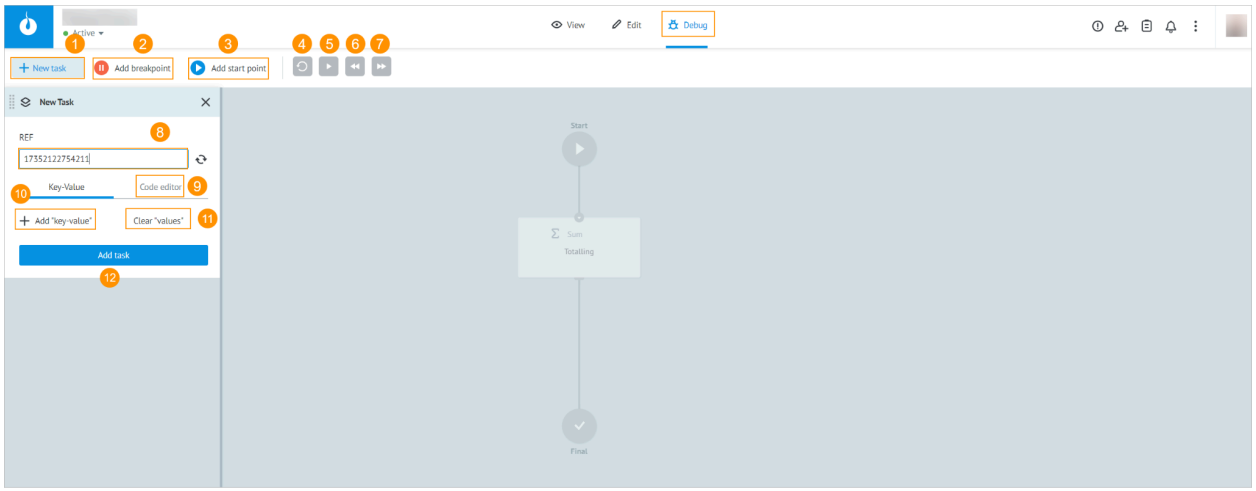
1. **Task search field**: Search a task by REF.
2. **New Task**: Add a new task to the Process.
3. **Import from CSV**: Import a task from a CSV file.

For more details, see Working with Tasks.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Debug Tab

On the **Debug** tab, you have the following:



Screenshot 30: **Debug tab controls.**

1. **New Task**: add a new task to the Process in Debug mode.
2. **Add breakpoint**: click the icon and drag it to where you want to add a breakpoint for tasks in your Process.
   Note: you can add breakpoints to the edges between nodes in your Process.
3. **Add start point**: click the icon and drag it to where you want to add a start point for tasks in your Process.
   Note: you can add only one start point to your Process.
4. **Repeat**: repeat the last play - the movement of an added task from the start point to the nearest breakpoint.
5. **Play**: move an added task from the start point to the nearest breakpoint.
6. **Rewind/back**: rewind the last task movement.
7. **Forward**: move a task to the next breakpoint.
8. **REF**: view and change the task reference. Click the icon to generate the new reference value automatically, or enter the value manually.
9. **Code editor** tab: switch to the code editor view of the current task content.
10. **Add "key-value"**: click to add a key-value pair to the task.
11. **Clear "values"**: clear values for all keys in the task.
12. **Add task**: click to add the task to the Process.

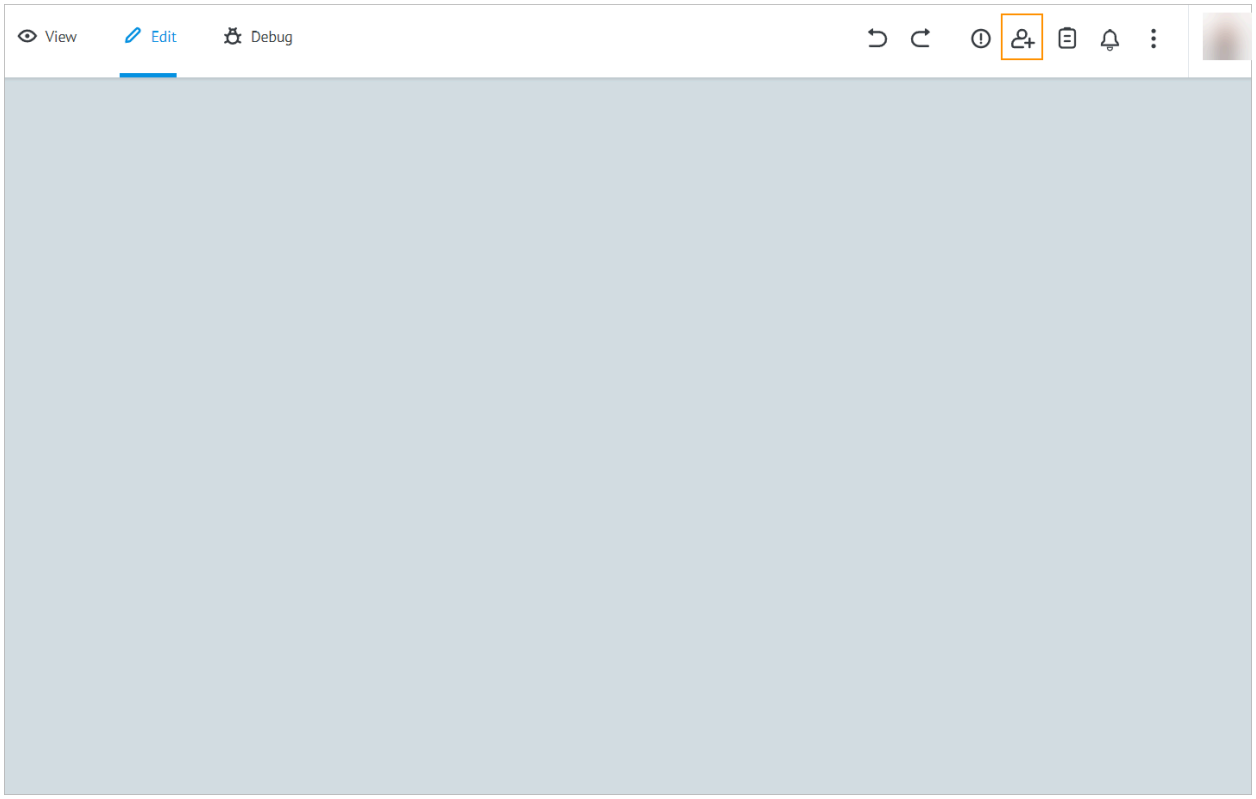
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

For more details, see Debugging a Process.

2.2.7 Sharing Settings Menu

You can use the Shared menu to share a Process or a State Diagram with new users or user groups, or change the existing sharing settings. To share with new users/user group, you need to invite them first:

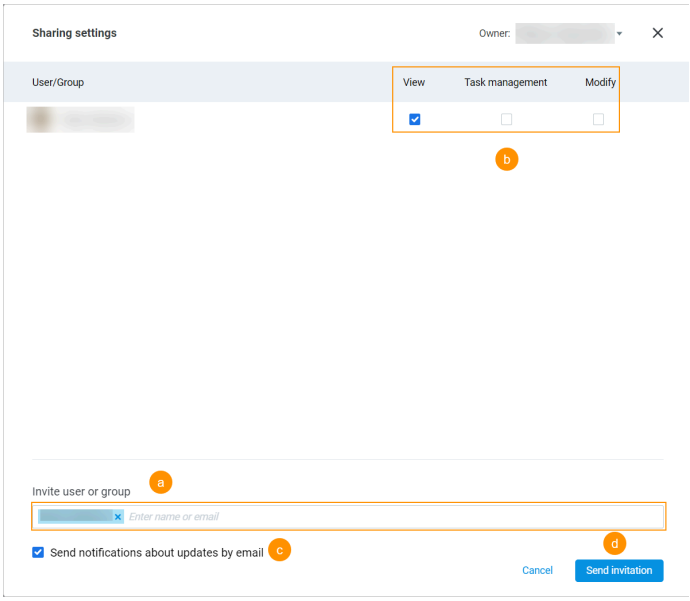
1. Open a Process or a State Diagram in the Process editor.
2. Click the share icon to open the **Sharing settings** menu:



Screenshot 31: **Sharing settings menu icon.**

3. In the menu:
    a. Enter the user/user group email or name.
    b. Select the sharing permissions to be granted to the user/user group: **View** (view only), **Task management** (allow sending tasks), **Modify** (allow editing the Process or a State Diagram).
    c. Select the **Send notifications about update by email** checkbox if you want the notifications on the sharing settings updates to be sent to the user/user group.
    d. Click **Send invitation**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



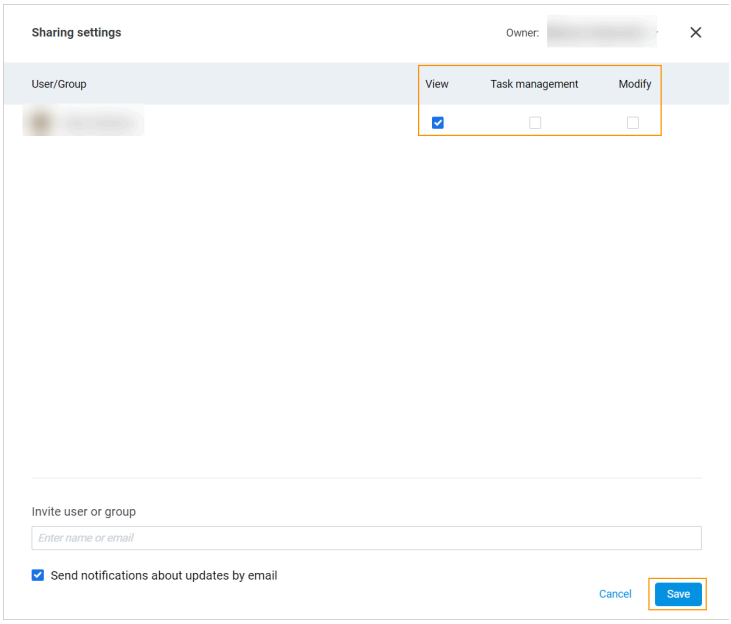
Screenshot 32: **Configuring share permissions and sending an invitation.**

The invitation has been sent. After the user confirms the invitation in their email, they are redirected to the shared Process or a State Diagram with the set permissions.

To change the sharing settings:

1. Open the Sharing settings menu for the needed Process or a State Diagram.
2. Set the new permissions in the list.
3. Click **Save**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 33: **Saving sharing changes.**

2.3 Core Concepts

Corezoid enables users to build dynamic workflows using a Process-oriented model. A **Process** represents a sequence of tasks or events, while **states** act as the checkpoints or stages within the Process. Events transition the workflow between states, enabling logic-based automation.

- **Node:** A building element for Processes, a unit of work in the Process.
- **Process:** A structured sequence of states defining the logic and actions to accomplish a specific task.
- **State:** Represents a condition within the Process. Examples include "Data Received," "API Called," or "Task Completed."
- **State Diagram:** A diagram that tracks and stores an object state.
- **Event:** A trigger that causes transitions between states. Events may depend on conditions, time, or external inputs.
- **Task:** A discrete unit of work within a Process. Tasks are sets of data in JSON format saved as key-value pairs (e.g., `{ "key": "value" }`). They serve as inputs for nodes to execute actions such as sending notifications, making API calls, or updating databases. Tasks can be added manually, imported from CSV files, loaded via direct URL, or submitted using the Corezoid API.

2.3.1 Nodes

Nodes in Corezoid are building blocks that determine how tasks are processed, routed, or manipulated at each step within your Processes and State Diagrams. By adding and connecting

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

nodes, you design the structure of your workflow. Based on their functions, nodes are organized into the following categories:

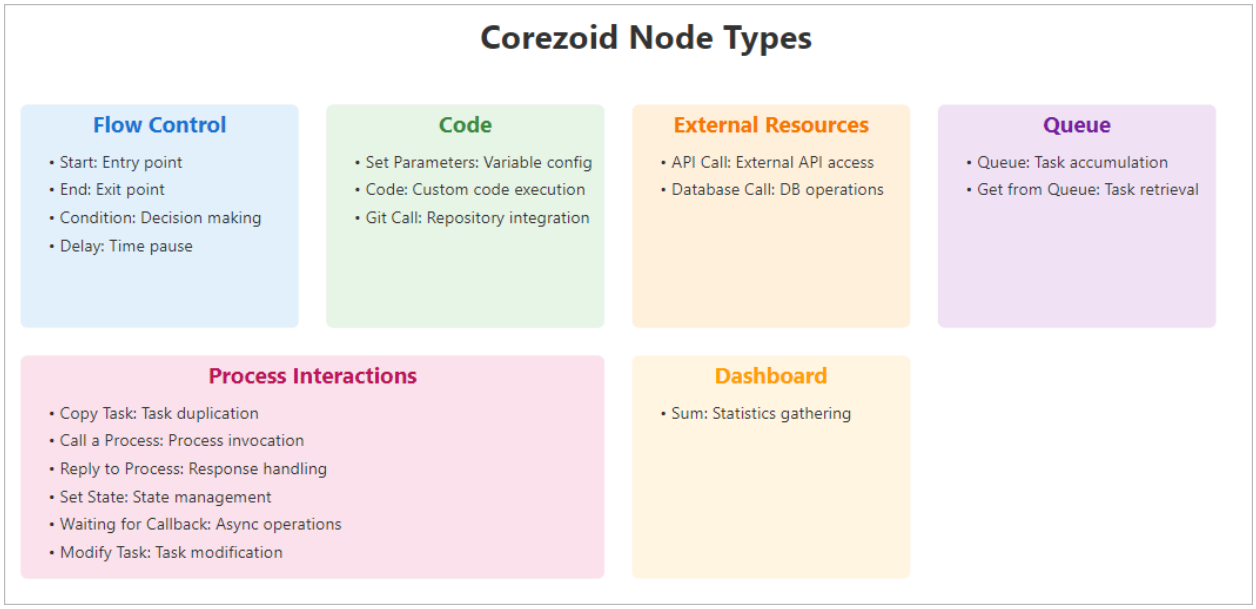


Diagram 3: **Node palette showcasing available node types with descriptions.**

1. **Flow Control:** These are the fundamental building blocks that control the basic flow and timing of tasks through a Process.
2. **Code:** Focused on manipulating and transforming task parameters and data.
3. **External Resources:** Designed for integrating with external systems and data sources.
4. **Queue (accumulating and retrieving tasks):** Manage task accumulation and retrieval for asynchronous processing.
5. **Process Interactions:** Facilitate communication and data flow between different Processes.
6. **Dashboard (gathering statistics on tasks):** Tracks and accumulates values from task parameters and provides statistical insights and monitoring capabilities for Process metrics.

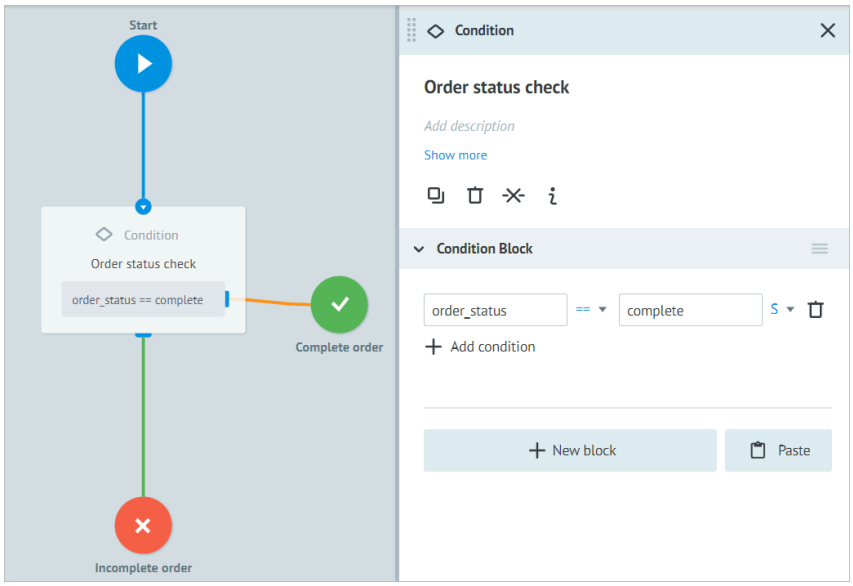
Each node has specific parameters and configurations that allow users to define its behavior.

Flow Control (basic building nodes):

- **Start**: The entry point into a Process or State Diagram for tasks, initializing logic execution. Mandatory for every Process and State Diagram.
- **End**: The final node in a Process or State Diagram where tasks are collected after going through all the nodes in the workflow. Mandatory for every Process and State Diagram.
- **Condition**: Routes tasks within the workflow based on specified parameter values. You can use it to trigger alerts and verify payments.
- **Delay**: Pauses task processing for a specified time. Use it for scheduled mailing, SMS sending.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- **Parallel Processing** (coming soon).



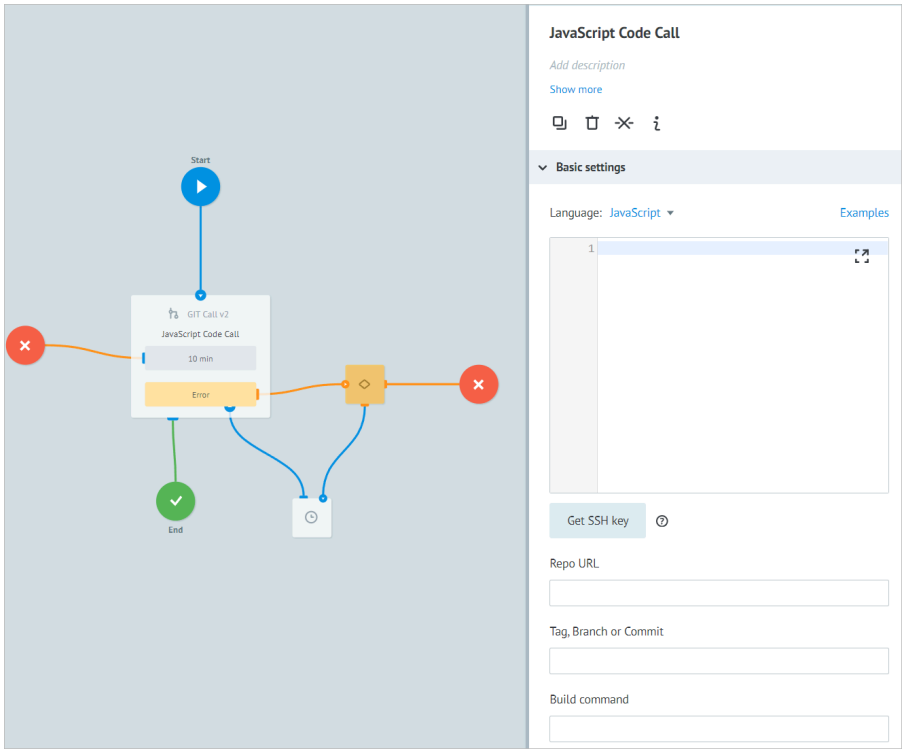
Screenshot 34: **Configuration panel for the Condition node.**

- Screenshot showing the configuration panel for a Condition node with an example condition (e.g., if `order_status == 'complete'`).

Operations on Task Parameters:

- **Set Parameters**: Adds new or updates existing parameters for a task. Use it for data modification stored in a State Diagram.
- **Code**: Executes your custom JavaScript or Erlang code for advanced data manipulation.
- **Git Call**: Runs code stored in a Git repository written in JavaScript, Golang, Python, Java, PHP, Closure, Lisp, and Prolog out of the box. Use the custom Dockerfiles feature to run code in any language.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

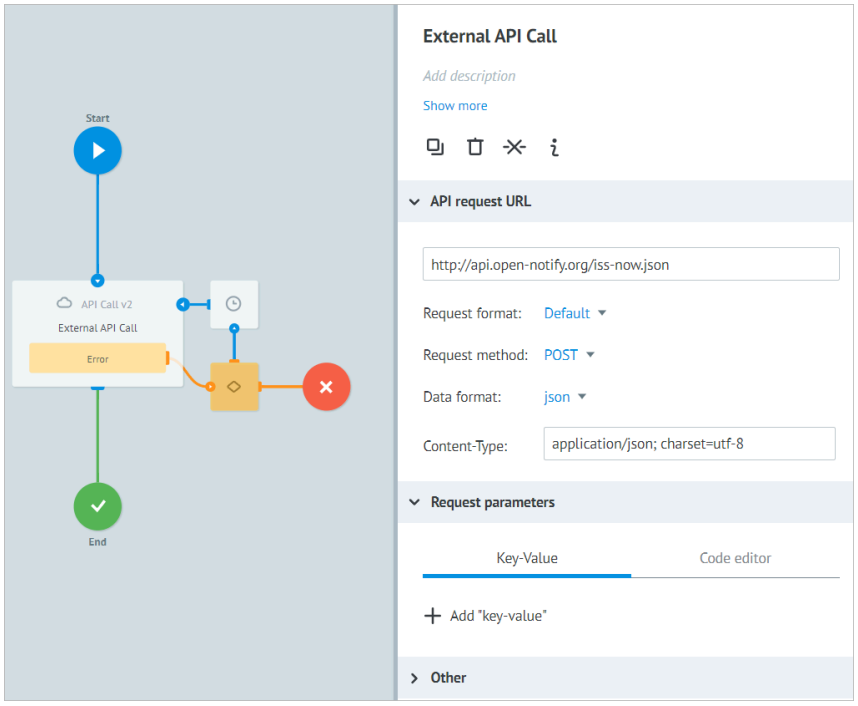


Screenshot 35: **Configuration panel for the Git Call node.**

External Resources (accessing external resources)

- **API Call:** Interacts with external APIs by sending requests and processing responses. You can use it with social media, payment gateways, or weather services.
- **Database Call:** Executes SQL queries on an external database for interaction with stored data on the task level. Use this node to update an external database or select and retrieve data from it.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 36: **Configuration panel for the API Call node.**

- Example of an API Call node with input fields for the URL, headers, and payload.

Queue (accumulating and retrieving tasks):

- **Queue**: Stores tasks for later processing. Use it for load balancing or implementing asynchronous workflows, like batch notification sending.
- **Get from Queue**: Retrieves tasks from a specified queue. Use it together with Queue nodes to resume task processing in your workflow.

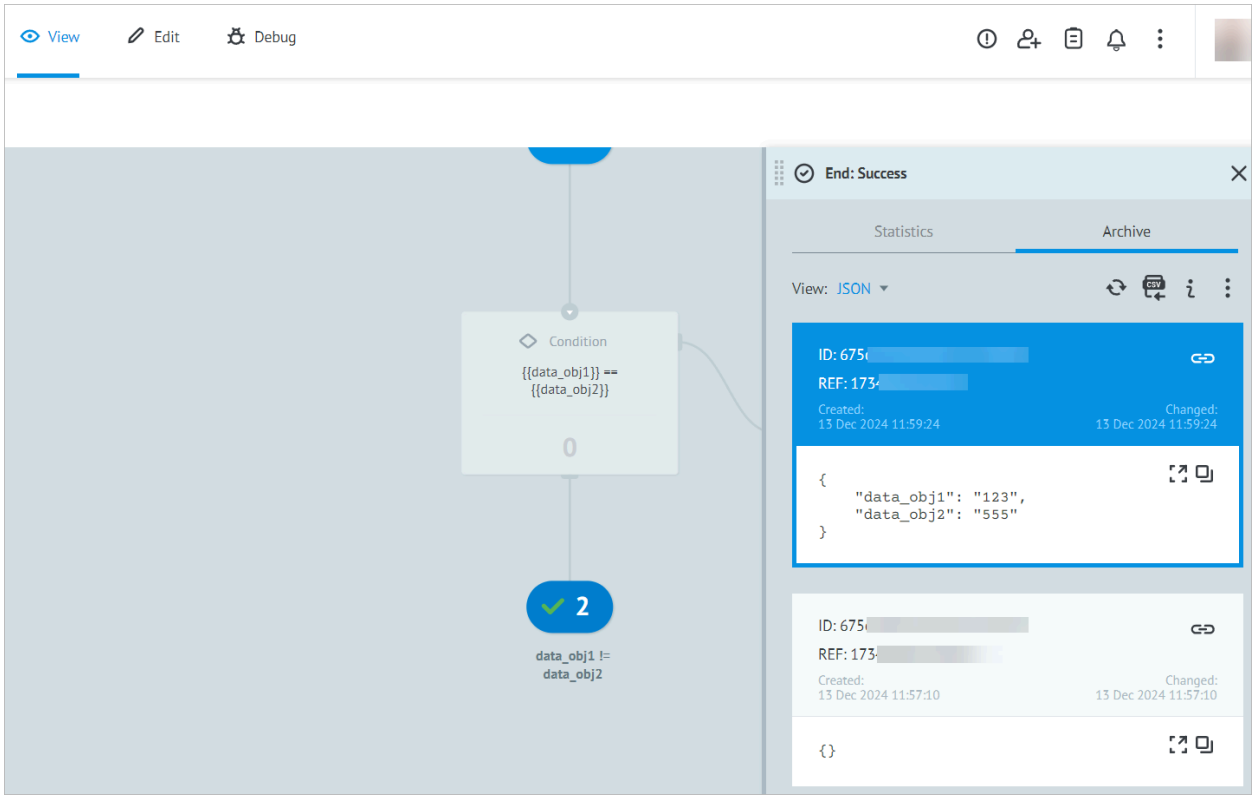
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

2.3.2 Tasks

Overview

A task is a set of data in JSON format saved in key-value pairs ("key": "value"). Tasks can:

- Execute specific actions, such as API calls or data manipulation.
- Be dynamically assigned based on Process logic.
- Report real-time status and results.

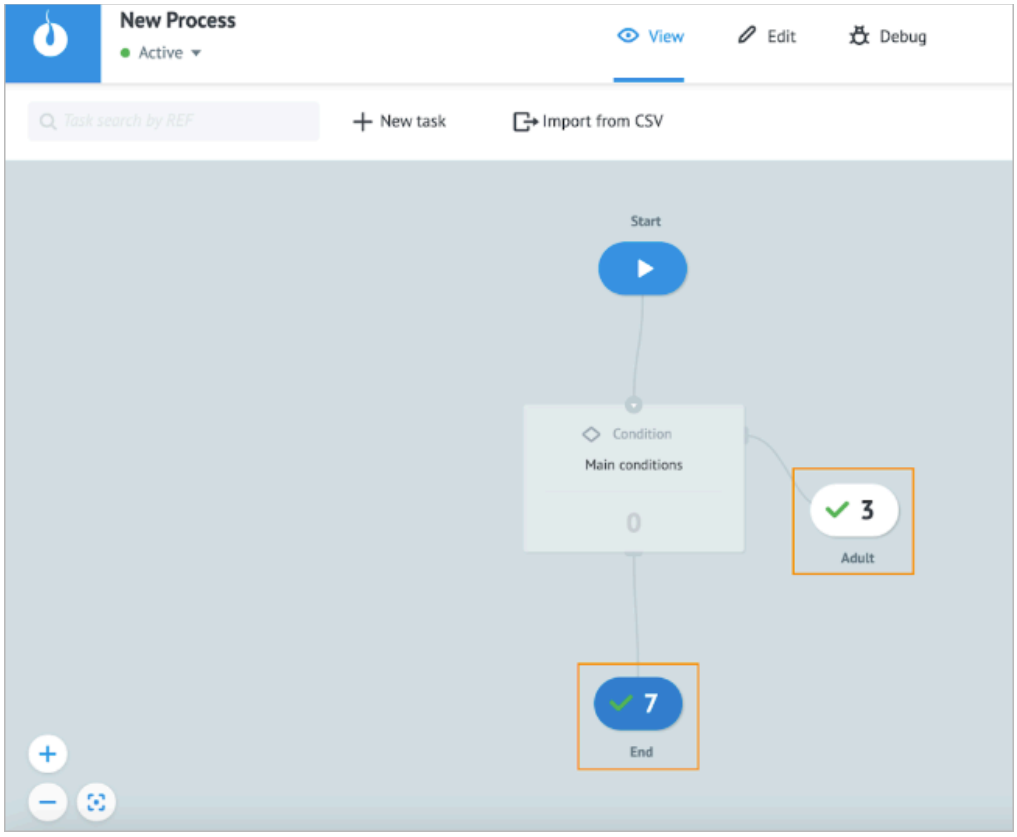


Screenshot 37: **Task execution log showing task details, status, and timestamps.**

Task details

The node that the task reached has a counter showing the number of received tasks.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 38: **Task distribution.**

The received task has the following data:

- ID: A unique value in the entire system.
- REF: A unique value in the current Process.
- Created: Date and time when the task was created.
- Changed: Date and time when the task was changed.
- Task parameters in the JSON or Table format.

REF parameter

REF parameter must have a unique value within a Process for all the tasks that are not located in the Process final nodes.
You can specify the REF (reference) parameter for new tasks in Corezoid UI and via the Corezoid API. When adding a new task in the UI, the `getRandomRef` function is used to generate and suggest a REF value. You have the option to change the suggested REF manually.

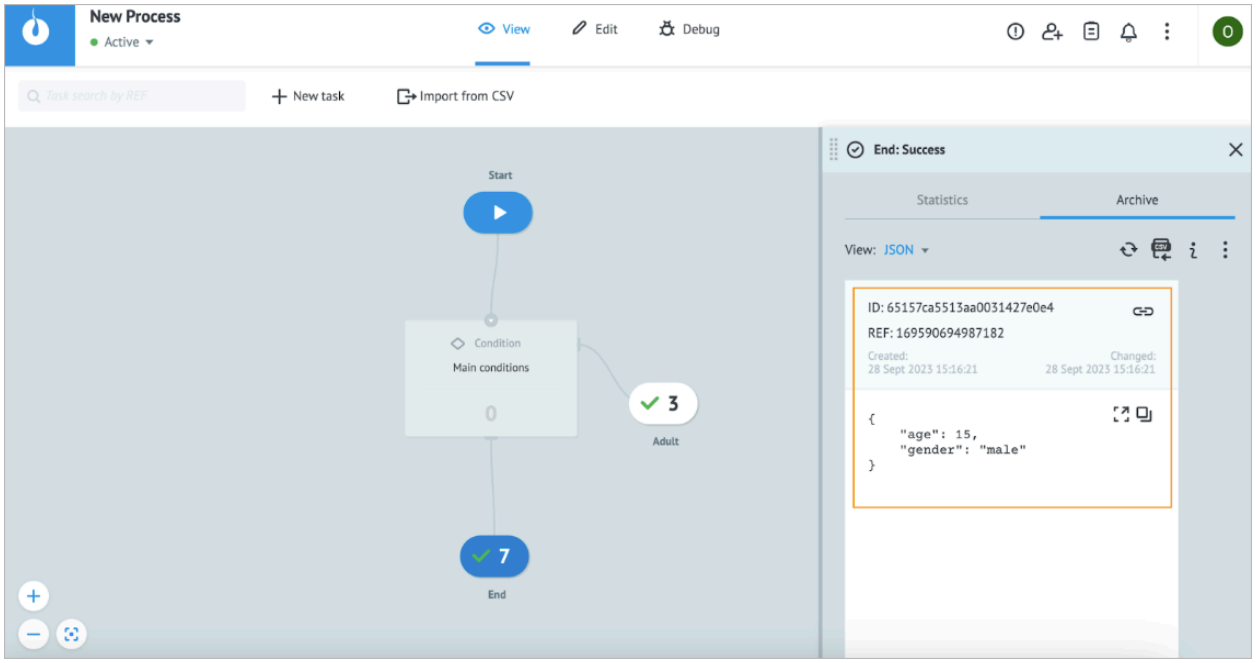
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

For tasks added through the UI, the `getRandomRef` function creates a unique reference by combining two parts:

1. `Date.now().toString()` - This retrieves the current timestamp in milliseconds since January 1, 1970, and converts it to a string, providing a unique number that increases over time.

2. `Math.floor((Math.random() * 100) + 1)` - This generates a random number between 1 and 100 (inclusive) and rounds it to the nearest whole number.

The function combines these two components to create a unique reference. For example, it might look like this: 167890123456789 (timestamp) + 42 (random number) = 16789012345678942.



Screenshot 39: **Task view in the archive.**

2.3.2.1 Task Adding

In Corezoid, you can add a task to a Process differently. Use one of the following ways.

1. Send a task manually.
2. Import a task from the CSV file.
3. Load a task via direct URL.
4. Use the Corezoid API.

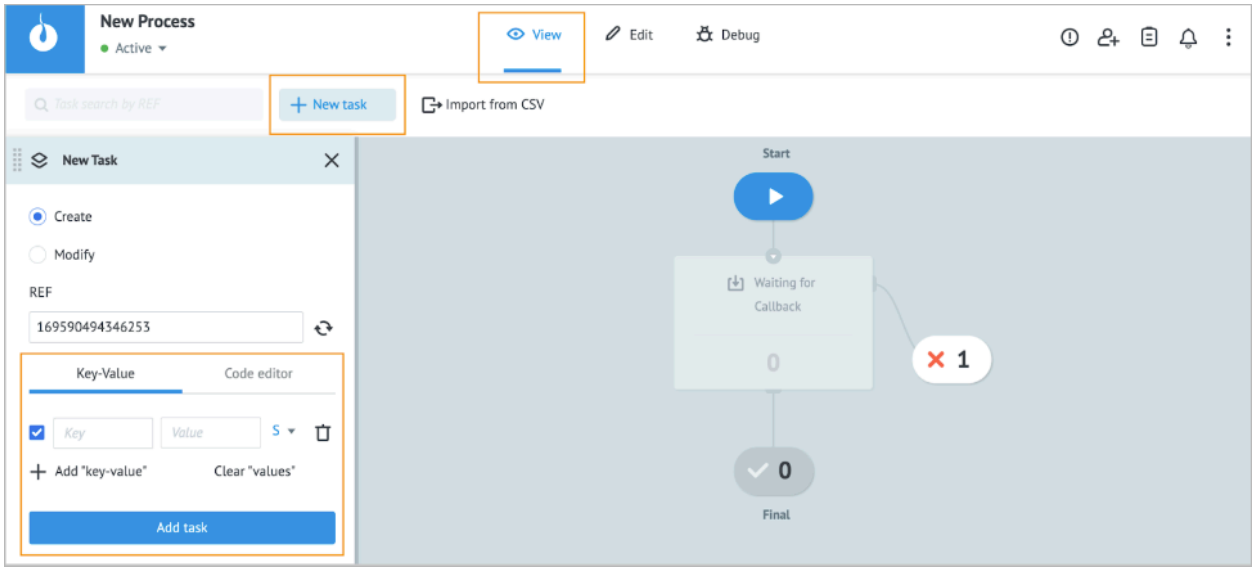
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

For one Process, the recommended number of tasks is ⩽100 000.

Send Task Manually

After you have set all the parameters in your Process, you can send the task to it manually. To do so:

1. Switch to the View mode by clicking View on the upper panel.
2. In the upper-left corner of the page, click + New task.
3. In the New task dialog that opens, enter the key and value in the Key and Value fields correspondingly, and then select the needed type of data in the dropdown list (S (String), N (Number), B (Boolean), A (Array), O (Object)).
4. Click Add task.



Screenshot 40: **Task view in the archive.**

The task goes to:

- The Success End node if it meets the requirements specified in the node.
- The Error End node if it doesn't meet the requirements specified in the node.

Below you can see a short video about sending a task manually to a Process.

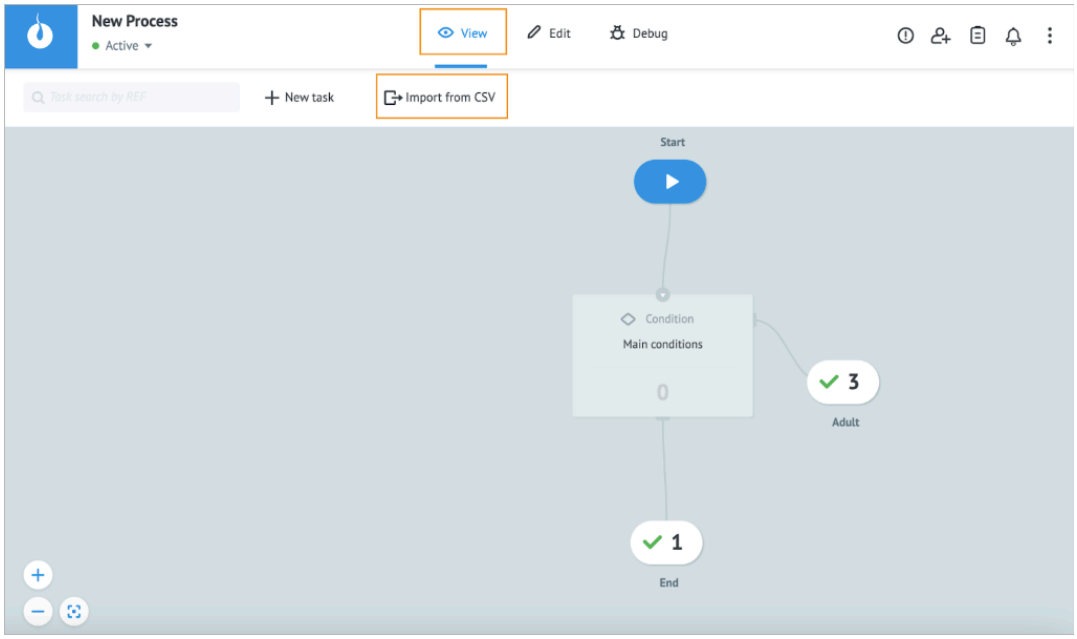
Screenshot 41: **Sending a task manually**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Import Task from CSV File

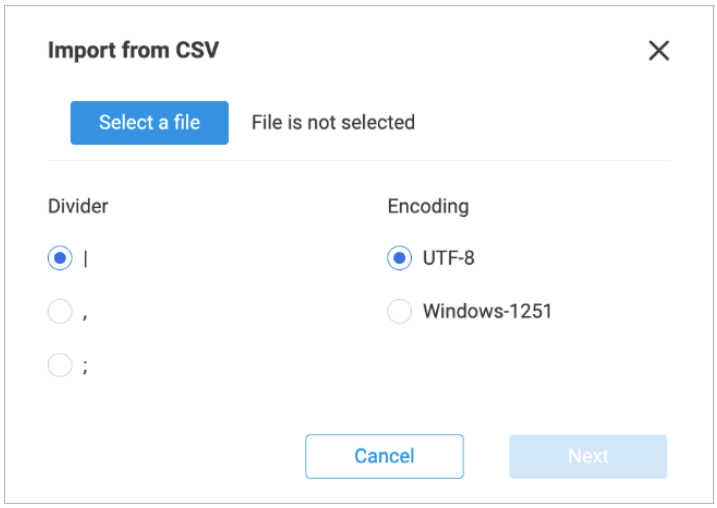
After you set all the parameters in your Process, you can import a task from a file. To do so:

1. Switch to the View mode by clicking View on the upper panel.
2. In the upper-left corner of the page, click Import from CSV.



Screenshot 42: **Task view in the archive.**

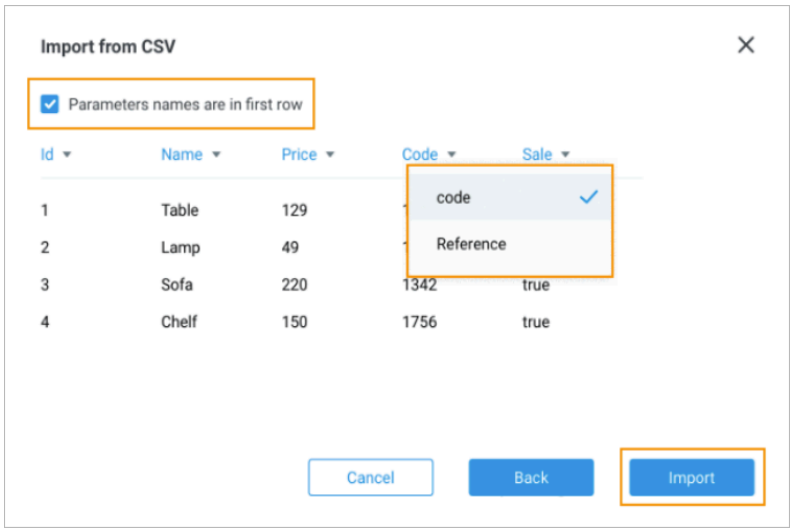
3. In the Import from CSV dialog that opens, click Select a file to select the needed file, select Divider and Encoding, and then click Next.



Screenshot 43: **Task view in the archive.**

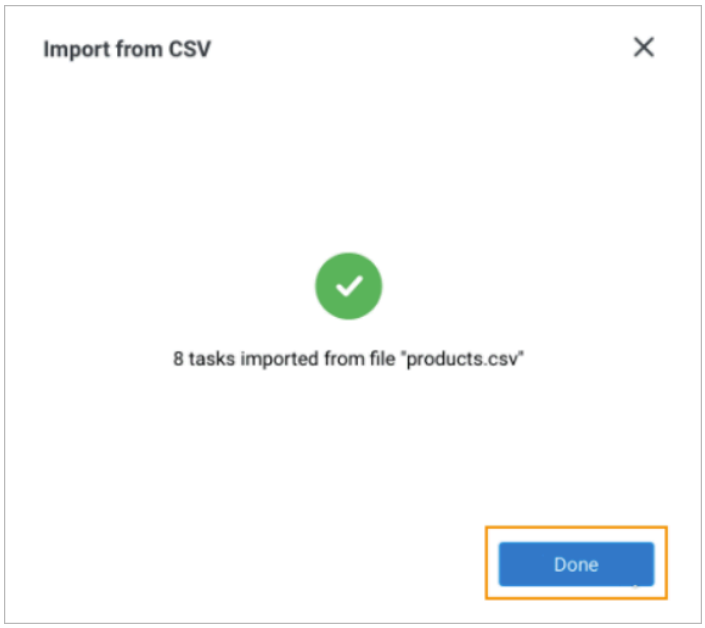
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

4. In the next Import from CSV dialog that opens, select the Parameters names are in first row checkbox, select the needed column name for each column, and then click Import.



Screenshot 44: **Task view in the archive.**

5. After the file has been imported, click Done.



Screenshot 45: **Task view in the archive.**

The task goes to:

- The Success End node if it meets the requirements specified in the node.
- The Error End node if it doesn't meet the requirements specified in the node.

Below, you can see a short video about importing a task from the CSV file to a Process.

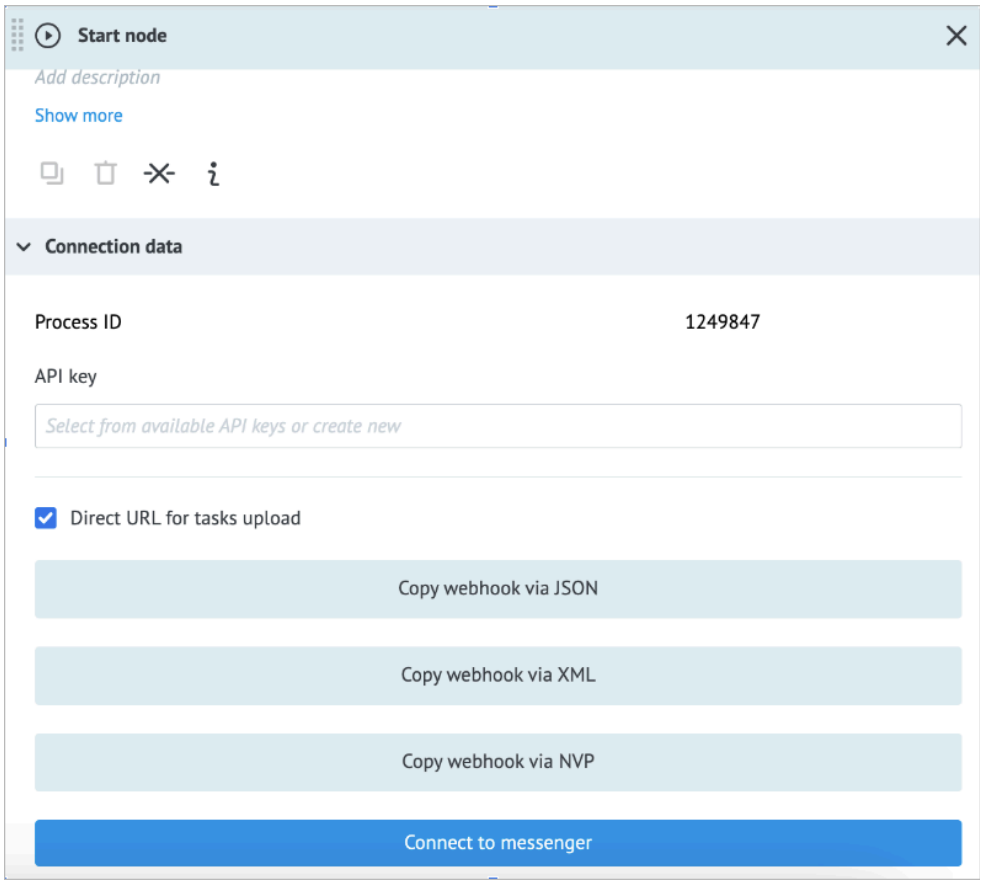
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Screenshot 46: **Task import from a CSV file.**

Load Task via Direct URL

By using the Direct URL option, you can load a task via JSON, XML, or NVP to a Process and enable the authorization for task loading (see Start for details on authorization use). To do so:

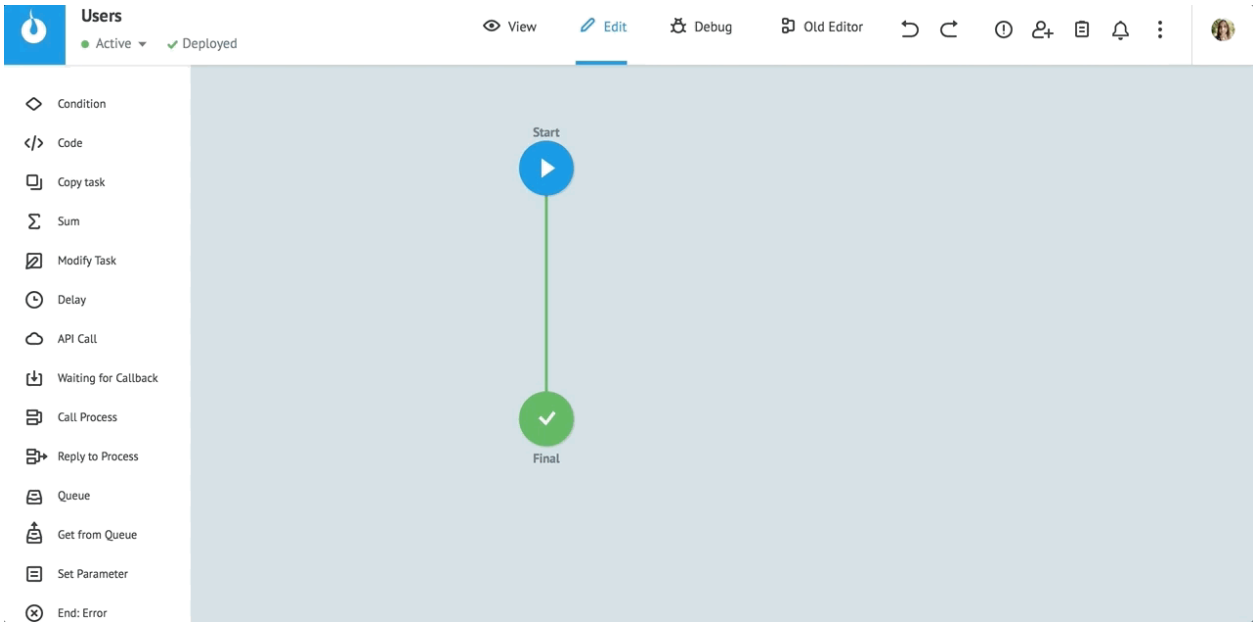
1. Open the needed Process in the Edit mode, and then click the Start node.
2. On the Start node details panel that opens, leave the Direct URL for tasks upload checkbox selected by default, and then click the needed way to load a task:
    - Copy webhook via JSON
    - Copy webhook via XML
    - Copy webhook via NVP
3. Send your payload to the URL by using the POST method.



Screenshot 47: **Task view in the archive.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Below, you can see a short video about loading task data via direct URL.



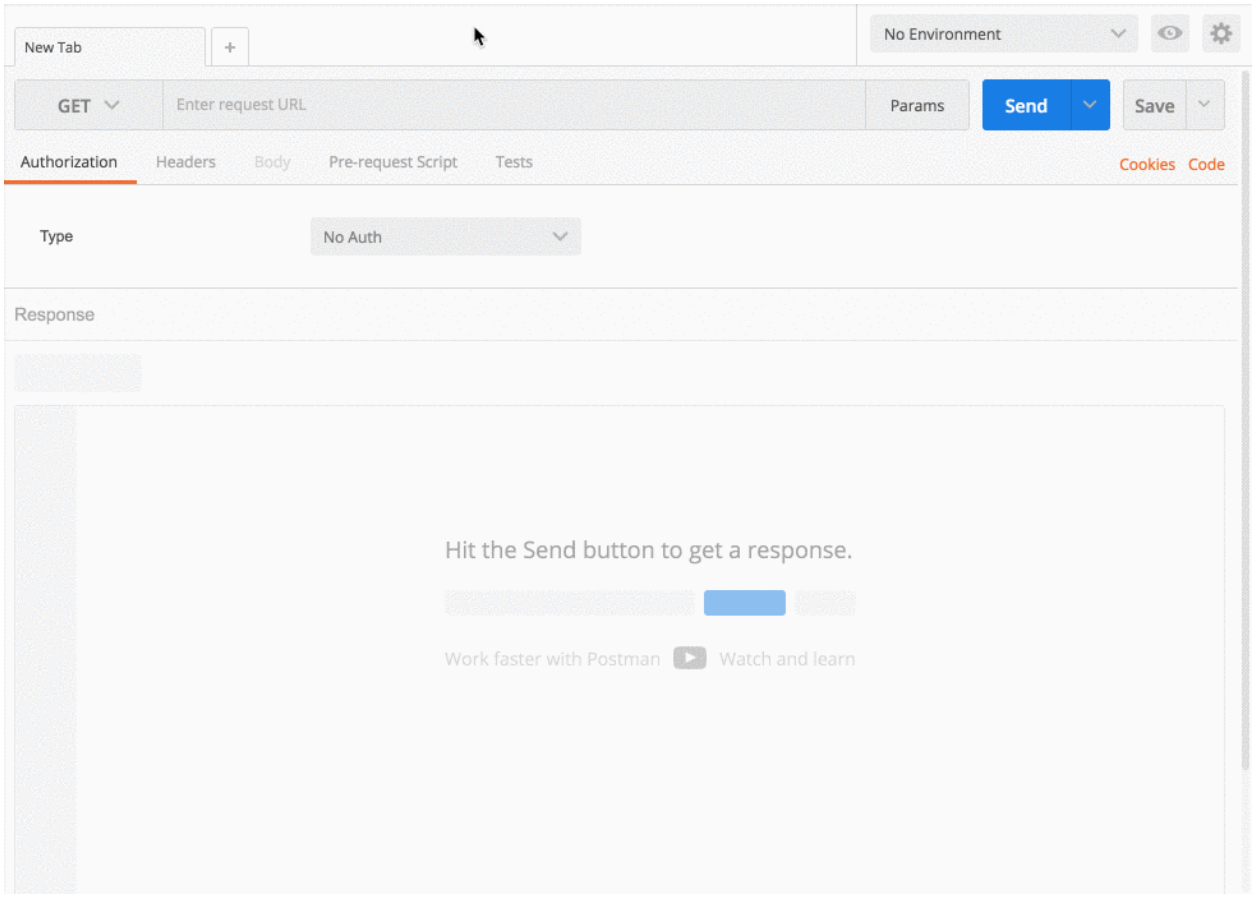
Screenshot 48: **Task view in the archive.**

To test how it works, use Postman:

- Type: POST
- URL: Direct URL from the clipboard
- Body: Request body, for example {"text":"Hello!"}
- Type of Body: raw
- Content/type: JSON (application/json)

The response contains the task ID (the obj_id parameter of the ops object). See the video below.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 49: **Task view in the archive.**

2.3.2.2 Task Parameters

Task parameter name

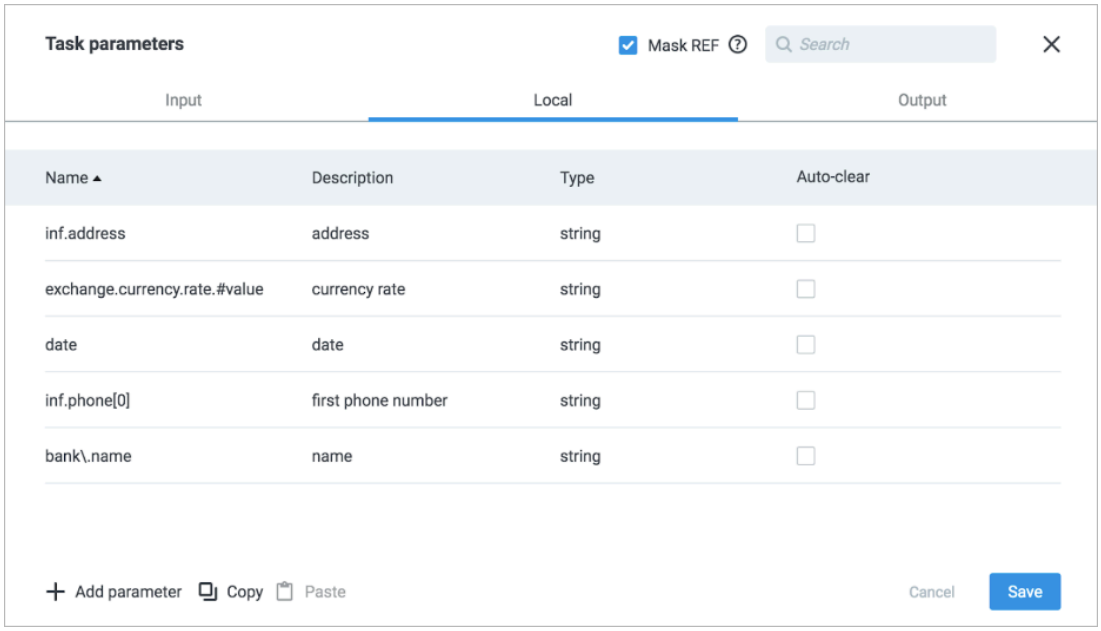
The task parameter name must be unique for all tabs and can contain:

- Big and small Latin letters
- Numbers
- Special characters: _, #, @, $, , [,], .

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

If the parameter name contains special characters like ".", "[" or "]", they have to be separated with a backslash:

- `phone[0]`: Element of the "phone" array
- `phone\[0\]`: Parameter "phone[0]"



Screenshot 50: **Task view in the archive.**

Task parameter type

The task parameter can be of the following types:

String (text and numbers together)

```
{
   "name": "John"

}
```

Number (integer, float):
Note: The maximum value for the Number type is 2^53 = 9 007 199 254 740 992. For larger values, use the String type.

```
{
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
    "card_number": 123456789,
    "bonus": 10.50
}
```

Boolean (true, false)

```
{
    "send_notifications": false,
    "subscribed": true
}
```

Array:

```
{
    "phones": [
        "+380991234567",
        "+380661234567"
    ]
}
```

Object (set of Key-value pairs)

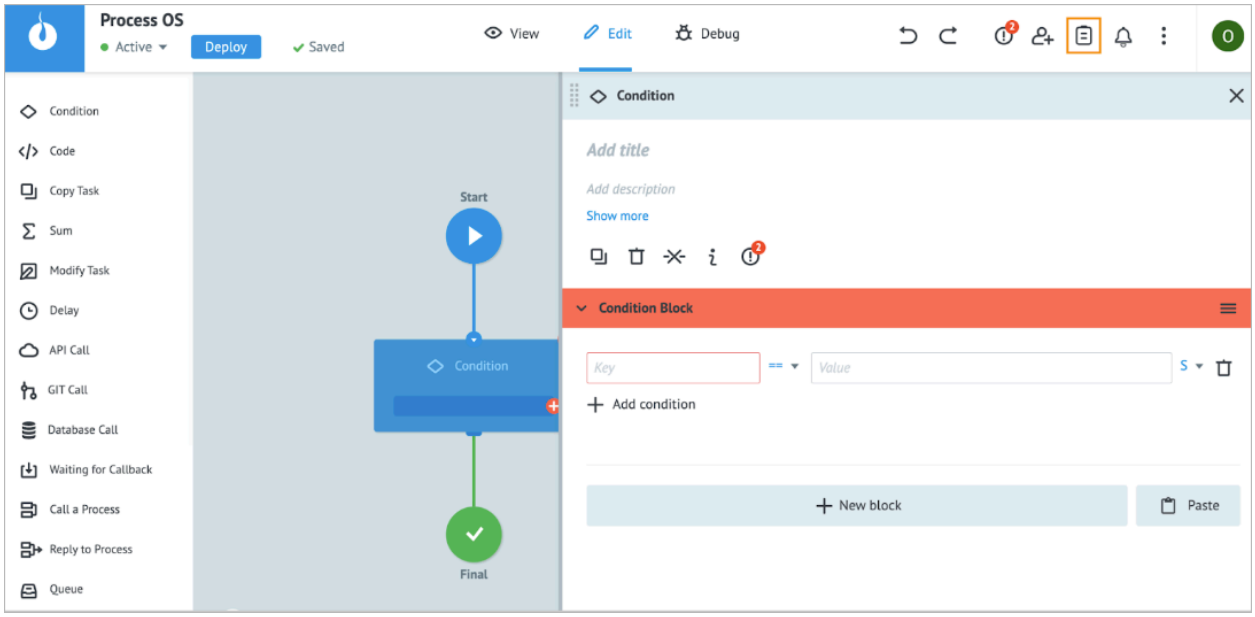
```
{
    "address": {
        "city": "Odessa",
        "zip_code": 65000
    }
}
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Set task parameters

Setting task parameters helps you save time when configuring data in the node or entering parameters in the task before sending it. To set task parameters:

1. In the upper-right corner of the page of your Process, click the parameters icon .



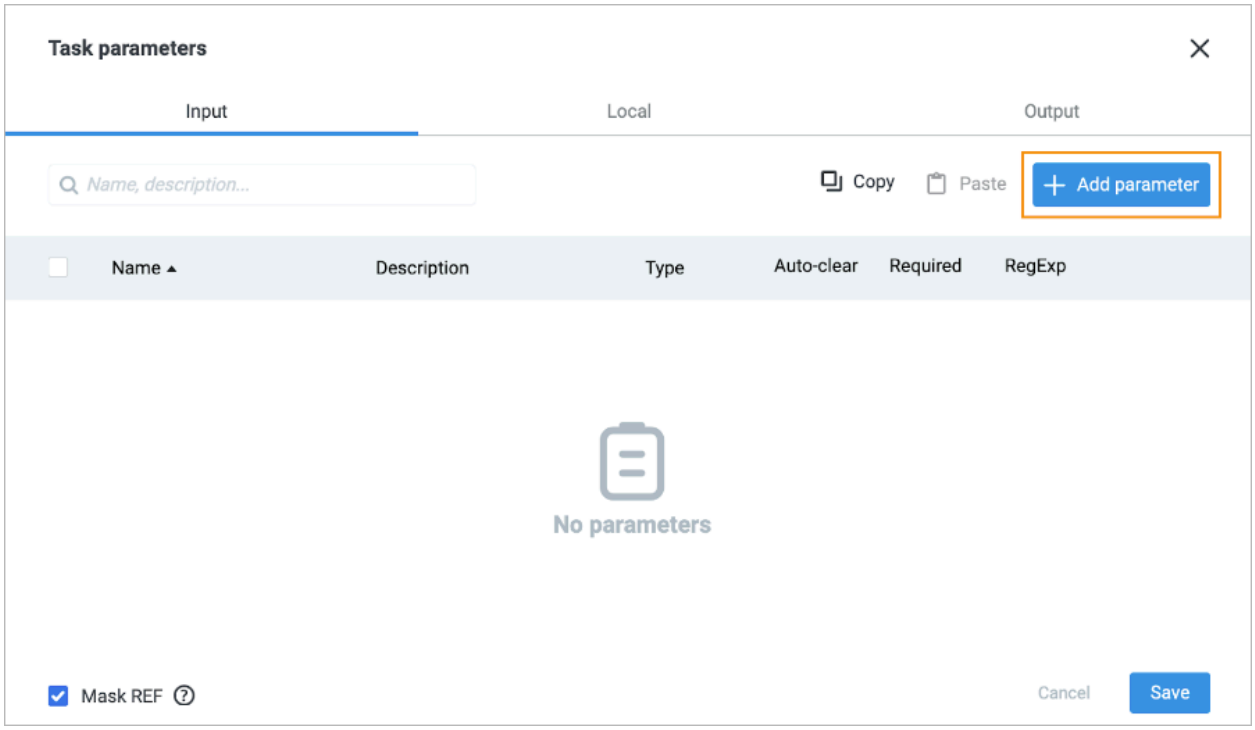
Screenshot 51: **Task view in the archive.**

2. In the Task parameters dialog that opens, click the needed tab (Input, Local, or Output), and then click + Add parameter.

   The Task parameters dialog has the following tab with parameters:

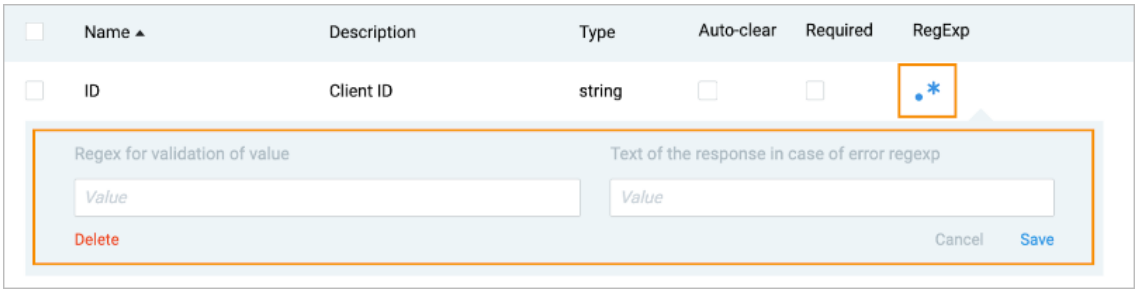
   - Input: Parameters which go to the Start node of a Process.
   - Local: Internal frequently used parameters for fast and convenient use in logic.
   - Output: Parameters, which are the result of a Process.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3. 

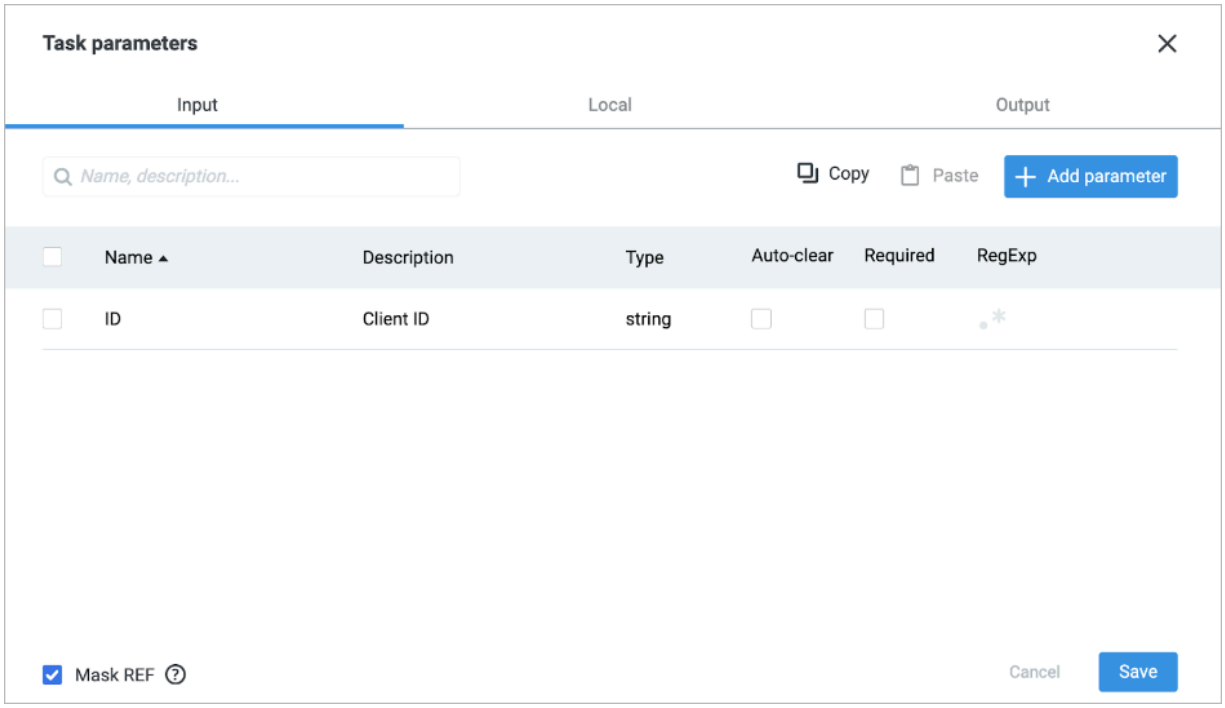
Screenshot 52: **Task view in the archive.**

4. In the table that opens:
   - Enter the parameter name in the Name column.
   - (Optional) Enter the parameter description in the Description column.
   - Specify the parameter type (S (String), N (Number), B (Boolean), A (Array), O (Object)) in the Type column.
   - Select the checkbox in the Auto-clear column to hide the parameter value (display as “***") in the Archive tab.
   - Select the checkbox in the Required column to set the parameter as required.
     Note: The Required column is only available for the Input and Output tabs.
   - Click .* in the RegExp column to use Regular Expression: enter the needed values (RegExp for validation of value and Text of the response in case of error regexp), and then click Save.
     Note: The RegExp column is available for the Output tab only



Screenshot 53: **Task view in the archive.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

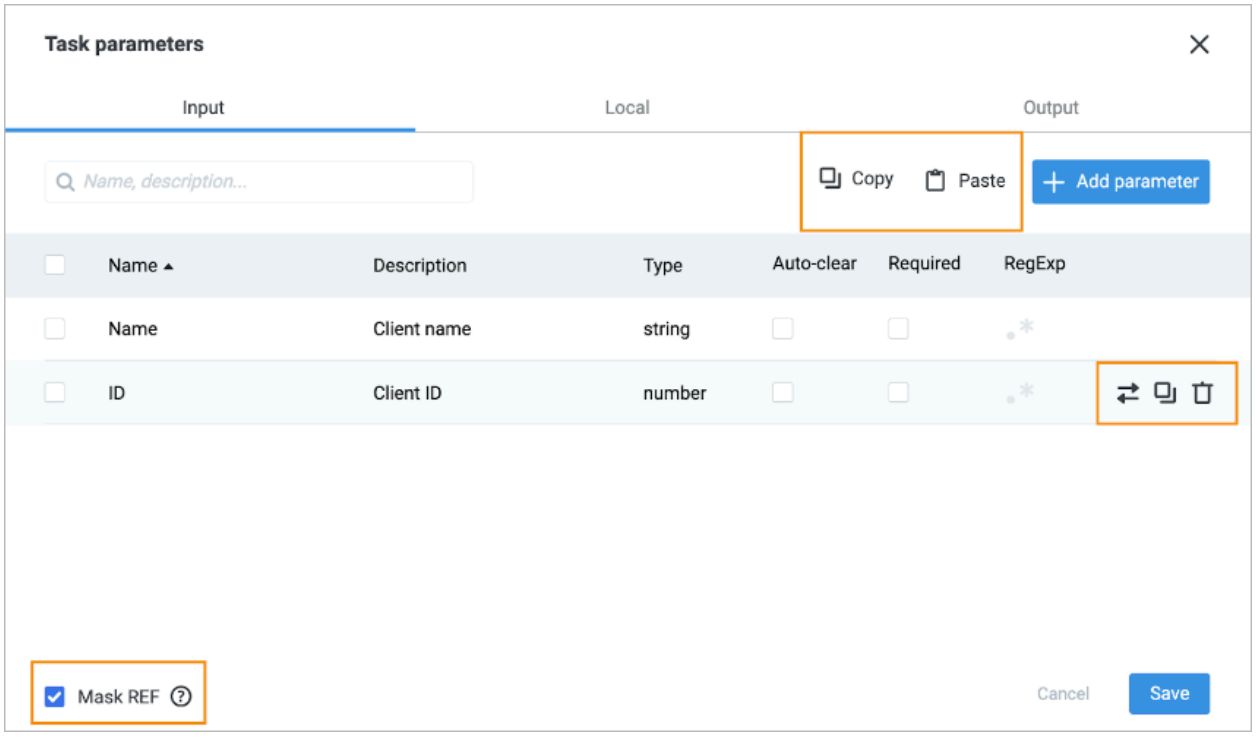


Screenshot 54: **Task view in the archive.**

In the Task parameters dialog, you can perform additional actions:

- Copy and paste the needed parameter by selecting the checkbox and clicking Copy and Paste consecutively
- Move the needed parameter to another tab (Local or Output) by hovering over the needed parameter row and clicking the arrows icon
- Copy the needed parameter by hovering over the needed parameter row and clicking the copy icon
- Delete the needed parameter by hovering over the needed parameter row and clicking the trash icon
- To turn off task reference masking (for 3-digit and 16-digit numbers that fall within the range of possible CVV and PAN pay card numbers), select the Mask REF checkbox.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



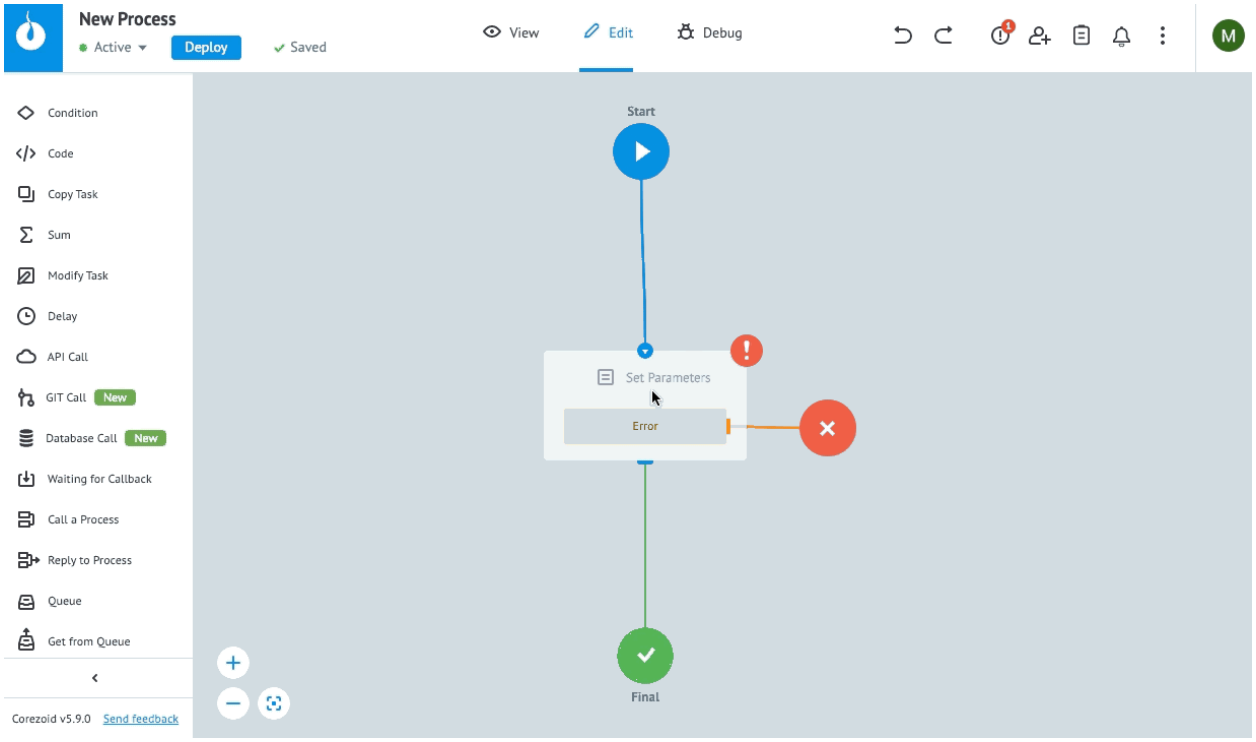
Screenshot 55: **Task view in the archive.**

5. In the lower-right corner of the Task parameters dialog, click Save.

The added parameters are saved, and you can use them when configuring data in the node or sending tasks to a Process.

Below, you can see a short video about adding a task parameter on the node details panel. By default, the parameter is added to the Local tab.

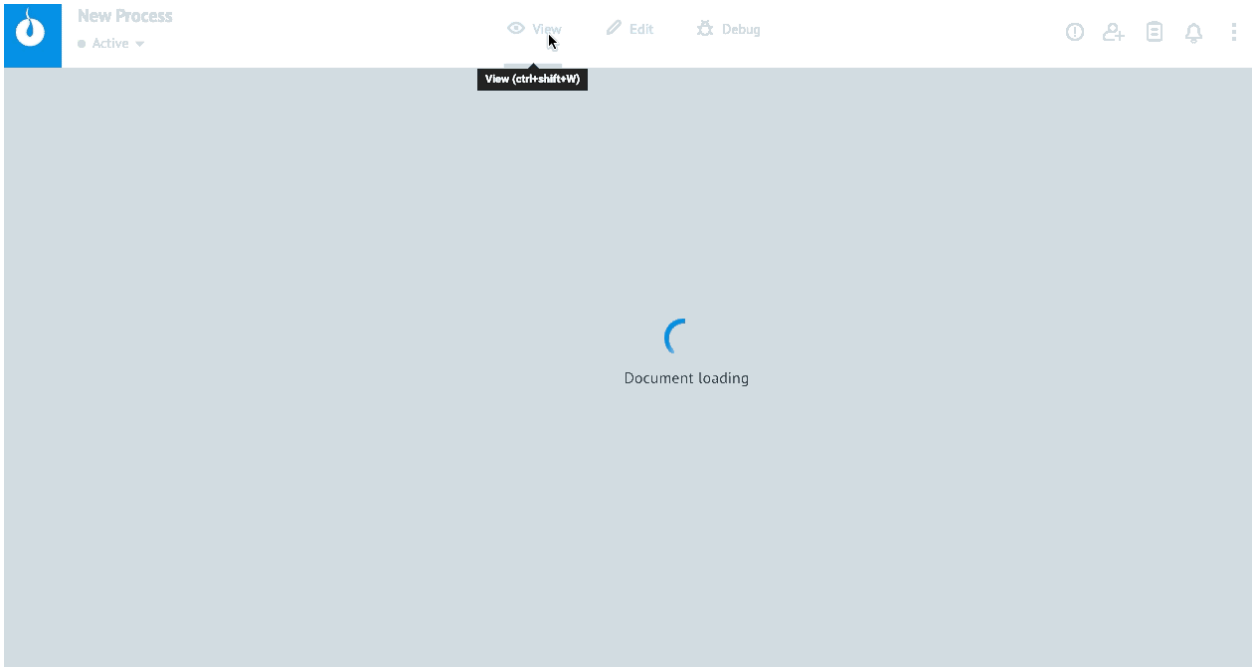
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 56: **Task view in the archive.**

Why set task parameters

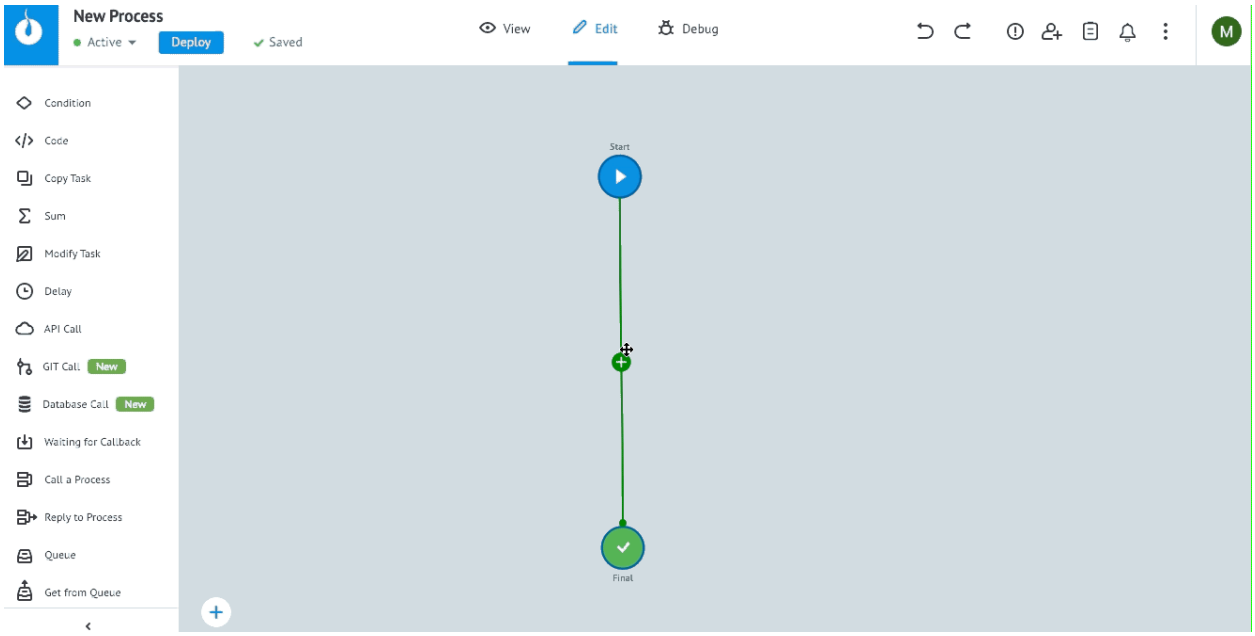
- Input tab: The task parameters of the Input tab are automatically copied to a new task.



Screenshot 57: **Task view in the archive.**

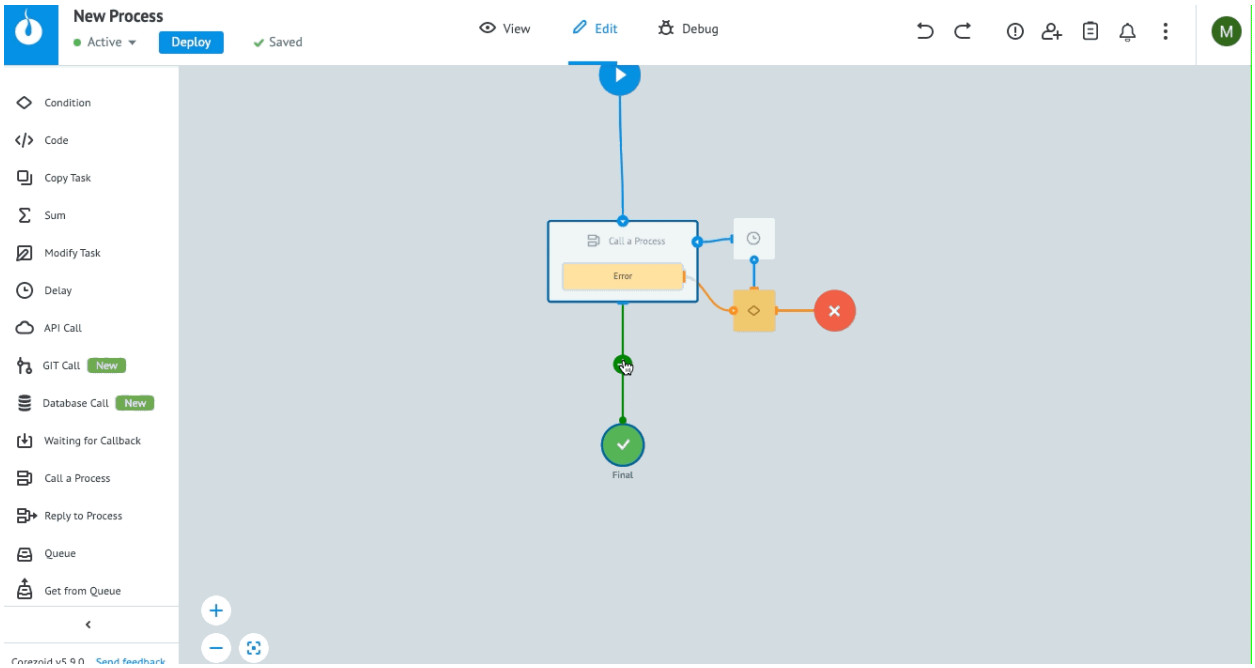
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

When adding the Call a Process, Copy Task, and Modify Task nodes to a Process, the Input parameters of the selected Process are automatically filled in.



Screenshot 58: **Task view in the archive.**

- Output tab: The task parameters of the Output tab are used to configure the Reply to Process node.



Screenshot 59: **Output tab.**

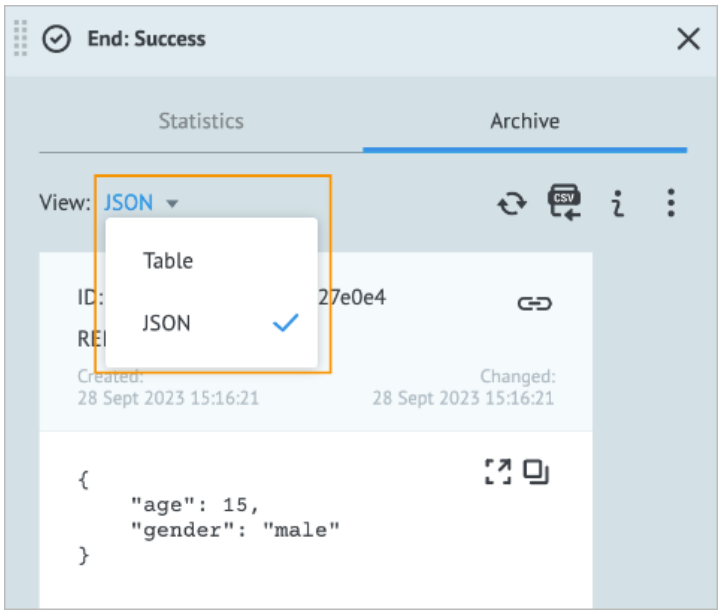
- Local tab: The task parameters of the Local tab are shown in the dropdown list when creating a task.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

2.3.2.3 Task Management

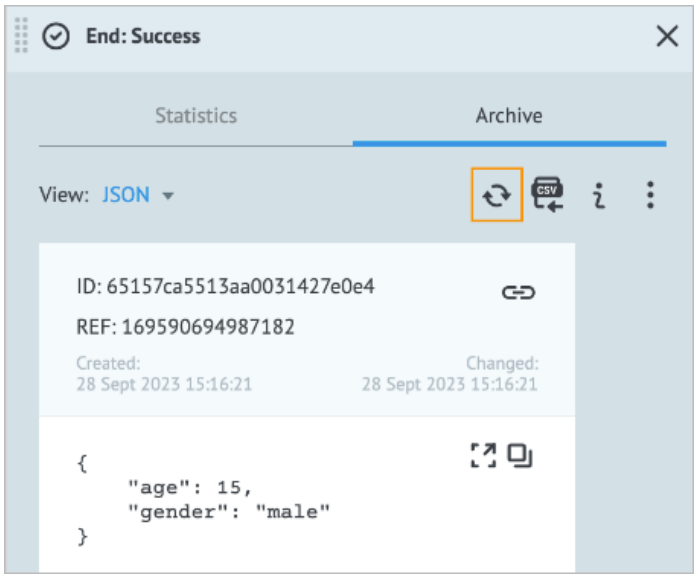
In the task dialog of the End node, you can:

- View task data in the JSON or Table formats by clicking the expand icon next to View



Screenshot 60: **View task data.**

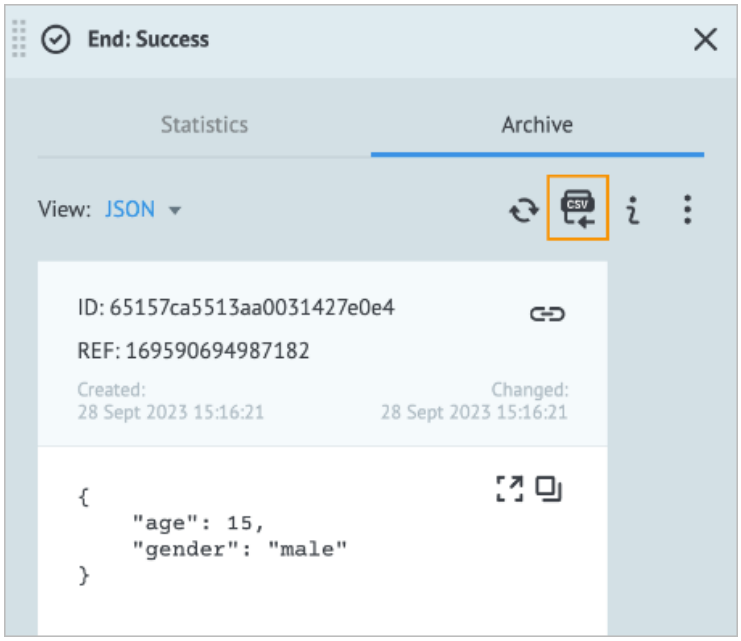
- Renew task data by clicking the refresh icon



Screenshot 61: **Renew task data.**

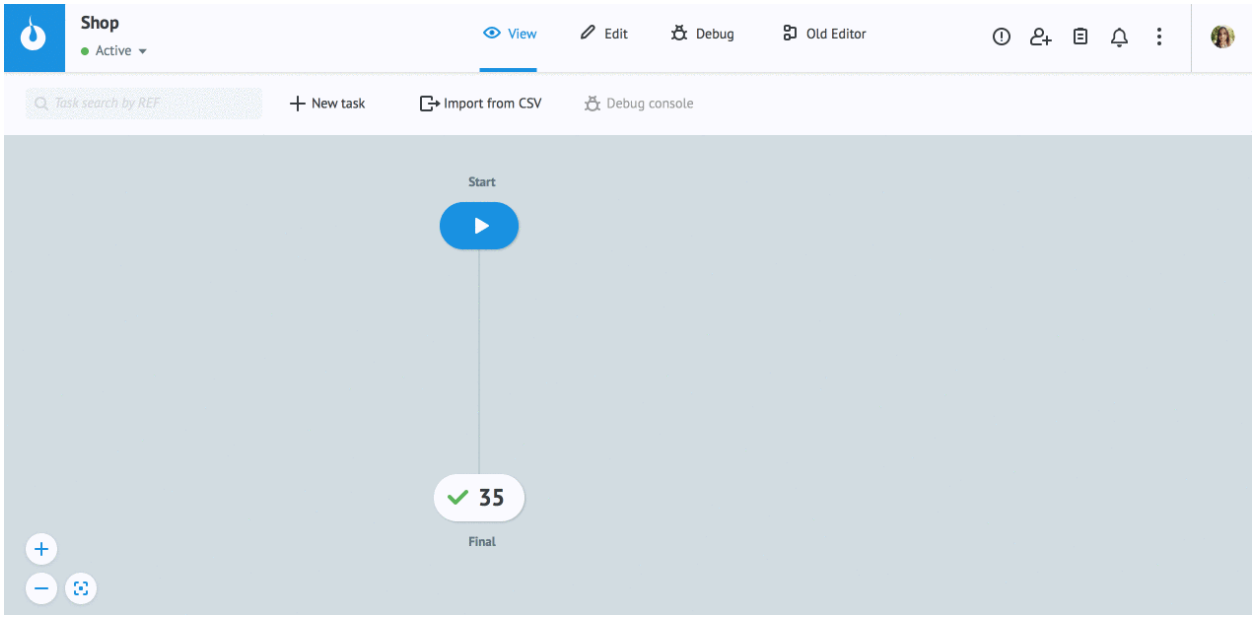
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- Export a task to a CSV file by clicking the CSV icon



Screenshot 62: **Export to CSV icon.**

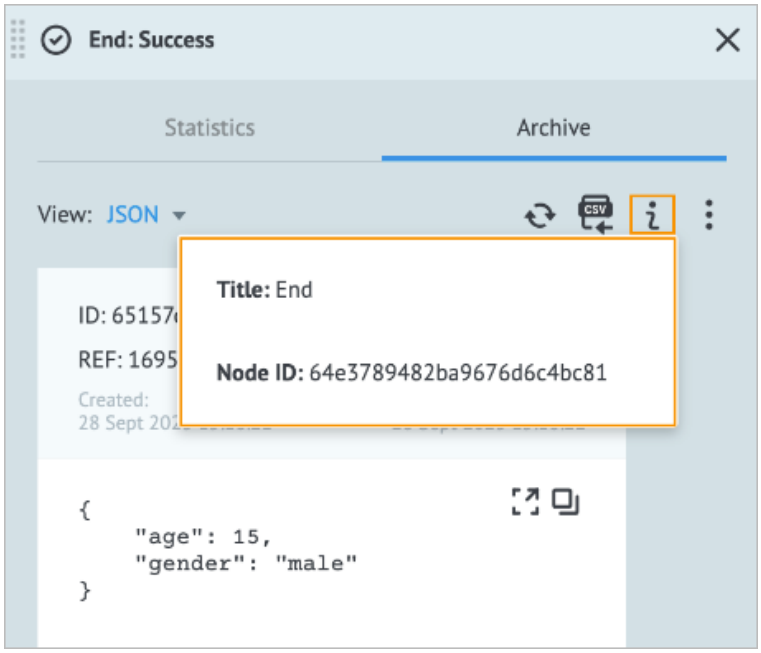
Below, you can see a short video about exporting a task to a CSV file.



Screenshot 62: **Export to CSV.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

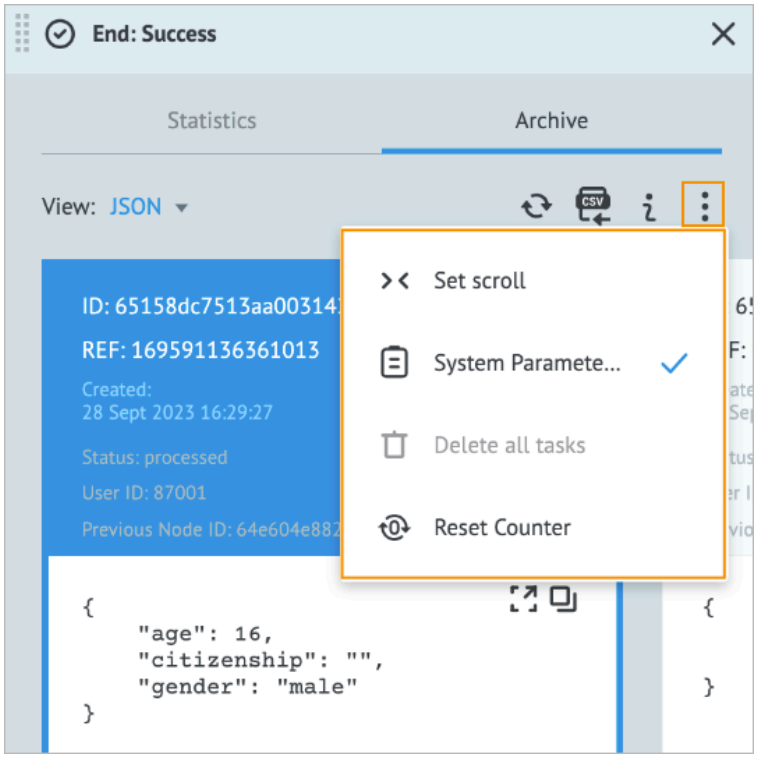
- View the node info by clicking the info icon



Screenshot 63: **Info menu.**

- By clicking the options icon :
  - Set scroll (vertical or horizontal)
  - View system parameters
  - Delete all tasks (except Success and Error End nodes)
  - Reset counter

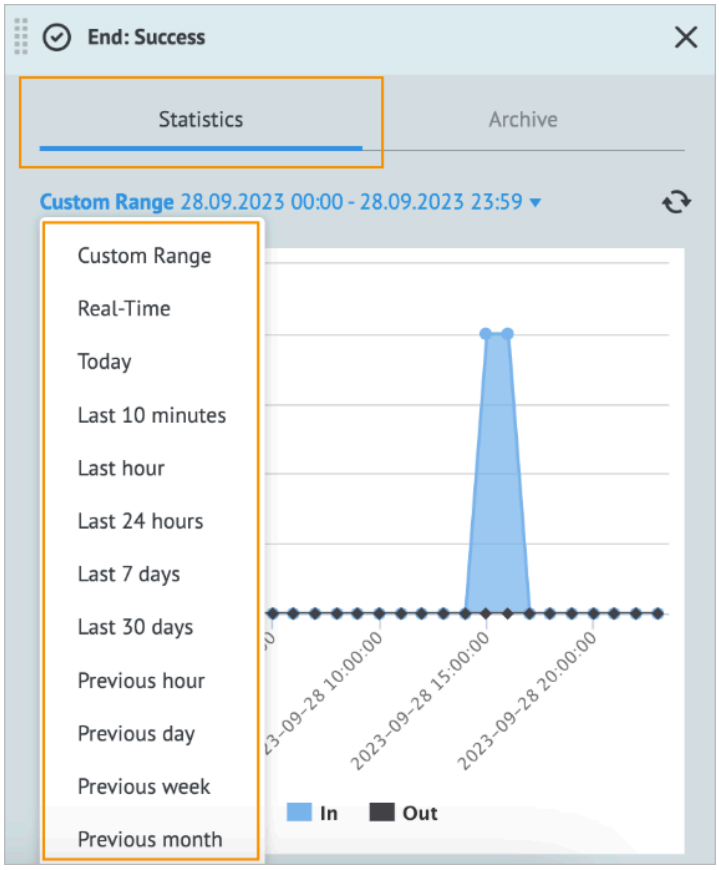
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 64: **Three-dot menu.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

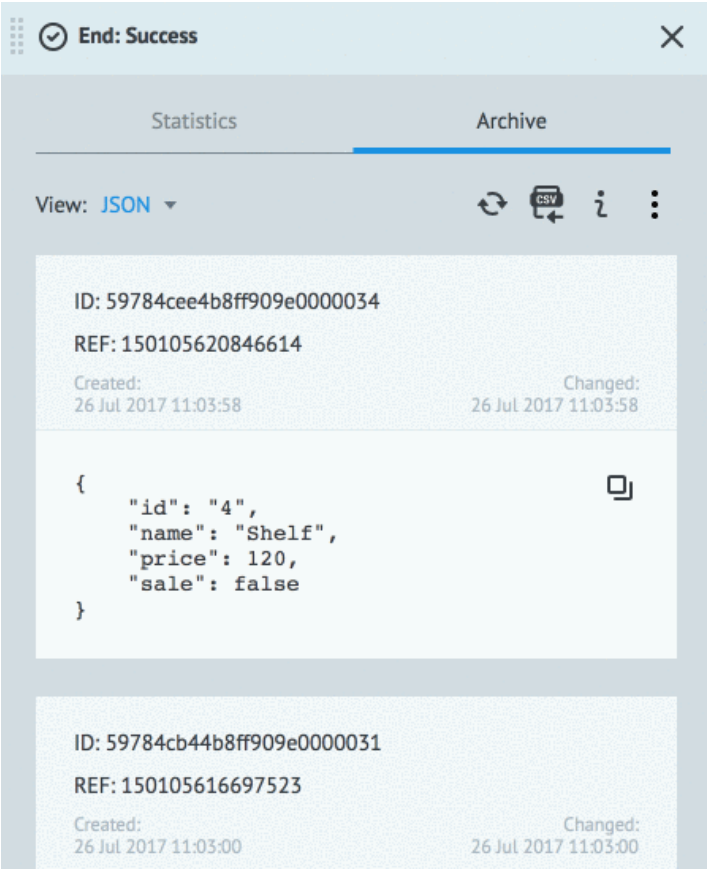
- View tasks statistics by clicking Statistics and selecting the needed time range



Screenshot 65: **Statistics tab.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Below you can see a short video about viewing task statistics.



Screenshot 66: **Statistics display in real time.**

Note: The tasks are stored in a Process until the end of the month, and on the first day of each month they are deleted. But, the final node counters always show the total number of tasks received.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

2.3.3 Processes

A **Process** represents a sequence of steps or tasks designed to achieve a specific goal. Processes are the backbone of Corezoid workflows. They include:

- **Start and End Points**: Define where the Process begins and concludes.
- **Transitions**: Logical connections between tasks or nodes.
- **Triggers**: Events that initiate or modify Process execution.

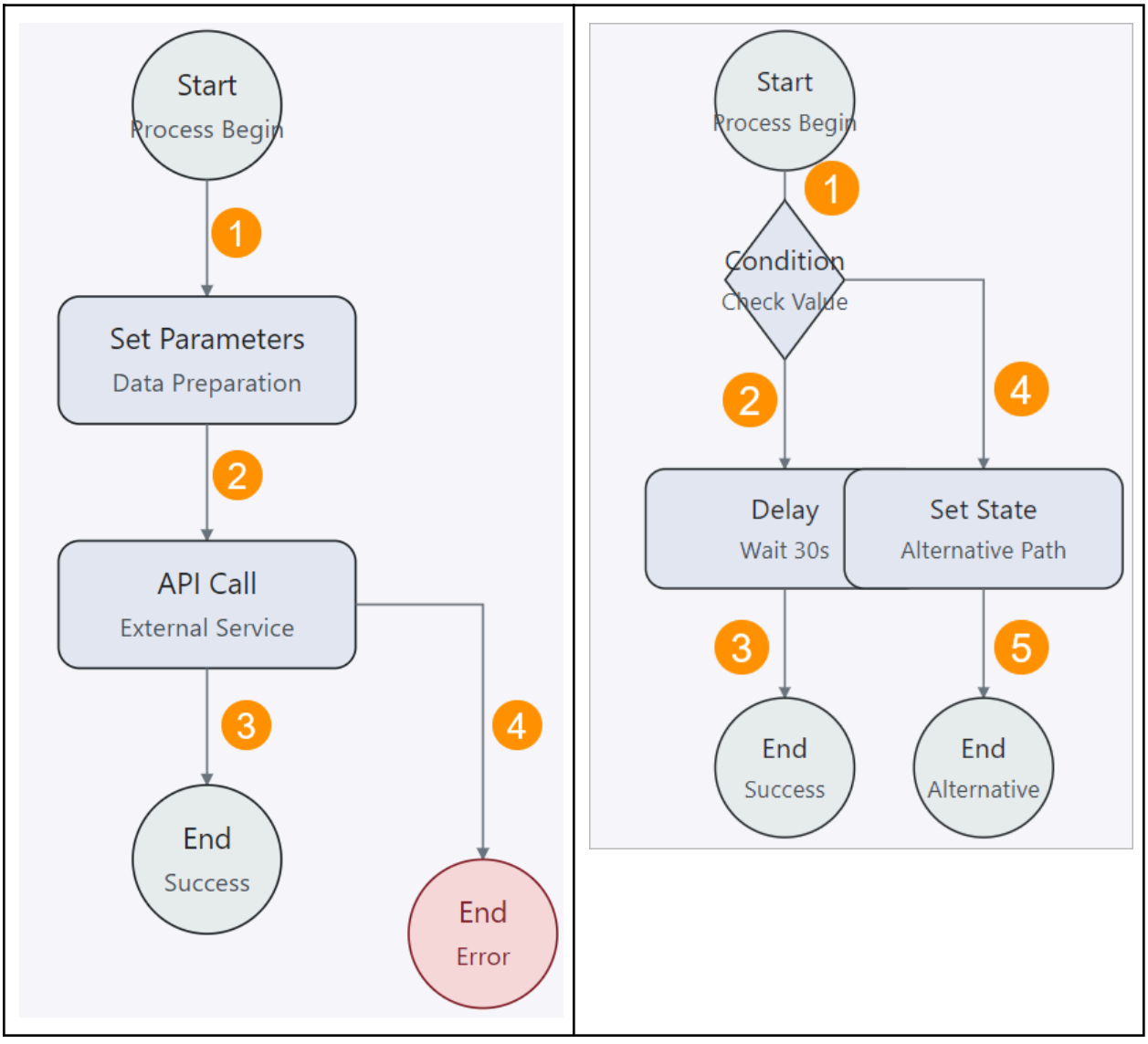


Diagram 4: **Example of a simple Process flow with start, action, and end nodes.**

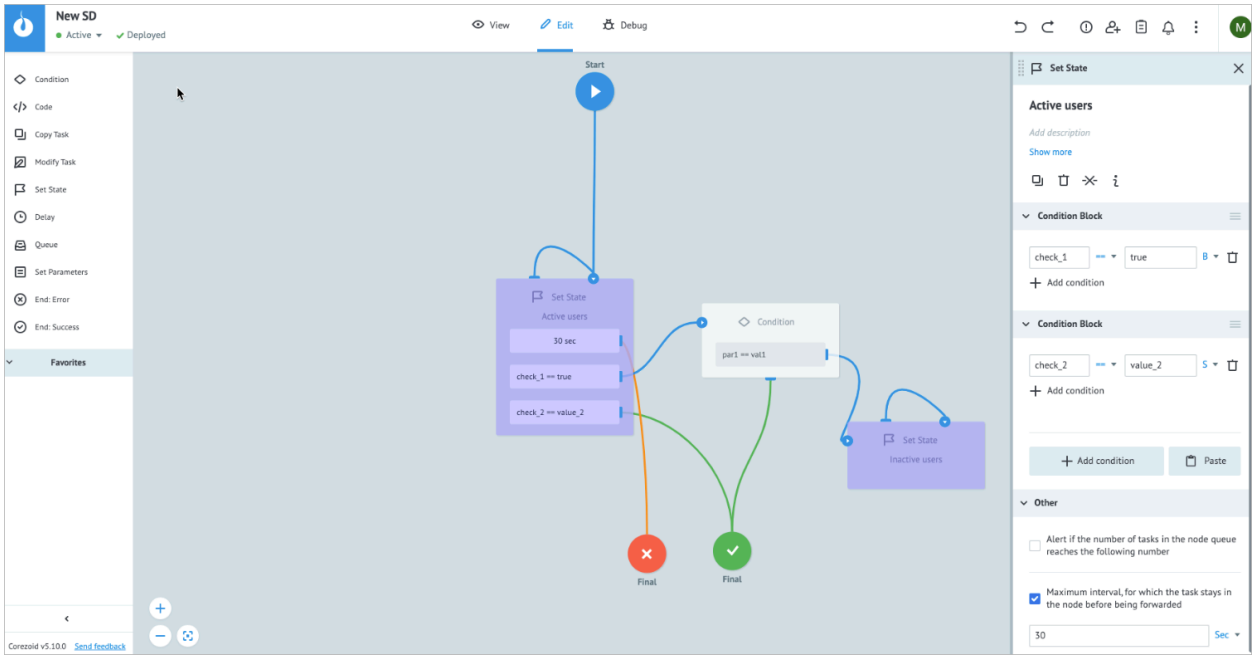
For more details, see **Processes** in the Glossary.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

2.3.4 State Diagrams

A **State Diagram** is a graphical representation of a Process, showing how data flows between different states and nodes. Benefits include:

- Visual clarity for complex workflows.
- Simplified debugging by tracing transitions.
- Enhanced collaboration with a shared visual model.



Screenshot 67: **State diagram example.**

More about this can be found in the Glossary under **State Diagrams**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

2.3.5 Parameters

Parameters can be of the following types:

- Task parameters (Global and Local)
- Node parameters (Callback URL, Count, and Sum)

For more information about how to get task parameters from a State Diagram, go to Getting task parameters from State Diagram.

Task parameters

Task parameters can be Global and Local.

Global parameters

System-defined, read-only parameters available in Condition, Set Parameters, API Call, Queue, Copy Task, Call a Process, Reply to Process, Modify Task, and Sum nodes.

The **Global** parameters are the parameters defined by the system. You can use them, but you can't customize them. In the table below, you can see the list of **Global** parameters, their types, and description.

Parameter name	Data type	Description
root.prev_node_id	string	Previous node ID.
root.user_id	number	ID of the user, who created or changed the task last.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

root.end_time	number	Dynamic value of the start timer (for example, the timer that has to wake you up at 8 a.m.).
root.change_time	number	Task change time in the **Unix Time** format.
root.create_time	number	Task creation time in the **Unix Time** format.
root.task_id	string	Task ID generated by the Process.
root.node_id	string	Node ID.
root.conv_id	number	Process ID.
root.ref	string	Task reference.

Table 1: **Global parameters**.

In the video below, you can see how to access the **Global** parameters from the node details panel.

Screenshot 68: **Global parameters access.**

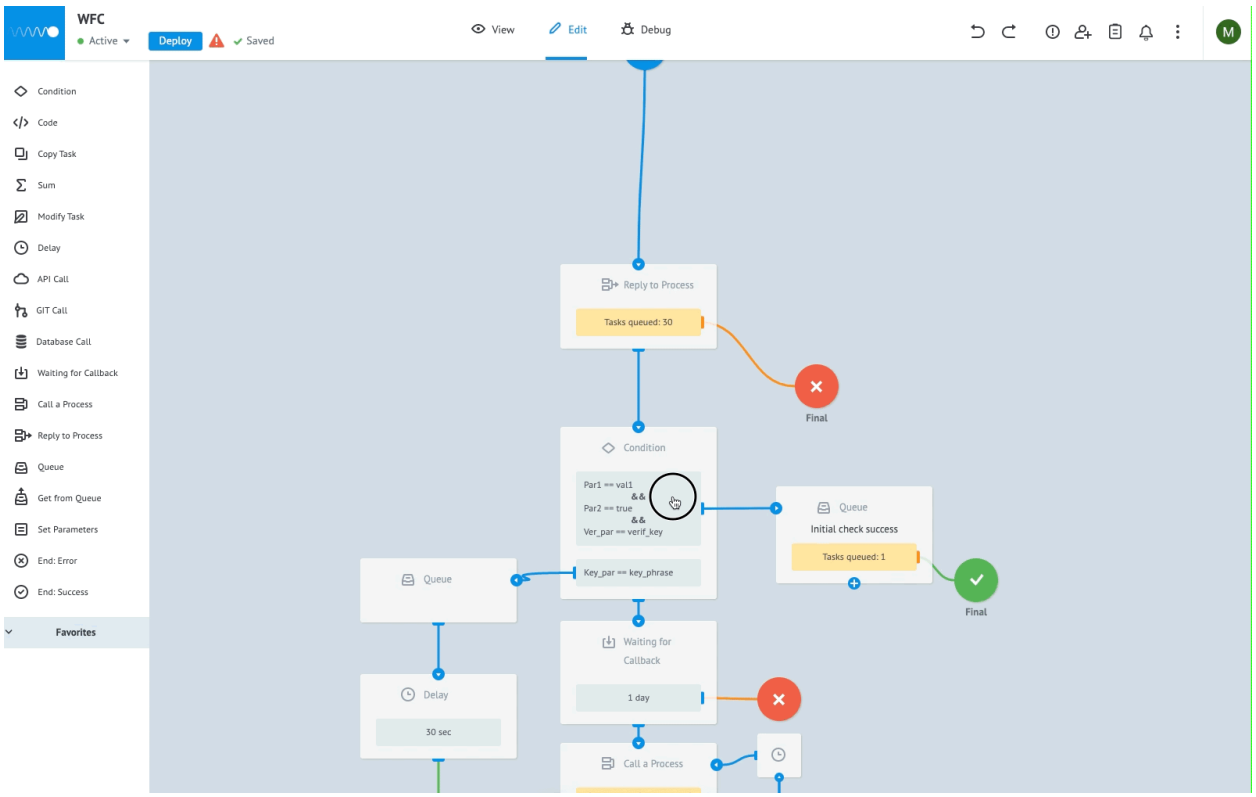
Local parameters

User-defined parameters added manually to Process or State Diagram.

For more information on how to add **Local** parameters, go to Set task parameters.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

In the video below, you can see how to access the Local parameters from the node details panel.



Screenshot 69: **Access the Local parameters from the node details panel**.

Getting task parameters from State Diagram

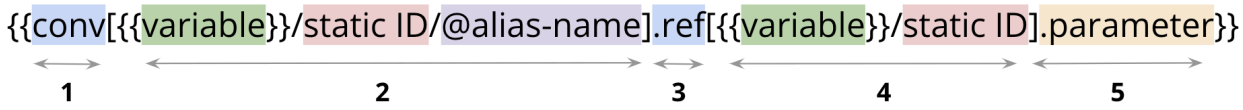
Precondition:

- Existing State Diagram.
- Stored tasks.
- Access to the State Diagram.

Corezoid tasks are stored in the JSON format, meaning that they are represented as objects.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

You can use a State Diagram as a data storage from where you can get a single or multiple parameters by using the following template for your request:



Screenshot 70: **State Diagram parameters scheme**

Where:

1. **conv**: Is a constant keyword used to get data from a State Diagram.
2. Methods to define the State Diagram:
    - **variable**: Is data that varies depending on the entity that you refer to. For example, {{Process_ID}} that stores a different ID number for each Process.
    - **static ID**: Is an exact data, for example, a Process ID.
    - **@alias-name**: Is the name of the alias you are calling.
3. **.ref**: Is a constant keyword used to get data from a task with a corresponding reference number.
4. Data type:
    - **variable**: Is data that varies depending on the entity that you refer to. For example, {{Reference_ID}} that stores a different ID number for each reference.
    - **static ID**: Is an exact data, for example, a reference ID.
5. (Optional) **.parameter**: Is a specific parameter name that you are retrieving.

When working with a State Diagram, you can get:

- The whole task by using the following parameter request:
  `{{conv[{{Process_ID}}].ref[{{REF}}]}}`
- Where:
    - **Process_ID** is the Process ID of the State Diagram (can be found by clicking the **Start** node in the **Edit** mode).
    - **REF** is the reference of the task.
- For example: The following statement will return all the task parameters of the **12345** task from the **Customers** State Diagram:
  `{{conv[customers].ref[12345]}}`
- A single task parameter by using the following static parameter request:
  `{{conv[{{Process_ID}}].ref[{{REF}}].parameter}}`
- Where:
    - **Process_ID** is the Process ID of the State Diagram (can be found by clicking the **Start** node in the **Edit** mode).
    - **REF** is the reference of the task.
    - **parameter** is the parameter you want to retrieve.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

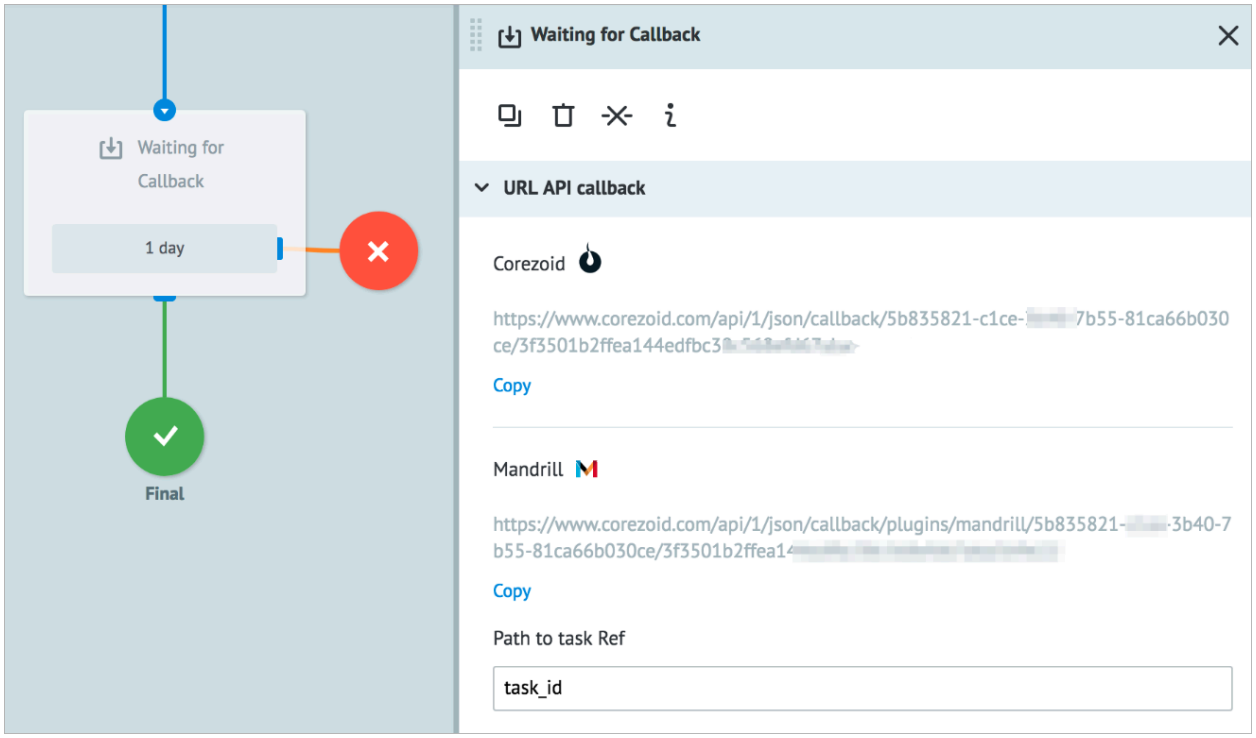
- For example: The following statement will return the **amount_owed** task parameter of the **12345** task stored in the **Customers** State Diagram:
  `{{conv[customers].ref[12345].amount_owed}}`
- Corezoid will try to automatically match the type if the accessed value type doesn't match the one specified. To avoid potential errors, specify the correct parameter type of the value you are trying to get.
- A single task parameter by using a dynamic parameter request, for example:
  `{{conv[{{my_proccess_name}}].ref[{{my_task_reference}}].amount_owed}}`
- Where:
  - **Process_ID** and **REF** are names of variables that store their values in double curly braces and not the exact values.
  - **parameter** is a static value.
- A single task parameter by using their alias, for example:
  `{{conv[@alias-name].ref[{{REF}}]}}`

Node parameter examples

Callback URL

With the **Callback URL** function, you can get a Callback URL in the **Waiting for Callback** node dynamically.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 71: **Getting a Callback URL in the Waiting for Callback node dynamically**.

For example:

```
// Returns Corezoid or Mandrill CallbackURL by {{node_id}}

        {{node[{{node_id}}].public_callback_corezoid}}

        {{node[{{node_id}}].public_callback_mandrill}}

// Returns Corezoid or Mandrill CallbackURL by {{node_id}} from
{{conv_id}} process

{{conv[{{conv_id}}].node[{{node_id}}].public_callback_corezoid}}
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
{{conv[{{conv_id}}].node[{{node_id}}].public_callback_mandrill}}
```

Count

The **Count** function returns the amount of tasks in a node with the specified ID. For example:

```
// Node ID = 561a272782ba961374d44178
```

```
        {{node[561a272782ba961374d44178].count}}
```

```
// Returns amount of tasks in the node specified by the parameter
{{node_id}}
```

```
        {{node[{{node_id}}].count}}
```

```
// Returns amount of tasks in the node specified by the parameter
{{node_id}} from {{conv_id}} process
```

```
        {{conv[{{conv_id}}].node[{{node_id}}].count}}
```

Sum

The **Sum** function returns the sum value for the selected task parameter. For example:

```
// Node ID = 561a272782ba961374d44178
```

```
        {{node[561a272782ba961374d44178].SumID}}
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
// Returns amount by SumID parameter from node {{node_id}}

        {{node[{{node_id}}].SumID}}

// Retuns amount by SumID parameter from {{node_id}} from {{conv_id}}
process

        {{conv[{{conv_id}}].node[{{node_id}}].SumID}}
```

SumID value

Where the **SumID** is the Sum node ID value. See the article.

2.3.6 Functions

The functions listed below can be used in the Condition and Set Parameters nodes. Please refer to the Erlang documentation for more details on the functions supported in Code nodes.

Using functions, you can perform various operations on data to get desired values in your Processes and State Diagrams.

Basic functions in use

Below are the basic functions for configuring nodes in your Processes and State Diagrams.

$.math()

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

With the $.math() function, you can perform mathematical operations (addition, subtraction, multiplication, and division).
For example:

- To calculate the 1+1 result, use:
  `$.math(1+1)`
- To calculate the sum of the specified parameter's value + 10, use:
  `$.math({{parameter_name}}+10)`
- To calculate the difference between two parameters, use:
  `$.math({{parameter_name1}}-{{parameter_name2}})`

$.random(N)

The $.random(N) function returns a random number depending on the specified N, where:

- .random(0): always returns 0.
- .random(N), N>1: returns a positive random number.
- $.random(N), N<0: returns a negative random number.

If the N value is bigger than int64, int64 will be used instead.

$.sha1_hex

The $.sha1_hex(content) function returns sha1 in hex from a constant or task parameter if it is specified in the {{parameter_name}} format. For example, to get the sha1 in hex for the constant 100, use:

$.sha1_hex(100)

$.md5_hex

The $.md5_hex(content) function returns md5 hash in hex generated from a constant or task parameter if it is specified in the {{parameter_name}} format. For example, to get the md5 in hex for the constant 100, use:

$.md5_hex(100)

$.base64_encode

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

The $.base64_encode(content) function converts a constant or a task parameter in the base64 format if it is specified in the {{parameter_name}} format, where content means a constant or {{parameter_name}}.

$.unixtime()

The $.unixtime() function returns the current time in Unix Time in GMT 0. You can use arithmetic arguments to adjust the current time value.
For example:

- To return the current time in Unix Time, use:
  `$.unixtime()`
- To return Unix Time for current time + 60 min, use:
  `$.unixtime(%y-%m-%d %h:%i+60:%s)`
- To return Unix Time for the current date with the set time 02:00:00, use:
  `$.unixtime(%y-%m-%d 02:00:00)`

$.date(%y-%m-%d %h:%i:%s)

The $.date(%y-%m-%d %h:%i:%s) function converts a date in the day and time format. You can use arithmetic arguments. For example:

- To return the current date and time in the YYYY-MM-DD HH:MM:SS format, use:
  `$.date(%y-%m-%d %h:%i:%s)`
- To add +1 day to the returned date in the YYYY-MM-DD HH:MM:SS format, use:
  `$.date(%y-%m-%d+1 %h:%i:%s)`

$.unixtime().tz

With the $.unixtime().tz function, you can get the current time of a specific timezone. For example, to return the current time in Kyiv (Eastern European time zone) in the Unix Time format, use:

$.unixtime().tz('Europe/Kiev')

$.map() and $.filter()

**2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.**

You can use the $.map() and $.filter() Erlang functions in the Set Parameters node in a similar way to how you use the .math() and .unixtime() functions. The use of the $.map() and $.filter() functions aims at making responses from external APIs more lightweight and tasks smaller after passing through the API Call nodes.

The $.map() and $.filter() functions are used in the following constructions:

- $.map(fun(Item) -> end, arr)
- $.filter(fun(Item) -> end, value)
  where:
- fun is the supported function.
- Item is an arbitrary value and the Item value must be capitalized.
- arr is a parameter of an input task, which contains an array.
- value is a single value.

A function body follows the arrow ->, in which you can write allowed expressions using simple operations from ALLOWED_INF_EXPRS (expressions allowed in $.map() and $.filter() functions), and some functions from ALLOWED_EXT_FUNS (allowed in $.map() and $.filter() functions).

Using the the fun() function with the {{...}} construction

You can't use the `{{...}}` **construction in the** `fun()` **function.**

ALLOWED_EXT_FUNS

{proplists,'_','_'},

{base64,'_','_'},

{binary, split, '_'},

{binary, replace, '_'},

{eutils, get_value, 2},

{eutils, from_json, '_'},

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

{eutils, to_json, 1},

{erlang,binary_to_float,1},

{erlang,binary_to_integer,1},

{erlang,integer_to_binary,1},

{erlang,integer_to_binary,2},

{erlang,round,1},

{erlang, is_integer, 1},

{erlang, is_binary, 1},

{erlang, is_list, 1},

{erlang, is_float, 1},

{erlang, is_boolean, 1},

{erlang, is_number, 1},

{erlang, hd, 1},

{erlang, tl, 1}

ALLOWED_EXT_FUNS performance

The use of the functions from the ALLOWED_EXT_FUNS modules list and adhering to the specified syntax enable maximum execution performance. In this case, the performance will be kept at a high level and gradually improve when support for newer Erlang versions in Corezoid is implemented.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

proplists - let's take a closer look at the `get_value` and `delete` functions. For example, we need to get a value from the test and increase it by 1. To do this, we'll use the following code:

```
Test = proplists:get_value(<<"test">>, Item),

[{<<"test">>, Test + 1} | proplists:delete(<<"test">>, Item)]
```

In this example, we've written the "test" value to the Test variable. As the Item is a list, we added a new test variable with the "Test +1" value to the head of the list and removed the old test variable from the Item. Note that we'll have both test variables processed in the code without removing the old variable.

base64 - note the use of `encode` and `decode` functions; they are used the same way as in other languages:

```
Test = proplists:get_value(<<"test">>, Item),

[{<<"base64">>, base64:encode(Test)} | Item]
```

Analogous to the previous example, we've got a value from the test and written it to the Test variable. Then, we updated the base64 value with the new base64 value and added it to the Item.

binary - `split` and `replace` functions are available. They allow splitting a string and replacing it with another string.

eutils - internal Corezoid library. It offers the following functions:

- get_value - similar to proplists and serves the same purpose;
- from_json - converts JSON to list;
- to_json - converts list to JSON.

For example, if your task contains the following data:

```
[{"test":"{\"a\":1}"}, {"test":"{\"a\":2}"}, {"test":"{\"a\":4}"}].
```

you can use the following code:

```
Test0 = eutils:get_value(<<"test">>, Item),

Test1 = eutils:from_json(Test0),
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

[{<<"a">>, eutils:get_value(<<"a">>, Test1)} | Item]

The code takes the test value, converts the string to a list, and assigns the Test1 the following value

[

{<<"a">>, 1}

]

We take the "a" value and remove it from the test, then we add it to the Item.

erlang - only the functions for converting binary to number and vice versa are available.

ALLOWED_INF_EXPRS

'+',

'-',

'*',

'/',

'bnot',

'div',

'rem',

'band',

'bor',

'bxor',

'bsl',

'bsr',

'not',

'and',

'or',

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

'xor',

'andalso',

'orelse',

'==',

'/=',

'=<',

'<',

'>=',

'>',

'=:=',

'=/=',

'++',

'--'

Examples

Multiplying all array elements by a specified value ($.map() function)

1. In the Set Parameters node input, take a task containing the a array:
   `"a": [1,2,3]`
2. In the Set Parameters node, enter the $.map() function :
   `"b": "$.map(fun(Item) -> Item*2 end, {{a}})"`

Each a array element is multiplied by 2, and the results are written to the b array:

"b": "[2,4,6]"

Filtering out uneven numbers and writing even numbers ($.filter() function)

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

1. Set the b task:
   `[1,2,3,4,5,6,7,8,9,10,11,12,13,14]`
2. In the Set Parameters node, write the $.filter() function:
   `"b1": "$.filter(fun(Item) when Item rem 2 < 1 -> true; (_) -> false end, {{b}})"`

The response is:

"b1":[2,4,6,8,10,12,14]

Conversion of an array with objects containing the "fieldName" key into an array that includes only values of this key:

1. Generic method:
   `$.map(fun(E) -> FieldName = eutils:get_value(<<"fieldName">>, E), FieldName end, {{arr}})`
2. Simplified method:
   `$.map(fun(E) -> eutils:get_value(<<"fieldName">>, E) end, {{arr}})`

Conversion reverse to the one in the previous example - an array of simple primitives is conversed to an object array:

1. By declaring a key-value pairs collection:
   `$.map(fun(Value) -> #{<<"fieldName1">> => Value, <<"fieldName2">> => Value} end, {{arr}})`
2. By declaring a tuples collection:
   `$.map(fun(Value) -> [{<<"fieldName1">>, Value}, {<<"fieldName2">>, Value}] end, {{arr}})`

2.3.7 Dashboards

Overview

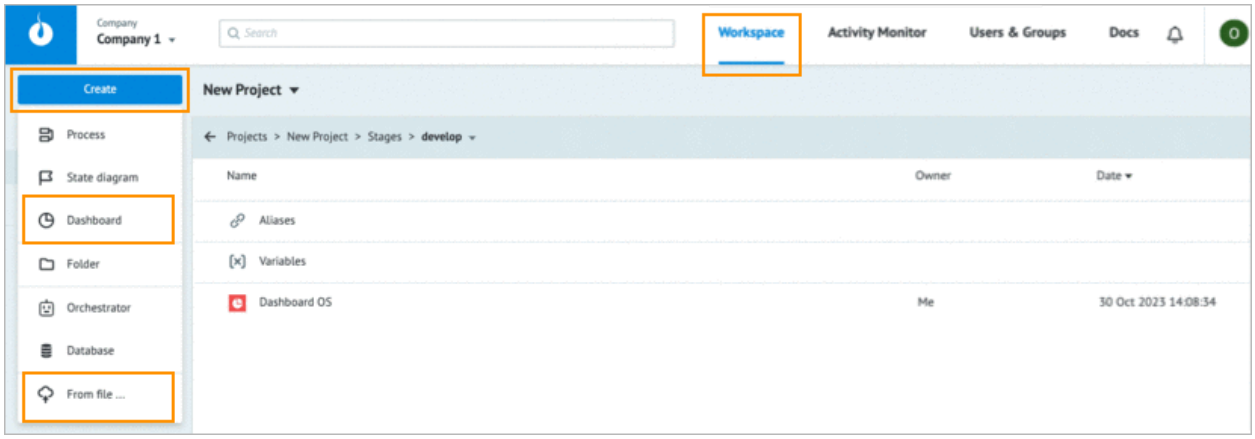
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

A dashboard is a tool in Corezoid designed for collecting and visualizing statistics and reports related to Processes and State Diagrams. Dashboards present data in the form of charts, allowing users to gain insights from the data visually. The charts, in turn, use data from nodes of built Processes and State Diagrams. The tasks in these nodes (counters) serve as the source of data (metrics) of the charts.

Create Dashboard

To create a Dashboard:

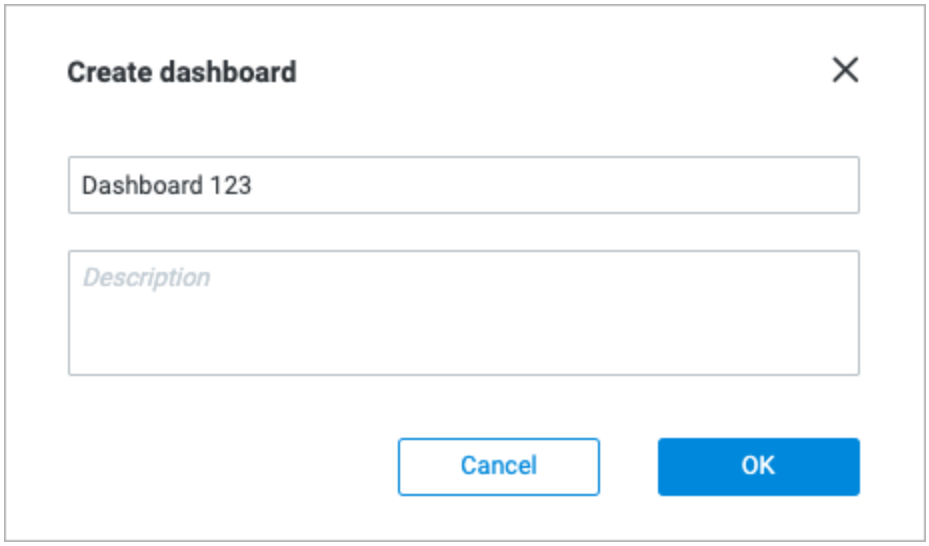
1. On the **Workspace** tab, click **Create**, and then in the dropdown list, select **Dashboard**. **Note:** You can import a dashboard from a JSON file that contains a folder with a dashboard and related Processes and charts.To do so, in the dropdown list, select **From file** and then choose the file from your device.



Screenshot 72: **Dashboard creation on the Workspace tab.**

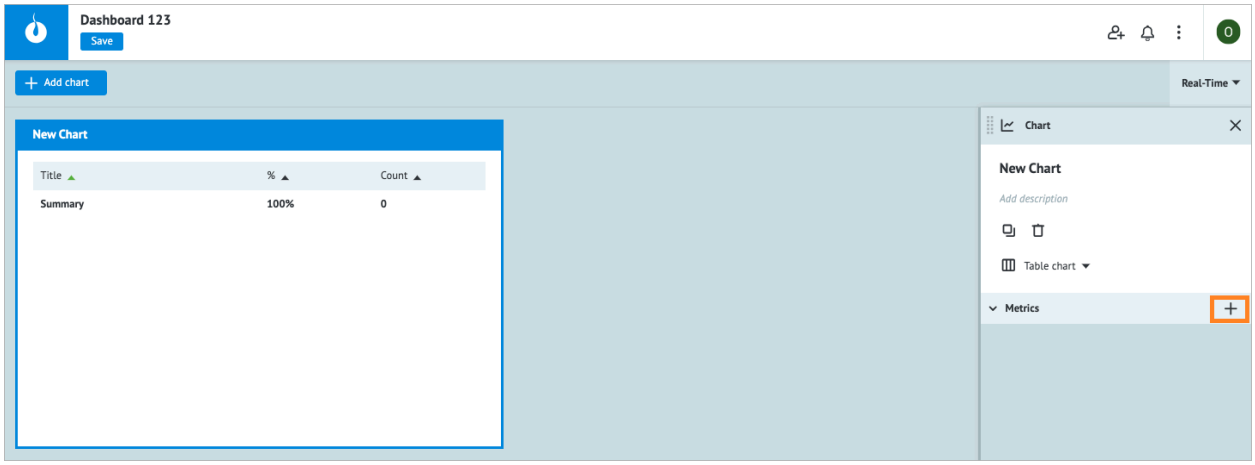
2. In the **Create dashboard** dialog that opens, enter a dashboard name and description (optional), and then click **Ok**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 73: **Create Dashboard window.**

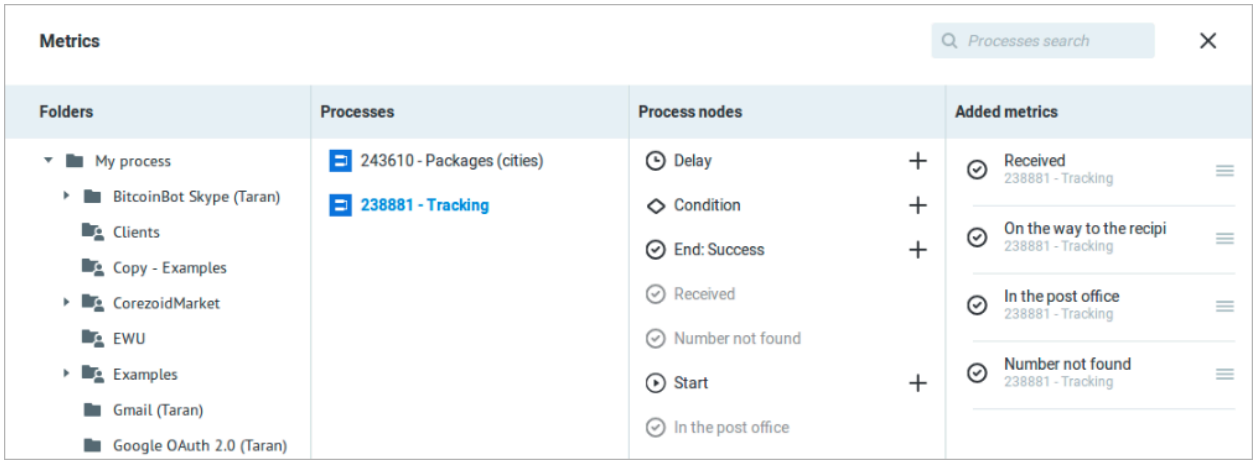
3. On the **Chart** details panel of the created dashboard page, click the plus icon **+** next to **Metrics**.



Screenshot 74: **Adding metrics to a Dashboard.**

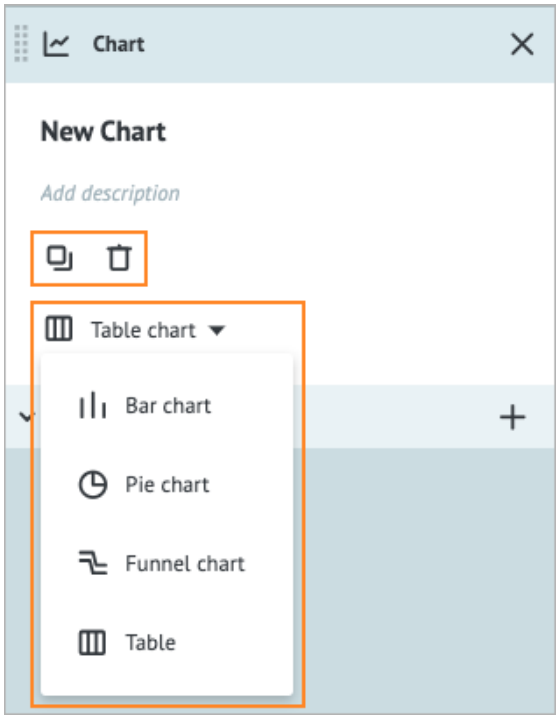
4. In the **Metrics** table, select the needed data source for the chart:
   - **Folders**: A list of folders with Processes
   - **Processes**: Processes located in the selected folder
   - **Process nodes**: Nodes of the selected Process.
     Note: Click the plus icon **+** to add a node to the chart.
   - **Added metrics**: Lists the nodes added to the dashboard
5. **Note:** After you have selected the needed data, in the upper-right corner of the table, click the close icon ✕.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 75: **Metrics menu.**

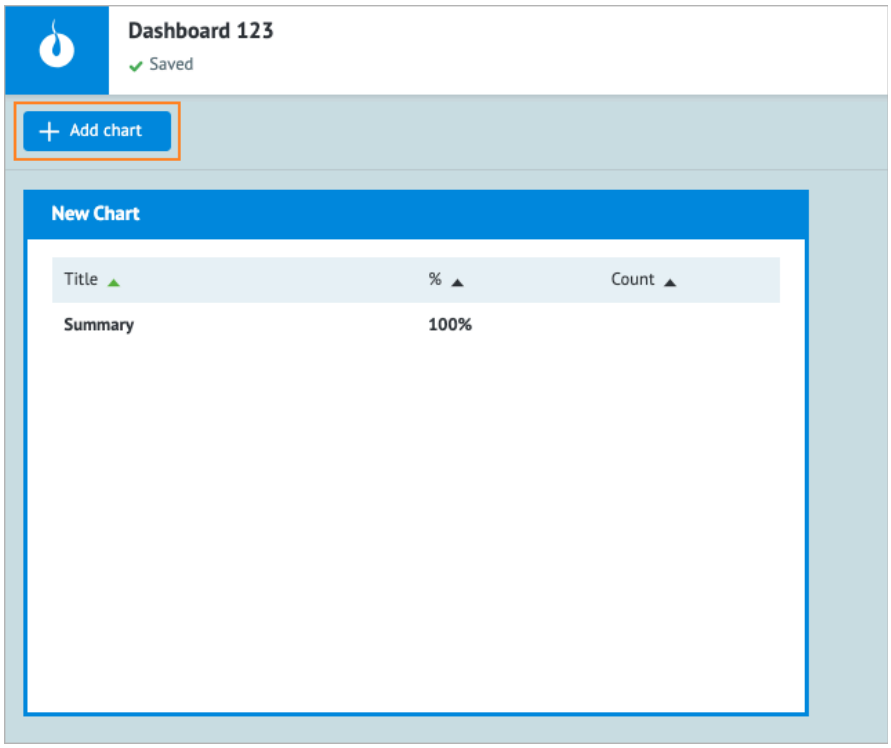
6. (Optional) On the **Chart** details panel:
   - Click **Table chart**, and then select the needed chart type (**Bar chart**, **Pie chart**, **Funnel chart**).
   - Click the copy icon to make a copy of the chart.
   - Click the delete icon to delete the chart.



Screenshot 76: **Chart menu.**

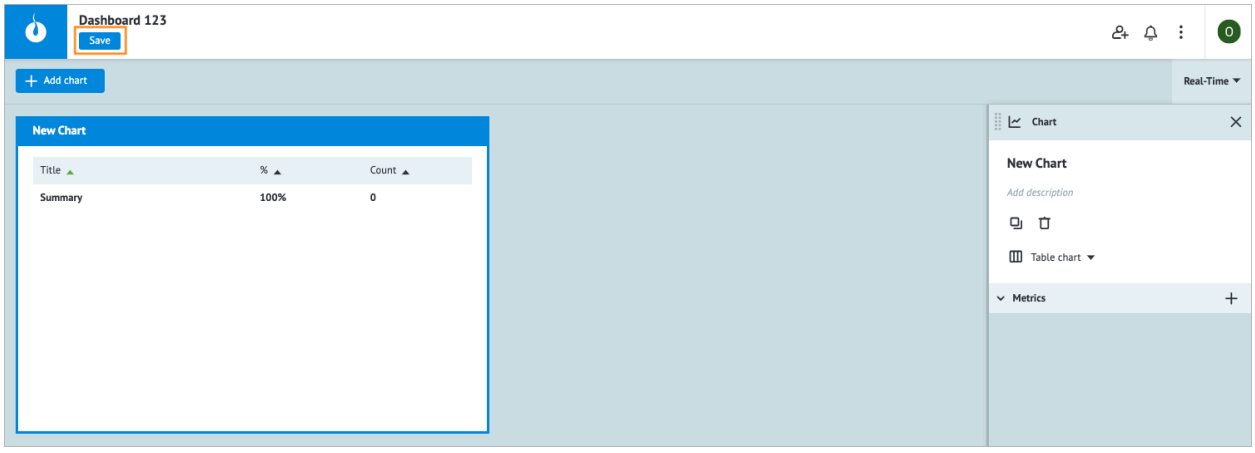
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

7. (Optional) In the upper-left corner of the created dashboard page, click **+ Add chart** and customize a new chart by following steps **3-6**.
Note: You can add multiple charts to your dashboard.



Screenshot 77: **Adding a chart to a Dashboard.**

8. In the upper-left corner, click **Save**.



Screenshot 78: **Saving a Dashboard.**

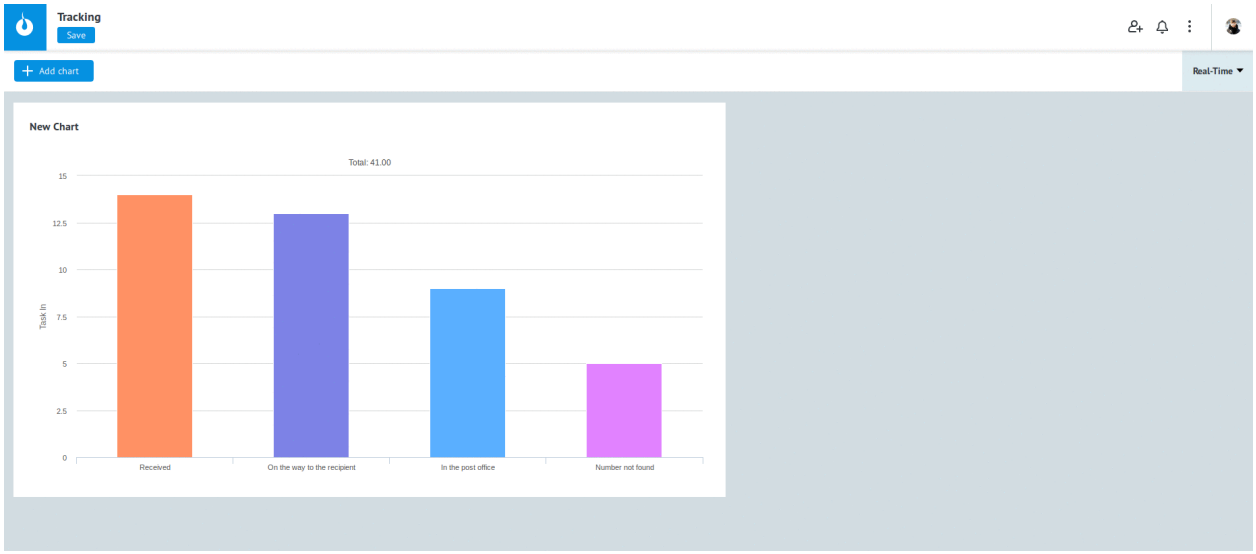
You've created the dashboard, and now can easily customize it.

Additional actions

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Select time range

You can select the needed time range for your dashboard by clicking **Real-Time** in the upper-right corner of the page.



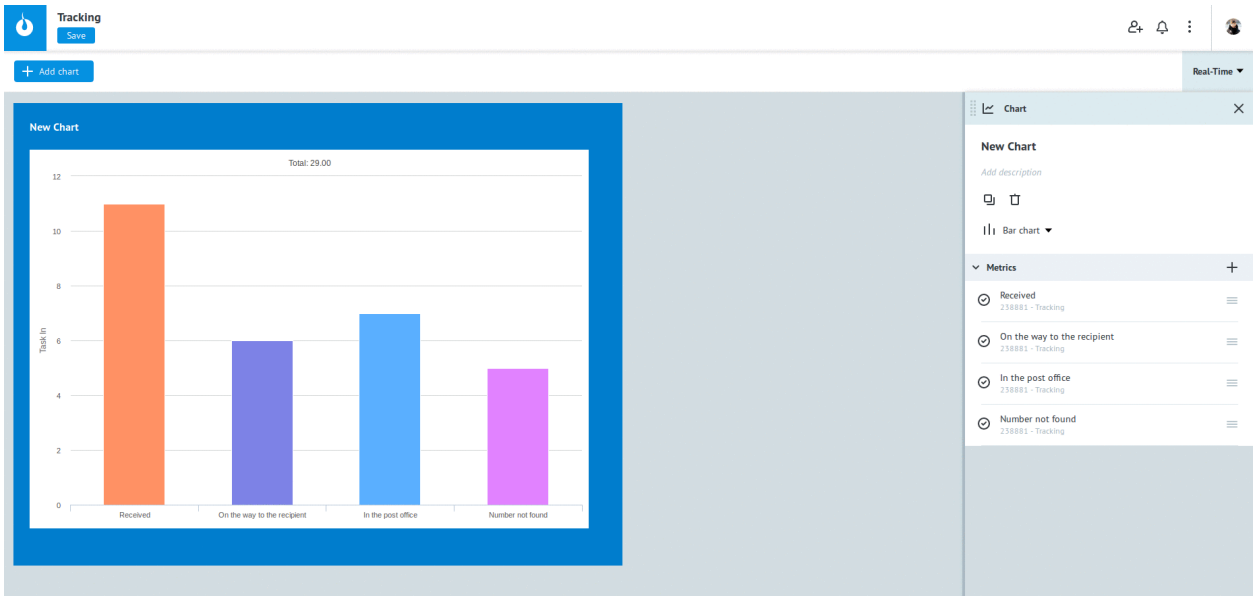
Screenshot 79: **Viewing Dashboard real-time metrics.**

The **Real-Time** mode works only for the **End** node where the tasks are accumulated and for the **Waiting for Callback**, **Delay** and **Set State** nodes (in a State Diagram) where the tasks are delayed for the time specified in the Process. Nodes of other types do not accumulate tasks, resulting in empty chart data when the **Real-Time** mode is selected.

Resize chart

Adjust the size of a chart by moving the cursor to its left or right side and setting the desired size.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

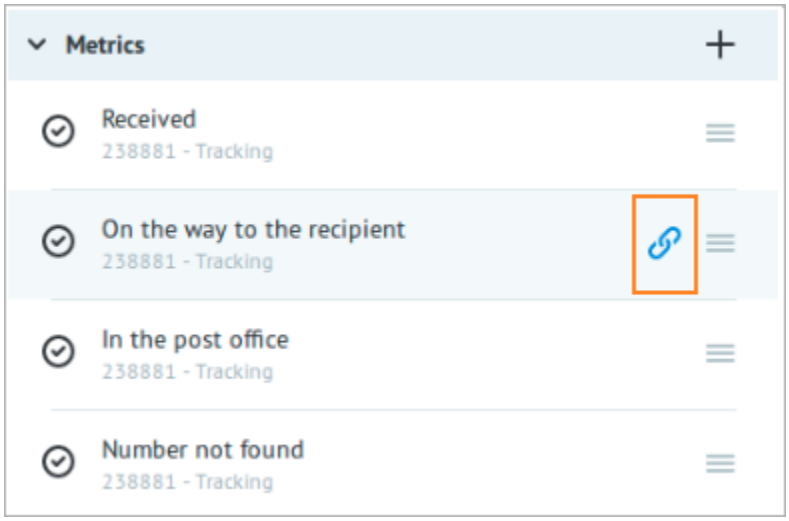


Screenshot 80: **Viewing Dashboard**

Add link to chart

If your data has a hierarchical structure with nested elements, you can display detailed information by linking the chart to another dashboard that provides in-depth metric details. To add a link to a chart:

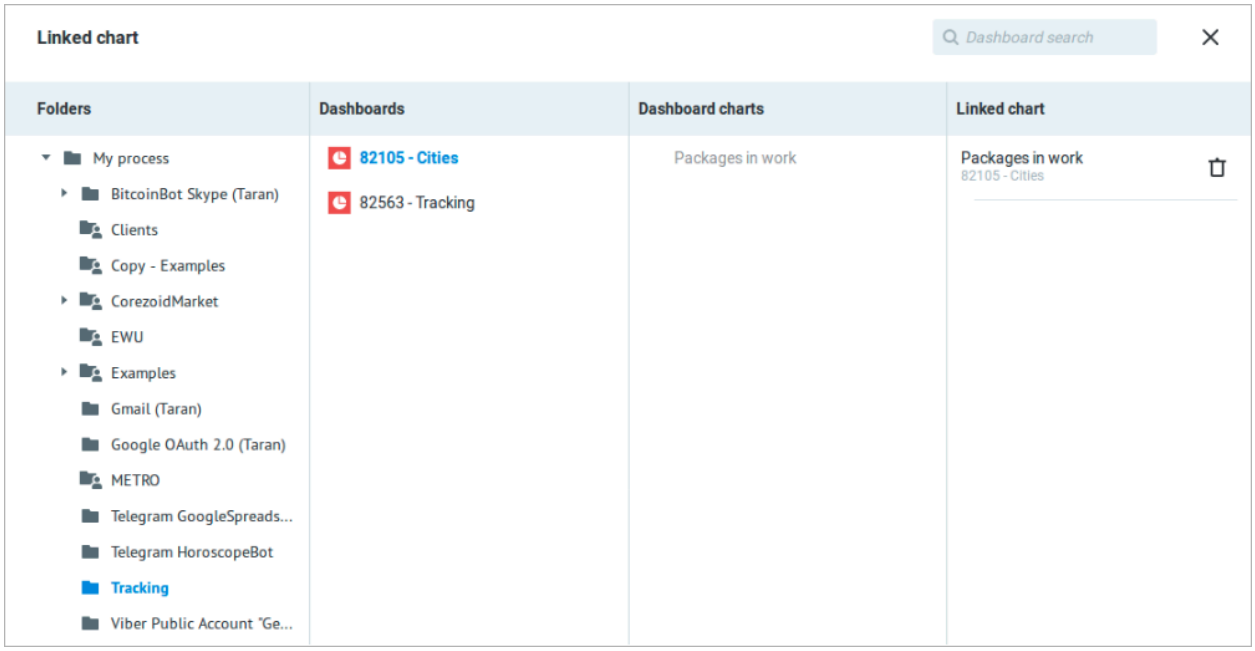
1. In **Metrics** of the **Chart** details panel, click the link icon .



Screenshot 81: **Adding a link to a chart.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

2. In the **Linked chart** table, select the needed dashboard and a chart in it.



Screenshot 82: **Linked chart table.**

After you've added the link, you can open it by clicking the corresponding metric link icon.

2.3.8 Projects

Overview

A project is a workspace where you can store and manage Processes, State Diagrams, Dashboards, API keys, aliases and folders.

Core Components

Stages

- Development environments for Processes
- Default: Production and Develop

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- Group Processes by environment type

Aliases

- Replace Process IDs with readable names
- Enable flexible Process interaction

Variables

- Stage-level settings
- Allow Process reuse with different values
- Environment-specific configuration

Versions

- Stage snapshots
- Track and restore previous states
- Support rollback capabilities

Best Practices

Development

- Use distinct stages for development phases
- Implement continuous integration flow
- Maintain organized Process groups

Testing

- Create dedicated testing environments
- Validate Process interactions
- Verify variable configurations

Deployment

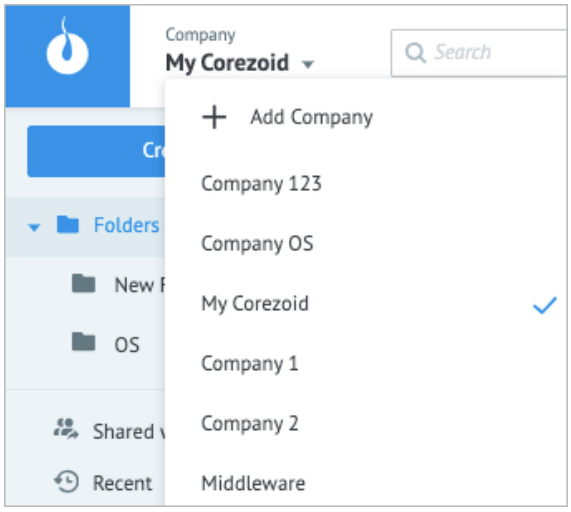
- Use merge functionality for updates
- Follow stage-to-stage promotion
- Maintain version control

Create project

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

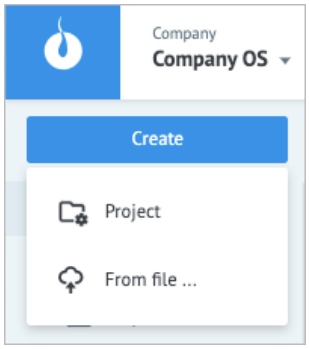
To create a project:

1. In the upper-left corner of the **Workspace** tab, click the expand icon, and in the dropdown list, select the needed company.
   Note: Don't select **My Corezoid** as selecting it means no user company is selected.



Screenshot 83: **Selecting a workspace.**

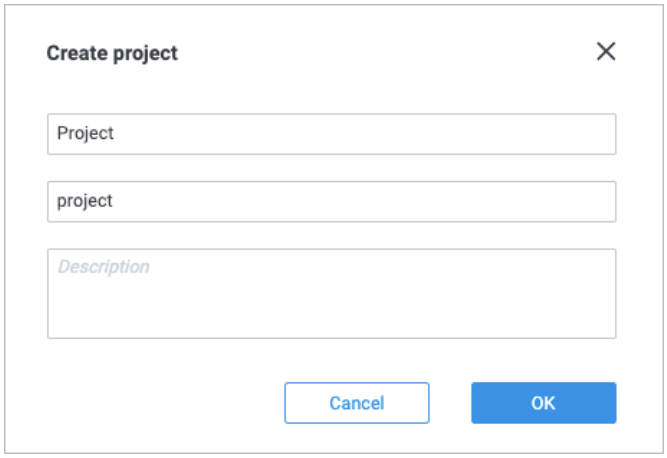
2. In the upper-left corner, click **Create**, and in the dropdown list, select **Project**.
   Note: To import a project file from your device, select **From file**.



Screenshot 84: **Create-Project.**

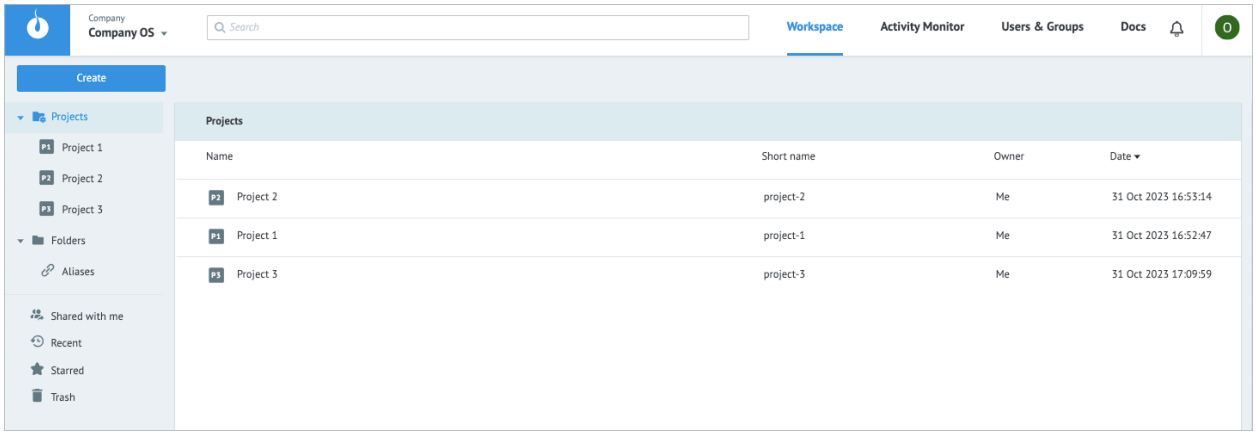
3. In the **Create project** dialog, enter your project name and description (optional), and then click **Ok**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 85: **Create project window.**

The new project is created and automatically added to the **Projects** folder.

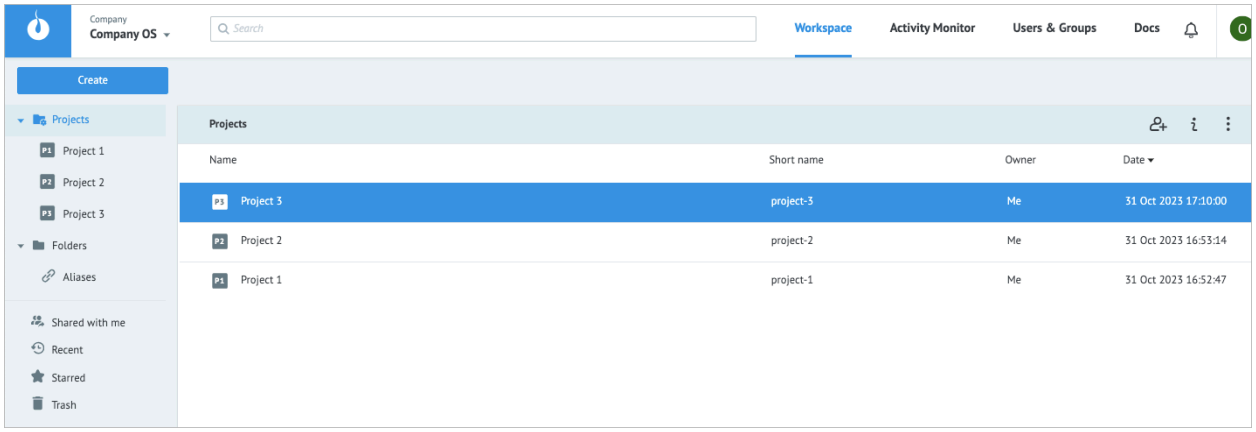


Screenshot 86: **New project in the list.**

Manage project

To manage a project, click the desired project in the **Projects** folder.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

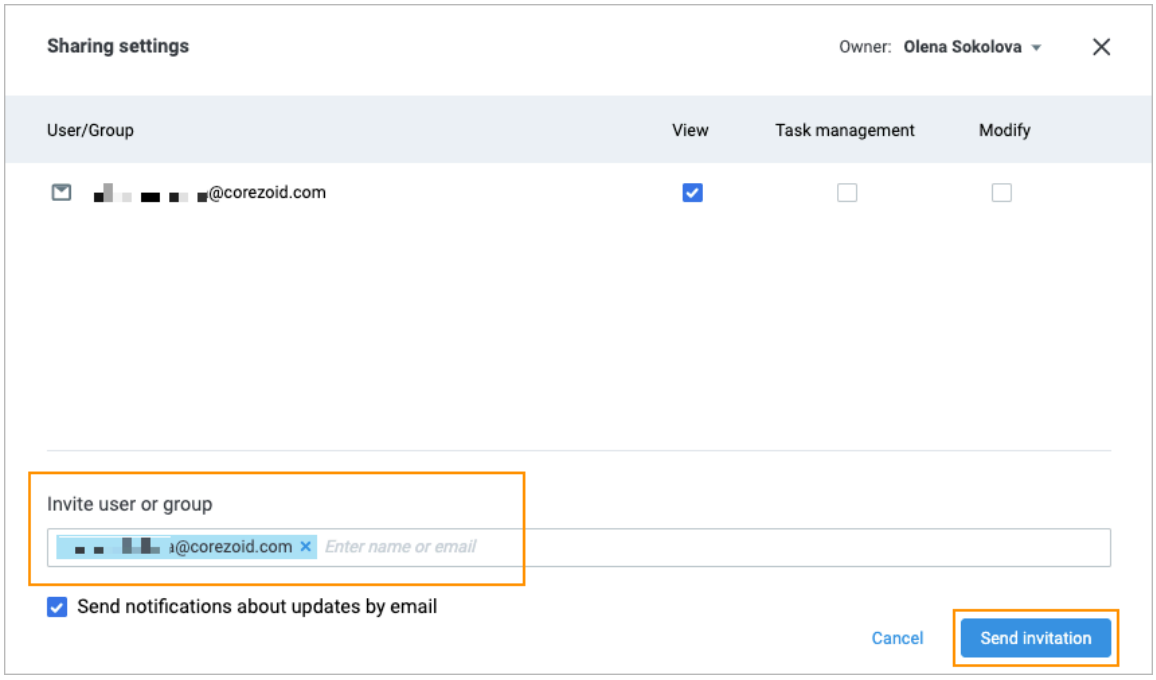


Screenshot 87: **Select a project in the list to manage.**

In the Projects, you can:

- Share the project with other users and groups:

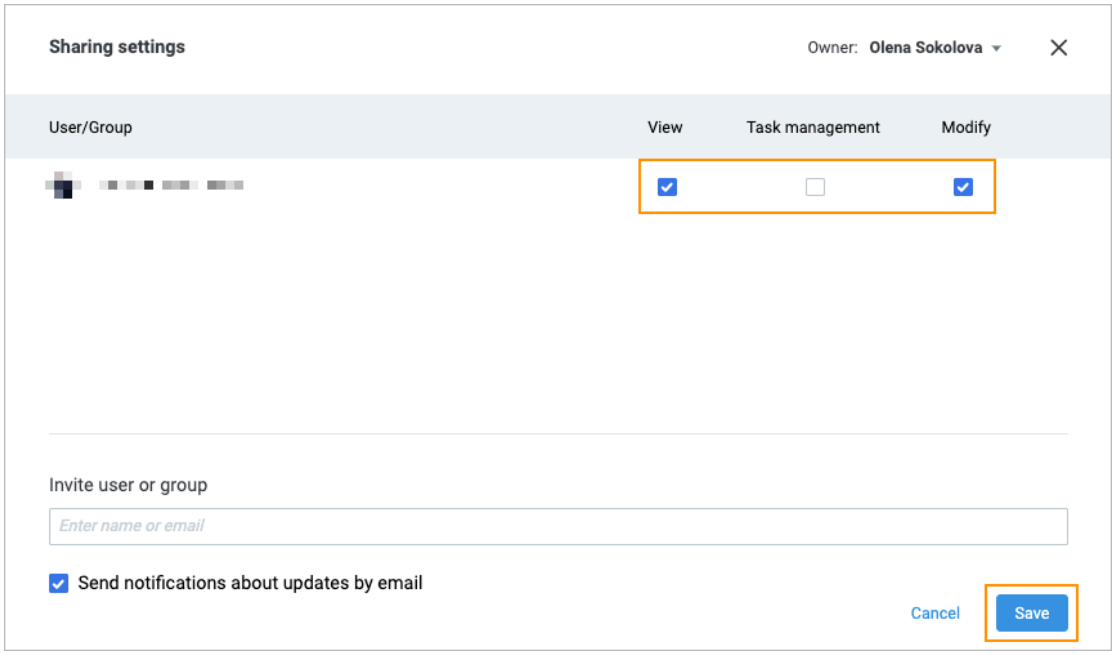
  a. In the upper-right corner, click the share icon .
  b. In the **Invite user or group** field of the **Sharing settings** dialog that opens, enter the name or email of the needed user or group, press **Enter** on your keyboard, and then click **Send invitation**.
  Note: Users and user groups that you want to provide access to the project, must be registered in your company.



Screenshot 88: **Project sharing-invite users.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

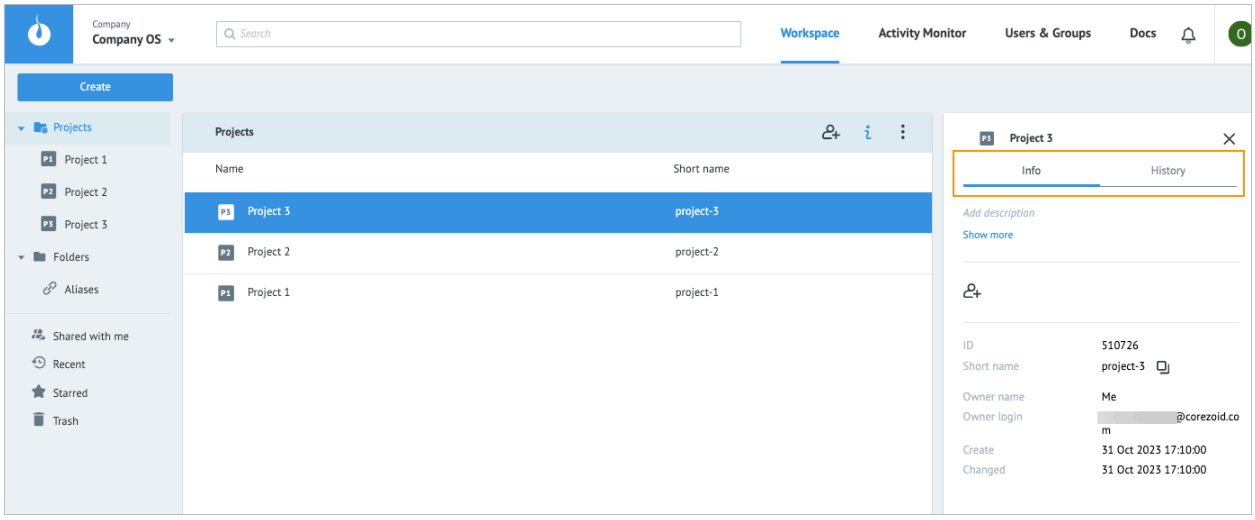
c. After the selected user has confirmed the invitation by email, set the needed sharing permissions checkboxes (**View**, **Task management**, and **Modify**), and then click **Save**. **Note:** You can select the permissions for the user you want to share with before clicking **Send invitation**. Setting the permissions for a user after the user has clicked the link in the email sets the new permissions. Every time permissions change, the user receives the corresponding notification by email.



Screenshot 89: **Project sharing-setting permissions.**

- Check the project information:

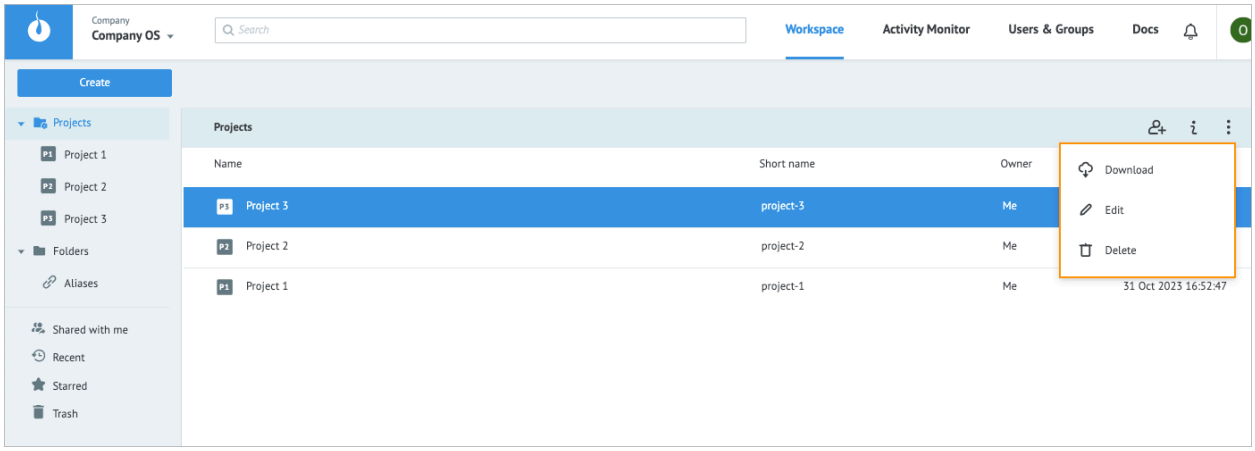
  a. In the upper-right corner, click the info icon .
  b. View project details by clicking the **Info** and **History** tabs.



Screenshot 90: **Project sharing-project info check.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- Perform more actions by using the options icon ⋮ and clicking:
  - **Download** to download the project
  - **Edit** to change the project name
  - **Delete** to delete the project
- **Note:** To perform an action on multiple objects, press **Shift** or **Command** on your keyboard, click the needed objects, and then click the options icon ⋮ .



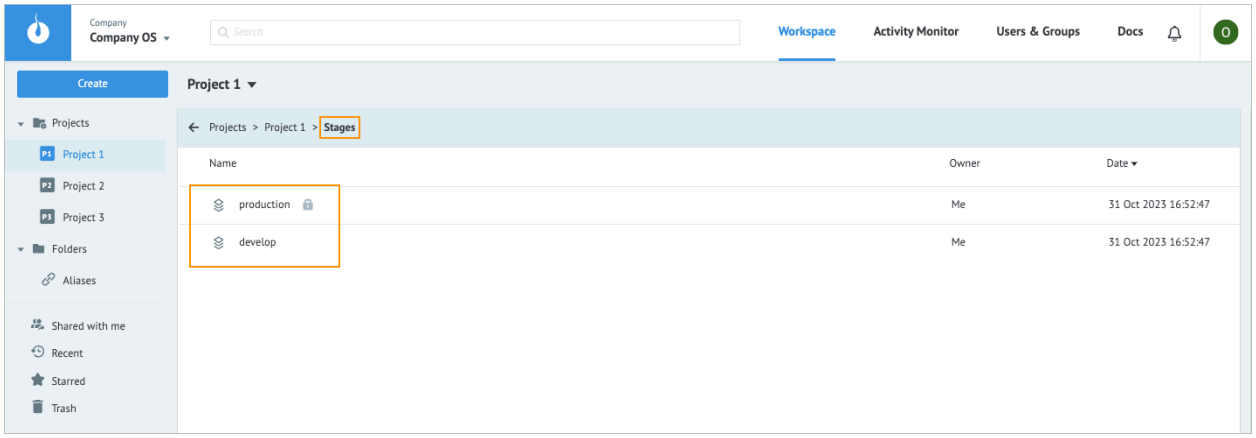
Screenshot 91: **Project sharing-project info check.**

2.3.8.1 Stages

Overview

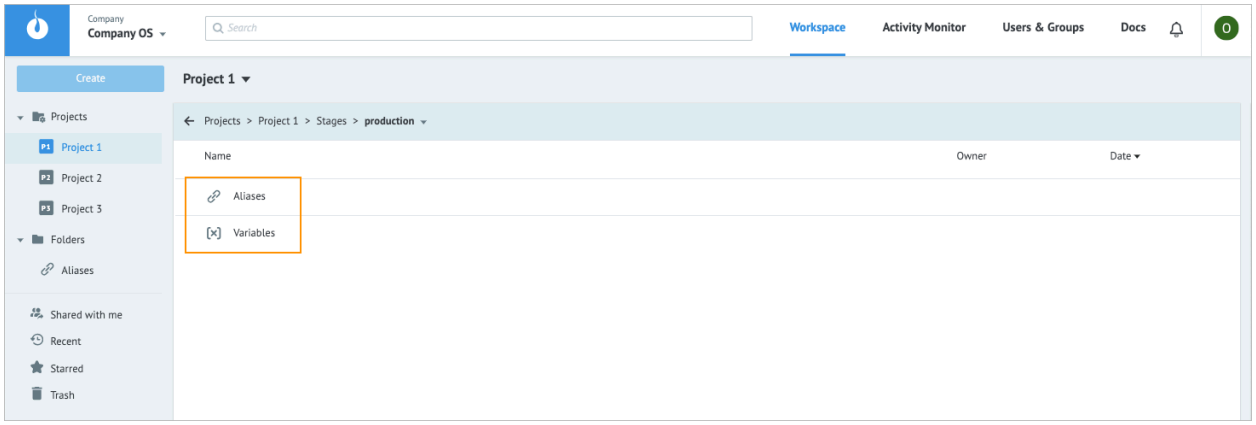
Stage is an environment where you can store your Processes, State Diagrams, and so on to test and work on. After you've created a project, it has two stages by default: **Production** and **Develop**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 92: **Stages subsection.**

By default, each stage includes the **Aliases** and **Variables** sections where you create and manage these objects. For more information, go to Alias and Variable.



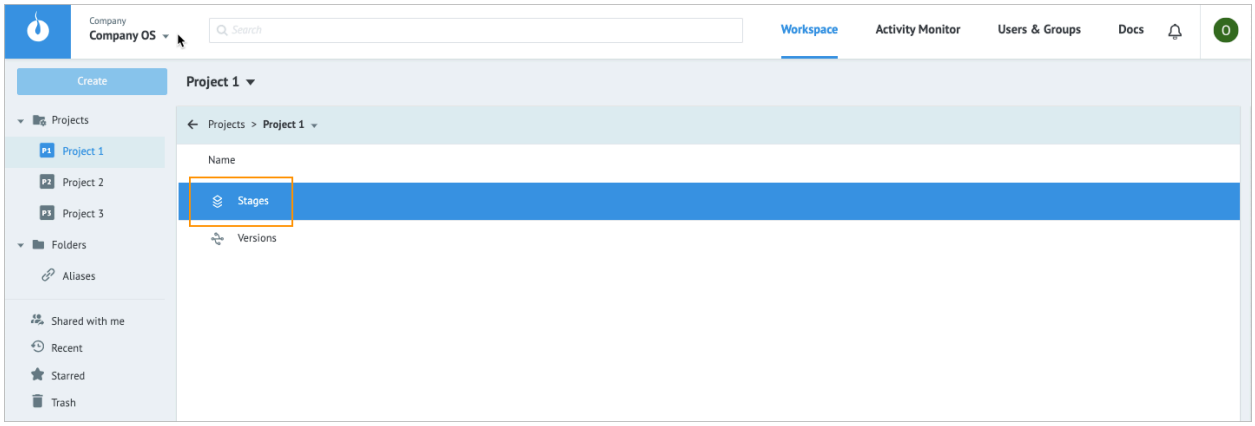
Screenshot 93: **Aliases and variables.**

Create stage

To create a stage:

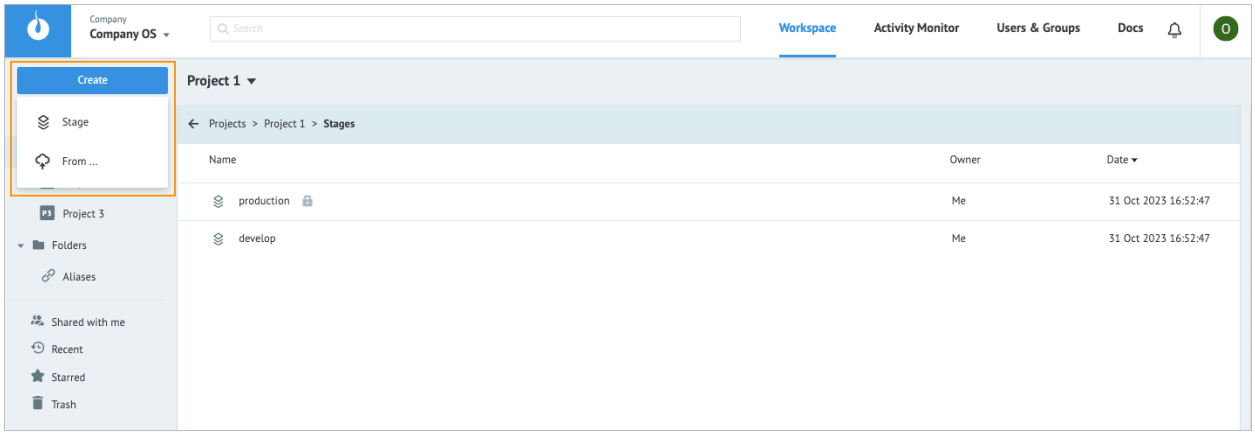
1. On the **Workspace** tab of your company, select the needed project, and then double-click **Stages**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 94: **Stages folder.**

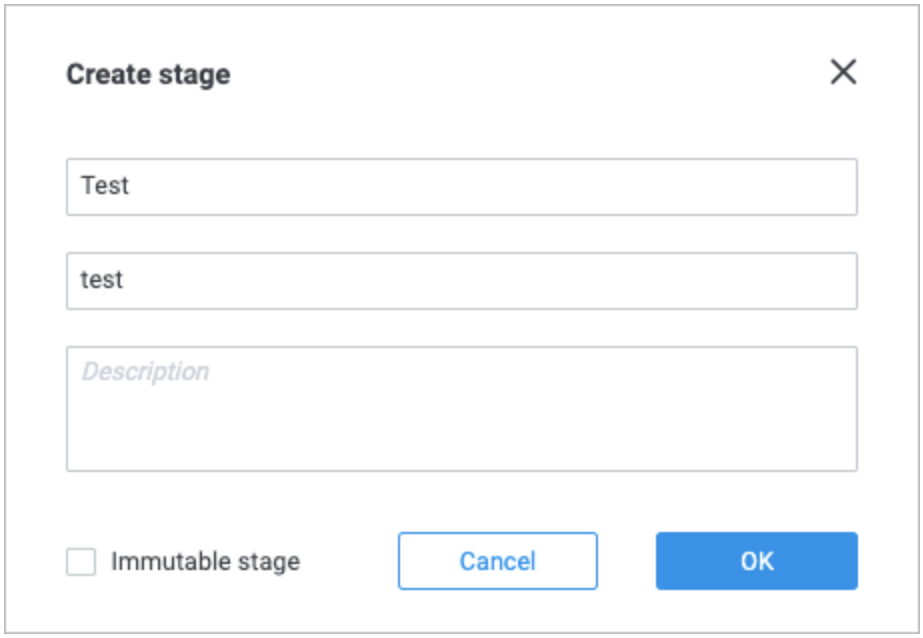
2. In the upper-left corner, click **Create**, and in the dropdown list, select **Stage**.
   Note: You can import a stage file from your device by selecting **From ...**.



Screenshot 95: **Create menu for stages folder.**

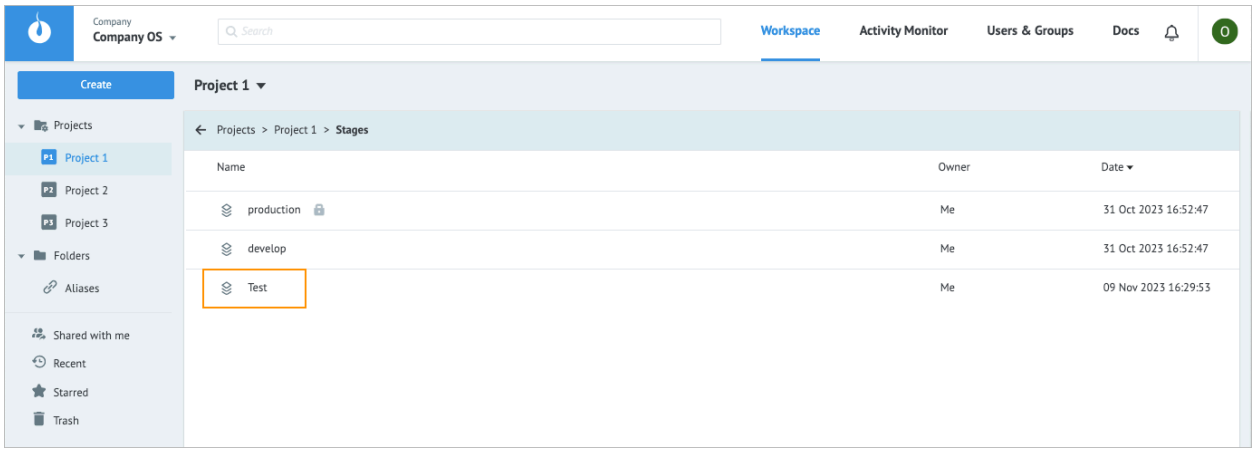
3. In the **Create stage** dialog, enter your stage name and description (optional), and then click **Ok**.
   Note: To make the stage protected of any changes, select the **Immutable stage** checkbox. For more information on immutable stages, go to Immutable stage.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 96: **Create stage window.**

The new stage has been created and automatically added to **Stages**.

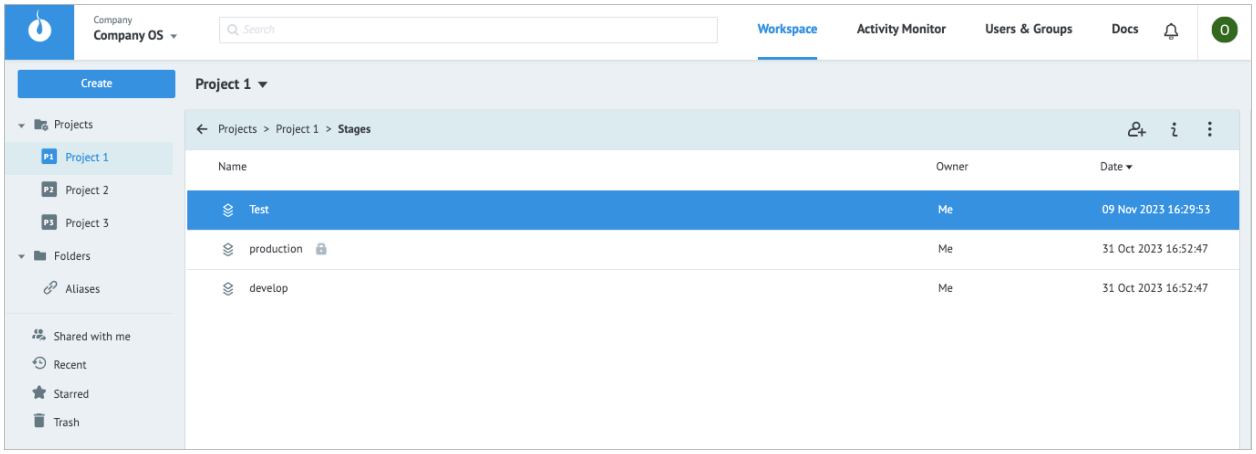


Screenshot 97: **Created stage in the list.**

Manage stage

To see what additional actions you can perform on your stage, click the needed stage in the selected project.

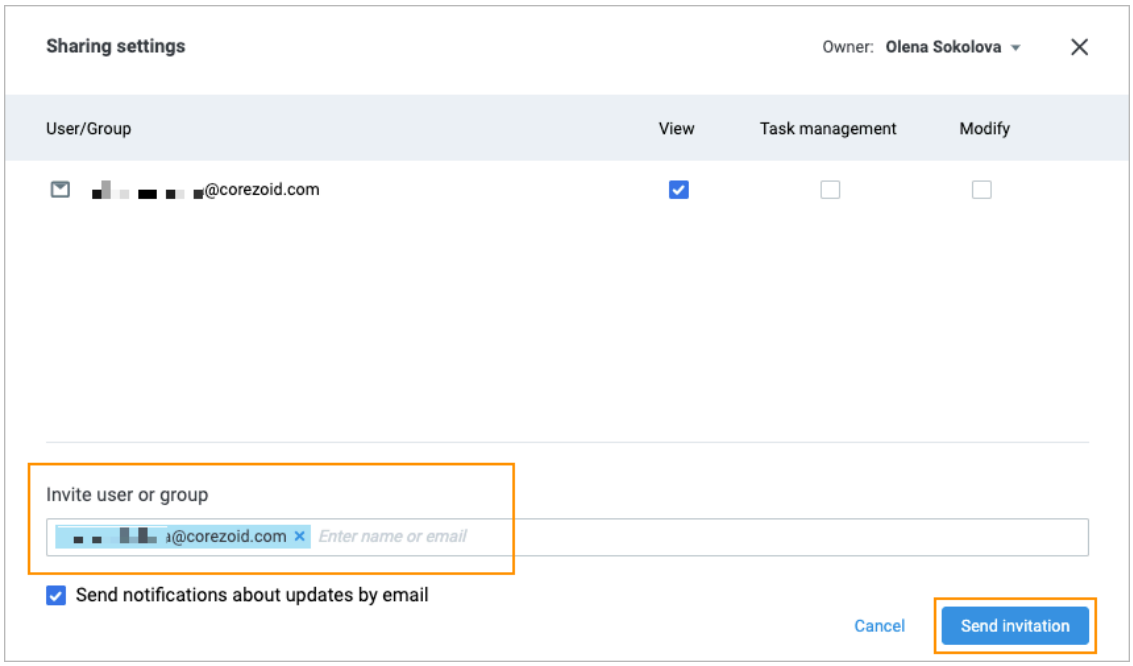
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 98: **Stage selected in the list.**

Now, you can:

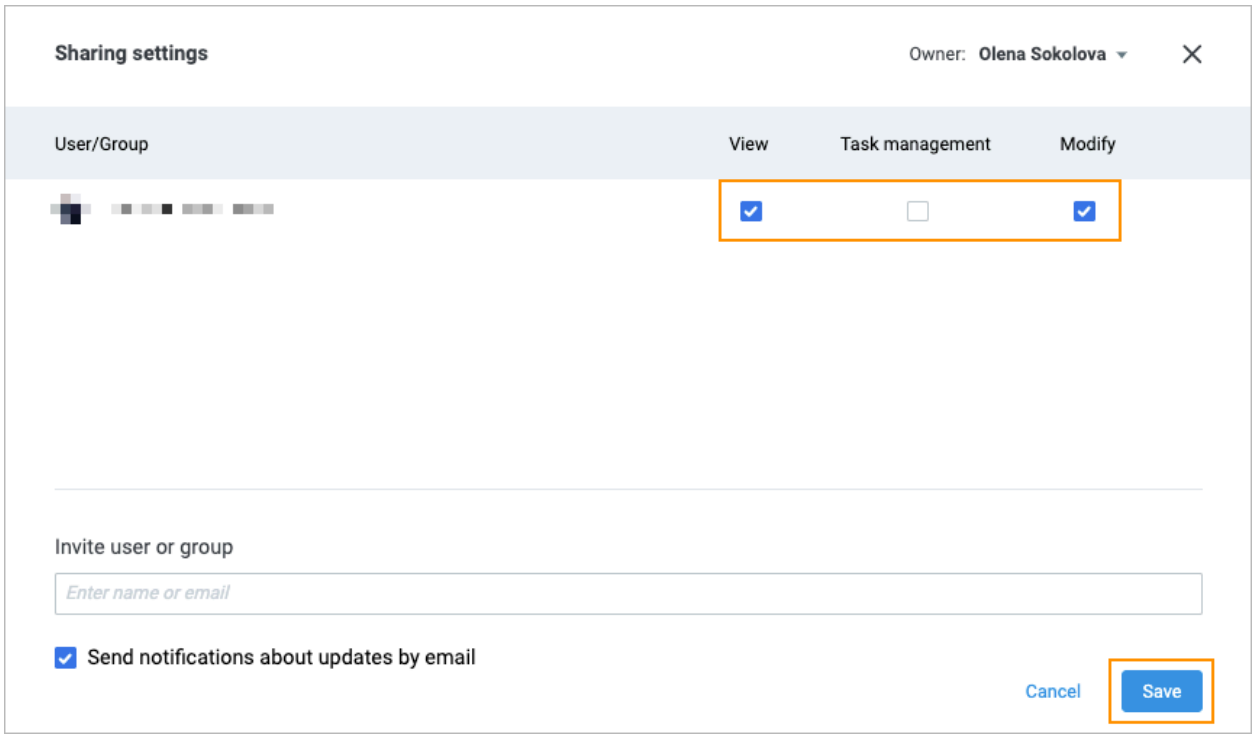
- Share the stage with other users and groups:
  - In the upper-right corner, click the share icon .
  - In the **Invite user or group** field of the **Sharing settings** dialog that opens, enter the name or email of the needed user or group, press **Enter** on your keyboard, and then click **Send invitation**.
    Note: Users and user groups that you want to provide access to the stage, must be registered in your company.



Screenshot 99: **Invitation sending.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

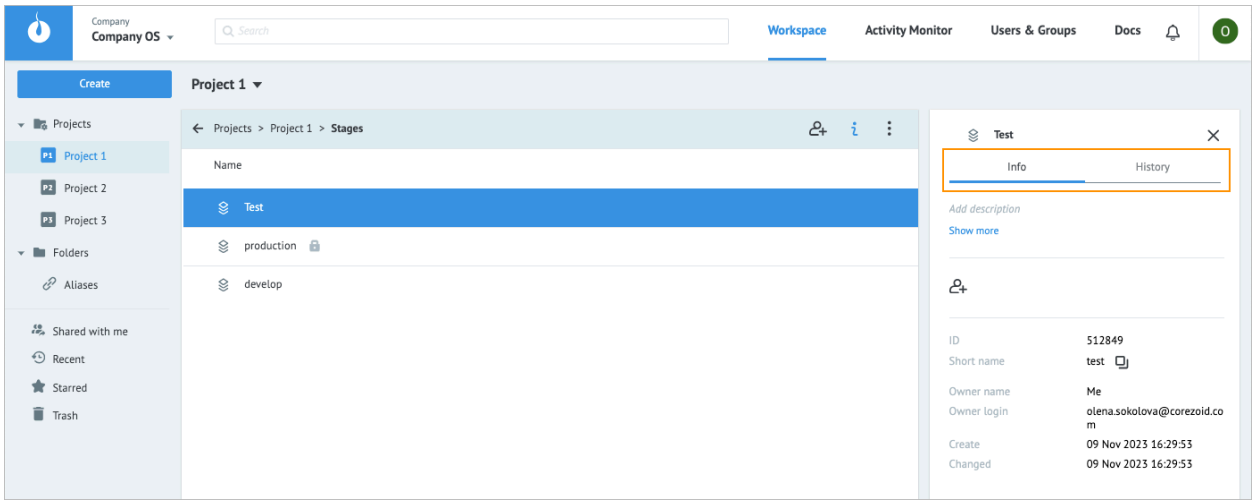
- After the selected user has confirmed the invitation by email, set the needed sharing permissions checkboxes (**View**, **Task managemen**t, and **Modify**), and then click **Save**. **Note:** You can select the permissions for the user you want to share the stage with before clicking **Send invitation**. Setting the permissions for a user after the user has clicked the link in the email sets the new permissions. Every time, the user receives the corresponding notification by email.



Screenshot 100: **Permissions configuration in sharing settings.**

- Check the stage information:
  - In the upper-right corner, click the info icon .
  - View alias details by clicking the **Info** and **History** tabs.

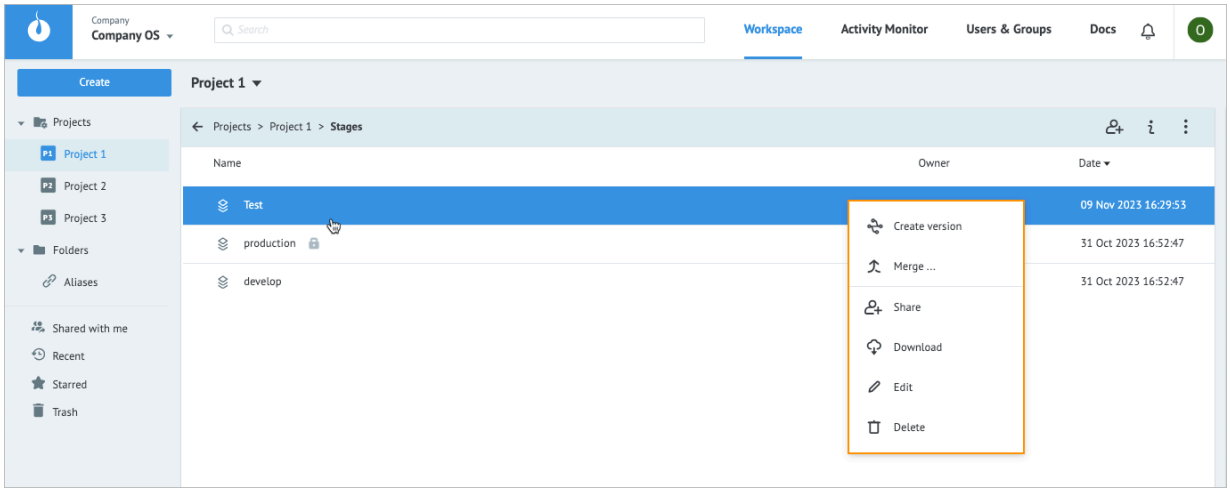
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 101: **Info and History tabs.**

- Perform more actions by using the options icon ⋮ and clicking:
  - **Create version** to create a new version of a stage
    Note: You can create versions only from the stages that contain at least one object (Process, State Diagram, and so on). If the selected stage does not have any objects, then the **Create version** option will be unavailable for selection and will be grayed out. For more information, go to Create version.
  - **Merge** to merge a stage or a version with another stage
    Note: For more information, go to Merge.
  - **Download** to download the stage
  - **Edit** to change the stage name
  - **Delete** to delete the stage
- **Note:** To perform an action on multiple stages, press Shift or Command on your keyboard, click the needed stages, and then click the options icon ⋮.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 102: **Version three-dot menu options.**

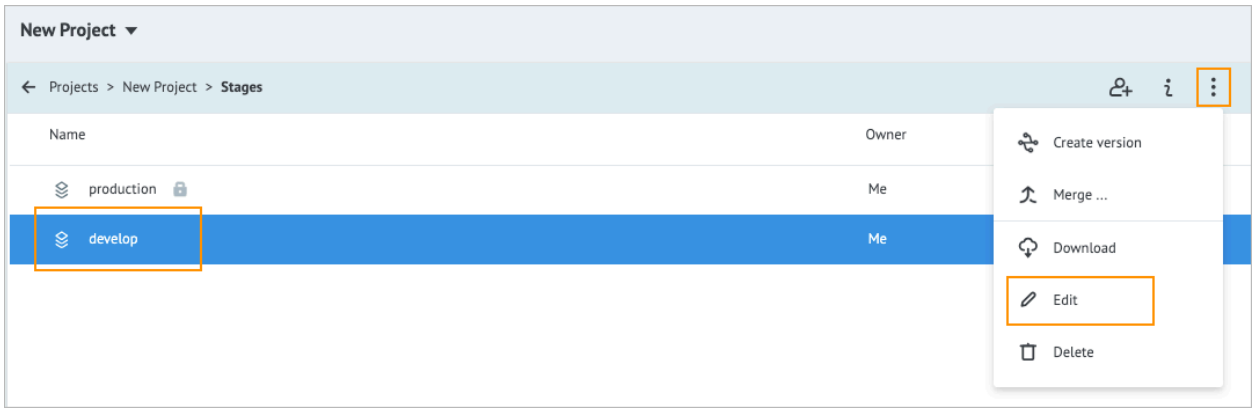
Immutable stage

When you create a project, it has two stages by default: **Production** and **Develop**. By default, the **Production** stage is immutable, that is protected from any changes so that no user can add any object to the immutable stage.

Note: The Immutable parameter for Stages can be toggled by a Project owner, Stage owner, or Superadmin user.

To make a stage immutable:

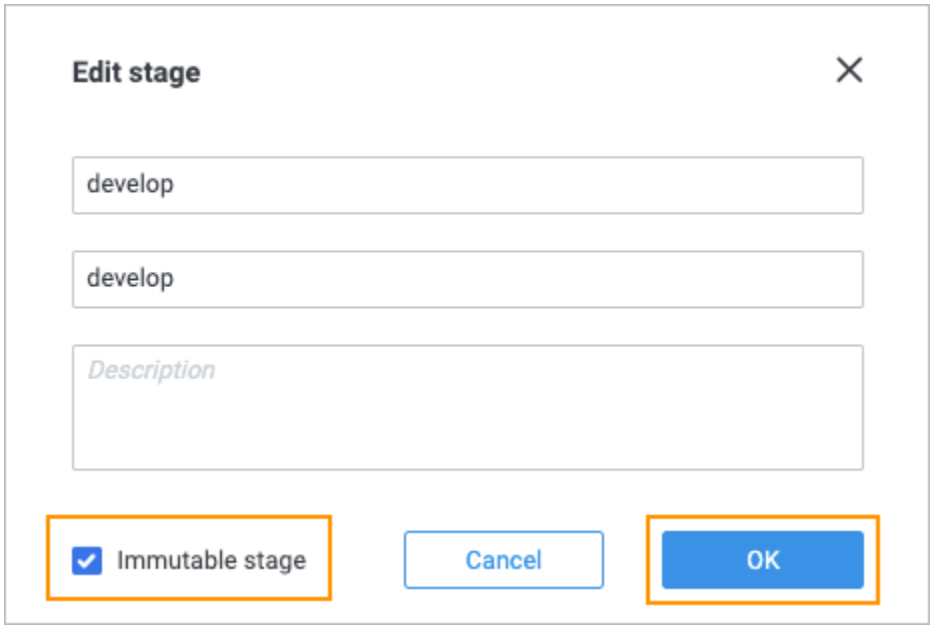
1. In the **Stages** section, click the needed stage, click the options icon, and in the dropdown list, select **Edit**.



Screenshot 103: **Stage edit item.**

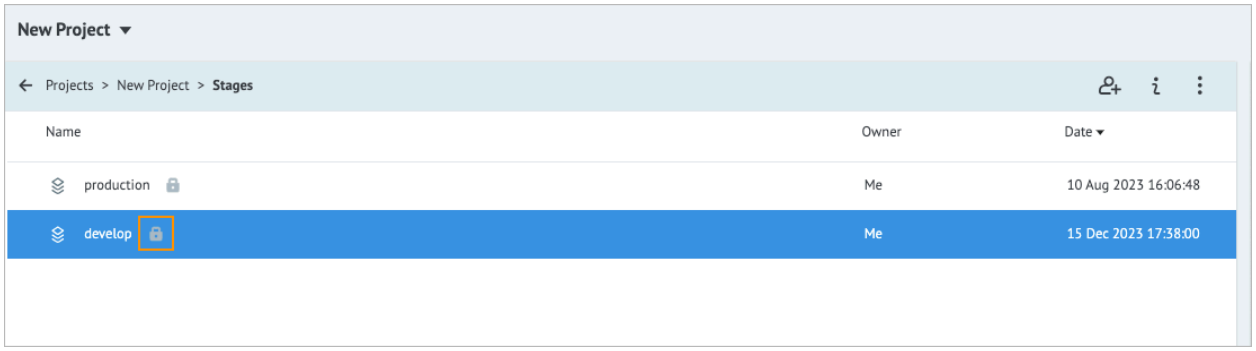
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

2. In the **Edit stage** dialog, select the **Immutable stage** checkbox, and then click **Ok**.



Screenshot 104: **Edit stage window.**

The stage becomes immutable and gets the lock icon next to its name.



Screenshot 105: **Immutable stage icon.**

Note: You can make your stage accessible for other users by deselecting the **Immutable stage** checkbox when editing it. So, the stage will be editable, and you can add needed objects to it.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

2.3.8.2 Aliases

Overview

Alias is a type of Corezoid object that is used to replace numeric object identifiers with symbolic ones specified by a user. Aliases can be used to interact with Processes and State Diagrams both in the nodes and via the Corezoid API.

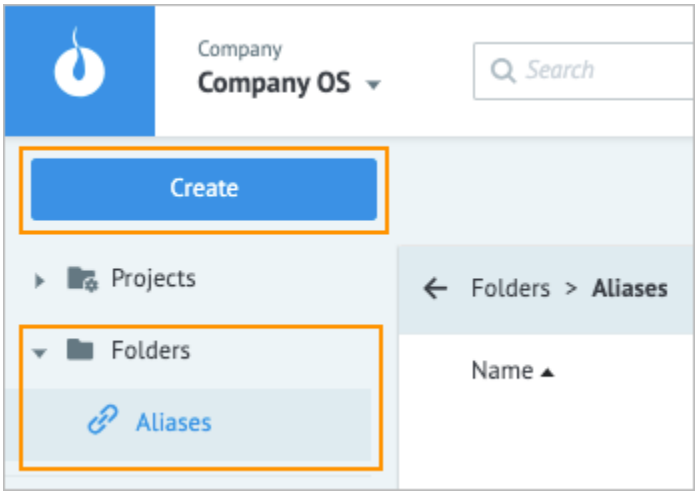
Create alias

You can create an alias:

- At the company level
- At the stage level

To create an alias:

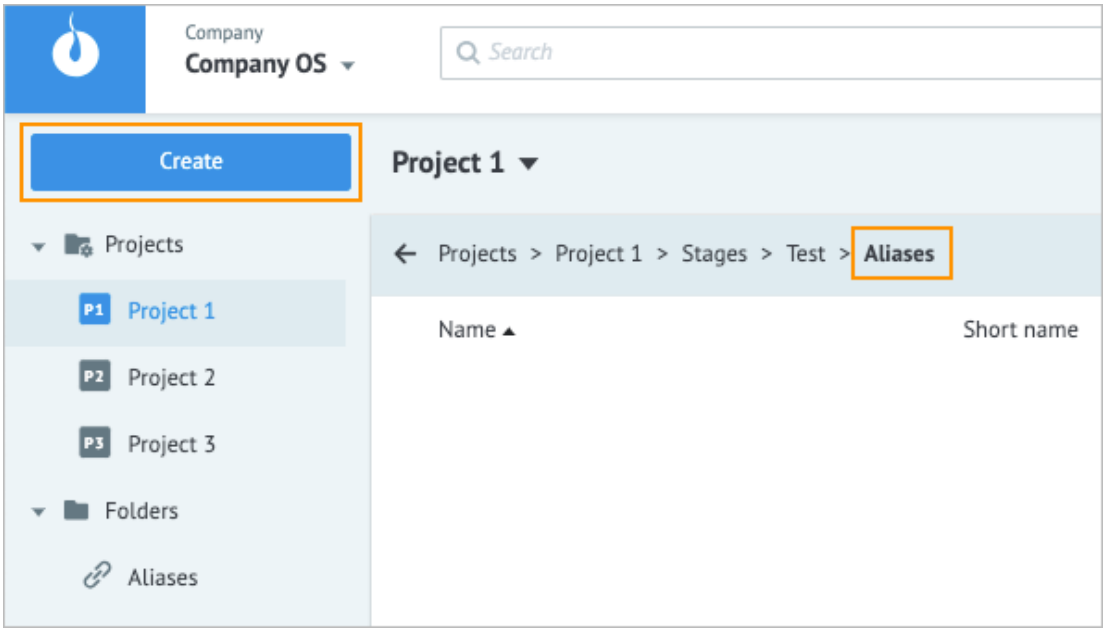
1. On the **Workspace** tab of your company:
    a. (At the company level) Click the **Folders** folder, click **Aliases**, and then click **Create**.



Screenshot 106: **Aliases in folder.**

    b. (At the stage level) In the **Projects** folder, select the needed project and stage, double-click **Aliases**, and then click **Create**.

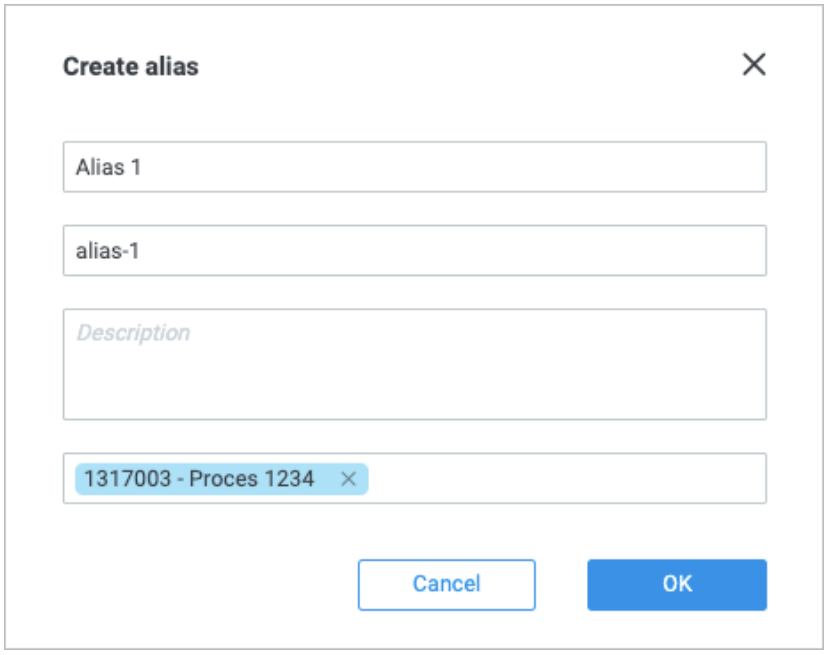
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 107: **Create alias.**

2. In the **Create alias** dialog:
    a. In the **Name** and **Description** fields, enter your alias name and description (optional).
    Note: The **Short name** field repeats the alias name in the **Name** field and must be unique within the stage; you can edit it at any time.
    b. In the **Linked object** field, click the directory icon and select the needed Process to link the alias to.
    Note: You can link a Process later when editing the alias.
    c. Click **Ok**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

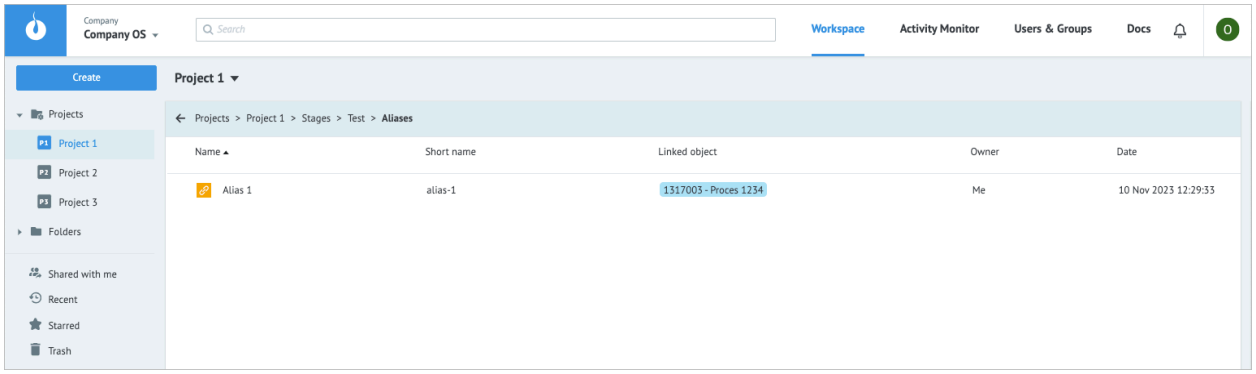


Screenshot 108: **Create alias window.**

The alias has been created, and you can see its linked Process in the field. By default, the aliases provide the following access rules:

- All users of a company can view the list of all aliases created in the company.
- Only the alias owner can manage the alias linking to a Process or a State Diagram.
- The alias owner can link the alias only to those Processes and State Diagrams to which he/she has task management and editing access.
- The alias owner cannot link Processes or State Diagrams to the alias to which he/she has access only for viewing.
- If the alias owner is denied access rights to a Process or a State Diagram linked to the alias, he/she cannot view the linked object name (the linked object identifier is always displayed regardless of access settings).
- A user can open a linked Process or a State Diagram in the **View** mode if he/she has the **View** or **Edit** access to the object.

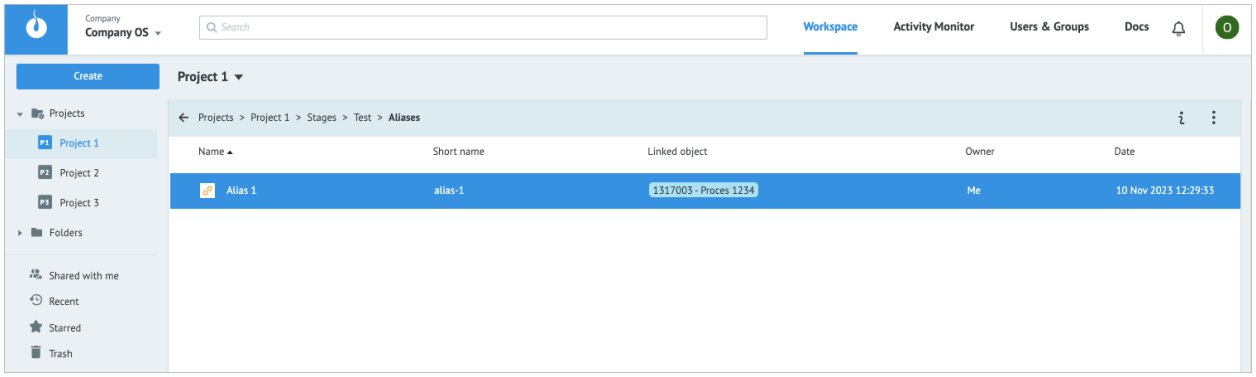
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 109: **Created alias in the list.**

Manage alias

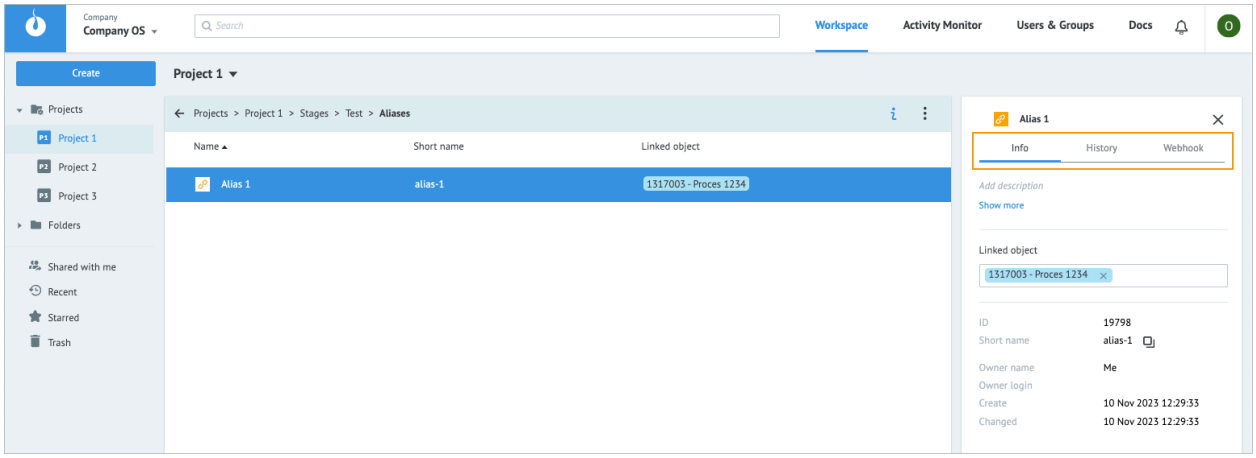
To see what additional actions you can perform on your alias, click the needed alias.



Screenshot 110: **An alias selected in the list.**

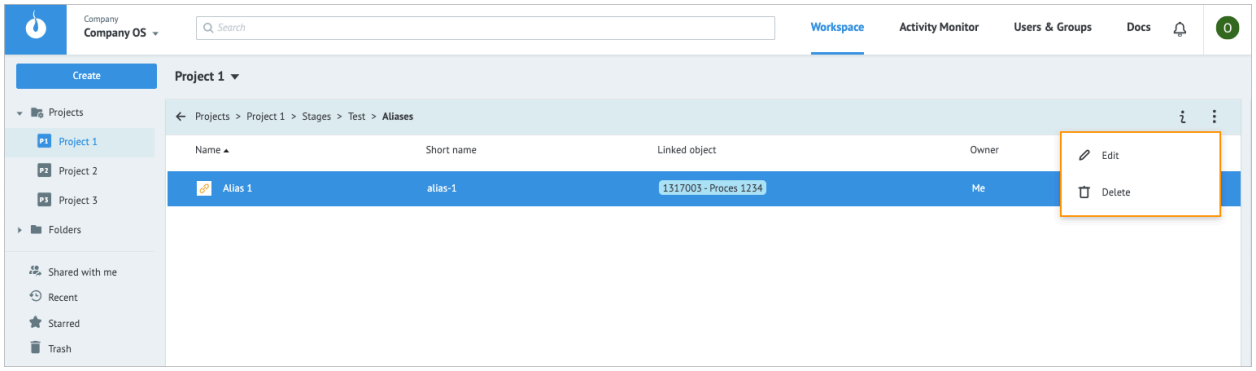
- Check the alias information:
    - In the upper-right corner, click the info icon .
    - View alias details by clicking the **Info** and **History** tabs.
    - Copy the webhook via **JSON**, **XML**, or **NVP** in the **Webhook** tab.
      Warning: The **Webhook** tab is available to the alias owner only. After clearing and selecting the **Direct URL for tasks upload** checkbox again, the link changes and you cannot send data via the initial link.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 111: **Selected alias tabs.**

- Perform more actions by using the options icon ⋮ and clicking:
  - **Edit** to change the alias name
  - **Delete** to delete the alias
    Note: To perform an action on multiple aliases, press **Shift** or **Command** on your keyboard, click the needed ones, and then click the options icon ⋮.



Screenshot 112: **Alias three-dot menu.**

Import alias

You can import aliases together with your project in a folder. As a result, the project folder will be created with linked aliases in the **Aliases** tab. There may be a conflict between already existing and aliases you want to import with the message “Aliases already exist” displayed. There are two ways to solve this problem:

- When uploading a file, click **Upload only processes**. Only folders, Processes, State Diagrams, and dashboards will be imported, while the aliases linked to them will be completely ignored.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Screenshot 113: **Upload only processes.**

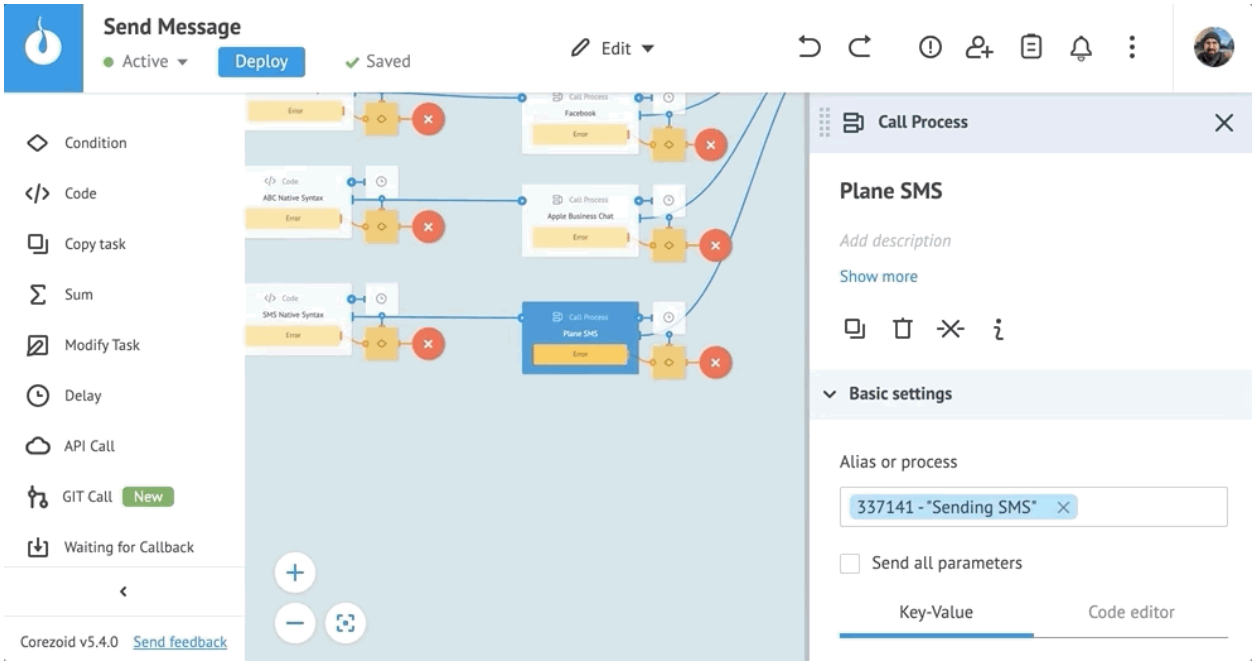
- When a file has been uploaded, click **Show conflicts**, and then **Force upload**. You can view all conflicting aliases in the file and import them by replacing the existing linked objects with imported ones.

Screenshot 114: **Show conflicts.**

Examples

You can specify an alias in the **Call Process**, **Copy Task**, and **Modify Task** nodes of a Process. You have several ways to do this in the **Basic settings** of the node details panel:

- Select an alias by using the directory icon
- Search for an alias by its name by entering the @ symbol at the beginning of the line
- Enter an alias name as a task parameter (for example, @{{alias_name}}) which contains a value equal to the alias short name



2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Screenshot 115: **Alias selection.**

2.3.8.3 Variables

Overview

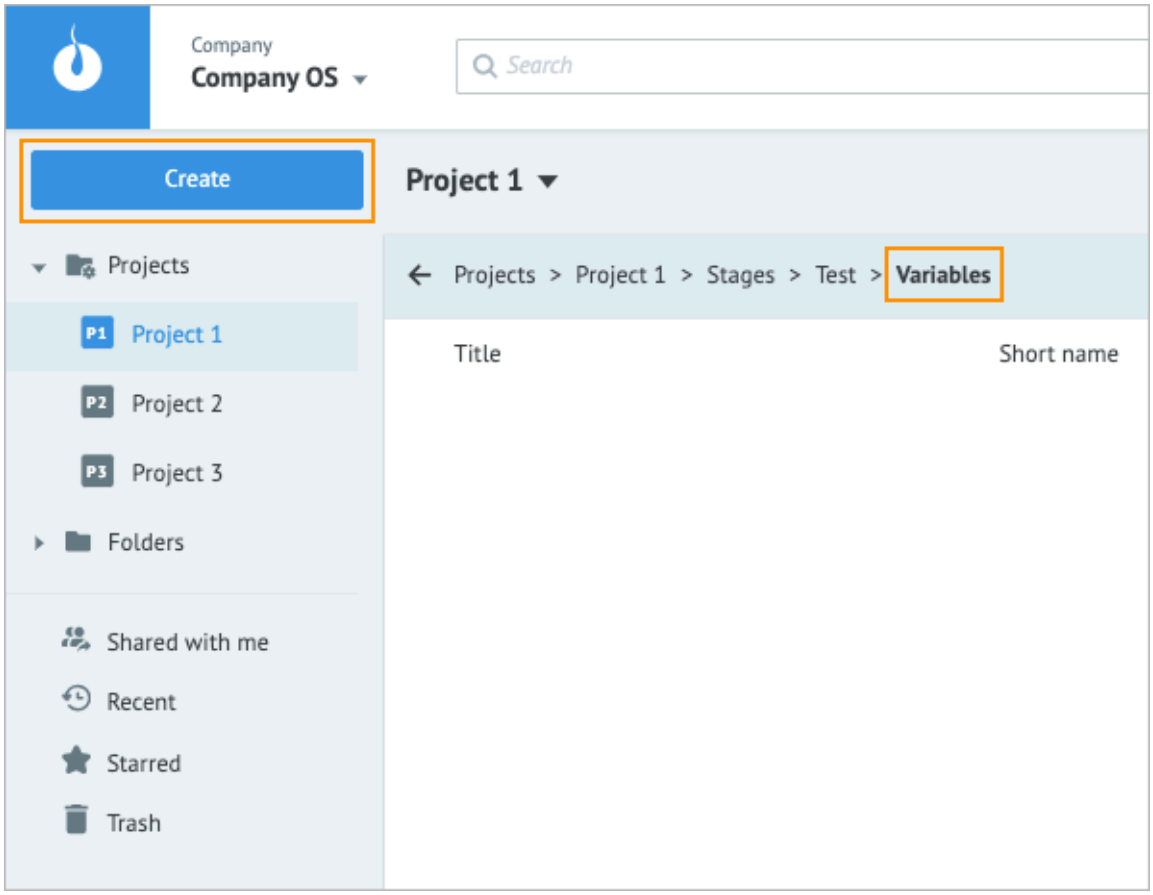
Variables are stage objects which store data for various purposes (secret, public keys and so on), which you can edit and reuse in **Corezoid** nodes. Variables store data in the **RAW** and **JSON** formats.

Create variable

To create a variable:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

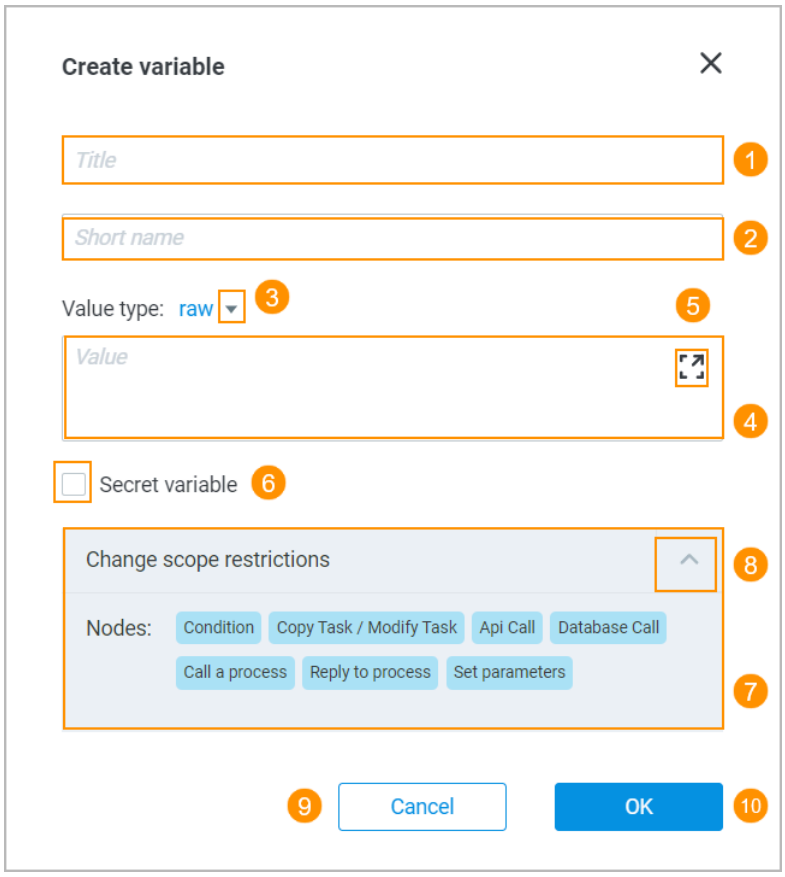
1. On the **Workspace** tab of your company, in the **Projects** folder, select the needed project and stage, double-click **Variables**, and then click **Create**.



Screenshot 116: **Create-variable.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

2. In the **Create variable** dialog:



Screenshot 117: **Create variable window.**

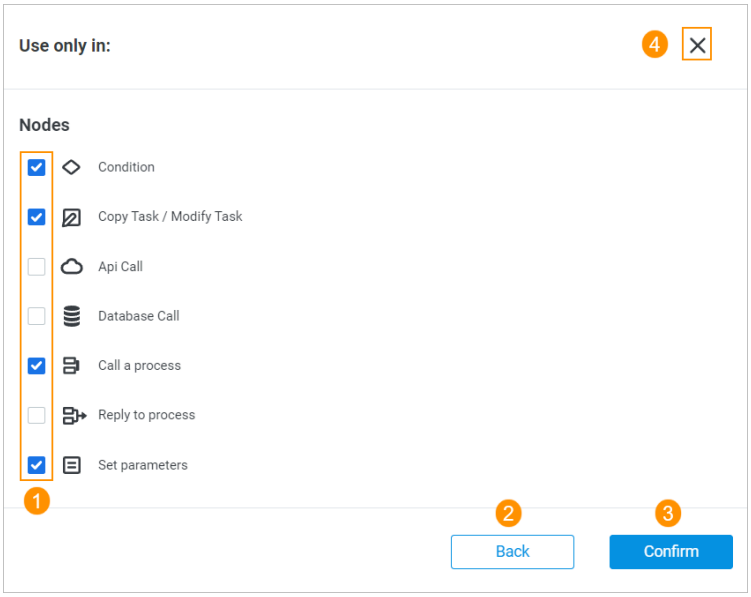
1. In the **Title** field, enter your variable title.
2. In the **Short name** field, your variable short name will be generated automatically based on the title. You can edit the short name manually.
3. In the **Value type** list, select **RAW** or **JSON** format.
4. In the **Value** field, enter the needed value or click the code editor icon and enter the value in the **Code editor** dialog that opens.
   Note: You can link a Process later when editing the alias.

5. Click to open the Code editor window in the full-screen mode.
6. Select the **Secret variable** checkbox to make your variable secret.
   Note: Making a variable secret is irreversible and automatically restricts its use to the **API Call** nodes only.

7. In the **Change scope restrictions** block, click the title to open the **Use only in** window. In the window, select the nodes, in which the variable will be available for the use by selecting the corresponding checkboxes (1) and clicking **Confirm**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

(3).
You can discard the scope restriction changes by clicking **Back** (2) or window close icon (4):

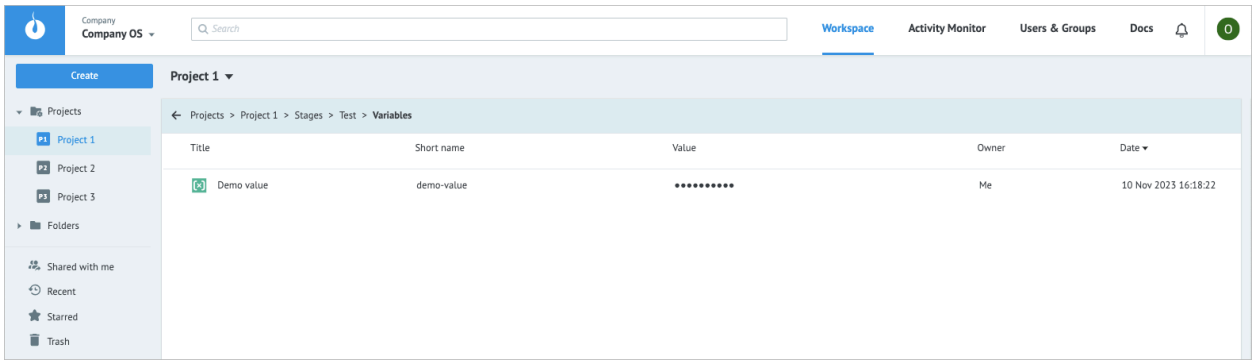


Screenshot 118: **Use only in: window.**

8. Click to hide the list of nodes where the variable will be available.
   Note: You must specify at least one node for the variable.

9. Click **OK** to create a variable, or **Cancel** to cancel.

The variable has been created, and you can use it in your Processes.



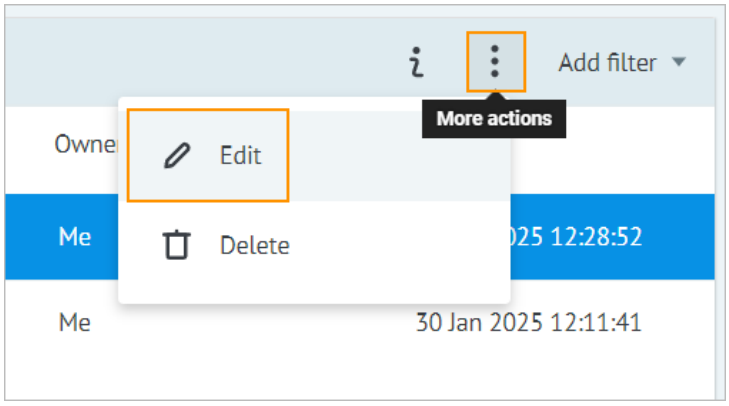
Screenshot 119: **Created variable in the list.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Variable edit

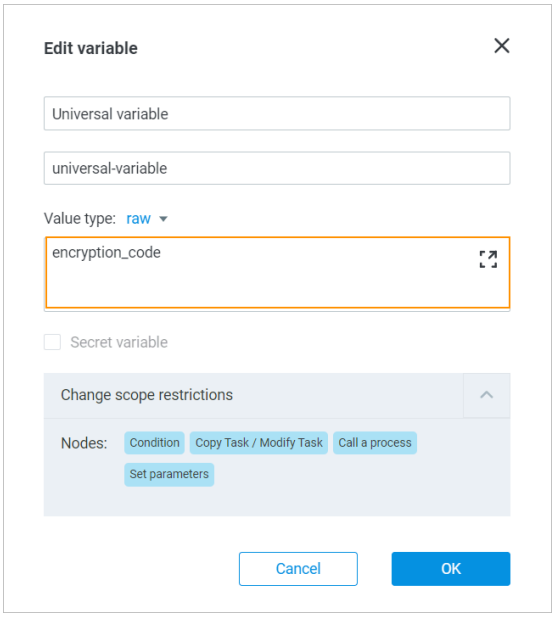
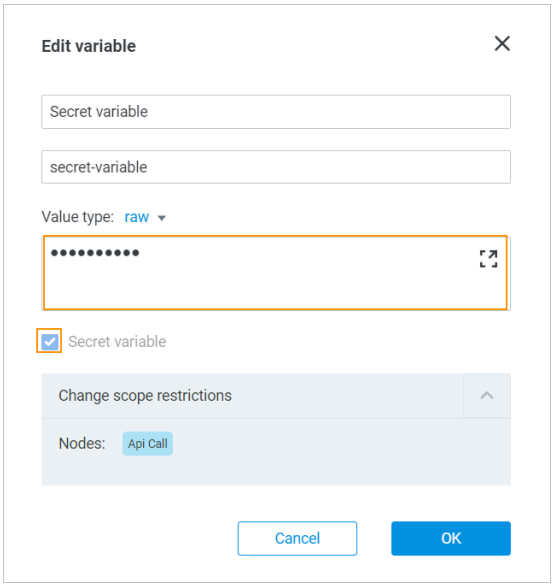
To edit a variable:

1. Select the variable in the list.
2. Click the variable three-dot menu icon or right-click the selected variable to open the context menu.
3. Select **Edit** in the menu list:



Screenshot 120: **Variable edit.**

4. In the **Edit variable** window, edit the variable parameters and scope restrictions. The value is not shown for secret variables, and the **Secret variable** box can't be cleared when editing:

Edit a non-secret variable	Edit a secret variable
	

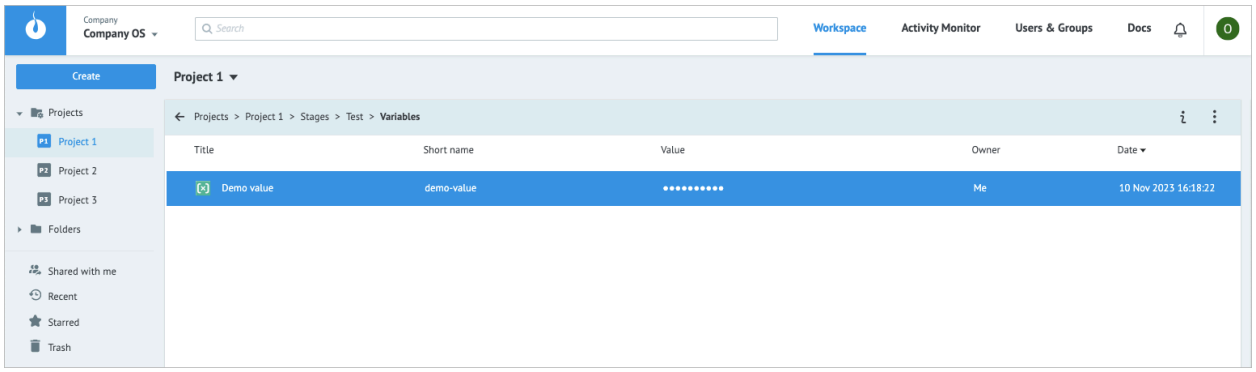
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Screenshot 121: **Edit variable window for secret and non-secret variables.**

5. Click **OK** to save the changes.

Manage variable

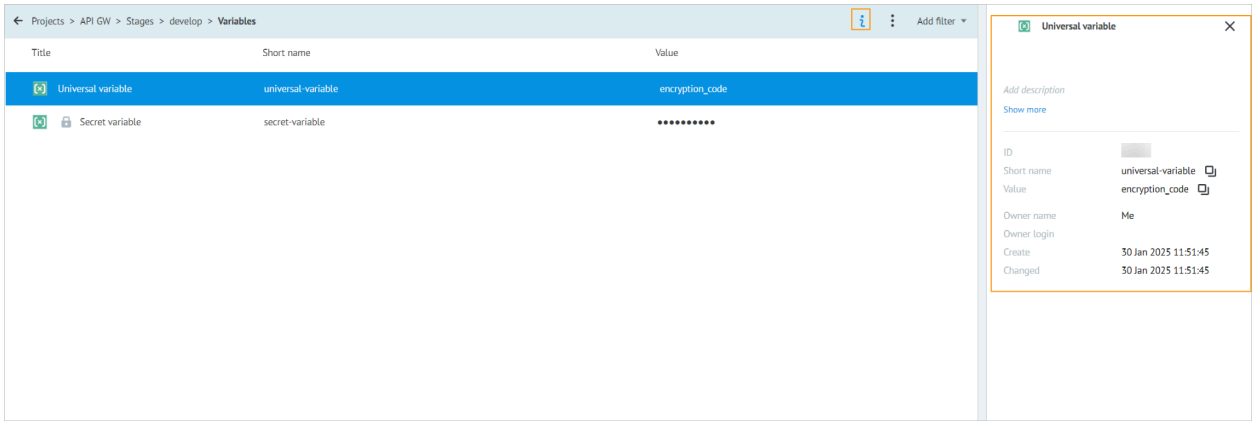
To see what additional actions you can perform on your variable, click the needed variable.



Screenshot 122: **Select a variable.**

Now you can:

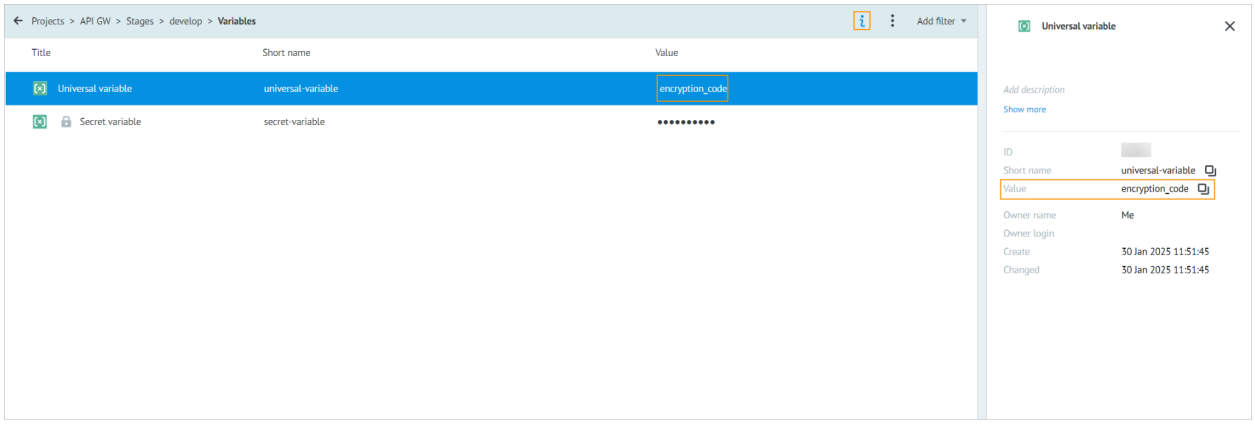
- Check the variable information, by clicking the info icon .



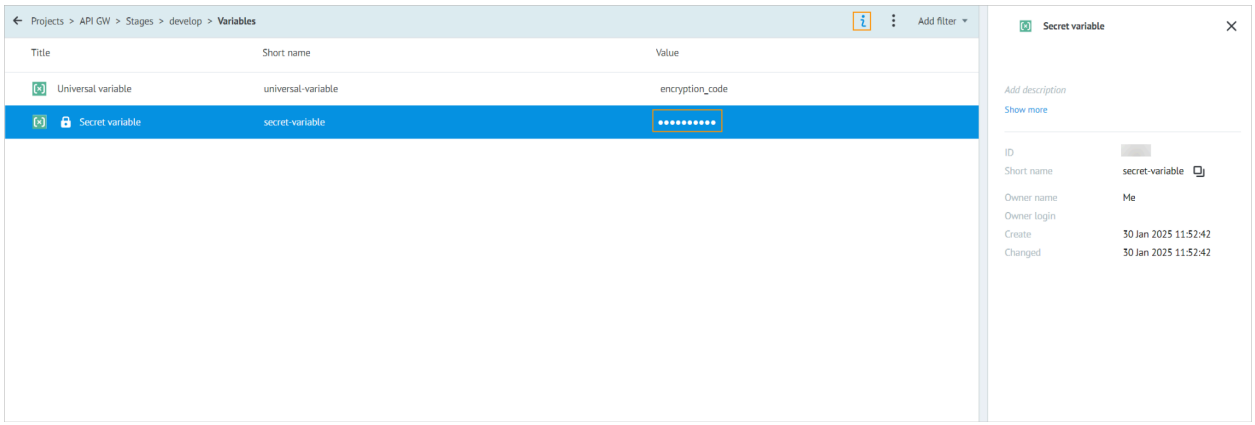
Screenshot 123: **Variable info view.**

When viewing a secret variable info, the value is obfuscated in the **Value** column of the variables list and not shown in the variable info panel:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

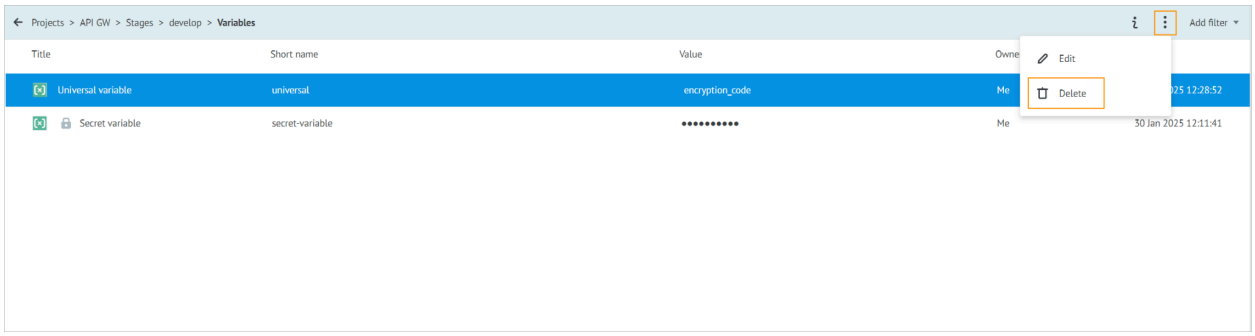


Screenshot 124: **Variable value view for non-secret variables.**



Screenshot 125: **Variable value view for secret variables.**

- Delete the variable, by clicking the options icon ⋮, and then clicking **Delete**.



Screenshot 126: **Delete a variable.**

Note: To perform an action on multiple variables, press **Shift** or **Command** on your keyboard, click the needed variables, and then click the needed icon.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Examples

Variable syntax:

For getting data stored in the **RAW** and **JSON** formats in variables, use:
`{{env_var[id]}}`

- `{{env_var[@shortname]}}`

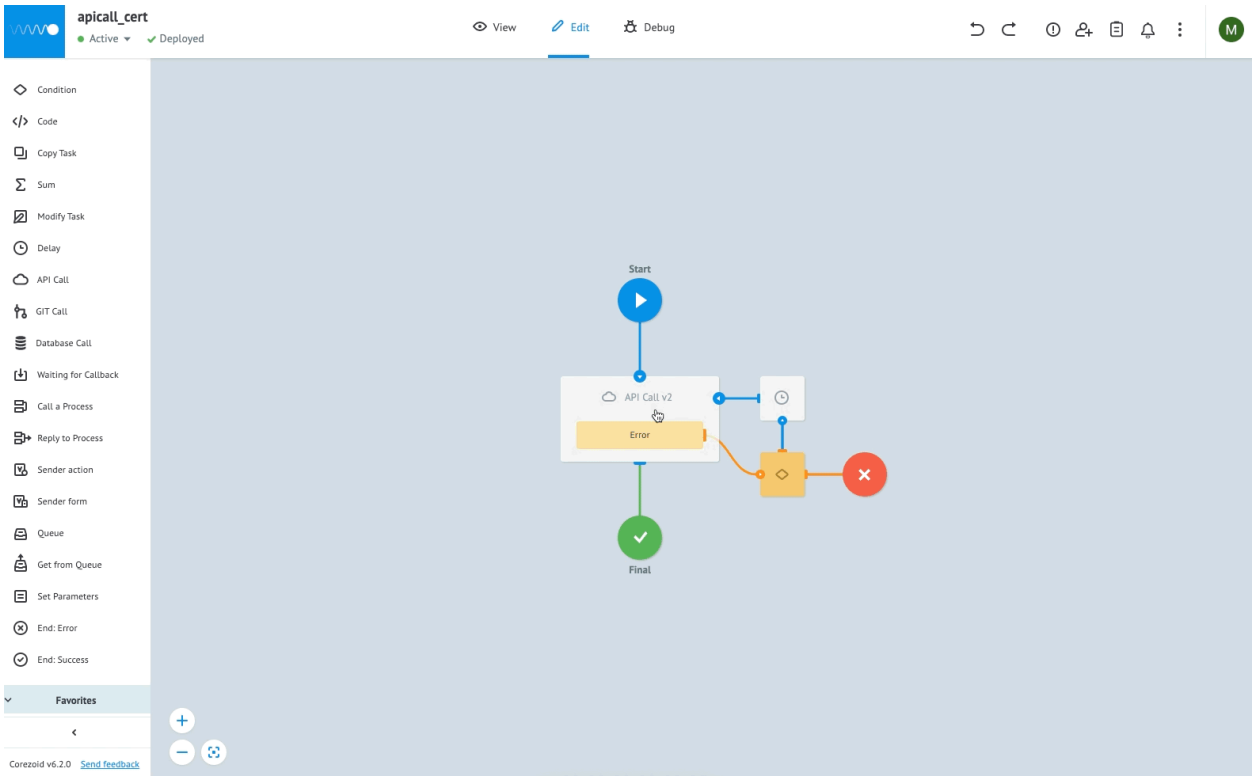
For getting data stored in the JSON format in variables containing nested structures, use:
`{{env_var[id].key}}`

- `{{env_var[id].key[1].param}}`

Variables use:

- Use of a variable to store a certificate and sign requests that you send to the API in the **API Call** node.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 127: **Storing a certificate in a variable.**

- Signing a request with a certificate by using a variable.

Screenshot 128: **Signing a request using a variable.**

- Using the **Sign the request with the secret key** feature to store a secret key and sign the requests you send with it:
    - Create an API key and call it **Key**.
    - Create a variable, select the **JSON** value type, and insert the following construction (where **secret** is the value of the secret key.):

```
{
  "key": {
          "param": "12312",
            "param2": "secret"
        }


        }
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- Save the variable with the **secret** key part.

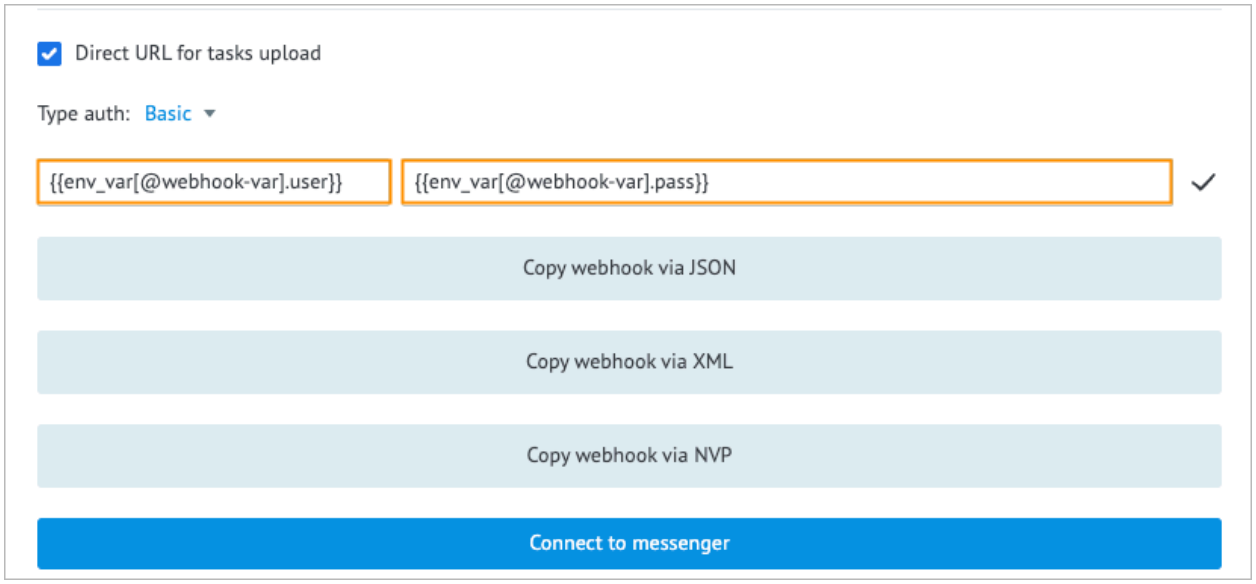
Open the **API Call** node in the **Edit** mode and enter the request you want to use (with the actual **conv_id** and **company_id** parameter values from your environment):

```
{
     "ops": [
        {
             "type": "create",
             "conv_id": 0000000,
            "obj": "task",
           "action": "user",
           "data": {
               "code": 200
         },
              "ref": "test",
              "company_id": "i000000000"
        }
]


 }
```

- In **Other**, select the **Sign the request with the secret key** checkbox.
- In the field below, enter the **{{env_var[@secret-key-part].key.param2}}** construction to read the necessary data from the JSON array stored in the **secret-key-part** variable.
- In the upper-left corner of the page, click **Deploy** to save the changes.
- Your requests sent from the **API Call** node will be signed with the **secret** key stored in the key variable.

- Use variables to establish more secure authorization in **Start** nodes and conceal basic authentication credentials. You can use a variable, for example the `webhook-var`, with the following constructions:
  - *{{env_var[@webhook-var].user}}* for the username field
  - *{{env_var[@webhook-var].pass}}* for the user password field.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 129: **Using constructions in a variable.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

2.3.8.4 Versions

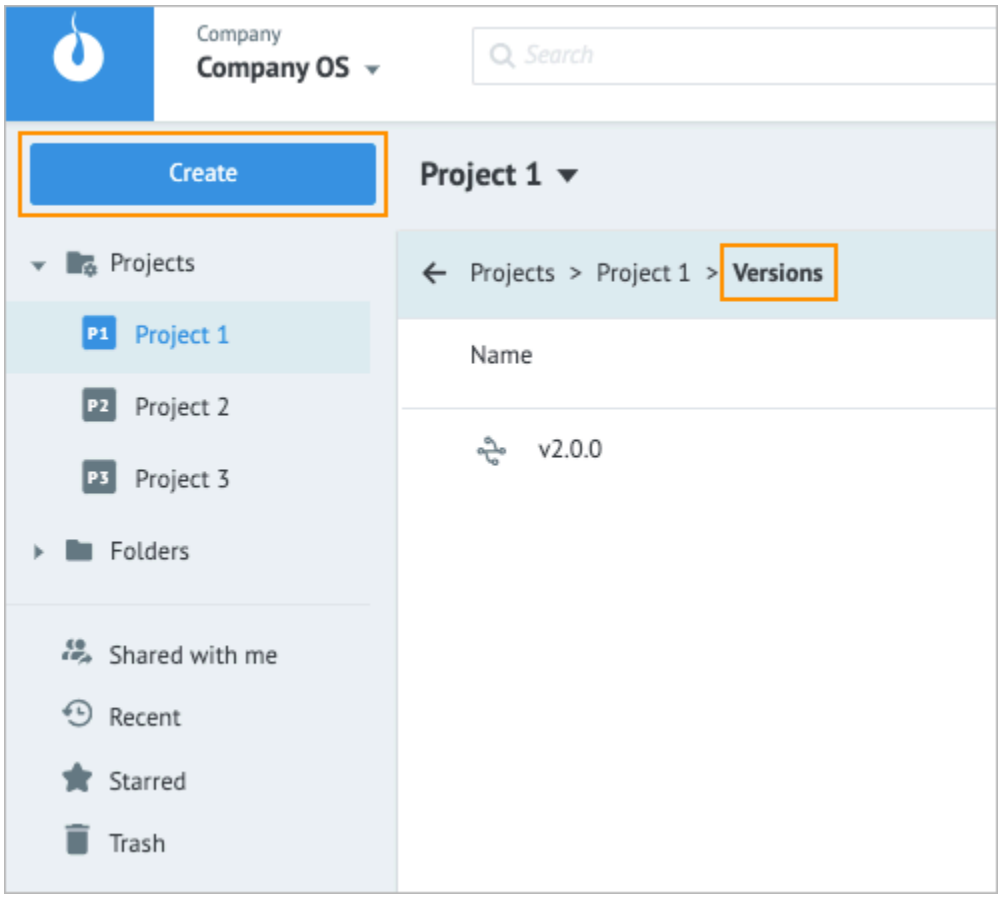
Overview

Versions are created from stages. A version is a snapshot of a stage, containing all its objects, but not containing its tasks.

Create version

To create a version:

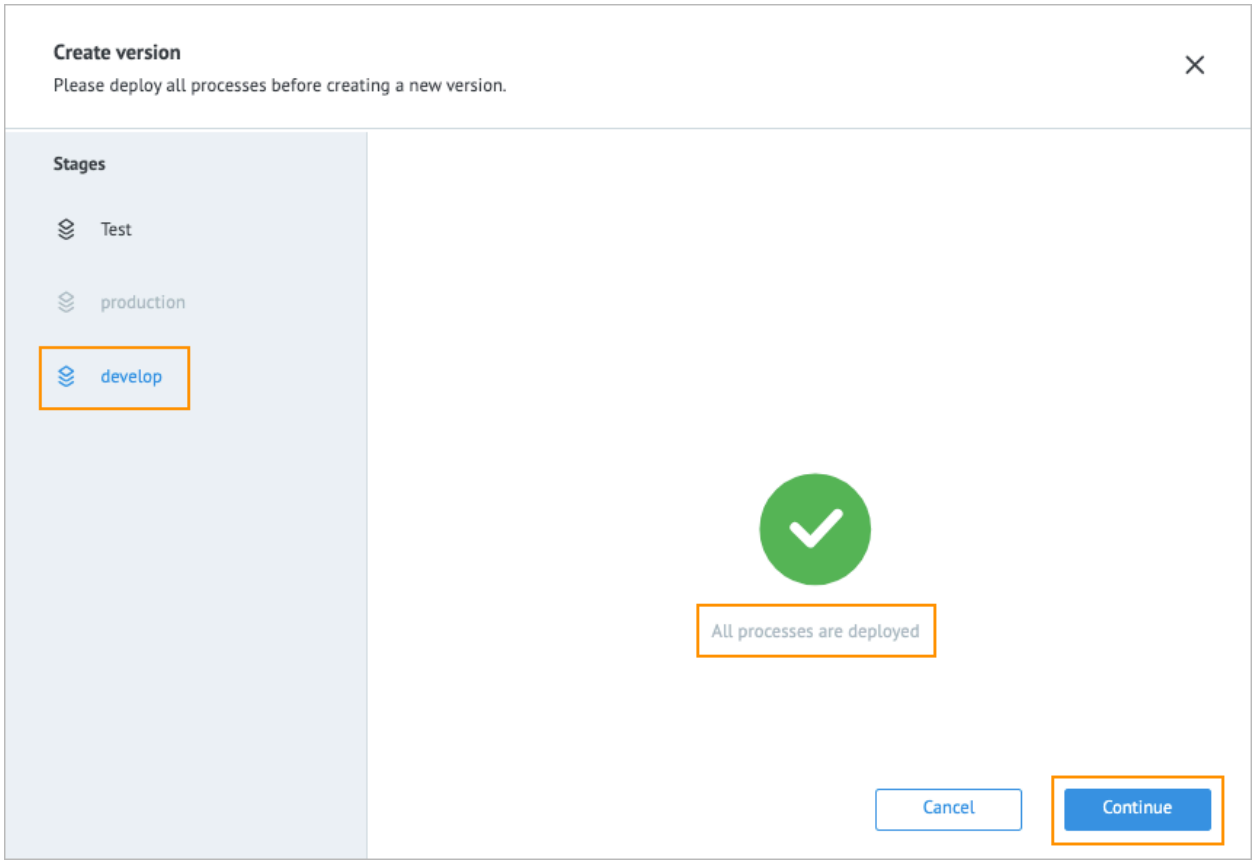
1. On the **Workspace** tab of your company, select the needed project, double-click **Versions**, and then in the upper-left corner, click **Create**.



Screenshot 130: **Create a version.**

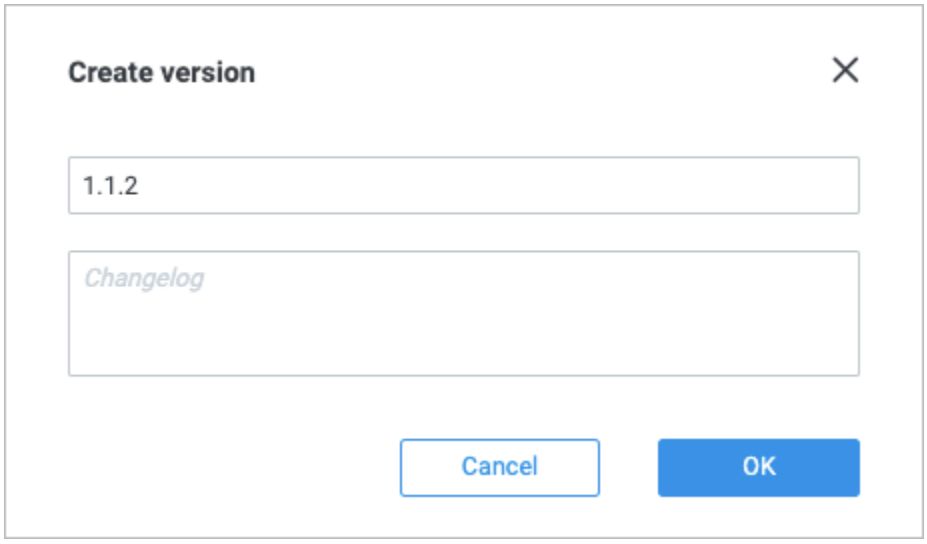
2. In the **Stages** dialog, select the needed stage, and then click **Continue**.
Note: Empty stages are not available for selection. After selecting the stage, the success message appears if all the Processes have the **Deployed** status.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 131: **Create version page.**

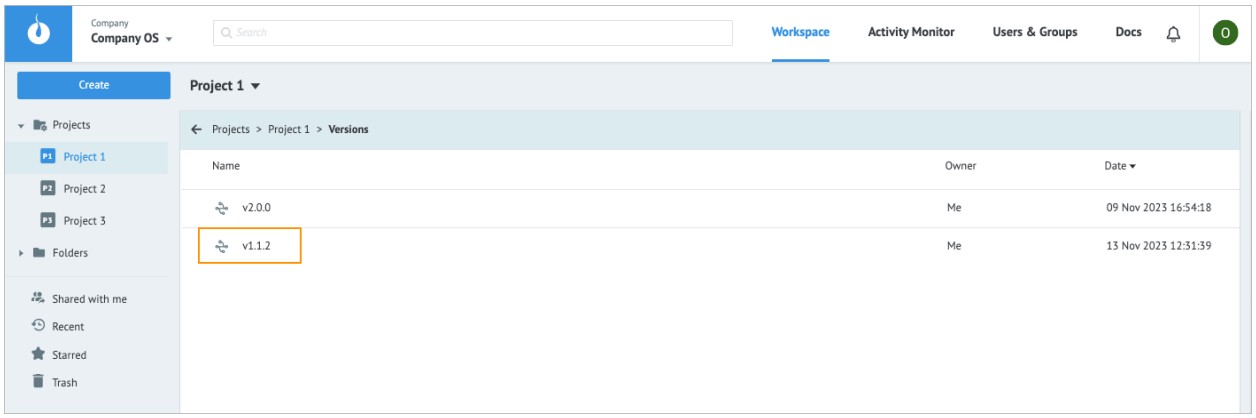
3. In the **Create version** dialog, enter the version number and change details (optional), and then click **Ok**.



Screenshot 132: **Create version window.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

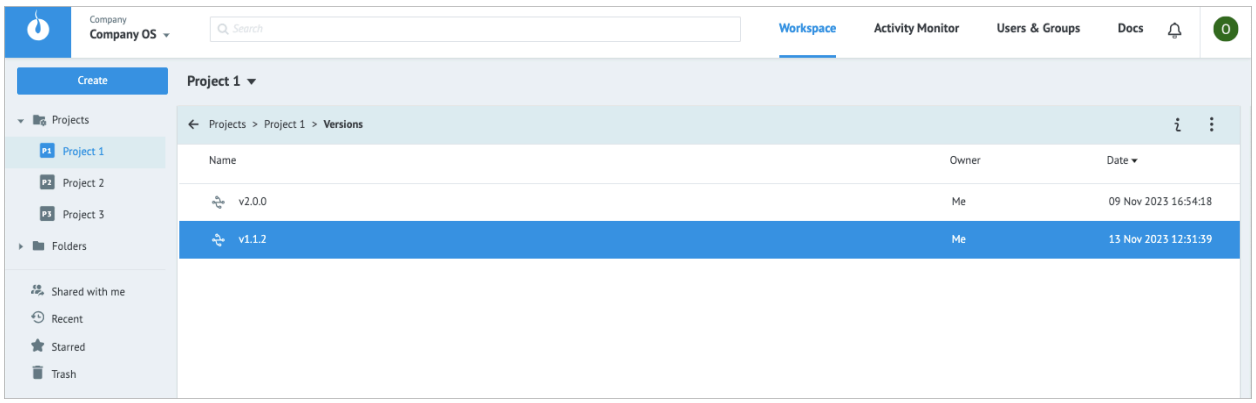
The new version has been created.



Screenshot 133: **Created version in the list.**

Manage version

To see what additional actions you can perform on your version, click the needed one in the selected project.

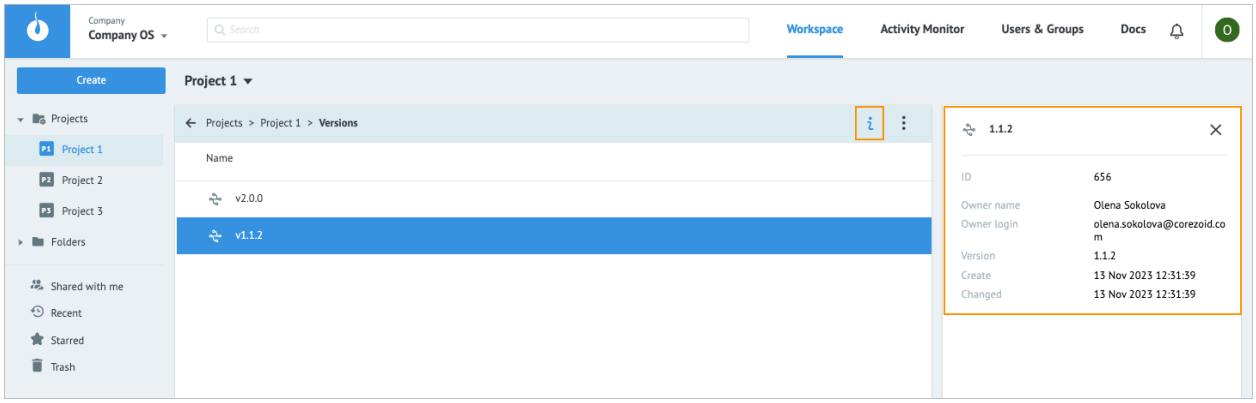


Screenshot 134: **Selected version in the list.**

Now you can:

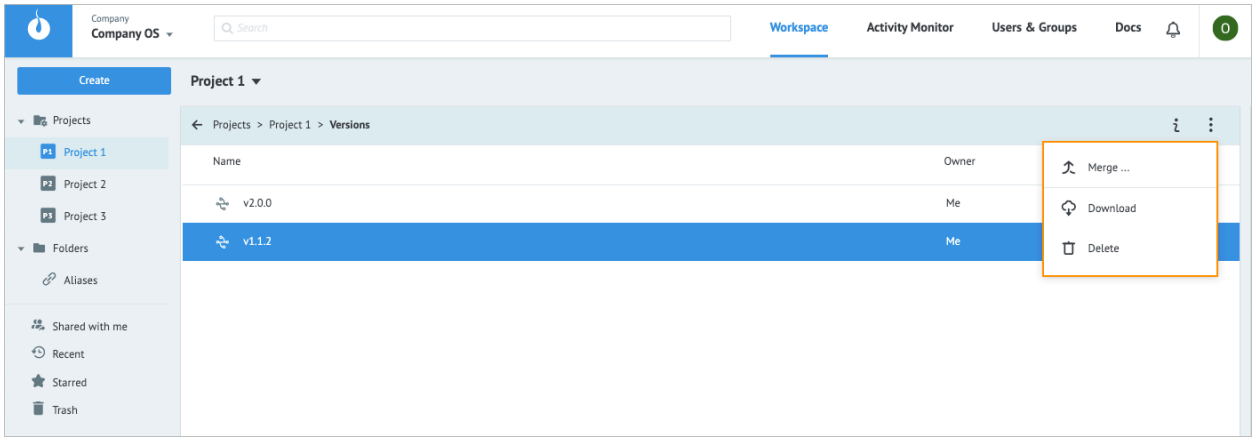
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- Check the variable information, by clicking the info icon .



Screenshot 135: **Version info.**

- Perform more actions by clicking the options icon and selecting:
  - **Merge** to merge the version with a stage.
    Note: For more information, go to Merge.
  - **Download** to download the version
  - **Delete** to delete the version
    Note: To perform an action on several versions, press **Shift** or **Command** on your keyboard, click the needed ones, and then click the options icon .



Screenshot 136: **Version three-dot menu.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

2.3.8.5 Merge

Overview

Merge is an operation available for stages and versions in projects. Merging is adding data from a stage, version or a file (exported stage or version) to another stage. Thus, you can have a stage or version or a stage/version file as a source and an existing stage as a target.

Perform merge

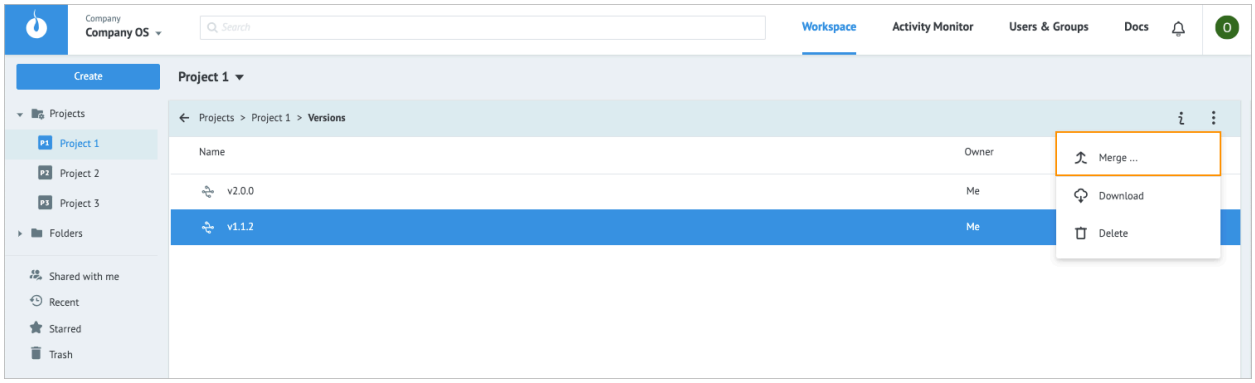
Precondition

To perform a merge, you need to have at least one company and one Project in it. Before performing a merge, check the following:

- No source stage Process has any errors.
- All source stage Processes have the **Active** status.
- The source stage is immutable: All source stage Processes are deployed.
- The target stage is immutable: All target stage Processes are deployed.

To merge the version with another version, stage, or file:

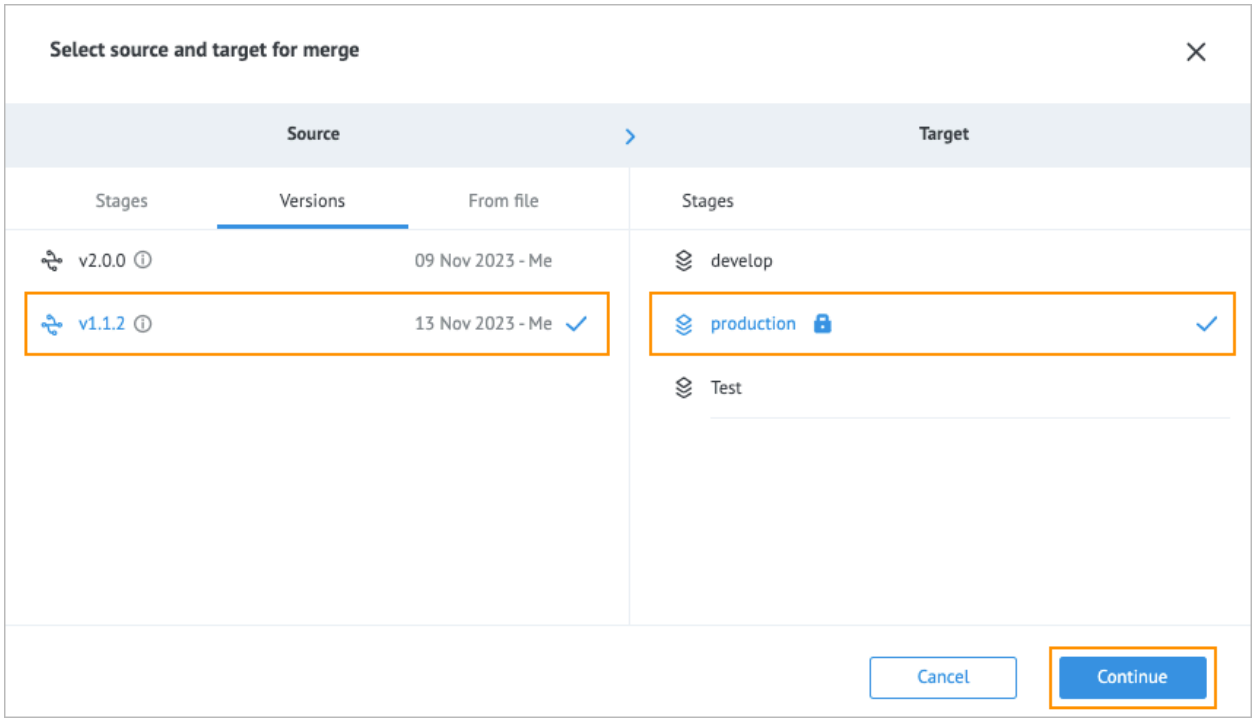
1. On the **Workspace** tab of your company, select the needed project, and then the stage or version that you want to use as a merge source.
2. In the upper-left corner, click the options icon ⋮, and then click **Merge**.



Screenshot 137: **Version Merge selection.**

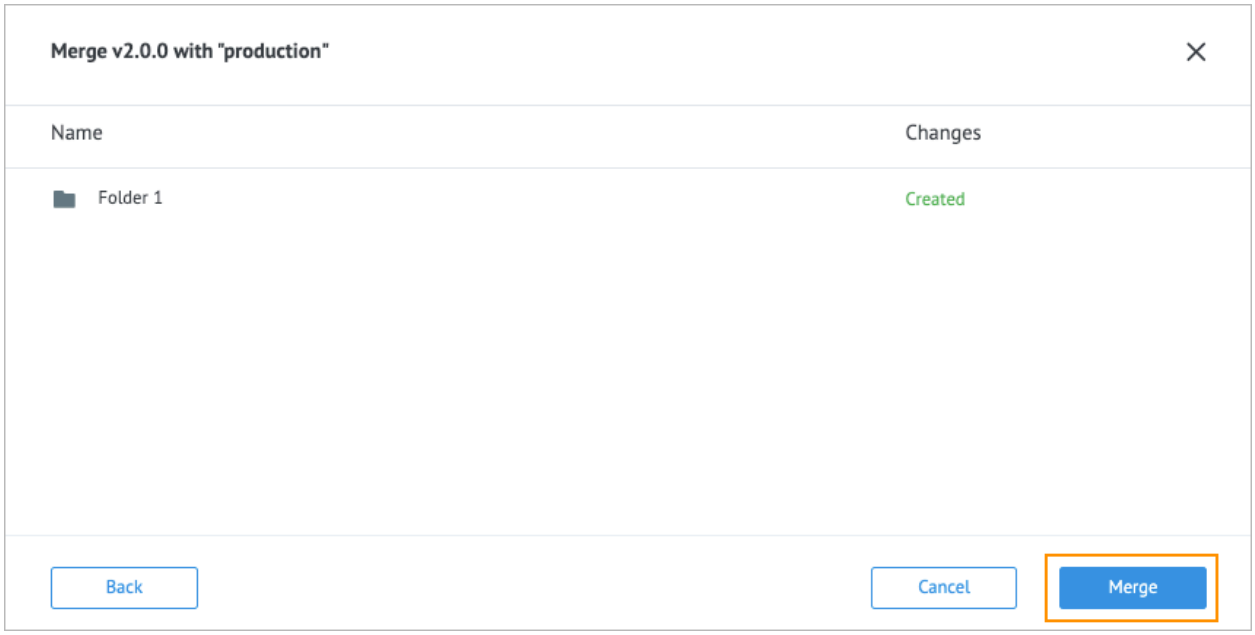
3. In the **Source** and **Target** parts of the dialog, select the needed source (from the **Stages**, **Versions**, or **From file** tabs) and target correspondingly, and then click **Continue**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 138: **Version merge window: source and target selection.**

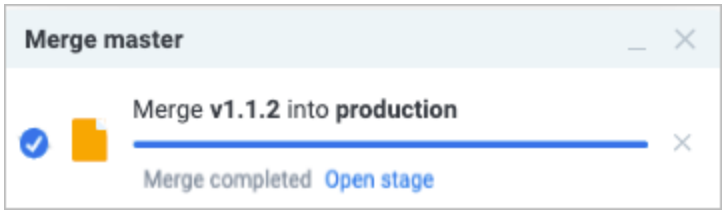
4. In the newly created folder dialog, click **Merge**.



Screenshot 139: **Version merge window: merge confirmation.**

The version has been merged to the selected stage, the success message appears in the lower-right corner of the page, and you can open the newly created stage by clicking **Open stage**.

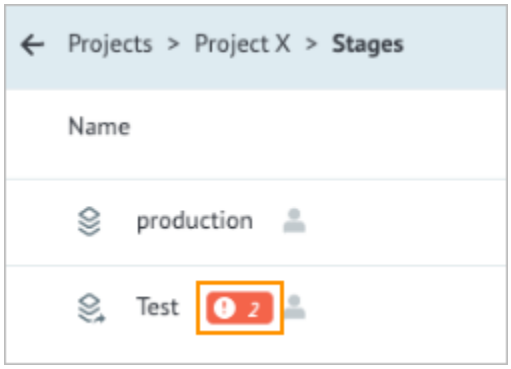
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 140: **Version merge progress indicator.**

Error handling & troubleshooting

When performing a merge, you may encounter the following errors.
1. The stage is not immutable: There are undeployed Processes in a stage. The stage name has the warning icon with the number of undeployed Processes inside.



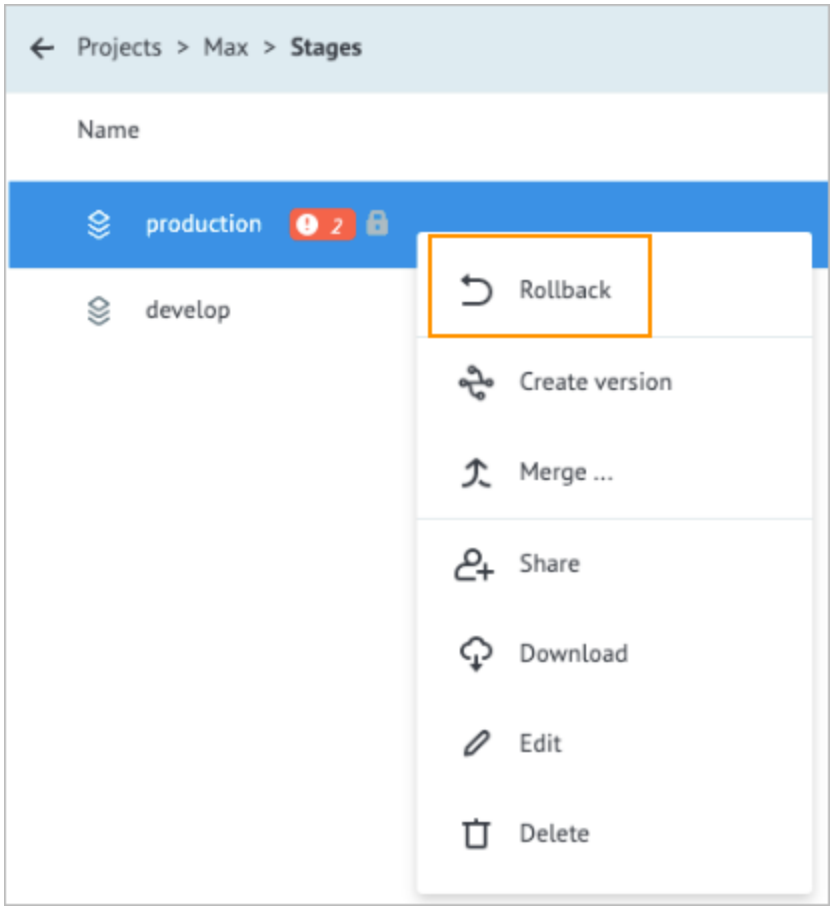
Screenshot 141: **Merge errors.**

Solution
First option: Open each Process in the **Edit** mode, and in the upper-right corner of the page, click Deploy.

Second option:

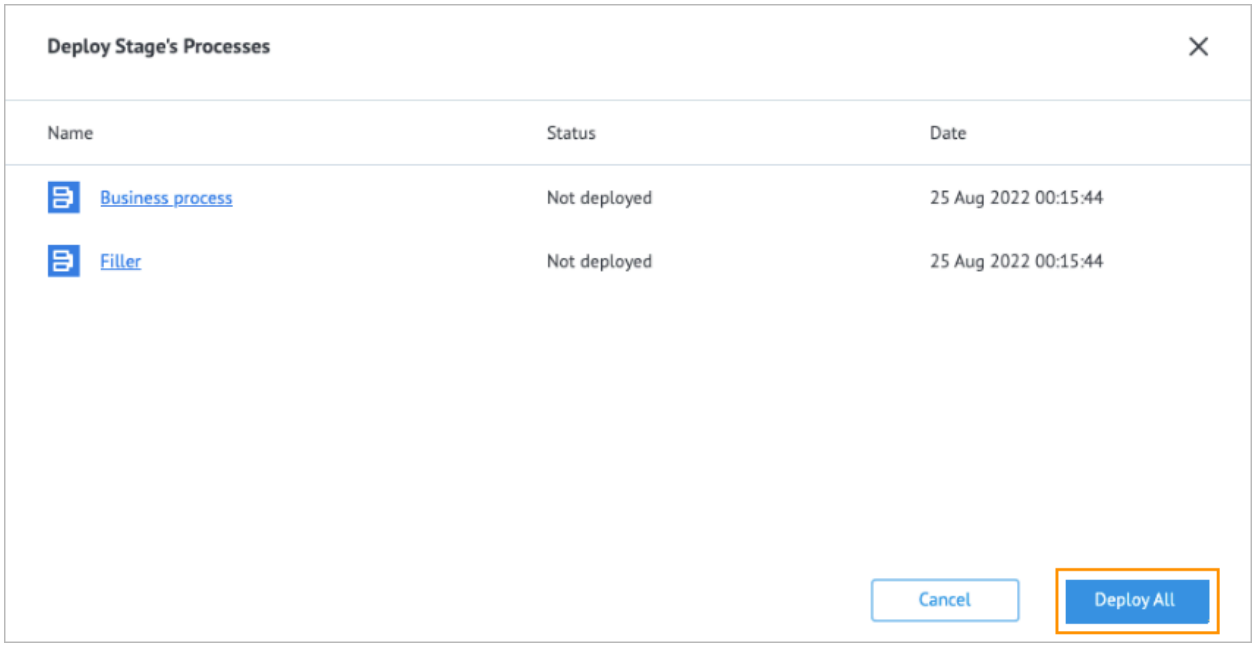
1. Right-click the not immutable stage with undeployed Processes and in the dropdown list, select **Rollback**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 142: **Merge rollback.**

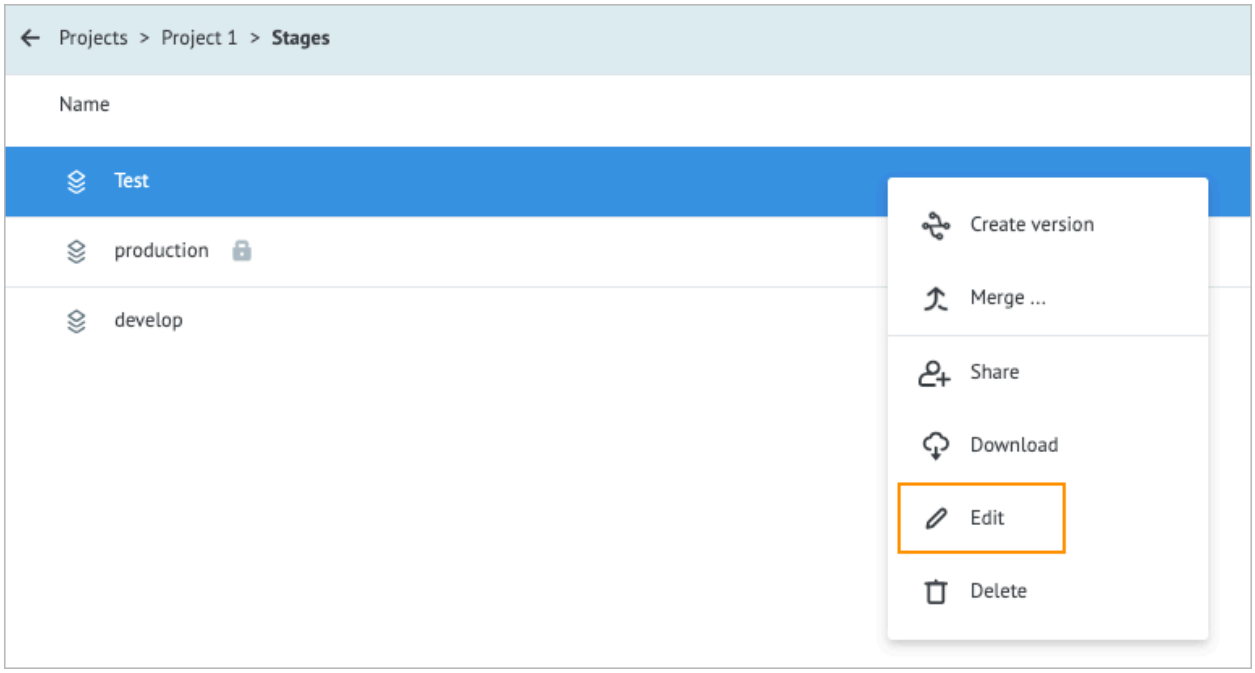
2. In the **Deploy Stage's Processes** dialog, click **Deploy All**.



2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

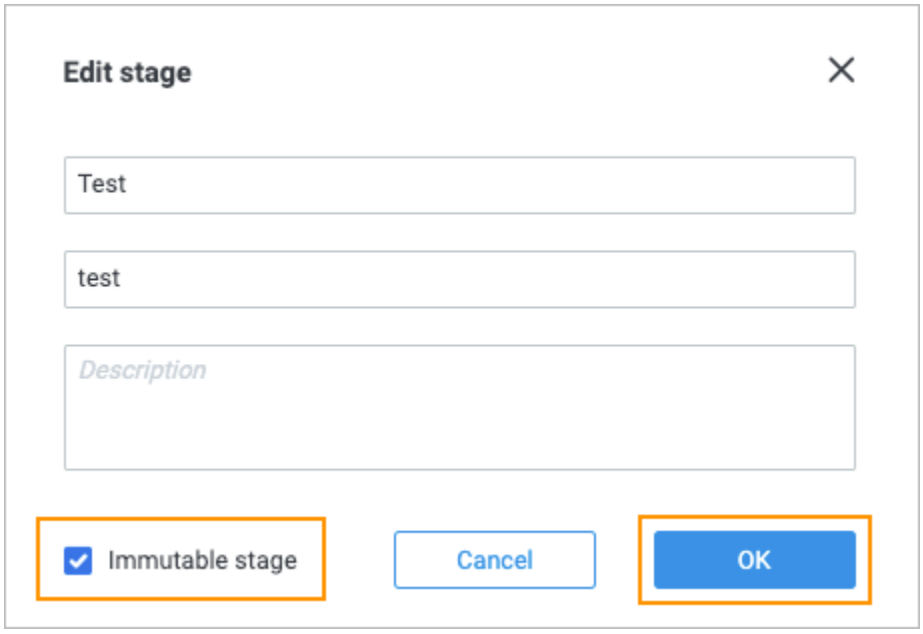
Screenshot 143: **Stage Processes deployment.**

3. Right-click the needed not immutable stage, and in the dropdown list, select **Edit**.



Screenshot 144: **Stage edit.**

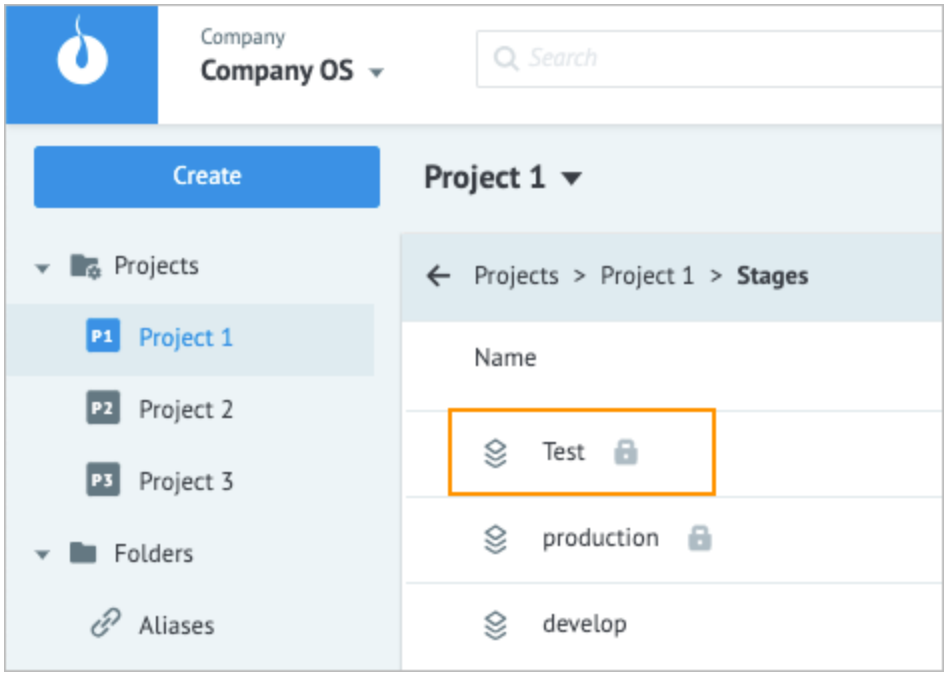
4. In the **Edit stage** dialog, select the **Immutable stage** checkbox, and then click **Ok**.



Screenshot 145: **Immutable stage checkbox.**

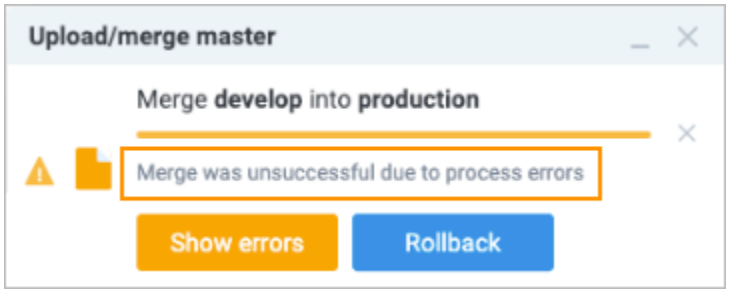
The stage becomes immutable and gets the lock icon next to its name.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 146: **Stage immutability indicator.**

2. Unsuccessful merge. The merge was unsuccessful, and you get the corresponding notification in the upload master window.

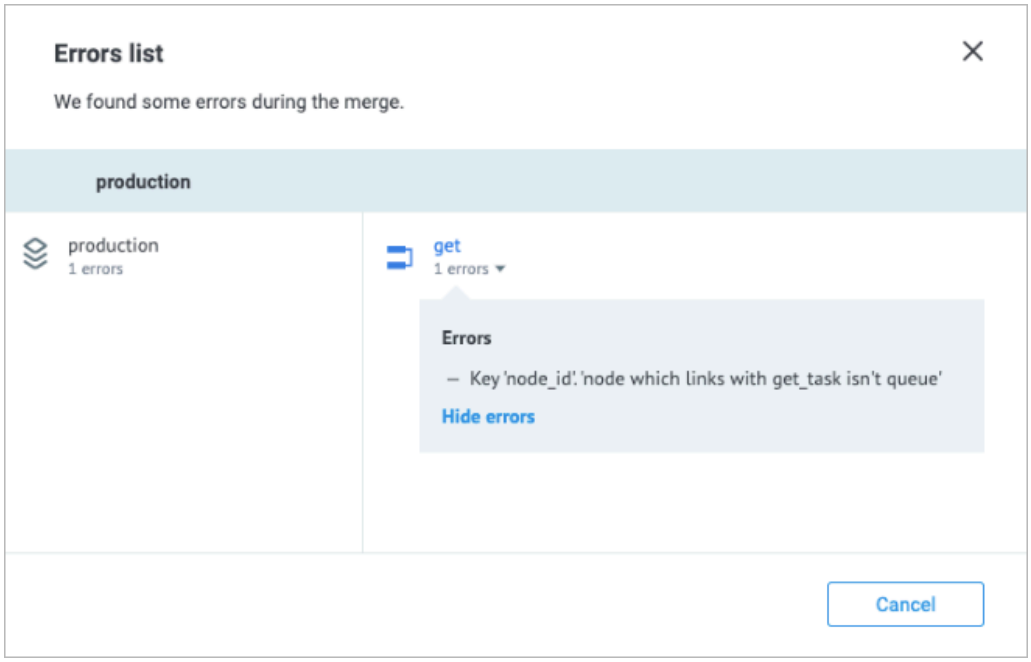


Screenshot 147: **Unsuccessful merge indicator.**

Solution

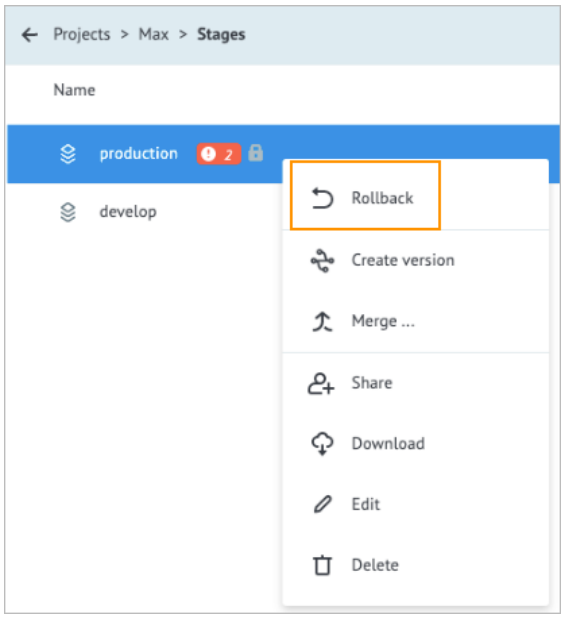
First option: In the upload master window, click **Show errors**. The **Errors list** dialog opens with error details. You can find the corresponding Processes and fix the errors.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 148: **Merge errors display.**

Second option: In the upload master window, click **Rollback**, which means reverting unsuccessful merge changes. Now you can fix errors and try to perform a merge once again.



Screenshot 149: **Merge rollback.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

2.3.9 Notation Schema for Corezoid Node Parameters

Description of Corezoid node parameters follows the general scheme:

Parameter Name [Required Status] [UI Element Type] [Data Type]: Brief description.

1. **Parameter Required Status**

- [Required]: Mandatory parameter
- [Optional]: Optional parameter

2. **UI Element Type**

- [Field]: Text input field
- [Action]: Action or command

3. **Data Type**

1. [String]: Sequence of characters, digits, or symbols treated as text. Includes alphanumeric and special characters.
   Example: hello, +1-999-666-3333, %$@#.
2. [Number]: Positive and negative numbers with and without fractions.
   Example: 0, 707, -100, 707.07.
3. [Boolean]: True or false values.
   Example: true, false.
4. [Array]: List with a number of elements in a specific order—typically of the same type.
   Example: rock (0), jazz (1), blues (2), pop (3).
5. [Object]: Object with nested structure, for example {{conv[{{Process_ID}}].ref[{{REF}}].parameter}}. See 2.3.5 Parameters for more details.
6. [Timestamp]: The unix time stamp is a number of seconds that have elapsed since midnight (00:00:00 UTC), 1st January 1970 (Unix time).
   Example: 1632855600.
7. [Any]: Any of the data types mentioned above, and:

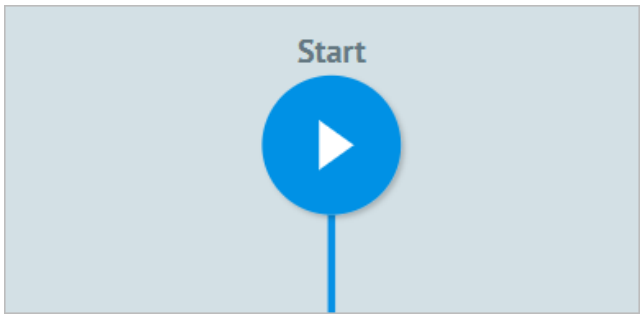
- variables in double brackets, for example {{variable}},
- dynamic variable contained in the currently processed tasks,
- variables in nested structures (constructions), see 2.3.5 Parameters for more details.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3. Nodes Description

3.1 Start

Overview



Screenshot 150: **Start node view.**

The **Start** node is the single entry point for tasks in a Process or State Diagram. It receives tasks from:

- **View Mode**: Manual task creation for testing, debugging, and configuration
- **CSV**: Bulk data import processing
- **Webhooks**: External task sources
- **Copy Task**: Task duplicates from other Processes
- **Call a Process**: Function-like Process invocation
- **Corezoid API**: Direct platform integration for task management

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

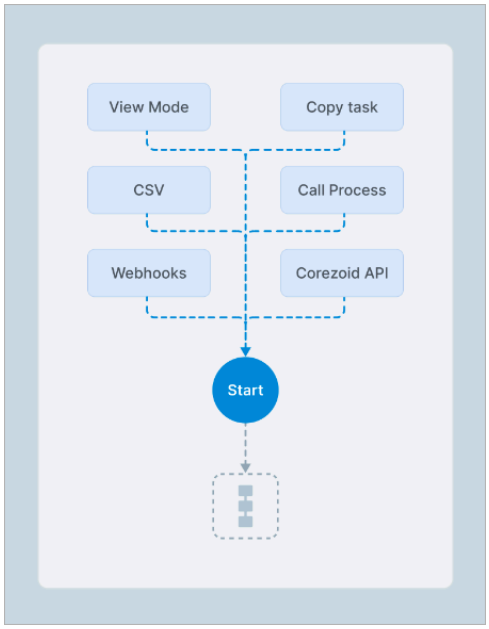


Diagram 5: **Start node flow.**

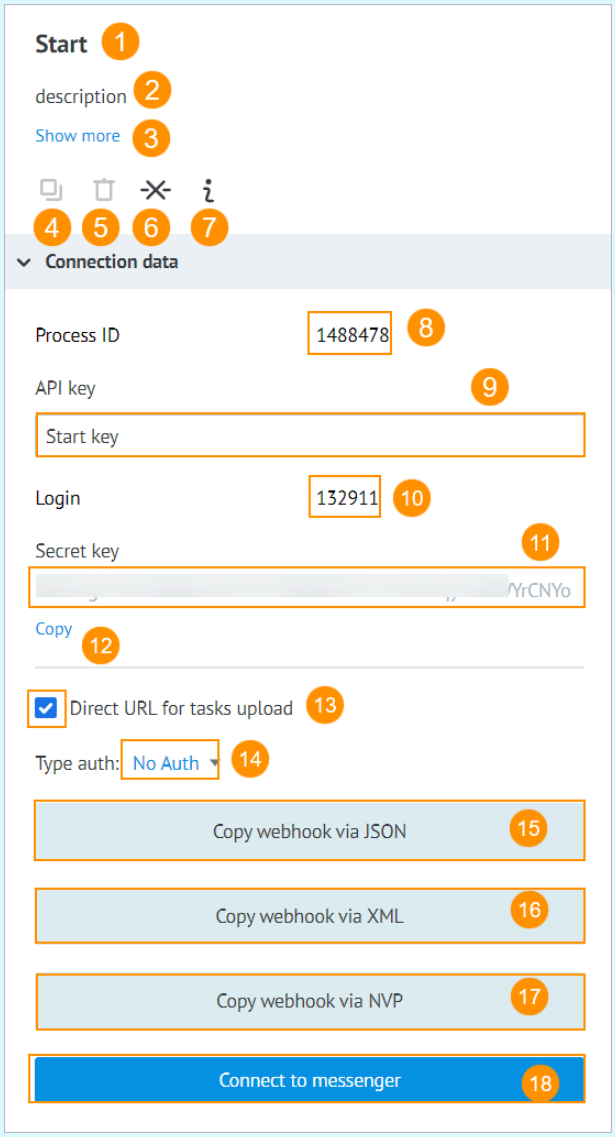
Key points:

- Added automatically when creating a Process/State Diagram
- Only one Start node per workflow
- Serves as webhook endpoint for external tasks

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Node Settings

The **Start** node has the following parameters:



Screenshot 151: **Start node settings.**

1. **Title** [Optional] [Field] [String]**:** Node title.
2. **Description** [Optional] [Field] [String]: Node description.
3. **Show more** [Optional] [Action]: Long descriptions are reduced to a compact view to simplify node settings. Click to see the whole description.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

4. **Copy node** [Optional] [Action]: Creates a copy of the current node with all settings**.**
   Note: Cannot be used with Start node as only one Start node is allowed per Process.
5. **Remove node** [Optional] [Action]: Deletes current node from Process.
   Note: Cannot be used with Start node as only one Start node is allowed per Process.
6. **Remove links** [Optional] [Action]: Removes all connections to/from current node.
7. **Info** [Optional] [Action]: Displays node ID.
8. **Process ID** [Required] [Field] [String]: Unique identifier that allows you to direct your tasks to the needed Process from any other Process within Corezoid.
9. **API key** [Optional] [Field] [String]: Authentication key for secure system access. After clicking the field, you can enter the API key name, and the following key parameters will be generated automatically.
10. **Login** [Preset] [Field] [String]: Associated API login.
11. **Secret** [Preset] [Field] [String]: Generated secret key value.
12. **Copy** [Optional] [Action]: Copy key to clipboard.
13. **Direct URL for tasks upload** [Optional] [Action] [Boolean]: Enable webhook endpoint for receiving tasks. The checkbox is selected by default, which means you have to specify the generated endpoint URL value when subscribing to a webhook.
    Supported input formats: JSON, XML, and NVP.
    Note: If you clear the checkbox, the endpoint URL will be turned off. When you select it again, a new endpoint URL will be generated, and you will need to update your webhook subscription.
14. **Type auth** [Optional] [Action] [String]: Authorization type to secure received requests processing by using the **Basic** option and to process all requests by using the **No Auth** option.
    Note: For more information, go to Enable secure authorization.

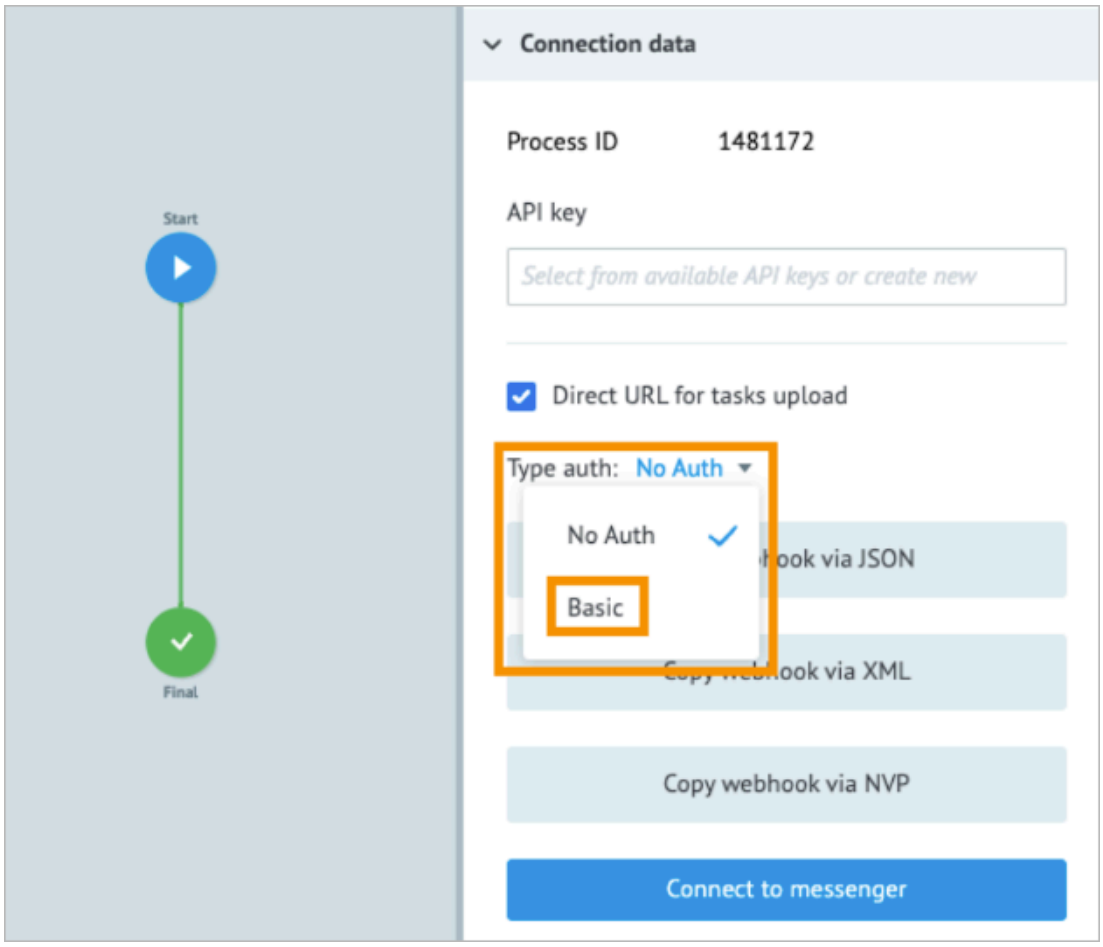
15. **Copy Webhook via JSON** [Optional] [Action]: Copy endpoint URL in JSON format.
16. **Copy Webhook via XML** [Optional] [Action]: Copy endpoint URL in XML format.
17. **Copy Webhook via NVP** [Optional] [Action]: Copy endpoint URL in NVP format.
18. **Connect to messenger** [Optional] [Action]: Automatically configures your Process to receive messages from messengers (Telegram, Viber, Apple Business Messages, Line (partial support), and Facebook and Google Messengers).

Enable secure authorization

Now, you can enhance security for the **Start** node in a Process by enabling basic authentication, ensuring the username and password protection for receiving requests via Direct URL in user companies. To do so:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

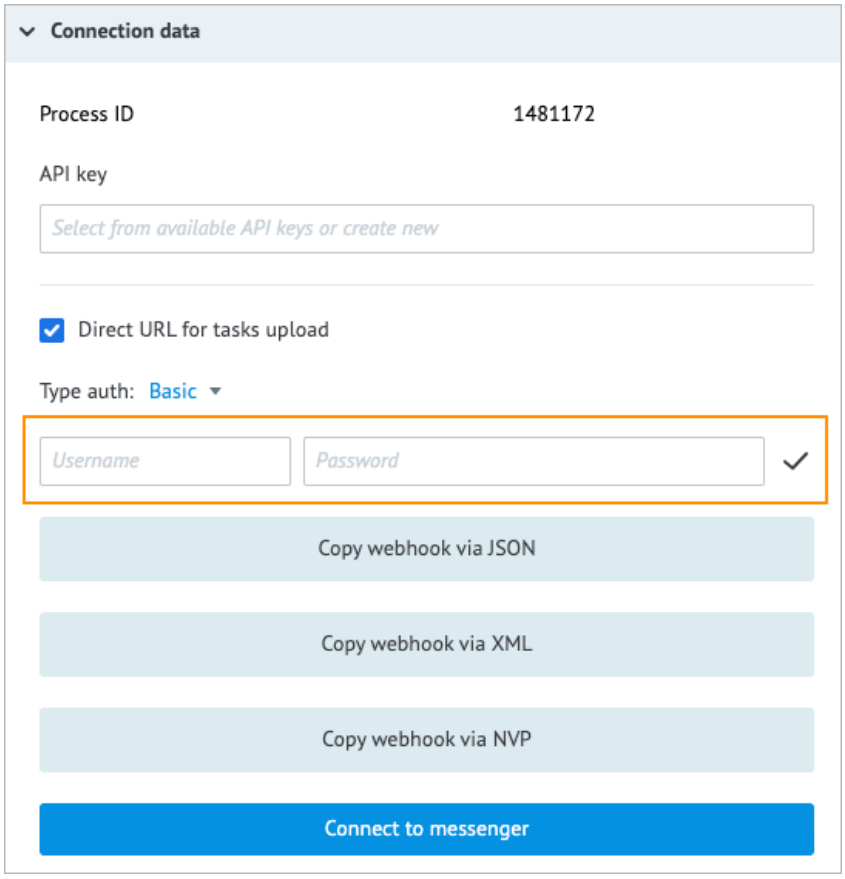
1. In the **Start** node settings, select **Basic** in the **Type auth** dropdown list.



Screenshot 152: **Authorization type selection.**

2. Enter the username and password in the corresponding fields and select the checkbox to save your authentication.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 153: **Specifying username and password.**

Now, your node webhook is protected, and only the requests containing authorization credentials will be processed.

Username and password credentials are fully displayed in the **Start** node settings in the **Edit** mode and on the **Webhook** tab of the **View** details menu for a Process. To conceal basic authentication credentials, use variables with credentials. For example, you can use the **webhook-var** variable with the following constructions:

- {{env_var[@webhook-var].user}} for the **Username** field
- {{env_var[@webhook-var].pass}} for the **Password** field

Where:
- {{env_var[]}}: construction containing the @webhook-var variable
- [@webhook-var].parameter: variable in the construction with Username or Password parameter.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

For more information on how to create a variable, go to Variable.

Important:

- If you change settings, and select **No Auth** in the **Type auth** dropdown list, you will turn off the authorization and will have to enter credentials next time.
- All the requests that don't contain authentication will not be processed, and the **Access denied** error will appear in the task.
- If you have an alias linked to a Process in which you've activated the basic authorization, sending data to the alias' Direct URL also requires sending authorization credentials. Sending requests to the alias' Direct URL with no credentials will result in the Access denied error.

Examples

Add tasks manually

This is the fastest and easiest way to create a new task in Corezoid.

Screenshot 154: **Manual task adding.**

Importing task from CSV

If you have data stored in a file that you want to turn into tasks, you can easily do so by using the **Import from CSV** option. Thus, you can quickly import large amounts of data and create tasks from it.

Screenshot 155: **Importing a task from CSV.**

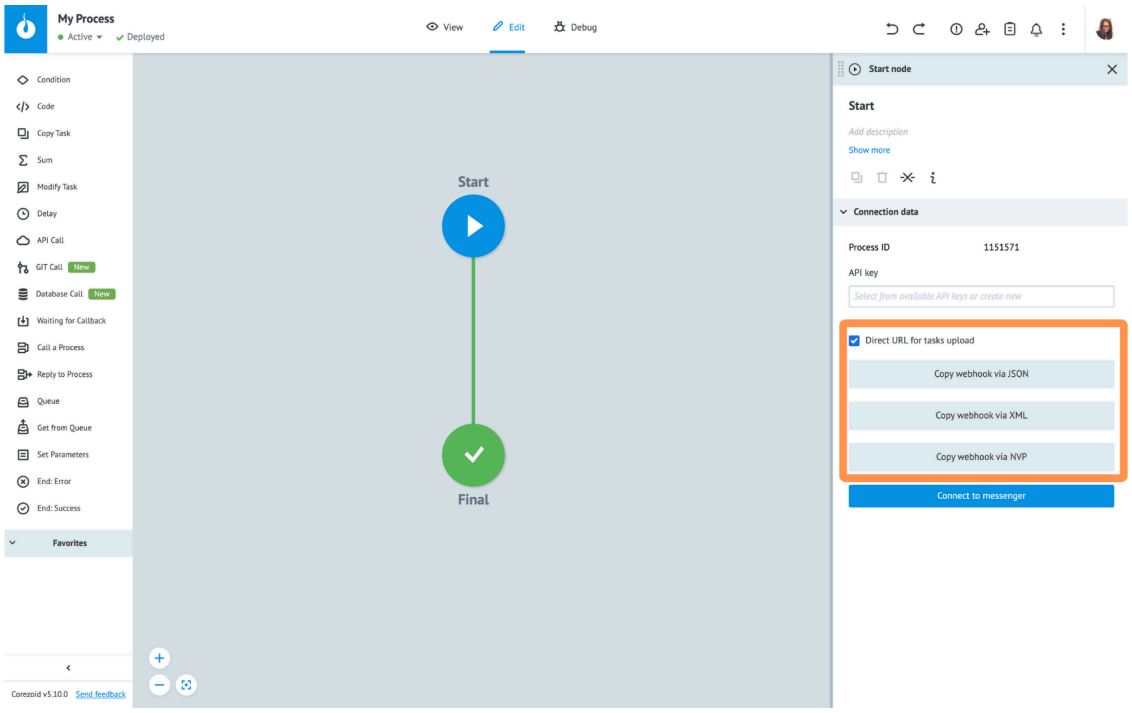
If your Process has required parameters, the CSV file must have columns for these parameters. Select any column as a **Task reference (REF)** and specify the data type for each column, such as **String** or **Number**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Sending task to Webhook

In Corezoid, every **Start** node serves as an endpoint with its unique URL. This endpoint URL can be used to subscribe to webhooks, and you can receive external data and trigger your Corezoid Processes automatically.
The endpoint URL is generated and copied to your clipboard:

- **JSON**
  `https://www.corezoid.com/api/1/json/public/0060000/*8b568f9c430f045b0c207ff8221`
- **XML**
  `https://www.corezoid.com/api/1/xml/public/0060000/*8b568f9c430f045b0c207ff8221`
- **NVP**
  `https://www.corezoid.com/api/1/nvp/public/0060000/*8b568f9c430f045b0c207ff8221`



Screenshot 156: **Sending a task to a webhook.**

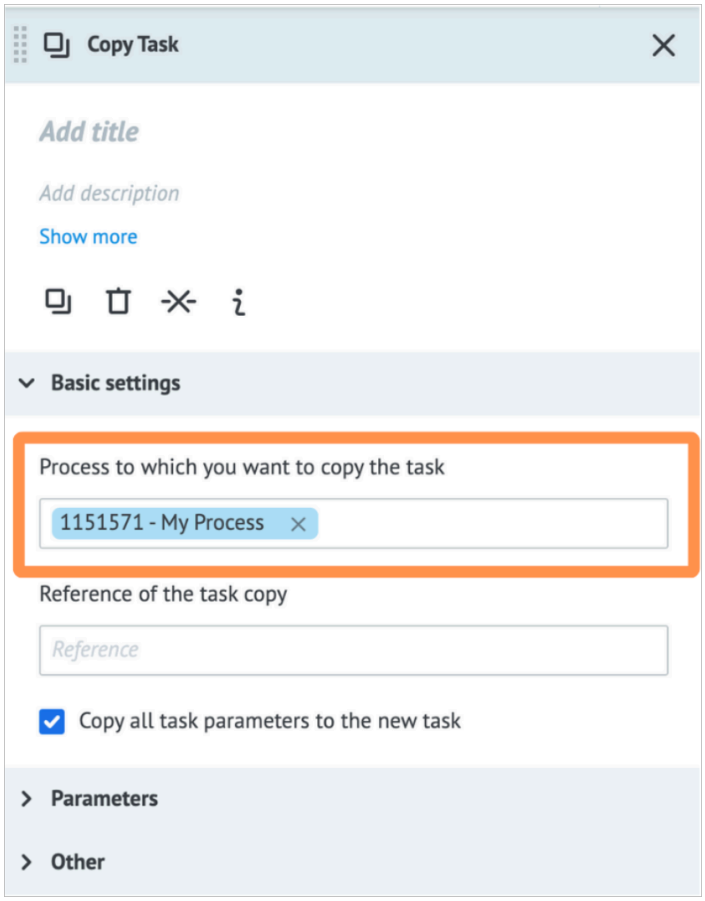
The endpoint URLs generated by Corezoid are identical except for their payload format.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Sending task from another Process

In Corezoid, it's possible to send tasks from one Process to another by using the **Copy Task** or **Call a Process** nodes:

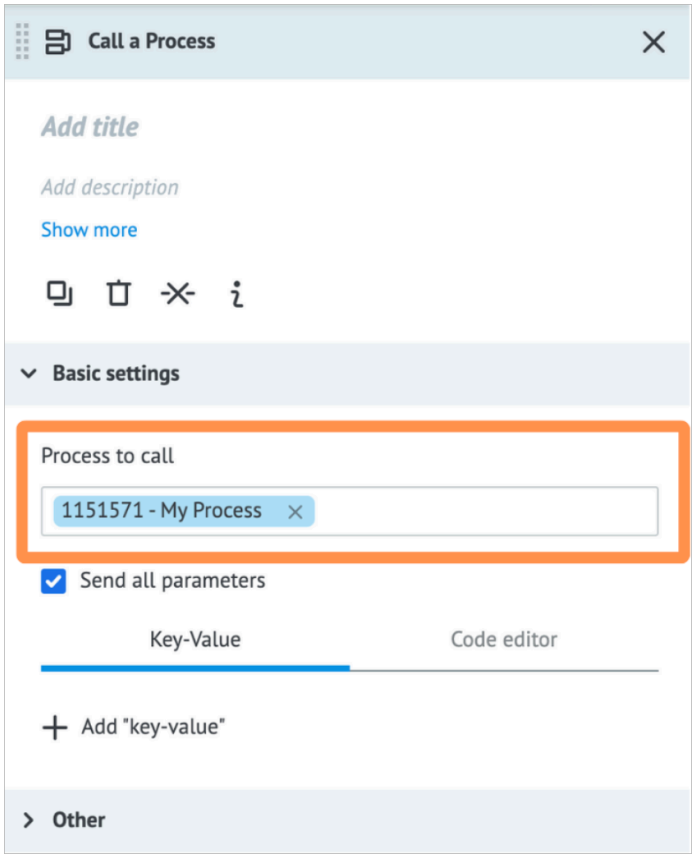
- With the **Copy Task** node, you can duplicate a task and send it to a different Process:



Screenshot 157: **Duplicating a task to a different Process.**

- With the **Call a Process** node, you can call another Process, like a function in programming

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 158: **Calling another Process.**

Corezoid API

With the Corezoid API (asynchronous), you can easily send a task to the **Start** from any API source connected.

Error handling & troubleshooting

When working with your Process, you may encounter the following issues.

Issue	Cause	Solution
Process is not receiving tasks.	Process is in the **Paused** mode.	Switch Process to **Active**.
	Process is in the **Debug** mode.	Switch Process to **Active**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

	The **Direct URL for task upload** is cleared.	Select the checkbox and update webhooks with a new URL.
	RPS limit has been exceeded.	Check **Activity Monitor**.

Table 2: **Start node issues**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3.2 End

Overview

Screenshot 159: **End node options view: error and success.**

The **End** node marks the final point in a Process or State Diagram. Key characteristics:

- Multiple End nodes allowed per workflow
- No outgoing connections
- Automatically added when creating a Process
- Shows task counter in View mode

Status indicators:

- Green: Successful task completion
- Red: Task processing error

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

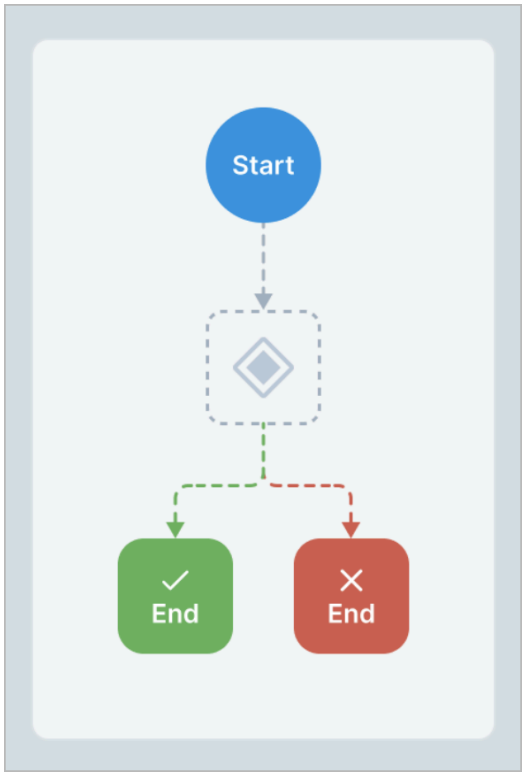
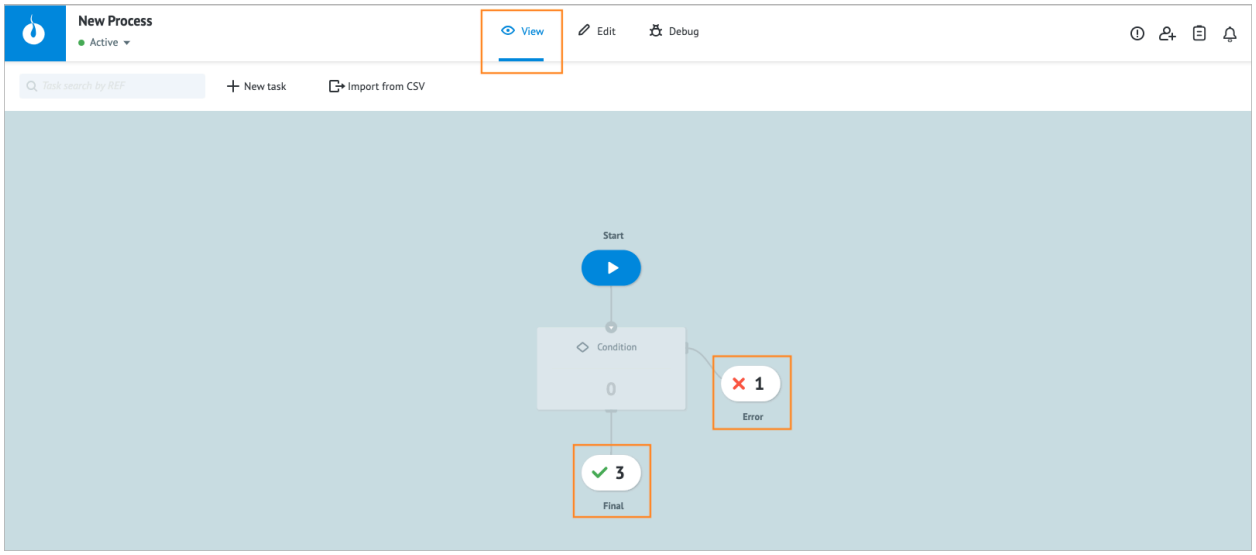


Diagram 6: **End node flow.**

In the **View** mode, each **End** node has a counter that displays the number of tasks that have reached the node.

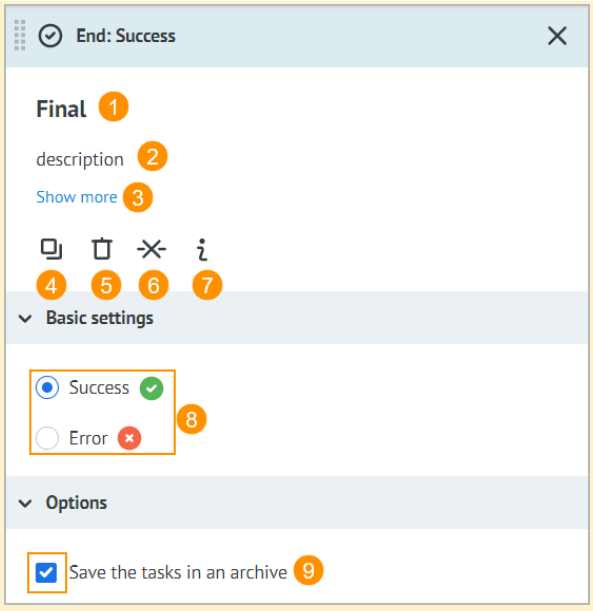


Screenshot 160: **Task counters.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Node Settings

The **End** node has the following parameters:



Screenshot 161: **End node settings.**

1. **Title** [Optional] [Field] [String]: Node title.
2. **Description** [Optional] [Field] [String]: Node description.
3. **Show more** [Optional] [Action]: Long descriptions are reduced to a compact view to simplify node settings. Click to see the whole description.
4. **Copy node** [Optional] [Action]: Creates a copy of the current node with all settings.
5. **Remove node** [Optional] [Action]: Deletes current node from Process.
6. **Remove links** [Optional] [Action]: Removes all connections to/from current node.
7. **Info** [Optional] [Action]: Displays node ID.
8. **Success/Error** [Required] [Action]:

- **Success**: Indicates that the task processing was successful with the color green.
- **Error**: Indicates that there was an error during the task processing with the color red.

9. **Save the tasks in an archive** [Optional] [Action]: The checkbox is selected by default and indicates that the task will be saved in an archive until the end of the current month. Thus, you can view and manage processed tasks (debugging and short-term monitoring purposes). If you clear the checkbox, the task will be deleted after processing.

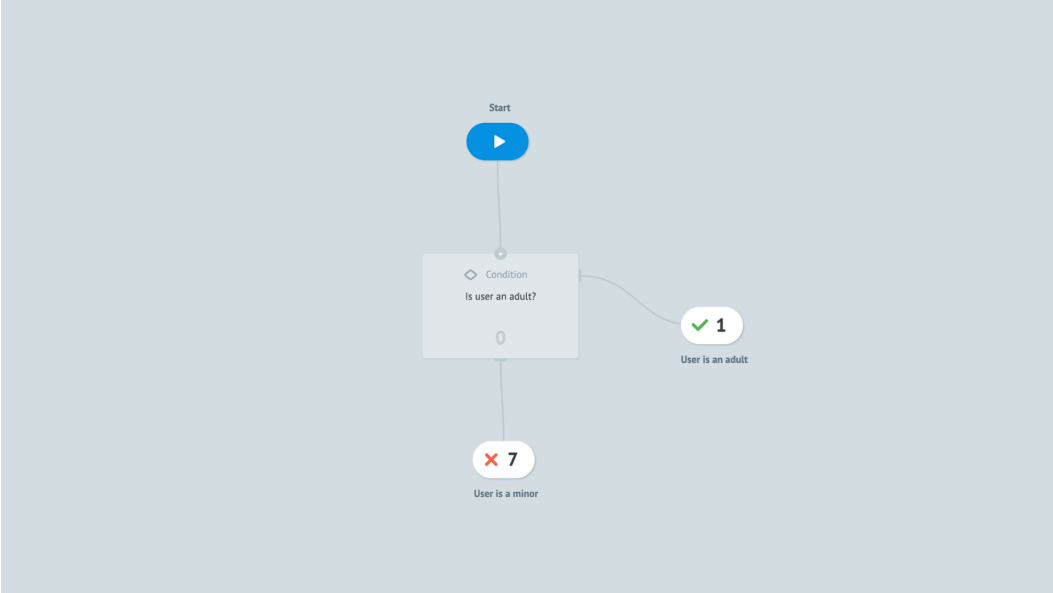
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

On the first day of each month, all the archives are cleared, thus all the tasks are deleted regardless of whether the checkbox Save the tasks in an archive is selected.

Examples

Moving tasks to End node to get statistics

In the example below, you can see the segmentation of the users by age, representing the distribution of users across various age categories.



Screenshot 162: **Distribution of tasks between End nodes.**

Using End node to break out of infinite loops

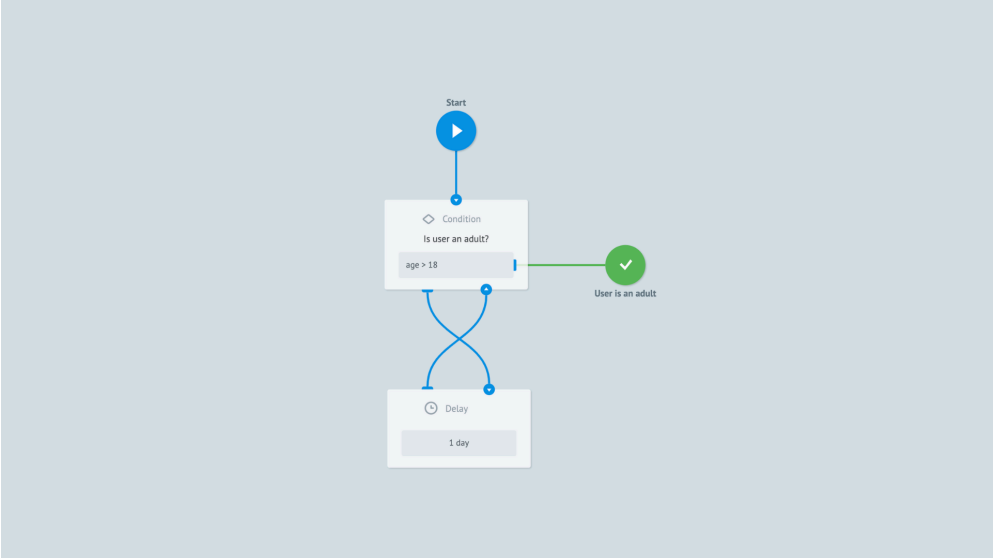
In Corezoid, an infinite loop occurs when a task continuously cycles through a sequence of nodes without reaching an End node. This can happen if the Process logic doesn't include conditions or exit points that allow the task to terminate:

1. **Task enters a node.** The task, a unit of data in JSON format, enters a node in the Corezoid Process.
2. **Node execution.** The node performs its designated function, which could involve data manipulation, API calls, or conditional checks.
3. **Transition.** Based on the node's configuration and the task's parameters, the task transitions to the next node in the Process.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

4. **Loop formation.** If the Process logic directs the task back to a previously visited node without encountering an End node, an infinite loop is formed.
5. **Continuous cycling.** The task continues to cycle through the same sequence of nodes repeatedly, consuming resources and preventing the Process from completing.

In the example below, you can see the End node usage for breaking out of infinite loops.



Screenshot 163: **Task looping prevention using the End node.**

Error handling & troubleshooting

When working with your Process, you may encounter the following issues.

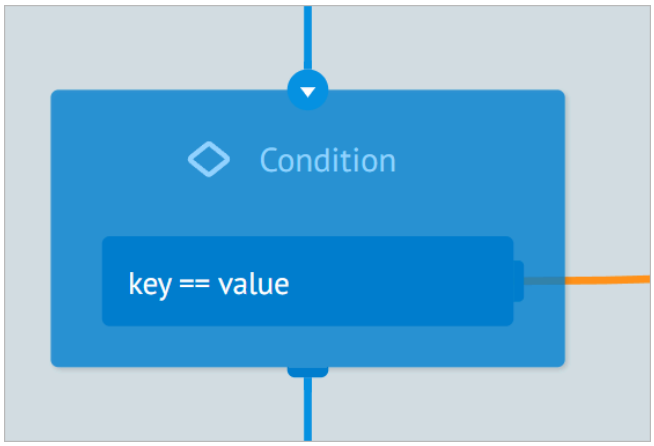
Issue	Cause	Solution
Tasks don't reach the **End** node.	The **Save the tasks in an archive** checkbox is cleared.	Select the box.
	Archive was cleared.	Store tasks in the **Set State** nodes.

Table 3: **End node issues**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3.3 Condition

Overview



Screenshot 164: **Condition node view**

The **Condition** node works as an "if" statement and enables you to evaluate task parameters and direct the flow of your tasks based on specific requirements set by the user. The **Condition** node may consist of one or more **Condition blocks**; each block includes one or more conditions and a single output edge, which leads to the End node. By using multiple **Condition blocks**, you can create a nested if/switch-case structure and achieve more flexibility in controlling the task flow based on various conditions.

When a task enters the **Condition** node, it is evaluated against each condition in the topmost **Condition block**:

- If the task satisfies every condition in the block, it goes to the **End** node of the block.
- If the task doesn’t satisfy at least one condition in the block, it goes to the next **Condition block**.
- If the task doesn’t satisfy all the conditions of any **Condition block**, it goes to the default **End** node.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

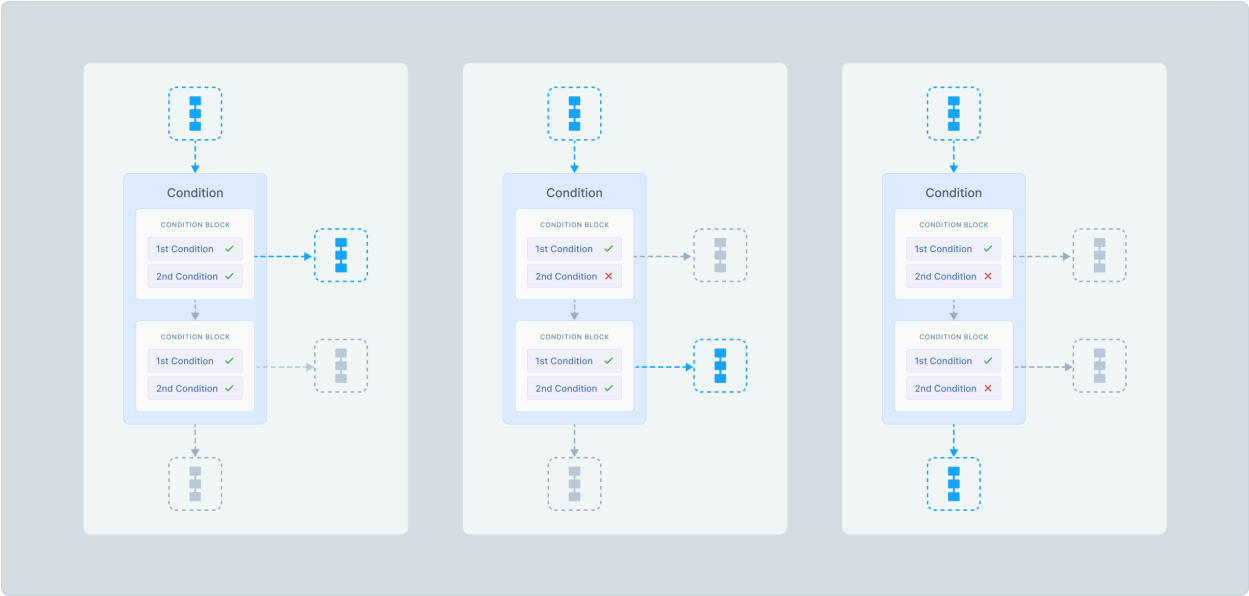


Diagram 7: **Condition node flow.**

Task Flow

1. Task enters Condition node
2. Evaluation starts with topmost Condition Block
3. Two possible outcomes:
    - **Success**: Task matches all conditions → proceeds to block's End node
    - **Failure**: Task fails any condition → moves to next block

Note: If task fails all blocks, it routes to default End node.

Example Structure

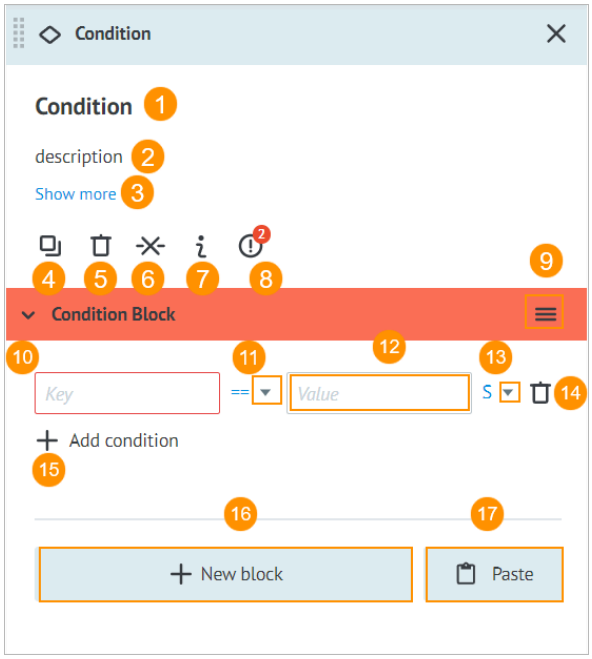
```
Condition Node
├── Block 1: if (condition A AND condition B)
│   └── End Node 1
├── Block 2: if (condition C)
│   └── End Node 2
└── Default End Node (no conditions met)
```

Schema 1: **Condition node structure**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Node Settings

The **Condition** node has the following parameters:



Screenshot 165: **Condition node settings.**

1. **Title** [Optional] [Field] [String]**:** Node title.
2. **Description** [Optional] [Field] [String]: Node description**.**
3. **Show more** [Optional] [Action]: Long descriptions are reduced to a compact view to simplify node settings. Click to see the whole description.
4. **Copy node** [Optional] [Action]: Creates a copy of the current node with all settings**.**
5. **Remove node** [Optional] [Action]: Deletes current node from Process.
6. **Remove links** [Optional] [Action]: Removes all connections to/from current node.
7. **Info** [Optional] [Action]: Displays node ID.
8. **Errors info** [Optional] [Action]: Displays node errors.
9. **Drag Condition Block** [Optional] [Action]: point the mouse cursor on the icon and hold down the left mouse button to drag the Condition Block up or down. Thus, you can change the arrangement of Condition Blocks in the node.
10. **Key** [Required] [Field] [String]: Key to specify a static name or a variable in {{}} curly brackets.
11. **Comparison operator** [Required] [Action]: The way the key and the value have to be compared:
    - Equal to (==)
    - Not equal (!=)

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- More than (>)
- More than or equal to (>=)
- Less than (<)
- Less than or equal to(<=)
- Regular Expression (RegExp)).

12. **Value** [Required] [Field] [Any]: Value to compare. Data type depends on the **Type of data** selected. See data types descrition for more details.

Note: For more information, go to Parameters, Functions.

13. **Type of data** [Required] [Action]: Selects the data type for the **Value**:
* String (**S**)
* Number (**N**)
* Boolean (**B**)
* Array (**A**)
* Object (**O**)

Note: For more information, go to Data types.

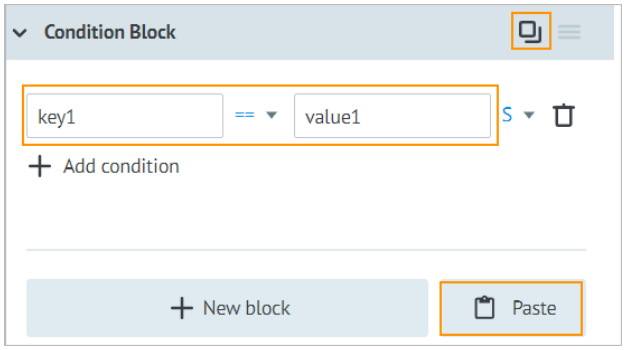
14. **Delete condition** [Optional] [Action]: delete the condition line from the Condition Block.

15. **Add condition** [Optional] [Action]: add a new condition line to the block.

16. **New block** [Optional] [Action]: add a new Condition Block to the node.

17. **Paste** [Optional] [Action]: paste the copied condition block. For at least one condition (key and value) specified in the block when pointing the cursor over the Drag Condition Block icon, the copy icon appears to the left. You can click this icon to copy the Condition Block and paste it to another one using the **Paste** button:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



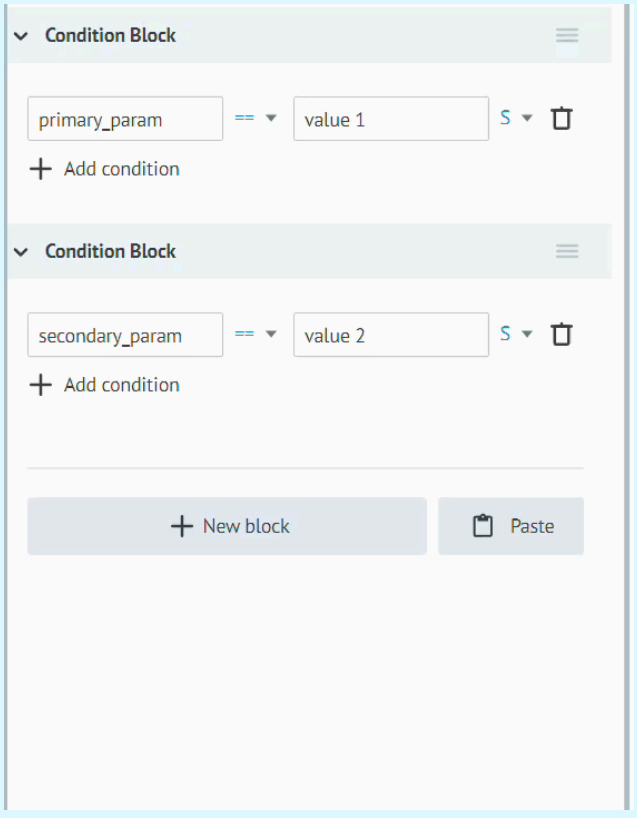
Screenshot 166: **Condition block copy and paste.**

Actions available in the Condition Block

You can do the following actions in the **Condition Block**:

- Add a new condition: Below the **Key** field, click **+ Add condition**, and then fill in the needed information in the condition line that appears.
- Delete a condition: On the right side of the condition line, click the trash icon
- Add a new **Condition Block**: Below the condition line, click **+ New block**, and then fill in the needed information in the block that opens.
- Copy a **Condition Block**: On the right side of the **Condition Block**, hover over the red/gray line, and then click the copy icon
- Paste a **Condition Block**: In the lower-right corner of the panel, click **Paste**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



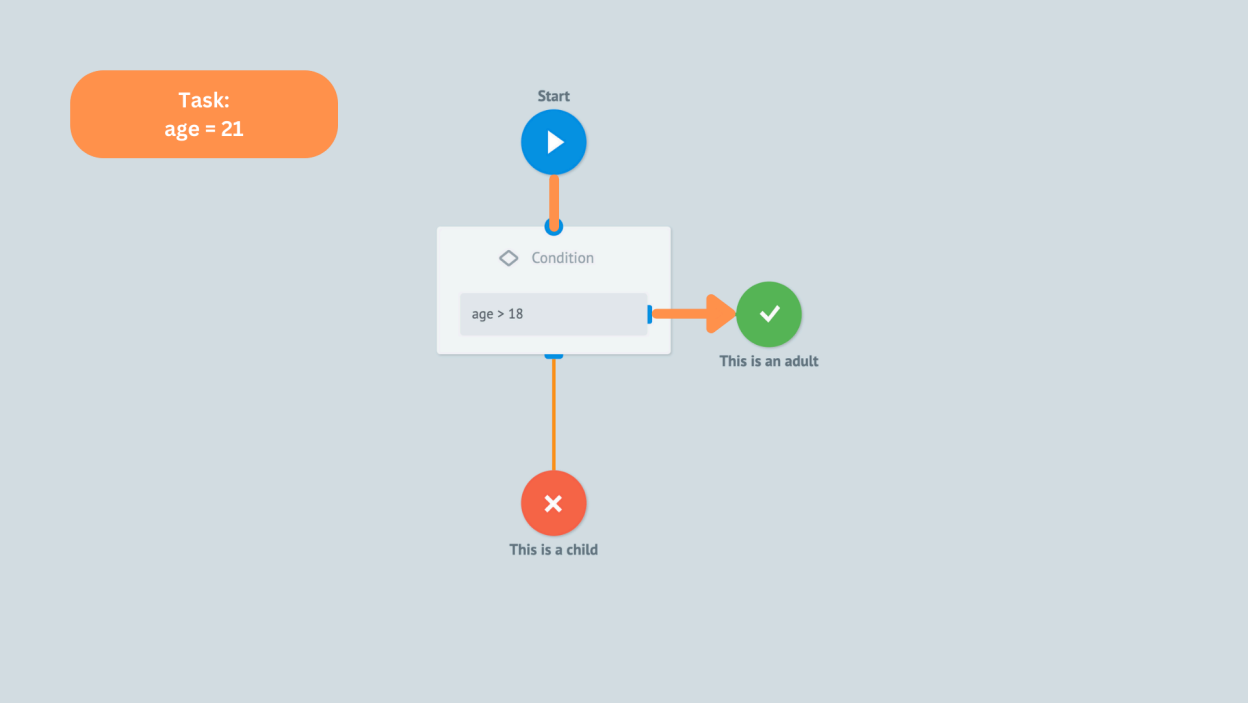
Screenshot 167: **Condition block operations.**

Examples

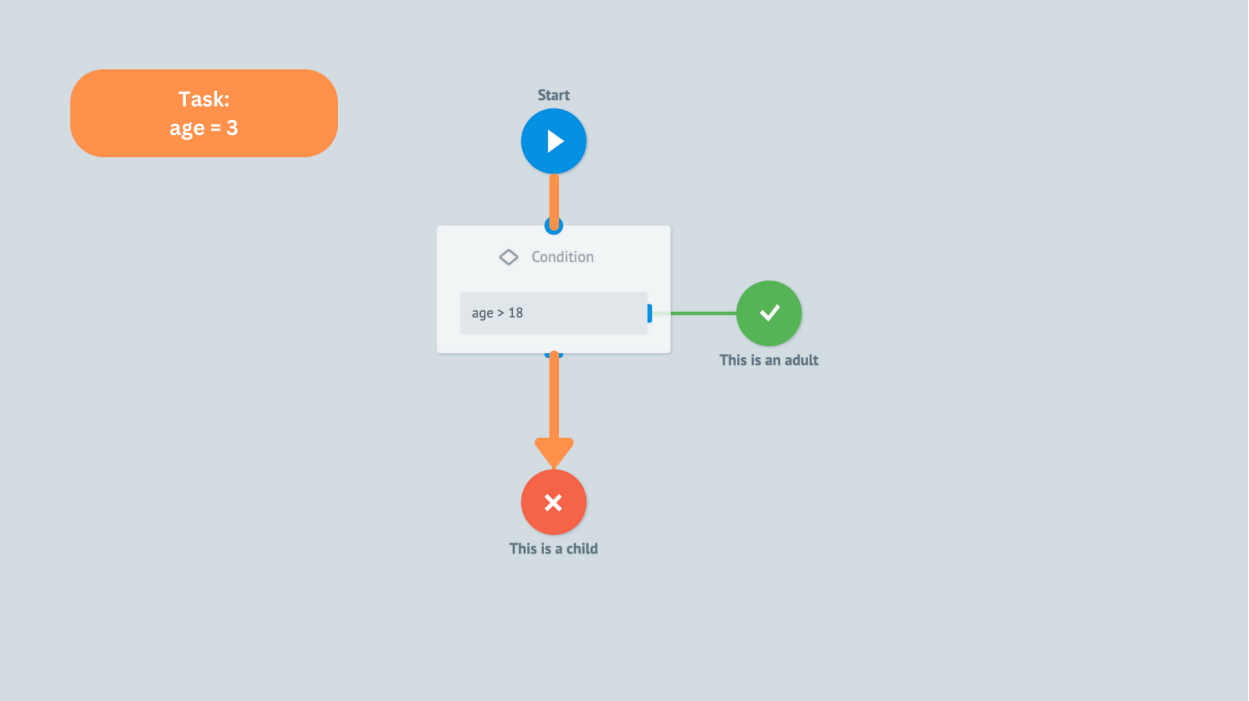
Adding one Condition block and one condition

In the examples below, we check if the condition is met: whether the value of the age parameter in the task is above 18:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 168: **Condition is met.**



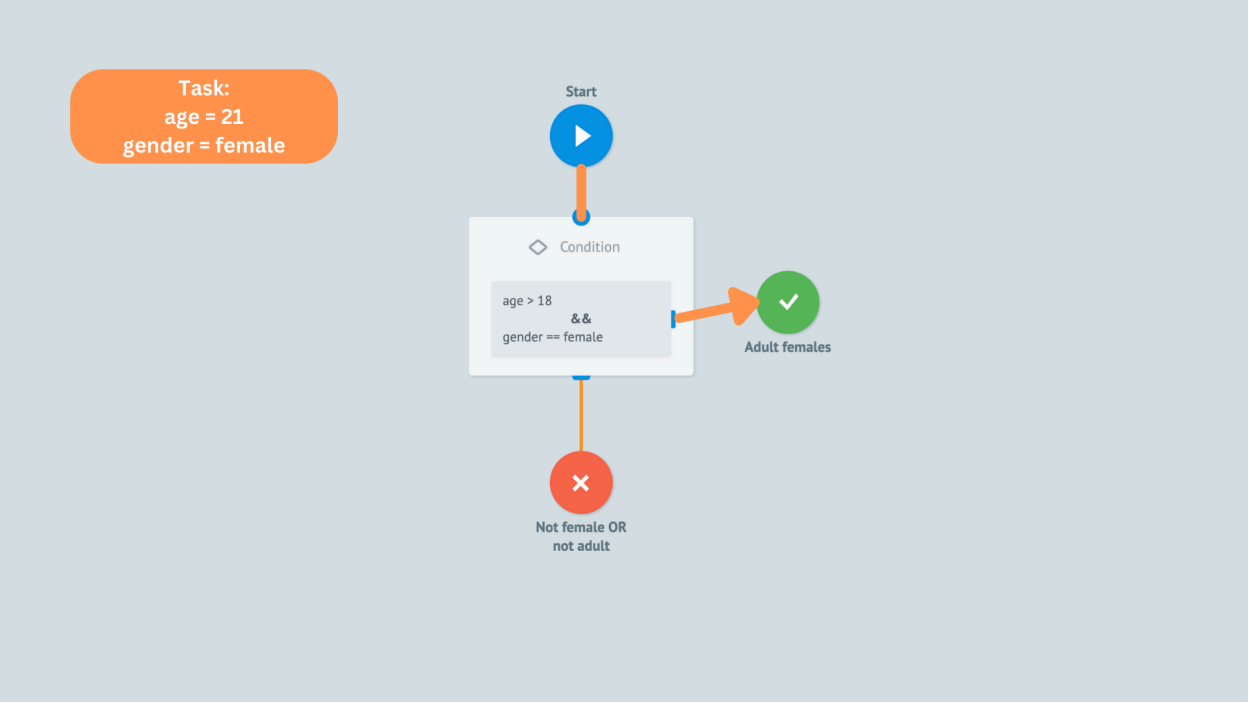
Screenshot 169: **Condition is not met.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

To pass a condition check, the task's parameters must meet the condition: if a task does not contain the age parameter, it fails to meet the condition.

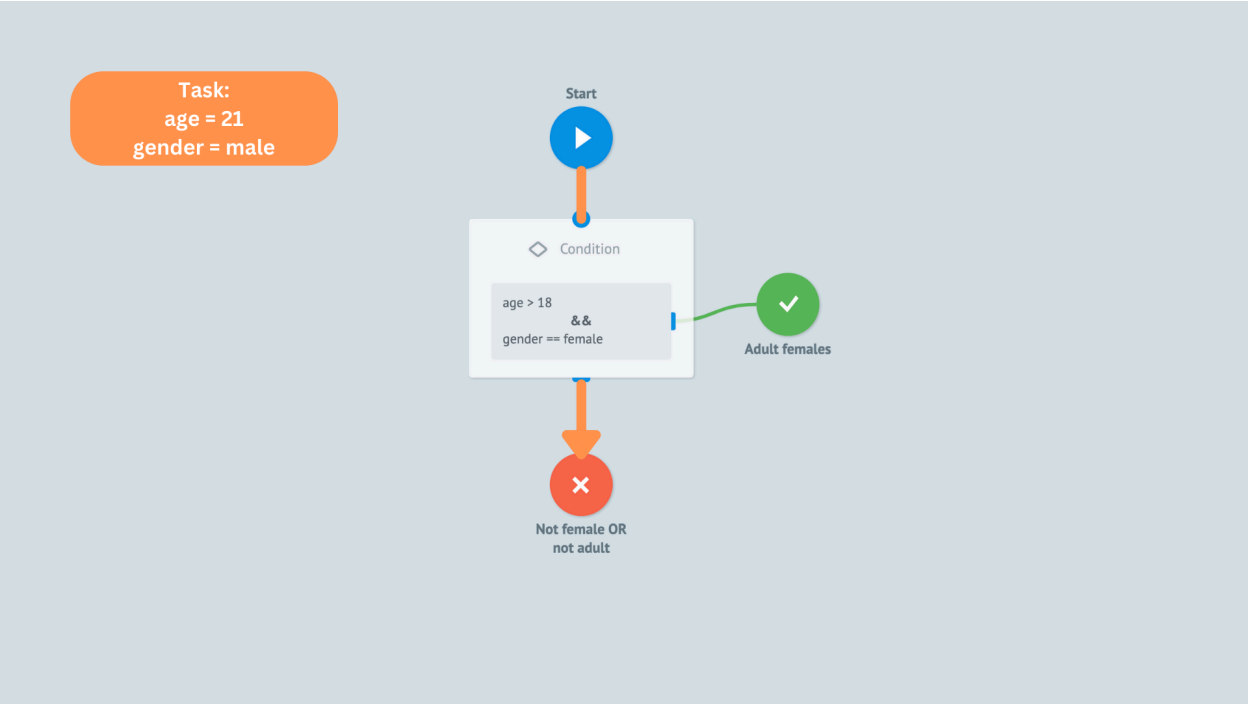
Adding one Condition block with two Conditions

In the examples below, we check whether the two conditions of the **Condition block** are met: whether the value of the **age** parameter in the task is above 18, and the value of the **gender** parameter is female:



Screenshot 170: **Both conditions are met.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



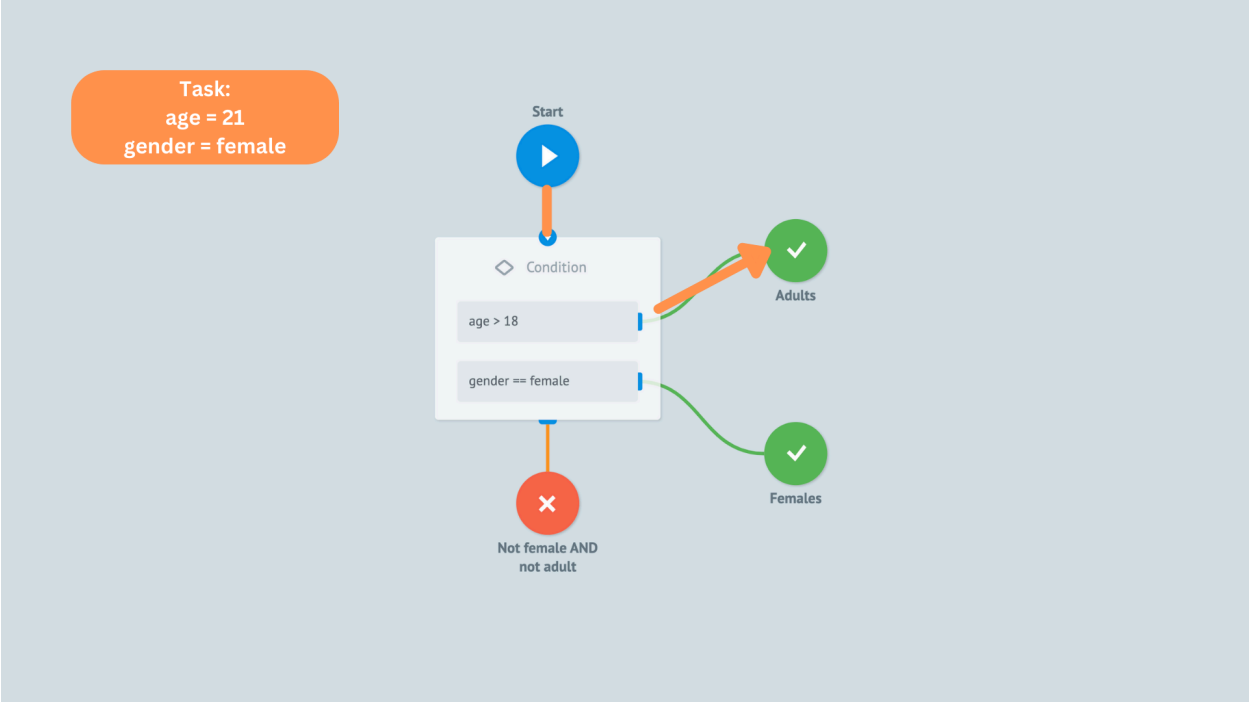
Screenshot 171: **Only one condition is met.**

If one block contains several conditions, all of them must be met for the block check to be considered true.

Adding two Condition blocks with one Condition in each

In the example below, the task's parameters are evaluated against two separate **Condition blocks**. Even though the conditions sets in both blocks are met, the task will only follow the output edge of the top block since it is the first one to be checked against.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 172: **Adding two blocks each with one condition.**

You can access a single task parameter or a whole task stored in a State Diagram by Getting Task Parameters from State Diagrams.

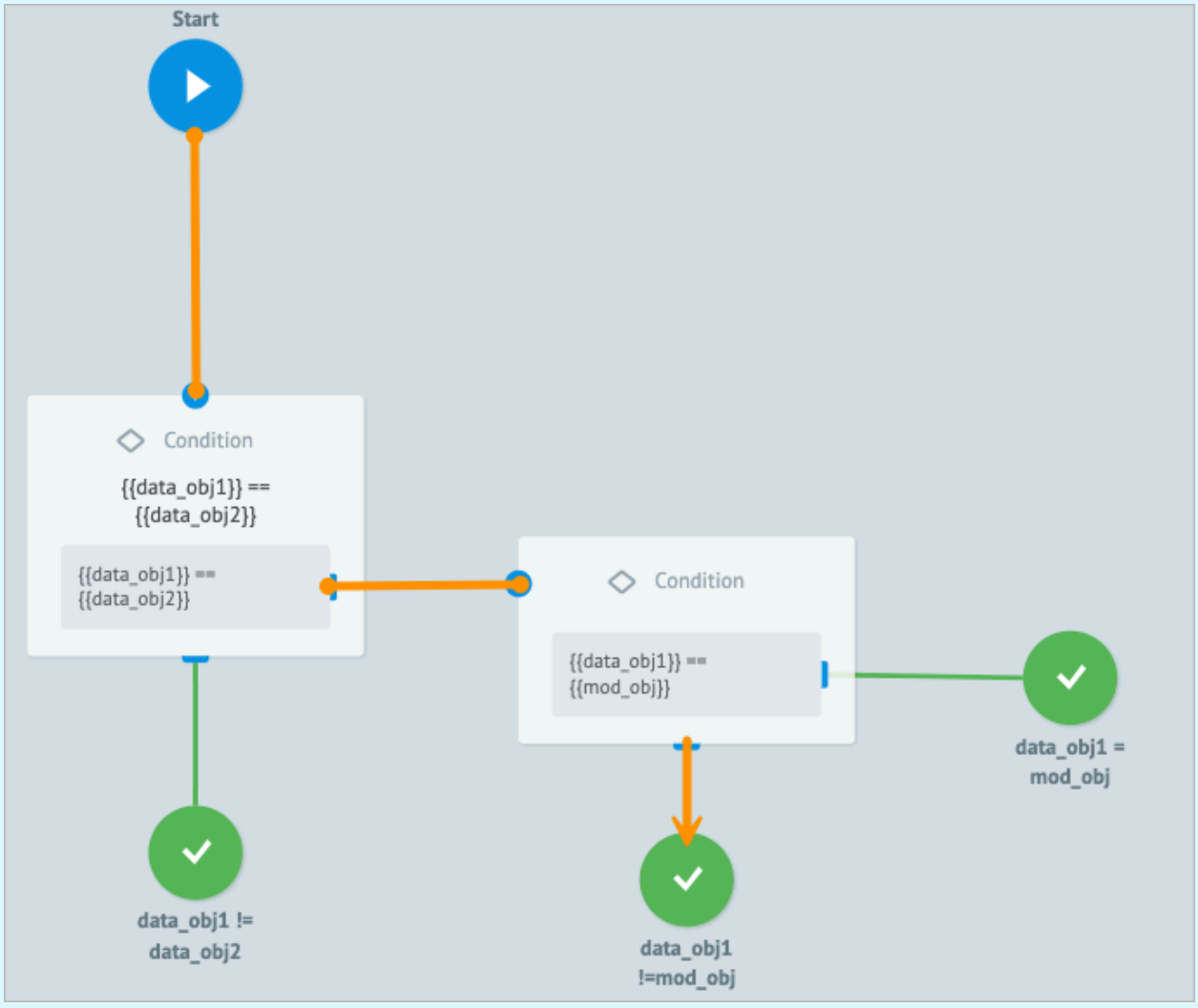
Comparing objects in Condition blocks

In the example Process below, we use two **Condition** nodes in sequence to compare object data type parameters in the following objects with nested parameters: `data_obj1`, `data_obj2` and `mod_obj`.

In-depth comparison

In **Condition** nodes, we can compare objects with as many nesting levels and parameters as we need, owing to the in-depth object comparison.

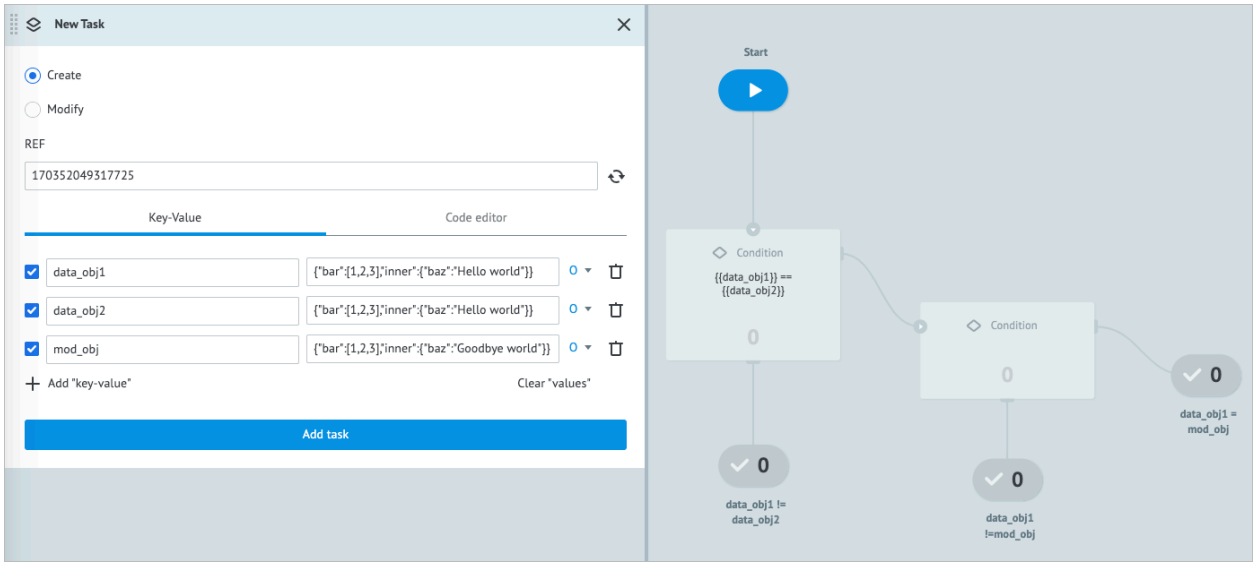
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 173: **In-depth comparison of object in the Condition node.**

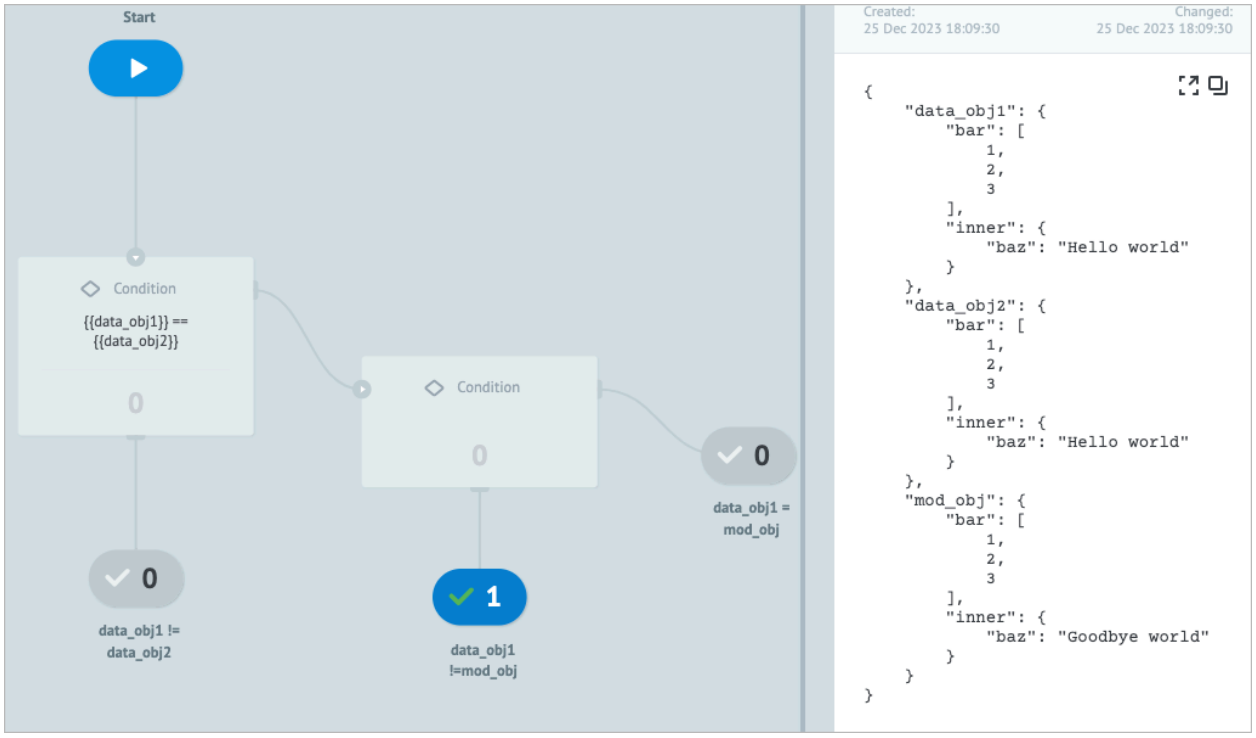
First, we define the objects and their parameters in the task.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 174: **Objects comparison: defining parameters.**

Then, we send the task to our Process. It passes through the first **Condition** node, where the check is performed for the `data_obj1` and `data_obj2` objects whether they are identical, and proceeds to the second **Condition** node. In the second **Condition** node, the `data_obj1` and `mod_obj1` are compared. As these objects are different, the task ends up in the final **data_obj1 !=mod_obj** node.



Screenshot 175: **Objects comparison: result.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Error handling & troubleshooting

When working with your Process or State Diagram, you may encounter the following issues.

Issue	Cause	Solution
The **Condition** node is not working properly.	There is a typo in the parameter name.	Parameter names are case-sensitive. For example, if the task parameter is “Phone”, but you entered it as "phone" in the Condition node, the node will not recognize them as one parameter. You have to enter task parameter names the same way, either “Phone” or "phone".
	An incorrect value type was entered.	The compared value types have to be of the same data type. For example, if the task value is "Count": 10 (Number), but the condition is specified as "Count"=10 (String), the value check will always fail. You have to use the same data type when comparing similar values.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

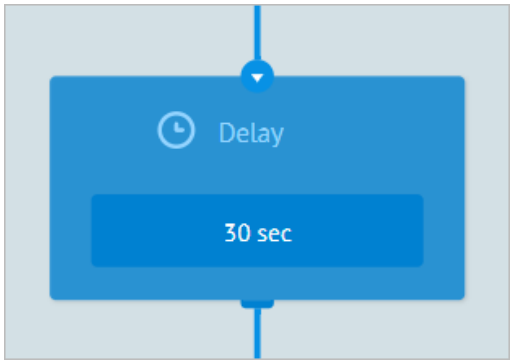
	Two variables are compared.	If the value is a variable, you must enter it in double braces. For example, the condition “amount == limit” will not work correctly, as by default limit will be considered a string. Use double braces when specifying a variable: {{limit}}.

Table 4: **Condition node issues**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3.4 Delay

Overview



Screenshot 176: **Delay node view.**

Manages workflow timing by pausing task progression.

Core Features

- Introduces specified time delays in specified time measurement units
- Schedules task execution
- Controls workflow timing

Use Cases

- Schedule task release at specific times (e.g., 7 PM)
- Add processing intervals between steps
- Automate time-dependent actions

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

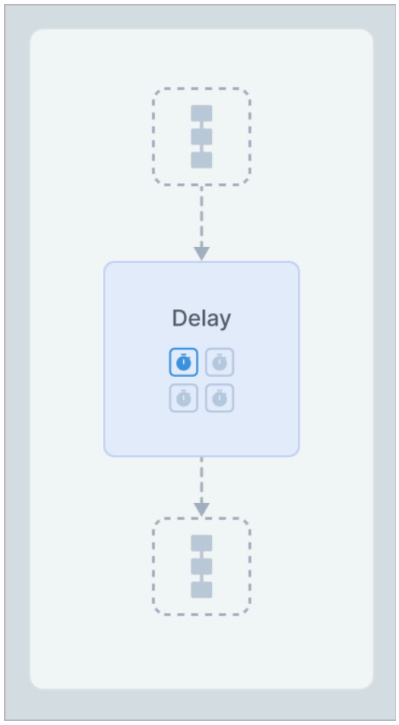
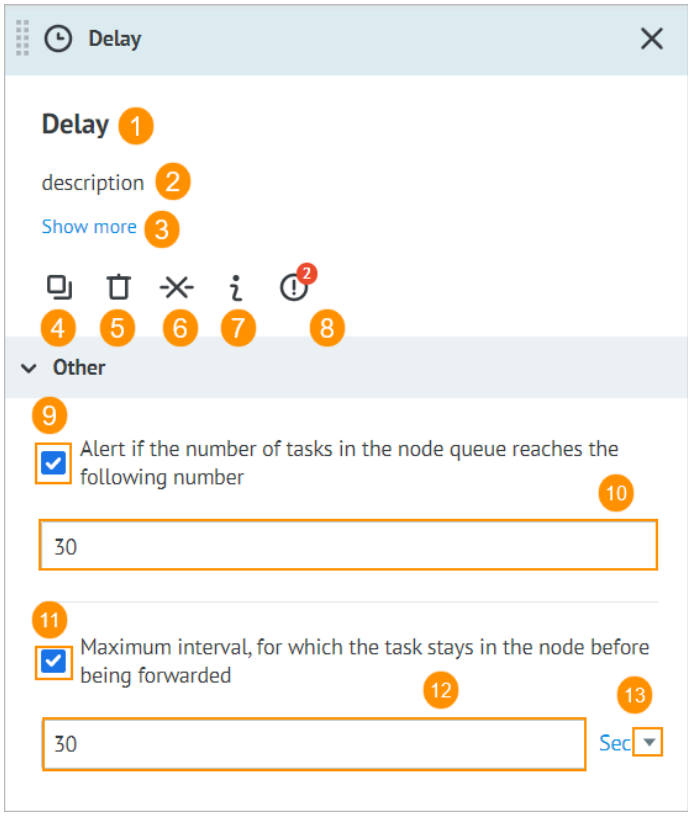


Diagram 8: **Delay node flow.**

Node Settings

The **Delay** node has the following parameters:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 177: **Delay node settings.**

1. **Title** [Optional] [Field] [String]**:** Node title.
2. **Description** [Optional] [Field] [String]: Node description**.**
3. **Show more** [Optional] [Action]: Long descriptions are reduced to a compact view to simplify node settings. Click to see the whole description.
4. **Copy node** [Optional] [Action]: Creates a copy of the current node with all settings**.**
5. **Remove node** [Optional] [Action]: Deletes current node from Process.
6. **Remove links** [Optional] [Action]: Removes all connections to/from current node.
7. **Info** [Optional] [Action]: Displays node ID.
8. **Errors info** [Optional] [Action]: Displays node errors.
9. **Alert if the number of tasks in the node queue reaches the following number** [Optional] [Action]: Helps monitor whether the number of tasks in the node exceeds the specified threshold.
10. **Number of tasks in the queue** [Optional/Required when 9 is selected] [Field] [Number]: Number of tasks in the queue which triggers an alert when reached. Set to 30 by default.
11. **Maximum interval, for which the task stays in the node before being forwarded** [Required] [Action]: The amount of time a task is allowed to be in the node can be set in seconds, minutes, hours, and days. Selected by default.
12. **Maximum interval** [Required] [Field] [Number]: The amount of time a task is allowed to be in the node. Has a minimum value of 30 seconds. You can set a shorter interval by using the **Unixtime** function.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

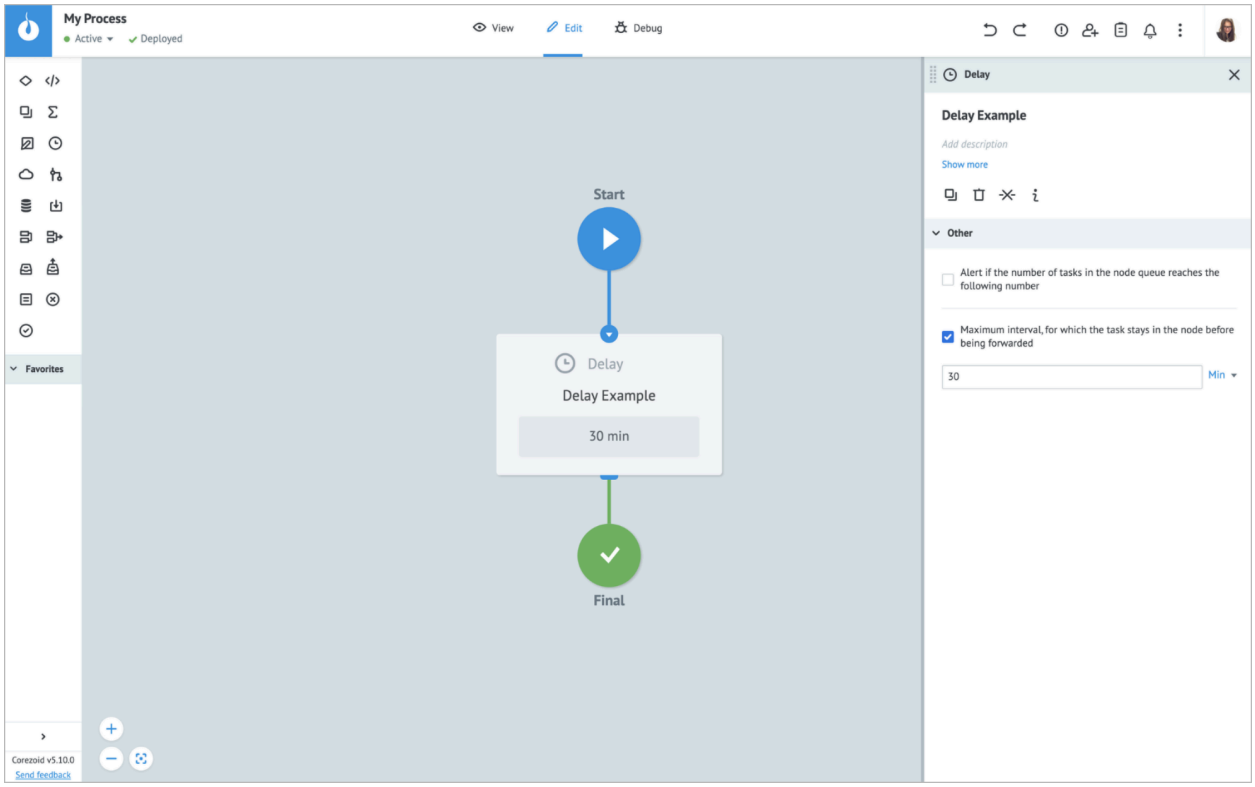
13. **Maximum interval measuring unit** [Required] [Action]: Selects the measuring unit for the maximum interval for which the task stays in the node before being forwarded:
    - Seconds
    - Minutes
    - Hours
    - Days.

Examples

Delaying a task by an x amount of time

Below, you can see the following examples:

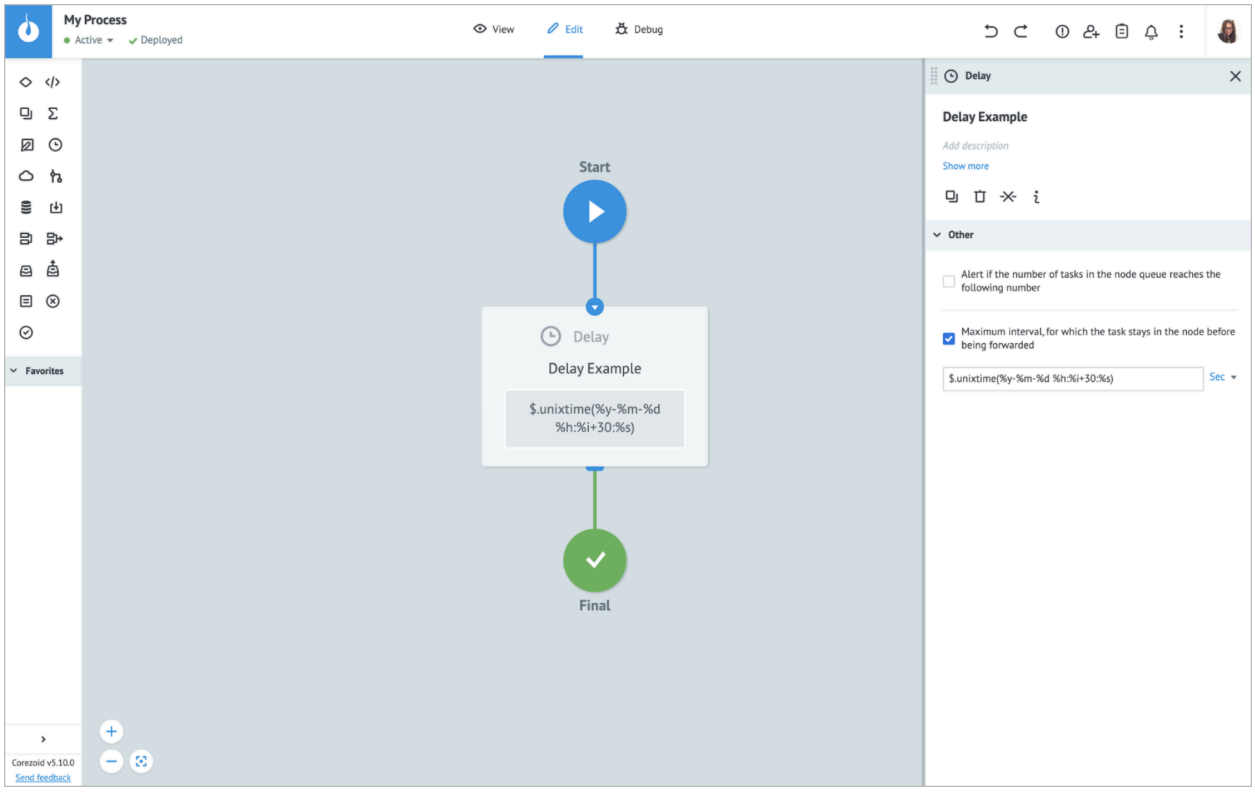
- Delaying a task for a time interval specified with an integer number. For example, you can use it in a Process for sending an email to your customers 30 minutes after they sign up for your service.



Screenshot 178: **Delaying a task for specified amount of time.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

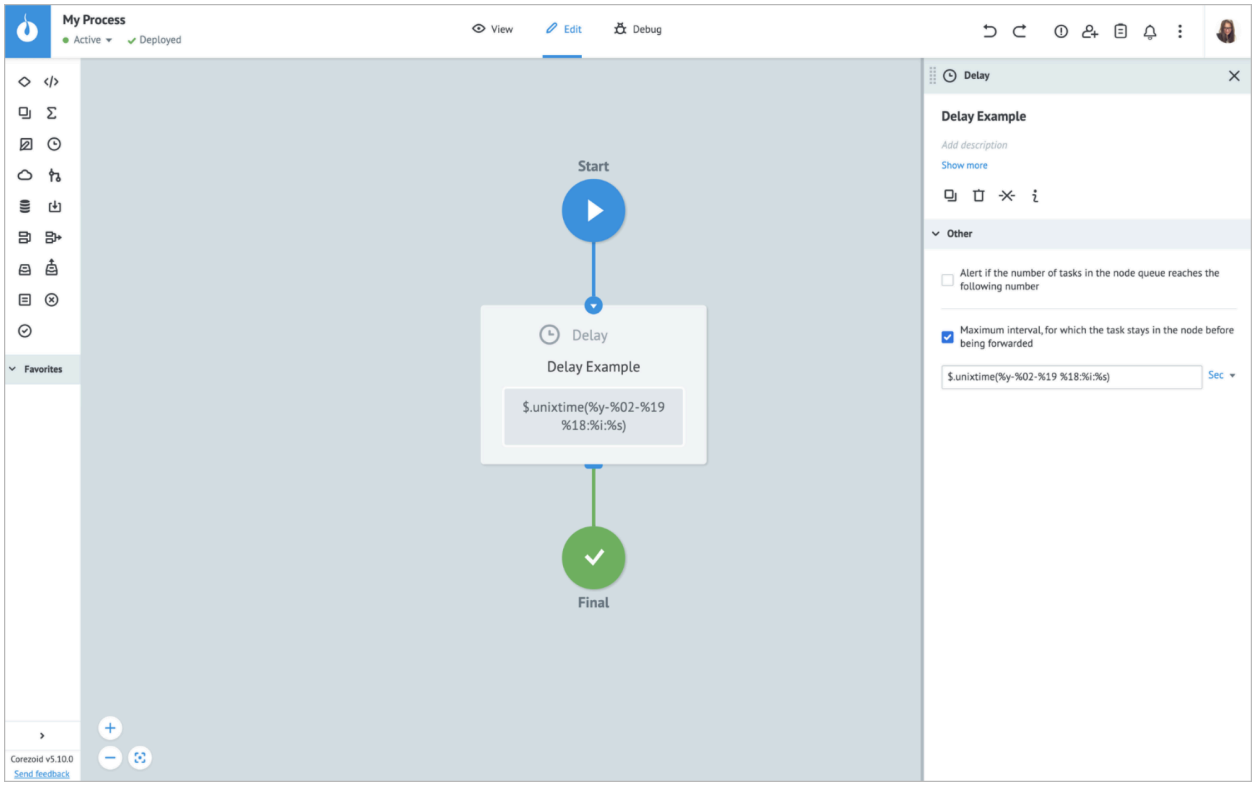
- Delaying a task for a time interval specified with Unix time: sending an email to your customers after a certain period of time after they sign up for your service.



Screenshot 179: **Delaying a task for the Unix time-specified interval.**

- Delaying a task until a time specified with Unix time: sending an email to your customers at a certain time.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



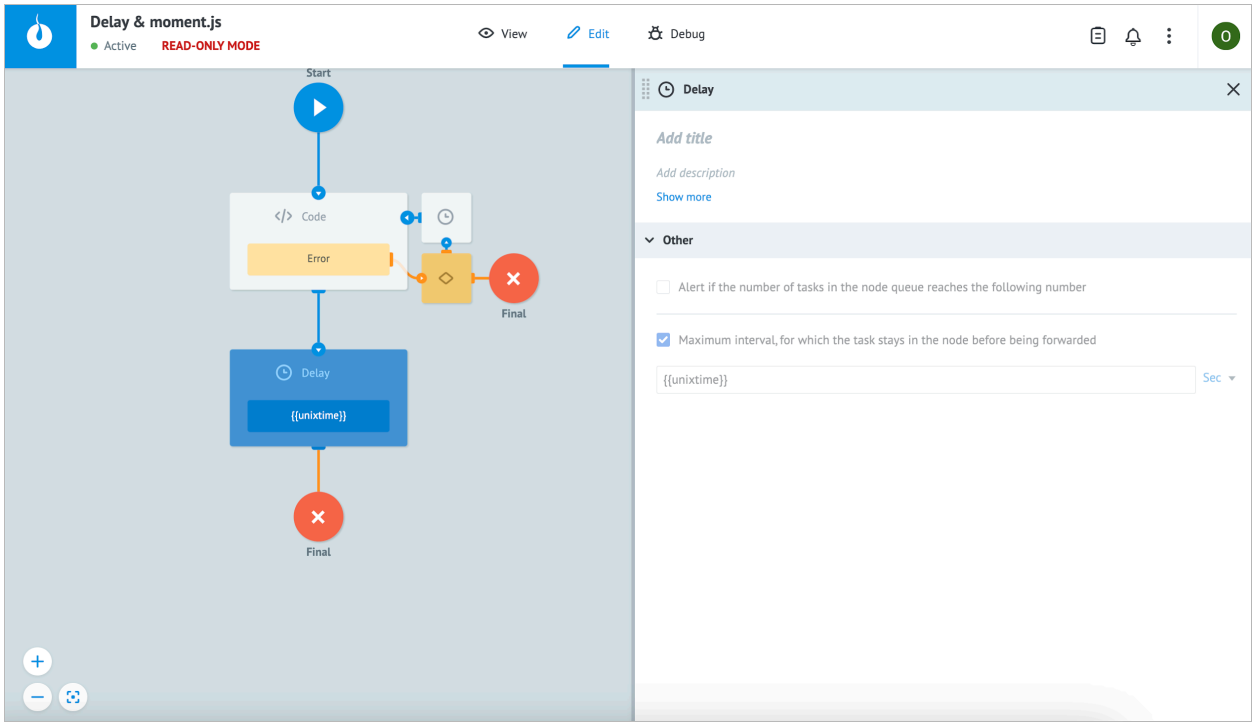
Screenshot 180: **Delaying a task until the Unix time-specified moment.**

Using dynamic timer for each task

Below, you can see the example where:

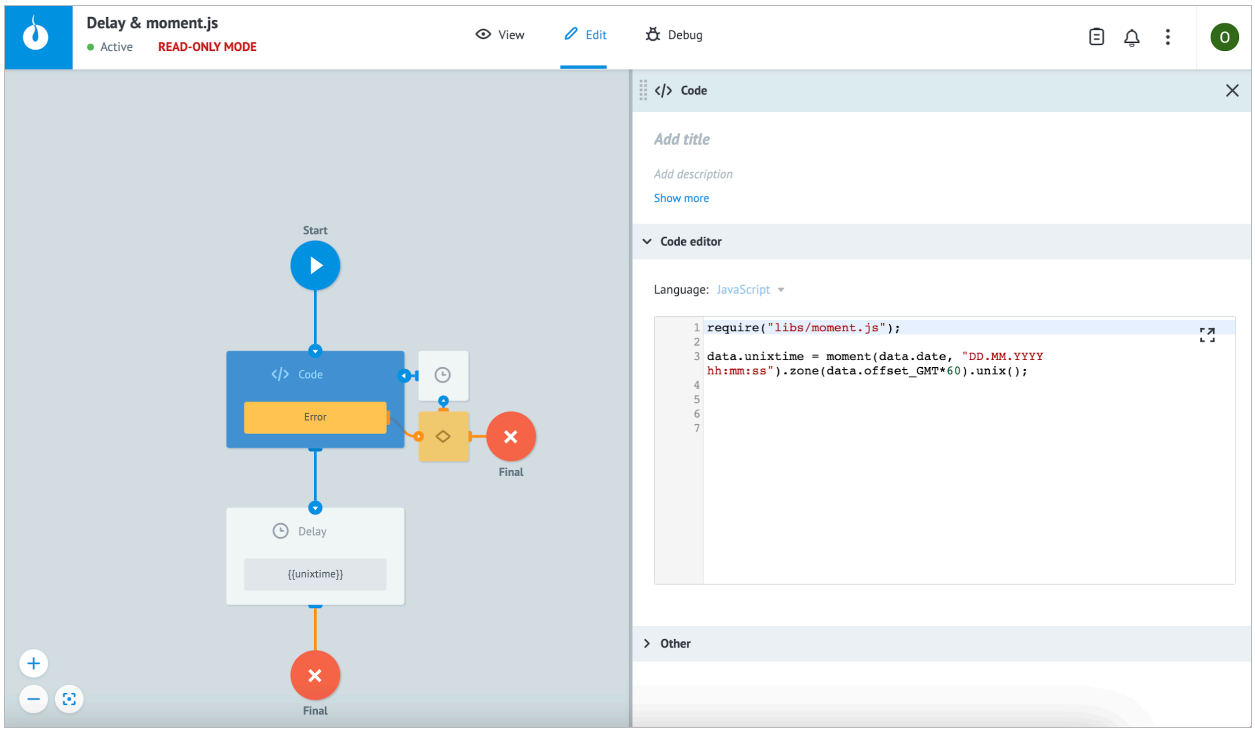
- **date** variable is converted to Unixtime to be used in the **Delay** logic (the format is DD.MM.YYYY hh:mm:ss).

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 181: **Delaying a task using the date variable converted to Unix time.**

- **offset_GMT** variable is used to adjust the Corezoid server time zone value.



Screenshot 182: **Using offset_GMT variable to adjust the Corezoid server time zone value.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Error handling & troubleshooting

When working with your Process or State Diagram, you may encounter the following issues.

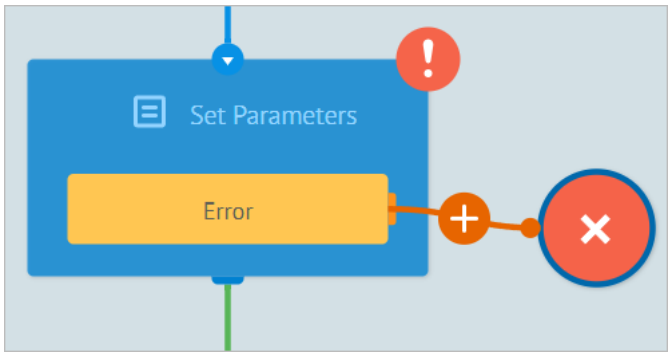
Issue	Cause	Solution
The task was not released from the **Delay** node at the expected time.	The specified Unix time is in the past.	Specify a date in the future. Make sure the specified timezone is correct.
	The Unix time formatting is incorrect.	Fix the Unix time formatting.

Table 5: **Delay node issues**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3.5 Set Parameters

Overview



Screenshot 183: **Set Parameters node view**

The **Set Parameters** node offers a simple way to work with task variables and parameters. By using this node, you can:

1. Add new parameters to a task going through the Process.
2. Update the existing task parameters.

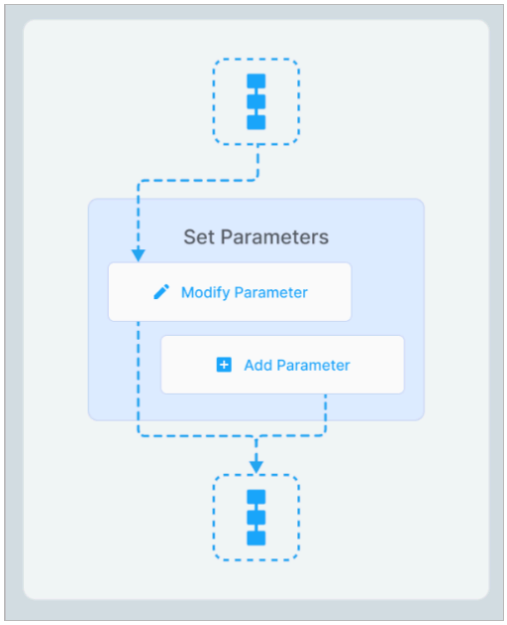
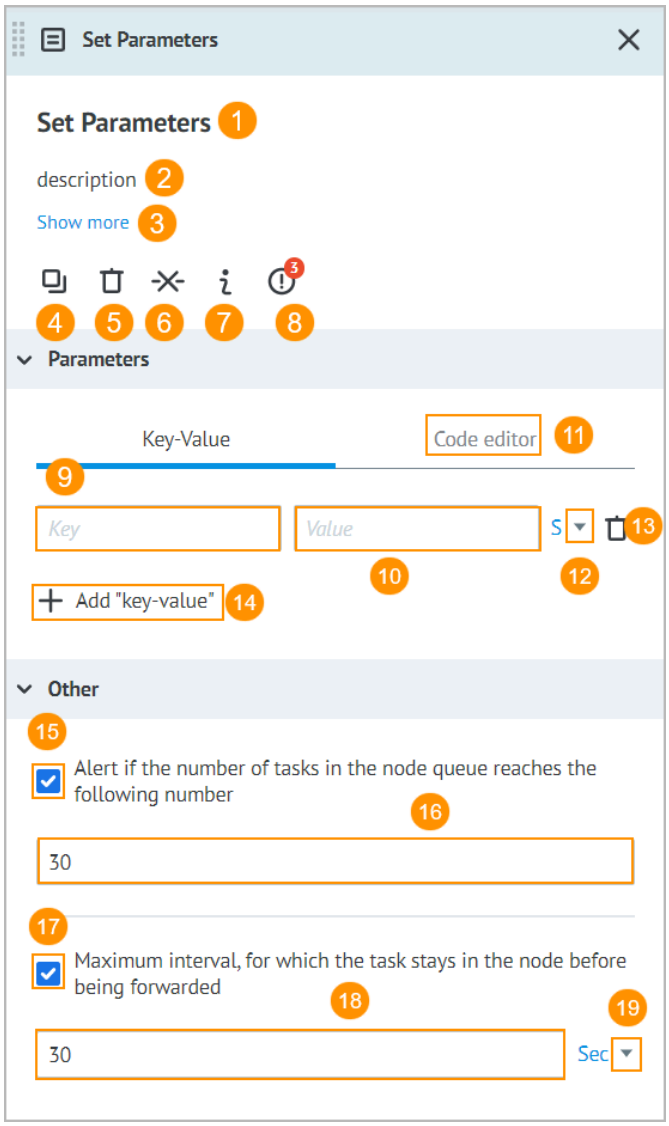


Diagram 9: **Set Parameters node flow**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Node Settings

The **Set Parameters** node has the following parameters:

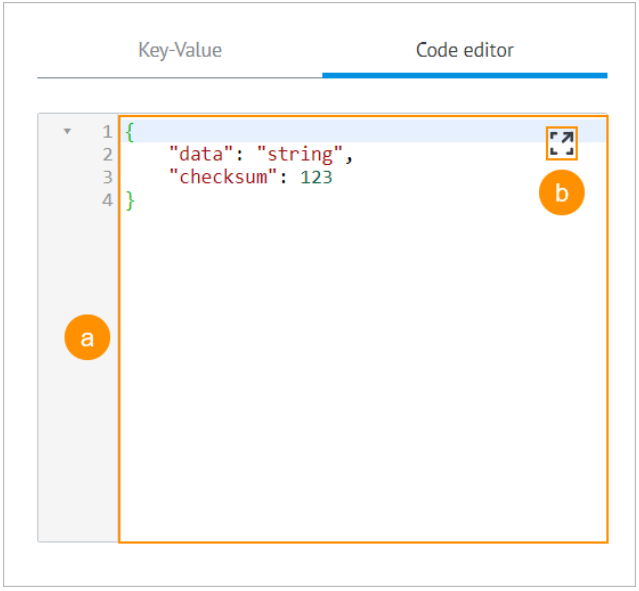


Screenshot 184: **Set Parameters node settings**.

1. **Title** [Optional] [Field] [String]**:** Node title.
2. **Description** [Optional] [Field] [String]: Node description**.**
3. **Show more** [Optional] [Action]: Long descriptions are reduced to a compact view to simplify node settings. Click to see the whole description.
4. **Copy node** [Optional] [Action]: Creates a copy of the current node with all settings**.**
5. **Remove node** [Optional] [Action]: Deletes current node from Process.
6. **Remove links** [Optional] [Action]: Removes all connections to/from current node.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

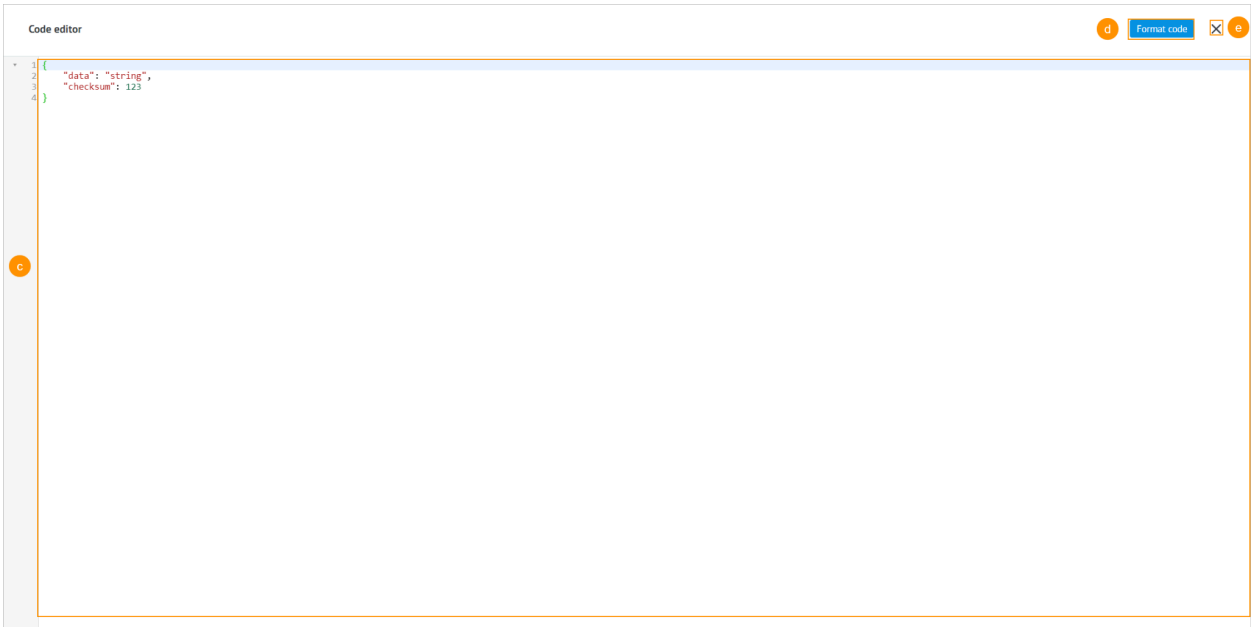
7. **Info** [Optional] [Action]: Displays node ID.
8. **Errors info** [Optional] [Action]: Displays node errors.
9. **Key** [Required] [Field] [String]: Key to specify a static name or a variable in {{}} curly brackets
   To add a new parameter, specify a unique name, to update an existing parameter, use its original name as a key.
10. **Value** [Required] [Field] [Any]: The value you want to assign to the corresponding Key parameter. See data types descrition for more details.
11. **Code editor** [Optional] [Field] [Any]: Shows the JSON format of the entered Key and Value. Click **Code editor** to open the code editor tab, where:



Screenshot 185: **Code editor tab.**

a. **Code editor field** [Optional] [Field] [Any]: Enter new keys and values or edit the existing ones in JSON format.
b. **Full-screen mode** [Optional] [Action]: Opens the **Code editor** field in the full-screen mode, where:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 186: **Code editor full-screen view.**

c. **Code editor field** [Optional] [Field] [Any]: Edit or add keys in values.
d. **Format code** [Optional] [Action]: Format your current code.
e. **Quit the full-screen mode** [Optional] [Action]: Quit the full-screen mode.

12. **Type of data** [Required] [Action]: Selects the data type for the **Value**:
    * String (**S**)
    * Number (**N**)
    * Boolean (**B**)
    * Array (**A**)
    * Object (**O**)

    Note: For more information, go to Data types.

13. **Delete key-value** [Optional] [Action]: Delete the key-value pair.
14. **Add key-value** [Optional] [Action]: Add a new key-value pair.
15. **Alert if the number of tasks in the node queue reaches the following number** [Optional] [Action]: Helps monitor whether the number of tasks in the node exceeds the specified threshold.
16. **Number of tasks in the queue** [Optional/Required if **Alert if the number of tasks in the node queue reaches the following number** is selected] [Field] [Number]: Number of tasks in the queue which triggers an alert when reached. Set to 30 by default.
17. **Maximum interval, for which the task stays in the node before being forwarded** [Optional] [Action]: The amount of time a task is allowed to be in the node can be set in seconds, minutes, hours, and days.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

18. **Maximum interval** [Optional/Required if **Maximum interval, for which the task stays in the node before being forwarded** is selected] [Field] [Number]: The amount of time a task is allowed to be in the node. Has a minimum value of 30 seconds. You can set a shorter interval by using the **Unixtime** function.
19. **Maximum interval measuring unit** [Required] [Action]: Selects the measuring unit for the maximum interval:
    - Seconds
    - Minutes
    - Hours
    - Days.

Examples

Defining a new parameter

In the example below, you can see how to define a new parameter as a constant in a Process.

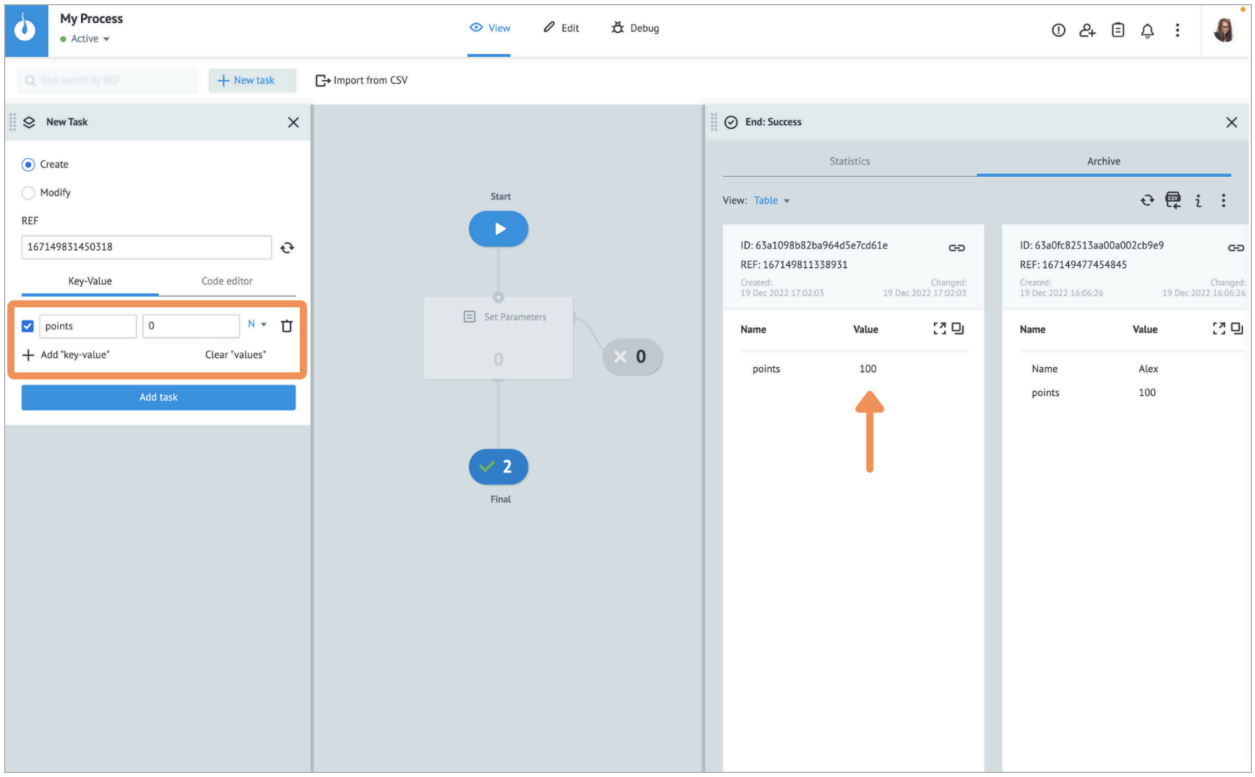
The value of a new parameter doesn't have to be a constant. You can assign it a value based on a task parameter or a value stored in a State Diagram.

Changing parameter value

Below, you can see examples of two ways of modifying the parameter value:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- Replacing a value of an existing parameter with a new one.

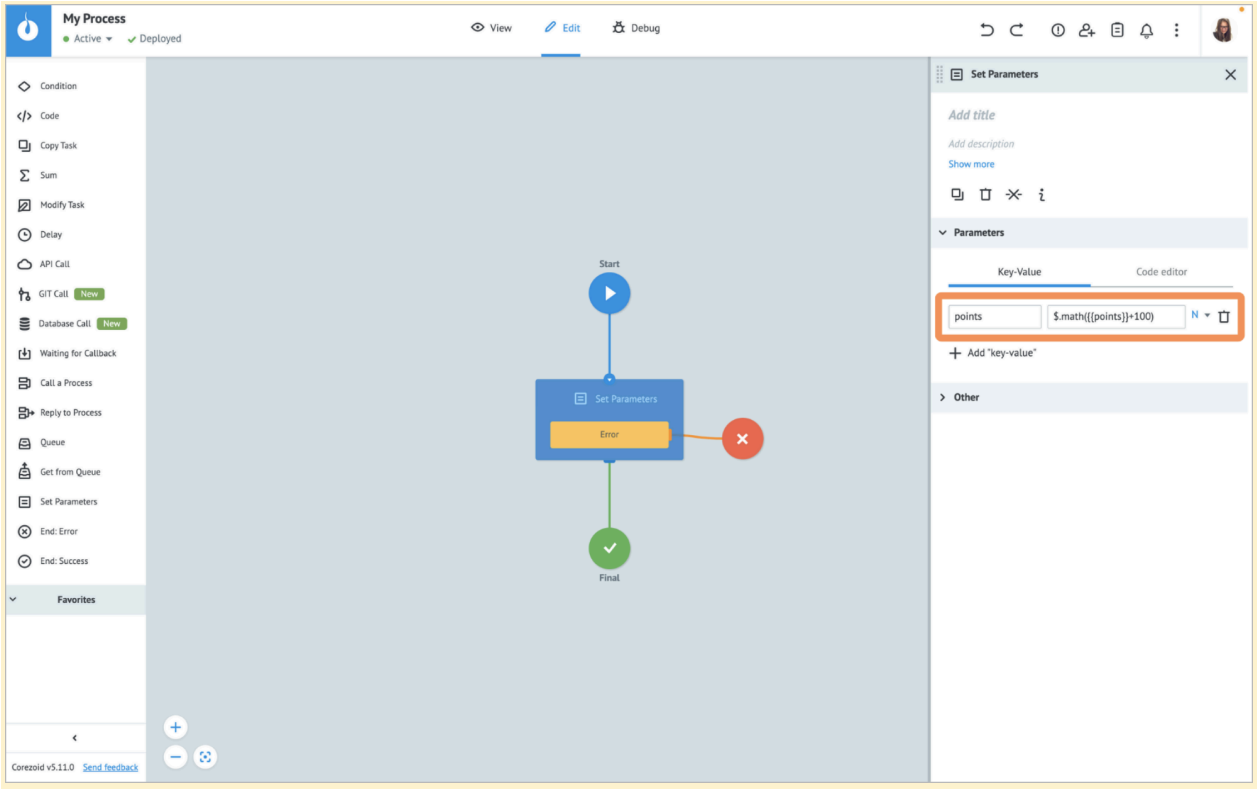


Screenshot 187: **Updating a parameter value in Set Parameters**.

To perform the replacement, the parameter key specified in the **Set Parameters** node must exactly match the key of an existing task parameter. The parameter matching the key ensures that the correct parameter value is updated and prevents accidentally creating a new parameter with the same name.

- Modifying a value of an existing parameter by using the **$.math()** function to add or subtract a certain value from the original value.
  If the task goes to the **Error** node for any reason after passing through the **Set Parameters** node, no parameters will be modified or added, and none of the changes specified by the **Set Parameters** node will be applied.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 188: **Modifying a parameter in Set Parameters**.

You can access a single task parameter or a whole task stored in a State Diagram by Getting Task Parameters from State Diagrams.

Error handling & troubleshooting

When an error occurs during the task processing, you may see the following error parameter names in the task.

Error parameter name	Parameter type
`__conveyor_set_param_return_type_error__`	**Hardware** (system error), **Software** (error in a node logic/settings).

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

`__conveyor_set_param_return_type_tag__`	*Error tag.
`__conveyor_set_param_return_type_description__`	Error description in human-readable language; can be static or dynamic.

*The error `__conveyor_set_param_return_type_tag__` may have the following values.

Table 6: **Error parameter names**.

Value	Cause	Solution
access_denied	You don't have access to the Process.	Contact the Process owner to give you access.
access_denied	You are trying to send or receive tasks from another stage.	Getting and sending tasks between stages is not permitted. Use the same stage to send and receive tasks.
access_refused	The Process you are working in is not a State Diagram but it must be a State Diagram.	Check the logic of using a State Diagram. For more information, go to State Diagram.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

conv_not_found_or_deleted	You are trying to work with a non-existent Process.	Use an existing Process.
not_found_conv_by_alias	Your alias refers to a Process that no longer exists.	Use an existing Process.
set_param_task_size_overflow_limit	The size of the task data exceeds the specified limit set for your environment.	Reduce the size of your task.
set_param_wrong_convert_param	Parameter value conversion failed with the specified parameter type.	Check and correct the parameter's data type specified in the error message.
set_param_wrong_create_obj	Object conversion failed.	Contact the support team to send ExceptionReason.
bad_callback_node	The Process you are calling doesn't have the **Callback** node.	Create the **Callback** node in the Process you are calling.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Table 7: **Set Parameters errors.**

When working with your Process or State Diagram, you may encounter the following issues.

Issue	Cause	Solution
The **Set Parameters** node sends its tasks to the associated Error End node.	The function was called by using a missing or incorrectly formatted parameter.	Ensure all parameters used by your functions are declared and have the appropriate type. For example, to use the **$.math({{x}}+{{y}})** function in a task, both x and y parameters must exist and have the **Number** type values.
	The parameter accessed from another State diagram using **conv/proc** is missing in the State Diagram or has a different type than specified.	Change the parameter type you are requesting or add the required parameter to the State Diagram. **Note:** Parameters of the **String** type are an exception. If the **String** parameter requested using conv/proc doesn't exist in a State Diagram, the **Set Parameters** node will work without errors, but the new parameter will be empty.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

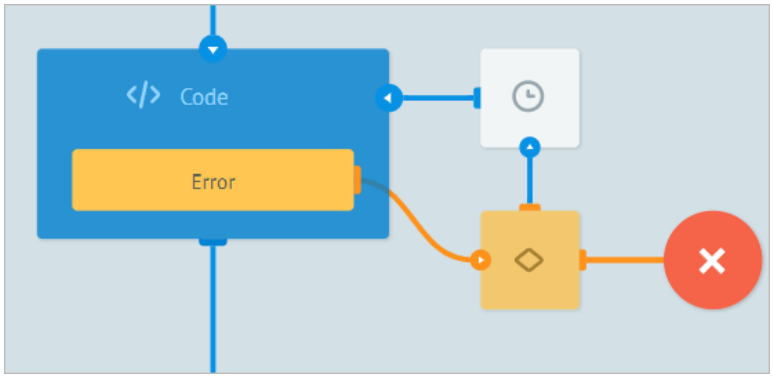
	The parameter has a non-numerical value, even though it was declared a Number. For example, x = 1abc (Number).	When declaring the parameter, remove everything that isn't numeric (values that are not 0-9 numbers, decimal points, negative signs, and so on) from the **Value** field.
	The parameter was initialized in one key-value input field of the **Set Parameters** node, and then the same parameter was assigned a new value in another input field.	Parameters get assigned a value after passing through the node. Therefore, you cannot modify the parameter in the same node it was initially set. To solve this issue, assign the parameter the needed value immediately or modify it later by using another node.

Table 8: **Set Parameters issues.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3.6 Code

Overview



Screenshot 189: **Code node view**

The **Code** node provides a convenient way to add custom functionality to your Process by allowing you to access, modify, and create task parameters based on JavaScript or Erlang code you input in the node.

Compared to the **Git Call** node, the **Code** node offers faster execution and does not require warming up before the first use. It also makes more efficient use of resources.

Although the **Code** node is highly efficient in speed, you may encounter a timeout error if your code is computationally intensive and takes an extended period to execute or if it includes an infinite loop. If your code requires extra processing power, you can create a dedicated microservice and call it from Corezoid. If you need further assistance, don't hesitate to contact us for custom solutions that will better fit your needs.

The **Code** node is designed for handling simple code snippets. For more complex code running, we recommend using the **Git Call** node as a better alternative.

Key Features

- Parameter manipulation
- Custom logic implementation
- Fast execution
- No warm-up required
- Resource efficient

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Limitations

- Simple code snippets only
- Risk of timeout with:
    - Computationally intensive operations
    - Infinite loops
    - Extended execution times

Recommendations

- Use for basic code operations
- For complex code, consider:
    - **Git Call** node
    - Dedicated microservices
    - Custom solutions

Compared to Git Call

Feature	Code Node	Git Call Node
Speed	Faster	Slower
Warm-up	None	Required
Complexity	Simple	Complex
Resources	Efficient	More intensive

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Table 9: **Code vs Git Call comparison.**

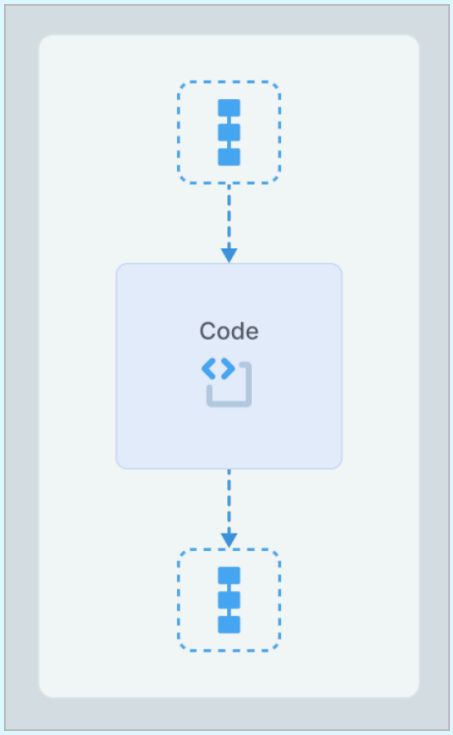
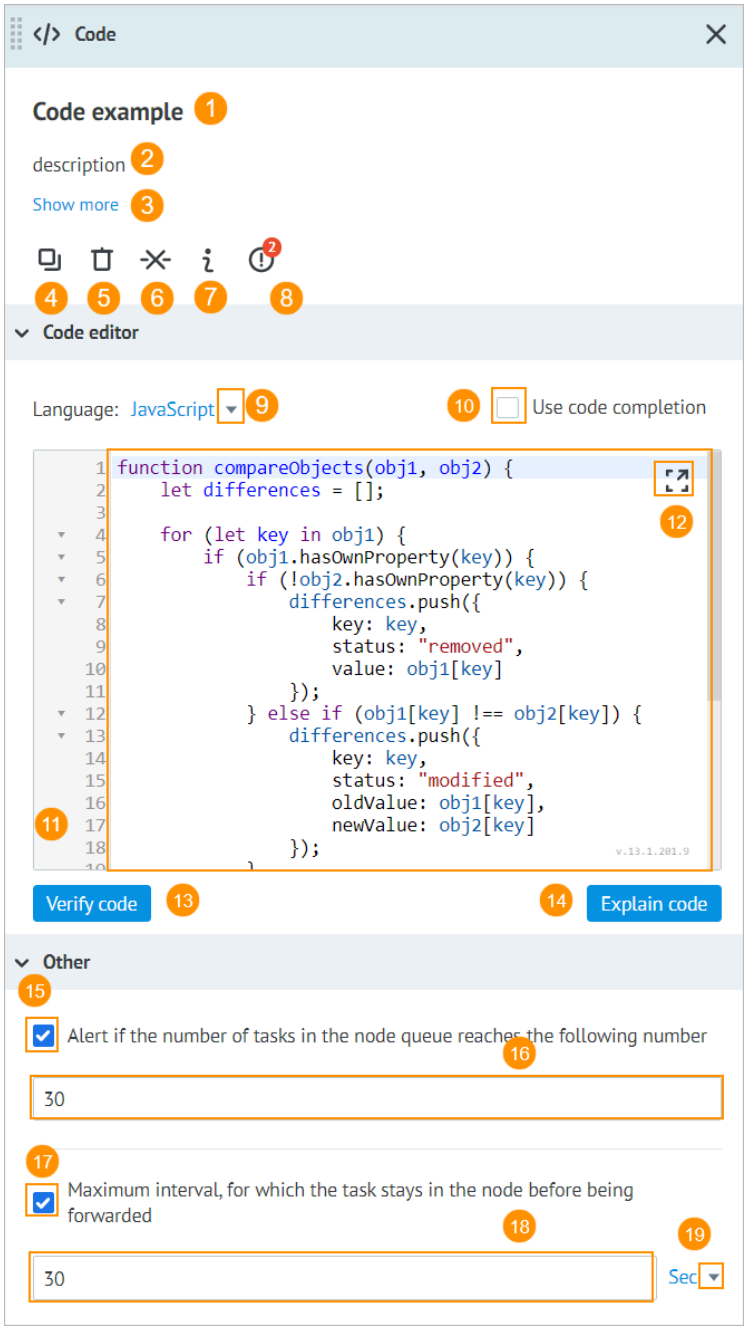


Diagram 10: **Code node flow**.

Node Settings

The **Code** node has the following parameters:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 190: **Code node settings.**

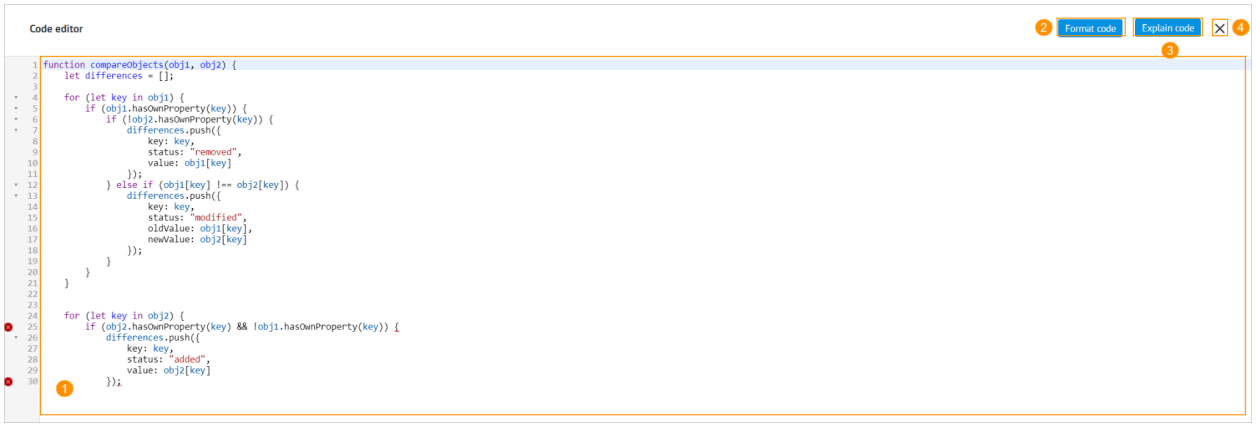
1. **Title** [Optional] [Field] [String]**:** Node title.
2. **Description** [Optional] [Field] [String]: Node description**.**
3. **Show more** [Optional] [Action]: Long descriptions are reduced to a compact view to simplify node settings. Click to see the whole description.
4. **Copy node** [Optional] [Action]: Creates a copy of the current node with all settings**.**
5. **Remove node** [Optional] [Action]: Deletes current node from Process.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

6. **Remove links** [Optional] [Action]: Removes all connections to/from current node.
7. **Info** [Optional] [Action]: Displays node ID.
8. **Errors info** [Optional] [Action]: Displays node errors.
9. **Language** [Required] [Action]: Selects the programming language to be used in the Code editor:

- JavaScript (default)
- Erlang.

10. **Use code completion** [Optional] [Action]: When checked, activates the AI to automatically complete the code in the Code editor. For instance, you can type "//" to generate automatic comments for the current code fragment or type command for the AI to complete the code line automatically.
11. **Code editor** [Required] [Field] [Any]: Code editor field to enter your code.
12. **Full-screen mode** [Optional] [Action]: Opens the **Code editor** field in the full-screen mode, where:



Screenshot 191: **Code editor full-screen view.**

12.1 **Code editor field** [Optional] [Field] [Any]: Edit or add keys in values.
12.2 **Format code** [Optional] [Action]: Format your current code.
12.3 **Explain code** [Optional] [Action]: Activates the AI to generate the explanation for the code in the Code editor.
12.4 **Quit the full-screen mode** [Optional] [Action]: Quit the full-screen mode.

13. **Verify code** [Optional] [Action]: Check your code in the Code editor. If verification succeeds, the message “Code is valid” will appear instead of the Verify code button:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Screenshot 191.1: **Code editor: code validation success.**

14. **Explain code** [Optional] [Action]: Activates the AI to generate an explanation for the code in the Code editor. The explanation will be displayed in a panel below the code:

Screenshot 191.2: **Code editor: code explanation panel.**

To close the generated explanation, click the cross icon in the explanation panel.

15. **Alert if the number of tasks in the node queue reaches the following number** [Optional] [Action]: Helps monitor whether the number of tasks in the node exceeds the specified threshold.
16. **Number of tasks in the queue** [Optional/Required if **Alert if the number of tasks in the node queue reaches the following number** is selected] [Field] [Number]: Number of tasks in the queue which triggers an alert when reached. Set to 30 by default.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

17. **Maximum interval, for which the task stays in the node before being forwarded** [Optional] [Action]: The amount of time a task is allowed to be in the node can be set in seconds, minutes, hours, and days.
18. **Maximum interval** [Optional/Required if **Maximum interval, for which the task stays in the node before being forwarded** is selected] [Field] [Number]: The amount of time a task is allowed to be in the node. It has a minimum value of 30 seconds. You can set a shorter interval by using the **Unixtime** function.
19. **Maximum interval measuring unit** [Required] [Action]: Selects the measuring unit for the maximum interval:
    - Seconds
    - Minutes
    - Hours
    - Days.

Languages Supported in the Code Editor

The Code editor supports the following programming languages:

- **Javascript**: Task parameters can be found within the data object, and you can access them using dot notation.
  `data.parameter_name`
- List of supported JavaScript libraries:
  - moment v2.11.1 (example)
  - moment-timezone v2.11.1 (example)
  - CryptoJS v3.1.2 (limited)
- Available CryptoJS methods:
  - Hashing: MD5, SHA-1, SHA-512, HMAC MD5, HMAC SHA-1, HMAC SHA-256
  - Ciphers: AES, DES, Triple DES, Rabbit, RC4
  - Encoders: base64
    Below you can see an example of your code where **Data** is the task object, and **param** is one of its parameters.

```
-module(node).
-export([main/1]).
```

- `main(Data) -> [{<<"param">>, <<"Hello World!!!">>} | Data].`

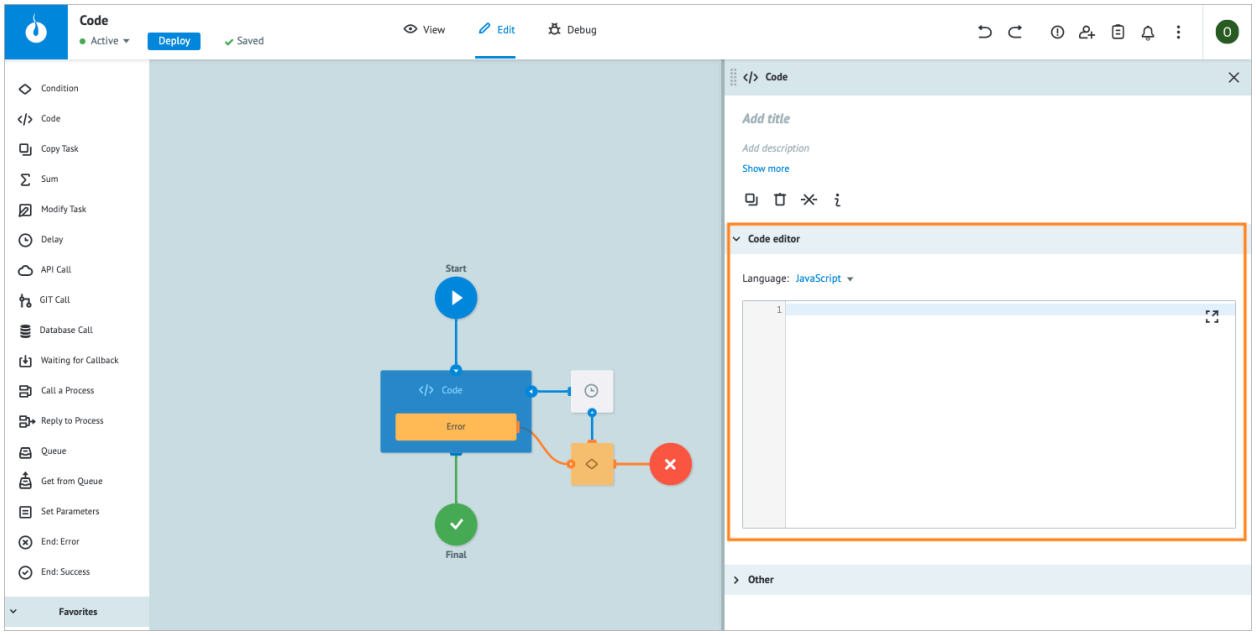
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- **XRegExp**: XRegExp is an extensive regular expression library for JavaScript, enhancing native RegExp with new capabilities and syntaxes. It simplifies complex regex operations and supports Unicode matching.

Examples

Working with date/time using moment.js

In the **Code editor** field of the **Code** node details panel, you can write your code in JavaScript using the supported libraries.



Screenshot 192: **Code editor field with JavaScript language selected**.

You can see the example below.

```
require("libs/moment.js");

let datetime = moment().format('MMMM Do YYYY, h:mm a');


data.message = "Hi " + data.name + ". Today is " + datetime +". What a great time to learn Corezoid!" ;
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- `require("libs/moment.js")`: Import the **moment.js** library used to obtain the current date and time.
- `let datetime = moment().format('MMMM Do YYYY, h:mm a')`: Use the **moment()** method to obtain the current date and time and the format() method to convert it into a string. The resulting value is assigned to a local variable.
- `data.message = "Hi " + data.name + ". Today is " + datetime +". What a great time to learn Corezoid!"`: Create a string using the value of the **name** parameter from the data object and the value of the datetime variable generated. Assign this string to **data.message**, which creates a new task parameter with the **message** key and the value of the string.

The data object stores the task parameters: **name** is an existing task parameter, and **message** is added as a new one. **datetime** is a local variable that will not be part of the task; it only exists temporarily while the node processes it.

Moment-timezone

```
require("libs/moment-timezone.js");

data.date = moment().tz('Europe/Kiev').format("DD-MM-YYYY HH:mm:ss");
```

- `require("libs/moment-timezone.js")`: Import the **moment-timezone.js** library used to convert and format dates for different time zones.
- `data.date = moment().tz('Europe/Kiev').format("DD-MM-YYYY HH:mm:ss")`: Use the moment() method to obtain the current time, the **format()** method to define the desired format, and the **tz()** method to change the time zone to Europe/Kiev.

Hashing

MD5

```
require("libs/md5.js");
```

- `data.md5 = CryptoJS.MD5(data.variable).toString();`
- JavaScript
- Copy
    - `data.variable`: Task parameter that will be hashed.
    - `data.md5`: Task parameter that will store the hash.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

SHA-1

```
require("libs/sha1.js");
```

- `data.sha1 = CryptoJS.SHA1(data.variable).toString();`
  - `data.variable`: Task parameter that will be hashed.
  - `data.sha1`: Task parameter that will store the hash.

SHA-512

```
require("libs/sha512.js");
```

- `data.sha512 = CryptoJS.SHA512(data.variable).toString();`
  - `data.variable`: Task parameter that will be hashed.
  - `data.sha512`: Task parameter that will store the hash.

HMAC SHA-1

```
require("libs/hmac-sha1.js");
var hash = CryptoJS.HmacSHA1(data.message, data.secret);
```

- `data.hashInHex = hash.toString(CryptoJS.enc.Hex);`
  - `data.message`: Task parameter that will be hashed.
  - `data.secret`: Task parameter that stores the secret.
  - `data.hashInHex`: Task parameter that will store the hash as a hexadecimal string.

HMAC SHA-256

```
require("libs/hmac-sha256.js");
var hash = CryptoJS.HmacSHA256(data.message, data.secret);
```

- `data.hashInHex = hash.toString(CryptoJS.enc.Hex);`
  - `data.message`: Task parameter that will be hashed.
  - `data.secret`: Task parameter that stores the secret.
  - `data.hashInHex`: Task parameter that will store the hash as a hexadecimal string.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Ciphers

AES

Encrypting

```
require("libs/aes.js");
```

- `data.encrypted = CryptoJS.AES.encrypt(data.message, data.password).toString();`
    - `data.message`: Task parameter that will be encrypted.
    - `data.password`: Task parameter that contains the password.
    - `data.encrypted`: Task parameter that will store the encrypted string.

Decrypting

```
require("libs/aes.js");
```

- `data.decrypted = CryptoJS.AES.decrypt(data.encrypted, data.password).toString(CryptoJS.enc.Utf8);`
    - `data.encrypted`: Task parameter that will be decrypted.
    - `data.password`: Task parameter that contains the password.
    - `data.decrypted`: Task parameter that will store the decrypted string.

DES

Encrypting

```
require("libs/tripledes.js");
```

- `data.encrypted = CryptoJS.DES.encrypt(data.message, data.password).toString();`
    - `data.message`: Task parameter that will be encrypted.
    - `data.password`: Task parameter that contains the password.
    - `data.encrypted`: Task parameter that will store the encrypted string.

Decrypting

```
require("libs/tripledes.js");
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- `data.decrypted = CryptoJS.DES.decrypt(data.encrypted, data.password).toString(CryptoJS.enc.Utf8);`
    - `data.encrypted`: Task parameter that will be decrypted.
    - `data.password`: Task parameter that contains the password.
    - `data.decrypted`: Task parameter that will store the decrypted string.

Triple DES

Encrypting

```
require("libs/tripledes.js");
```

- `data.encrypted = CryptoJS.TripleDES.encrypt(data.message, data.password).toString();`
    - `data.message`: Task parameter that will be encrypted.
    - `data.password`: Task parameter that contains the password.
    - `data.encrypted`: Task parameter that will store the encrypted string.

Decrypting

```
require("libs/tripledes.js");
```

- `data.decrypted = CryptoJS.TripleDES.decrypt(data.encrypted, data.password).toString(CryptoJS.enc.Utf8);`
    - `data.encrypted`: Task parameter that will be decrypted.
    - `data.password`: Task parameter that contains the password.
    - `data.decrypted`: Task parameter that will store the decrypted string.

Rabbit

Encrypting

```
require("libs/rabbit.js");
```

- `data.encrypted = CryptoJS.Rabbit.encrypt(data.message, data.password).toString();`
    - `data.message`: Task parameter that will be encrypted.
    - `data.password`: Task parameter that contains the password.
    - `data.encrypted`: Task parameter that will store the encrypted string.

Decrypting

```
require("libs/rabbit.js");
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- `data.decrypted = CryptoJS.Rabbit.decrypt(data.encrypted, data.password).toString(CryptoJS.enc.Utf8);`
    - `data.encrypted`: Task parameter that will be decrypted.
    - `data.password`: Task parameter that contains the password.
    - `data.decrypted`: Task parameter that will store the decrypted string.

RC4

Encrypting

```
require("libs/rc4.js");
```

- `data.encrypted = CryptoJS.RC4.encrypt(data.message, data.password).toString();`
    - `data.message`: Task parameter that will be encrypted.
    - `data.password`: Task parameter that contains the password.
    - `data.encrypted`: Task parameter that will store the encrypted string.

Decrypting

```
require("libs/rc4.js");
```

- `data.decrypted = CryptoJS.RC4.decrypt(data.encrypted, data.password).toString(CryptoJS.enc.Utf8);`
    - `data.encrypted`: Task parameter that will be decrypted.
    - `data.password`: Task parameter that contains the password.
    - `data.decrypted`: Task parameter that will store the decrypted string.

Encoders

Base64

Encoding

```
require("libs/base64.js");
var wordArray = CryptoJS.enc.Utf8.parse(data.variable);
```

- `data.base64 = CryptoJS.enc.Base64.stringify(wordArray);`
    - `data.variable`: Task parameter that will be encoded.
    - `wordArray`: Tocal variable that stores a wordArray representation of `data.variable`.
    - `data.base64`: Task parameter that will store the encoded string.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Decoding

```
require("libs/base64.js");
var words = CryptoJS.enc.Base64.parse(data.base64);
```

- `data.parsedStr = words.toString(CryptoJS.enc.Utf8);`
  - `data.base64`: Task parameter that will be decoded.
  - `words`: Tocal variable that stores a wordArray representation of `data.base64`.
  - `data.parsedStr`: Task parameter that will store the decoded string.

JWT Token

Encoding

```
require("libs/hmac-sha256.js");
require("libs/base64.js");

// Custom function to base64URL encode a string manually (pure JS)
function base64UrlEncode(input) {
// Convert the input to a word array
const wordArray = CryptoJS.enc.Utf8.parse(input);
// Convert the word array to a base64 string
let base64String = CryptoJS.enc.Base64.stringify(wordArray);
// Perform base64URL encoding (replace characters and remove padding)
base64String = base64String.replace(/\+/g, '-').replace(/\//g,
'_').replace(/=+$/, '');
return base64String;
}
// Function to create a secure HMAC SHA-256 signature
function createSignature(message, secret) {
// Generate HMAC SHA-256 signature using the loaded library
const hash = CryptoJS.HmacSHA256(message, secret);
// Convert the hash to a base64-encoded string
const base64 = CryptoJS.enc.Base64.stringify(hash);
// Convert base64 to base64url
return base64.replace(/\+/g, '-').replace(/\//g, '_').replace(/=+$/,
'');
}
// Function to generate the JWT token
function generateJwtToken(header, payload, secret) {
// Step 1: Encode the header and payload
const encodedHeader = base64UrlEncode(JSON.stringify(header));
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
const encodedPayload = base64UrlEncode(JSON.stringify(payload));
// Step 2: Create the signature using HMAC-SHA256
const message = `${encodedHeader}.${encodedPayload}`;
const signature = createSignature(message, secret);
// Step 3: Return the final JWT
return `${message}.${signature}`;
}

const header = {
alg: "HS256",
typ: "JWT"
};

const payload = data.payload;
const secret = data.secret;
```

- `data.jwtToken = generateJwtToken(header, payload, secret);`

Decoding

```
require("libs/base64.js");
require("libs/hmac-sha256.js");

// Base64Url encode function
function base64UrlEncode(input) {
return CryptoJS.enc.Base64.stringify(input)
.replace(/\+/g, '-')
.replace(/\//g, '_')
.replace(/=+$/, '');
}

// Base64Url decode function
function base64UrlDecode(input) {
let base64 = input.replace(/-/g, '+').replace(/_/g, '/');
while (base64.length % 4) {
base64 += '=';
}
return CryptoJS.enc.Base64.parse(base64);
}

// Create HMAC SHA-256 signature
function createSignature(message, secret) {
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
const hash = CryptoJS.HmacSHA256(message, secret);
return base64UrlEncode(hash);
}

// Decode and validate JWT token
function decodeAndValidateJwtToken(token, secret) {
const parts = token.split('.');
if (parts.length !== 3) {
throw new Error('Invalid JWT token format');
}

// Decode header and payload
const header =
JSON.parse(CryptoJS.enc.Utf8.stringify(base64UrlDecode(parts[0])));
const payload =
JSON.parse(CryptoJS.enc.Utf8.stringify(base64UrlDecode(parts[1])));
const providedSignature = parts[2];

// Store decoded parts and original signature
data.jwtHeader = header;
data.jwtPayload = payload;
data.jwtSignature = providedSignature;

// Validate signature if secret is provided
if (secret) {
const message = `${parts[0]}.${parts[1]}`;
const expectedSignature = createSignature(message, secret);

data.signatureValidation = (expectedSignature === providedSignature)
? 'success' : 'failed';
} else {
data.signatureValidation = 'not performed';
}

// Validate token expiration
const currentTime = Math.floor(Date.now() / 1000);
if (payload.exp && payload.exp < currentTime) {
data.tokenStatus = 'expired';
} else {
data.tokenStatus = 'valid';
}
}
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
// Main execution
try {
const token = data.token;
const secret = data.secret || null; // Optional secret for validation

decodeAndValidateJwtToken(token, secret);
} catch (error) {
data.error = error.message;


    }
```

XRegExp

- **Extended Syntax**: This example demonstrates named capture groups for improved readability and maintainability.

Input (data.input - a text string):

```
[2024-11-06 14:23:00] INFO: User login successful, User ID: 12345,
IP: 192.168.1.1
[2024-11-06 14:23:15] ERROR: Failed to load data, User ID: 12345

[2024-11-06 14:23:30] WARN: User requested invalid action, IP:
192.168.1.2
```

Code:

```
require("libs/xregexp.js");

// Define the complex multiline regex pattern
const logPattern = XRegExp(`
    ^                                   # Start of the line
    (?<timestamp>\\[[^\\]]+\\])         # Capture timestamp in square
brackets
    \\s+                                # Match one or more spaces
    (?<level>\\w+)                      # Capture the log level (INFO,
ERROR, etc.)
    :\\s+                               # Match colon and space after
level
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
    (?<action>[^,]+)                    # Capture the user action
description (until the first comma)
    (?:,\\s+User\\sID:\\s+(?<userId>\\d+))?  # Optionally match User
ID (if available)
    (?:,\\s+IP:\\s+(?<ip>[^\\s,]+))?       # Optionally match IP
address (if available)
    $                                   # End of the line
`, 'gm'); // Use 'g' for global matching and 'm' for multiline flag
data.output = [];

// Parse the log text using XRegExp.exec and iterate through matches
let match;
while ((match = XRegExp.exec(data.input, logPattern)) !== null) {
  const logEntry = {
    timestamp: match.timestamp,
    level: match.level,
    action: match.action,
  };

  if (match.userId) {
    logEntry.userId = match.userId;
  }
  if (match.ip) {
    logEntry.ip = match.ip;
  }

  // Output will now be stored in `data.output`
  data.output.push(logEntry);

}
```

Output (data.output - a JSON array):

```
[
  {
    "timestamp": "[2024-11-06 14:23:00]",
    "level": "INFO",
    "action": "User login successful",
    "userId": "12345",
    "ip": "192.168.1.1"
  },
  {
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
    "timestamp": "[2024-11-06 14:23:15]",
    "level": "ERROR",
    "action": "Failed to load data",
    "userId": "12345"
  },
  {
    "timestamp": "[2024-11-06 14:23:30]",
    "level": "WARN",
    "action": "User requested invalid action",
    "ip": "192.168.1.2"
  }

]
```

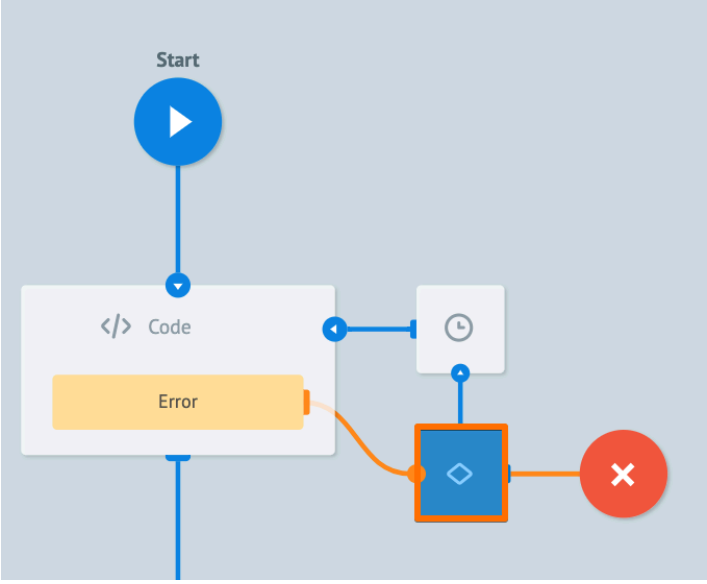
- **Unicode Matching**: This pattern matches any Unicode letter, showcasing XRegExp's Unicode property support.

```
require("libs/xregexp.js");

XRegExp('\\p{L}');
```

Error handling & troubleshooting

When an error occurs in the **Code** node, a task goes to the auxiliary **Condition** output node that is used for storing error parameters.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 193: **Forwarding an error occurred in the Code node to the auxiliary Condition node.**

When an error occurs during the task processing, you may see the following error parameter names in the task.

Error parameter name	Error type
`__conveyor_code_return_type_error__`	**Hardware** (system error), **Software** (error in a node logic/settings).
`__conveyor_code_return_type_tag__`	*Error tag.
`__conveyor_code_return_type_description__`	Error description in human-readable language; can be static or dynamic.

*The error tag `__conveyor_code_return_type_tag__` may have the following values.

Table 10: **Error parameter names.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Value	Cause	Solution
code_timeout	Code has been running for too long.	Check if your program contains an infinite loop.
code_executing_error	Your code contains an error.	Debug your code.
code_return_format_error	Your code has a return statement that doesn't belong in a function and returns a value that isn't an object. The **Code** node wraps your code into a function, so an unexpected return statement can cause the program to end abruptly.	Remove the return statement to resolve the error.
code_return_size_overflow	The size of your code exceeds the specified limit set for your environment.	Reduce the size of your code.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

code_fatal_error	A hardware error has occurred.	Contact the support team for further assistance.

Table 11: **Code node errors.**

When working with your Process or State Diagram, you may encounter the following issues.

Issue	**Cause**	**Solution**
Failing to access task parameters.	The code refers to objects or object properties absent from the task.	Define the missing objects or object properties within the **Code** node or adjust the code to remove all references to them.
	A task parameter is referenced without using the data object.	Add a data object reference in front of the name of a task parameter. For example: data.amount = 1 is correct; amount = 1 is not correct.
Task parameters are gone.	The data object was assigned a new value in the **Code** node.	Data is a special object used by Corezoid to store task parameters. Therefore, access to task parameters will be removed if it is changed. You should avoid reassigning this object and use a different name for your variables instead.

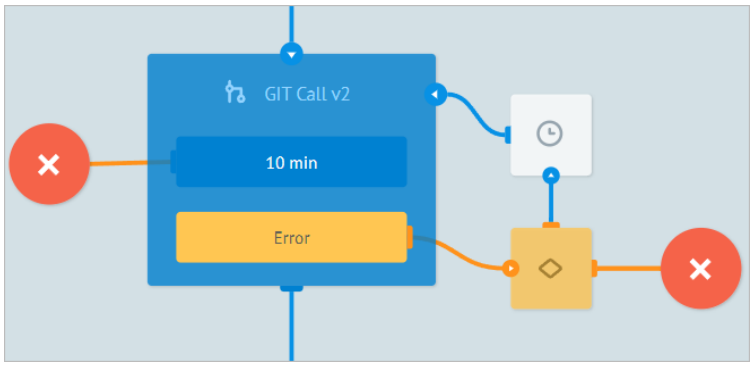
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Table 12: **Code node issues.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3.7 Git Call

Overview



Screenshot 194: **Git Call node view.**

With the **Git Call** node, you can easily fetch and execute code from any platform that supports Git protocol for code storing and managing. For example, you can use GitHub, GitLab platforms. All you need to do is specify the needed repository, write a program that uses it (or include it in the repository), and give proper build instructions.

Experience with Git and the command line is recommended before you start working with the **Git Call** node.

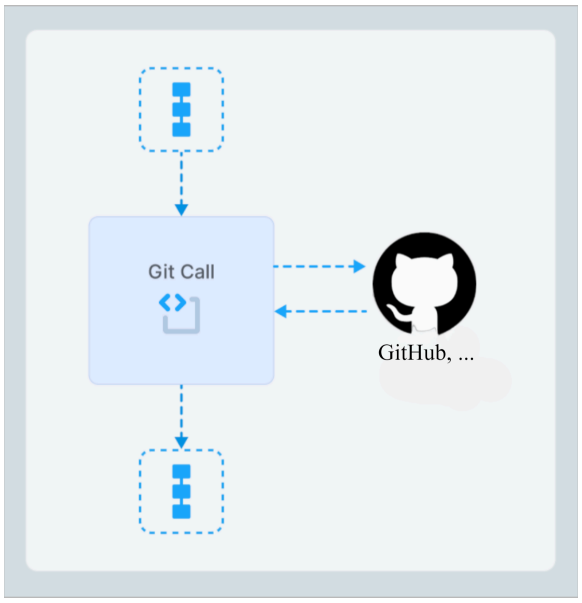


Diagram 11: **Git Call node flow**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

All Git Call calls are made from the following IP addresses:

- 54.171.15.37/32
- 108.128.68.222/32
- 63.33.226.230/32

If you encounter access errors when using Git Call nodes, you might need to white-list these addresses.

Supported languages

In GitCall, you can run code for the following programming languages:

JavaScript (node) v20; PM: yarn, npm; OS: alpine v3.17

- JavaScript (NodeJS) (node) v20; PM: yarn, npm; OS: alpine v3.17
- GoLang v1.23; PM: go mod; OS: alpine v3.20
- Python v.3.12; PM: pip; OS: alpine v3.17
- Java v22; PM: gradle; OS: alpine v3.15
- PHP v8.3; PM: composer v2.5.4; OS: alpine v3.17
- Closure v1.11.1; PM: lein 2.10.0; OS: alpine v3.17
- Lisp v2.4.8; PM: roswell v20.05.14.106; OS: Ubuntu v18.04.4
- Prolog **PM**: swipl v9.2.7; OS: debian:bullseye-slim
- GitCall allows **custom Dockerfiles** to provide more flexibility and broader capabilities for configuring code-running environments. This feature makes using code in any language, be it C, C++, Rust, Swift, or any other language possible.

Code dependencies management

In the GitCall, two options for using your code are available:

1. Code editor - you enter user code directly in the Git Call node's Code editor.
2. Git Repo - you specify the link to a git repository that contains the code.
   The code dependencies management differs depending on the option you choose.

JS

For managing dependencies in JS, the following package manager is used:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

NPM

Code editor
Build command:

```
npm install crypto-js@4.1.1 moment@2.29.4
```

Git Repo
You need to add the file with the following dependencies specified to your project:
package.json

```
{
"name": "gitcall-example",
"version": "1.0.0",
"description": "gitcall example",
"dependencies": {
"crypto-js": "^4.1.1",
"moment": "^2.29.4"
}

}
```

Git Example

Python

For managing dependencies in Python, the following package manager is used:

```
https://pip.pypa.io/
```

Code editor
Build command:

```
pip install 'pkg_info==0.1.2' 'pycryptodomex==3.20'
```

Git Repo
You need to add the file with the following dependencies specified to your project:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

`requirements.txt`

```
pycryptodomex==3.20
```

Git Example

Java

To manage dependencies and building in Java, you can use any available build options by specifying any command in the **Build command** field.

The main tools for Java project building:

https://gradle.org/
https://maven.apache.org/

Code editor
The dependencies command is not available.

Git Repo
You need to add gradle files (use the structure from the example below) to your project and specify the needed dependencies in the build.gradle.

The example with dependencies specified (implementation: 'com.squareup.okhttp3:okhttp:4.9.0'): https://github.com/corezoid/gitcall-examples/blob/master/java/http_request/usercode/build.gradle#L9
If the system detects a gradlew file in your project, gitcall will automatically execute the **Build command**:

```
./gradlew build
```

Also, you can specify any command you wish to use in the **Build command** field.

Git Example

Golang

For managing dependencies in Golang, the following package manager is used:

```
https://go.dev/ref/mod
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Code editor
In Golang, the last versions of dependencies are set automatically.

Git Repo
In Golang, the last versions of dependencies are set automatically.
If you need to specify specific dependency versions, add the following file to your project:

go.mod

```
module uuid_deps

go 1.22.5

require (
  github.com/corezoid/gitcall-go-runner v1.0.0
  github.com/google/uuid v1.6.0

)
```

Git Example

PHP

In PHP, you can install packets and extensions.
For dependencies management in PHP, use the following:

https://getcomposer.org/

For extensions management in PHP, use the following:

https://github.com/mlocati/docker-php-extension-installer

Code editor
Build command:

```
composer require pdeans/xml-builder:^1.0.3 guzzlehttp/guzzle:^7.0
```

or:

```
docker-php-ext-install pcntl soap bcmath pdo_mysql
```

or the combined Build command:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
docker-php-ext-install bcmath && composer require
guzzlehttp/guzzle:^7.0
```

Git Repo
You need to add the file with the following dependencies specified to your project:
composer.json

```
{
"name": "gitcall/example",
"type": "project",
"require": {
"guzzlehttp/guzzle": "^7.2"
}

}
```

Git example 1, example 2

List of supported PHP extensions

Clojure

For managing dependencies in Clojure, use the following manager:

https://leiningen.org/

Code editor
Build command:

```
lein change :dependencies conj '[danlentz/clj-uuid "0.1.9"]' && lein
install
```

For multiple dependencies:
Build command:

```
lein change :dependencies conj '[danlentz/clj-uuid "0.1.9"]'
'[org.clojure/data.json "2.5.0"]' && lein install
```

**2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.**

Git Repo
You need to use the structure from the example below and add the file with specified dependencies to your project:

project.clj

```
(defproject usercode "1.0.0"
:description "gitcall-example"

:dependencies [[danlentz/clj-uuid "0.1.9"]])
```

Git Example

Lisp

For managing dependencies in Lisp, use the following managers:

Roswell
Quicklisp

Code editor
You don't need to use the **Build command** to upload dependencies to Lisp. It's sufficient to specify the needed dependency in your code:

```
(ql:quickload '(:cl-mustache) :silent t)
```

Code example

Git Repo
In Git repositories there are two ways of dependencies management:

1. Similar to the dependencies management in Code editor for Lisp, you need to specify the needed dependency in your code:

```
(ql:quickload '(:cl-mustache) :silent t)
```

Code example

2. asd files use example

Prolog

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

For managing dependencies in Prolog, use the following package manager:

SWI Prolog

Code editor

Build command:

```
swipl -g "pack_install(matrix, [interactive(false)])."
```

Git Repo

Build command:

```
swipl -g "pack_install(matrix, [interactive(false)])."
```

Git example

Custom Dockerfiles Use

Git Call supports the use of custom Dockerfiles, providing more flexible options for configuring the code execution environment. This feature allows users to create their own Docker containers with necessary dependencies and configurations.

Key aspects

1. **Creating a Dockerfile**
- Users can create their own Dockerfile for their project.
- The Dockerfile can be placed in the repository along with the project code.
- Git Call will use this Dockerfile to create the execution environment.
2. **Mandatory requirements**
- In the Dockerfile, it's necessary to create a user and group with ID 501:

```
RUN addgroup -g 501 usercode && adduser -u 501 -G usercode -s /bin/sh
-D usercode
```

```
USER usercode
```

- The container will be run in read-only mode.
- The /tmp directory is available for temporary files.
3. **HTTP server**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- The Dockerfile needs to set up an HTTP server that will listen on the port specified in the GIT_CALL_PORT environment variable.
- The server should handle POST requests and work with the JSON-RPC 2.0 protocol.

4. **Example Dockerfile structure**

```
FROM node:20-alpine
WORKDIR /app
COPY src/ .
RUN addgroup -g 501 usercode && adduser -u 501 -G usercode -s /bin/sh
-D usercode
USER usercode

CMD ["node", "main.js"]
```

5. **Advantages of using Dockerfile**

- Ability to install additional dependencies and libraries.
- Flexible environment configuration for specific project needs.
- Possibility to use languages and tools not supported by the standard **Git Call** configuration (e.g., C++, Rust).

6. **Working with Dockerfile**

- When selecting the Dockerfile option in **Git Call** settings, the user specifies the repository URL and path to the Dockerfile.
- Git Call clones the repository, builds a Docker image based on the provided Dockerfile.
- The built image is used to execute the user's code.

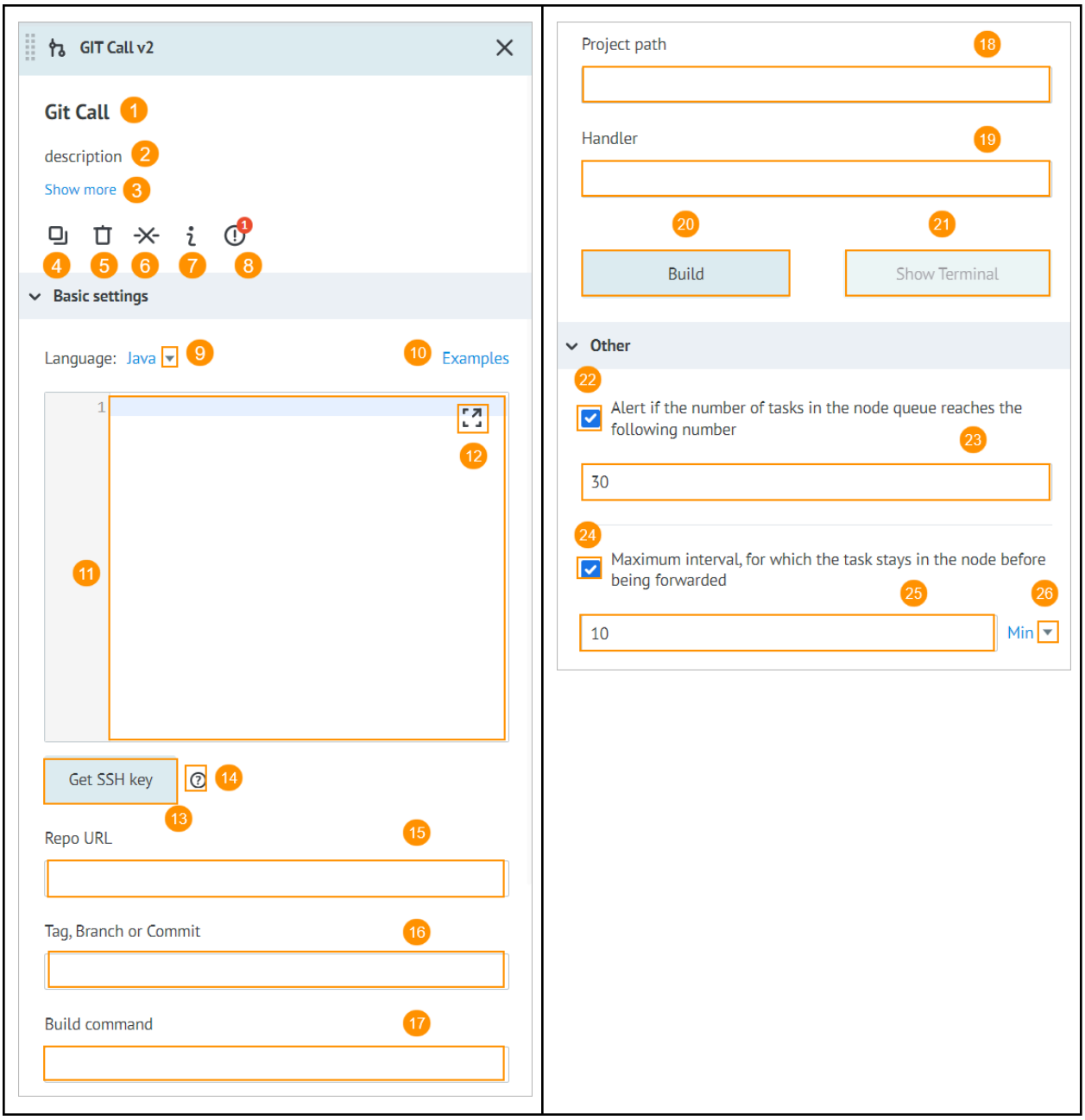
7. **Usage recommendations**

- Dockerfile is suitable for more complex projects with non-standard requirements.
- For simple tasks, it's recommended to use the standard **Git Call** code editor.
- When using Dockerfile, the user takes responsibility for correctly configuring the execution environment.

Node Settings

The **Git Call** node has the following parameters:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 195: **Git Call node settings.**

1. **Title** [Optional] [Field] [String]**:** Node title.
2. **Description** [Optional] [Field] [String]: Node description**.**
3. **Show more** [Optional] [Action]: Long descriptions are reduced to a compact view to simplify node settings. Click to see the whole description.
4. **Copy node** [Optional] [Action]: Creates a copy of the current node with all settings**.**
5. **Remove node** [Optional] [Action]: Deletes current node from Process.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

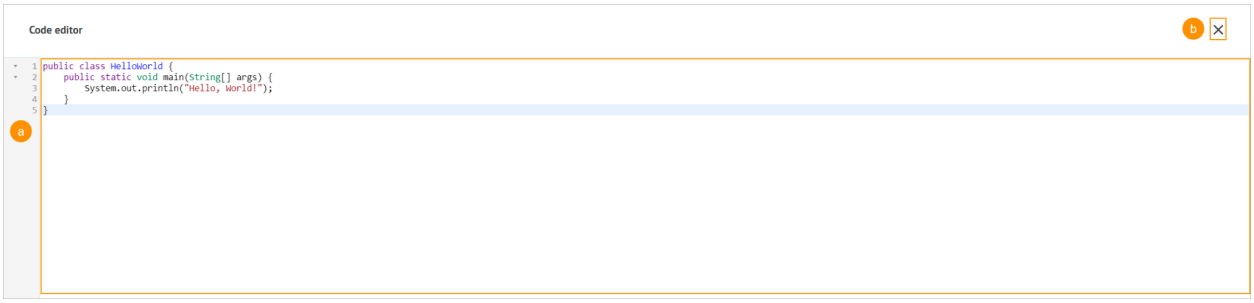
6. **Remove links** [Optional] [Action]: Removes all connections to/from current node.
7. **Info** [Optional] [Action]: Displays node ID.
8. **Errors info** [Optional] [Action]: Displays node errors.
9. **Language** [Required] [Action]: Selects the programming language to be processed by the node. The following languages are supported with their Package Managers (PM) and OS:
    - **JavaScript**
    - **GoLang**
    - **Python**
    - **PHP**
    - **Java**
    - **Closure**
    - **Lisp**
    - **Prolog**
    - **Dockerfile**

10. **Examples** [Optional] [Action]: Opens the repository with examples.
11. **Code editor** [Optional/Required if **Repo URL** is empty] [Field] [Any]: Code editor field to enter your custom code which will be used with the existing codebase from the Git repository and executed as a part of the overall workflow.

Note: As another option, you can include the **Usercode** file in your Git repository, which allows you to manage and control versions of your custom code alongside the main codebase. If the file **Usercode** is already included in the Git repository, but you also write code in the **Code editor**, the original file will be overwritten.
To correctly integrate and execute your custom code within the **Git Call** node, it must be written within a wrapper. You can find detailed examples of the appropriate wrapper syntax for each programming language on our GitHub page.
JSON-RPC 2.0 Protocol: Git Call uses the JSON-RPC 2.0 protocol for communication between the Corezoid platform and the user's code. Understanding this protocol is crucial for effective development and testing of Git Call nodes.

12. **Full-screen mode** [Optional] [Action]: Click to open your code in the full-screen mode, where:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 196: **Code editor full-screen mode.**

a. **Code editor field** [Optional] [Field] [Any]: Edit or add keys in values.
b. **Quit the full-screen mode** [Optional] [Action]: Quit the full-screen mode.

13. **Get SSH key** [Optional] [Action]: Generates an SSH key that can be added to your private Git repository and automatically authenticates the node.

14. **Node article link** [Optional] [Action]: Opens the node article.

15. **Repo URL** [Optional/Required if **Code editor** is empty] [Field] [String]: Indicates the URL of the Git repository that you will retrieve. If you choose to authenticate your request to access the repository with a username and password, you can do so by adding them to the URL in the following format:
`https://username:password@github.com/account/repository`

- For security reasons, it is not recommended to include credentials in the URL. Instead, we suggest using an SSH key to authenticate your requests.

16. **Tag Branch Commit** [Required if **Repo URL** is specified] [Field] [String]: Indicates a specific branch from your repository when you specify the **Repo URL.** In most cases, it is the **Main** branch.

17. **Build command** [Optional] [Field] [String]: Commands to build your code and install any missing libraries by using Package Manager. The Build command is executed within a containerized environment, typically based on a Docker image. This ensures that the build process is isolated and reproducible.
The Build command uses standard sh syntax (Bourne Shell). You can use any valid sh commands, including:

- Package managers (npm, pip, etc.) to install dependencies.
- Build tools (make, Gradle, etc.) to compile code.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- Testing frameworks (Jest, pytest, etc.) to run tests.

After clicking this button, a terminal window opens, and you can monitor the execution of your commands.

18. **Project path** [Optional] [Field] [String]: Specifies an optional custom path to your code when the **Repo URL** is specified.

19. **Handler** [Optional] [Field] [String]: The name of the class that implements UsercodeHandler. Available only when **Java** is selected in the **Language**.

20.**Build/Rebuild** [Optional] [Action]: Builds your code for the node based on the **Build command** input. After the first use, the button title changes to **Rebuild**.

21. **Show terminal** [Optional] [Action]: Shows code build/rebuild logs.

22. **Alert if the number of tasks in the node queue reaches the following number** [Optional] [Action]: Helps monitor whether the number of tasks in the node exceeds the specified threshold.

23. **Number of tasks in the queue** [Optional/Required if **Alert if the number of tasks in the node queue reaches the following number** is selected] [Field] [Number]: Number of tasks in the queue which triggers an alert when reached. Set to 30 by default.

24. **Maximum interval, for which the task stays in the node before being forwarded** [Optional] [Action]: The amount of time a task is allowed to be in the node can be set in seconds, minutes, hours, and days.

25. **Maximum interval** [Optional/Required if **Maximum interval, for which the task stays in the node before being forwarded** is selected] [Field] [Number]: The amount of time a task is allowed to be in the node. It is set to 10 minutes by default and has a minimum of 30 seconds. You can set a shorter interval by using the **Unixtime** function.

26. **Maximum interval measuring unit** [Required] [Action]: Selects the measuring unit for the maximum interval:

- Seconds
- Minutes
- Hours
- Days.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

The **Git Call** node is different from other nodes in that every time you make a change, you'll need to rebuild the node by clicking **Build** before deploying it.

Resource Management

Git Call provides a containerized environment for code execution with allocated resources. Understanding how these resources are managed and can be adjusted is crucial for optimizing performance and handling more demanding tasks.

a. Default Resource Allocation

By default, each Git Call container is allocated:

- 100 millicpu (0.1 CPU core)
- 50 MB of RAM

b. Resource Limitations

These default allocations may be insufficient for computationally intensive tasks or operations requiring more memory. Examples of resource-intensive tasks include complex calculations, cryptographic operations, or processing large datasets.

c. Current Resource Management

- At present, resource allocation is managed globally for all Git Call nodes.
- Administrators can adjust the global settings to increase resources, e.g., to 500 millicpu or 500 MB of RAM.
- However, this global approach affects all nodes, even those that don't require additional resources.

d. Requesting Additional Resources

If your code requires more resources than the default allocation, you should contact support or your environment's super admin. Explain your use case and the specific resource requirements of your task.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

e. Dockerfiles and Resources

- When using custom Dockerfiles, be aware that the same resource limitations apply.
- Your Dockerfile should be optimized to work within these constraints or be prepared to request additional resources.

JSON-RPC 2.0 protocol use

1. Protocol overview

- JSON-RPC 2.0 is a stateless, light-weight remote procedure call (RPC) protocol.
- It uses JSON as its data format.
- **Git Call** uses this protocol for both incoming requests to user code and outgoing responses.

2. Request structure

Every JSON-RPC request must contain the following fields:

- **jsonrpc:** Must be exactly "2.0"
- **method:** A string containing the method name (typically "handle" for Git Call)
- **params:** An object containing the parameters for the method
- **id:** An identifier established by the client.

Request example:

```
{
  "jsonrpc": "2.0",
  "method": "handle",
  "id": "1234567890",
  "params": {
    "key1": "value1",
    "key2": "value2"
  }

}
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3. Response structure

3.1. Successful response must contain:

- **jsonrpc:** Must be exactly "2.0"
- **result:** The data returned by the method
- **id:** The same ID as the request it's responding to.

Successful response example:

```
{
  "jsonrpc": "2.0",
  "id": "1234567890",
  "result": {
      "key1": "modified_value1",
      "key2": "modified_value2"
    }

  }
```

3.2. Error response must contain:

- **jsonrpc:** Must be exactly "2.0"
- **error:** An object describing the error
- **id:** The same ID as the request it's responding to.

Example error response:

```
{
  "jsonrpc": "2.0",
  "id": "1234567890",
  "error": {
    "code": -32000,
    "message": "An error occurred during processing"
  }

}
```

4. Handling requests in User Code

- The `handle` function (or equivalent in other languages) should process this data and return a result.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

5. Returning responses

- User code should return a JSON object that will be used as the `result` in the JSON-RPC response.
- In case of an error, user code should throw an exception or return an error object, which Git Call will format into a proper JSON-RPC error response.

6. HTTP server configuration

- When using a custom Dockerfile, the user must set up an HTTP server that listens on the port specified by the `GIT_CALL_PORT` environment variable.
- This server should accept **POST** requests and handle them according to the JSON-RPC 2.0 protocol.

7. Example implementation (Node.js)

Node.js

See the Node.js and Dockerfile examples for other languages on GitHub for details.

```
const http = require('node:http');

const port = process.env.GITCALL_PORT;
if (!port) {
    console.error('GITCALL_PORT env is required but not set');
    process.exit(1);
}

process.on('SIGTERM', () => process.exit(1));
process.on('SIGINT', () => process.exit(1));
process.on('uncaughtException', function (err) {
    console.error(`[uncaughtException] time=${(new Date()).getTime()}
error=${err}`);
});
process.on('unhandledRejection', function (reason, p) {
    console.error(`[unhandledRejection] time=${(new
Date()).getTime()} reason=${reason} promise=`, p);
});

const server = http.createServer((request, response) => {
    if (request.method === 'POST' && request.url === '/') {
        let body = [];
        request
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
            .on('data', chunk => body.push(chunk))
            .on('end', () => {
                response.statusCode = 200;
                handler(Buffer.concat(body).toString(), response);
            });
    } else {
        response.statusCode = 404;
        response.end();
    }
});

const handler = async (body, response) => {
    const req = JSON.parse(body);
    const jsonrpc = req.jsonrpc;
    const id = req.id;
    const params = req.params;
    console.log(`[req] time=${(new Date()).getTime()} id=${id}`);
    try {
        if (typeof params !== 'object') {
            throw Error('expected request params is object');
        }
        let result = usercode(params);
        if (result instanceof Promise) {
            result = await result;
        }
        response.end(JSON.stringify({
            jsonrpc: jsonrpc,
            id: id,
            result: result,
        }));
        console.log(`[res] time=${(new Date()).getTime()} id=${id}`);
    } catch (e) {
        response.end(JSON.stringify({
            jsonrpc: jsonrpc,
            id: id,
            error: {
                code: 1,
                message: e.toString(),
            },
        }));
        console.log(`[res] time=${(new Date()).getTime()} id=${id}
error=${e.toString()}`);
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
    }

};

const usercode = (data) => {
    data["js"] = "Hello, world!"
    return data
};

console.log('listening on 0.0.0.0:' + port);

server.listen(port);
```

8. Testing JSON-RPC communication

Local testing

Git Call provides the ability to test your code locally before deploying it to the Corezoid platform. This feature allows developers to simulate the **Git Call** environment on their local machines, ensuring code functionality and performance before integration.

- When testing locally or in CI/CD pipelines, ensure that your test requests follow the JSON-RPC 2.0 format.
- Verify that your code correctly handles the incoming request structure and returns properly formatted responses.

Key aspects

1. Local environment setup

- Users can create a local environment that mimics the **Git Call** execution environment using Docker.
- This setup allows testing without direct dependency on the Corezoid platform.

2. Docker image creation

- Users build a Docker image locally based on their Dockerfile or the standard **Git Call** environment.
- This image contains the necessary runtime, dependencies, and the user's code.

3. Running the local container

- The Docker container is run locally, simulating the **Git Call** node environment.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- Environment variables, such as `GIT_CALL_PORT`, are set to mimic the actual **Git Call** setup.

4. Testing procedure
Users can send test requests to their locally running container using tools like curl or Postman. This procedure emulates how Corezoid would interact with the **Git Call** node.
5. Example commands for local testing
a) Building the Docker image:

```
docker build -t gitcall-example
```

b) Running the Docker container:

```
docker run --rm -it -p 9999:9999 -e GITCALL_PORT=9999 --user 501:501
--read-only gitcall-example
```

c) Sending a test request:

```
curl http://127.0.0.1:9999 -H 'Content-Type: application/json' -d
'{"jsonrpc":"2.0","id":"task-id-1","method":"Usercode.Run","params":{
"key1":"val1"}}'
```

6. Benefits of local testing

- Faster development cycle by reducing dependency on the Corezoid platform for testing.
- Ability to debug and troubleshoot code in a controlled local environment.
- Easier integration of **Git Call** nodes into CI/CD pipelines.

7. JSON-RPC 2.0 protocol in local testing

- Local testing should adhere to the JSON-RPC 2.0 protocol used by **Git Call**.
- Requests should include the required fields: jsonrpc, method, params, and id.
- Responses should follow the protocol structure, including result or error objects.

8. Example JSON-RPC 2.0 request and response

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Request:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "handle",
  "params": {
      "key1": "value1",
      "key2": "value2"
    }

}
```

Response:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
      "key1": "modified_value1",
      "key2": "modified_value2"
    }

  }
```

CommonJS and ESM

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Previously, GitCall for JavaScript projects supported only CommonJS module loading. However, GitCall has added support for ECMAScript modules (ESM).

Many modern libraries no longer support CommonJS, transitioning entirely to ESM. For instance, starting from version 3.0, the `**node-fetch**` library only supports ESM.

1. **When using the Corezoid Code editor**

If you write your code in the Code editor, GitCall automatically detects whether you are using CommonJS or ESM.

For **CommonJS**, write the following:

`require('...')` / `module.exports`

For ESM, write the following:

`import ...` / `export default ...`

2. **When using a Git repository**

1. If your main file has the `.mjs` extension, GitCall will treat it as an ESM module.

2. Alternatively, you can set `"type": "module"` in your `package.json`, in which case any `.js` files will also be handled as ESM.

3. If you want to continue using CommonJS, you can keep the standard `.js` extension and omit `"type": "module"`.

3. **CommonJS and ESM Use Examples**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

CommonJS use example

Build command: npm install node-fetch@2.7.0

```
const fetch = require('node-fetch');

module.exports = async (data) => {
   const response = await
fetch('https://jsonplaceholder.typicode.com/posts/1');
   data.result = await response.json();
   return data;
};
```

ESM module use example

Build command: npm install node-fetch

```
import fetch from 'node-fetch';

export default async (data) => {
   const response = await
fetch('https://jsonplaceholder.typicode.com/posts/1');
   data.result = await response.json();
   return data;
};
```

For more details, see the Git examples

For .mjs:

https://github.com/corezoid/gitcall-examples/tree/master/js/esm_import

For package.json type=module:

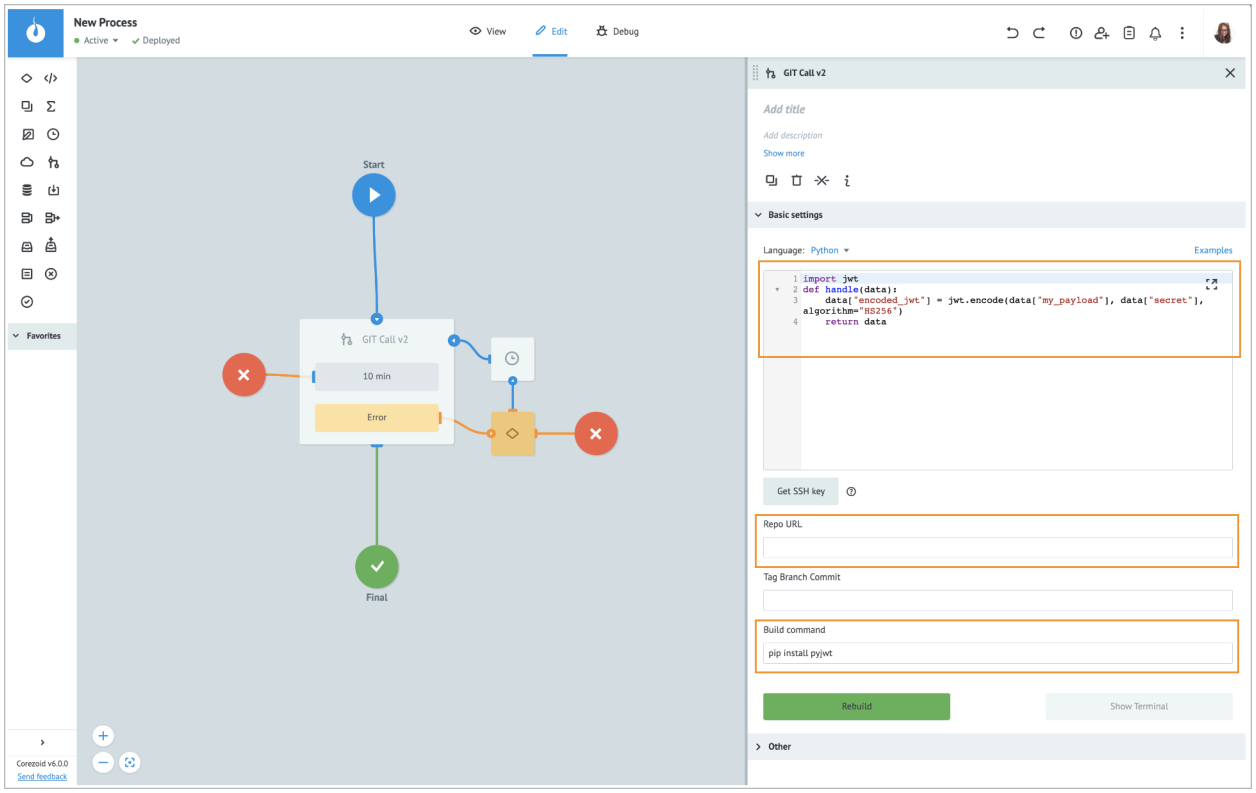
https://github.com/corezoid/gitcall-examples/tree/master/js/esm_module

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Examples

Accessing the Python library

Below, you can see the example of the **Git Call** node used to install a library and write code that uses it.



Screenshot 197: **Accessing the Python library**.

When writing your code:

- Place your code inside the handle function, which uses the data parameter containing the task parameters as an input.
- Since Python represents JSON objects as dictionaries, use dictionary syntax when accessing and manipulating the task parameters within the data variable.

See the GitHub example.

Using Dockerfile

An example of using Dockerfile in **Git Call** for Python:

1. Repository Structure:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
/your-repo
  ├── Dockerfile
  ├── src/
  │       └── main.py
  │       └── usercode.py
  ├── .gitignore
  ├── Dockerfile

  ├── requirements.txt
```

Schema 2: **Dockerfile repository structure**.

2. Dockerfile:

```
# Use Python Alpine as builder
FROM python:3.12-alpine as builder

# Set working directory
WORKDIR /app

# Copy and install requirements
COPY requirements.txt requirements.txt
RUN pip install -r requirements.txt

# Copy application code
COPY src src

# Create user with ID 501 (required by Git Call)
RUN addgroup --gid 501 usercode && \
    adduser --disabled-password \
    --gecos "" \
    --shell /usr/sbin/nologin \
    --ingroup usercode \
    --uid 501 \
    usercode

# Switch to usercode
USER usercode

# Set the entry point
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
ENTRYPOINT fastapi run src/main.py --host=0.0.0.0
--port="$GITCALL_PORT"
```

3. requirements.txt:

```
fastapi==0.111.0
```

4. main.py:

```
import time
import logging
from fastapi import FastAPI, Body
from usercode import handle

logger = logging.getLogger('uvicorn.error')
logger.setLevel(logging.DEBUG)

app = FastAPI()

@app.post("/")
def request(req = Body()):
    jsonrpc = req["jsonrpc"]
    id = req["id"]
    params = req["params"]
    try:
        logger.info("[req] time=%d id=%s" % (round(time.time() *
1000), id))
        result = handle(params)
        logger.info("[res] time=%d id=%s" % (round(time.time() *
1000), id))
        return {"jsonrpc": jsonrpc, "id": id, "result": result}
    except Exception as err:
        logger.info("[res] time=%d id=%s error=%s" %
(round(time.time() * 1000), id, str(err)))

        return {"jsonrpc": jsonrpc, "id": id, "error": {"code": 1,
"message": str(err)}}
```

5. usercode.py:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
# write your code here
def handle(data):
    data["python"] = "Hello, world!"

    return data
```

6. Local testing commands:

```
# Build the Docker image
docker build -t gitcall-stats .

# Run the container
docker run -p 8080:8080 -e GIT_CALL_PORT=8080 --read-only
gitcall-stats

# Test with curl
curl -X POST -H "Content-Type: application/json" -d '{
  "jsonrpc": "2.0",
  "method": "handle",
  "params": {
    "task_data": {
      "numbers": [1, 2, 3, 4, 5],
      "operation": "mean"
    }
  },
  "id": "1"

}' http://localhost:8080
```

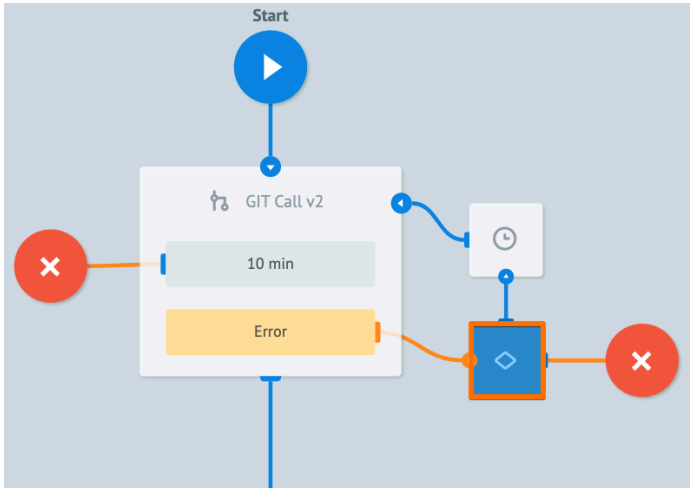
7. Git Call Setup:

- **Repository URL:** Your repository URL
- **Path to Dockerfile:** /Dockerfile.

Error handling & troubleshooting

When an error occurs in the **Git Call** node, a task goes to the auxiliary **Condition** output node that is used for storing error parameters.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 198: **Condition auxiliary node.**

When an error occurs during the task processing, you may see the following error parameter names in the task.

Error parameter name	Error type
`__conveyor_git_call_return_type_error__`	**Hardware** (system error), **Software** (error in a node logic/settings).
`__conveyor_git_call_return_type_tag__`	*Error tag.
`__conveyor_git_call_return_type_description__`	Error description in human-readable language; can be static or dynamic.

*The error tag `__conveyor_git_call_return_type_tag__` may have the following values.

Table 13: **Error parameter names.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Value	Cause	Solution
git_call_return_format_error	Incorrect response format from the **Git Call** node.	Contact the support team for further assistance.
git_call_executing_error	An incorrect code has been returned from the **Git Call** node.	Contact the support team for further assistance.
git_call_is_not_supported	The **Git Call** node you are using has the version 1, which is no longer supported.	Use the **Git Call** node version 2.
code_return_size_overflow	The size of your code exceeds the specified limit set for your environment.	Reduce the size of your code.
git_call_fatal_error	A hardware error has occurred.	Contact the support team for further assistance.

Table 14: **Git Call node errors.**

When working with your Process, you may encounter the following issues.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

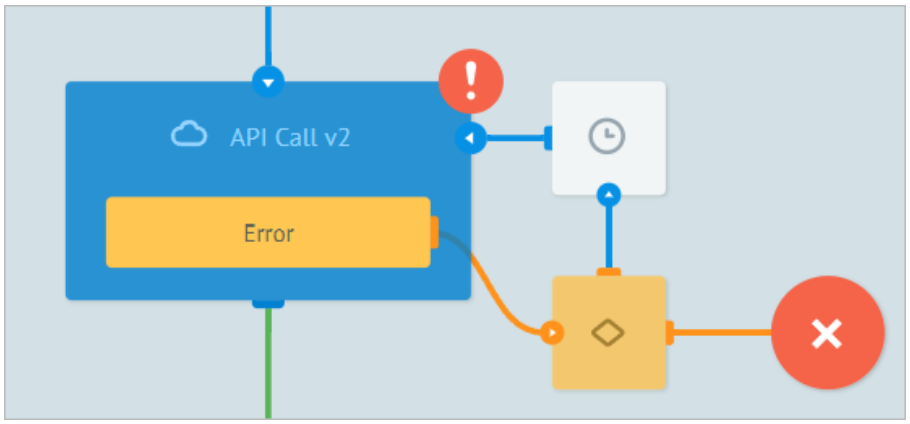
Issue	Cause	Solution
The build fails.	To use the repository, root access is required.	Corezoid does not permit access to the root directory. If your code requires root access, you need to modify it to work within the allowed directory structure or you can contact us to discuss custom solutions.
	The repository requires C/C++.	By default, Git Call does not support C/C++. If you need access to this feature, please contact our support team.
	There is no internet access.	**Git Call** requires internet access to retrieve code from an online repository. It doesn't function in a closed offline environment.

Table 15: **Git Call node issues.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3.8 API Call

Overview



Screenshot 199: **API Call node view**

The **API Call node** integrates external systems via APIs, enabling Processes to retrieve or send data to various platforms, including social media (e.g., LinkedIn, Facebook), banking services (e.g., Visa, Mastercard), and general web services. The node supports both dynamic and static parameter configurations.

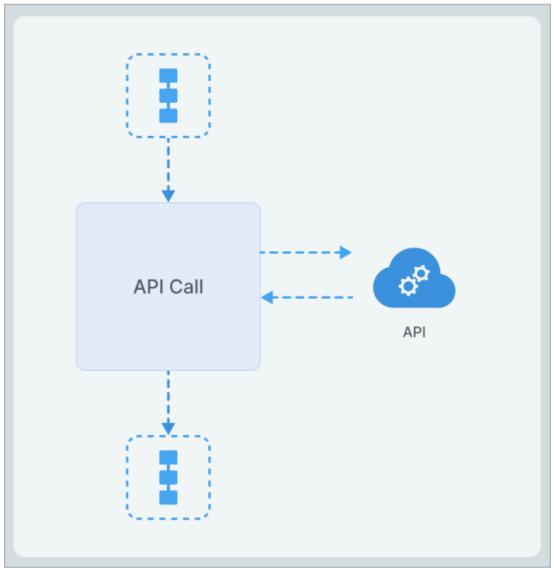


Diagram 12: **API Call node flow**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

All API calls are made from the following IP addresses:

- 54.154.124.100
- 54.154.124.224

If you encounter access errors when calling databases, you might need to white-list these addresses.

You can use the API Call for weather data retrieval, payment verification, customer feedback analysis and other cases. Utilizing Header Parameters (see **Node Settings**) allows you to use the node in the following cases:

1. Authentication
    a. Sending API keys or tokens (e.g., Authorization: Bearer <token>).
    b. Basic authentication credentials.
    c. Custom authentication headers required by specific APIs.
2. Content Negotiation
    a. Specifying accepted response formats (Accept: application/json).
    b. Defining the request content type (Content-Type: application/json).
    c. Character set specifications (charset=utf-8).
3. API Versioning
    a. Specifying API versions (e.g., api-version: 2.0).
    b. Indicating which version of the API you want to use.
4. Request Metadata
    a. Tracking IDs for request tracing.
    b. Client identification information.
    c. Session tokens
5. Caching Controls
    a. Cache-control directives.
    b. ETag headers for resource versioning.
    c. Last-modified information.
6. Security Headers
    a. Cross-Origin Resource Sharing (CORS) headers.
    b. Security tokens.
    c. Custom security-related headers.
7. Language and Localization
    a. Preferred language (`Accept-Language`).
    b. Timezone information.
    c. Regional preferences.
8. Custom Headers
    a. API-specific custom parameters.

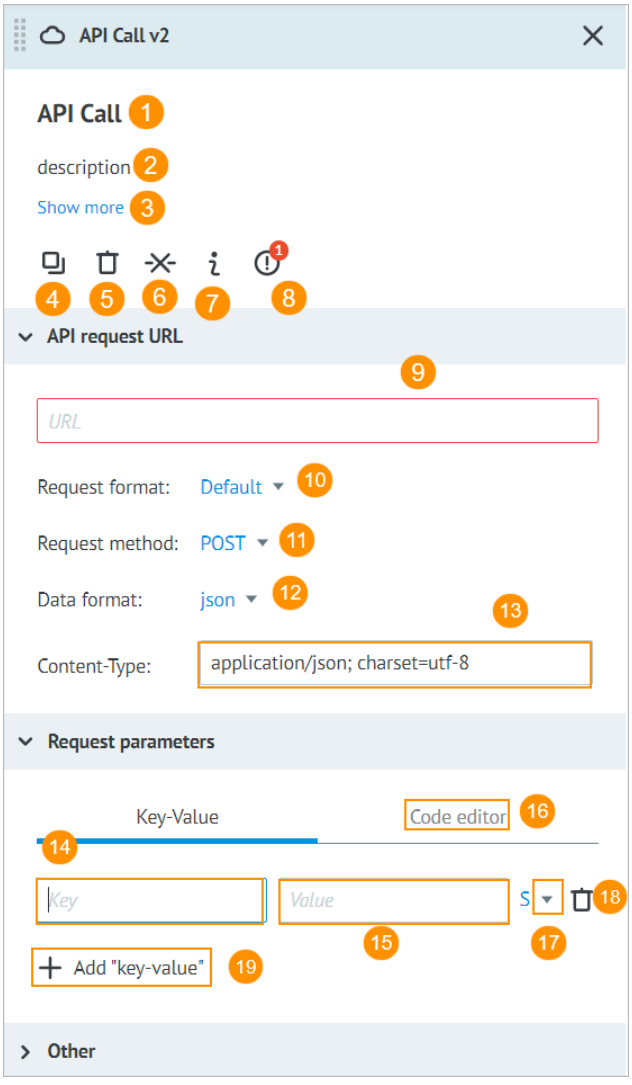
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

b. Platform-specific identifiers.
c. Application-specific metadata.

Node Settings

The **API Call** node has the following parameters:

Basic settings



2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Screenshot 200: **API Call node basic settings**.

1. **Title** [Optional] [Field] [String]**:** Node title.
2. **Description** [Optional] [Field] [String]: Node description**.**
3. **Show more** [Optional] [Action]: Long descriptions are reduced to a compact view to simplify node settings. Click to see the whole description.
4. **Copy node** [Optional] [Action]: Creates a copy of the current node with all settings**.**
5. **Remove node** [Optional] [Action]: Deletes current node from Process.
6. **Remove links** [Optional] [Action]: Removes all connections to/from current node.
7. **Info** [Optional] [Action]: Displays node ID.
8. **Errors info** [Optional] [Action]: Displays node errors.
9. **URL** [Required] [Field] [String]: The endpoint for the API Call specified in one of the formats:
    - **URL**: for example, https://account.corezoid.com/api/1
    - **Variable**: for example, {{url}}.
    - **Construction**: for example {{conv[state diagram id].ref[ref].url}}
10. **Request format** [Required] [Action]: Selects the request format:
    - **Default**: Sets standard HTTP/HTTPS request format with a Key-Value editor for query parameters and a request method dropdown menu.
    - **Corezoid**: Same as the **Default** format, but with no request method selection.
    - **Raw**: A free-form request format that allows you to write custom request bodies. Use this format when an API follows a non-standard request format.
11. **Request method** [Required] [Action]: HTTP methods supported:
    - **POST**: Send data.
    - **GET**: Retrieve data.
    - **PUT**: Create or update existing data.
    - **DELETE**: Delete data.
    - **HEAD**: Similar to POST, but with no response body.
    - **PATCH**: Update only a part of the data.

Note: Some APIs have the same endpoint for different request methods.

12. **Data format** [Required] [Action]: Select the format for sending payload data:
    - JSON
    - XML
    - Form

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

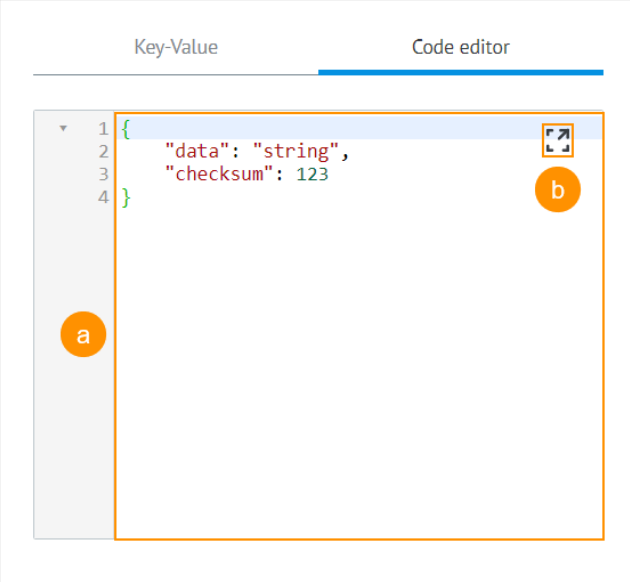
13. **Content-Type** [Required] [Field] [String]: Indicates the format of the data sent in the request body. The field is filled out automatically based on the data format that you have selected. Default types based on the data format:

- **JSON**: application/json; charset=utf-8.
- **XML**: application/xml; charset=utf-8.
- **Form**: application/x-www-form-urlencoded; charset=utf-8.

14. **Key** [Required] [Field] [String]: Key to specify a static name or a variable in {{}} curly brackets.

15. **Value** [Required] [Field] [Any]: Specify the value you want to assign to the Key-named parameter. See data types descrition for more details.

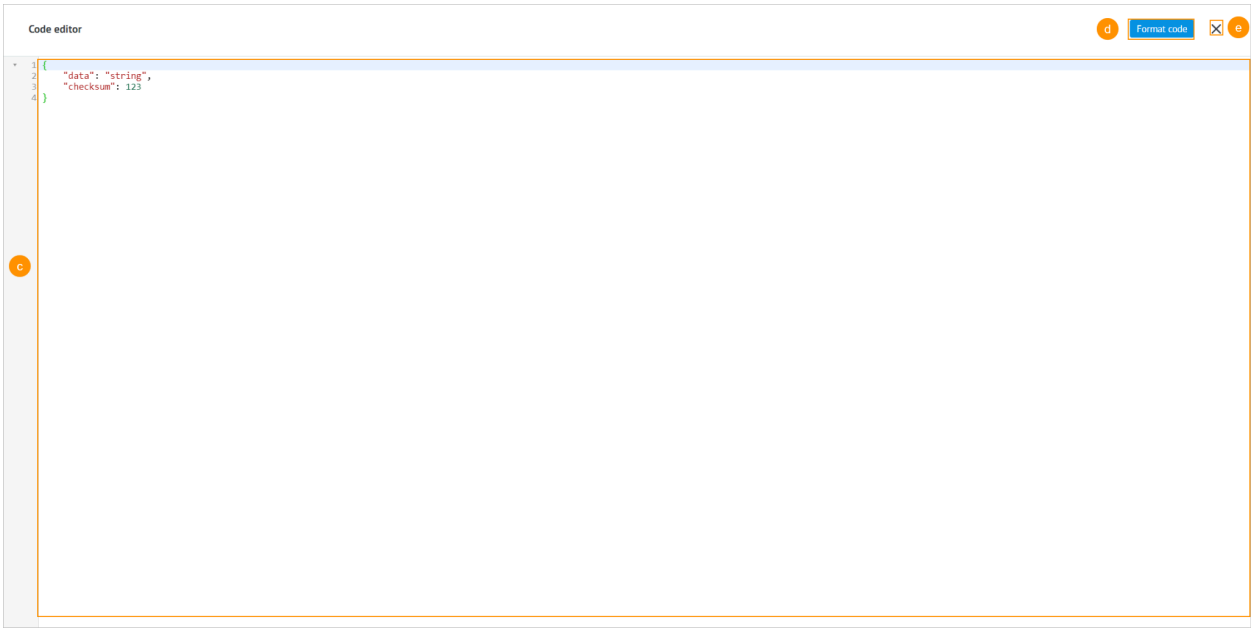
16. **Code editor** [Optional] [Action]: Click **Code editor** to open the code editor tab, where:



Screenshot 201: **Code editor tab.**

a. **Code editor field** [Optional] [Field] [Any]: Enter new keys and values or edit the existing ones in JSON format.
b. **Full-screen mode** [Optional] [Action]: Opens the **Code editor** field in the full-screen mode, where:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 202: **Code editor full-screen view.**

c. **Code editor field** [Optional] [Field] [Any]: Edit or add keys in values.
d. **Format code** [Optional] [Action]: Format your current code.
e. **Quit the full-screen mode** [Optional] [Action]: Quit the full-screen mode.

17. **Type of data** [Required] [Action]: Selects the data type for the **Value**:
    * String (**S**)
    * Number (**N**)
    * Boolean (**B**)
    * Array (**A**)
    * Object (**O**)

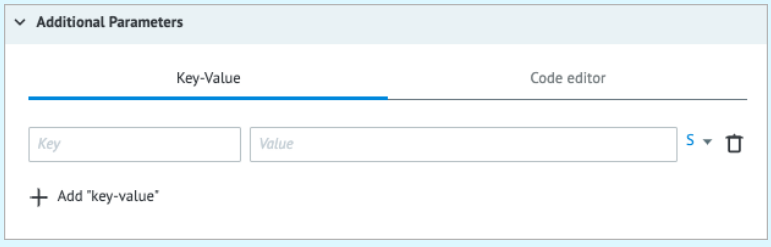
    Note: For more information, go to Data types.

18. **Delete key-value** [Optional] [Action]: Delete the key-value pair.

19. **Add key-value** [Optional] [Action]: Add a new key-value pair.

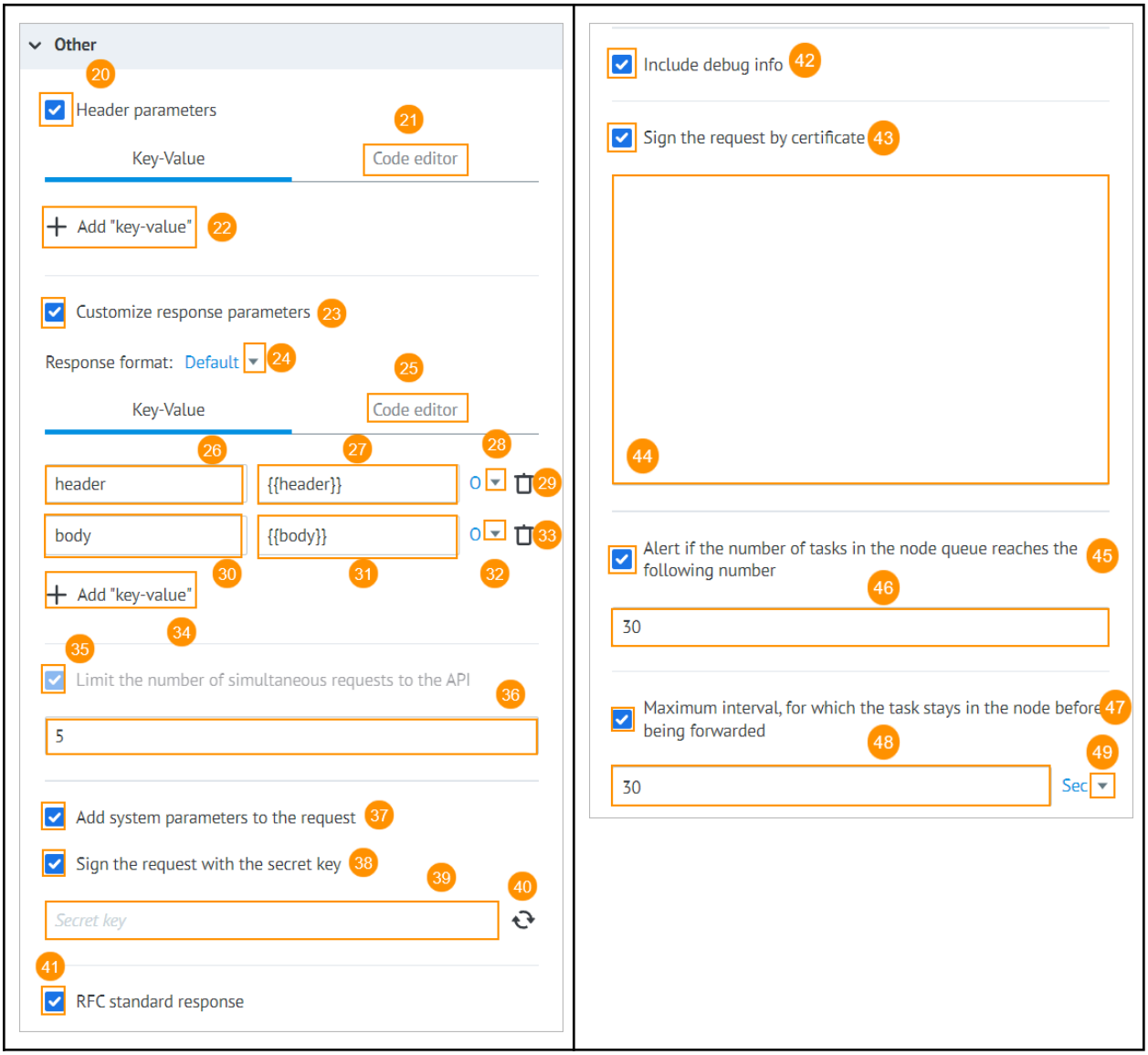
- To add a key-value, click **+ Add "key-value"** below the **Key** field.
- To delete the key-value line, click the trash icon on the right side of the key-value line.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 203: **Adding a key-value pair.**

Other settings



2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Screenshot 204: **API Call other parameters.**

20. **Header parameters** [Optional] [Action]: Select to specify header parameters for API requests providing additional information (authentication parameters, metadata, etc.). Use the same **Key-Value** format as in the **Request Parameters**.

21. **Code editor** [Optional] [Action]: Open the Code editor.

22. **Add key-value** [Optional] [Action]: Add a new key-value pair to the header parameters.

23. **Customize response parameters** [Optional] [Action]: Select to modify the response header and body parameters and specify the response format before the task is released from the node. Use standard icons to add parameters' keys and values.

You can use this feature to work with APIs that provide an incorrect response format or do not specify a format. By customizing the response parameters, you can ensure that the response is parsed correctly.

Also, it can be used to minimize the amount of data added to a task. When an API request returns a lot of data, use Customize response parameters to extract only the needed data.

24. **Response format** [Required] [Action]: Select the format for the response:
    * Default
    * Application/Json
    * Application/X-Www-Form-Urlencoded
    * Application/Xml
    * Text/Xml

25. **Code editor** [Optional] [Action]: Open the Code editor.

26. **Header key** [Required] [Field] [String]: The key preset to “header” to enable the processing of response header data from an external API and its inclusion in the task.
27. **Header value** [Required] [Field] [Any]: The key preset to “{{header}}” to enable the processing of response header data from an external API and its inclusion in the task.
28. **Type of data** [Required] [Action]: Selects the data type for the response header value:
    * String (**S**)
    * Number (**N**)
    * Boolean (**B**)
    * Array (**A**)
    * Object (**O**) selected by default.

29. **Delete key-value** [Optional] [Action]: Delete the key-value pair.
30. **Body key** [Required] [Field] [String]: The key preset to “body” to enable the processing of response body data from an external API and its inclusion in the task.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

31. **Body value** [Required] [Field] [Any]: The key preset to “{{body}}” to enable the processing of response body data from an external API and its inclusion in the task.
32. **Type of data** [Required] [Action]: Selects the data type for the response body value:
    * String (**S**)
    * Number (**N**)
    * Boolean (**B**)
    * Array (**A**)
    * Object (**O**) selected by default.
33. **Delete key-value** [Optional] [Action]: Delete the key-value pair.

34. **Add key-value** [Optional] [Action]: Add a new key-value pair to the header parameters.
35. **Limit the number of simultaneous requests to the API** [Required] [Action]: Limit the number of concurrent requests made to the API.
36. **Number of simultaneous requests to the API** [Required] [Field] [Number]: Specify the maximum number of concurrent requests made to the API. Set to 5 by default.

37. **Add system parameters to the request** [Optional] [Action]: Add Corezoid system parameters to an API request required to make the request to the Corezoid API. Parameters can be the following: **conv_signature**, **conv_time**, and **conv_id**. For more information, go to Corezoid API.
Warning (color): Some APIs have strict requirements for the parameters that can be included in a request. If this is the case for the API you are using, you may need to clear the checkbox to exclude Corezoid system parameters from your request.

38. **Sign the request with the secret key** [Optional] [Action]: Select to sign requests from the node with the secret key you can specify in the field that appears. You can use it to send requests to Corezoid API, which requires secret key authentication.
Note (color): To include the necessary system parameters along with the request, you have to select the **Add system parameters to the request** option.

39. **Secret key** [Optional/Required if **Sign the request with the secret key** is selected] [Field] [String]: Specify the secret key.

40. **Refresh the secret key** [Optional] [Action]: Refresh the secret key.
41. **RFC standard response** [Optional] [Action]: Select to Process the response from an external API according to the RFC standard.
Note (color): When this option is selected, the response body is mandatory for response codes 200 and 201 and is optional for the 202 response code.

42. **Include debug info** [Optional] [Action]: Adds the API request extra information to the conveyor_api_debug task parameter that includes execution time, request body size, response code, and response body size.

```
"__conveyor_api_debug__": {
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
    "http_exec_time": 328.552,
    "http_req_body_size": 0,
    "http_res_code": 200,
    "http_res_body_size": 1132
}
```

Parameter	Description
conveyor_api_debug	API debug info
http_exec_time	Request execution time
http_req_body_size	Request body size
http_res_code	Request code
http_res_body_size	Response body size

Table 16: Conveyor debug parameters.

43. **Sign the request by certificate** [Optional] [Action]: Select to add a certificate in the PEM format. Use this option for APIs that require a client certificate to be included in your request.

```
-----BEGIN RSA PRIVATE KEY-----

....your private key goes here....

-----END RSA PRIVATE KEY-----

-----BEGIN CERTIFICATE-----

....your certificate goes here....

-----END CERTIFICATE-----
```

44. **Certificate input field** [Optional/Required if **Sign the request by certificate** is selected] [Field] [String]: Input the certificate to sign the request with.

45. **Alert if the number of tasks in the node queue reaches the following number** [Optional] [Action]: Helps monitor whether the number of tasks in the node exceeds the specified threshold.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

46. **Number of tasks in the queue** [Optional/Required if **Alert if the number of tasks in the node queue reaches the following number** is selected] [Field] [Number]: Number of tasks in the queue which triggers an alert when reached.

47. **Maximum interval, for which the task stays in the node before being forwarded** [Optional] [Action]: Select to limit the time a task is allowed to be in the node in seconds, minutes, hours, and days.

48. **Maximum interval** [Optional/Required if **Maximum interval, for which the task stays in the node before being forwarded** is selected] [Field] [Number]: The amount of time a task is allowed to be in the node. It has a minimum value of 30 seconds. You can set a shorter interval by using the **Unixtime** function.

49. **Maximum interval measuring unit** [Required] [Action]: Selects the measuring unit for the maximum interval:

- Seconds
- Minutes
- Hours
- Days.

Example Usage

There are innumerable ways in which companies choose to implement their APIs:

- Using different data formats, including JSON and XML as the most popular ones
- Applying certificates
- Getting an API key or token
- Generating an authentication code and periodically renewing their access token

Note: There are no commonly used examples that can cover all the possible features.

API call to OpenWeatherMap
OpenWeatherMap is a popular free API that you can use to check the weather at any location. To check the weather in San Francisco:

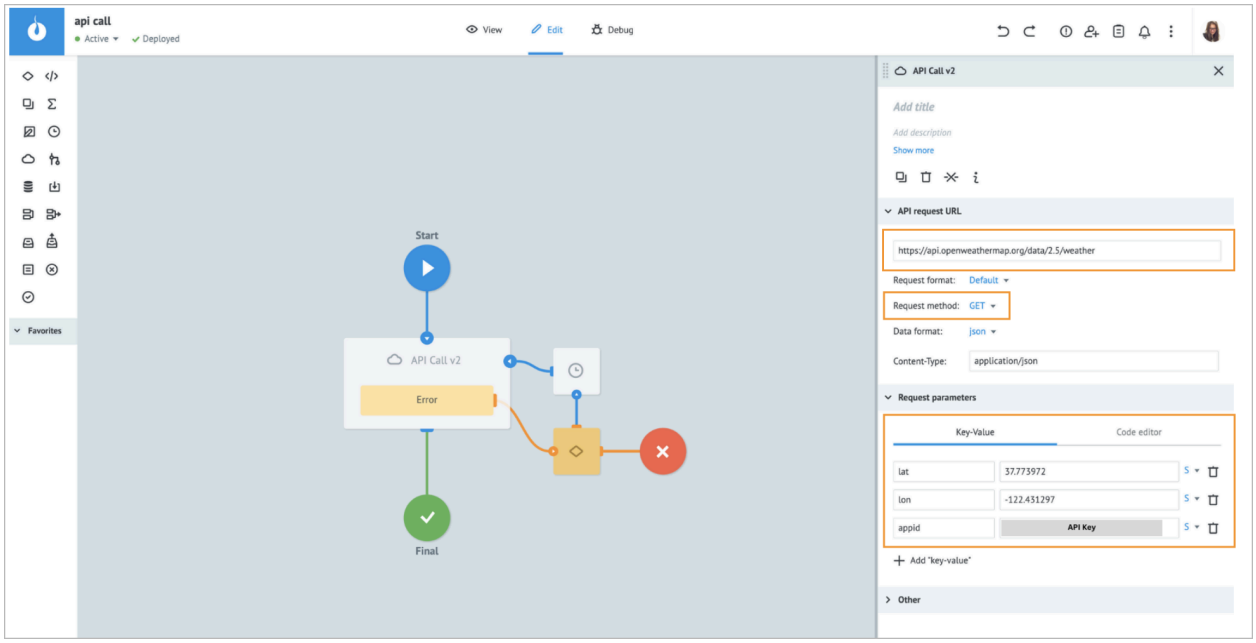
1. Create a Process and add the API Call node.
   Note: For more information, go to Create Process and Add node.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

2. Go to OpenWeatherMap and get the needed API where:
    a. https://api.openweathermap.org/data/2.5/weather is the endpoint URL.
    b. **lat**, **lon**, and **appid** are parameters.
3. The **appid** parameter is a unique API key that you get by creating an account with the service.

```
https://api.openweathermap.org/data/2.5/weather?lat={lat}&lon={lon}&appid={API key}
```

4. In **API request URL** of the **API Call** node details panel:
    a. In the **URL** field, enter the endpoint URL.
    b. In the **Request method** dropdown list, select **GET**.
5. In **Request parameters** of the **API Call** node details panel, enter the **lat**, **lon**, and **appid** parameter values.

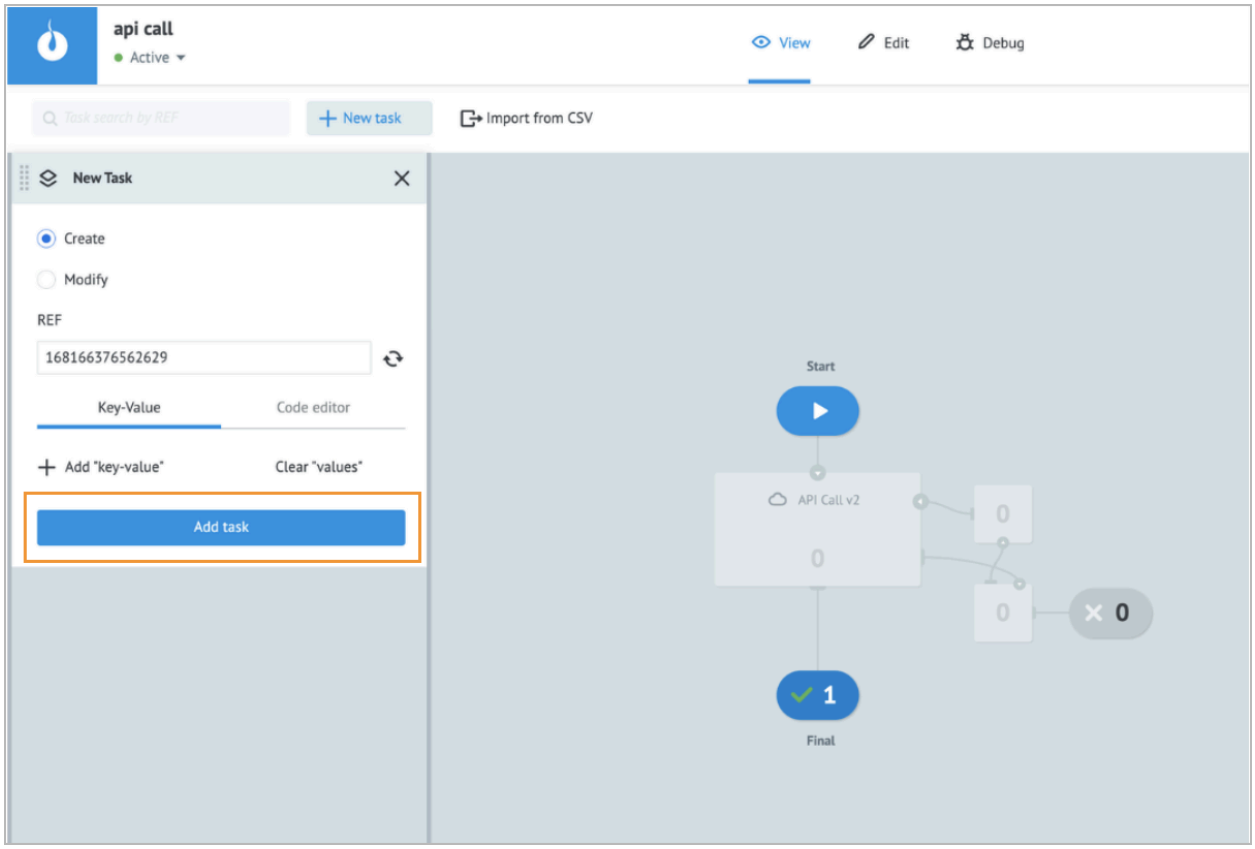


Screenshot 205: **Enter parameter for the GET request method**.

5. Switch to the View mode by clicking **View** on the upper panel.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

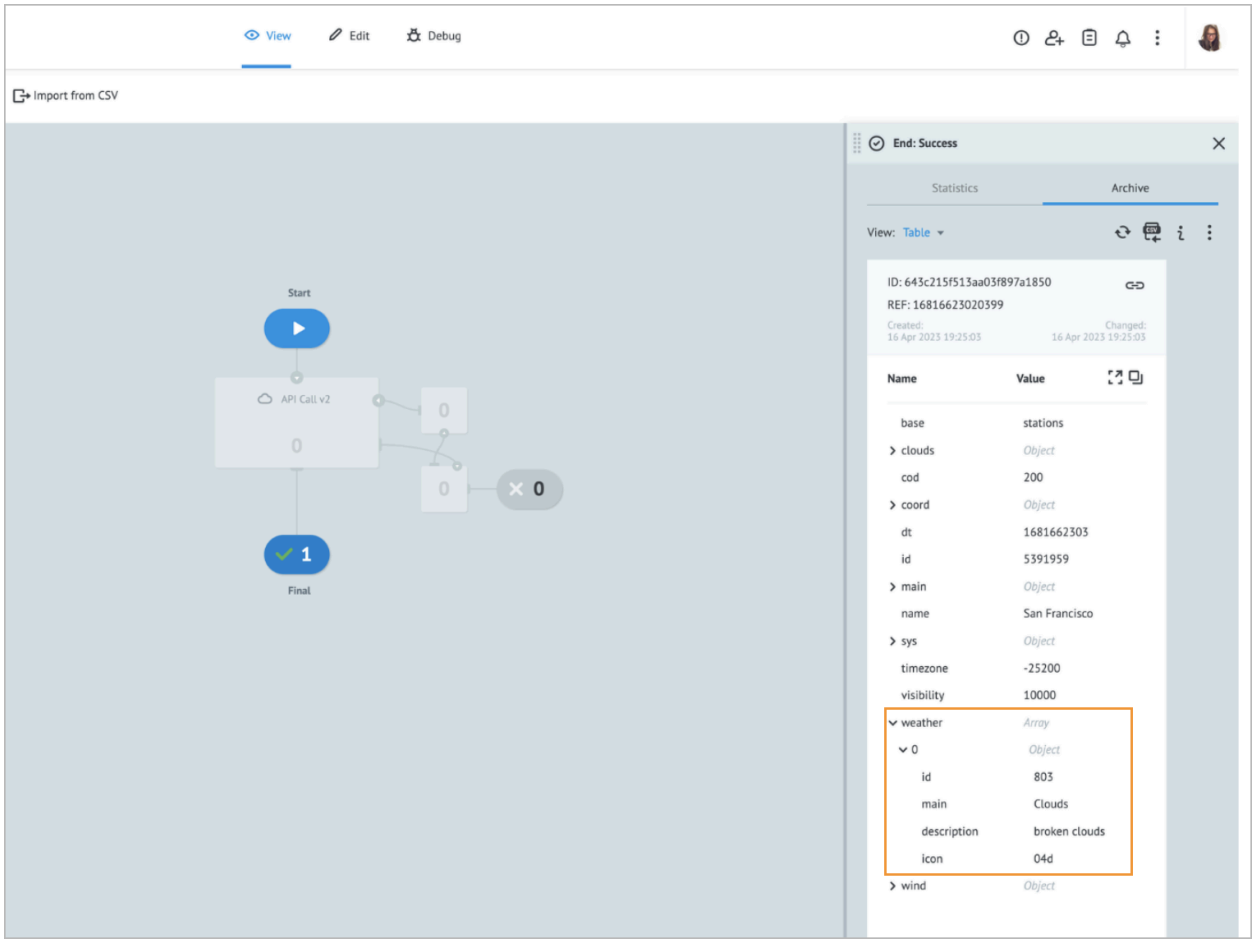
6. In the **New Task** dialog that opens, click **Add task**.



Screenshot 206: **Add the task**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

On the task details form, you can see that it's cloudy in San Francisco now.



Screenshot 207: **Check the weather in San Francisco**.

Best Practices

- **Secure API Keys**: Use environment variables or secure storage for sensitive data like API keys or tokens.
- **Optimize Timeouts**: Set appropriate timeout values to prevent blocking workflows unnecessarily.
- **Error Management**: Implement robust error handling strategies, such as retries and failover mechanisms, for critical operations.
- **Logging and Monitoring**: Ensure logs capture sufficient detail for diagnosing integration issues.

By effectively leveraging the API Call node, you can extend Corezoid Processes to interact seamlessly with external systems, enhancing automation and connectivity.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Error Handling and Troubleshooting

(current table under spoiler)

Error parameter name	Error type
`__conveyor_api_return_type_error__`	**Hardware** (system error), **Software** (error in a node logic/settings).
`__conveyor_api_return_type_tag__`	Error tag.
`__conveyor_api_return_type_description__`	Error description in human-readable language; can be static or dynamic.

Table 17: **Error parameter names.**

*The error tag `__conveyor_api_return_type_tag_` may have the following values.

Value	Cause	Solution
access_denied	You don't have access to the Process you are trying to call.	Contact the Process owner to get access.
access_denied	You are trying to send or receive tasks from another stage.	It's not allowed to get and send tasks between stages. Use the same stage to send and receive tasks.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

access_refused	The Process you are working in is not a State Diagram but it must be a State Diagram.	Check the logic of using a State Diagram. For more information, go to State Diagram.
api_expire_query	Your API query has expired: a) The API has taken too long to respond. b)The number you specified in the **Limit the number of simultaneous requests to the API** field has been exceeded.	Check possible solutions with the API provider.
api_bind_error	The request parameters you defined in the **Key** and **Value** fields could not be added to the URL to form a request.	Make sure the parameters are formatted correctly.
api_bad_answer: timeout or failed_connect	The **API Call** node did not receive a response from API or failed to establish a connection with it after 60 seconds.	Check possible solutions with the API provider. Make sure the URL you provided is correct. Corezoid will reattempt to send a request every 30 seconds.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

api_conv_not_active	The Process associated with an **API Call** node is either inactive or in a debug state.	Ensure the Process is active and not blocked before initiating the **API Call** node.
api_no_valid_json	The request header or body is formatted incorrectly.	Correct the request header or body format.
api_no_ops	The API response does not contain an ops block or contains an incorrect ops block.	For more information about the ops blocks, go to Corezoid API.
wrong_validate_params value when using the **Customize response parameters** option	The variable that the **API Call** node is trying to access has not been sent to the API or has a different type than the specified one.	Make sure the parameters you are trying to modify exist, or change the parameter type.
api_bad_answer: no_scheme	The **URL** field is empty.	The **URL** field can be empty when it refers to a variable with no value. Make sure to provide a valid URL.
api_no_request_proc	Incorrect tag “request_proc” is set.	Ensure that you specify "request_proc=ok" for your response. If you want to send an error, use "proc!=ok".

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

api_wrong_convert_param	Parameter value failed to convert with the specified parameter type.	Check and correct the data type of the parameter specified in the error message.
api_bad_res_data	The data format you are trying to send to the API is incorrect.	For more information about the correct format, go to Corezoid API.
not_found_conv_by_alias	Your alias is linked to the Process that no longer exists.	Use an existing Process.
api_task_size_overflow_limit	The size of the task data exceeds the specified limit set for your environment.	Reduce the size of your task.
api_size_overflow_limit	The size of the task data exceeds the specified limit set for your environment.	Reduce the size of your task.
api_bad_certificate	The API you are trying to connect to requires a certificate. Your certificate is missing or invalid.	Contact the API developer to get the certificate.
env_var_not_found	The variable or alias you are trying to connect doesn't exist.	Check the variable or alias or use another one that is in your list.
bad_callback_node	The Process you are calling doesn't have the **Callback** node.	Create the **Callback** node in the Process you are calling.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

api_no_valid_xml	The API you are trying to call does not correspond to the format selected in the **API Call** node.	Contact the API developer for further assistance.
api_no_valid_nvp	The API you are trying to call does not correspond to the format selected in the **API Call** node.	Contact the API developer for further assistance.
api_max_http_resp_size_exceed	There is a large response from the external API that exceeds the limit of the task.	Contact your administrator to increase the task's allowed size.
api_bad_url_error	The URL is invalid or doesn't exist.	Check whether the URL is correct.
bad_url_by_rfc	The URL is invalid or doesn't exist.	Check whether the URL is correct.
api_blocked_url	You can't call an API in this domain because it is blocked by Corezoid.	Contact your administrator for further assistance.
api_bad_header_error	The external API replies with an invalid header as it contains ASCII characters.	Check your external API header or contact API developers for further assistance.
api_fatal_error	There was an internal Process error while executing the API request.	Contact the support team for further assistance.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

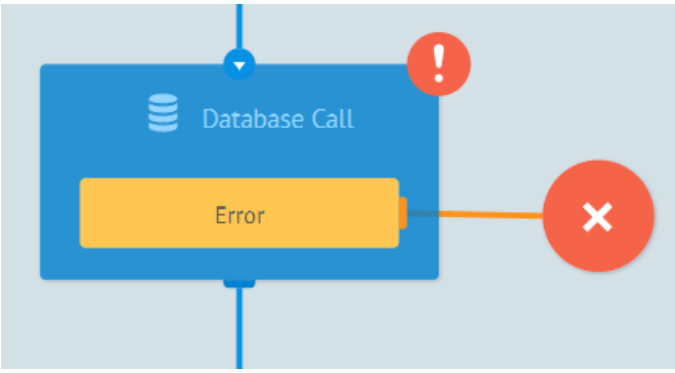
api_no_valid_json	Not valid http result json: The server did not return a valid JSON response.	Adding the **“te: trailers”** header ensures that the client can properly handle trailers (additional data in the response), preventing connection errors and allowing the request to complete successfully.
		Example:
		
		{
		"method": "GET",
		"url": "https://api.twitter.com/2/users/by/username/yevaaks",
		"headers": {
		"Authorization": "Bearer your_token",
		"te": "trailers"
		},
		}

Table 18: **API Call errors.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3.9 Database Call

Overview



Screenshot 208: **Database Call node view**

The **Database Call** node lets you write and execute SQL queries directly within Corezoid, removing the need for writing custom APIs to access databases. With the **Database Call** node, managing and accessing data in your databases becomes straightforward and streamlined. Supported database management systems (DBMS):

- MySQL
- Microsoft SQL Server
- PostgreSQL
- Oracle
- MongoDB

To use the **Database Call** node, first, you need to connect your database to Corezoid.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

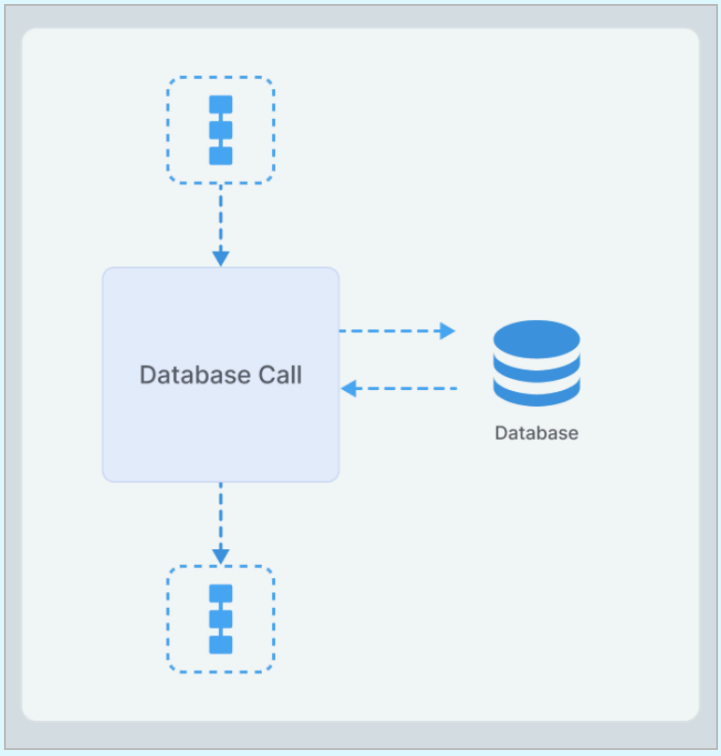


Diagram 13: **Database Call node flow**.

All Database calls are made from the following IP addresses:

- 54.171.15.37/32
- 108.128.68.222/32
- 63.33.226.230/32

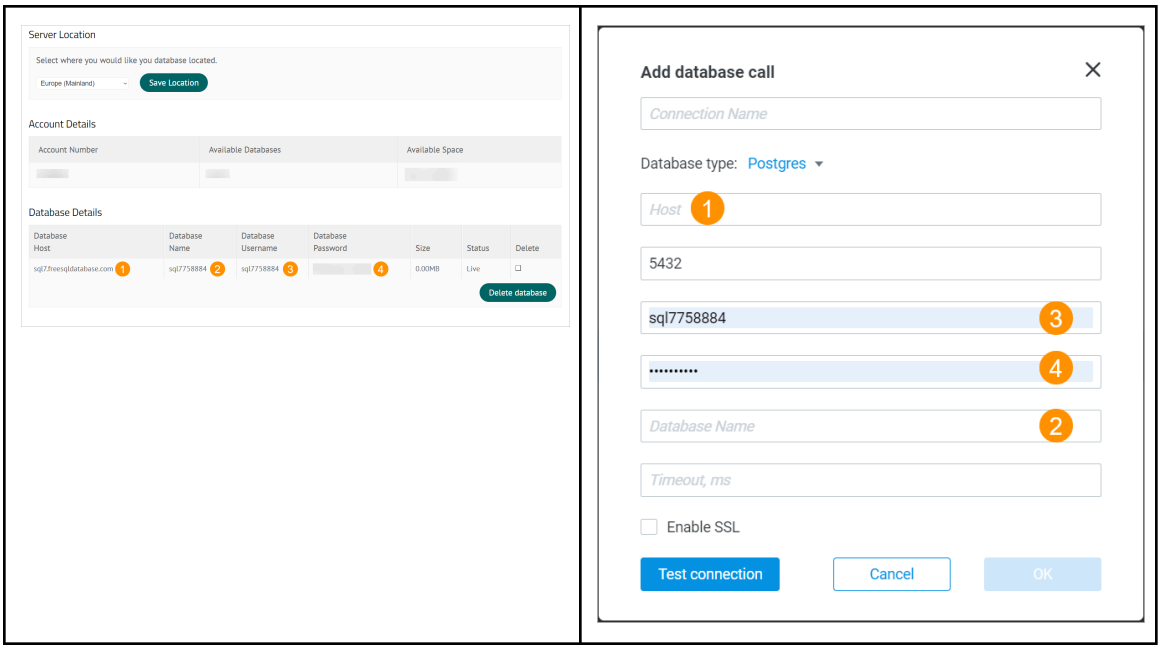
If you encounter access errors when calling databases, you might need to white-list these addresses.

Preparing to Work with the Database Call

1. Create a database you want to use with the Database Call.
2. Save your DB parameters to use when creating a connection in Corezoid:

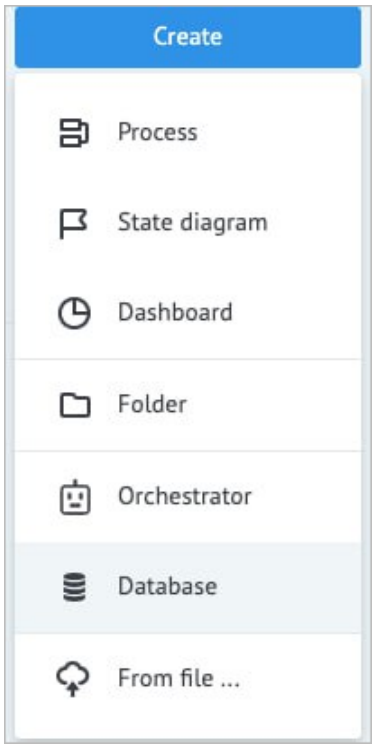
DBMS account example	Corresponding parameters in Create database window in Corezoid

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 209: **DB parameters example for Add database call window.**

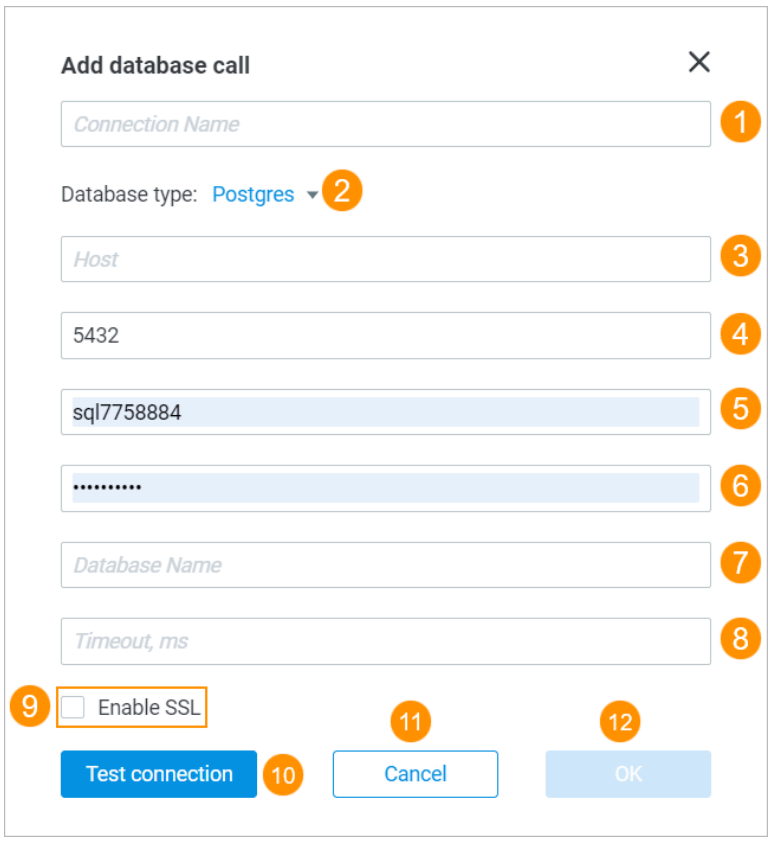
3. **In Corezoid, create a database connection** by clicking **Create-Database**.



Screenshot 210: **Create a database**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

4. **Fill in all required fields:** specify all the required parameters, such as database name, users, passwords, etc. and select the database type in the drop-down list:



Screenshot 211: **Add connection for the Database Call.**

1. **Connection Name** [Required] [Field] [String]: Connection name.

2. **Database type** [Required] [Action]: Select database type:

- Postgres
- Oracle
- MongoDB
- MySQL
- MSSQL

3. **Host** [Required] [Field] [String]: Database server address.

4. **Port** [Required] [Field] [String]: Connection port. Set to 5432 by default when PostgreSQL in the Database type field.

5. **Database username** [Required] [Field] [String]: database user name.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

6. **Database password** [Required] [Field] [String]: connection password.

7. **Database Name** [Required] [Field] [String]: database name.

8. **Timeout, ms** [Required] [Field] [String]: connection timeout in milliseconds (required).

9. **Enable SSL** [Optional] [Action]: SSL encryption option.

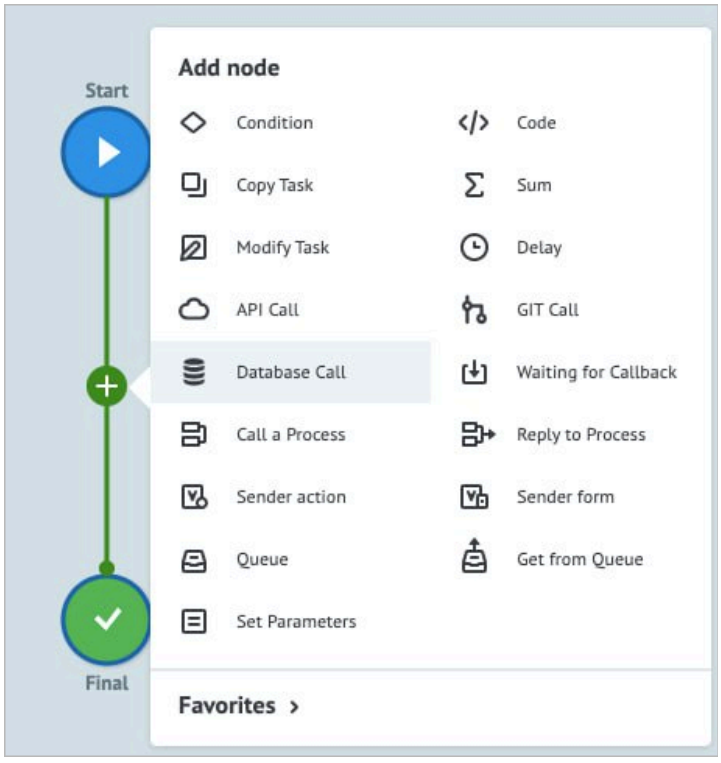
10. **Test connection** [Required] [Action]: button to verify connection.

11. **Cancel** [Optional] [Action]: cancel the connection configuration saving.

12. **OK** [Required] [Action]: save the connection configuration.

After you've successfully tested the connection to your database, click **OK** to save the connection configuration to the folder you select.

3. **Create a process:** create a new Process in your system and add the **DB Call** node to it.



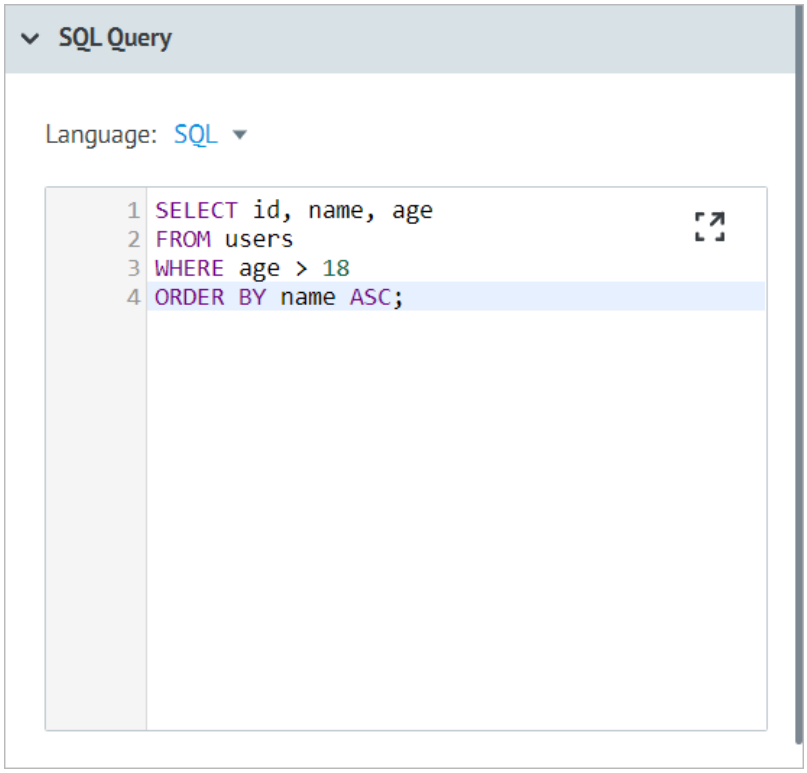
Screenshot 212: **Add the Database Call node.**

4. **Connect the node to the database:** link the **DB Call** node to your database by navigating to the connection parameters in the **Connection** field.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Screenshot 213: **Link the Database Call node to your database.**

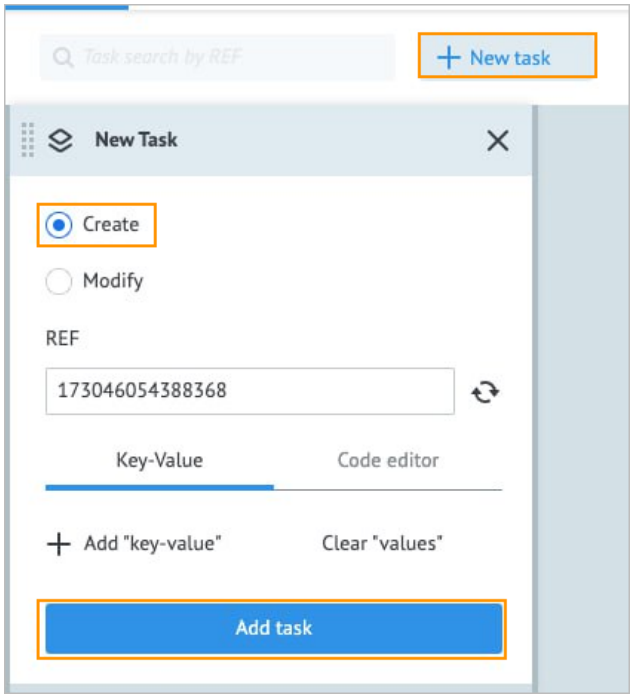
5. **Execute a query:** configure a query in the node. For example, select the **SQL** in the **Language** field and enter a query to retrieve information from your database.



Screenshot 214: **Enter an SQL query.**

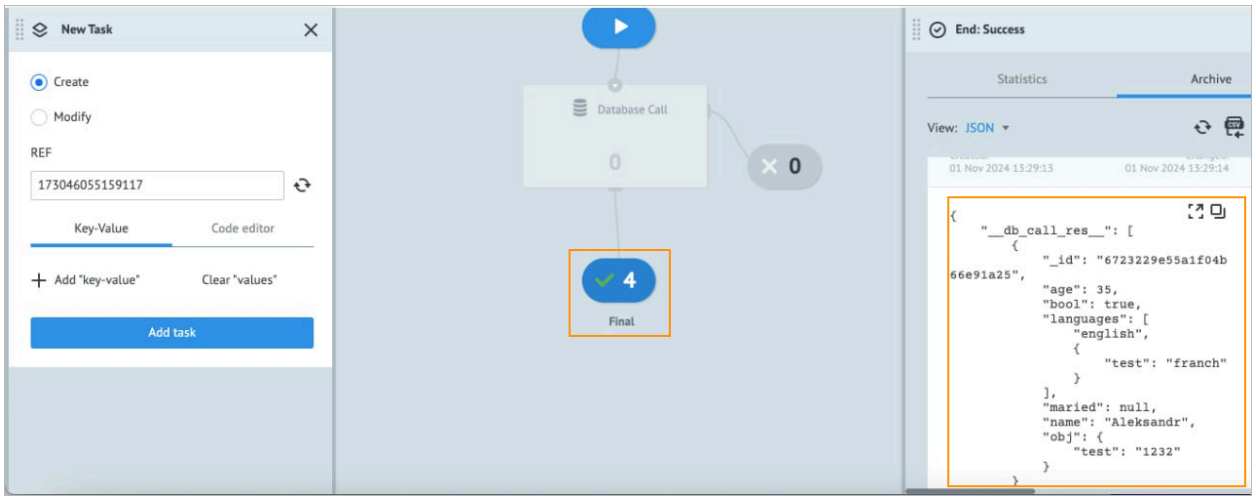
6. **Save changes and switch to the View mode:** Click **Deploy** to apply all changes and switch to the **View** mode.
7. **Send an empty task to your Process to check its operation:** In the **View** mode, click **+New task**, select **Create**, and click **Add task** to send a new empty task to your Process.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 215: **Sending an empty task.**

Check whether your task in the final node contains information retrieved from the database.



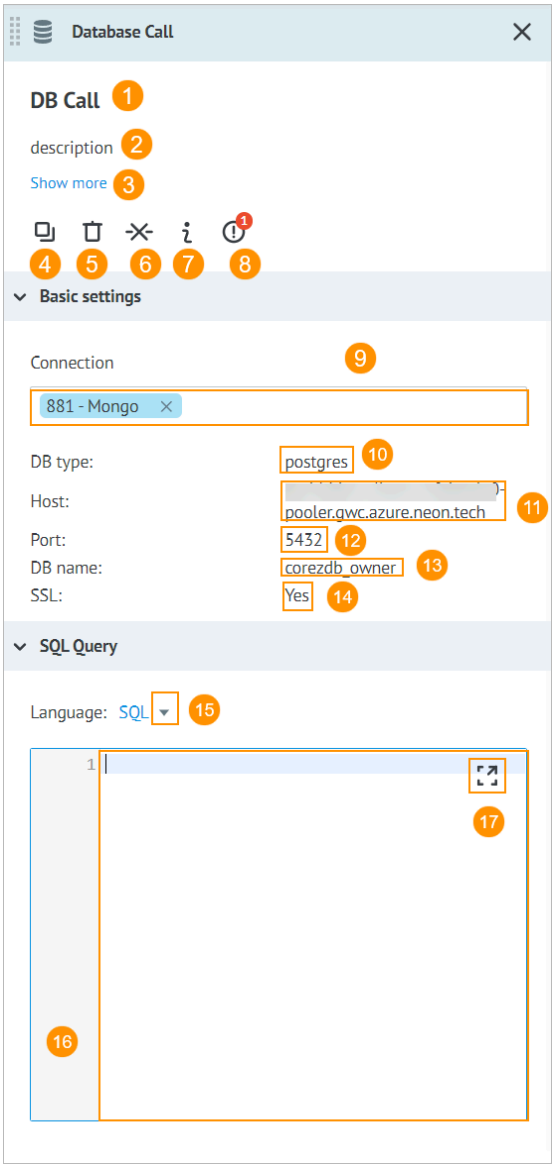
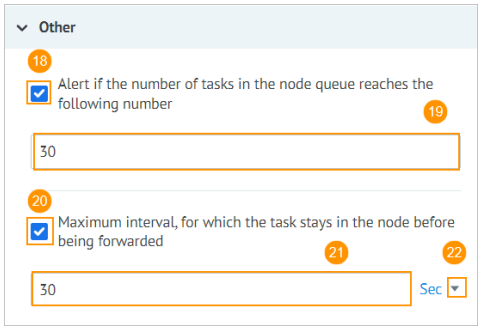
Screenshot 216: **Viewing the task in the Final node.**

After checking your Process operation, you can configure and use your Database Call node.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Node Settings

The **Database Call** node has the following parameters:

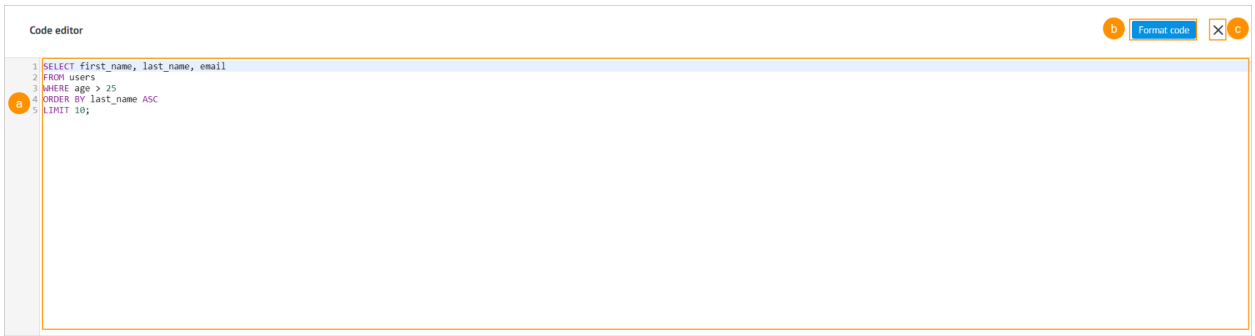
Basic settings	Other settings
	

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Screenshot 217: **Database Call settings.**

1. **Title** [Optional] [Field] [String]**:** Node title.
2. **Description** [Optional] [Field] [String]: Node description**.**
3. **Show more** [Optional] [Action]: Long descriptions are reduced to a compact view to simplify node settings. Click to see the whole description.
4. **Copy node** [Optional] [Action]: Creates a copy of the current node with all settings**.**
5. **Remove node** [Optional] [Action]: Deletes current node from Process.
6. **Remove links** [Optional] [Action]: Removes all connections to/from current node.
7. **Info** [Optional] [Action]: Displays node ID.
8. **Errors info** [Optional] [Action]: Displays node errors.

9. **Connection** [Required] [Action]: Selects the connection to your database. Click the directory icon and navigate to the saved connection configuration that you've created in **Preparing to Work with the Database Call.**
   Note: For more information about how to connect your database to Corezoid, go to Examples.
10. **DB type** [Required][Field][String]: Shows the selected DB type.
11. **Host** [Required][Field][String]: Shows the selected DB host.
12. **Port** [Required][Field][String]: Shows the DB port.
13. **DB name** [Required][Field][String]: Shows the selected DB name.
14. **SSL** [Required][Field][String]: Shows whether the SSL encryption is on.
15. **Language** [Required] [Action]: Selects the language of queries to the selected database. For now, only SQL is available.
16. **SQL Query** [Required] [Field]: Input your SQL query.
    Note: To access task parameters, use curly braces: {{some_parameter}}. Ensure that the query result does not surpass the maximum task size limit.
17. **Code editor** [Optional] [Action]: Click **Code editor** to open the code editor with your SQL request, where:



Screenshot 218: **Code editor full-screen mode.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

a. **Code editor field** [Optional] [Field] [Any]: Edit or add keys in values.
b. **Format code** [Optional] [Action]: Format your current code.
c. **Quit the full-screen mode** [Optional] [Action]: Quit the full-screen mode.

18. **Alert if the number of tasks in the node queue reaches the following number** [Optional] [Action]: Helps monitor whether the number of tasks in the node exceeds the specified threshold.

19. **Number of tasks in the queue** [Optional/Required if **Alert if the number of tasks in the node queue reaches the following number** is selected] [Field] [Number]: Number of tasks in the queue which triggers an alert when reached.

20. **Maximum interval, for which the task stays in the node before being forwarded** [Optional] [Action]: Select to limit the time a task is allowed to be in the node in seconds, minutes, hours, and days.

21. **Maximum interval** [Optional/Required if **Maximum interval, for which the task stays in the node before being forwarded**] [Field] [Number]: The amount of time a task is allowed to be in the node. It has a minimum value of 30 seconds. You can set a shorter interval by using the **Unixtime** function.

22. **Maximum interval measuring unit** [Required] [Action]: Selects the measuring unit for the maximum interval:

- Seconds
- Minutes
- Hours
- Days.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Examples

Create a table

```
CREATE TABLE alc (
    user_id integer,
    conveyor_id integer,
    name text,
    description text
```

- `);`

Insert a new row into a table

```
INSERT INTO alc (user_id, conveyor_id, name, description)
```

- `VALUES (13, 777000, 'Send Message', 'Use to send text messages to phone number')`
- Call stored function with one parameter (PostgreSQL)
  `SELECT function_name(arg)`

Select data from two tables using JOIN

```
SELECT alb.conveyor_id,
CONCAT('https://api.corezoid.com/api/2/json/', alb.hash) as URL,
alc.*
FROM "public"."alb" alb
LEFT JOIN "public"."alc" alc ON alc.conveyor_id = alb.conveyor_id
```

- `WHERE alc.conveyor_id is not null`

Update data based on task parameters
Change the value of the hash field with the value of {{URL}} parameter for a row with conveyor_id equals to {{conv_id}}.

```
UPDATE alb
SET hash = '{{URL}}'
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- `WHERE conveyor_id = '{{conv_id}}'`

Error handling & troubleshooting

When an error occurs during the task processing, you may see the following error parameter names in the task.

Error parameter name	Parameter description
`__conveyor_db_call_return_type_error__`	**Hardware** (system error), **Software** (error in a node logic/settings).
`__conveyor_db_call_return_type_tag__`	*Error tag.
`__conveyor_db_call_return_type_description__`	Error description in human-readable language; can be static or dynamic.

Table 19: **Error parameter names.**

*The error tag `__conveyor_db_call_return_type_tag__` may have the following values.

Value	Cause	Solution
access_denied	You don't have permission to send a	Contact the Process owner for further assistance.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

	reply to the indicated Process.	
access_denied	You are trying to connect to the database from another stage.	It's not allowed to connect to databases from other stages. Use the same stage to connect to the needed database.
db_call_return_format_error	Incorrect format of the response from the **Database Call** node.	Contact the support team for further assistance.
db_call_executing_error	The **Database Cal**l node returned an incorrect code.	Contact the support team for further assistance.
not_found_conv_by_alias	Your alias refers to a Process that no longer exists.	Use an existing Process.
db_call_fatal_error	A hardware error has occurred.	Contact the support team for further assistance.

Table 20: **Database Call node errors.**

When working with your Process, you may encounter the following issues.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

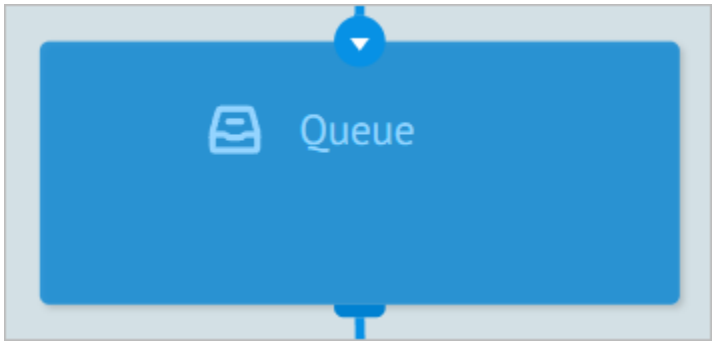
Issue	Cause	Solution
Connection to the database is lost.	The database you are trying to connect to is down.	This issue is caused by a database failure and is not related to the Corezoid platform.
Can't reach the component that connects to the database.	The on-premise connection has been lost because the database is not functioning.	Check your database connection and credentials.

Table 21: **Database Call node issues.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3.10 Queue

Overview



Screenshot 219: **Queue node view.**

The **Queue** node enables you to temporarily hold a task until you are ready to process it. It is commonly used when tasks require manual intervention, such as when an employee needs to perform certain actions to process an order, such as contacting a client or preparing an order for shipment.
A workflow involving the **Queue** node consists of two Processes:

- Includes the **Queue** node, which accumulates and holds all incoming tasks until they are ready for processing.
- Involves the **Get from Queue** node, which is used to retrieve tasks when you are prepared to process them.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

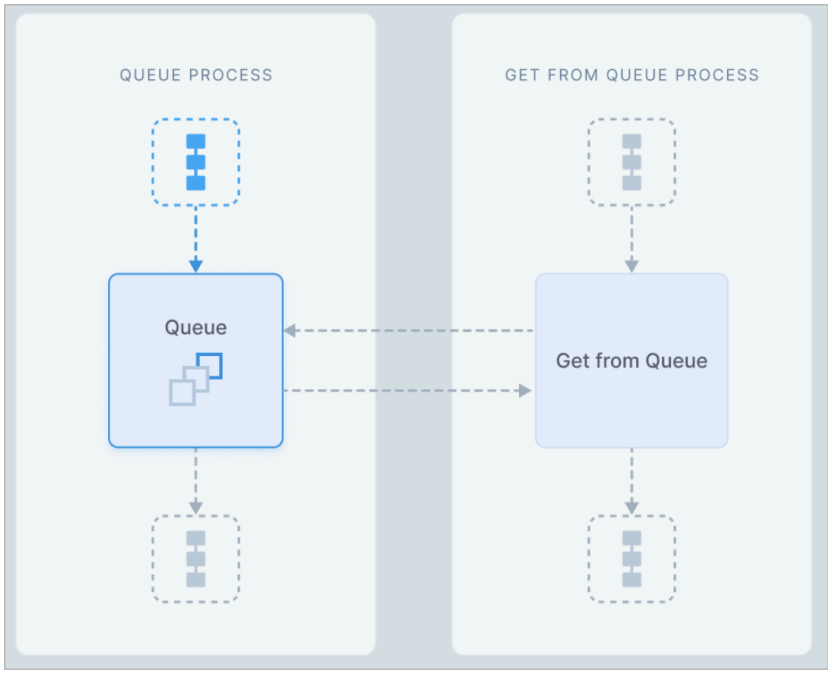
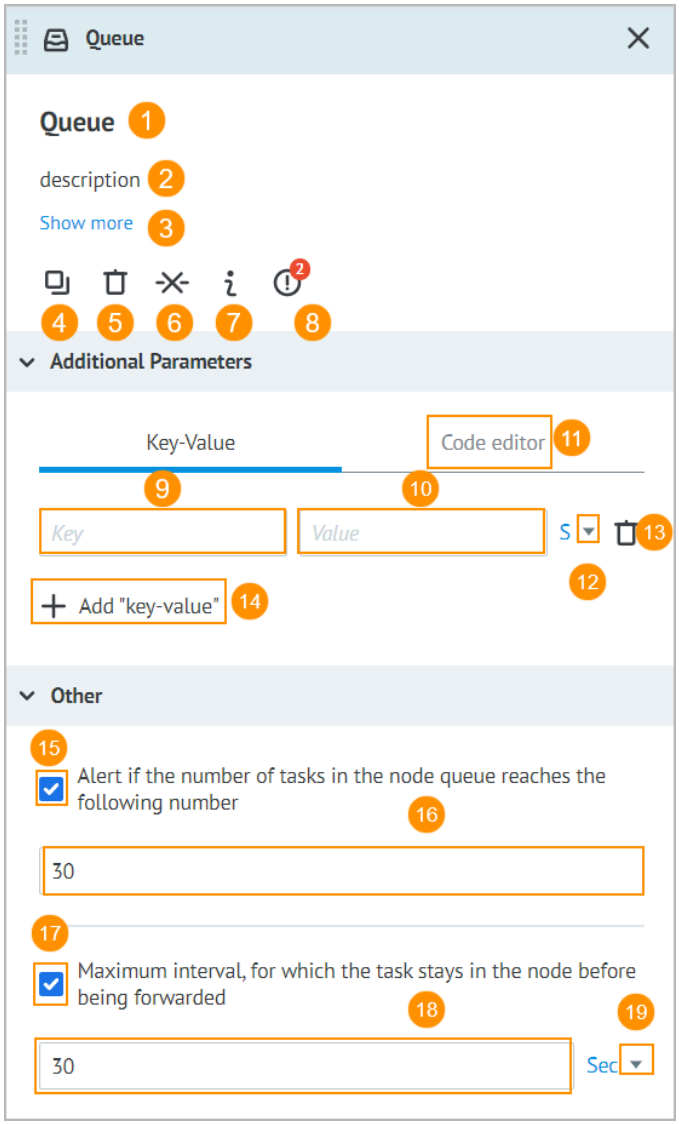


Diagram 14: **Queue and Get from Queue nodes interaction flow**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Node Settings

The **Queue** node has the following parameters:

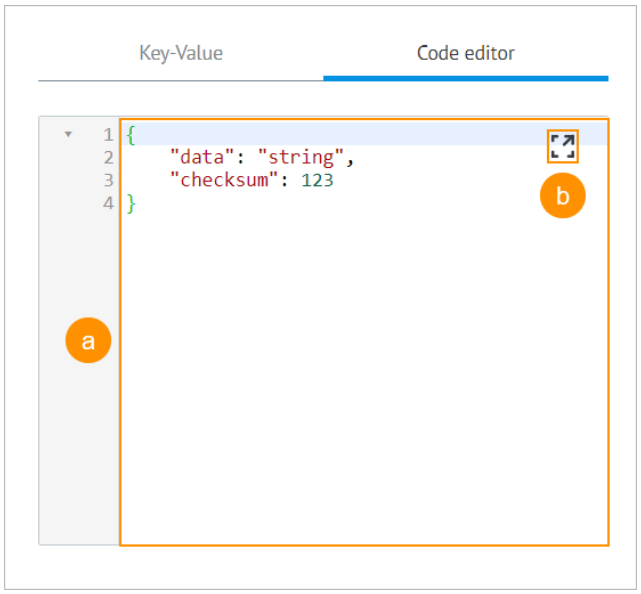


Screenshot 220: **Queue node settings.**

1. **Title** [Optional] [Field] [String]**:** Node title.
2. **Description** [Optional] [Field] [String]: Node description.
3. **Show more** [Optional] [Action]: Long descriptions are reduced to a compact view to simplify node settings. Click to see the whole description.
4. **Copy node** [Optional] [Action]: Creates a copy of the current node with all settings**.**
5. **Remove node** [Optional] [Action]: Deletes current node from Process.
6. **Remove links** [Optional] [Action]: Removes all connections to/from current node.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

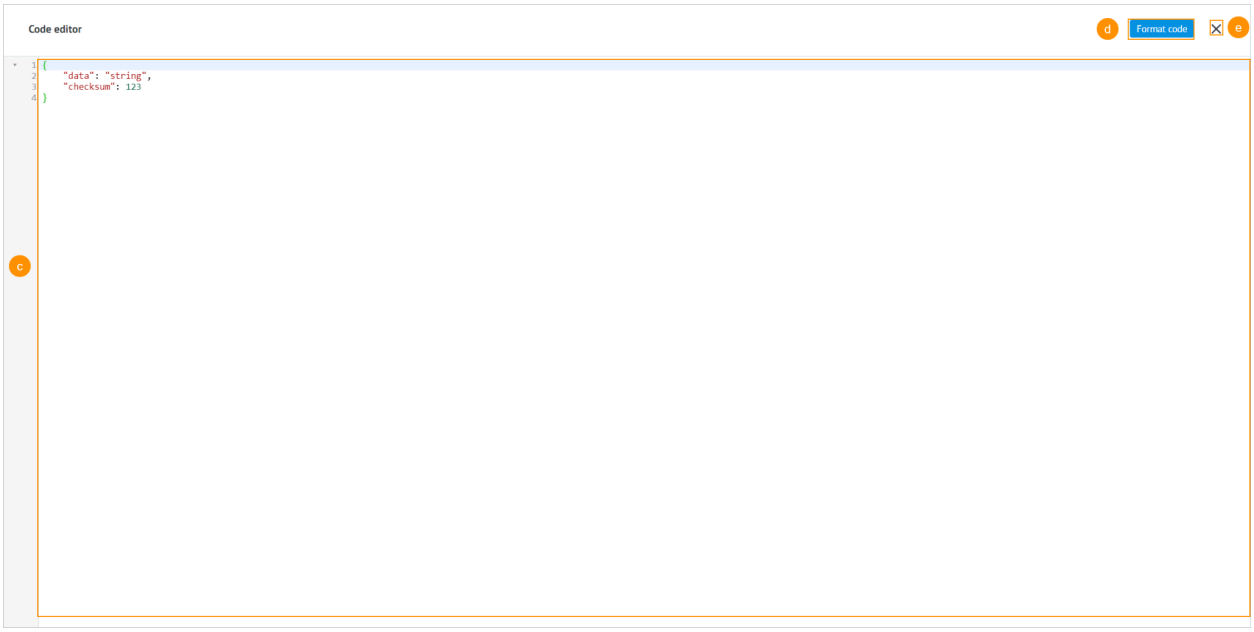
7. **Info** [Optional] [Action]: Displays node ID.
8. **Errors info** [Optional] [Action]: Displays node errors.
9. **Key** [Required] [Field] [String]: Key to specify a static name or a variable in {{}} curly brackets
   To add a new parameter, specify a unique name, to update an existing parameter, use its original name as a key.
10. **Value** [Required] [Field] [Any]: The value you want to assign to the corresponding Key parameter. See data types descrition for more details.
11. **Code editor** [Optional] [Action]: Shows the JSON format of the entered Key and Value. Click **Code editor** to open the code editor tab, where:



Screenshot 221: **Code editor tab.**

a. **Code editor field** [Optional] [Field] [Any]: Enter new keys and values or edit the existing ones in JSON format.
b. **Full-screen mode** [Optional] [Action]: Opens the **Code editor** field in the full-screen mode, where:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 222: **Code editor full-screen mode.**

c. **Code editor field** [Optional] [Field] [Any]: Edit or add keys in values.
d. **Format code** [Optional] [Action]: Format your current code.
e. **Quit the full-screen mode** [Optional] [Action]: Quit the full-screen mode.

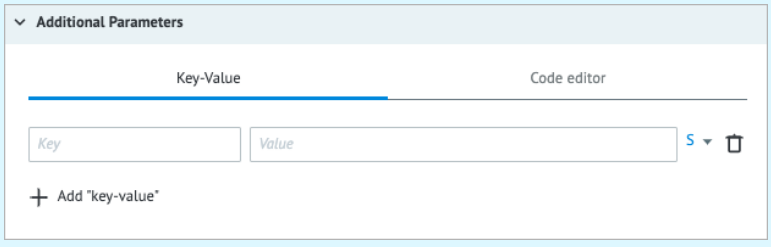
12. **Type of data** [Required] [Action]: Selects the data type for the **Value**:
    * String (**S**)
    * Number (**N**)
    * Boolean (**B**)
    * Array (**A**)
    * Object (**O**)

    Note: For more information, go to Data types.

13. **Delete key-value** [Optional] [Action]: Delete the key-value pair.
14. **Add key-value** [Optional] [Action]: Add a new key-value pair.

    - To add a key-value, click **+ Add "key-value"** below the **Key** field.
    - To delete the key-value line, click the trash icon on the right side of the key-value line.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 223: **Key-value pair adding.**

15. **Alert if the number of tasks in the node queue reaches the following number** [Optional] [Action]: Helps monitor whether the number of tasks in the node exceeds the specified threshold.
16. **Number of tasks in the queue** [Required] [Field] [Number]: Number of tasks in the queue which triggers an alert when reached.
17. **Maximum interval, for which the task stays in the node before being forwarded** [Optional] [Action]: Select to limit the time a task is allowed to be in the node in seconds, minutes, hours, and days.
18. **Maximum interval** [Required] [Field] [Number]: The amount of time a task is allowed to be in the node. It has a minimum value of 30 seconds. You can set a shorter interval by using the **Unixtime** function.
19. **Maximum interval measuring unit** [Required] [Action]: Selects the measuring unit for the maximum interval:
    - Seconds
    - Minutes
    - Hours
    - Days.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Examples

Using Queue and Get from Queue nodes together
In the example below, you can see how the **Queue** node in one Process and the **Get from Queue** node in another Process work together. The parameter specified in the **Queue** node is not added to the tasks until after they pass through that node. Two parameters are added to a task when it is released from the **Get from Queue** node:

- **queue_task_id**: Contains the task ID of the stored task and is useful for tracking and identifying individual tasks as they are retrieved from the Queue node.
- **queue_task_data**: Contains the task parameters of the stored task that include the data that the task originally had, as well as any additional parameters that were specified in the Queue node.

Screenshot 224: **Using Queue and Get from Queue.**

You can access a single task parameter or a whole task stored in a State Diagram by Getting Task Parameters from State Diagrams.

Error handling & troubleshooting

When an error occurs during the task processing, you may see the following error parameter names in the task.

Error parameter name	Error type
`__conveyor_queue_return_type_error__`	**Hardware** (system error), **Software** (error in a node logic/settings).
`__conveyor_queue_return_type_tag__`	*Error tag.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

`__conveyor_queue_return_type_description__`	Error description in human-readable language; can be static or dynamic.

Table 22: **Error parameter names.**

*The error tag `__conveyor_queue_return_type_tag__` may have the following values.

Value	Cause	Solution
access_denied	You don't have access to the Process you are trying to call.	Contact the Process owner to get access.
access_refused	The Process you are working in is not a State Diagram but it must be a State Diagram.	Check the logic of using a State Diagram. For more information, go to State Diagram.
queue_wrong_convert_param	Parameter value failed to convert with the specified parameter type.	Check and correct the data type of the parameter mentioned in the error message.

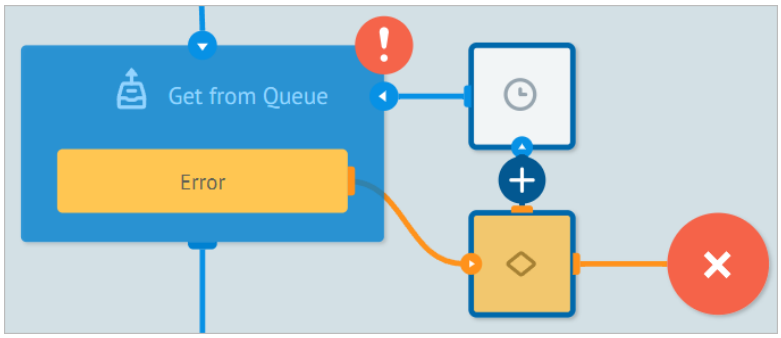
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

not_found_conv_by_alias	Your alias is linked to a Process that no longer exists.	Use an existing Process.
queue_wrong_logic	The **Queue** node is missing in the Process you are trying to call.	Check whether the Queue logic exists in your conv ID. For more information about the **Queue** node, go to Queue.

Table 23: **Queue node errors.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3.11 Get from Queue



Screenshot 225: **Get from Queue node view.**

Overview

The **Get from Queue** node enables you to retrieve tasks from any **Queue** node that you have set up in your Corezoid Processes.

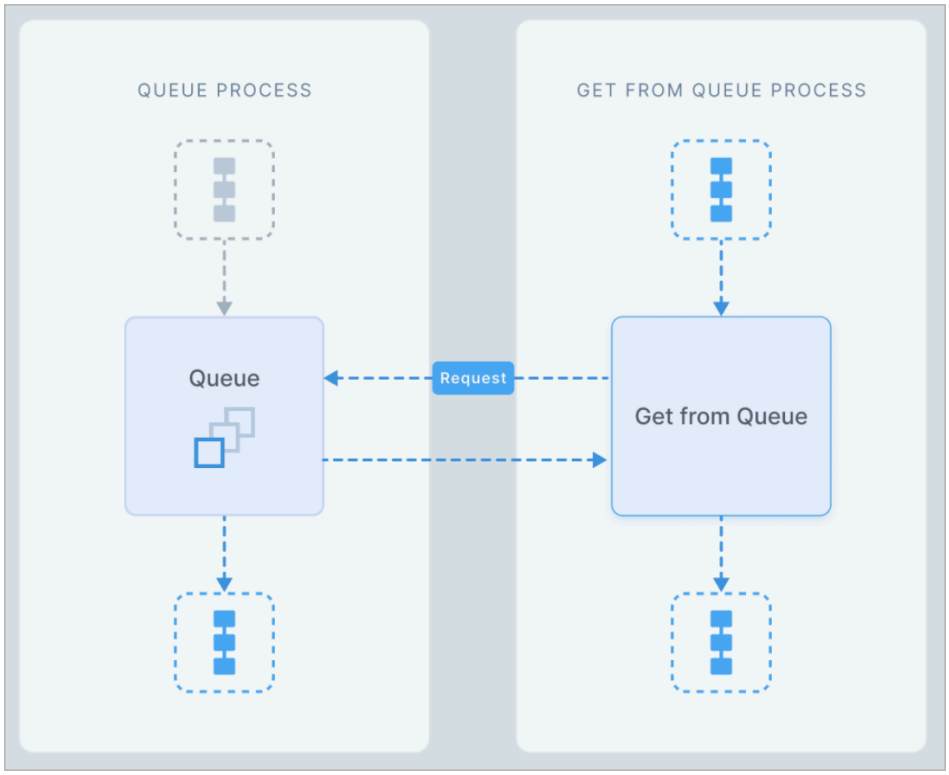
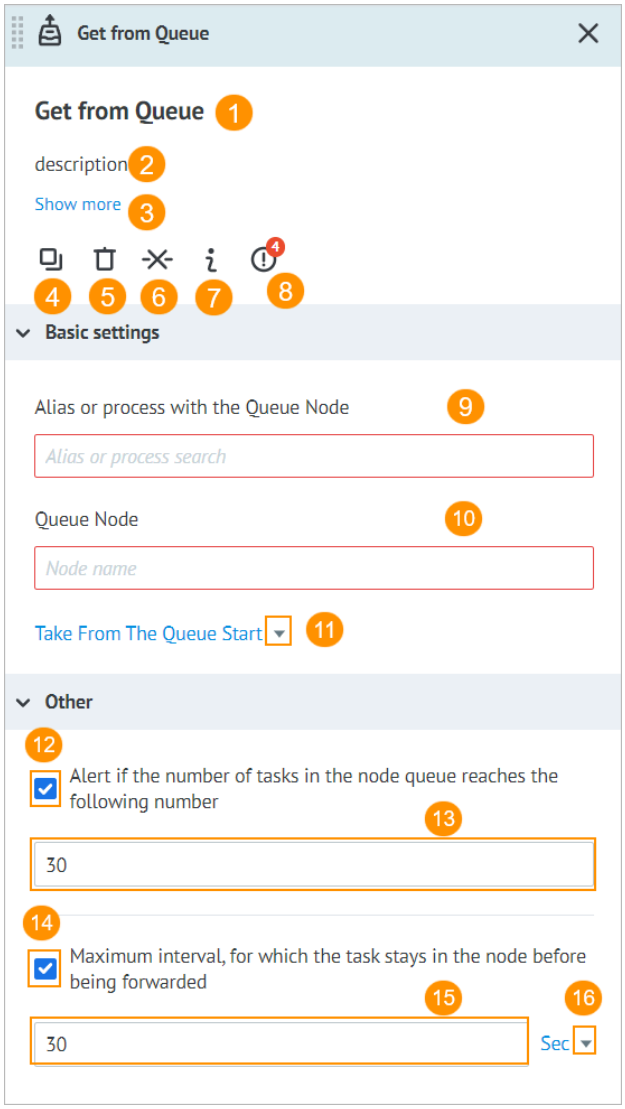


Diagram 15: **Queue and Get from Queue nodes interaction flow**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Node Settings

The **Get from Queue** node has the following parameters:



Screenshot 226: **Get from Queue node settings.**

1. **Title** [Optional] [Field] [String]**:** Node title.
2. **Description** [Optional] [Field] [String]: Node description**.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3. **Show more** [Optional] [Action]: Long descriptions are reduced to a compact view to simplify node settings. Click to see the whole description.
4. **Copy node** [Optional] [Action]: Creates a copy of the current node with all settings**.**
5. **Remove node** [Optional] [Action]: Deletes current node from Process.
6. **Remove links** [Optional] [Action]: Removes all connections to/from current node.
7. **Info** [Optional] [Action]: Displays node ID.
8. **Errors info** [Optional] [Action]: Displays node errors.
9. **Alias or process with the Queue Node** [Required] [Field/Action] [String]: Choose the Process or alias that contains the Queue node from which you want to take tasks by clicking the directory icon or by specifying its Process ID or Alias.
10. **Queue Node** [Required] [Field/Action] [String]: Select the specific Queue node from the Process you've selected above.
11. **Take From The Queue start**/ **Take From The Queue end** [Required] [Action]: Choose whether to take tasks from the start or the end of the selected Queue node.
12. **Alert if the number of tasks in the node queue reaches the following number** [Optional] [Action]: Helps monitor whether the number of tasks in the node exceeds the specified threshold.
13. **Number of tasks in the queue** [Optional/Required if **Alert if the number of tasks in the node queue reaches the following number** is selected] [Field] [Number]: Number of tasks in the queue which triggers an alert when reached.
14. **Maximum interval, for which the task stays in the node before being forwarded** [Optional] [Action]: Select to limit the time a task is allowed to be in the node in seconds, minutes, hours, and days.
15. **Maximum interval** [Optional/Required if **Maximum interval, for which the task stays in the node before being forwarded** is selected] [Field] [Number]: The amount of time a task is allowed to be in the node. It has a minimum value of 30 seconds. You can set a shorter interval by using the **Unixtime** function.
16. **Maximum interval measuring unit** [Required] [Action]: Selects the measuring unit for the maximum interval:
    - Seconds
    - Minutes
    - Hours
    - Days.

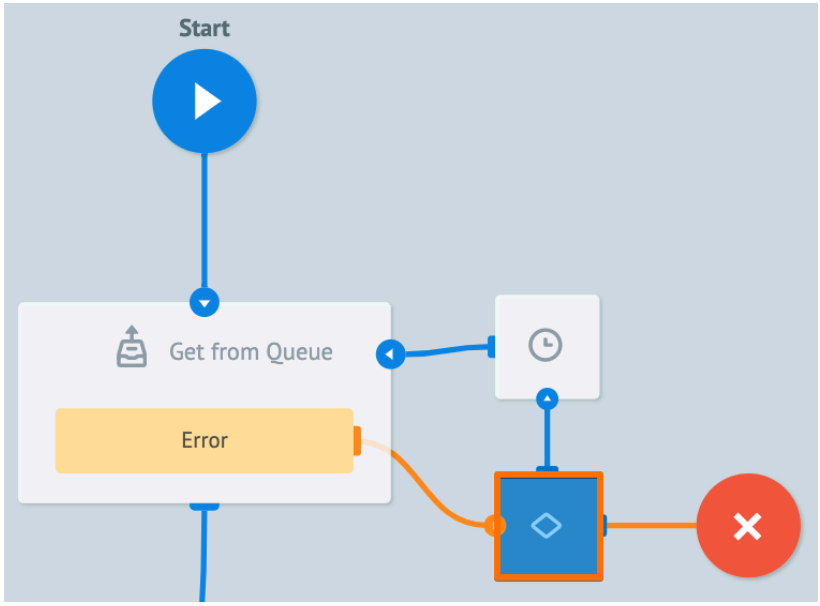
Examples

Using the Get from Queue node to retrieve a task stored in the Queue node
See the example in Queue.

Error handling & troubleshooting

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

When an error occurs in the **Get from Queue** node, a task goes to the auxiliary **Condition** output node that is used for storing error parameters.



Screenshot 227: **Get from Queue auxiliary debug node.**

When an error occurs during the task processing, you may see the following error parameter names in the task.

Error parameter name	Error type
`__conveyor_get_task_return_type_error__`	**Hardware** (system error), **Software** (error in a node logic/settings).
`__conveyor_get_task_return_type_tag__`	*Error tag.
`__conveyor_get_task_return_type_description__`	Error description in human-readable language; can be static or dynamic.

Table 22: **Error parameter names.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

*The error tag `__conveyor_get_task_return_type_tag__` may have the following values.

Value	Cause	Solution
access_denied	You don't have access to the Process you are trying to call.	Contact the Process owner to get access.
access_denied	You are trying to send or receive tasks from another stage.	It's not allowed to get and send tasks between stages. Use the same stage to send and receive tasks.
No queue logic in conv_id: {{ConvId}} node_id: {{NodeId}}	The Process you specified does not contain the **Queue** node.	Make sure you've specified the Process correctly.
conveyor_is_not_active	The target Process is not active.	Make sure that the **Queue** node Process is active.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

get_task_wrong_convert_param	Parameter value failed to convert.	Check and correct the data type of the parameter mentioned in the error message.
crash_api	Task retrieval error.	Contact your administrator for further assistance.
Error running get tas	Task copy error.	Check whether the specified task ID is valid (a task with this ID exists) or contact your administrator for further assistance.
not_found_conv_by_alias	Your alias is linked to a Process that no longer exists.	Use an existing Process.
get_task_size_overflow_limit	The size of the task data exceeds the specified limit set for your environment.	Reduce the size of your task.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

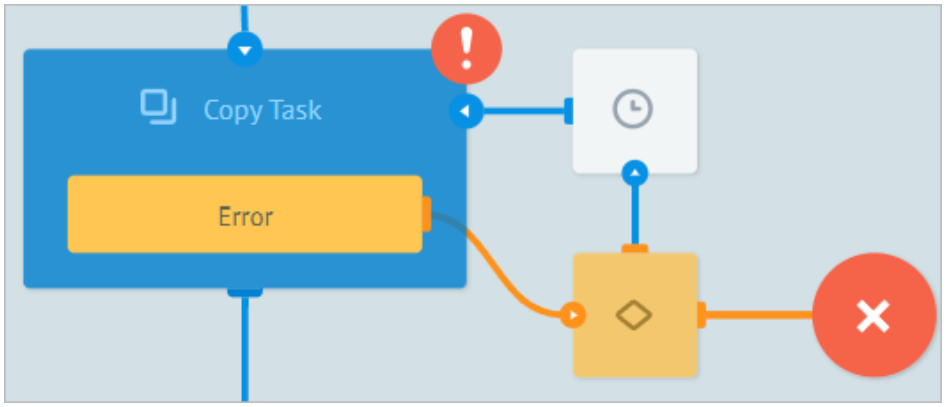
bad_callback_node	The Process you are calling doesn't have the **Callback** node.	Create the **Callback** node in the Process you are calling.
get_task_executing_error	There is an unexpected error.	Contact the support team for further assistance.
not_found_task	The **Queue** node is empty.	Make sure to consider this case when designing the Process by selecting the **Alert when the number of tasks in the node queue reaches the next number** option.

Table 23: **Get from Queue node errors.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3.12 Copy Task

Overview



Screenshot 228: **Copy Task node view.**

The **Copy Task** node allows you to send a copy of a task to another Process. When you use the node, the task is essentially duplicated with one copy proceeding in the current Process and a new copy being passed to another Process. The two copies are independent of one another, but unlike the **Call a Process** node, the **Copy Task** node does not wait for a response from the other Process.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

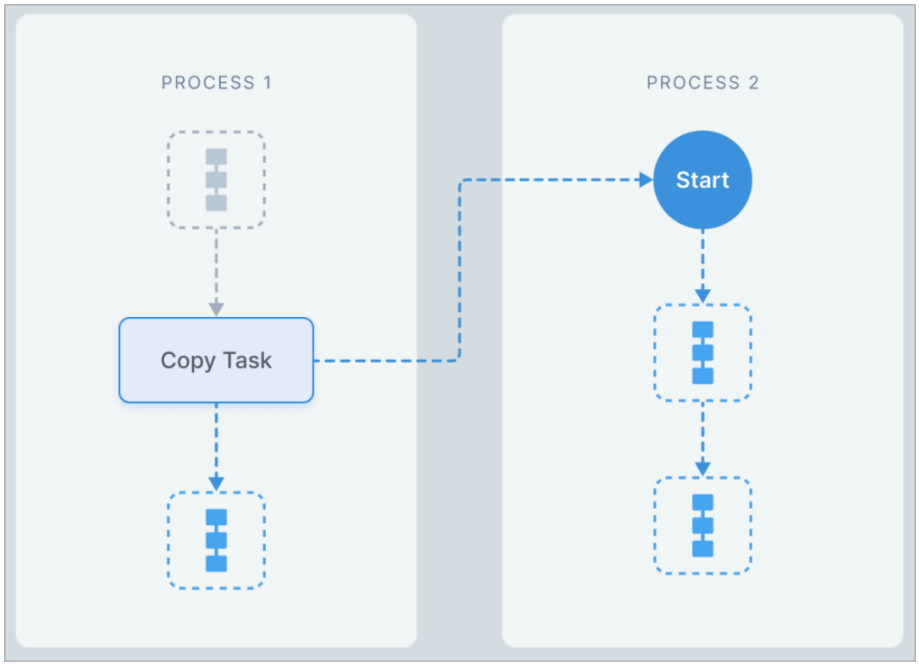


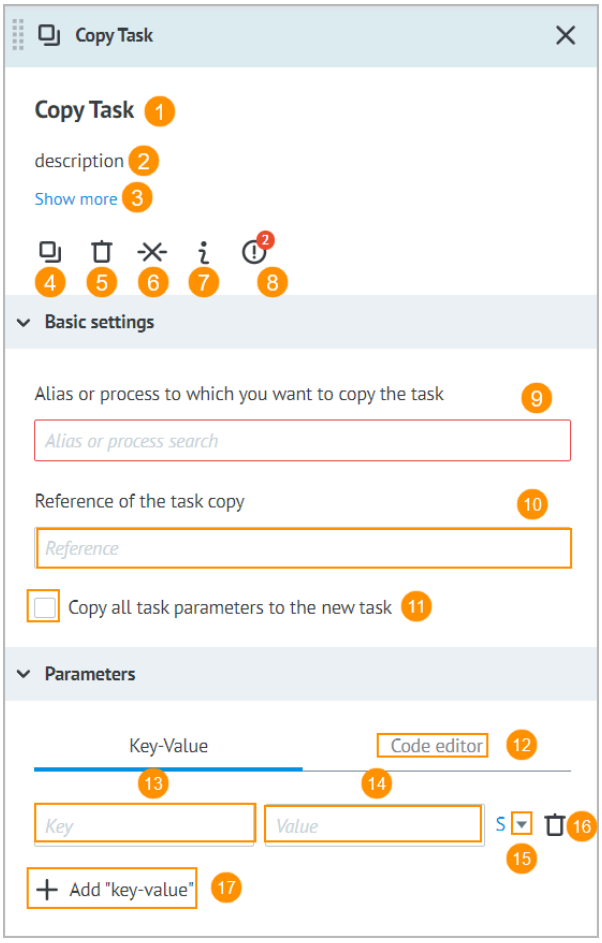
Diagram 16: **Copy Task node flow**.

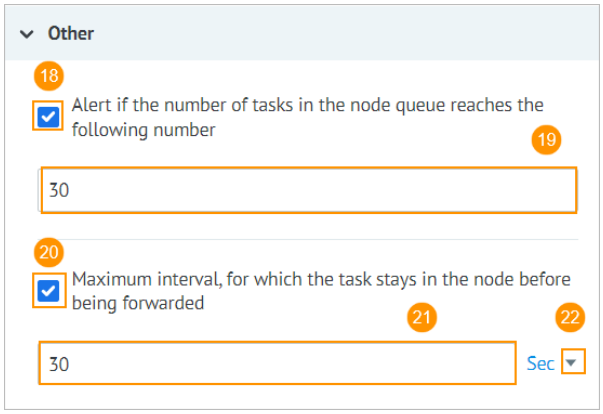
Node Settings

The **Copy Task** node has the following parameters:

Basic settings	Other settings

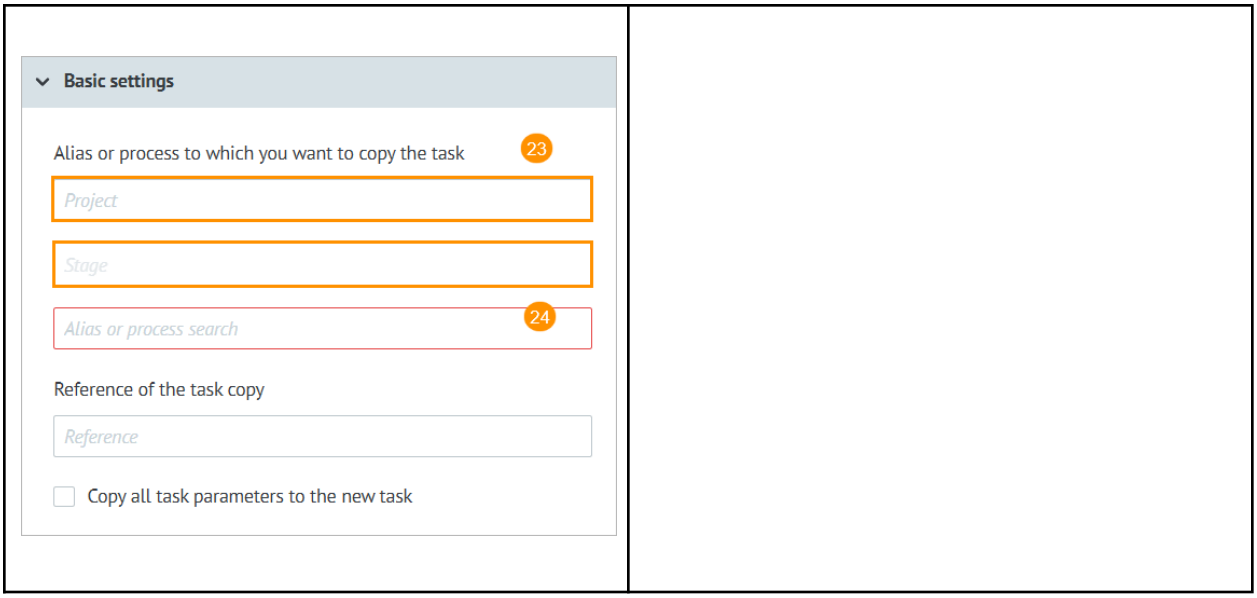
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.





Basic settings for nodes inside Stages

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



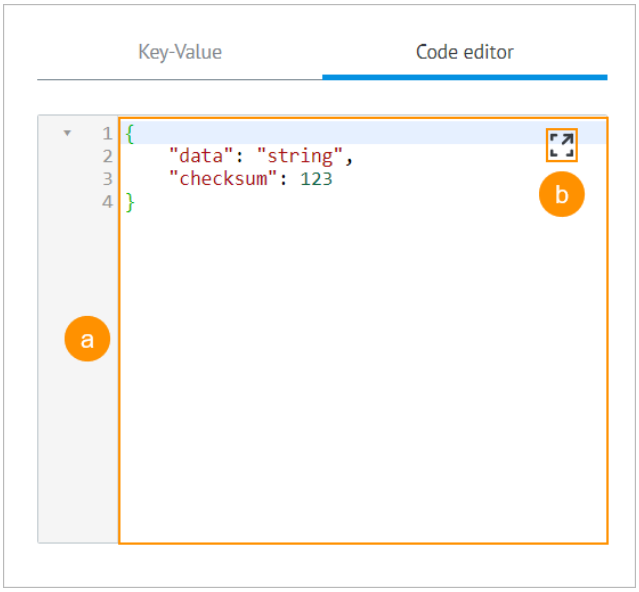
Screenshot 229: **Copy Task node settings.**

1. **Title** [Optional] [Field] [String]**:** Node title.
2. **Description** [Optional] [Field] [String]: Node description**.**
3. **Show more** [Optional] [Action]: Long descriptions are reduced to a compact view to simplify node settings. Click to see the whole description.
4. **Copy node** [Optional] [Action]: Creates a copy of the current node with all settings**.**
5. **Remove node** [Optional] [Action]: Deletes current node from Process.
6. **Remove links** [Optional] [Action]: Removes all connections to/from current node.
7. **Info** [Optional] [Action]: Displays node ID.
8. **Errors info** [Optional] [Action]: Displays node errors.
9. **Alias or process to which you want to copy the task** [Required] [Field/Action] [String]: Choose the Process to which you would like to copy a task by clicking the directory icon or by specifying its Process ID or Alias.
10. **Reference of the task copy** [Optional] [Action]: Specify the reference identifier for the copied task.
11. **Copy all task parameters to the new task** [Optional] [Action]:

    When selected: all current task parameters will be sent along with any parameters specified in the node.
    When not selected: Only the parameters specified in the node will be sent.

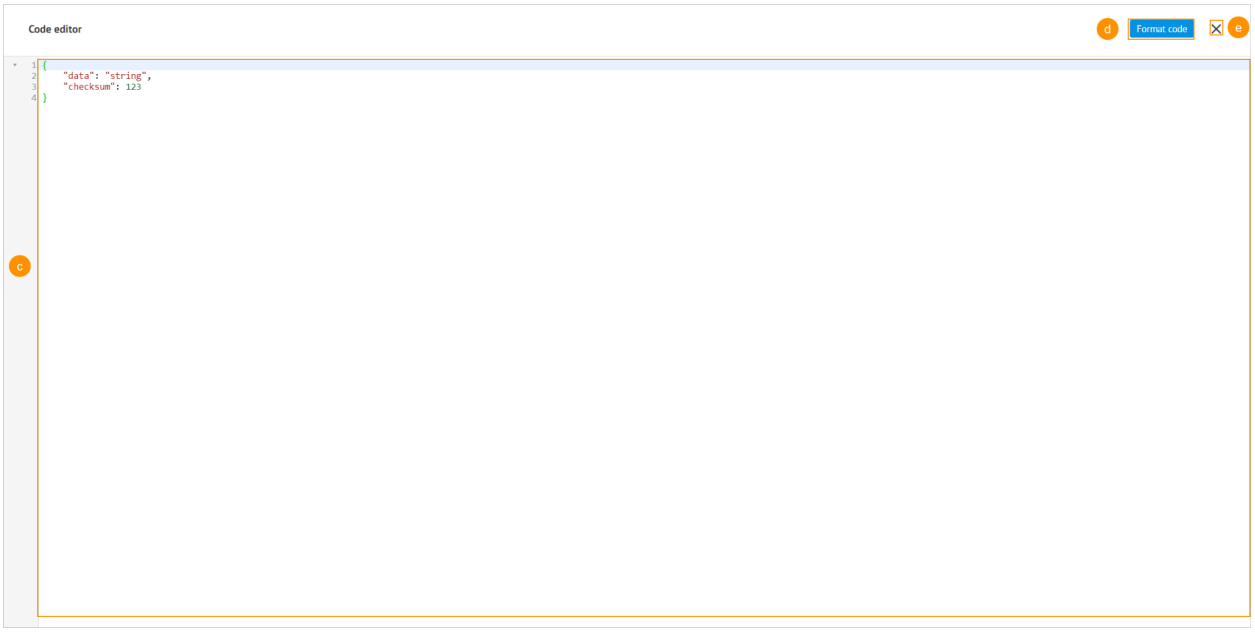
12. **Code editor** [Optional] [Action]: Shows the JSON format of the entered Key and Value. Click **Code editor** to open the code editor tab, where:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 230: **Code editor tab.**

a. **Code editor field** [Optional] [Field] [Any]: Enter new keys and values or edit the existing ones in JSON format.
b. **Full-screen mode** [Optional] [Action]: Opens the **Code editor** field in the full-screen mode, where:



Screenshot 231: **Code editor full-screen mode.**

c. **Code editor field** [Optional] [Field] [Any]: Edit or add keys in values.
d. **Format code** [Optional] [Action]: Format your current code.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

e. **Quit the full-screen mode** [Optional] [Action]: Quit the full-screen mode.

13. **Key** [Required] [Field] [String]: Key to specify a static name or a variable in {{}} curly brackets
    To add a new parameter, specify a unique name, to update an existing parameter, use its original name as a key.
14. **Value** [Required] [Field] [Any]: The value you want to assign to the corresponding Key parameter.
    See data types descrition for more details.
15. **Type of data** [Required] [Action]: Selects the data type for the **Value**:
    * String (**S**)
    * Number (**N**)
    * Boolean (**B**)
    * Array (**A**)
    * Object (**O**)

    Note: For more information, go to Data types.

16. **Delete key-value** [Optional] [Action]: Delete the key-value pair.
17. **Add key-value** [Optional] [Action]: Add a new key-value pair.

- To add a key-value, click **+ Add "key-value"** below the **Key** field.
- To delete the key-value line, click the trash icon on the right side of the key-value line.

18. **Alert if the number of tasks in the node queue reaches the following number** [Optional] [Action]: Helps monitor whether the number of tasks in the node exceeds the specified threshold.

19. **Number of tasks in the queue** [Optional/Required if **Alert if the number of tasks in the node queue reaches the following number** is selected] [Field] [Number]: Number of tasks in the queue which triggers an alert when reached.

20. **Maximum interval, for which the task stays in the node before being forwarded** [Optional] [Action]: Select to limit the time a task is allowed to be in the node in seconds, minutes, hours, and days.

21. **Maximum interval** [Optional/Required if **Maximum interval, for which the task stays in the node before being forwarded**] [Field] [Number]: The amount of time a task is allowed to be

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

in the node. It has a minimum value of 30 seconds. You can set a shorter interval by using the **Unixtime** function.

22. **Maximum interval measuring unit** [Required] [Action]: Selects the measuring unit for the maximum interval:

- Seconds
- Minutes
- Hours
- Days.

23. **Project** [Required] [Field/Action] [String]: Choose the Project containing the Process to which you would like to copy a task by clicking the directory icon or by specifying its Process ID, name or Alias.

Note: this parameter is available with the **Alias or process to which you want to copy the task** parameter and applies to **Copy Task** nodes created within Stages in Projects. This feature allows you to copy tasks to Processes located in Stages of different Projects.

24. **Stage** [Required] [Field/Action] [String]: Choose the Stage containing the Process to which you would like to copy a task by clicking the directory icon or by specifying its Process ID, name or Alias.

Note: this parameter is available with the **Alias or process to which you want to copy the task** parameter and applies to **Copy Task** nodes created within Stages in Projects. This feature allows you to copy tasks to Processes located in Stages of different Projects.

Examples

Duplicating a task and adding a new parameter to the task copy

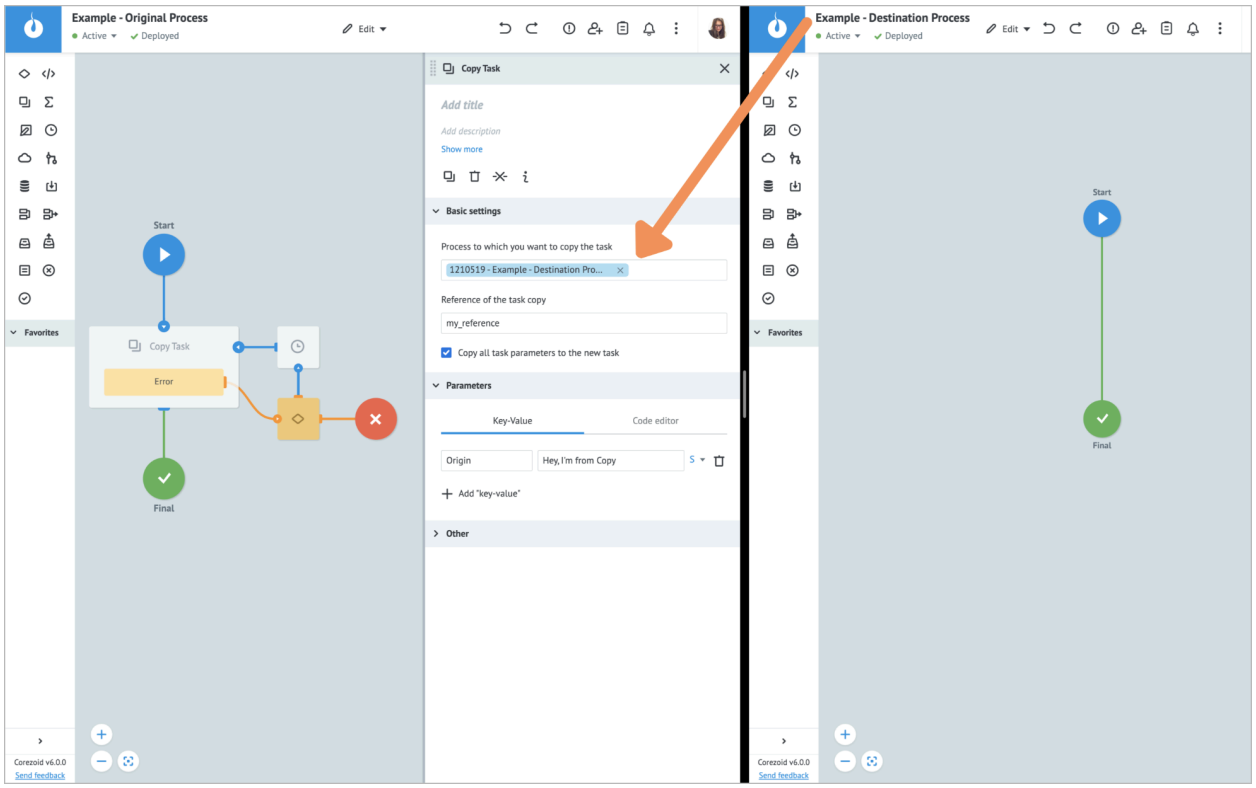
Description: Below, you can see the example where the **Copy Task** node is used to duplicate a task from the **Original** Process to the **Destination** Process.

Precondition:

1. Add the **Copy Task** node to your **Original** Process.
2. In **Basic settings** of the **Copy Task** node details panel, add:
    a. The **Destination** Process you want to copy a task from.
    b. The **my_reference** reference.
3. Select the **Copy all task parameters to the new task** checkbox.
4. In **Parameters**, add:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

a. The **Origin** parameter.
b. The **Hey, I'm from Copy** value.



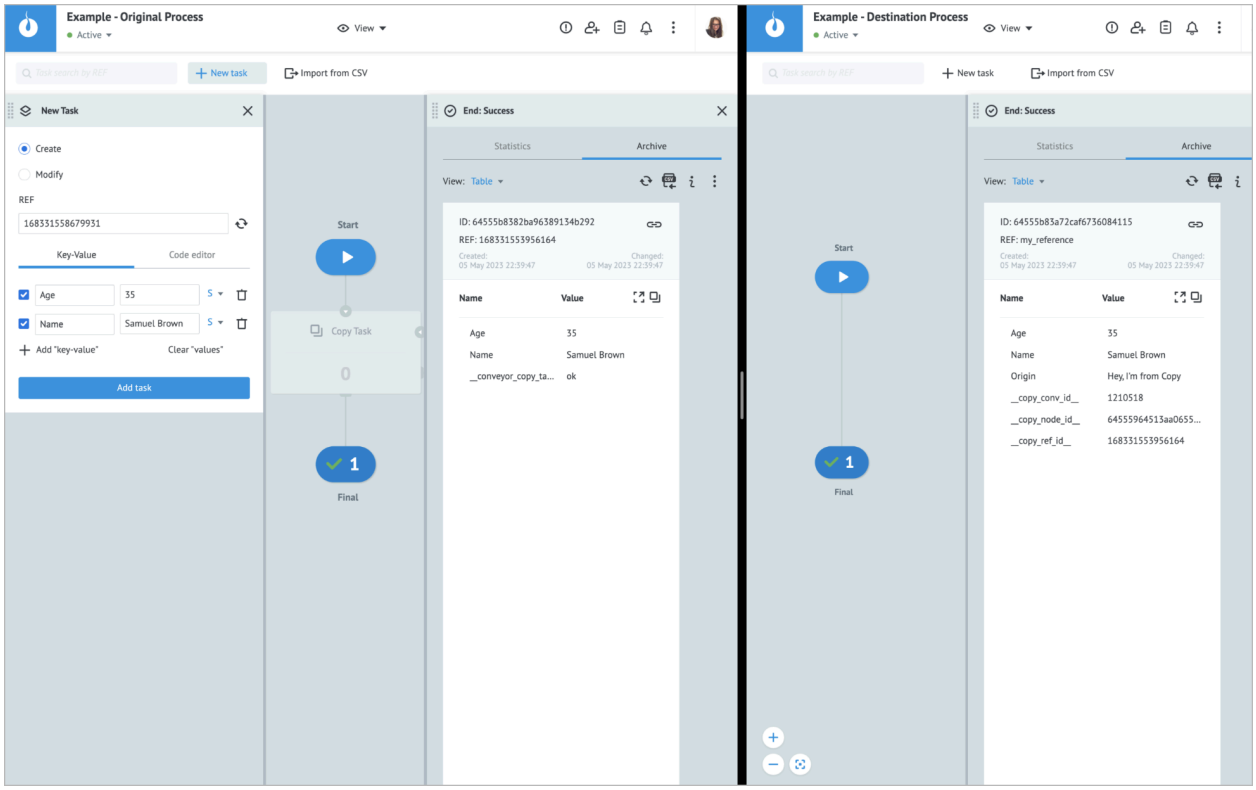
Screenshot 232: **Copy task original and destination Processes.**

Flow: After adding a task to the **Original** Process, you can see that the **Copy Task** node not only duplicates the original task parameters but also adds some new ones. These system parameters are automatically generated by the **Copy Task** node and help trace the relationship between the original and copied tasks:

- **copy_conv_id**: Stores the ID of the Process (Original Process) from which the task was copied.
- **copy_node_id**: Indicates the ID of the Copy Task node that created the copied task.
- **copy_ref_id**: Stores the ID of the initial task from which the copy was created.

The task in the **Original** Process does not include the parameter that was defined within the **Copy Task** node but has the **conveyor_copy_task_result** parameter to keep track of whether the duplication was successful.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



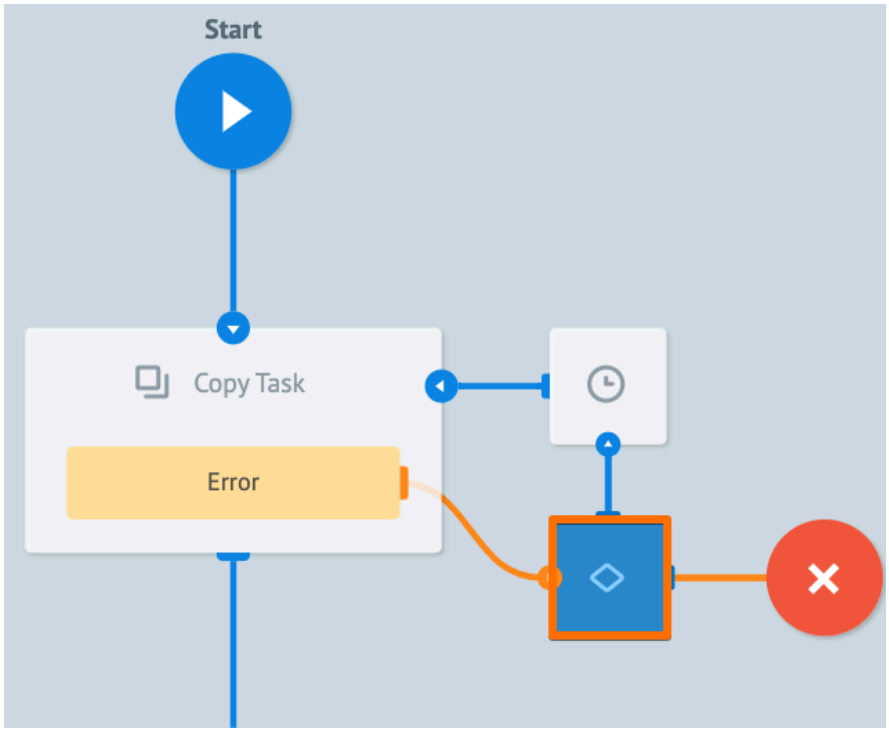
Screenshot 233: **Tasks in Copy Task Original and Destination Processes.**

You can access a single task parameter or a whole task stored in a State Diagram by Getting Task Parameters from State Diagrams.

Error handling & troubleshooting

When an error occurs in the **Copy Task** node, a task goes to the auxiliary Condition output node that is used for storing error parameters.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 234: **Copy Task auxiliary debug node.**

When an error occurs during the task processing, you may see the following error parameter names in the task.

Error parameter name	Error type
`__conveyor_copy_task_return_type_error__`	**Hardware** (system error), **Software** (error in a node logic/settings).
`__conveyor_copy_task_return_type_tag__`	*Error tag.
`__conveyor_copy_task_return_type_description__`	Error description in human-readable language; can be static or dynamic.

Table 23: **Error parameter names.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

*The error `tag __conveyor_copy_task_return_type_tag__` may have the following values.

Value	Cause	Solution
access_denied	You don't have access to the Process you are trying to call.	Contact the Process owner to get access.
access_denied	You are trying to send or receive tasks from another stage.	It's not allowed to get and send tasks between stages. Use the same stage to send and receive tasks.
access_refused	The Process you are working on is not a State Diagram but it must be a State Diagram.	Check the logic of using a State Diagram. For more information, go to State Diagram.
conveyor_is_not_active	The target Process is not active.	Make sure that the Process that you are trying to copy the task to is active.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

crash_api	Error occurred when sending the task to the Process with the **Reply to Process** node.	Contact your administrator for further assistance.
no_api_callback_in_node_with_task	No API callback logic found in the node with the task.	Specify the ID or alias of the **Waiting for Callback** node or add this type of node.
copy_task_timeout	Timeout when performing task copying.	Retry copying the task or contact your administrator for further assistance.
not_unical_ref	The task reference is not unique.	Check whether there is a task with the specified reference in the Process you are attempting to copy a task to.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

wrong_validate_params	Parameter validation error.	Check the types and values of the parameters you have specified in the **Copy Task** node.
copy_task_wrong_convert_param	Parameter value failed to convert with the specified parameter type.	Check and correct the data type of the parameter mentioned in the error message.
not_found_conv_by_alias	Your alias is linked to a Process that no longer exists.	Use an existing Process.
copy_task_size_overflow_limit	The size of the task you are trying to copy exceeds the specified limit set for your environment.	Reduce the size of your task by modifying the data or splitting it into several smaller tasks.
duplicate_callback	You are sending the next task with the same reference while the first one	Wait until the first task leaves the **Callback** node.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

	is still in the **Callback** node.	
conv_not_found_or_deleted	You are trying to work with a non-existent Process.	Use an existing Process.
bad_utf8_string	The key or value you are using is not valid.	Check whether the key and value have the **utf8** symbols.
copy_task_eq_convs	You are trying to copy your task to itself.	Choose another task to copy or as a target to copy yours in.
ref_is_not_string_format	The task reference specified in the parameters of the copied Process is incorrect.	Check whether the reference belongs to the **String** type of data.
limit_delayed_tasks_to_this_user_is_overflow	There is a specified limit for the user to set the time intervals (long	Contact your administrator for further assistance.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

	or short) in the **Delay** node	
bad_callback_node	The Process you are calling doesn't have the **Callback** node.	Create the **Callback** node in the Process you are calling.
not_found_task	No task with the specified reference exists in the Process.	Make sure you entered the task reference and Process correctly. **Note:** An empty reference is not considered unique: if only a single task with an empty reference exists, you cannot access it by reference.
not_found_main_node	The **Start** node is missing in the Process in which you are trying to copy a task.	Check whether the **Start** node exists in the Process you are copying to your task.
copy_task_fatal_error	The **Copy Task** node error.	Check the Process to which you are trying to copy the task or contact your administrator

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

		for further assistance.

Table 24: **Copy Task node errors.**

When working with your Process or State Diagram, you may encounter the following issues.

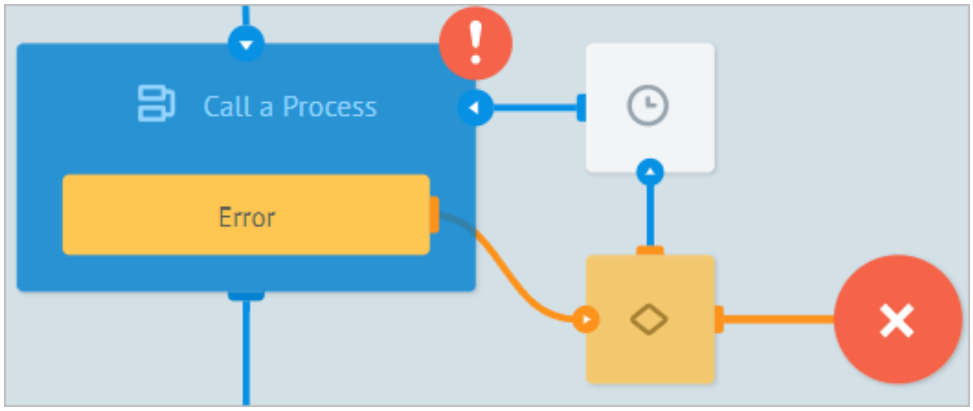
Issue	Cause	Solution
The specified task can't be modified.	The destination Process already has a task with the same reference.	Make sure to specify a unique task reference for each task within a Process.
	The task you are trying to send to a Process does not have the input parameters required by the destination Process.	Make sure to include all the required parameters or modify the list of those.

Table 25: **Copy Task node issues.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3.13 Call a Process

Overview



Screenshot 235: **Call a Process node view.**

The **Call a Process** node allows you to simplify and optimize workflows, making them more understandable and easier to maintain. You can increase efficiency by incorporating the logic and functionality of other Processes into your current Process. Simply select the desired Process from the catalog or specify its identifier – and the **Call a Process** node will send a task to that Process and receive the result.

Benefits of the **Call a Process** node:

- Breaking complex Processes into simpler parts
- Ability to reuse code without duplication
- Isolation of Processes from each other

In the called Process, you must use the **Process Response** node. Without it, tasks will get stuck in the **Call a Process** node and won't be able to return a result – like a function in programming without a return statement.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

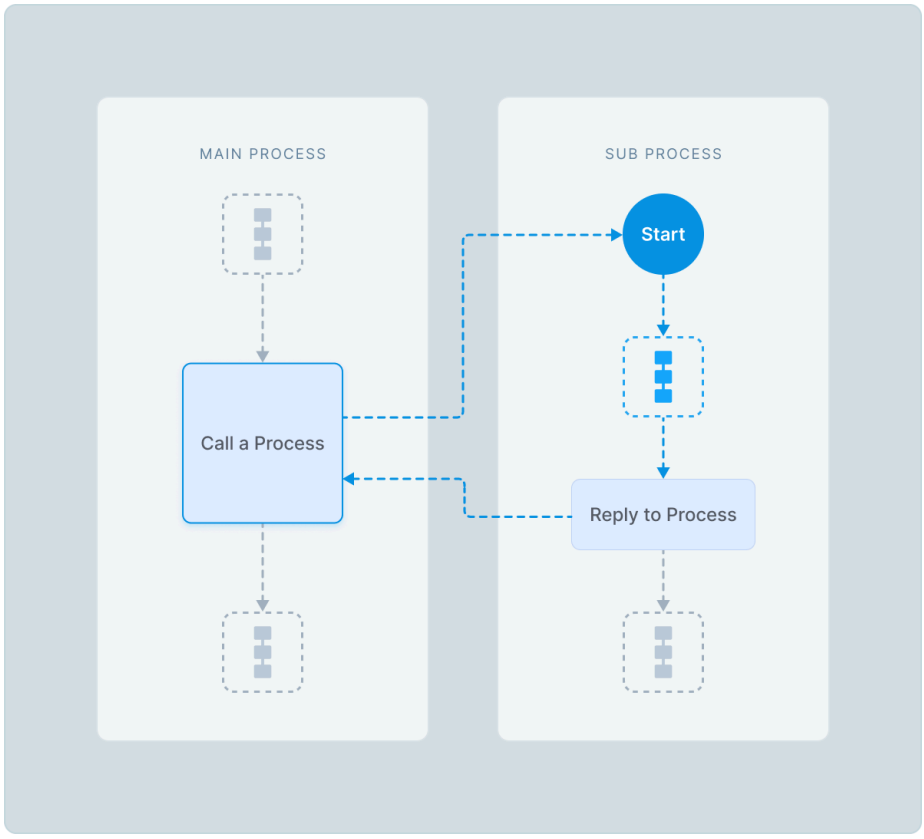


Diagram 17: **Call a Process node flow**.

Avoid recursion by using **Call a Process**. Although it is technically possible to use the **Reply to Process** node within the same Process, we advise against it, but instead, it is better to use the **Copy Task** node.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

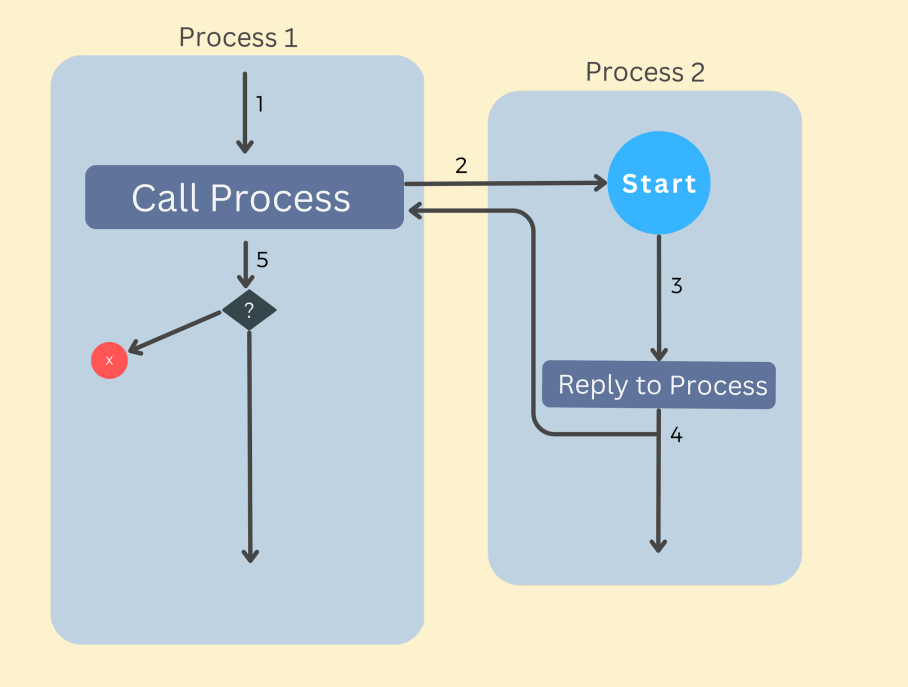
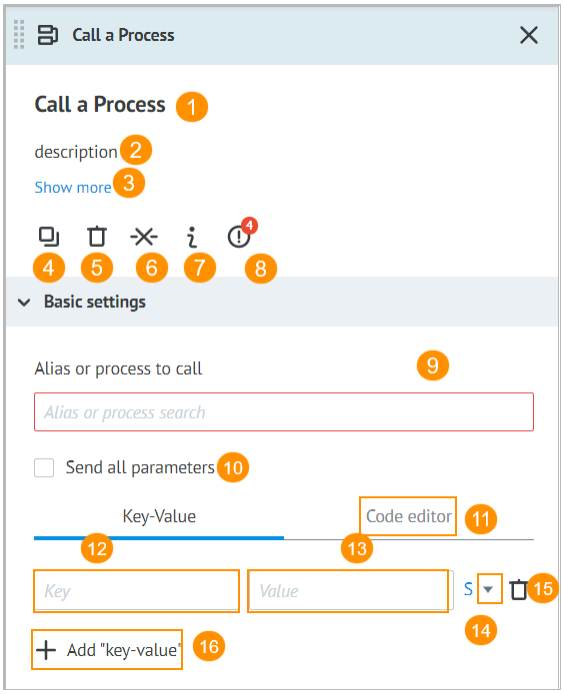
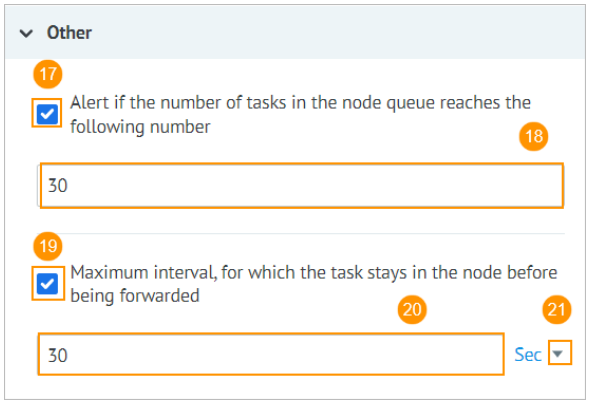
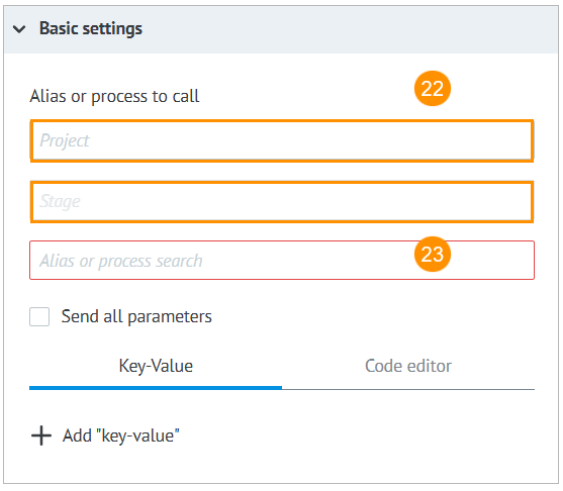


Diagram 18: **Two Processes interaction in Call a Process and Reply to Process scheme**.

Node Settings

The **Call a Process** node has the following parameters:

**2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.**

Basic settings	Other settings
	
Basic settings for nodes inside Stages	
	

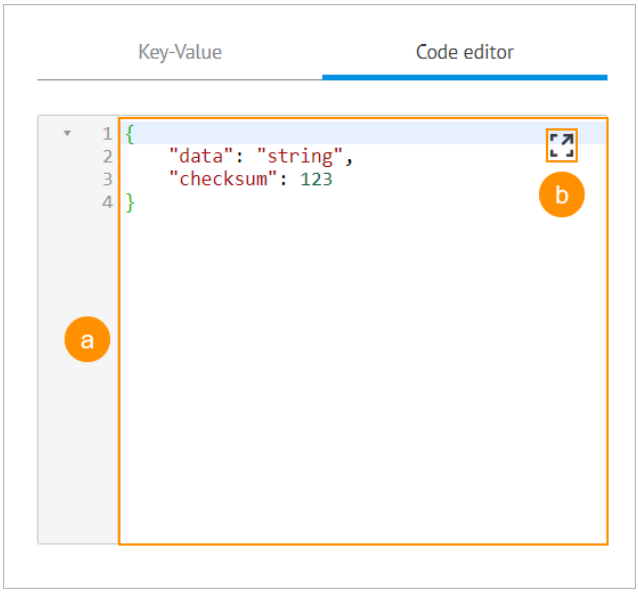
Screenshot 236: **Call a Process node settings.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

1. **Title** [Optional] [Field] [String]**:** Node title.
2. **Description** [Optional] [Field] [String]: Node description**.**
3. **Show more** [Optional] [Action]: Long descriptions are reduced to a compact view to simplify node settings. Click to see the whole description.
4. **Copy node** [Optional] [Action]: Creates a copy of the current node with all settings**.**
5. **Remove node** [Optional] [Action]: Deletes current node from Process.
6. **Remove links** [Optional] [Action]: Removes all connections to/from current node.
7. **Info** [Optional] [Action]: Displays node ID.
8. **Errors info** [Optional] [Action]: Displays node errors.
9. **Alias or process to call** [Required] [Field/Action] [String]: Select the Process to call by clicking the directory icon or by specifying its Process ID/Alias.
10. **Send all parameters** [Optional] [Action]: When the checkbox is selected: All current task parameters will be sent along with any parameters specified in the node.

    When the checkbox is not selected: Only the parameters specified in the node will be sent.

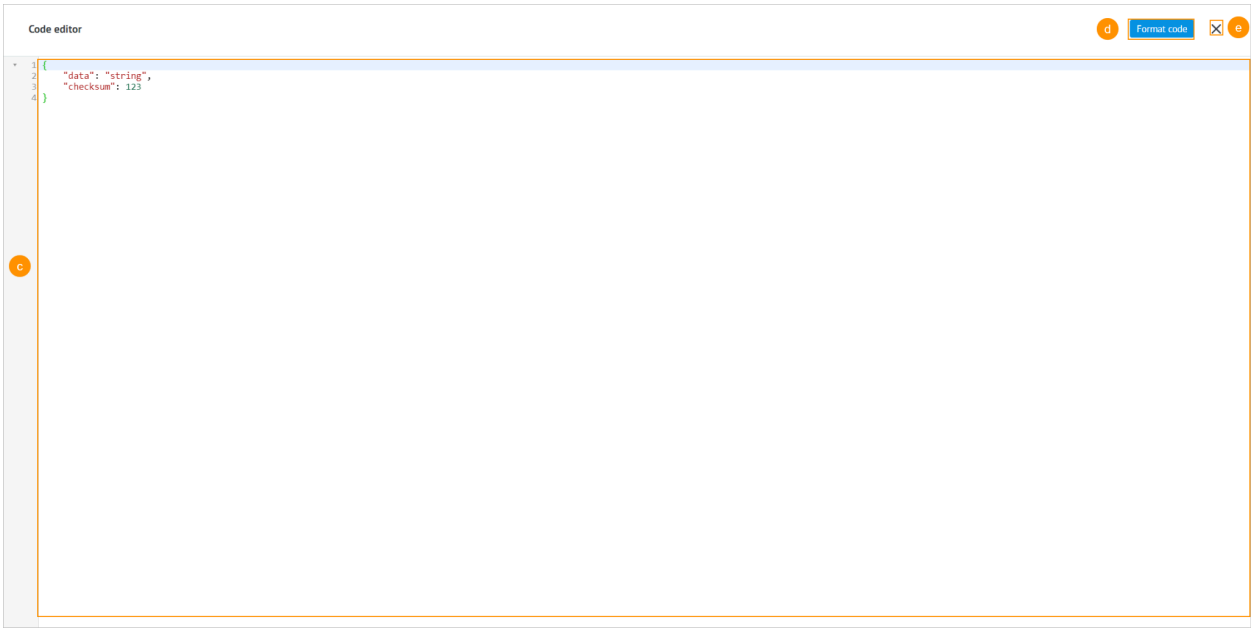
11. **Code editor** [Optional] [Action]: Shows the JSON format of the entered Key and Value. Click **Code editor** to open the code editor tab, where:



Screenshot 237: **Code editor tab.**

a. **Code editor field** [Optional] [Field] [Any]: Enter new keys and values or edit the existing ones in JSON format.
b. **Full-screen mode** [Optional] [Action]: Opens the **Code editor** field in the full-screen mode, where:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 238: **Code editor full-screen mode.**

c. **Code editor field** [Optional] [Field] [Any]: Edit or add keys in values.
d. **Format code** [Optional] [Action]: Format your current code.
e. **Quit the full-screen mode** [Optional] [Action]: Quit the full-screen mode.

12. **Key** [Required] [Field] [String]: Key to specify a static name or a variable in {{}} curly brackets
To add a new parameter, specify a unique name, to update an existing parameter, use its original name as a key.

13. **Value** [Required] [Field] [Any]: The value you want to assign to the corresponding Key parameter. See data types descrition for more details.

14. **Type of data** [Required] [Action]: Selects the data type for the **Value**:
* String (**S**)
* Number (**N**)
* Boolean (**B**)
* Array (**A**)
* Object (**O**)

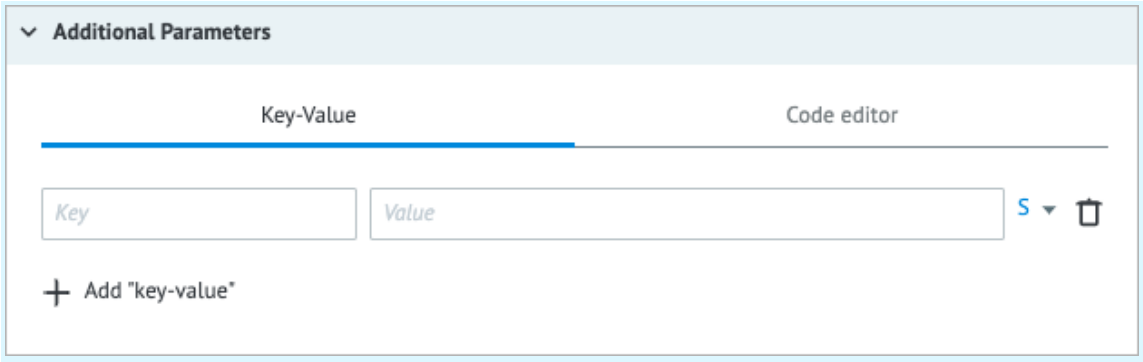
Note: For more information, go to Data types.

15. **Delete key-value** [Optional] [Action]: Delete the key-value pair.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

16. **Add key-value** [Optional] [Action]: Add a new key-value pair.

- To add a key-value, click **+ Add “key-value”** below the **Key** field.
- To delete the key-value line, click the trash icon on the right side of the key-value line.



Screenshot 239: **A key-value pair add**.

17. **Alert if the number of tasks in the node queue reaches the following number** [Optional] [Action]: Helps monitor whether the number of tasks in the node exceeds the specified threshold.

18. **Number of tasks in the queue** [Optional/Required if **Alert if the number of tasks in the node queue reaches the following number** is selected] [Field] [Number]: Number of tasks in the queue which triggers an alert when reached.

19. **Maximum interval, for which the task stays in the node before being forwarded** [Optional] [Action]: Select to limit the time a task is allowed to be in the node in seconds, minutes, hours, and days.

20. **Maximum interval** [Optional/Required if **Maximum interval, for which the task stays in the node before being forwarded**] [Field] [Number]: The amount of time a task is allowed to be in the node. It has a minimum value of 30 seconds. You can set a shorter interval by using the **Unixtime** function.

21. **Maximum interval measuring unit** [Required] [Action]: Selects the measuring unit for the maximum interval:

- Seconds
- Minutes
- Hours
- Days.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

22. **Project** [Required] [Field/Action] [String]: Choose the Project containing the Process which you would like to call by clicking the directory icon or by specifying its Process ID, name or Alias.

Note: this parameter is available with the **Alias or process to call** parameter and applies to **Call a Process** nodes created within Stages in Projects. This feature allows you to call Processes located in Stages of different Projects.

23. **Stage** [Required] [Field/Action] [String]: Choose the Stage containing the Process which you would like to call by clicking the directory icon or by specifying its Process ID, name or Alias.

Note: this parameter is available with the **Alias or process to call** parameter and applies to **Call a Process** nodes created within Stages in Projects. This feature allows you to call Processes located in Stages of different Projects.

Examples

Basic usage of the Call a Process node
Description: Below, you can see the example where the **Call a Process** node is used to send a task from the **Call** Process to the **Reply** Process that sends back a reply.

Precondition:

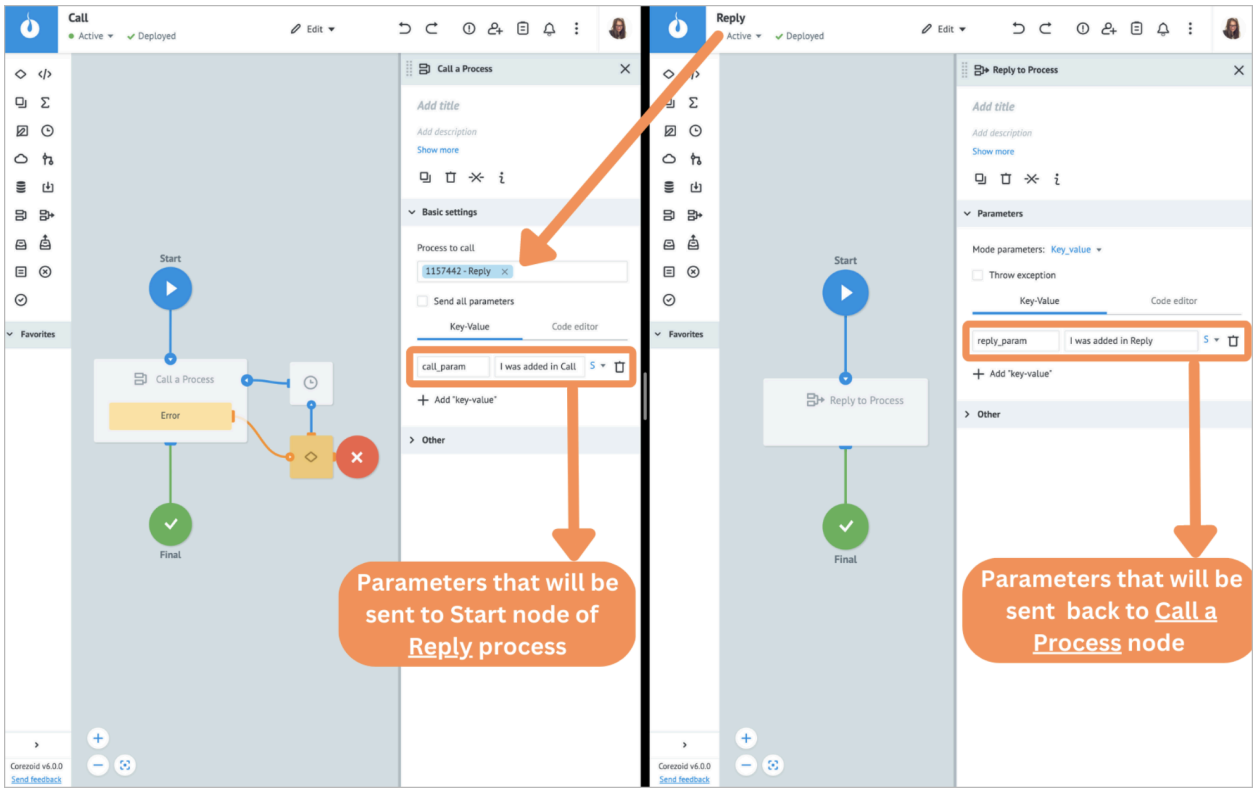
1. Add the **Call a Process** node to your **Call** Process.
2. In **Basic settings** of the **Call a Process** node details panel, add:
    a. The **Reply** Process you want to call.
    b. The **call_param** parameter.
    c. The **I was added in Call** value.
3. Clear the **Send all parameters** checkbox.

    Parameters can be sent when:

    a. The **Send all parameters** checkbox is selected + no parameters: All parameters that are part of the original task are sent.
    b. The **Send all parameters** checkbox is cleared + parameters: Only the specified parameters are sent. You can either create new parameters or send parameters that are part of the original task.
    c. The **Send all parameters** checkbox is selected + parameters: The specified parameters are added to the original task parameters. If you specify a parameter with the same name as one of the task parameters, only the specified parameter will be sent.
    d. The **Send all parameters** checkbox is cleared + no parameters: The option is not permitted; at least one parameter must be sent.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

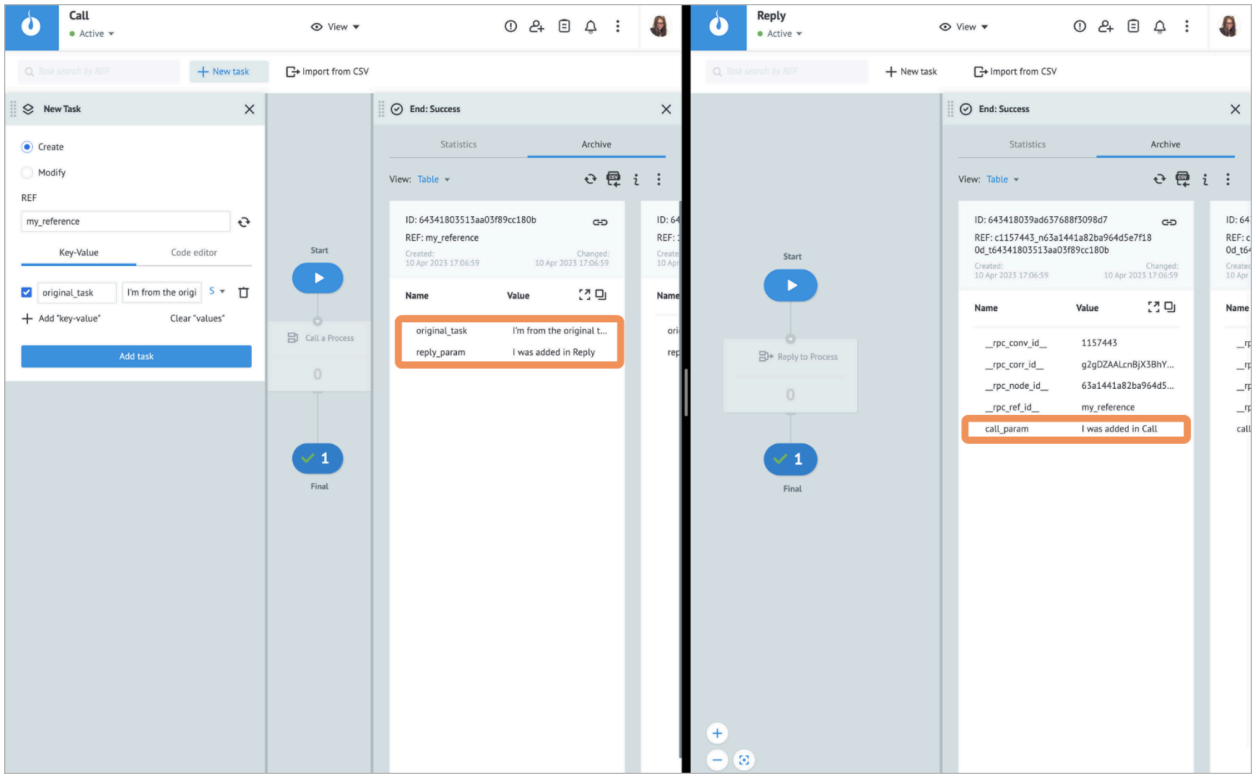
4.
5. Add the **Reply to Process** node to the **Reply** Process you are going to call.
6. In **Basic settings** of the **Reply to Process** node details panel:
    a. Add the **reply_param** parameter.
    b. Add the **I was added in Reply** value.
    c. Clear the **Send all parameters** checkbox.



Screenshot 240: **Parameters in tasks in Call a Process and Reply to Process nodes.**

Flow: Add a task with the **original_task** parameter and the **I'm from the original task** value to the **Call** Process. Once the task reaches the **Call a Process** node, it gets stuck in the node, a new task with the **call_param** and system parameters is created and sent to the **Reply** Process. After the new task enters the **Reply to Process** node of the **Reply** Process, a new task with the **reply_param** parameter is generated and sent back to the **Call a Process** node while the task that entered the node continues to move downstream. Once the task from the **Reply to Process** node is received in the **Call a Process** node, its parameters are merged with the original task parameters and added to the original_task and **reply_param** parameters.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



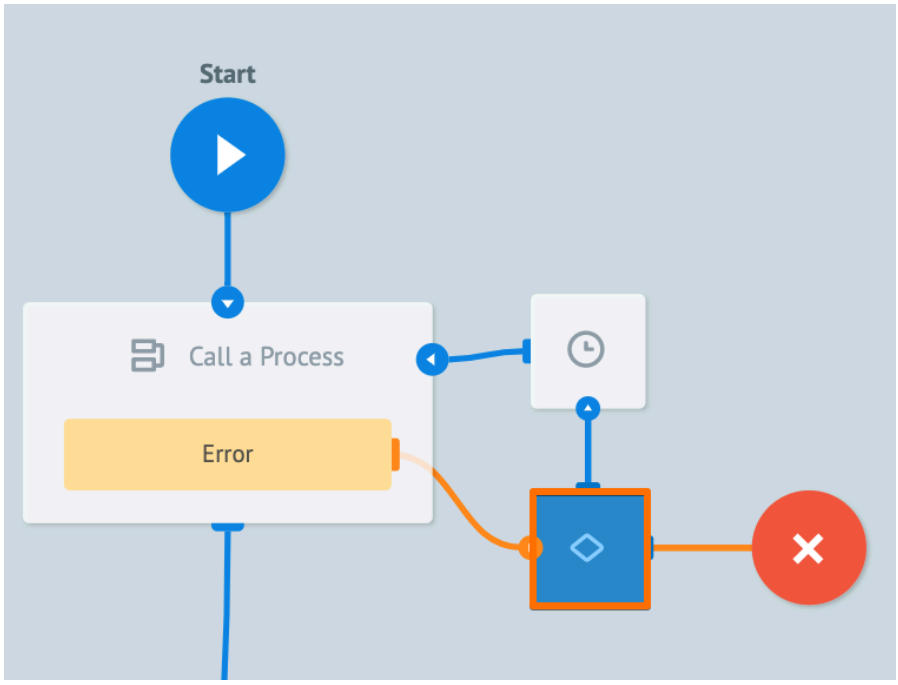
Screenshot 241: **Parameters in tasks in Call a Process and Reply to Process nodes.**

You can access a single task parameter or a whole task stored in a State Diagram by Getting Task Parameters from State Diagrams.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Error handling & troubleshooting

When an error occurs in the **Call a Process** node, a task goes to the auxiliary **Condition** output node that is used for storing error parameters.



Screenshot 242: **Auxiliary node for Call a Process errors handling.**

When an error occurs during the task processing, you may see the following error parameter names in the task.

Error parameter name	Error type
`__conveyor_rpc_return_type_error__`	**Hardware** (system error), **Software** (error in a node logic/settings.)
`__conveyor_rpc_return_type_tag__`	*Error tag.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

`__conveyor_rpc_return_type_description__`	Error description in human-readable language; can be static or dynamic.

Table 26: **Error parameter names.**

*The error `tag __conveyor_rpc_return_type_tag__` may have the following values.

Value	Cause	Solution
access_denied	You do not have permission to access the Process you are trying to call.	Normally, the UI doesn't allow you to select a Process you do not have access to, but you can circumvent it by passing the Process ID as a variable. Make sure you have access to the Process you are calling.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

wrong_validate_params	The called Process was configured to receive specific parameter types (**String**, **Number**, **Object**, and so on), but at least one parameter passed from the **Call a Process** node doesn't match the one that was specified.	Make sure that parameter types match the parameter values that you have specified in the **Call a Process** node.
rpc_task_wrong_convert_param	Parameter type specified in the **Call a Process** node does not match the actual value.	The parameter type specified in the **Call a Process** node does not match the actual value.

Table 27: **Call a Process node errors.**

When working with your Process, you may encounter the following issues.

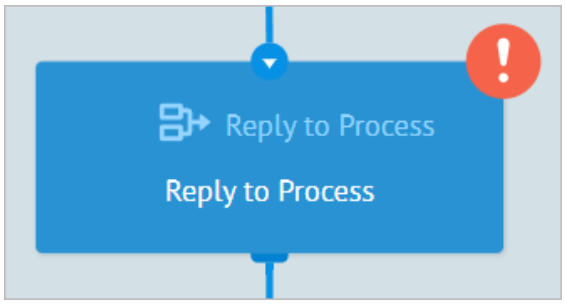
Issue	**Cause**	**Solution**
Task is stuck in the **Call a Process** node.	The task inside the called Process has not reached the **Reply to Process** node.	Make sure that the called Process has the **Reply to Process** node and it is accessible.

Table 28: **Call a Process node issues.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3.14 Reply to Process

Overview



Screenshot 243: **Reply to Process node view.**

The **Reply to Process** node serves to send data back from the Process that was called by the **Call a Process** node. This node must be used in conjunction with the **Call a Process** node and cannot be used alone.

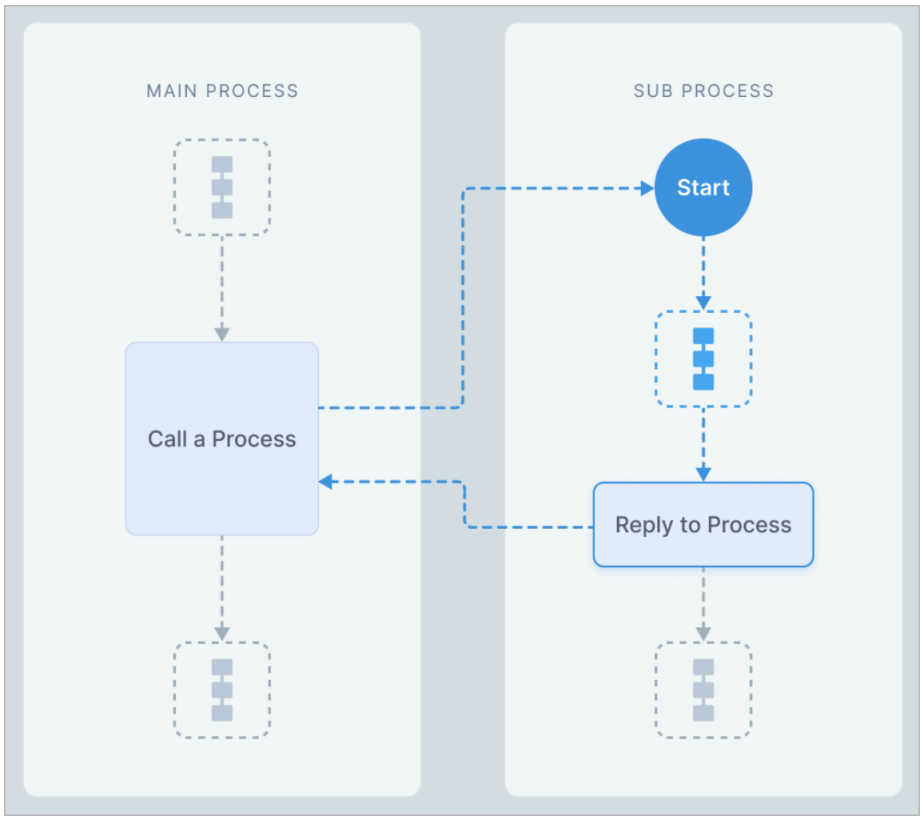


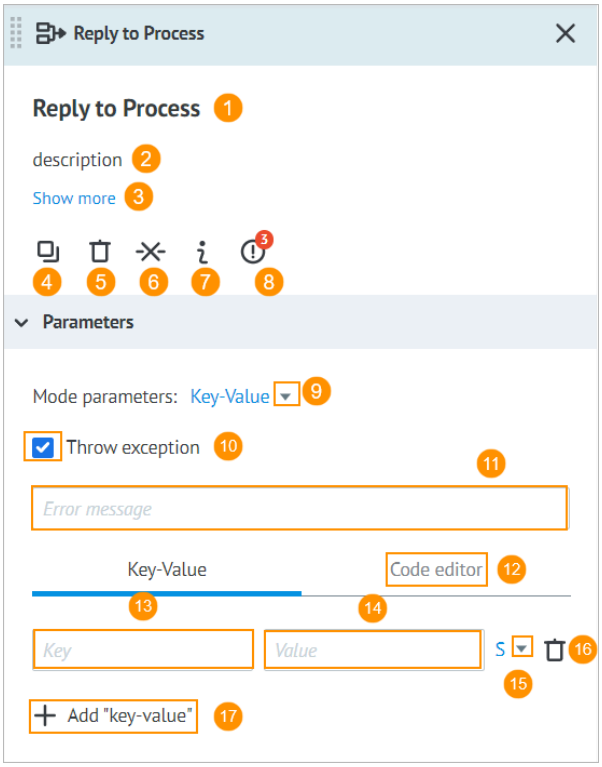
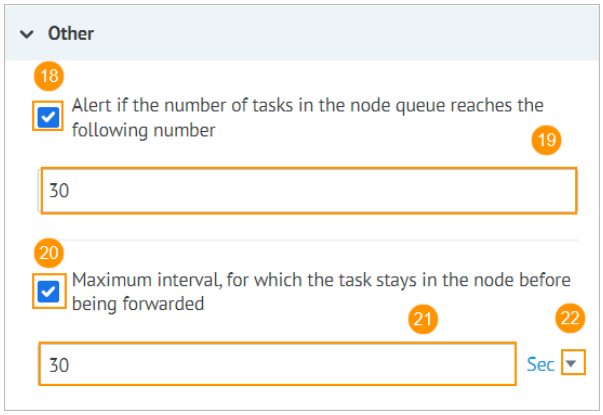
Diagram 19: **Reply to Process node flow**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

You can use the **Reply to Process** node to differentiate between valid and invalid responses by marking them later with the **Throw exception** checkbox. This will separate them into an error node.

Node Settings

The **Reply to Process** node has the following parameters:

Basic settings	Other settings
	

Screenshot 244: **Reply to Process parameters.**

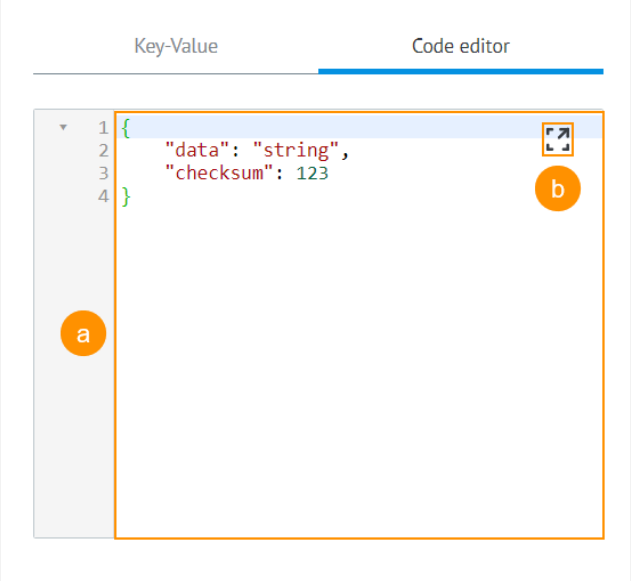
1. **Title** [Optional] [Field] [String]**:** Node title.
2. **Description** [Optional] [Field] [String]: Node description**.**
3. **Show more** [Optional] [Action]: Long descriptions are reduced to a compact view to simplify node settings. Click to see the whole description.
4. **Copy node** [Optional] [Action]: Creates a copy of the current node with all settings**.**
5. **Remove node** [Optional] [Action]: Deletes current node from Process.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

6. **Remove links** [Optional] [Action]: Removes all connections to/from current node.
7. **Info** [Optional] [Action]: Displays node ID.
8. **Errors info** [Optional] [Action]: Displays node errors.
9. **Mode parameters** [Required] [Action]: Select the format you can specify data in the node to send it to the parent Process:

- Key-Value: specify **Keys** and their **Values**.
- Keys: specify Keys only.

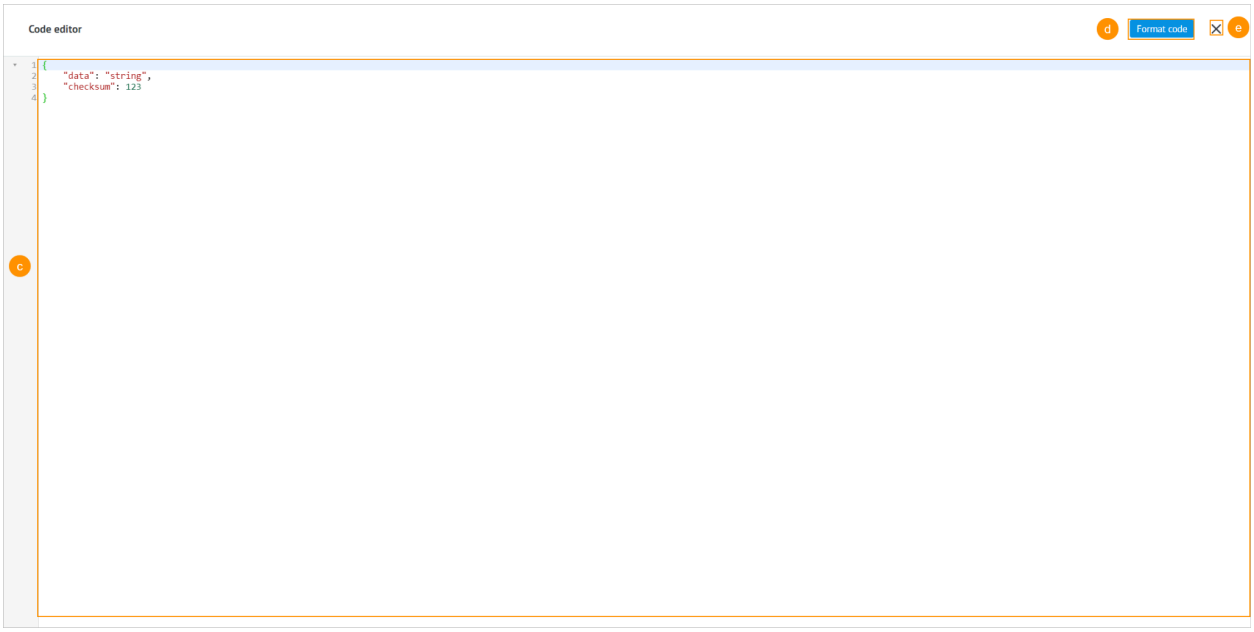
10. **Throw exception** [Optional] [Action]: When selected, notifies the main Process of an error. You can customize the notification by adding an error message.
11. **Error message** [Optional/Required if **Throw exception** is selected] [Field] [String]
12. **Code editor** [Optional] [Action]: Shows the JSON format of the entered Key and Value. Click **Code editor** to open the code editor tab, where:



Screenshot 245: **Code editor tab.**

a. **Code editor field** [Optional] [Field] [Any]: Enter new keys and values or edit the existing ones in JSON format.
b. **Full-screen mode** [Optional] [Action]: Opens the **Code editor** field in the full-screen mode, where:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 246: **Code editor full-screen mode.**

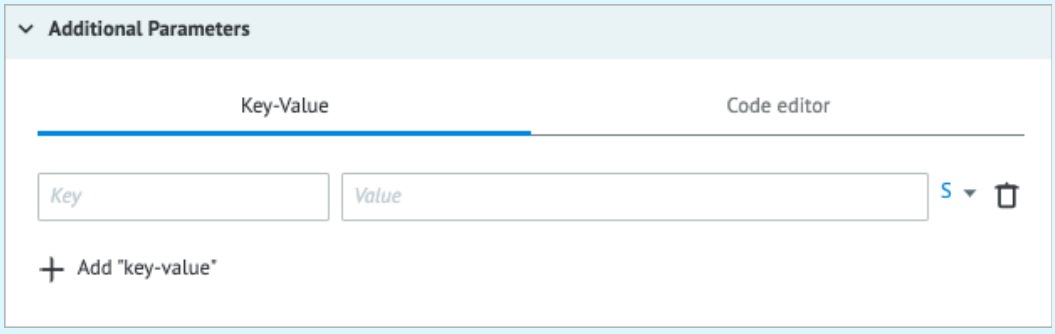
c. **Code editor field** [Optional] [Field] [Any]: Edit or add keys in values.
d. **Format code** [Optional] [Action]: Format your current code.
e. **Quit the full-screen mode** [Optional] [Action]: Quit the full-screen mode.

13. **Key** [Required] [Field] [String]: Key to specify a static name or a variable in {{}} curly brackets
    To add a new parameter, specify a unique name, to update an existing parameter, use its original name as a key.
14. **Value** [Required] [Field] [Any]: The value you want to assign to the corresponding Key parameter.
    See data types descrition for more details.
    Note: is inactive when **Keys** is selected in **Mode parameters**.

You must specify at least one parameter:

- To add a key-value, click **+ Add "key-value"** below the **Key** field.
- To delete the key-value line, click the trash icon on the right side of the key-value line.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 247: **Adding Key and Value to the node.**

15. **Type of data** [Required] [Action]: Selects the data type for the **Value**:
    * String (**S**)
    * Number (**N**)
    * Boolean (**B**)
    * Array (**A**)
    * Object (**O**)

    Note: For more information, go to Data types.

16. **Delete key-value** [Optional] [Action]: Delete the key-value pair.

17. **Add key-value** [Optional] [Action]: Add a new key-value pair.

18. **Alert if the number of tasks in the node queue reaches the following number** [Optional] [Action]: Helps monitor whether the number of tasks in the node exceeds the specified threshold.

19. **Number of tasks in the queue** [Optional/Required if **Alert if the number of tasks in the node queue reaches the following number** is selected] [Field] [Number]: Number of tasks in the queue which triggers an alert when reached.

20. **Maximum interval, for which the task stays in the node before being forwarded** [Optional] [Action]: Select to limit the time a task is allowed to be in the node in seconds, minutes, hours, and days.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

21. **Maximum interval** [Optional/Required if **Maximum interval, for which the task stays in the node before being forwarded**] [Field] [Number]: The amount of time a task is allowed to be in the node. It has a minimum value of 30 seconds. You can set a shorter interval by using the **Unixtime** function.

22. **Maximum interval measuring unit** [Required] [Action]: Selects the measuring unit for the maximum interval:

- Seconds
- Minutes
- Hours
- Days.

Examples

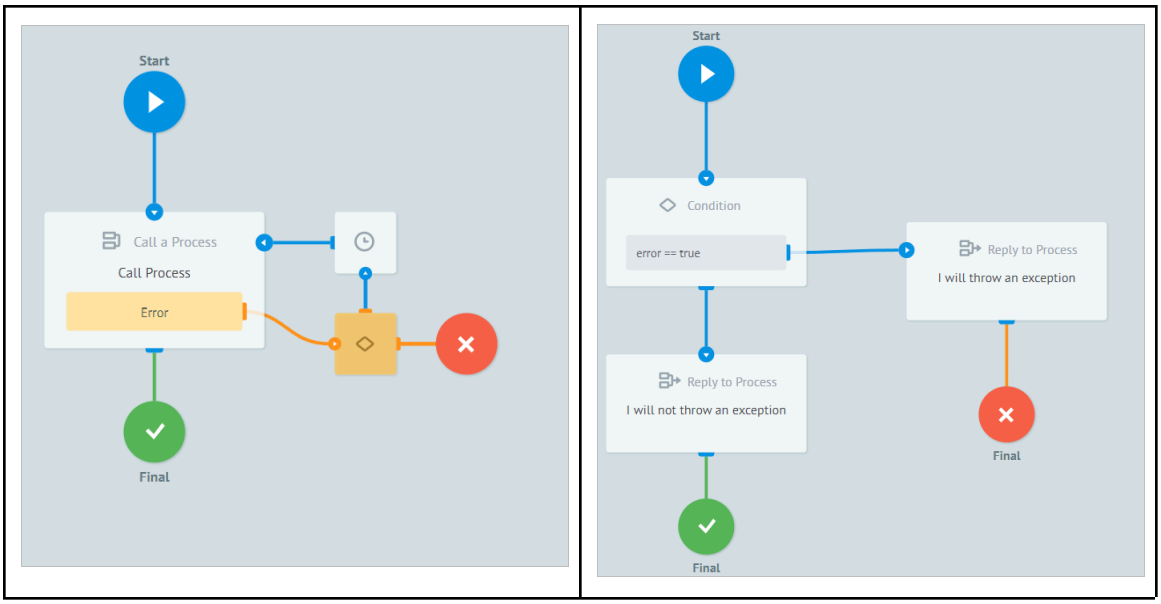
Basic usage of the Reply to Process node
Description: In the example below, you can see the usage of two **Reply to Process** nodes in one Process.

Precondition:

1. Create the **Reply** Process with one **Condition** node and two **Reply to Process** nodes.
2. Create the **Call** Process with the **Call a Process** node that includes the **call_param** parameter.

Call Process	**Reply Process**

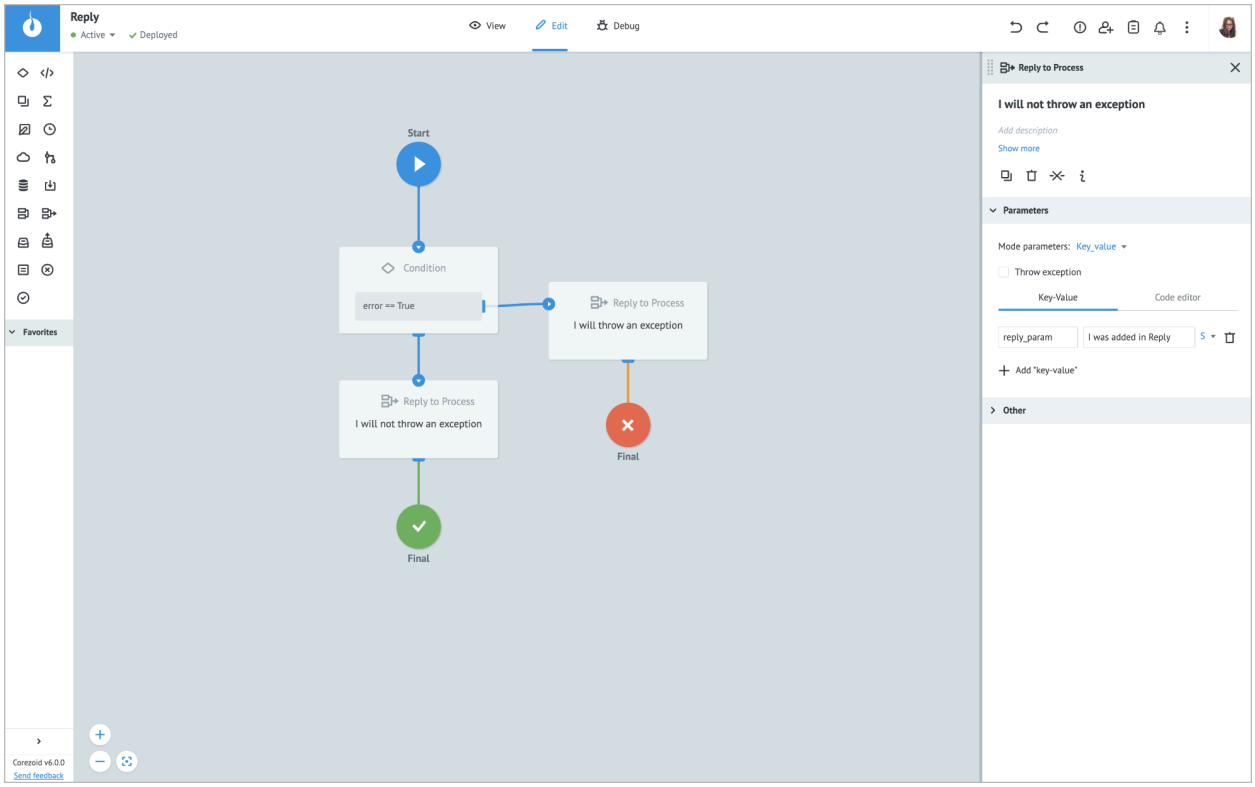
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 248: **Call Process and Reply Process Processes.**

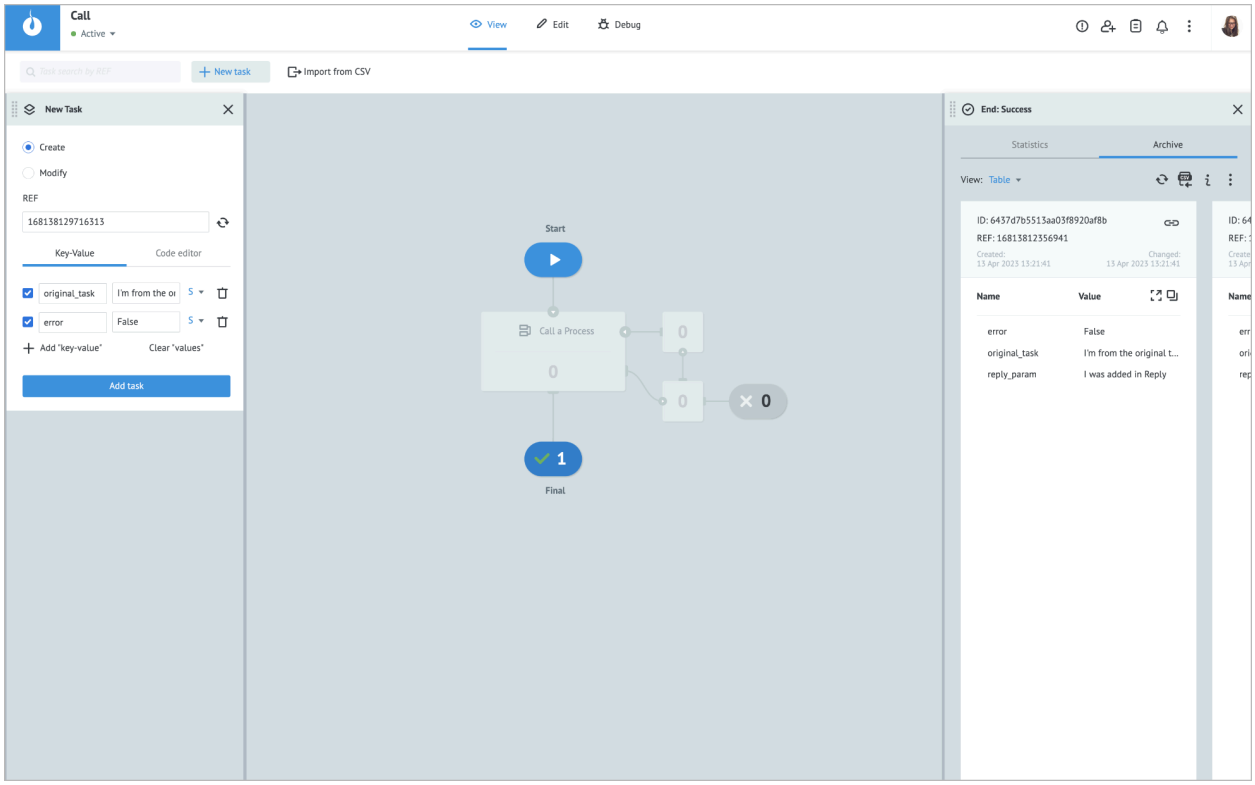
Flow:
The **Condition** node checks the value of the **error** parameter:

- If the value is **False**, it leads it to the **Reply to Process** node that has the **Throw an exception** checkbox cleared.



2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

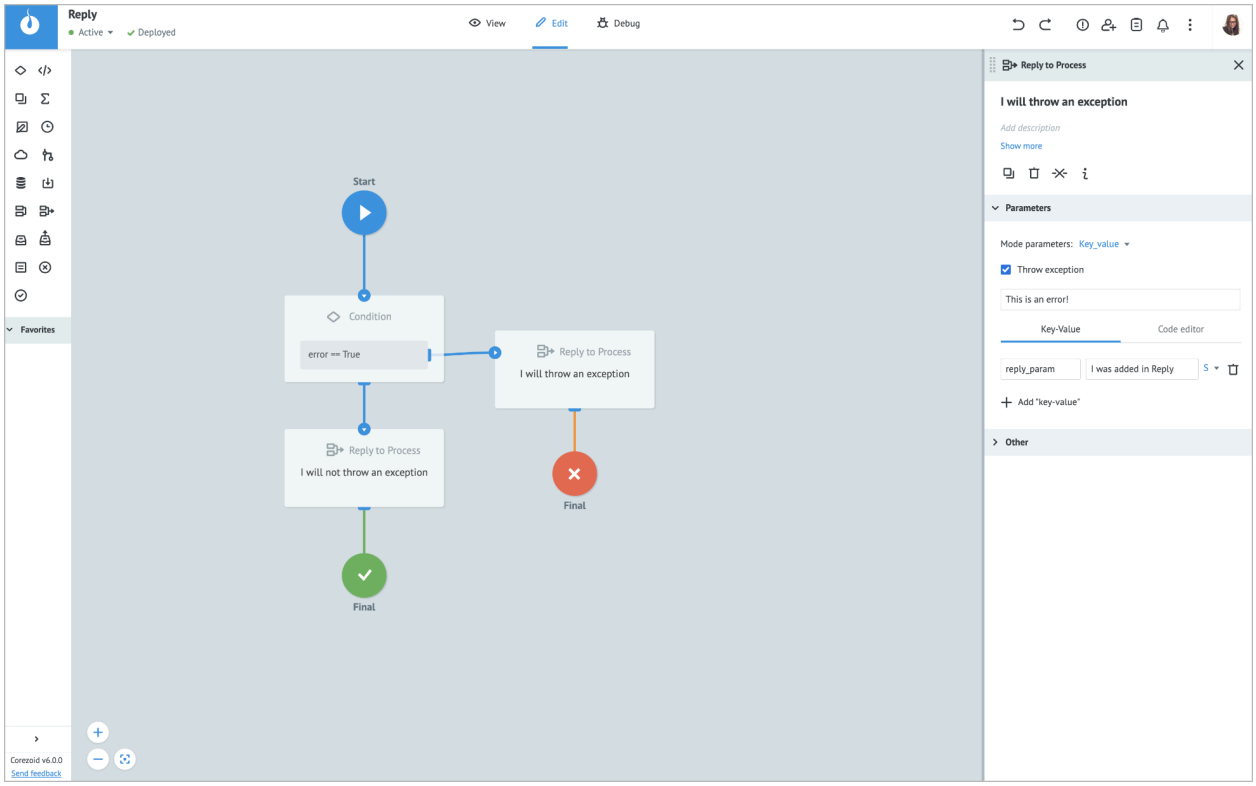
Screenshot 249: **Task movement from Call a Process to a Reply to Process.**
The task is sent back to the connected **Call a Process** node and has the parameters that were listed in the **Reply to Process** node.



Screenshot 250: **Task return to the Call a Process.**

- If the value is **True**, the task is routed to the **Reply to Process** node that has the **Throw an exception** checkbox selected and an error message.

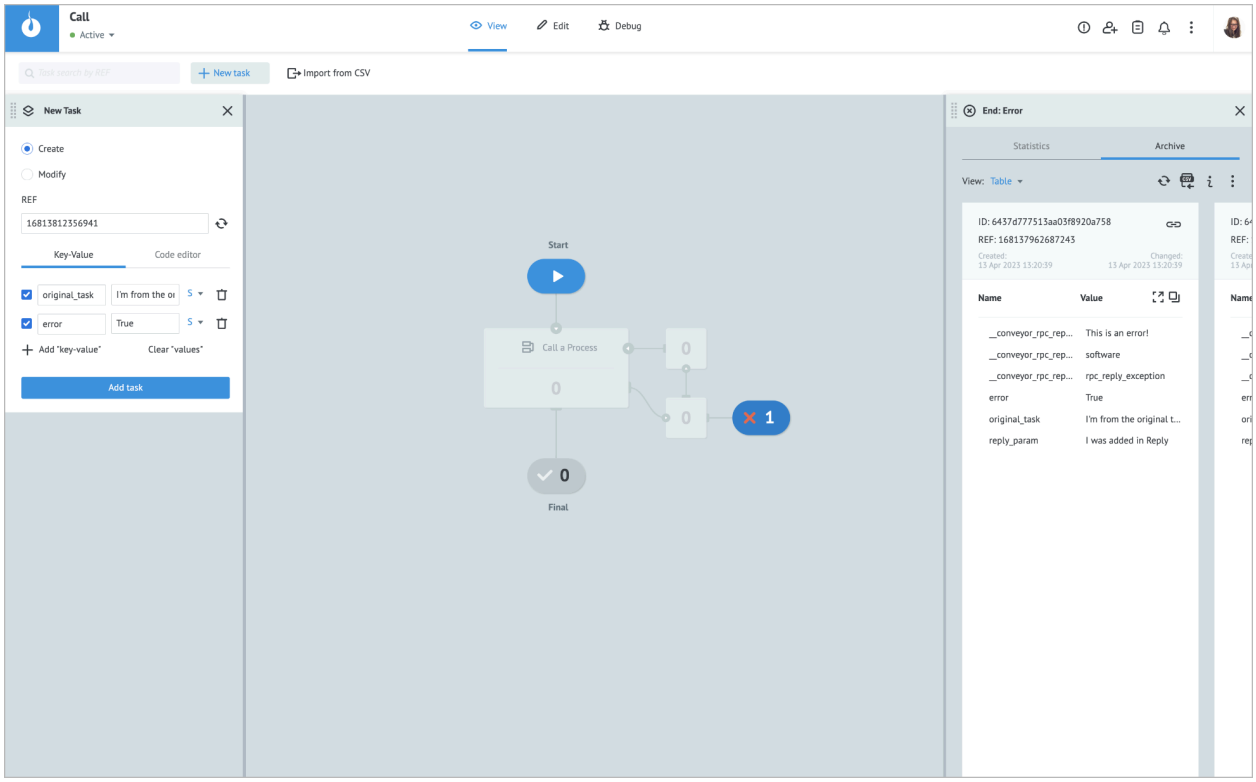
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 251: **Throw exception box selected.**

The task goes to the **Error** node of the **Call** Process and has all the typical error handling parameters, the error message specified before, along with the **reply_param** value.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 252: **Task is forwarded to the Error node.**

No matter which parameters the task has when entering the **Reply to Process** node, only the parameters listed in the node will be sent back.

Error handling & troubleshooting

When an error occurs during the task processing, you may see the following error parameter names in the task.

Error parameter name	Error type
`__conveyor_rpc_reply_return_type_error__`	**Hardware** (system error), **Software** (error in a node logic/settings).
`__conveyor_rpc_reply_return_type_tag__`	*Error tag.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

`__conveyor_rpc_reply_return_type_description__`	Error description in human-readable language; can be static or dynamic.

Table 29: **Error parameter names.**

*The error `tag __conveyor_rpc_reply_return_type_tag__` may have the following values.

Value	Cause	Solution
access_denied	You don't have access to the Process.	Contact the Process owner to get access.
access_denied	You are trying to send or receive tasks from another stage.	It's not allowed to get and send tasks between stages. Use the same stage to send and receive tasks.
access_refused	The Process you are working in is not a State Diagram but it must be a State Diagram.	Check the logic of using a State Diagram. For more information, go to State Diagram.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

conveyor_is_not_active	The target Process is not active.	Make sure that the Process you are trying to copy the task to is active.
crash_api	Error occurred when sending the task to the Process with the **Reply to Process** node.	Contact your administrator for further assistance.
rpc_task_size_overflow	The size of the response data exceeds the limit specified for your environment.	Reduce the size of your task by modifying the data or splitting it into several smaller tasks.
wrong_validate_params	Parameter validation error.	Check the types and values of the parameters you have specified in the **Reply to Process** node.
rpc_task_wrong_convert_param	Parameter value failed to convert.	Check and correct the data type of the parameter

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

		mentioned in the error message.
rpc_reply_task_wrong_convert_param	There is an incorrect value in dynamic parameters that cannot be converted to a given data type.	Correct the value in dynamic parameters.
not_unical_ref	Task reference is not unique.	Check if there is no task with the specified reference in the Process you are trying to send a task to.
not_found_main_node	The **Start** node is missing in the Process to which you are replying.	Check whether the **Start** node exists in the Process you are replying to.
conv_not_found_or_deleted	You are trying to work with a non-existent Process.	Use an existing Process.
not_found_conv_by_alias	Your alias refers to a Process that no longer exists.	Use an existing Process.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

conv_not_found_or_deleted	You are trying to work with a non-existent Process.	Use an existing Process.
rpc_reply_task_size_overflow_limit	The size of the task data exceeds the specified limit set for your environment.	Reduce the size of your task.
rpc_reply_exception	There is an unexpected error.	Contact the support team for further assistance.
rpc_task_eq_convs	You are trying to copy your task to itself.	Choose another task to copy or as a target to copy yours in.
bad_callback_node	The Process you are calling doesn't have the **Callback** node.	Create the **Callback** node in the Process you are calling.
rpc_task_fatal_error	Error of sending the task to the Process with the **Reply to Process** node.	Contact your administrator for further assistance.

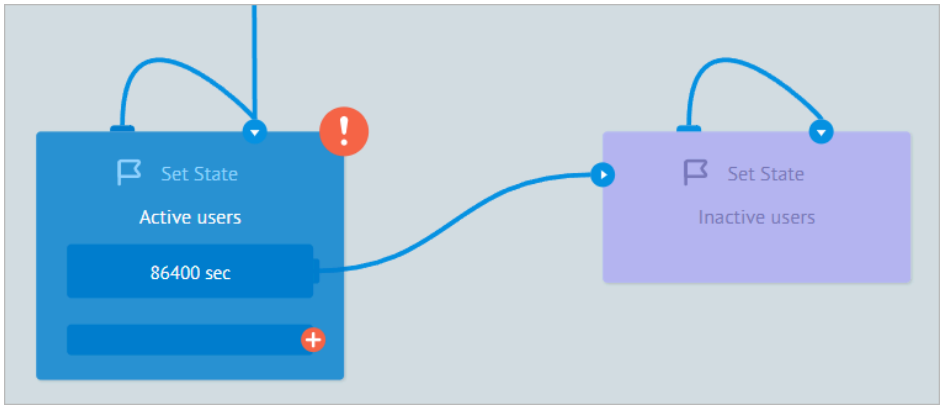
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Table 30: **Reply to Process node errors.**

3.15 Set State

Overview

The **Set State** node is used for a State Diagram only.



Screenshot 253: **Set State node view.**

The **Set State** node is designed to store tasks and control their flow through a State Diagram. It allows you to keep a task within the node until there is the task modification request and certain condition is met after which it is forwarded to the next node associated with that condition. In a way, the **Set State** node in a State Diagram combines the functionality of the **Condition** and Waiting for **Callback** nodes found in Processes.
The general flow:

1. A task enters Set State node and waits for a modification.
2. Waits for any type of modification:
- Manual
- API
- Modify Task node
3. Any modification triggers parameters check:
- Compares current parameters with conditions
4. The task moves forward when:
- Any modification occurred
- Parameters match conditions
5. Returns to wait if no match

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Note: Initial parameter match on entry doesn't trigger forwarding. Only modification + condition match enables forwarding.

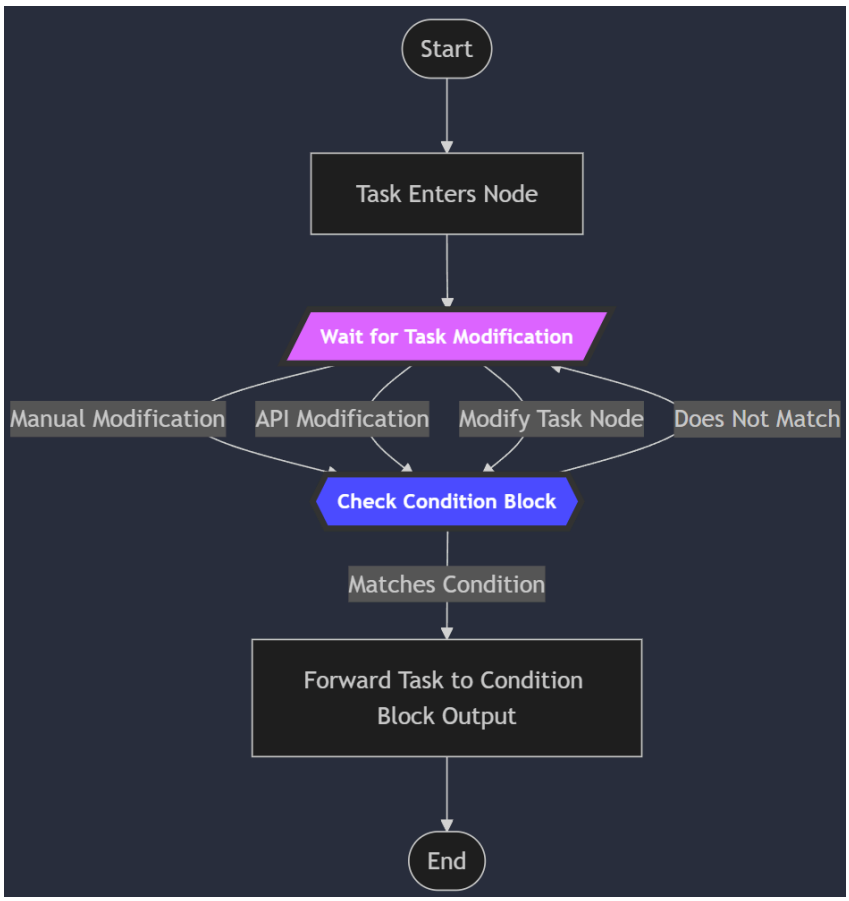
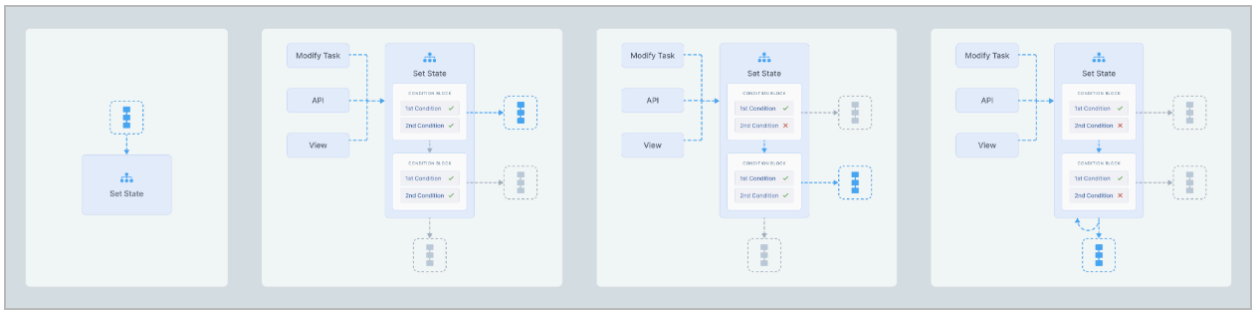


Diagram 20: **Set State task forwarding logic.**

In a State Diagram, you have the flexibility to manage tasks from outside the Diagram by using:

- **Modify Task** node
- **API Call** node
- Manually created or imported tasks in the **View** mode



2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Diagram 21: **Set State node flow.**

Modify Task Interaction:

- Starts with Modify Task node
- Flows to Set State
- Conditions check with decision point
- Proceeds to Next Node or returns to Set State

API Call Interaction:

- Initiates with API Call
- Moves to Set State
- Evaluates conditions
- Advances or remains in Set State

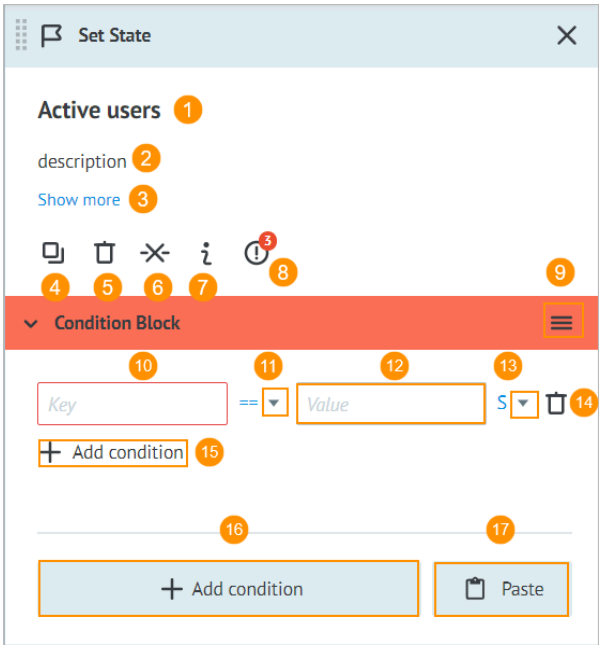
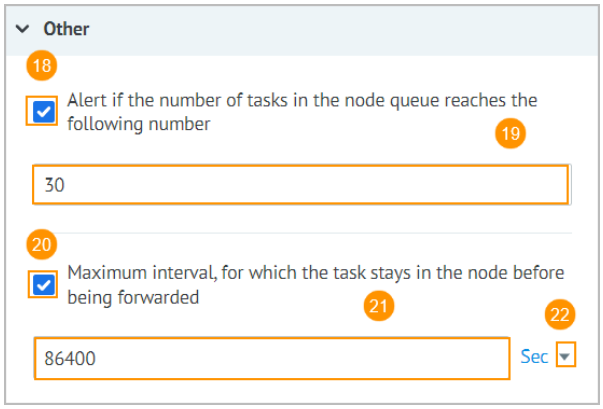
View Mode Interaction:

- Begins with manual task creation
- Enters Set State
- Checks conditions
- Continues or stays in Set State

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Node Settings

The **Set State** node has the following parameters:

Basic settings	Other settings
	

Screenshot 254: **Set State node settings.**

1. **Title** [Optional] [Field] [String]**:** Node title.
2. **Description** [Optional] [Field] [String]: Node description**.**
3. **Show more** [Optional] [Action]: Long descriptions are reduced to a compact view to simplify node settings. Click to see the whole description.
4. **Copy node** [Optional] [Action]: Creates a copy of the current node with all settings**.**
5. **Remove node** [Optional] [Action]: Deletes current node from Process.
6. **Remove links** [Optional] [Action]: Removes all connections to/from current node.
7. **Info** [Optional] [Action]: Displays node ID.
8. **Errors info** [Optional] [Action]: Displays node errors.
9. **Drag Condition Block** [Optional] [Action]: point the mouse cursor on the icon and hold down the left mouse button to drag the Condition Block up or down. Thus, you can change the arrangement of Condition Blocks in the node.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

10. **Key** [Required] [Field] [String]: Key to specify a static name or a variable in {{}} curly brackets.
11. **Comparison operator** [Required] [Action]: The way the key and the value have to be compared:
    - Equal to (==)
    - Not equal (!=)
    - More than (>)
    - More than or equal to (>=)
    - Less than (<)
    - Less than or equal to(<=)
    - Regular Expression (RegExp)).

12. **Value** [Required] [Field] [Any]: Value to compare. Data type depends on the **Type of data** selected. See data types descrition for more details.

Note: For more information, go to Parameters, Functions.

13. **Type of data** [Required] [Action]: Selects the data type for the **Value**:
* String (**S**)
* Number (**N**)
* Boolean (**B**)
* Array (**A**)
* Object (**O**)

Note: For more information, go to Data types.

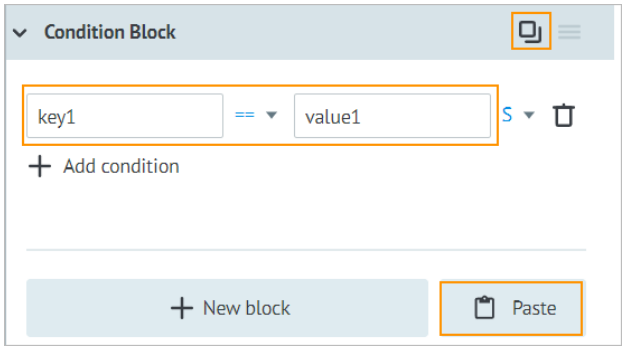
14. **Delete condition** [Optional] [Action]: delete the condition line from the Condition Block.

15. **Add condition** [Optional] [Action]: add a new condition line to the block.

16. **New block** [Optional] [Action]: add a new Condition Block to the node.

17. **Paste** [Optional] [Action]: paste the copied condition block. For at least one condition (key and value) specified in the block when pointing the cursor over the Drag Condition Block icon, the copy icon appears to the left. You can click this icon to copy the Condition Block and paste it to another one using the **Paste** button:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



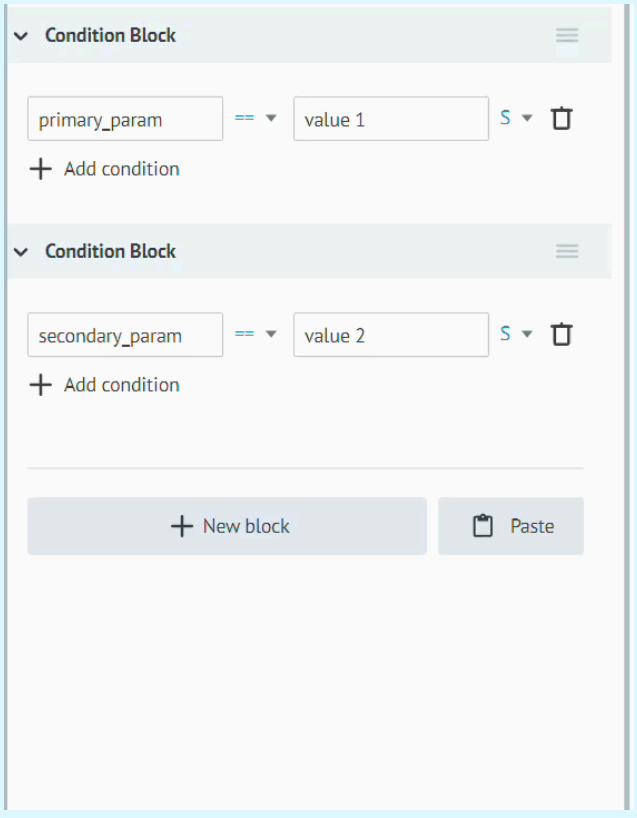
Screenshot 255: **Condition copy and paste.**

Actions available in the Condition Block

You can do the following actions in the **Condition Block**:

- Add a new condition: Below the **Key** field, click **+ Add condition**, and then fill in the needed information in the condition line that appears.
- Delete a condition: On the right side of the condition line, click the trash icon
- Add a new **Condition Block**: Below the condition line, click **+ New block**, and then fill in the needed information in the block that opens.
- Copy a **Condition Block**: On the right side of the **Condition Block**, hover over the red/gray line, and then click the copy icon
- Paste a **Condition Block**: In the lower-right corner of the panel, click **Paste**.
- Delete a **Condition Block**: On the right side of the **Condition Block**, hover over the red/gray line, and then click the trash icon that appears.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 256: **Condition block operations.**

The **Set State** node does not necessarily have to contain a condition. If it does not have a condition, the tasks will stay in it forever.

18. **Alert if the number of tasks in the node queue reaches the following number** [Optional] [Action]: Helps monitor whether the number of tasks in the node exceeds the specified threshold.

19. **Number of tasks in the queue** [Optional/Required if **Alert if the number of tasks in the node queue reaches the following number** is selected] [Field] [Number]: Number of tasks in the queue which triggers an alert when reached.

20. **Maximum interval, for which the task stays in the node before being forwarded** [Optional] [Action]: Select to limit the time a task is allowed to be in the node in seconds, minutes, hours, and days.

21. **Maximum interval** [Optional/Required if **Maximum interval, for which the task stays in the node before being forwarded**] [Field] [Number]: The amount of time a task is allowed to be

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

in the node. It has a default value of 86400 seconds. You can set a shorter interval manually or by using the **Unixtime** function.

22. **Maximum interval measuring unit** [Required] [Action]: Selects the measuring unit for the maximum interval:

- Seconds
- Minutes
- Hours
- Days.

Examples

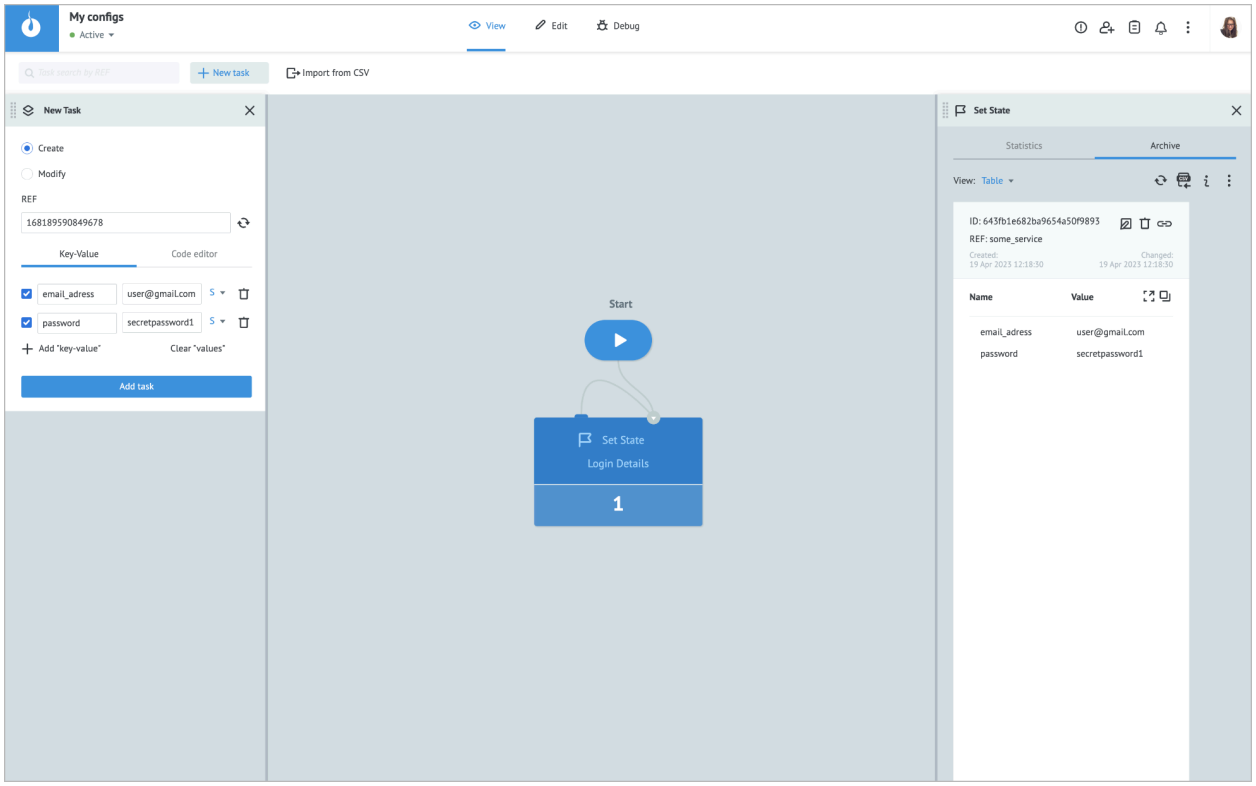
Storage
Description: In the example below, you can see how to safely store any values you might need to run your Processes, such as login and password, within the **Set State** node.

Precondition:

1. Create a State Diagram.
2. Add the **Set State** node.
3. Add tasks with sensitive parameters.

Flow: You can refer to sensitive parameters when you need to use them in other State Diagrams, without revealing their actual value.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 257: **Set State use for storage.**

Reacting to task parameters changes
Description: In the example below, you can see how the Set State node handles and modifies the received tasks.

Precondition:

1. Create a State Diagram.
2. Add the **Set State** node.
3. In the **Condition block** of the **Set State** node, add the **Status** key and the **Inactive** value.

Flow:

- The condition is met.

Screenshot 258: **Set State use-condition is met.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- The condition is not met.

Screenshot 259: **Set State use-condition is not met.**

Error handling & troubleshooting

When working with your State Diagram, you may encounter the following issues.

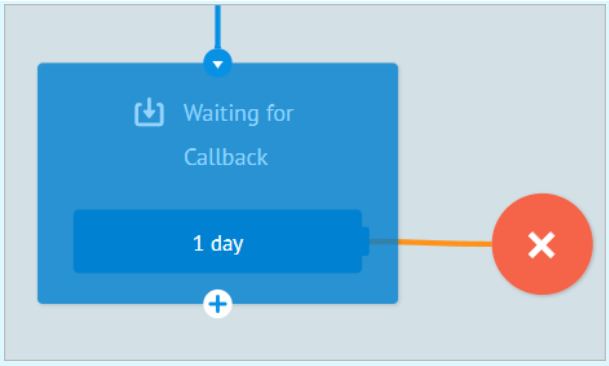
Issue	Cause	Solution
Task is not updated.	If the task has an empty reference, it cannot be updated. An empty reference is not considered to be unique.	Update the task reference manually.

Table 31: **Set State node issues.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3.16 Waiting for Callback

Overview



Screenshot 260: **Waiting for Callback node view.**

The **Waiting for Callback** node is used for a Process only.

The **Waiting for Callback** node suspends task processing until receiving a response. It's primarily used when waiting for user actions - for example, when a chatbot awaits user input.

Like the **Set State** node, tasks remain in the **Waiting for Callback** node until modified or expired. However, while **Set State** requires specific conditions to be met, **Waiting for Callback** releases the task upon any modification.

Ways to trigger a callback:

- Manual task creation in **View** mode
- **Modify Task** node action
- Corezoid API call
- Callback URL request

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

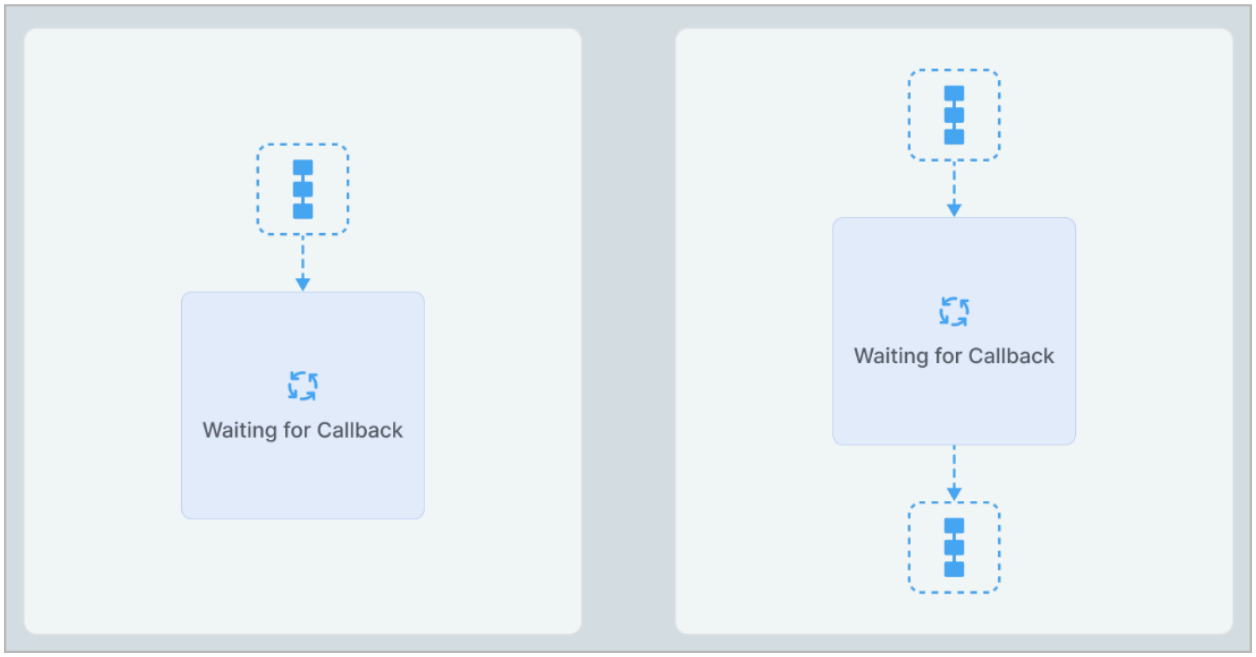


Diagram 22: **Waiting for Callback node flow.**

Corezoid API vs Request to Callback URL:

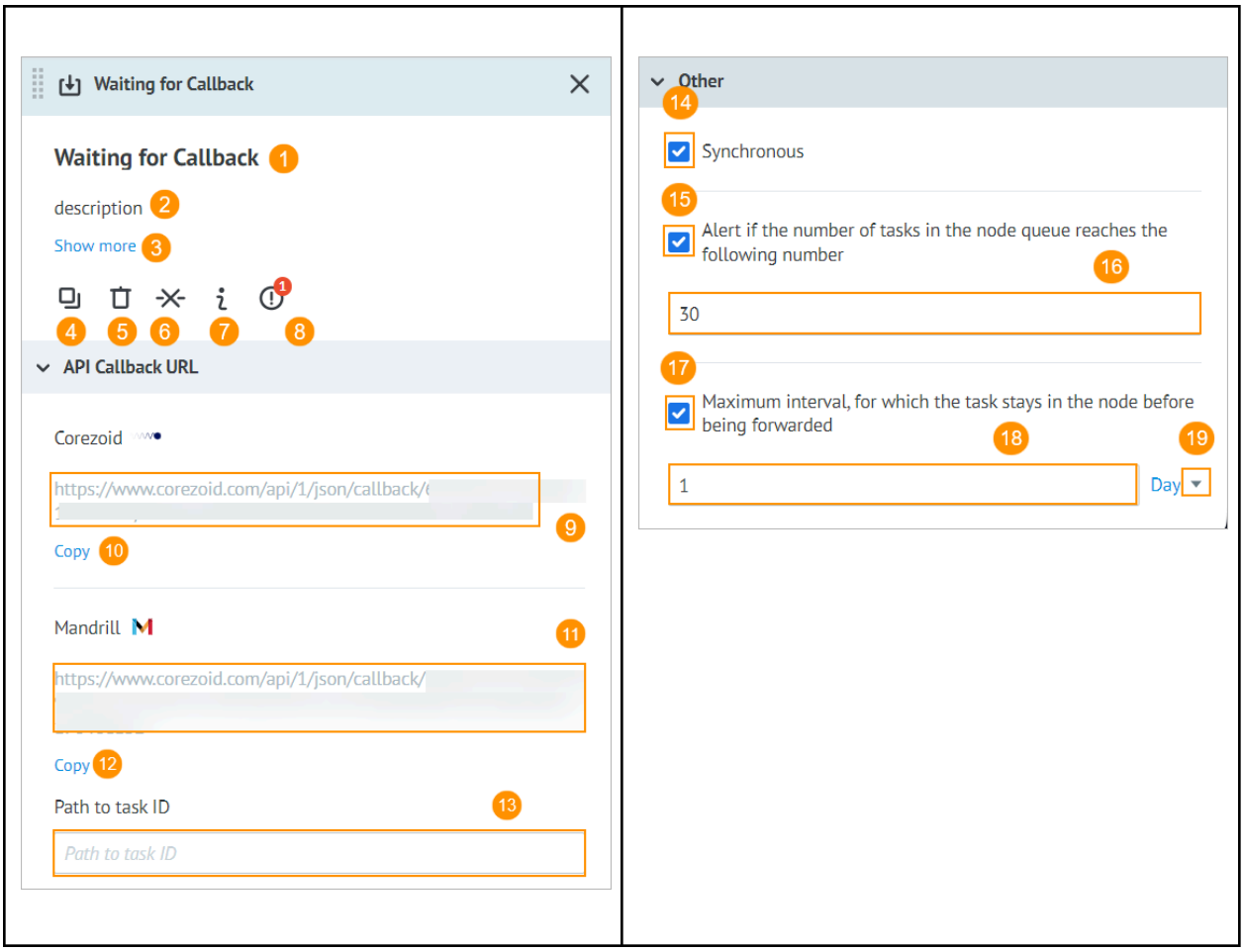
- Corezoid API provides a code-based framework for performing UI operations within a specified format. For more information, go to Corezoid API.
- Callback URL is the API request made specifically to the URL provided within the **Waiting for Callback** node. This method is recommended for replying to tasks from outside Corezoid.

Node Settings

The **Waiting for Callback** node has the following parameters:

Basic settings	Other settings

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 261: **Set State node settings.**

1. **Title** [Optional] [Field] [String]**:** Node title.
2. **Description** [Optional] [Field] [String]: Node description**.**
3. **Show more** [Optional] [Action]: Long descriptions are reduced to a compact view to simplify node settings. Click to see the whole description.
4. **Copy node** [Optional] [Action]: Creates a copy of the current node with all settings**.**
5. **Remove node** [Optional] [Action]: Deletes current node from Process.
6. **Remove links** [Optional] [Action]: Removes all connections to/from current node.
7. **Info** [Optional] [Action]: Displays node ID.
8. **Errors info** [Optional] [Action]: Displays node errors.
9. **Corezoid** [Required] [Field] [String]: Callback URL for an API call.
10. **Copy** [Optional] [Action]: Click to copy the Corezoid URL.
11. **Mandrill** [Required] [Field] [String]: Is a legacy feature, a Callback URL that allows you to receive callbacks directly from Mandrill.
12. **Copy** [Optional] [Action]: Click to copy the Mandrill URL.
13. **Path to task ID** [Optional] [Field] [String]: Displays called task ID in API Call parameters.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

14. **Synchronous** [Optional] [Action]:

- Returns error `task_not_in_current_sync_callback_node` if task not found in current node during Callback URL request.
- When disabled, request succeeds regardless of task location.

15. **Alert if the number of tasks in the node queue reaches the following number** [Optional] [Action]: Helps monitor whether the number of tasks in the node exceeds the specified threshold.

16. **Number of tasks in the queue** [Optional/Required if **Alert if the number of tasks in the node queue reaches the following number** is selected] [Field] [Number]: Number of tasks in the queue which triggers an alert when reached.

17. **Maximum interval, for which the task stays in the node before being forwarded** [Optional] [Action]: Select to limit the time a task is allowed to be in the node in seconds, minutes, hours, and days.

18. **Maximum interval** [Optional/Required if **Maximum interval, for which the task stays in the node before being forwarded**] [Field] [Number]: The amount of time a task is allowed to be in the node. It has a minimum value of 30 seconds and default value of 1 day. You can set a shorter interval by using the **Unixtime** function.

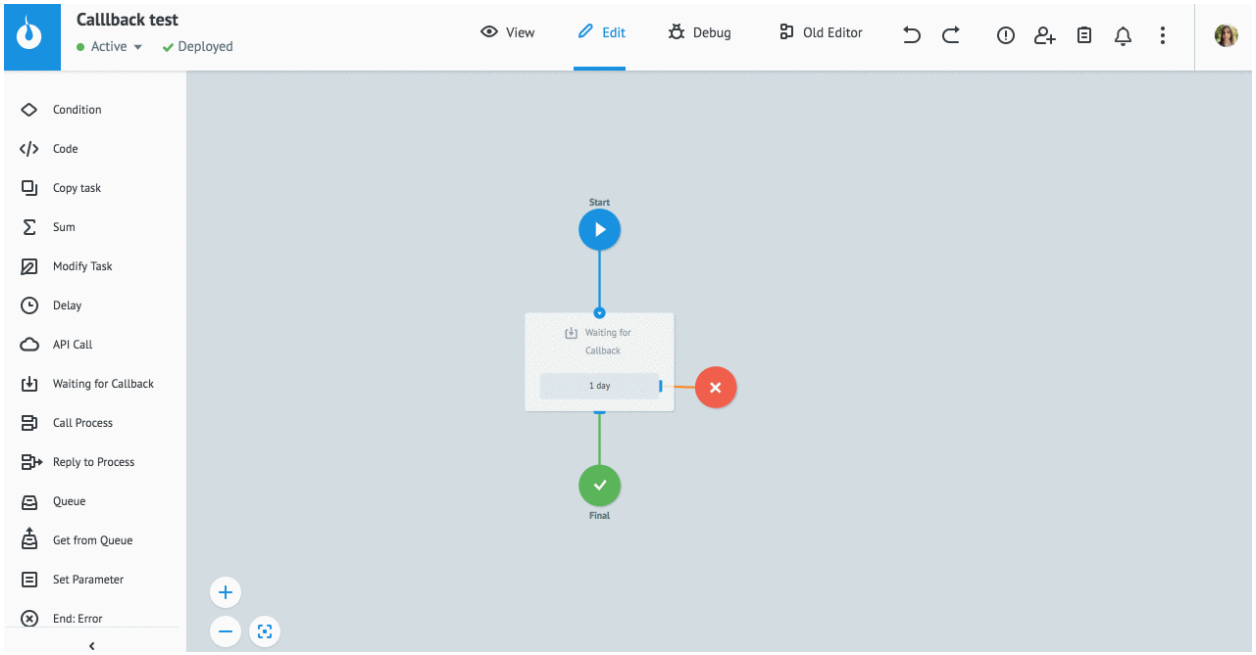
19. **Maximum interval measuring unit** [Required] [Action]: Selects the measuring unit for the maximum interval:

- Seconds
- Minutes
- Hours
- Days.

Examples

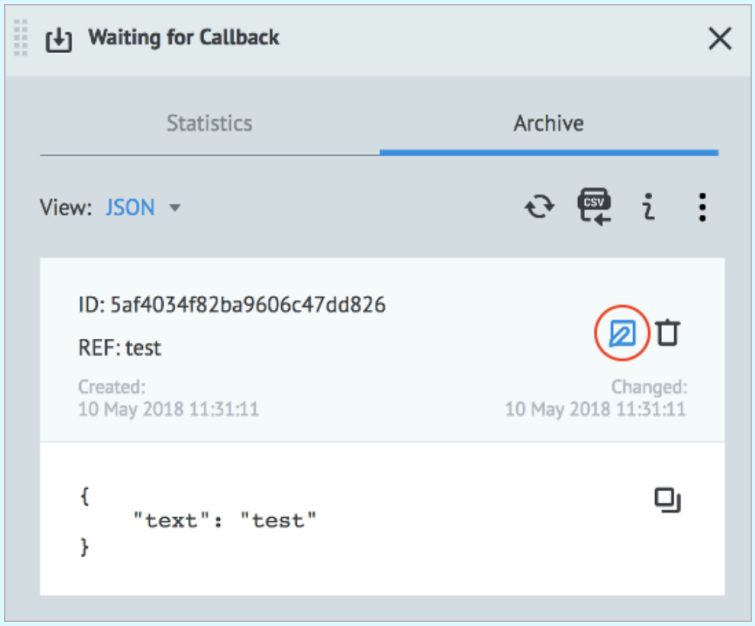
Manually modified task
In the example below, you can see how the task is modified manually.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 262: **Manually modified task in the Waiting for Callback node.**

The **Modify** button in the task window is available in the **Archive** tab that opens the **New Task** dialog.

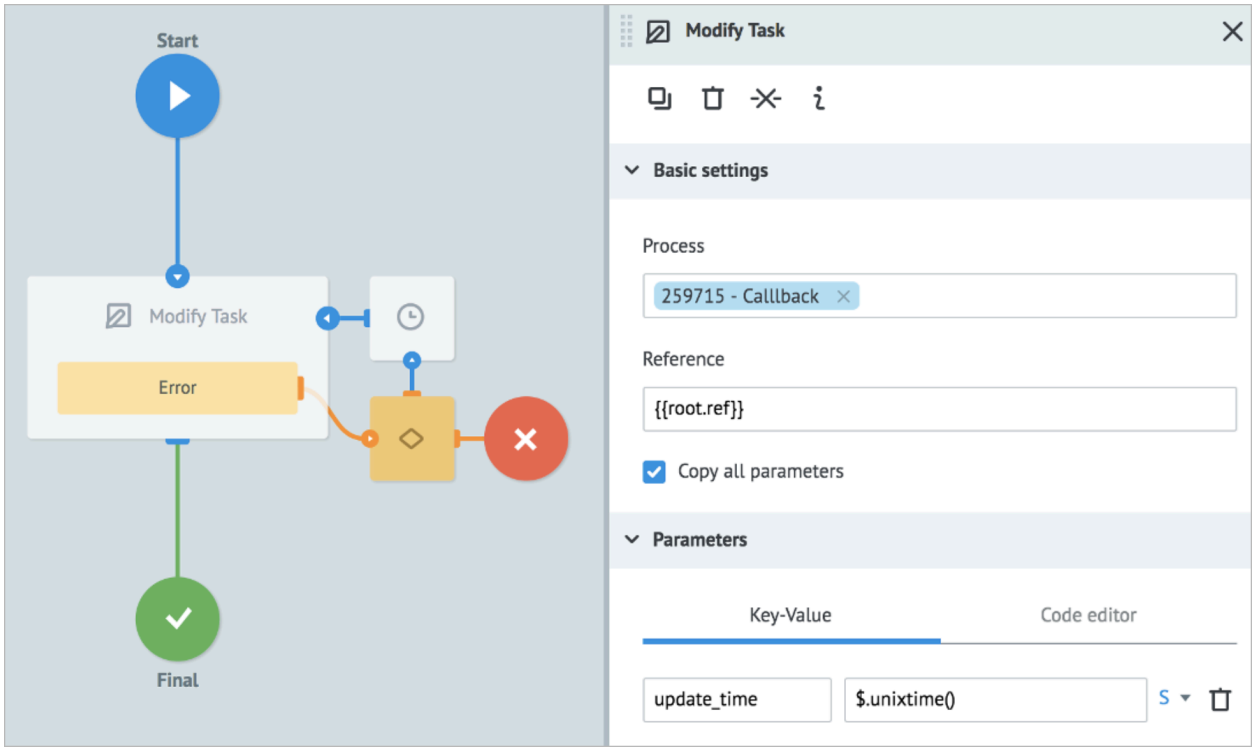


Screenshot 263: **A task in the Waiting for Callback node's archive.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Task modified in Modify Task node

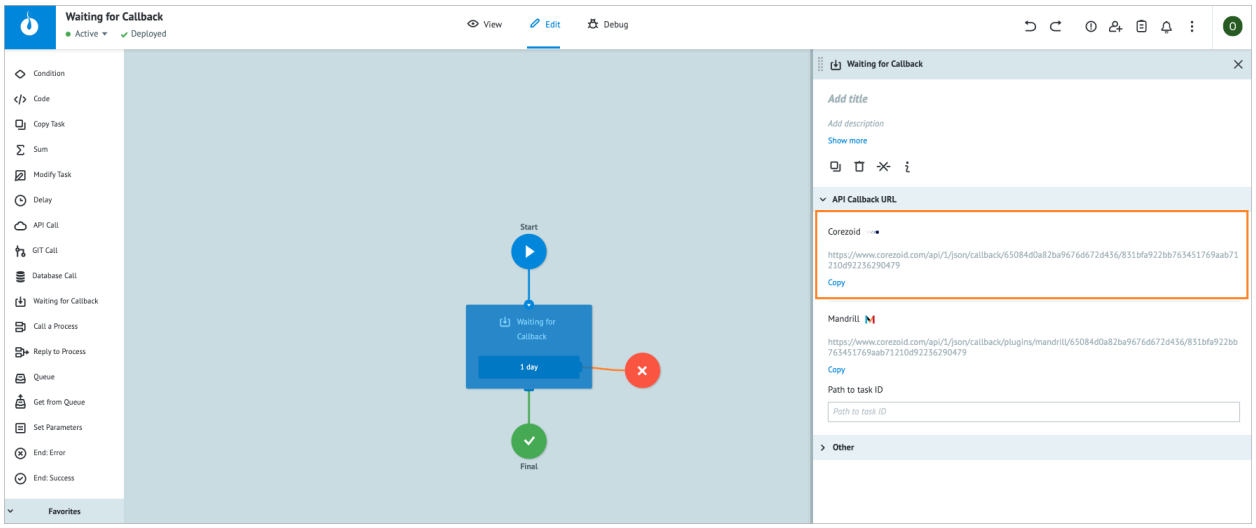
In the example below, you can see a task modified with the **Waiting for Callback** node by using the **Modify Task** node in another Process.



Screenshot 264: **A task modified in the Modify Task node.**

Task modified by Corezoid API

In the example below, a task was modified with Corezoid API by using the Waiting for Callback node.



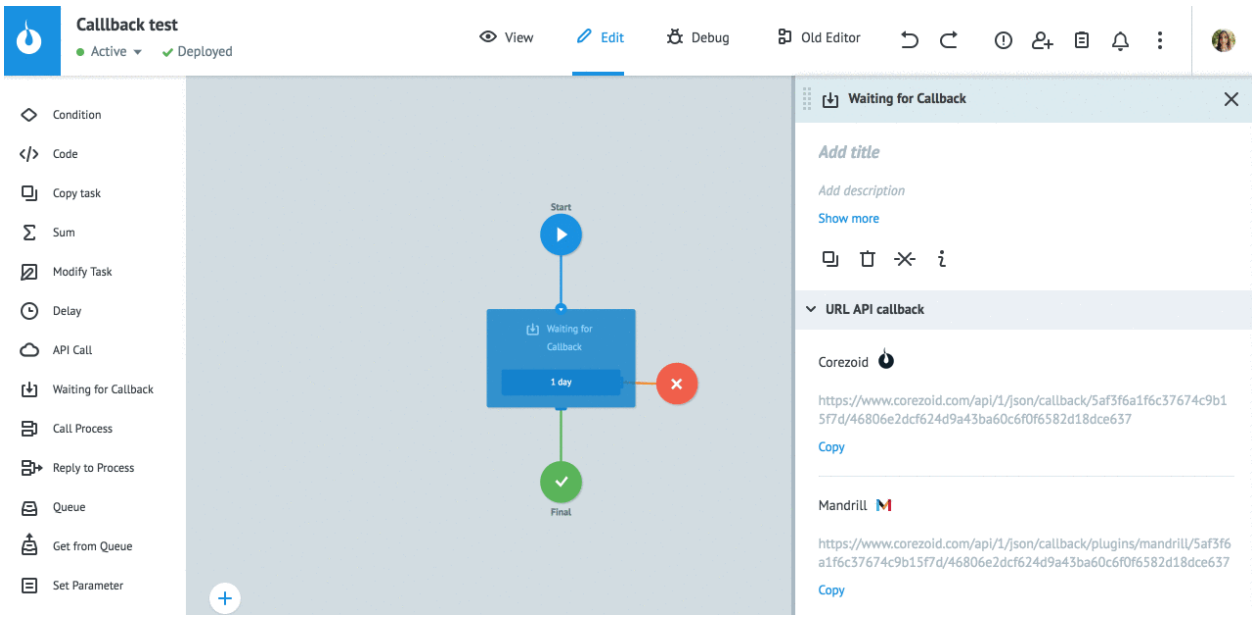
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Screenshot 265: **A task modified in the Corezoid API.**

You can use the following request body with the **task_id** parameter instead of ref for your requests.

```
{
"ops": [
    {
        "type": "modify",
        "obj": "task",
        "ref": "test_ref",
        "conv_id": "259715",
        "data": {
            "new_param": "test"
        }


}]}
```

Task modified by Callback URL
In the example below, you can see the task modified by submitting the POST request to the URL provided within the **Waiting for Callback** node.



Screenshot 266: **A task modified by the Callback URL.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Example request is sent in Postman.

Error handling & troubleshooting

When an error occurs during the task processing, you may see the following error parameter names in the task.

Error parameter name	Error type
`__conveyor_node_type_return_type_error__`	**Hardware** (system error), **Software** (error in a node logic/settings).
`__conveyor_node_type_return_type_tag__`	Error tag*.
`__conveyor_node_type_return_type_description__`	Error description in human-readable language; can be static or dynamic

Table 32: **Error parameter names.**

*The error `tag __conveyor_callback_return_type_tag__` may have the following values.

Value	Cause	Solution

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

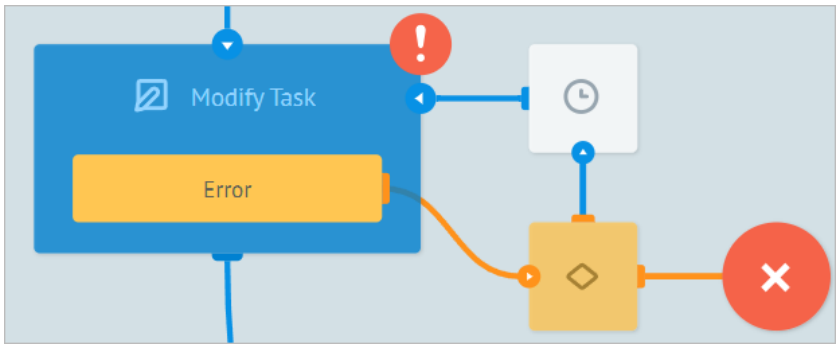
task_not_in_current_sync_callback_node	The task you are trying to modify from the **Modify Task** node is not inside the referenced **Waiting for Callback** node.	Make sure the task is in the specified node at the moment or clear the Synchronous checkbox in the **Waiting for Callback** node settings.

Table 33: **Waiting for Callback node errors.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3.17 Modify Task

Overview



Screenshot 267: **Modify Task node view.**

The **Modify Task** node updates parameters of tasks in other Processes without disrupting their workflow. Common use cases:

- Updating tasks in **Waiting for Callback** nodes to trigger their progression
- Adjusting task parameters in **Set State** nodes to influence their path through State Diagrams

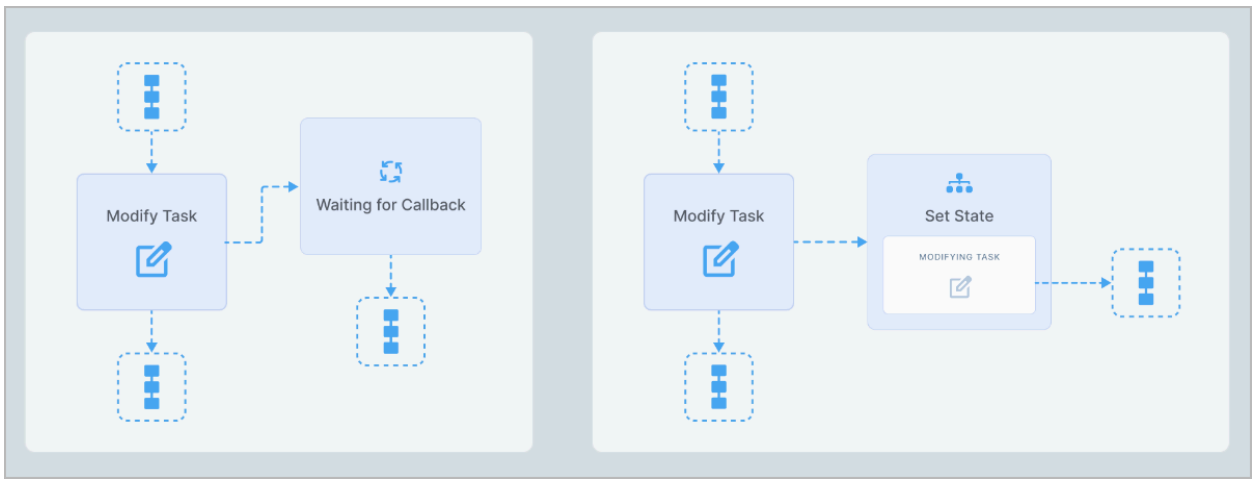


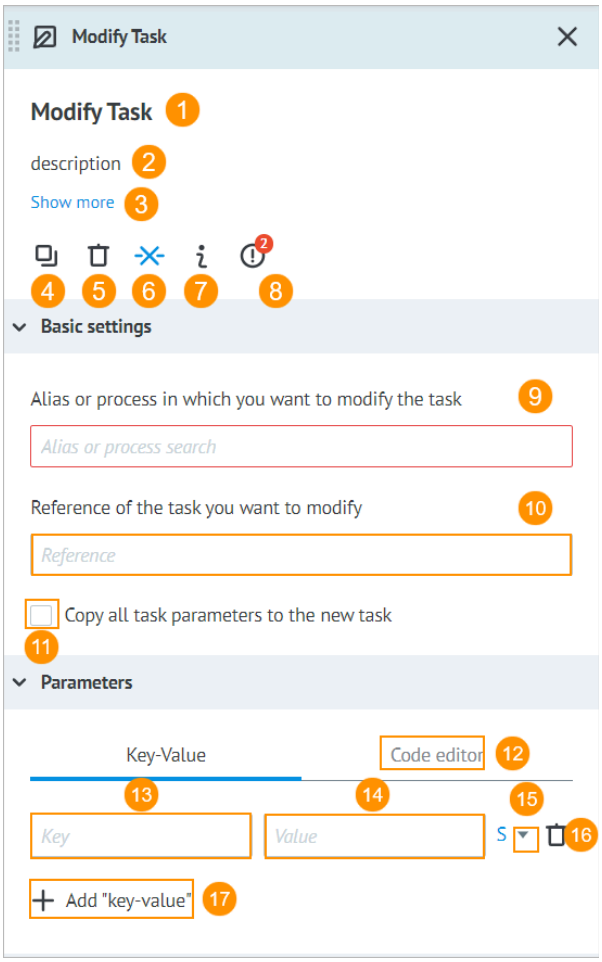
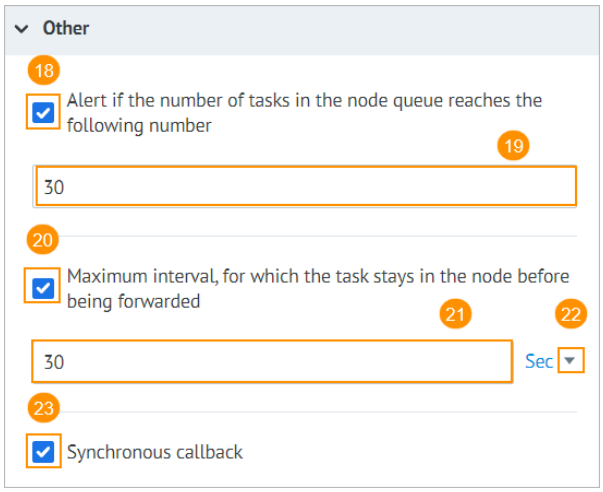
Diagram 23: **Modify Task node flow.**

You can only modify tasks that are in the **Waiting for Callback** or **Set State** nodes.

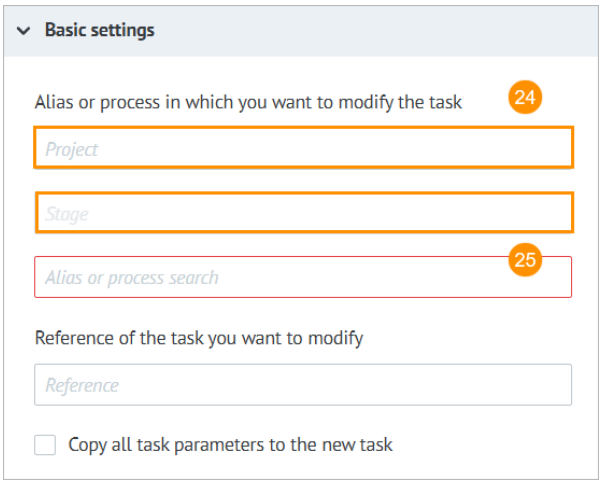
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Settings

The **Modify Task** node has the following parameters:

Basic settings	Other settings
	

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Basic settings for nodes inside Stages	
	

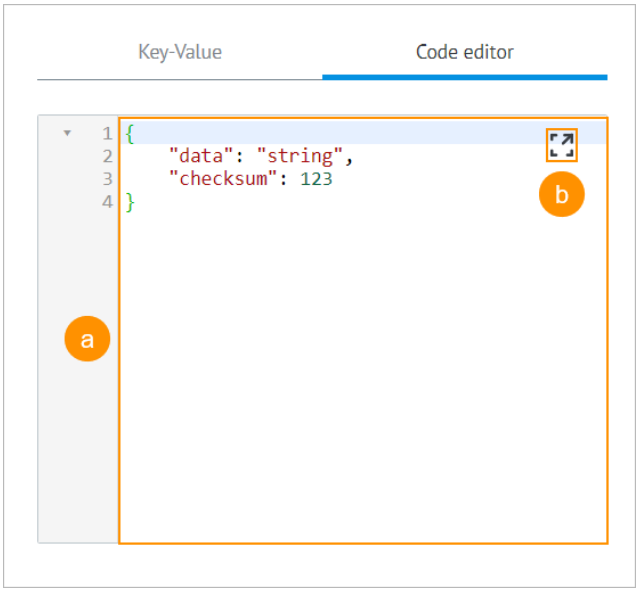
Screenshot 268: **Modify Task node settings.**

1. **Title** [Optional] [Field] [String]**:** Node title.
2. **Description** [Optional] [Field] [String]: Node description**.**
3. **Show more** [Optional] [Action]: Long descriptions are reduced to a compact view to simplify node settings. Click to see the whole description.
4. **Copy node** [Optional] [Action]: Creates a copy of the current node with all settings**.**
5. **Remove node** [Optional] [Action]: Deletes current node from Process.
6. **Remove links** [Optional] [Action]: Removes all connections to/from current node.
7. **Info** [Optional] [Action]: Displays node ID.
8. **Errors info** [Optional] [Action]: Displays node errors.
9. **Alias or Process in which you want to modify the task** [Required] [Field/Action] [String]: Select Process via directory icon or enter Process ID or Alias.
10. **Reference of the task you want to modify** [Option] [Field/Action] [String]: Specify identifier for task modification.
11. **Copy all task parameters to the new task** [Optional] [Action]:

- When selected: Sends all current task parameters plus node-specified parameters.
- When not selected: Sends only node-specified parameters.

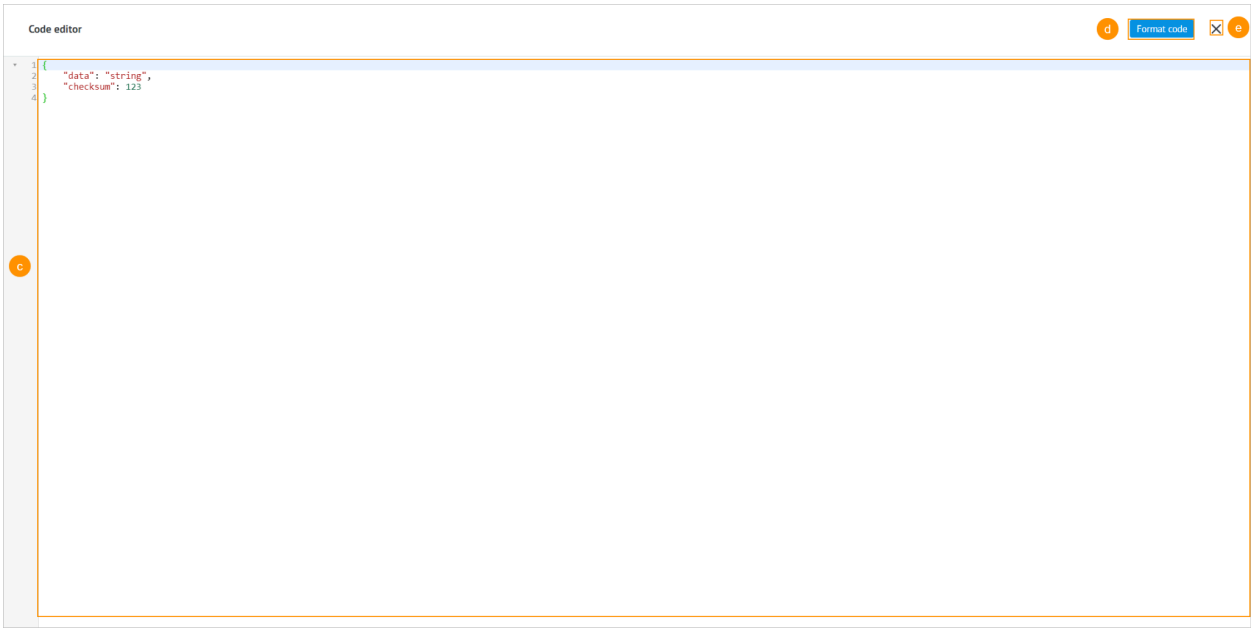
12. **Code editor** [Optional] [Action]: Shows the JSON format of the entered Key and Value. Click **Code editor** to open the code editor tab, where:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 269: **Code editor tab.**

a. **Code editor field** [Optional] [Field] [Any]: Enter new keys and values or edit the existing ones in JSON format.
b. **Full-screen mode** [Optional] [Action]: Opens the **Code editor** field in the full-screen mode, where:

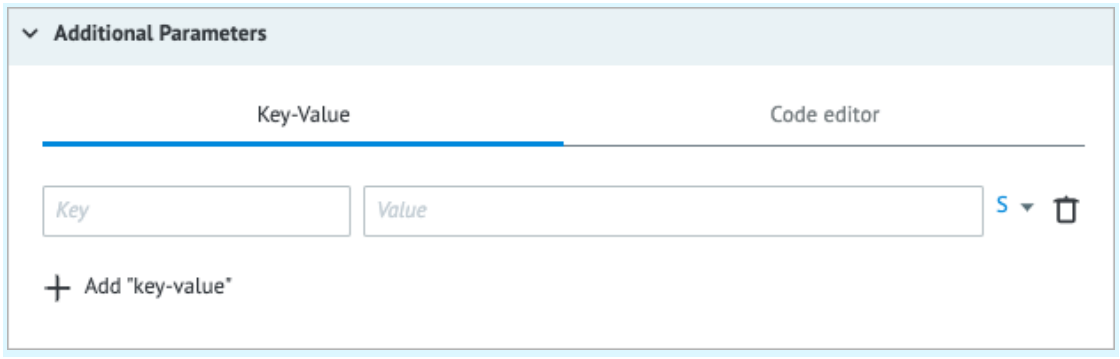


Screenshot 270: **Code editor full-screen view.**

c. **Code editor field** [Optional] [Field] [Any]: Edit or add keys in values.
d. **Format code** [Optional] [Action]: Format your current code.
e. **Quit the full-screen mode** [Optional] [Action]: Quit the full-screen mode.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

13. **Key** [Required] [Field] [String]: Key to specify a static name or a variable in {{}} curly brackets
    To add a new parameter, specify a unique name, to update an existing parameter, use its original name as a key.
14. **Value** [Required] [Field] [Any]: The value you want to assign to the corresponding Key parameter.
    See data types descrition for more details.
    - To add a key-value, click **+ Add "key-value"** below the **Key** field.
    - To delete the key-value line, click the trash icon on the right side of the key-value line.



Screenshot 271: **Add a key-value pair.**

15. **Type of data** [Required] [Action]: Selects the data type for the **Value**:
    * String (**S**)
    * Number (**N**)
    * Boolean (**B**)
    * Array (**A**)
    * Object (**O**)

    Note: For more information, go to Data types.

16. **Delete key-value** [Optional] [Action]: Delete the key-value pair.

17. **Add key-value** [Optional] [Action]: Add a new key-value pair.

18. **Alert if the number of tasks in the node queue reaches the following number** [Optional] [Action]: Helps monitor whether the number of tasks in the node exceeds the specified threshold.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

19. **Number of tasks in the queue** [Optional/Required if **Alert if the number of tasks in the node queue reaches the following number** is selected] [Field] [Number]: Number of tasks in the queue which triggers an alert when reached.

20. **Maximum interval, for which the task stays in the node before being forwarded** [Optional] [Action]: Select to limit the time a task is allowed to be in the node in seconds, minutes, hours, and days.

21. **Maximum interval** [Optional/Required if **Maximum interval, for which the task stays in the node before being forwarded**] [Field] [Number]: The amount of time a task is allowed to be in the node. It has a minimum value of 30 seconds. You can set a shorter interval by using the **Unixtime** function.

22. **Maximum interval measuring unit** [Required] [Action]: Selects the measuring unit for the maximum interval:

- Seconds
- Minutes
- Hours
- Days.

23. **Synchronous callback** [Optional] [Action]:

- The checkbox is selected: Updates tasks sequentially, checking whether it is being modified in another **Modify task** node.
- The checkbox is not selected: Only checks if task exists in the Process.

Note: Works only for tasks in **Waiting for Callback** or **Set State** nodes. Otherwise, returns `no_api_callback_in_node_with_task` error.

24. **Project** [Required] [Field/Action] [String]: Choose the Project containing the Process in which you would like to modify a task by clicking the directory icon or by specifying its Process ID, name or Alias.

Note: this parameter is available with the **Alias or process in which you want to modify the task** parameter and applies to **Modify Task** nodes created within Stages in Projects. This feature allows you to modify tasks in Processes located in Stages of different Projects.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

25. **Stage** [Required] [Field/Action] [String]: Choose the Stage containing the Process in which you would like to modify a task by clicking the directory icon or by specifying its Process ID, name or Alias.

Note: this parameter is available with the **Alias or process in which you want to modify the task** parameter and applies to **Modify Task** nodes created within Stages in Projects. This feature allows you to modify tasks in Processes located in Stages of different Projects.

Examples

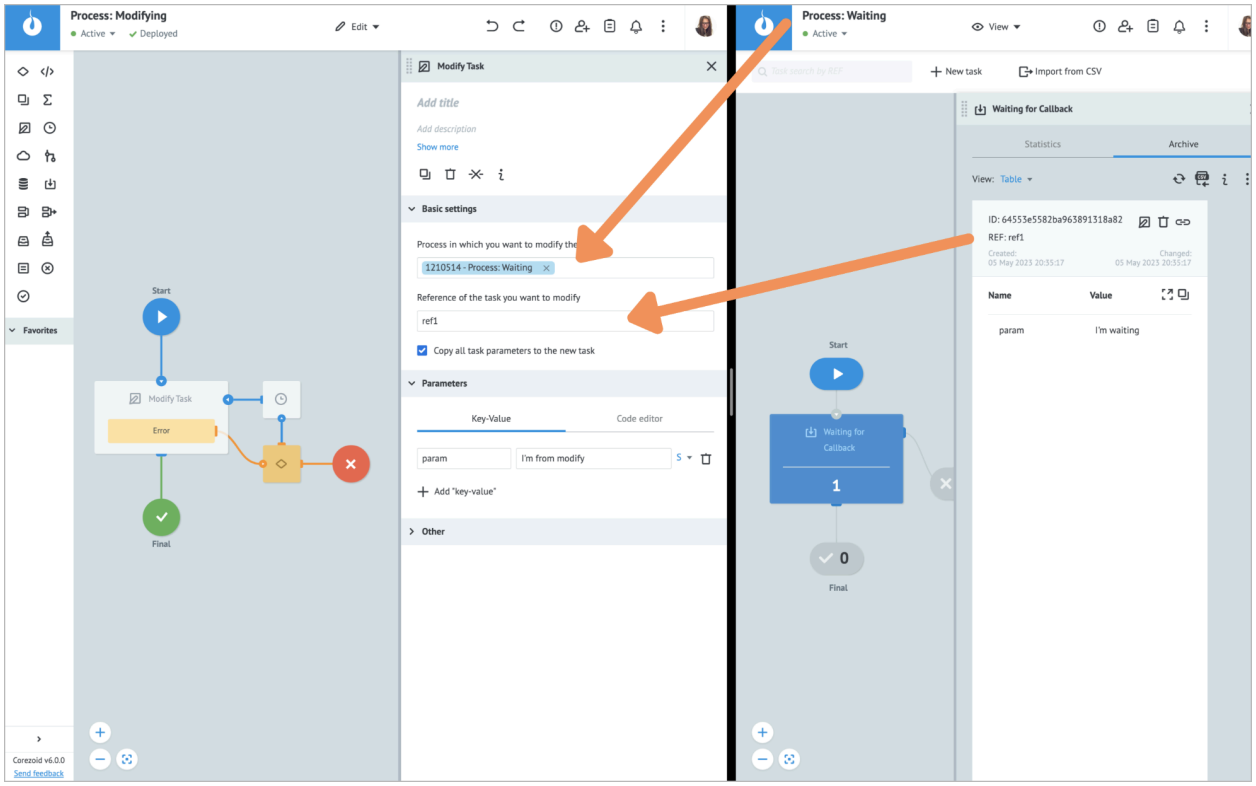
Modifying a task in the Waiting for Callback node

Description: In the example below, you can see how the **I'm waiting** task parameter is changed by **I'm from modify**.

Precondition:

1. Create a Process or a State Diagram, and add the **Waiting for Callback** node.
2. On the **Waiting for Callback** node details panel, add the **param** parameter, the **I'm waiting** value, and the **ref1** reference.
3. Create a Process and add the **Modify Task** node.
4. In **Basic settings** of the **Modify Task** node details panel, add the **Waiting for Callback** node, the **ref1** reference, the **param** parameter, and the **I'm from modify** value.

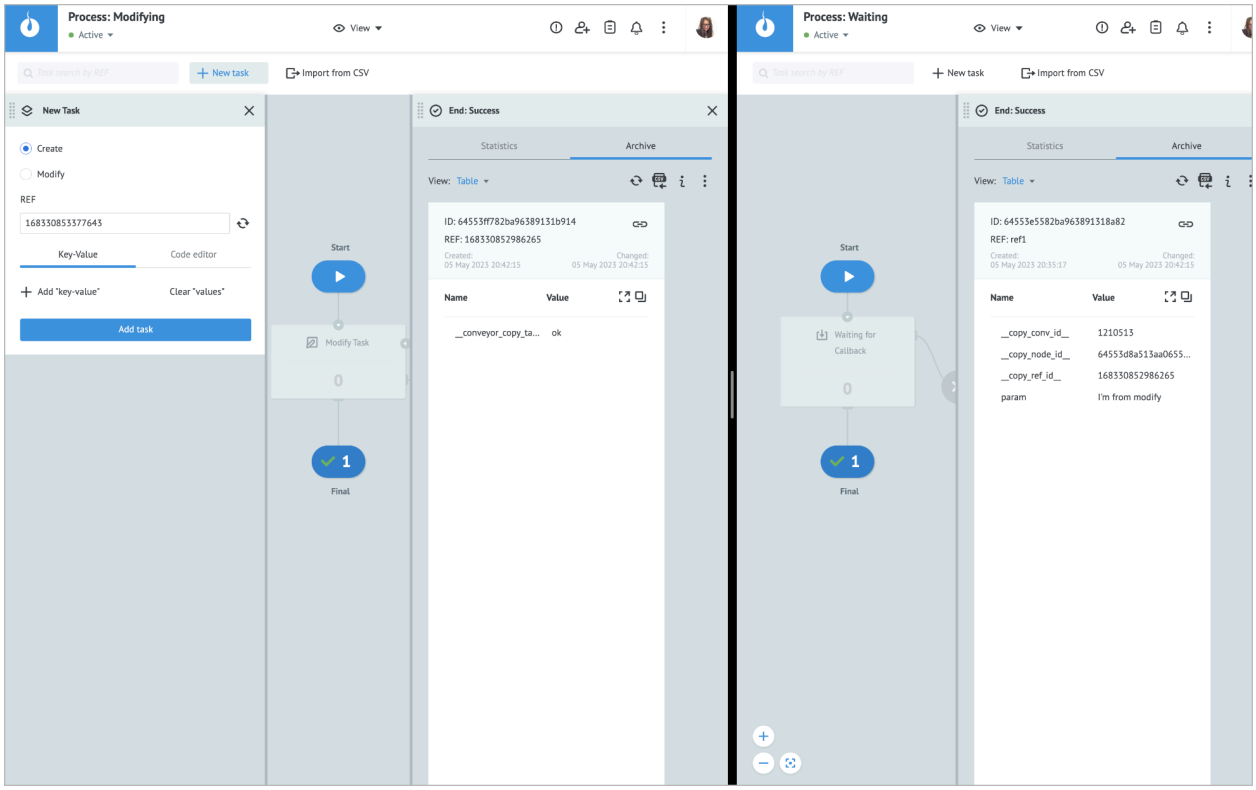
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 272: **Modify Task and Waiting for Callback nodes combined use.**

Flow: The task passes through the **Modify Task** node and gets a new parameter indicating the status of the modification. In this example, it is the ok value. When the modified task returns to the **Waiting for Callback** node, its parameter changes from **I'm waiting** to **I'm from modify**, reflecting the changes specified in the **Modify Task** node. System parameters are added to the task, so you can track where this modification came from.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



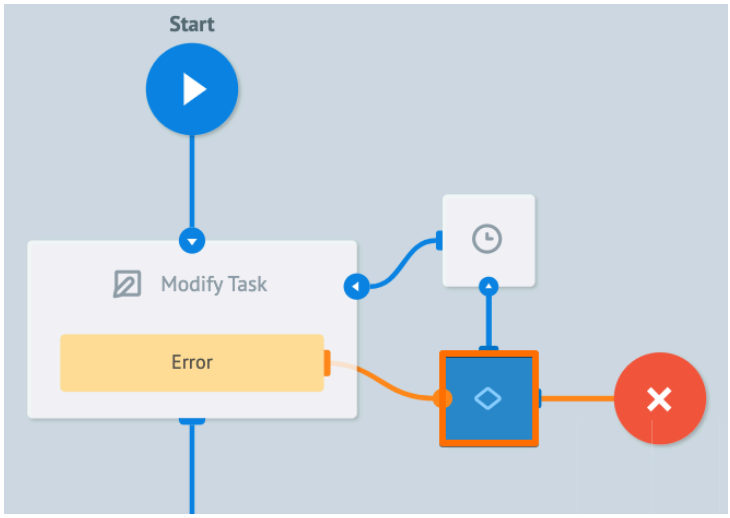
Screenshot 273: **Task parameters in Modify Task and Waiting for Callback nodes.**

You can access a single task parameter or a whole task stored in a State Diagram by Getting Task Parameters from State Diagrams.

Error handling & troubleshooting

When an error occurs in the **Modify Task** node, a task goes to the auxiliary Condition output node that is used for storing error parameters.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 274: **Modify Task auxiliary debug node.**

When an error occurs during the task processing, you may see the following error parameter names in the task.

Error parameter name	Error type
`__conveyor_modify_task_return_type_error__`	**Hardware** (system error), **Software** (error in a node logic/settings).
`__conveyor_modify_task_return_type_tag__`	*Error tag.
`__conveyor_modify_task_return_type_description__`	Error description in human-readable language; can be static or dynamic.

Table 34: **Error parameter names.**

*The error `tag __conveyor_modify_task_return_type_tag__` may have the following values.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Value	Cause	Solution
access_denied	You don't have access to the Process you are trying to call.	Contact the Process owner to give you access.
not_found_task	No task with the specified reference exists in the Process.	Make sure you entered the task reference and Process correctly. **Note:** An empty reference is not considered unique: if only a single task with an empty reference exists, you cannot access it by reference.
Not_found_task **Note:** The task is present in the Process.	The task might have existed the **Set State** or **Waiting for Callback** nodes just before it was supposed to be modified.	Try to modify the task again or select the **Synchronous callback** checkbox.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

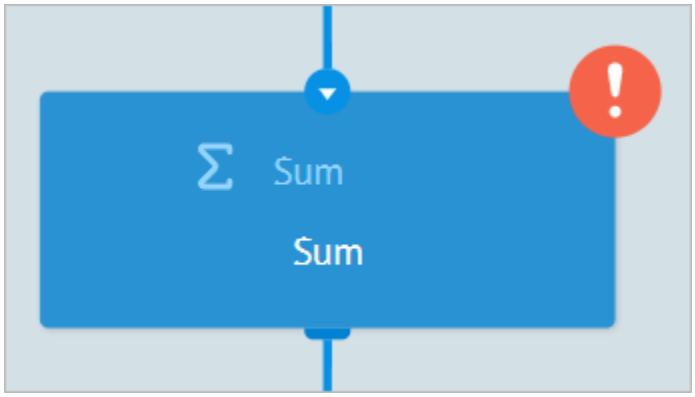
no_api_callback_in_node_with_task	The task you are trying to modify is not in the **Set State** or **Waiting for Callback** node.	Make sure the task is in the correct type of node.
duplicate_callback	The **Synchronous callback** checkbox is selected, and the task is being modified.	Corezoid will reattempt to send the request after 30 seconds.
modify_task_size_overflow_limit	The size of the task data exceeds the specified limit set for your environment.	Reduce the size of your task.

Table 35: **Modify Task node errors.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3.18 Sum

Overview



Screenshot 275: **Sum node view.**

The **Sum** node:

- Calculates and accumulates totals from specified task parameters
- Tracks parameter statistics
- Integrates with other nodes for workflow analytics

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

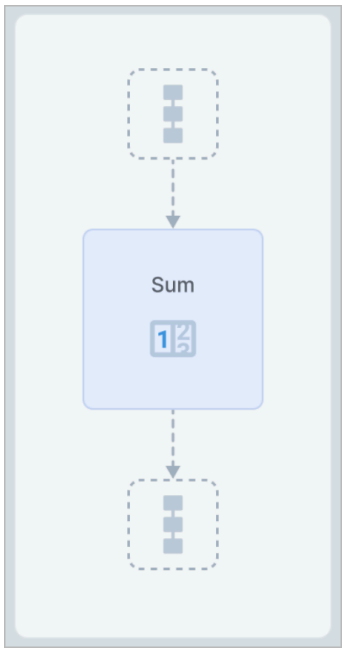


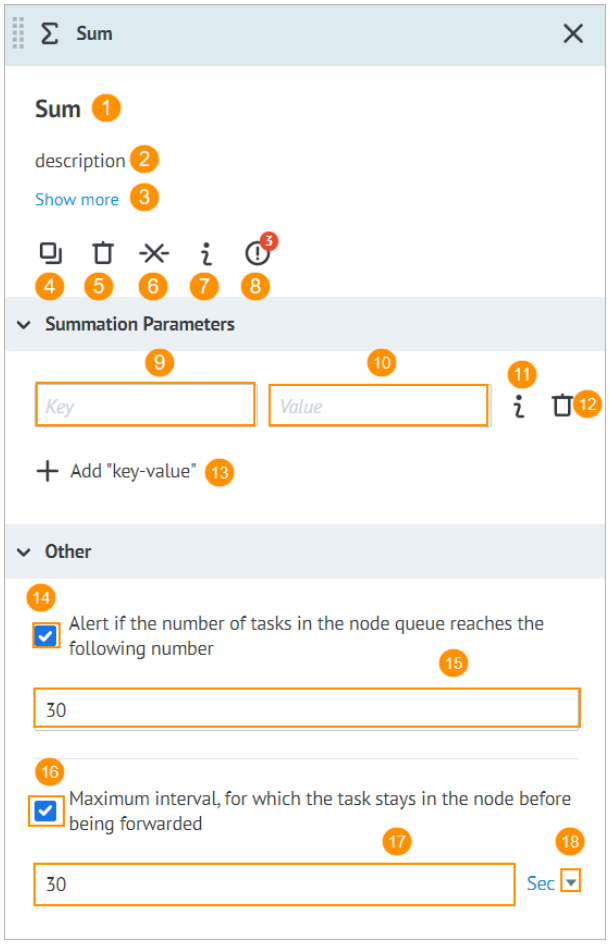
Diagram 24: **Sum node flow.**

Unlike other nodes, in the **Sum** node, the total value is not added to the tasks but is stored within the node itself.

Node Settings

The **Sum** node has the following parameters:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



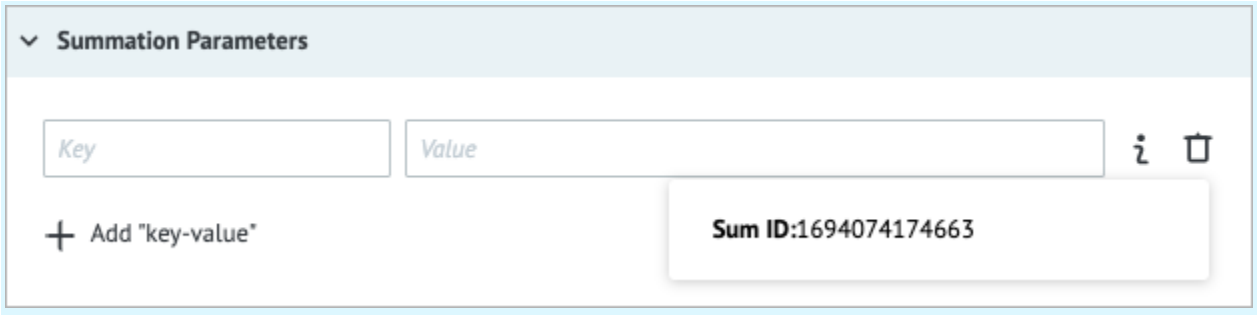
Screenshot 276: **Sum node settings.**

1. **Title** [Optional] [Field] [String]**:** Node title.
2. **Description** [Optional] [Field] [String]: Node description**.**
3. **Show more** [Optional] [Action]: Long descriptions are reduced to a compact view to simplify node settings. Click to see the whole description.
4. **Copy node** [Optional] [Action]: Creates a copy of the current node with all settings**.**
5. **Remove node** [Optional] [Action]: Deletes current node from Process.
6. **Remove links** [Optional] [Action]: Removes all connections to/from current node.
7. **Info** [Optional] [Action]: Displays node ID.
8. **Errors info** [Optional] [Action]: Displays node errors.
9. **Key** [Required] [Field] [String]: Key to specify a static name or a variable in {{}} curly brackets
   To add a new parameter, specify a unique name, to update an existing parameter, use its original name as a key.
10. **Value** [Required] [Field] [Any]: The value you want to assign to the corresponding Key parameter.
    See data types description for more details.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Note: For more information, go to Parameters, Functions.

- To add a key-value, click **+ Add "key-value"** below the **Key** field.
- To delete the key-value line, click the trash icon on the right side of the **Value** field.



Screenshot 277: **A key-value pair add.**

11. **Sum ID** [Optional] [Action]: Click to view the Sum ID.
12. **Delete key-value** [Optional] [Action]: Delete the key-value pair.
13. **Add key-value** [Optional] [Action]: Add a new key-value pair.
14. **Alert if the number of tasks in the node queue reaches the following number** [Optional] [Action]: Helps monitor whether the number of tasks in the node exceeds the specified threshold.
15. **Number of tasks in the queue** [Optional/Required if **Alert if the number of tasks in the node queue reaches the following number** is selected] [Field] [Number]: Number of tasks in the queue which triggers an alert when reached.
16. **Maximum interval, for which the task stays in the node before being forwarded** [Optional] [Action]: Select to limit the time a task is allowed to be in the node in seconds, minutes, hours, and days.
17. **Maximum interval** [Optional/Required if **Maximum interval, for which the task stays in the node before being forwarded**] [Field] [Number]: The amount of time a task is allowed to be in the node. It has a minimum value of 30 seconds. You can set a shorter interval by using the **Unixtime** function.
18. **Maximum interval measuring unit** [Required] [Action]: Selects the measuring unit for the maximum interval:
    - Seconds
    - Minutes
    - Hours
    - Days.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Examples

Using the Sum node as a counter

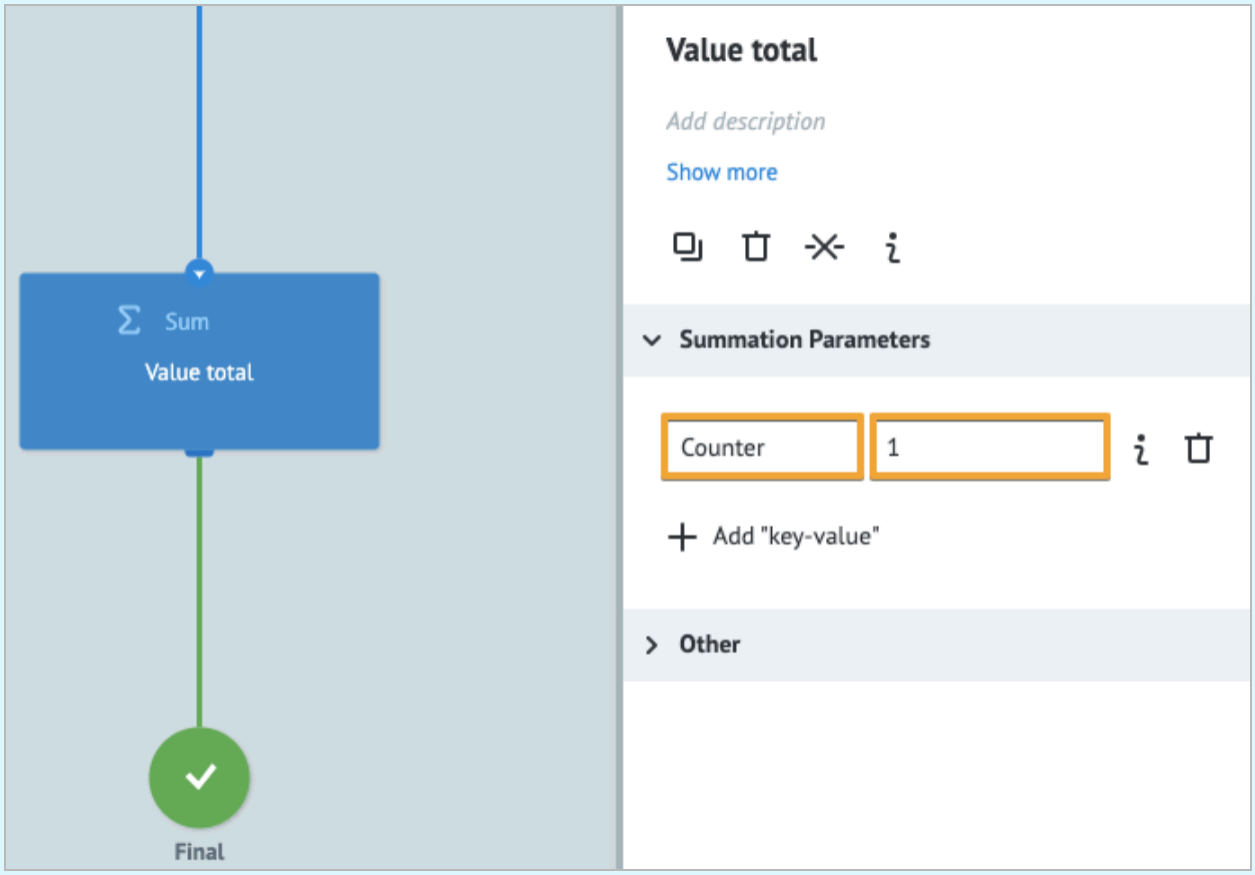
Description: In the example below, you can see how to use the **Sum** node as a counter.

Precondition:

1. Add the **Sum** node to your Process.
2. In **Summation Parameters** of the **Sum** node details panel, add the **Count** parameter and the **1** value (a positive or negative number).

Flow: Each time a task passes through the Sum node, the Counter value increases by 1, effectively counting the number of tasks that have passed through the node.

Such a counter can be used to monitor the number of payments or charges on an account or to record different types of payments/multiple accounts.



Screenshot 278: **Sum node counter.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

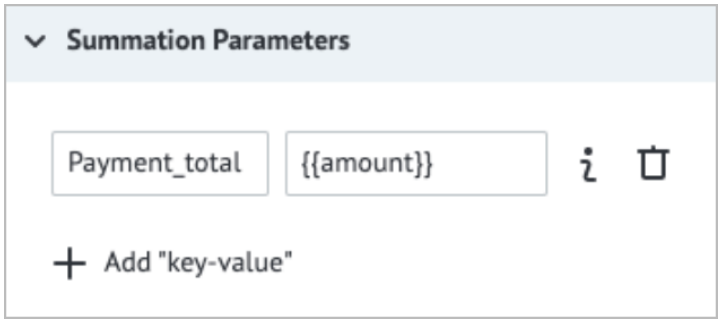
You can view the current **Counter** value on a Dashboard or access its value in a Process or a State Diagram.

Summing up a parameter

Description: In the example below, you can see how to use the **Sum** node to calculate a parameter value.

Precondition:

1. Add the **Sum** node to your Process or State Diagram.
2. In **Summation Parameters** of the **Sum** node details panel, add the **Payment_total** parameter and the **{{amount}}** value (as a variable).



Screenshot 279: **Summation parameters.**

Flow: Each time a task passes through the **Sum** node, the **Sum** node tracks and accumulates the values of the **{{amount}}** task parameter, calculates the total, and stores it in the **Payment_total** parameter.

You can sum multiple task parameters within the **Sum** parameter by using the **$.math({{param1}}+{{param2}})** math function.

Error handling & troubleshooting

When an error occurs during the task processing, you may see the following error parameter names in the task.

Error parameter name	Error type

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

`__conveyor_api_sum_return_type_error__`	**Hardware** (system error), **Software** (error in a node logic/settings).
`__conveyor_api_sum_return_type_tag__`	*Error tag.
`__conveyor_api_sum_return_type_description__`	Error description in human-readable language; can be static or dynamic.

Table 36: **Error parameter names.**

*The error tag `__conveyor_api_sum_return_type_tag__` may have the following values.

Value	Cause	Solution
access_denied	You don't have access to the Process.	Contact the Process owner to give you access.
access_denied	You are trying to send or receive tasks from another stage.	It's not allowed to get and send tasks between stages. Use the same stage to send and receive tasks.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

access_refused	The Process you are working in is not a State Diagram but has to be it.	Check the logic of using a State Diagram. For more information, go to State Diagram.
api_sum_wrong_convert_param	Parameter value failed to convert with the specified parameter type.	Check and correct the data type of the parameter mentioned in the error message.
conv_not_found_or_deleted	You are trying to work with a non-existent Process.	Use an existing Process.
not_found_conv_by_alias	Your alias refers to a Process that no longer exists.	Use an existing Process.
bad_callback_node	The Process you are calling doesn't have the **Callback** node.	Create the **Callback** node in the Process you are calling.
api_sum_fatal_error	A hardware error has occurred.	Contact the support team for further assistance.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Table 37: **Sum node errors.**

When working with your Process, you may encounter the following issues.

Issue	Cause	Solution
The summed parameter is not a number.	A parameter in the **Sum** node is not of the **Number** type.	Change the parameter type to **Number**.

Table 38: **Sum node issues.**

3.19 Parallel Processing Node (Coming Soon)

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

4. Deployment Options

Virtualization Platforms notes

- Corezoid is officially supported only on VMware (ESXi 6.5 or later) because VMware offers global, production-ready solutions optimized for marketplace deployments.
- Other virtualization platforms (e.g., Hyper-V, KVM) are not officially supported due to limited use by enterprise clients and suboptimal production performance. However, converting Corezoid images for other virtualization platforms is possible.
- Only RHEL 7 and CentOS 7 are officially supported for marketplace solutions, ensuring consistent performance and stability. Corezoid can run on other Linux distributions, such as CentOS 8, CentOS 9, Amazon Linux 2, and Amazon Linux 2023, but these are available only under specific agreements for production environments.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

4.1 VMware

Requirements

VMware	ESXi 6.5 and later
OS	RHEL7/CentOS7
CPU	6
RAM	10
SSD	500GB
Note: VM version 13. If your version is lower, you need to additionally convert the image.	

Table 39: **VMware requirements.**

Deployment options

We offer two options of Corezoid Actor Engine deployment on VMware:

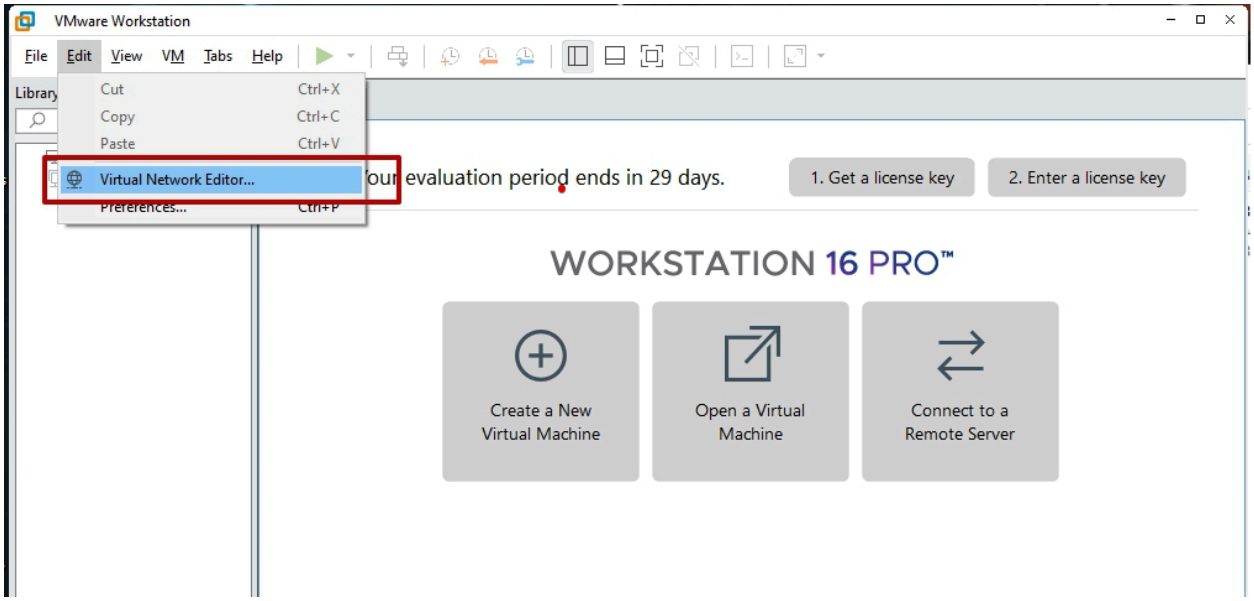
1. Deploying via OVF + VMDK
2. Attaching a disk image to a new virtual machine

Installation

Deploying via OVF + VMDK

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

1. Create a network: Click the **Edit** tab and in the dropdown list, select **Virtual Network Editor**.

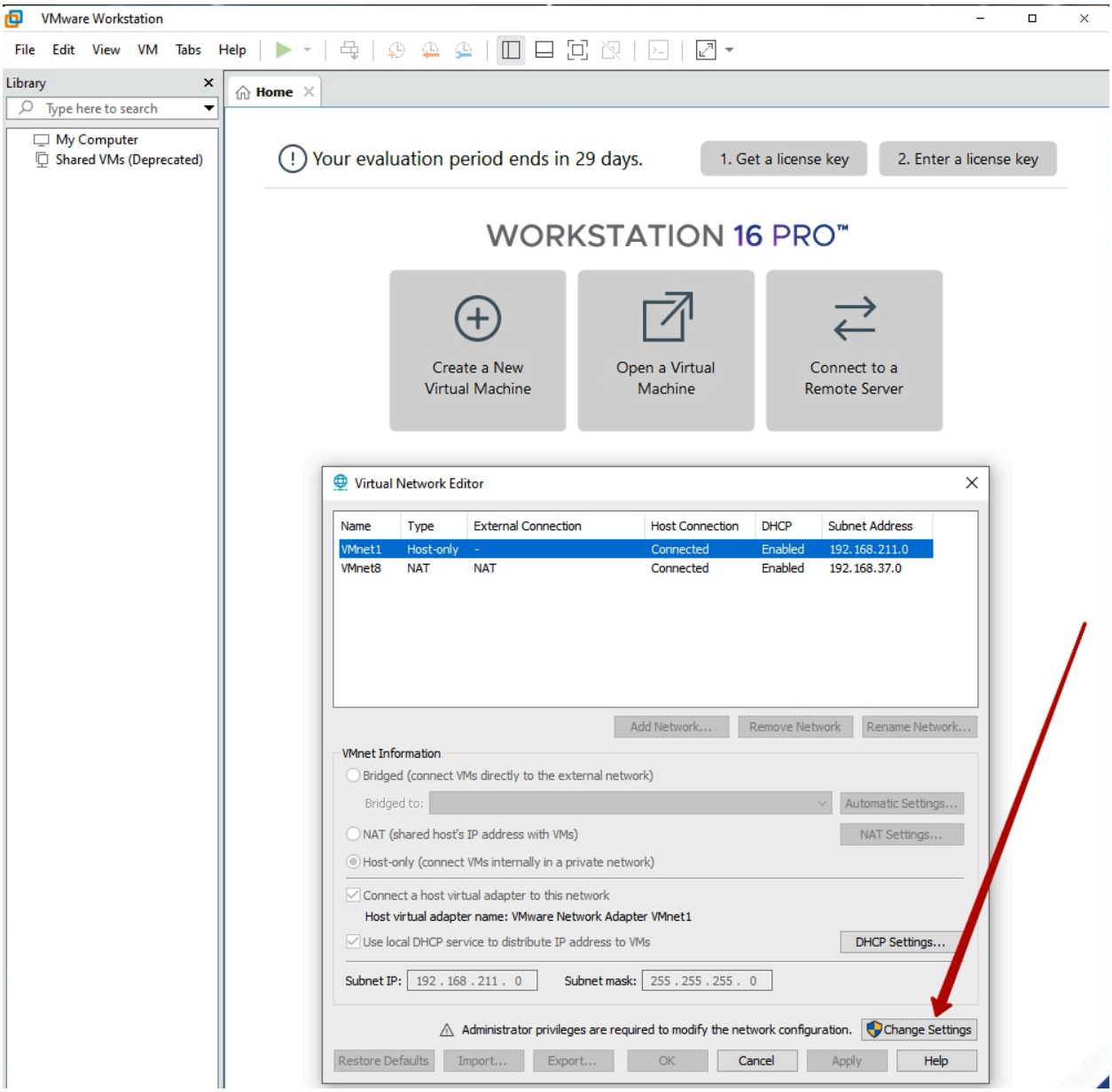


Screenshot 280: **VMware deployment.**

2. In the upper-right corner of the **Virtual Network Editor** dialog, click **Change Settings**.

   Note: You need to have the admin permission to change the configuration.

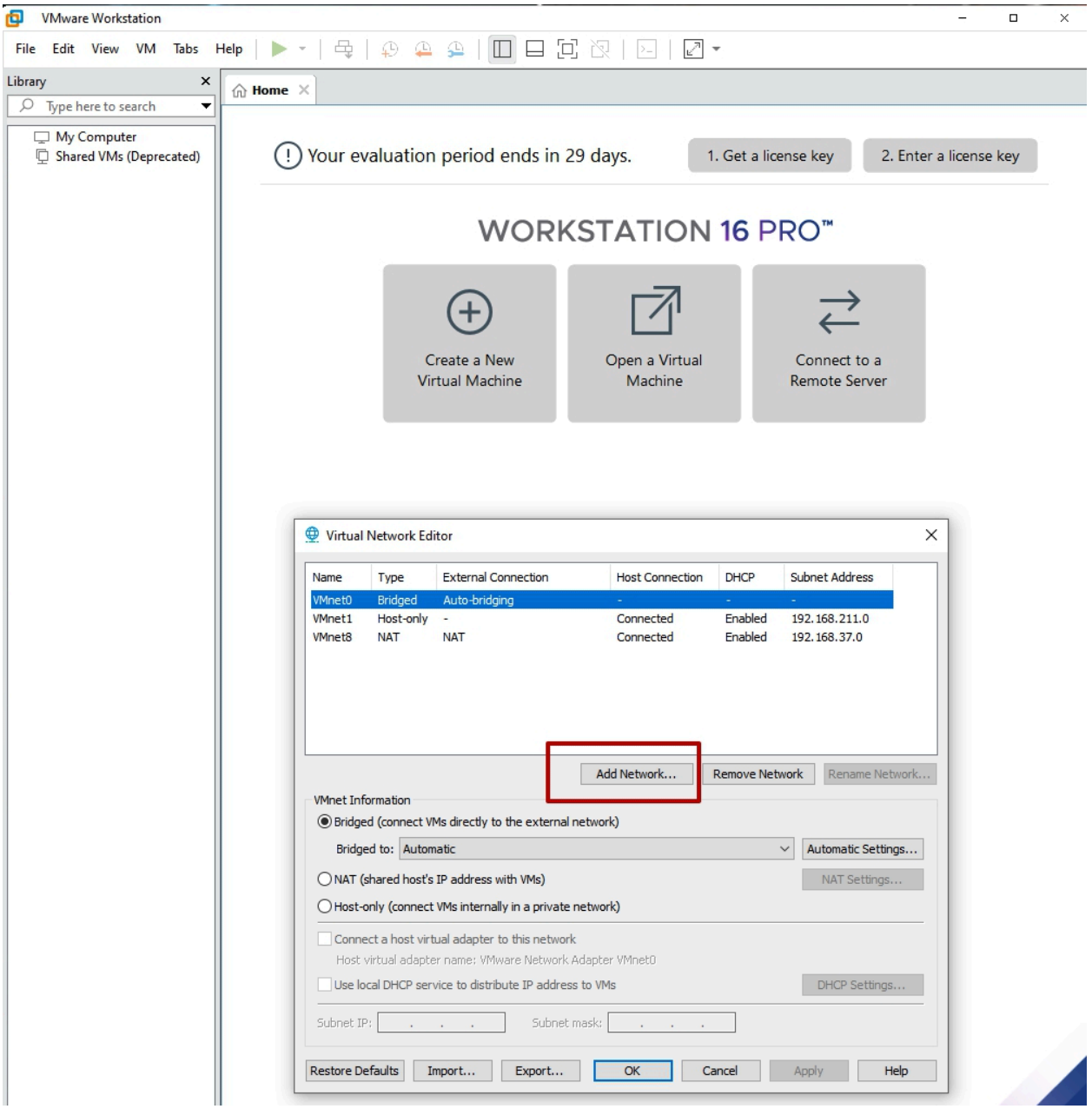
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 281: **VMware deployment.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

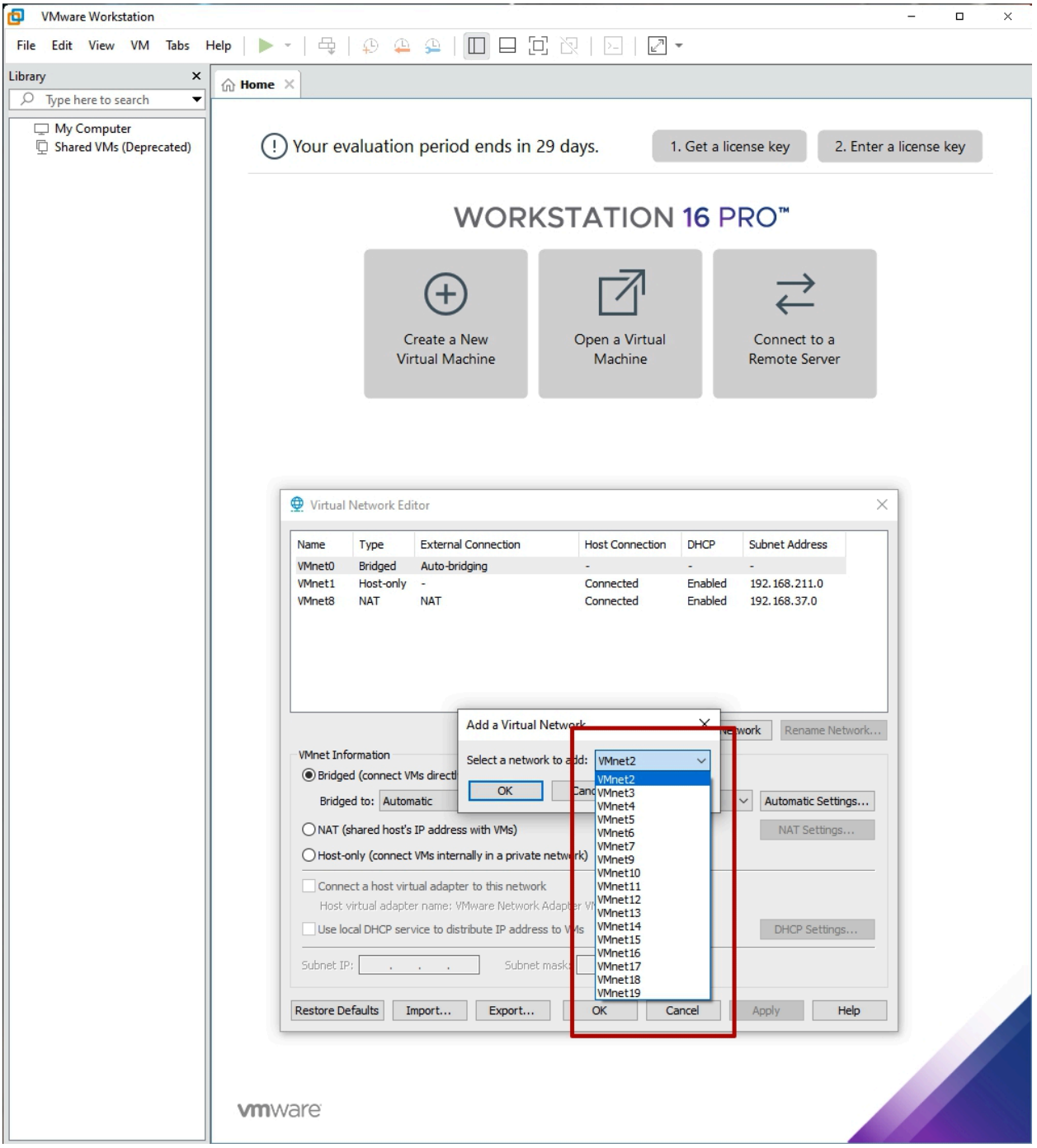
3. Click **Add Network**.



Screenshot 282: **VMware deployment.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

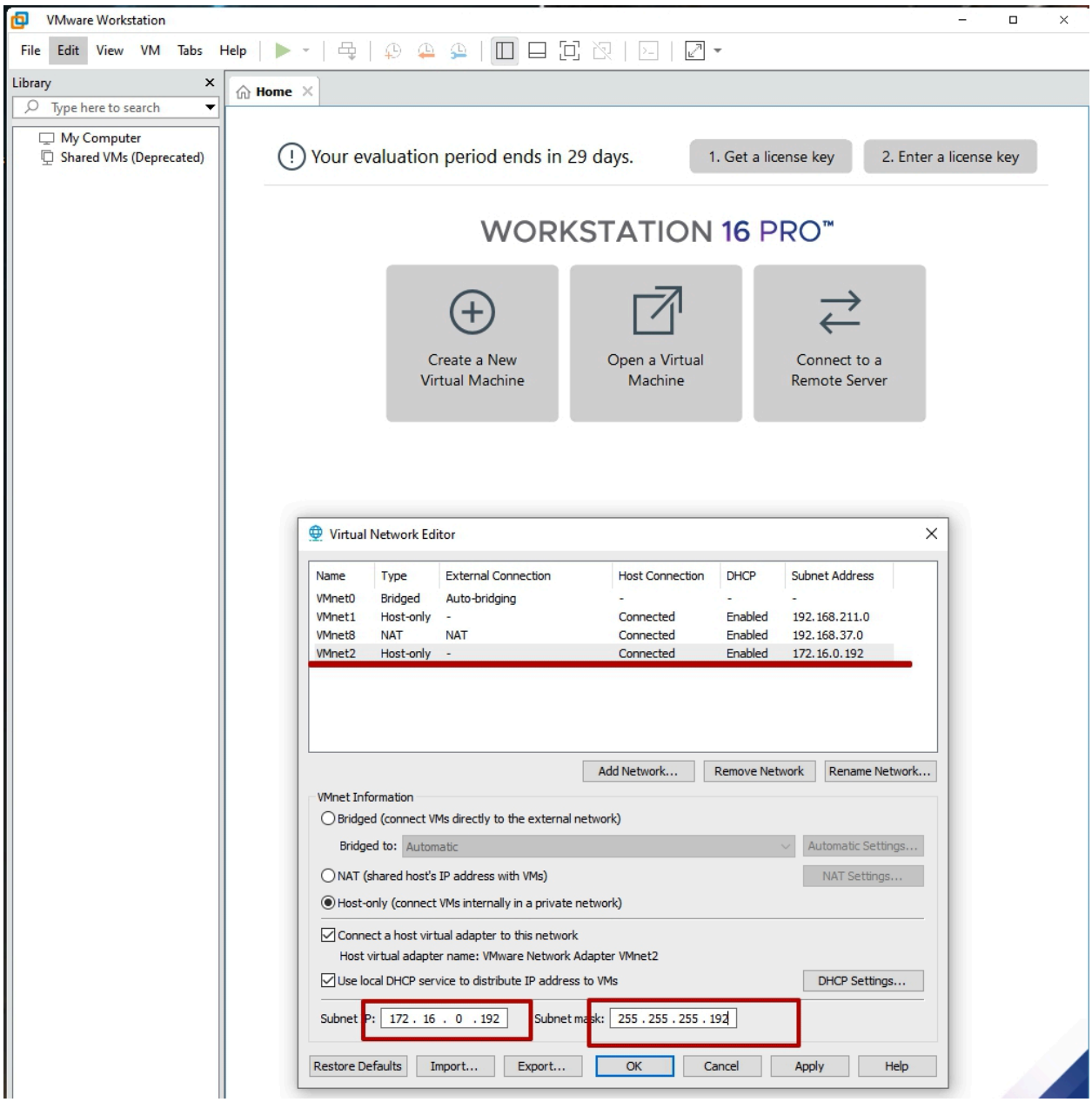
4. Select the network to add from the dropdown list.



Screenshot 283: **VMware deployment.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

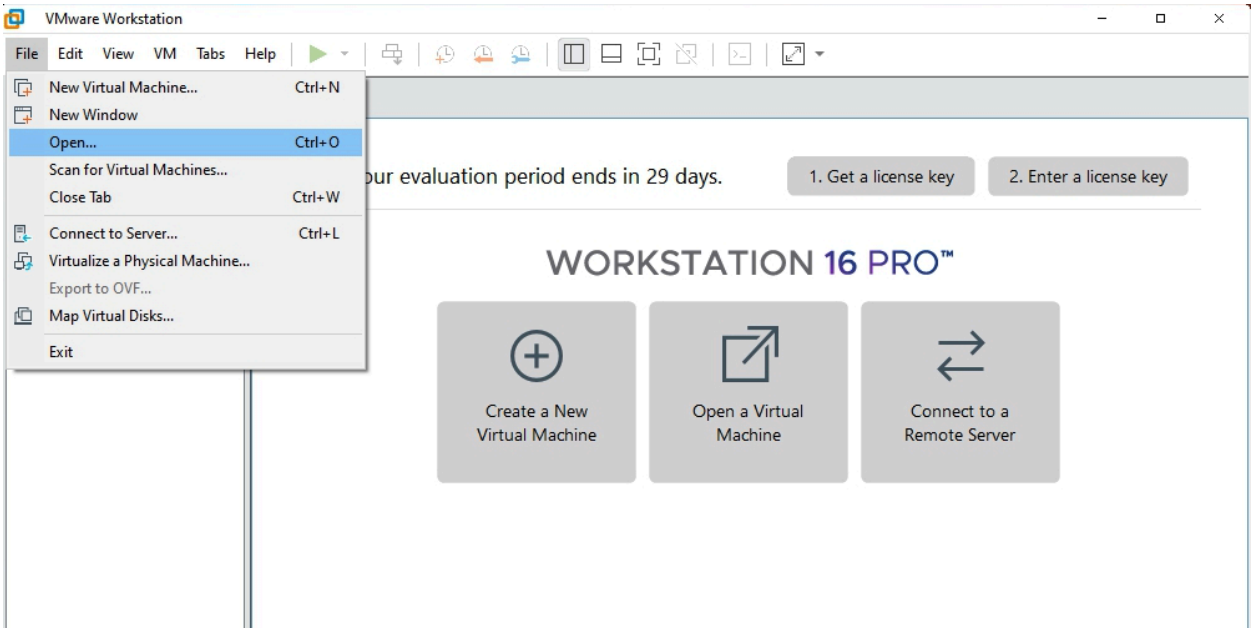
5. Specify the subnet IP and mask, and click **Apply**.



Screenshot 284: **VMware deployment.**

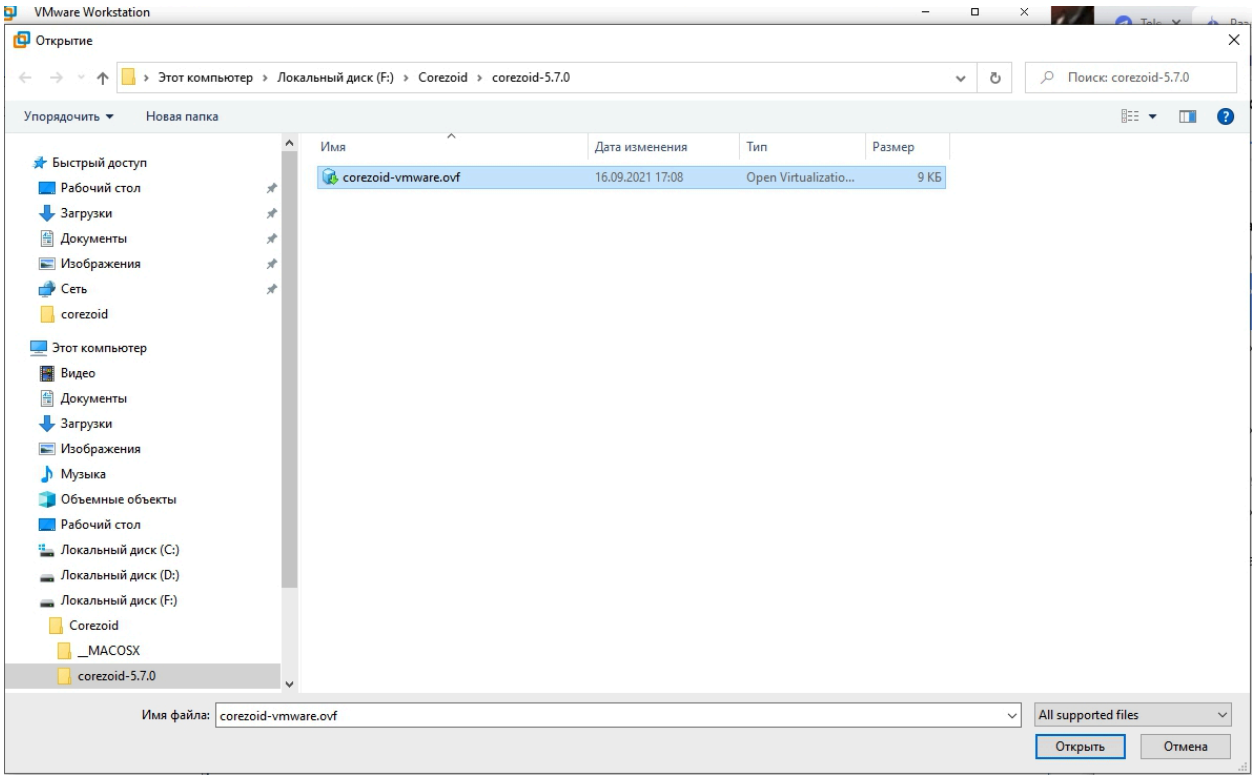
6. Upload the previously downloaded **ovf-image** to the virtual machine: click the **File** tab and in the dropdown list, select **Open**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 285: **VMware deployment.**

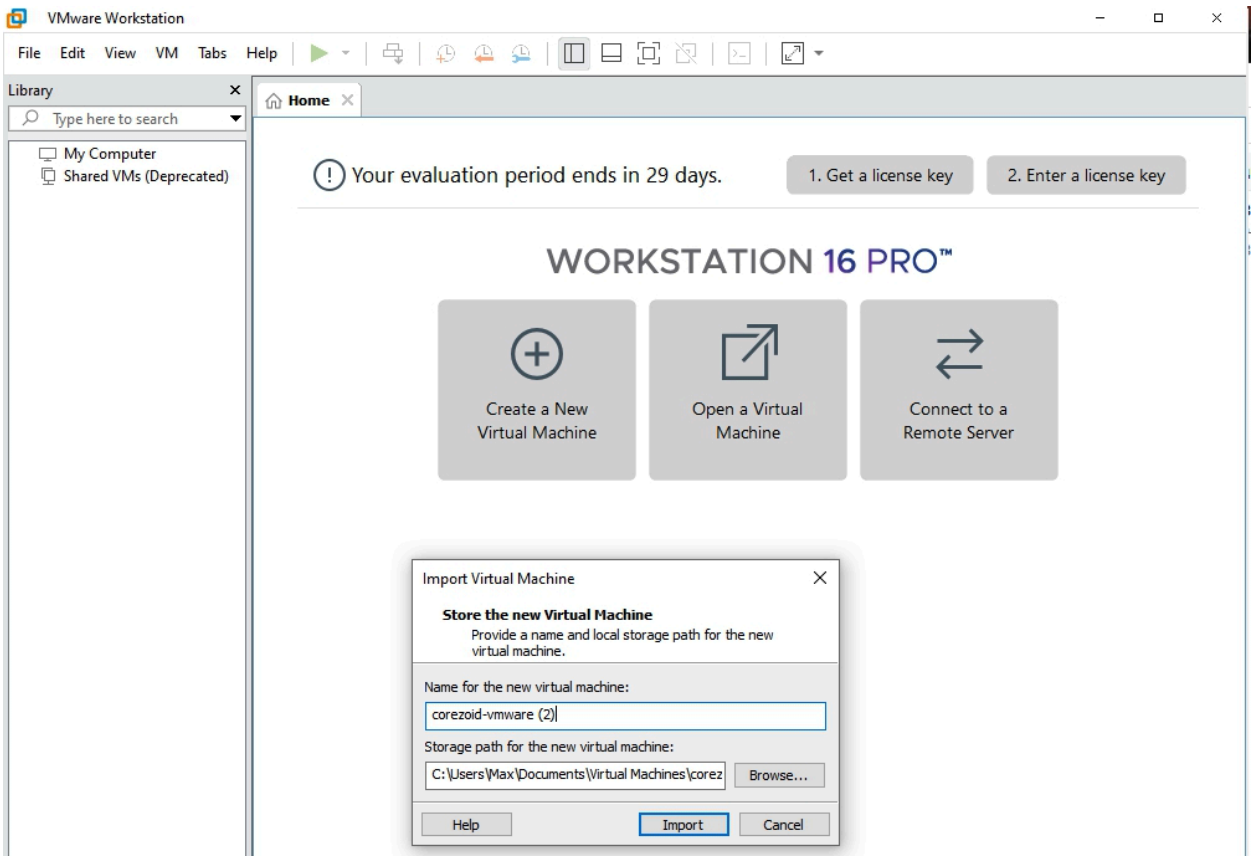
7. Browse to the **ovf-image** file.



Screenshot 286: **VMware deployment.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

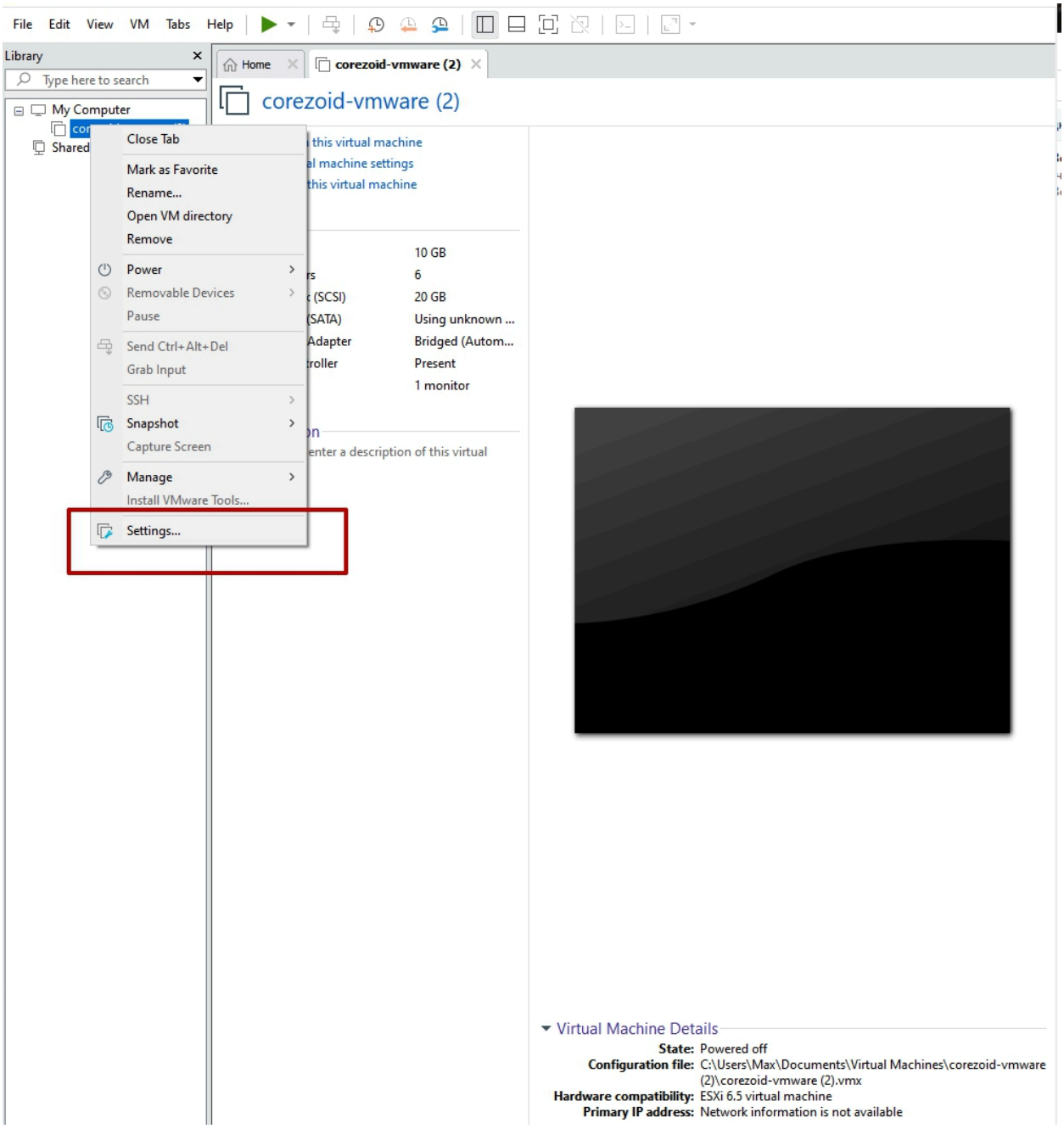
8. Specify the name for the new virtual machine.



Screenshot 287: **VMware deployment.**

9. Right click the uploaded image in the list on the left, and in the dropdown list, select **Settings**.

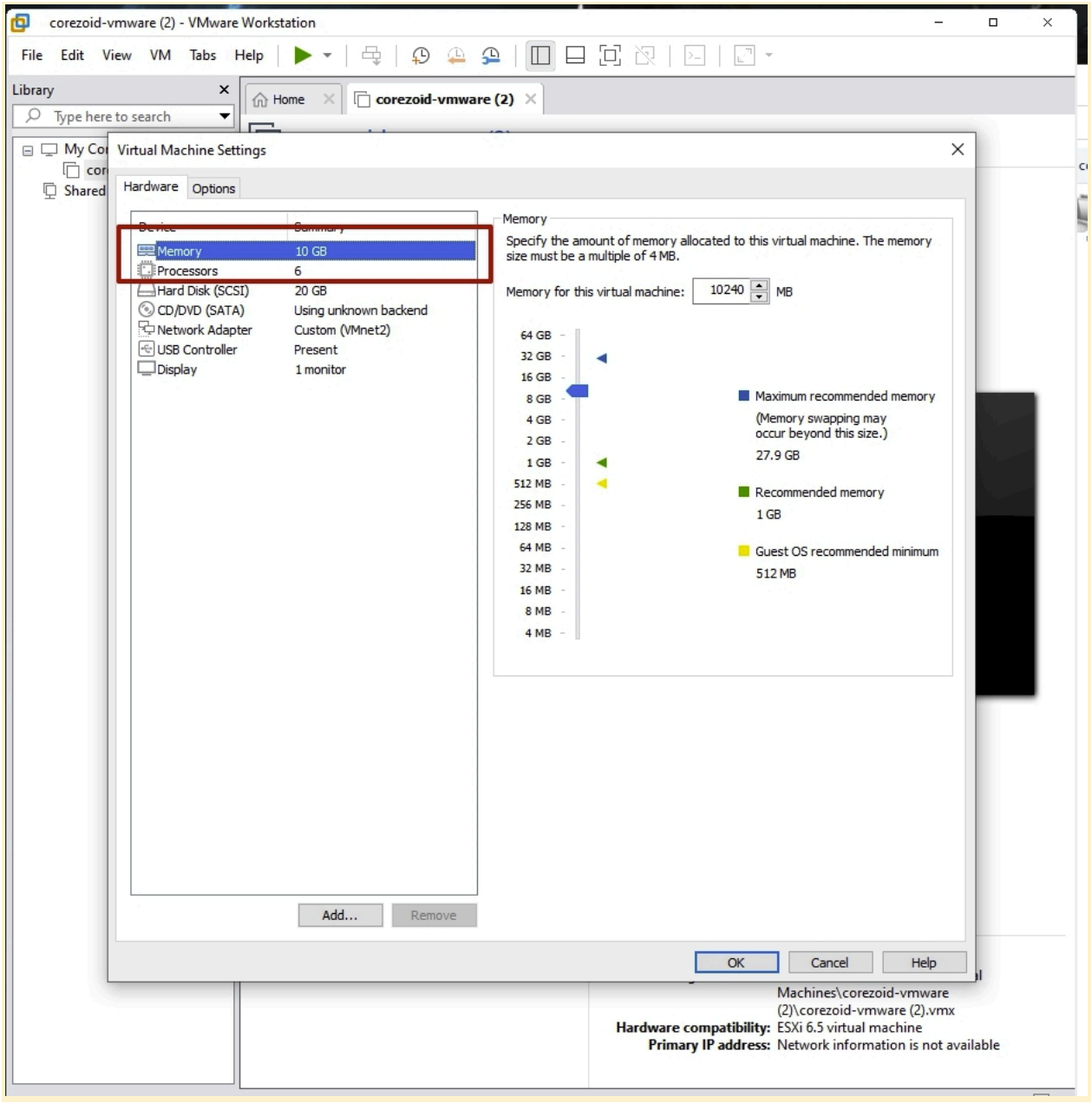
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 288: **VMware deployment.**

Before proceeding with the configuration of your virtual machine, make sure your hardware meets the minimum requirements (6 CPU cores, 10 GB RAM). To do that, select your virtual machine in the list, open the **Settings** menu on the **Hardware** tab.

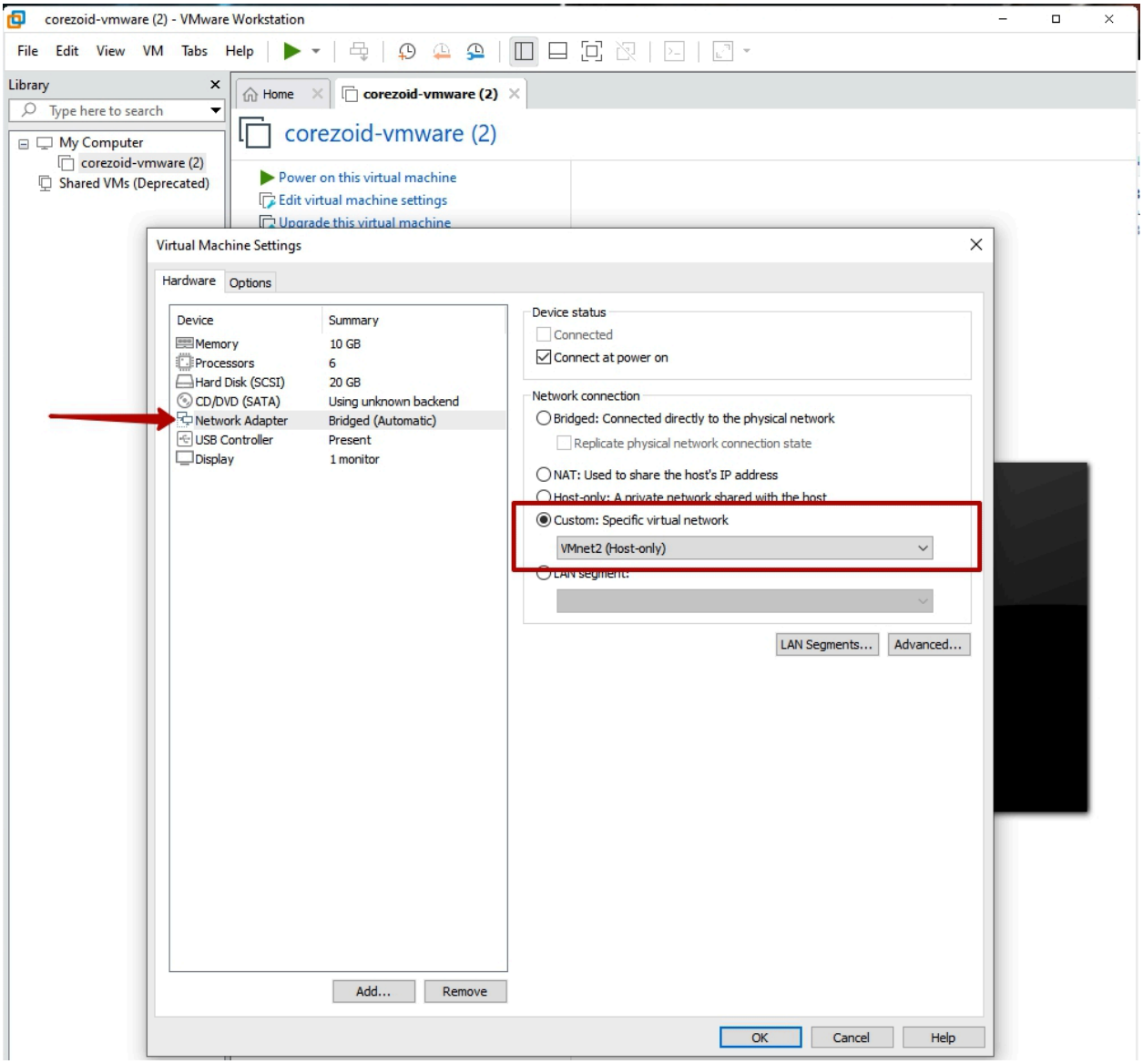
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 289: **VMware deployment.**

10. In the **Hardware** list, select **Network adapter**, and in the adapter's settings select the network, which you've created in the **Virtual network editor** on the step 1. Click **OK**.

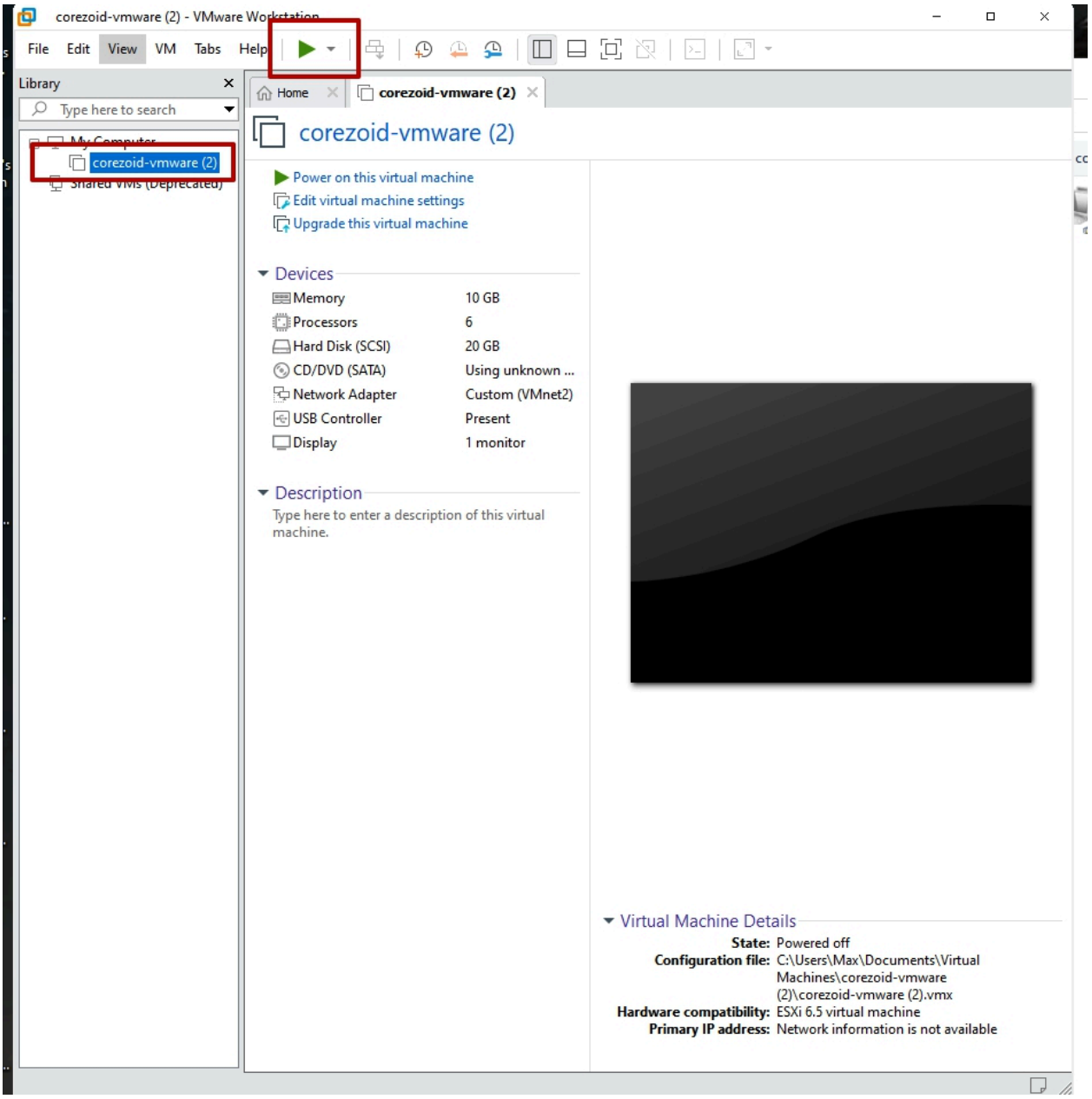
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 290: **VMware deployment.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

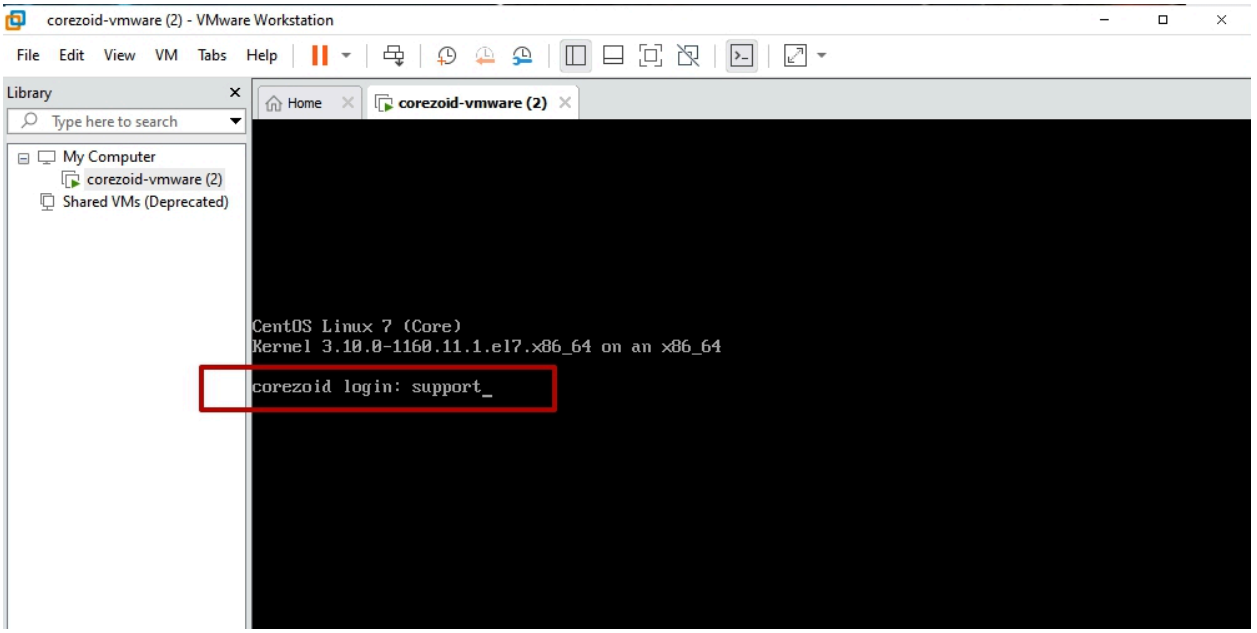
11. Select your virtual machine in the list and start it.



Screenshot 291: **VMware deployment.**

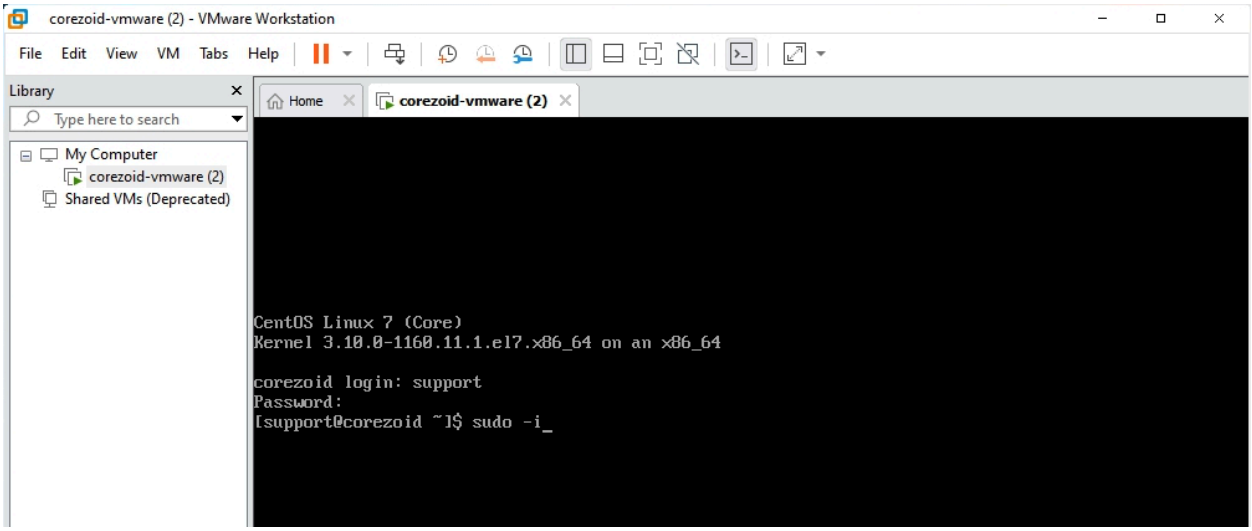
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

12. Enter the login.



Screenshot 292: **VMware deployment.**

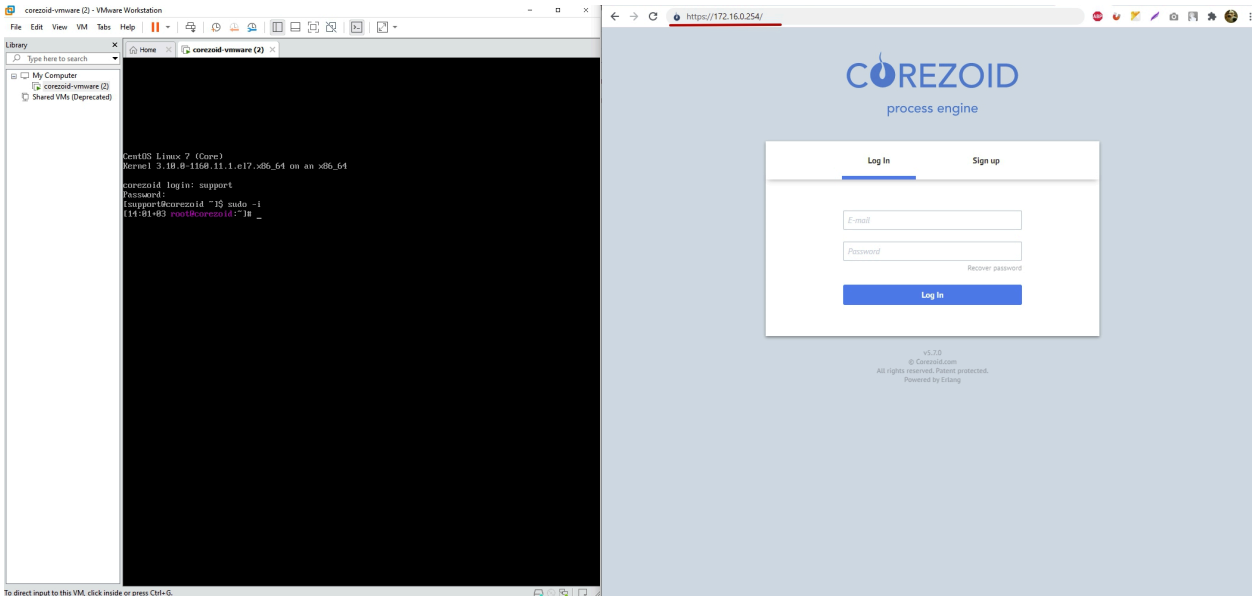
13. Enter the password.



Screenshot 293: **VMware deployment.**

After logging in successfully, you see the Corezoid login screen.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 294: **VMware deployment.**

Attaching disk image to new virtual machine

Requirements to the newly created virtual machine: **6 CPU cores, 10 GB RAM**.

Launching

After installing the image, the IP address of the server is 172.16.0.254.

If necessary, you can change the IP address.

To change the web endpoint (domain or IP), you must run a script that is located in:

`/root/fix_domain.sh`.

Web access: https://172.16.0.254 (or your value)

Web login: admin@corezoid.loc

Web password: Middleware2020-

SSH access:

Login: support

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Password: corezoid

Get superuser by command: `sudo -i`

In case of having any questions, please do not hesitate to contact us: support@corezoid.com.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

4.2 VMware Fusion

Requirements

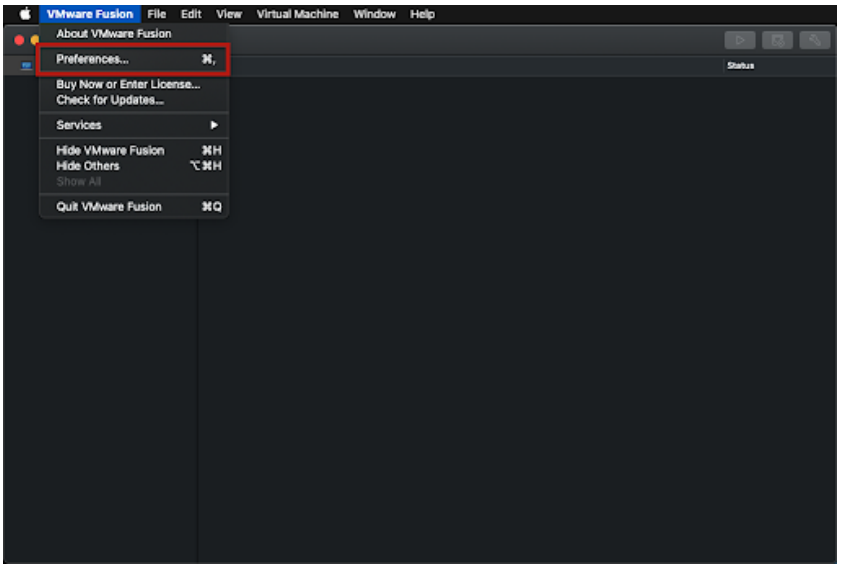
VMware Fusion version (Activated)	11 or later
CPU	4 (minimum) - 6 (recommended)
RAM	6GB (minimum) - 10GB (recommended)
HDD	minimum 20GB free

Table 40: VMware fusion requirements.

Installation

Note: Before installing an image, make sure you have installed and activated a paid version of VMware Fusion. You won't be able to deploy Corezoid on the Trial version of VMware Fusion.

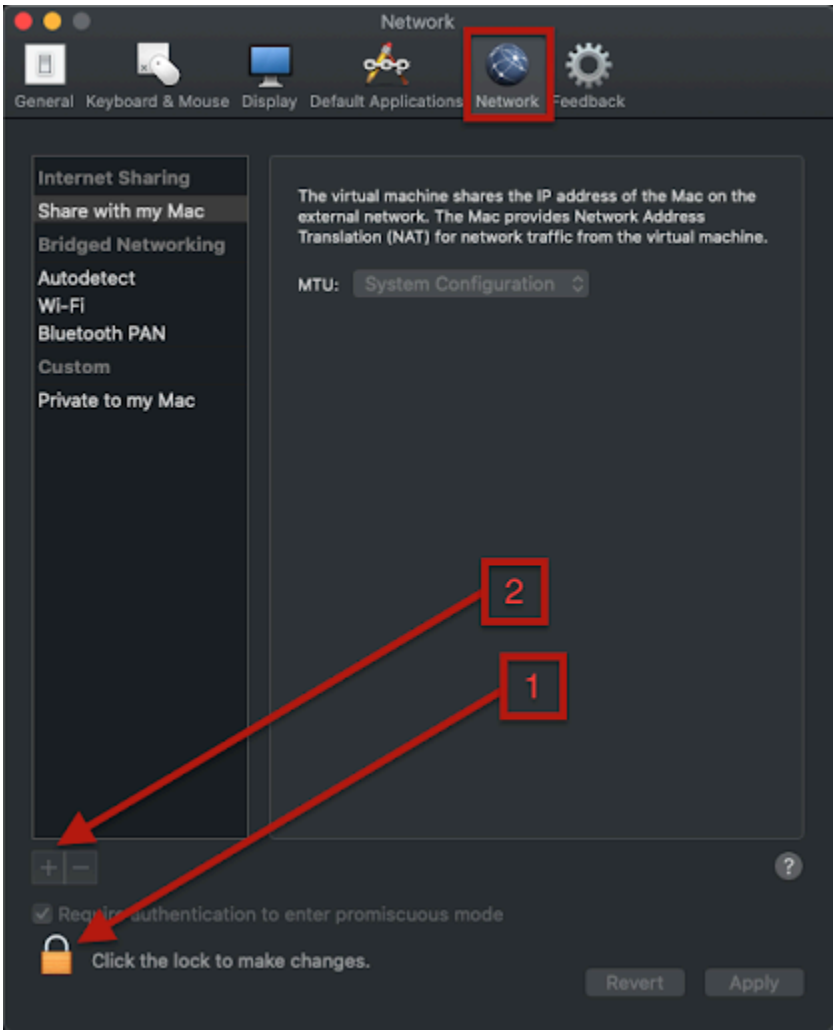
1. Click VMwareFusion and in the dropdown list, select Preferences.



Screenshot 295: **VMware fusion deployment.**

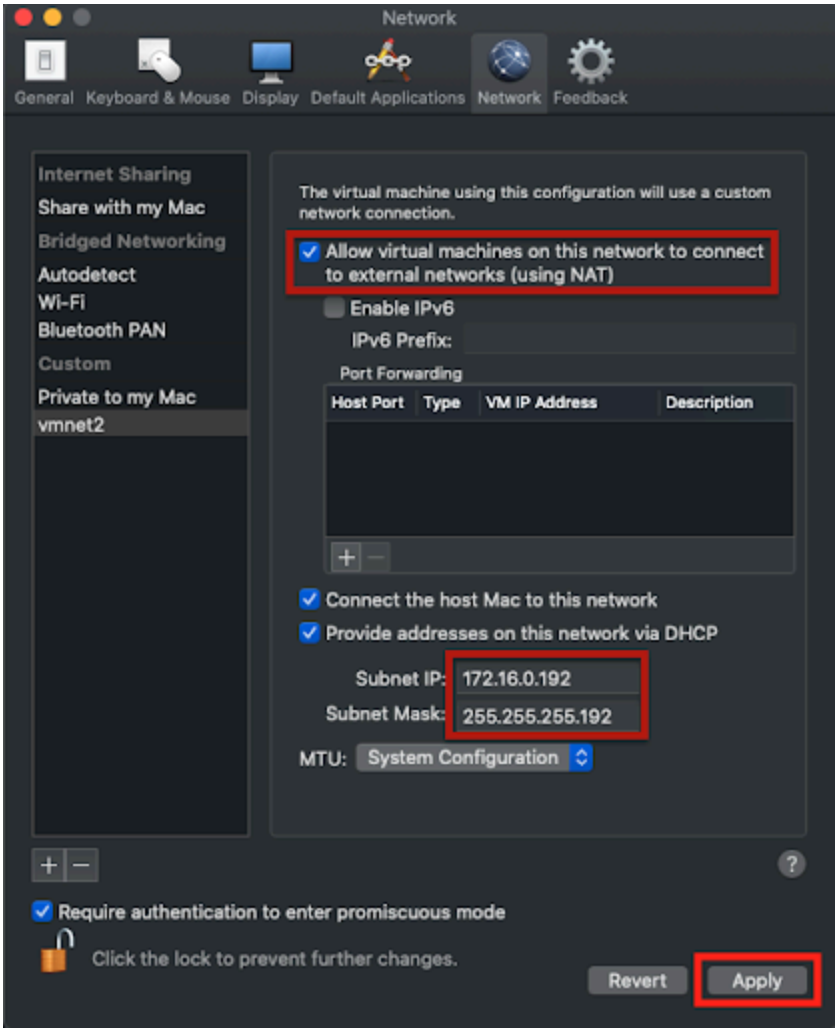
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

2. In the Network dialog, click Network and add a new Network interface.



Screenshot 296: **VMware fusion deployment.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

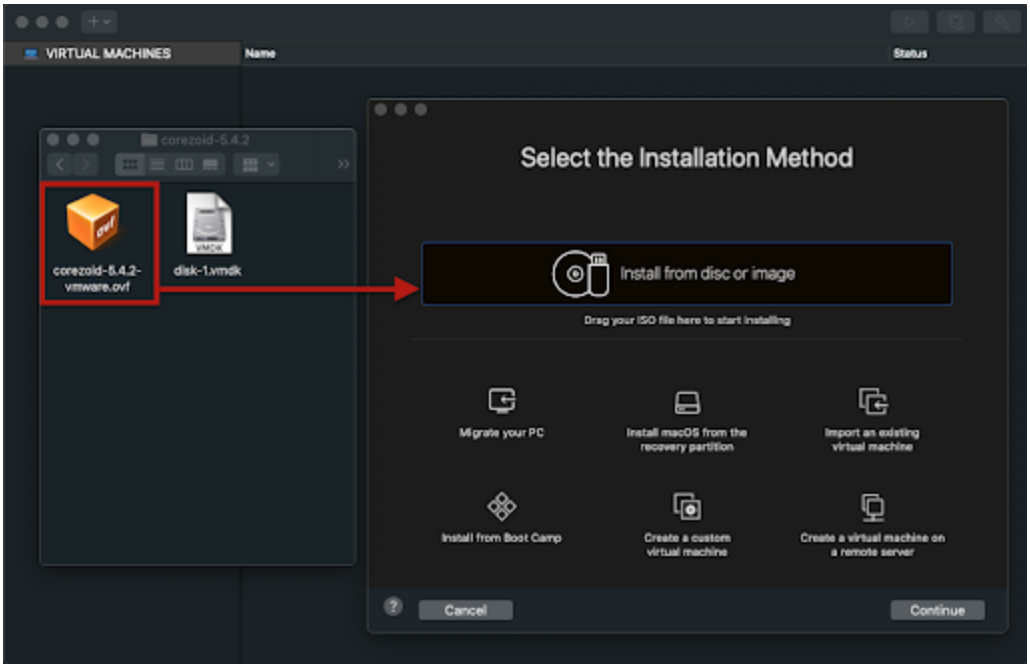


Screenshot 297: **VMware fusion deployment.**

If you have troubles with setting Subnet IP and Subnet Mask parameters, go to Troubleshooting.

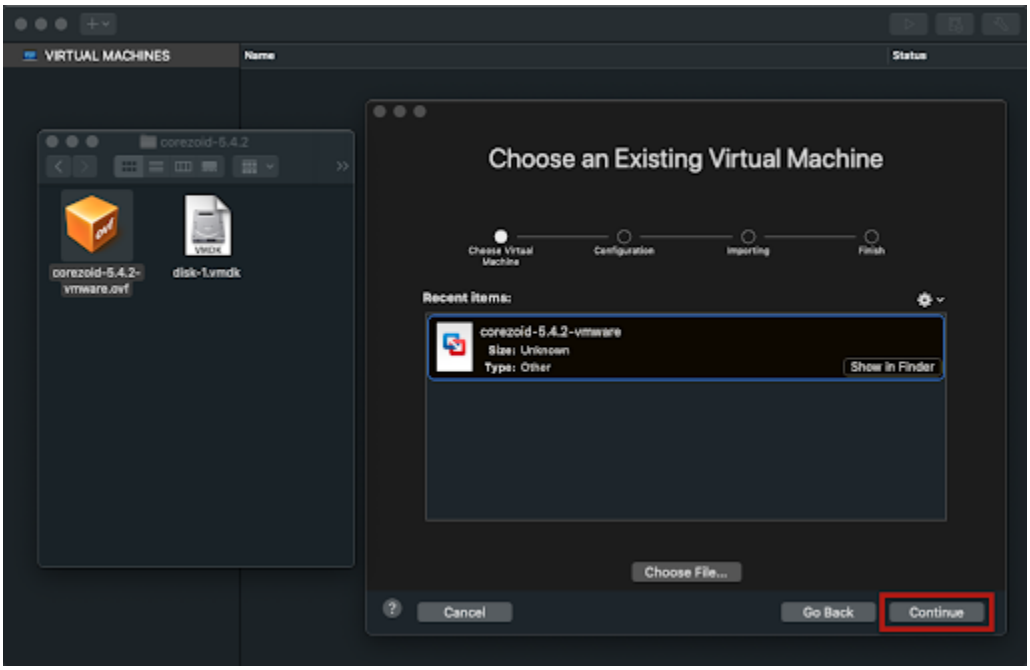
3. Create a new virtual machine: drag the .ovf file to the Select the Installation Method modal window.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 298: **VMware fusion deployment.**

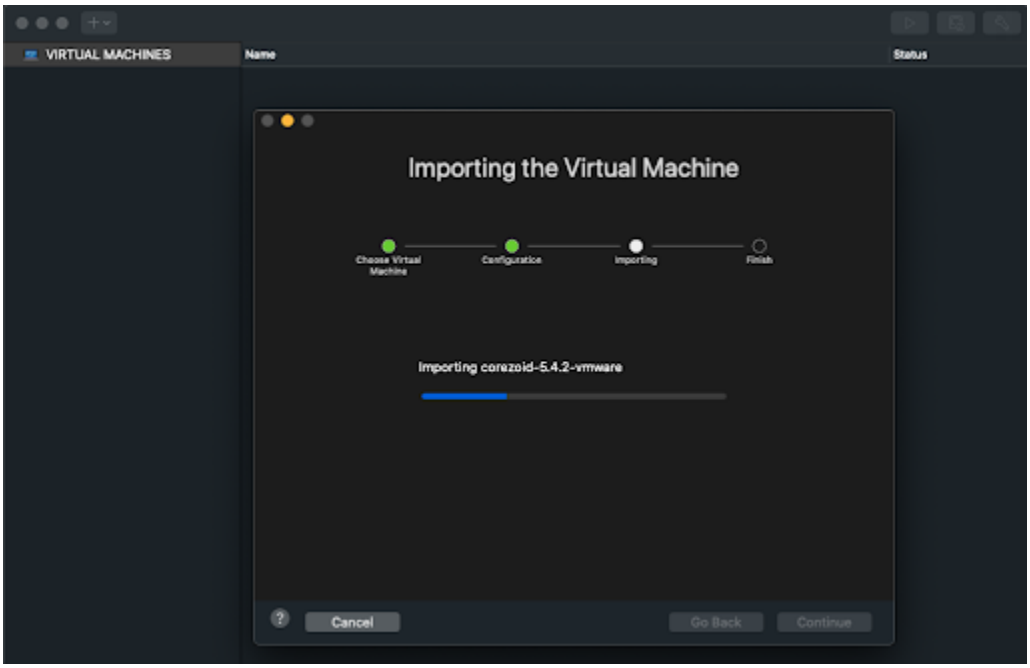
4. Select the existing virtual machine and click Continue.



Screenshot 299: **VMware fusion deployment.**

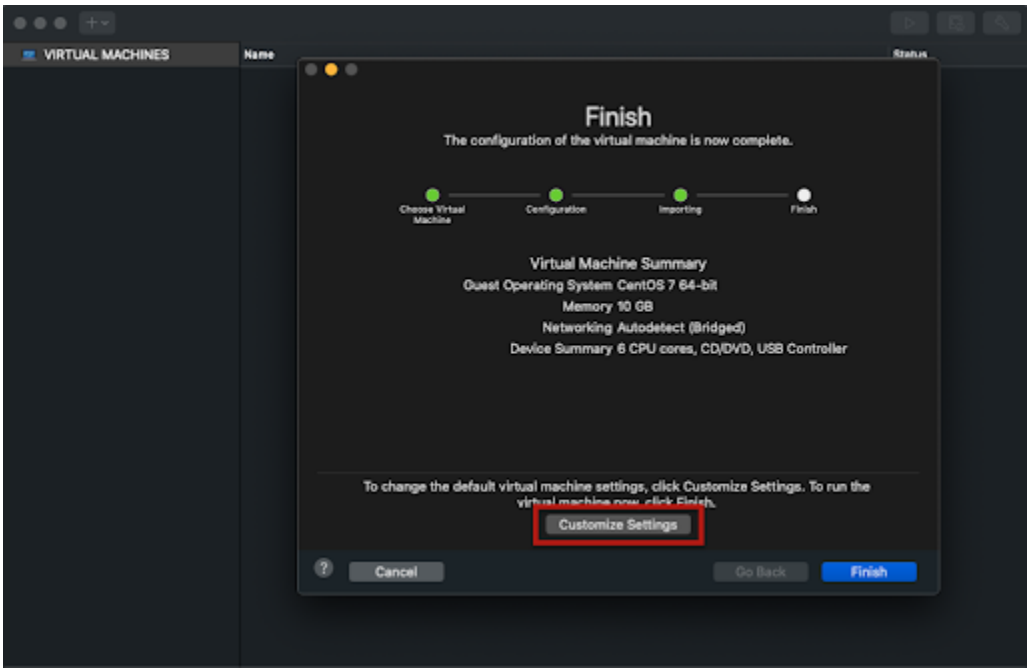
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

5. The installation starts.



Screenshot 300: **VMware fusion deployment.**

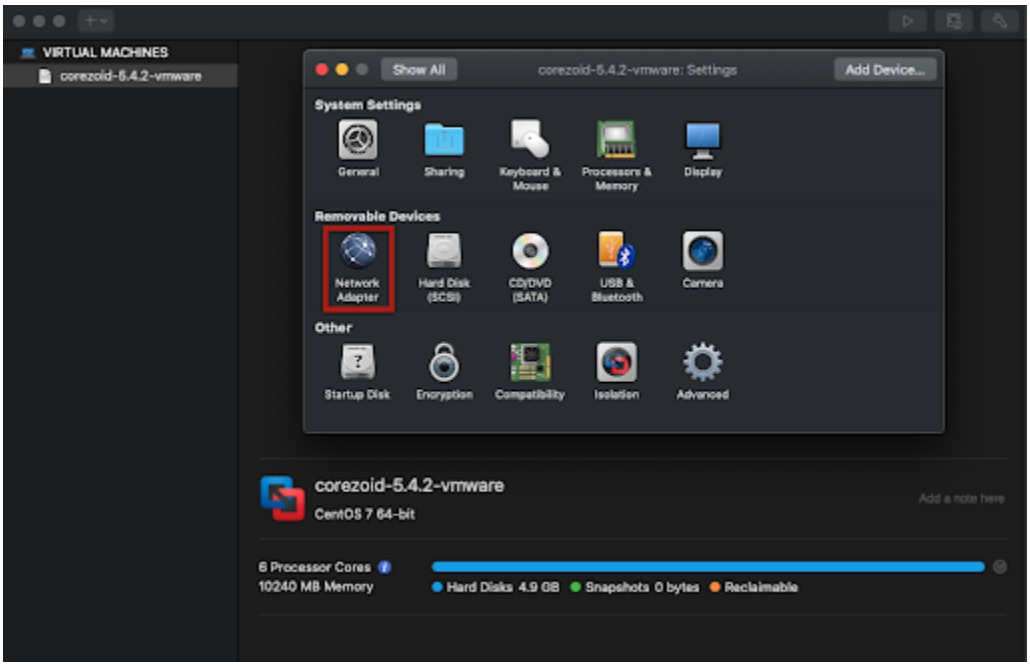
6. Click Customize Settings.



Screenshot 301: **VMware fusion deployment.**

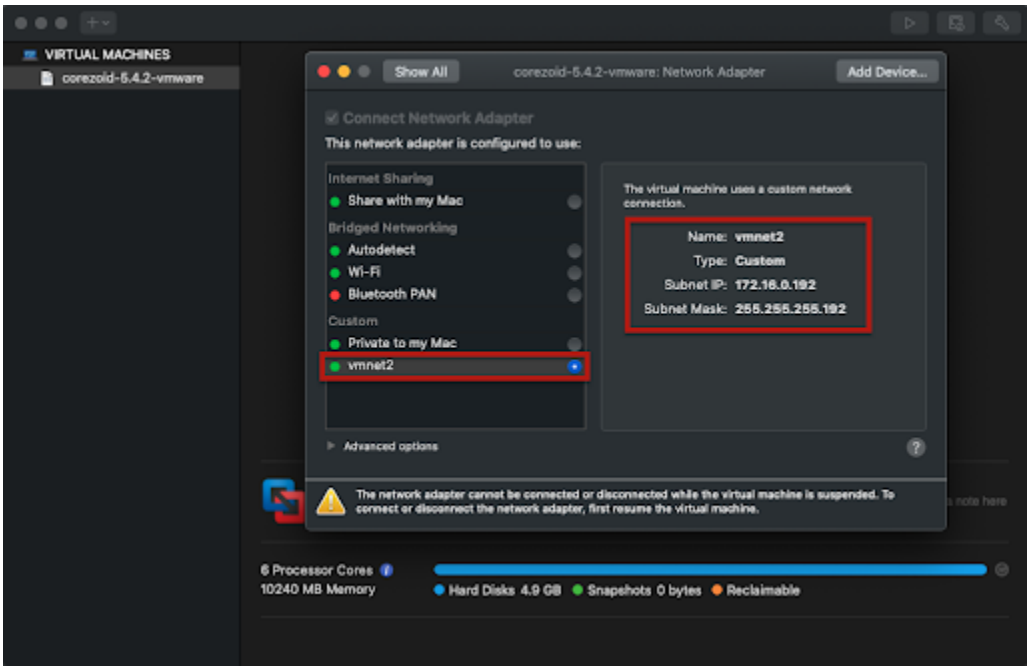
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

7. In the menu, select Network Adapter.



Screenshot 302: **VMware fusion deployment.**

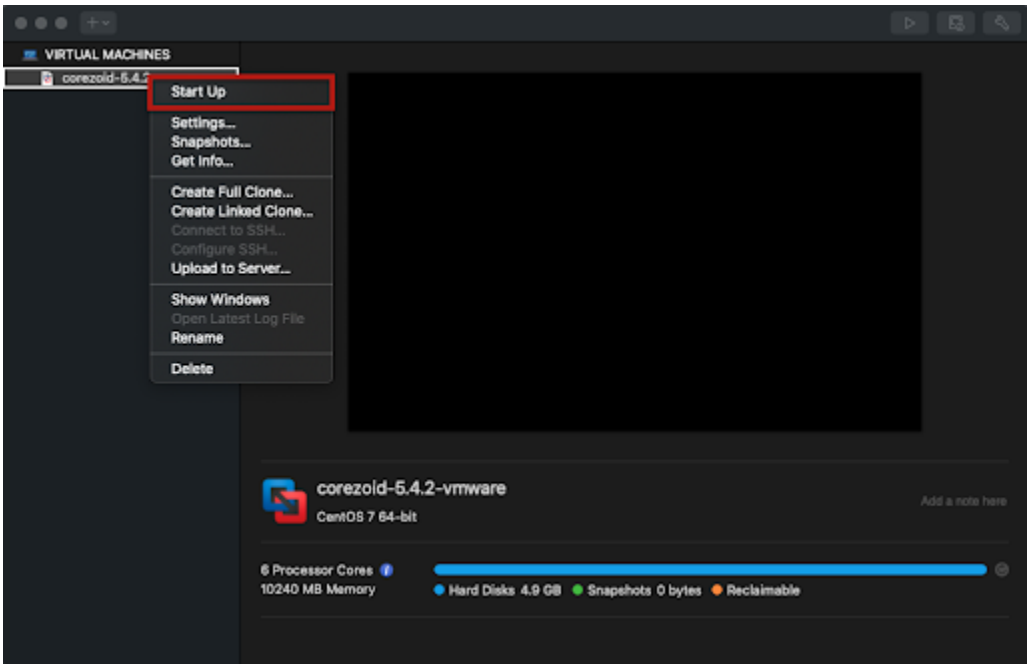
8. Select your network interface.



Screenshot 303: **VMware fusion deployment.**

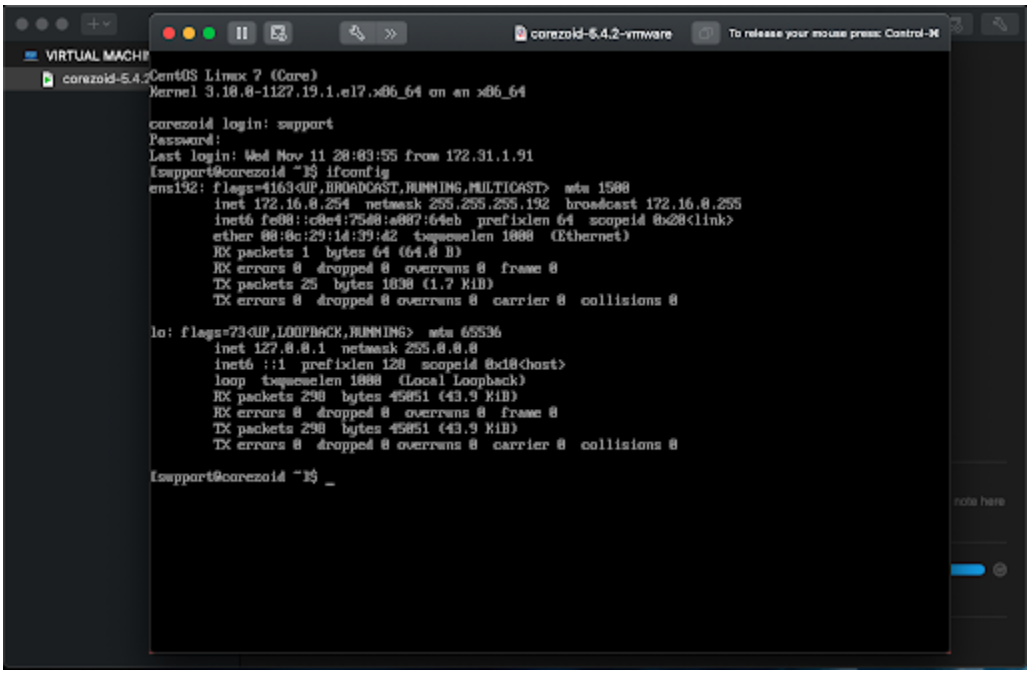
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

9. Start up your virtual machine.



Screenshot 304: **VMware fusion deployment.**

10. Finish the deployment.



Screenshot 305: **VMware fusion deployment.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Launching

A few minutes later, after the successful launch of the system, you can open a Corezoid in your browser.

Web access: https://172.16.0.254/ (or your value)

Login: admin@corezoid.loc

Password: Middleware2020-

SSH access:

Login: support

Password: corezoid

Get superuser by command: `sudo -i`

Additional Information

After the image is installed, the IP address of the server is 172.16.0.254.

Note: You can change the IP address if needed: run a script which is located in /root/fix_domain_crz.sh.

SSH access:

Login: support

Password: corezoid

Get superuser by command: `sudo -i`

Monitoring and restarting

You can also monitor and restart applications via the monit:

- Output of all the monitoring processes: `sudo monit status`

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

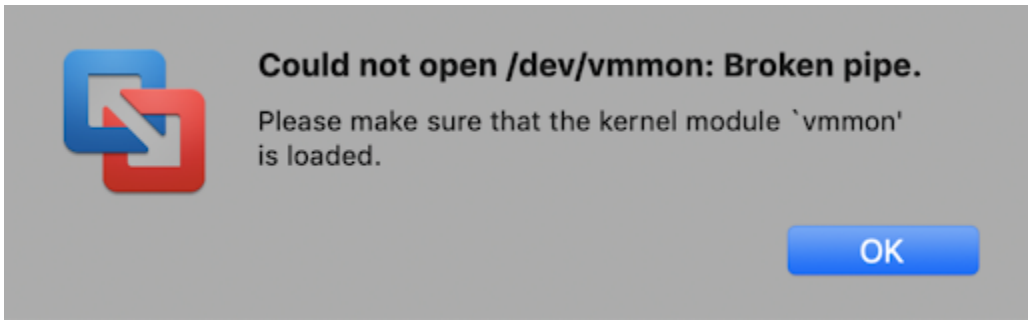
- Disabling monitoring of monit applications:`monit unmonitor all`
- Enabling the monitoring:`monit monitor all`

If monit is used to start / stop the application, then stop / start the application using the monit commands:

- monit stop conveyor_api_multipart
- monit start conveyor_api_multipart

Troubleshooting

When starting the virtual machine, the following error appears.

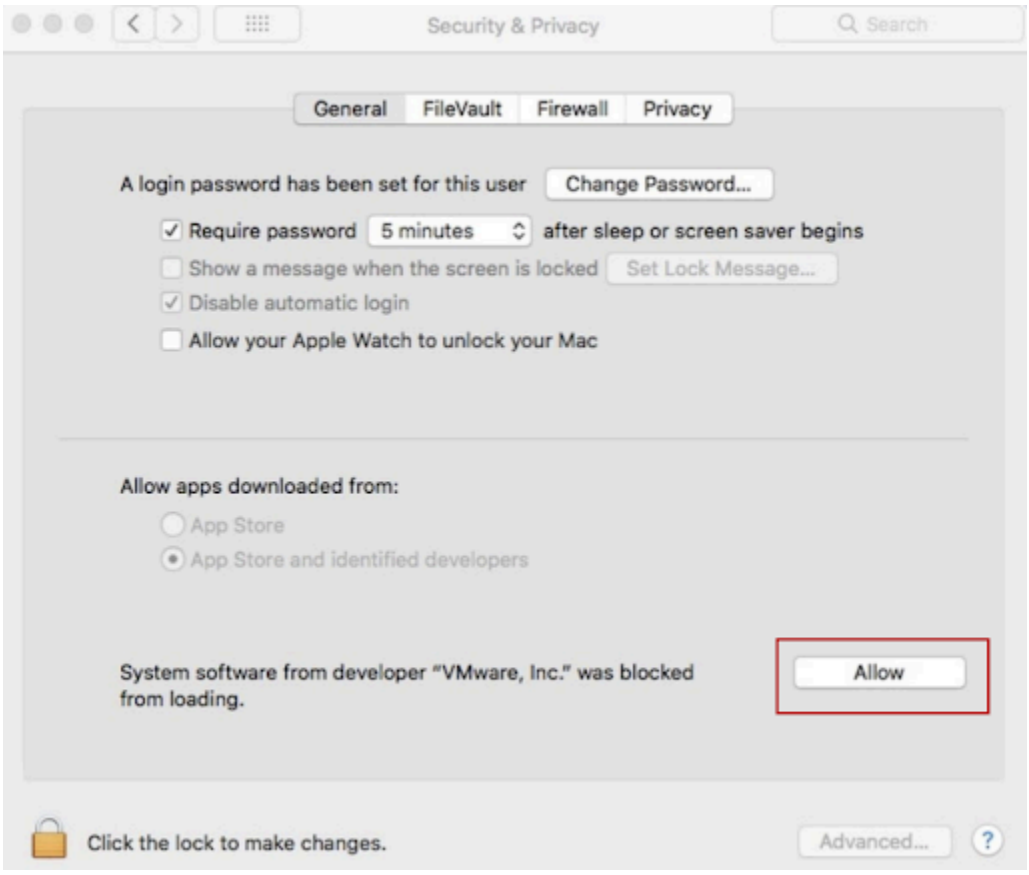


Screenshot 306: **VMware fusion VM start error.**

Perform the following actions in the system security settings:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

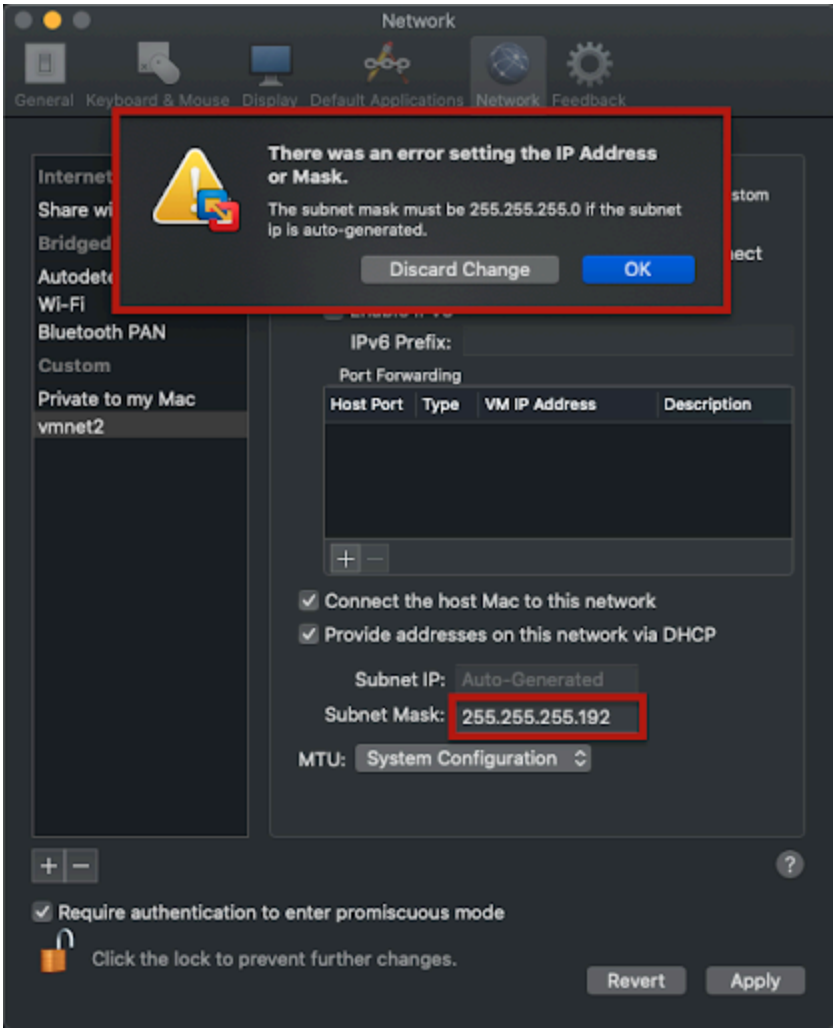
1. Go to the System Preferences tab, open the Security & Privacy, and click Allow.



Screenshot 307: **VMware fusion troubleshooting.**

2. If you get these errors when configuring the subnet, set the Subnet Mask to 255.255.255.0 and Subnet IP value to 172.16.0.0.

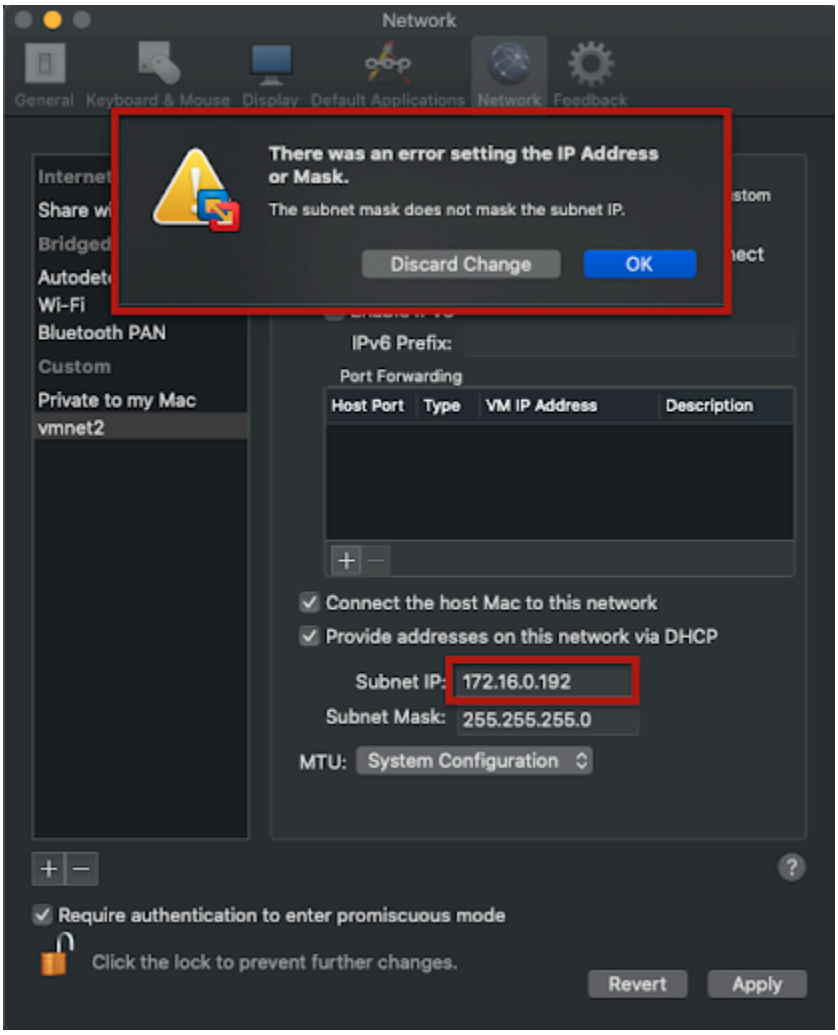
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 308: **VMware fusion troubleshooting.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3. Change the Subnet Mask value to 255.255.255.192 and Subnet IP to 172.16.0.192.



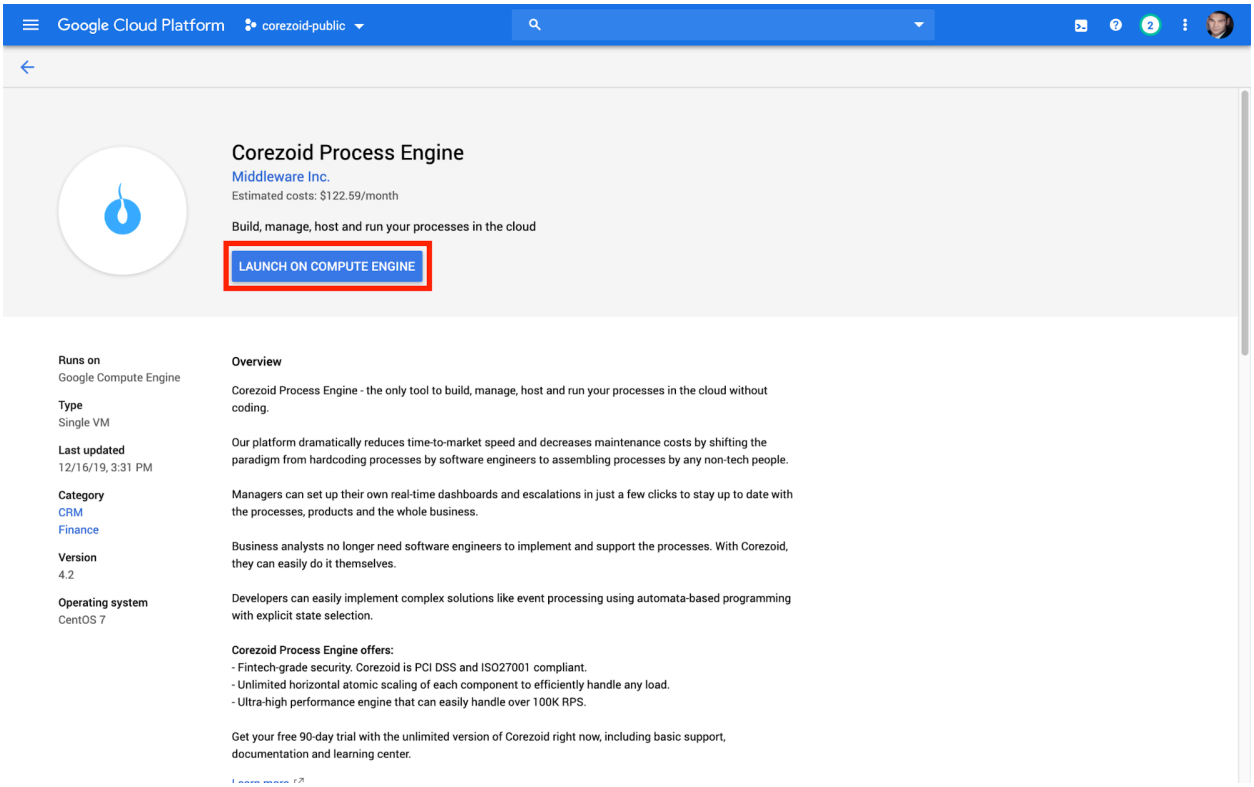
Screenshot 309: **VMware fusion troubleshooting.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

4.3 Deploy Corezoid on Google Cloud Engine

To deploy Corezoid on Google Cloud Engine:

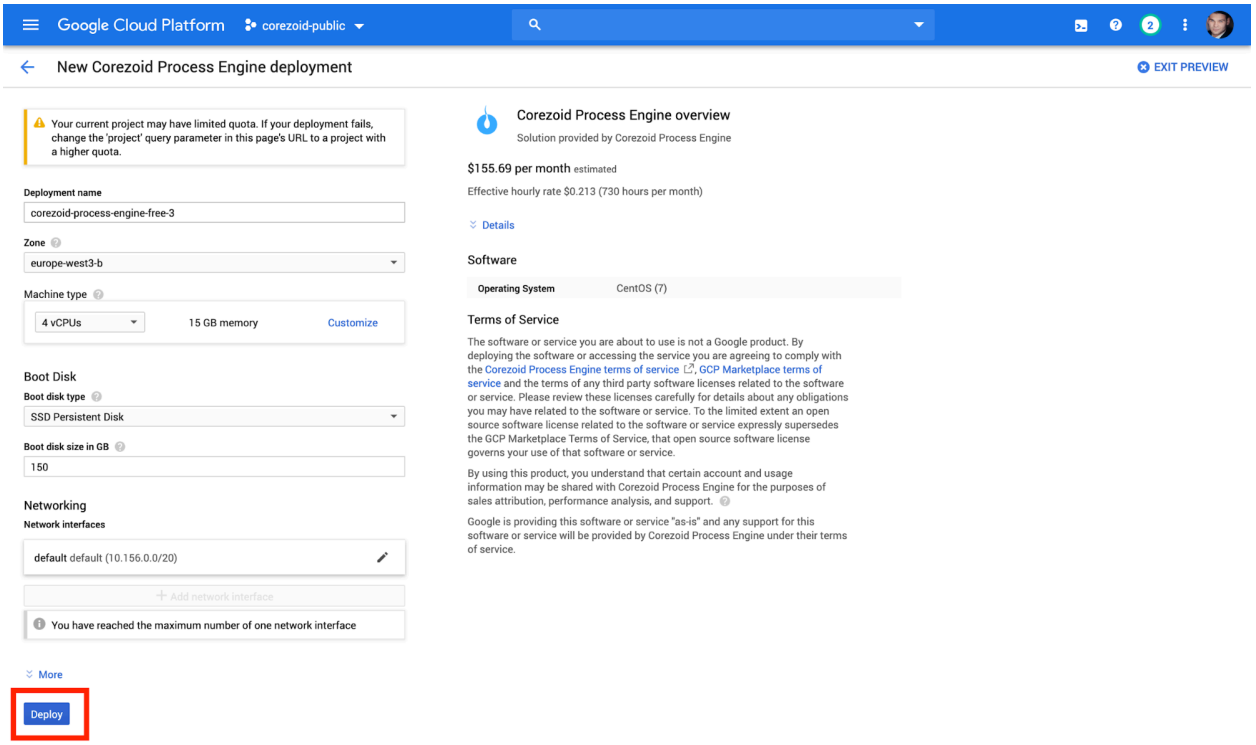
1. On the **Corezoid Process Engine** page, click **Launch On Compute Engine**.



Screenshot 310: **Deployment on GCE.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

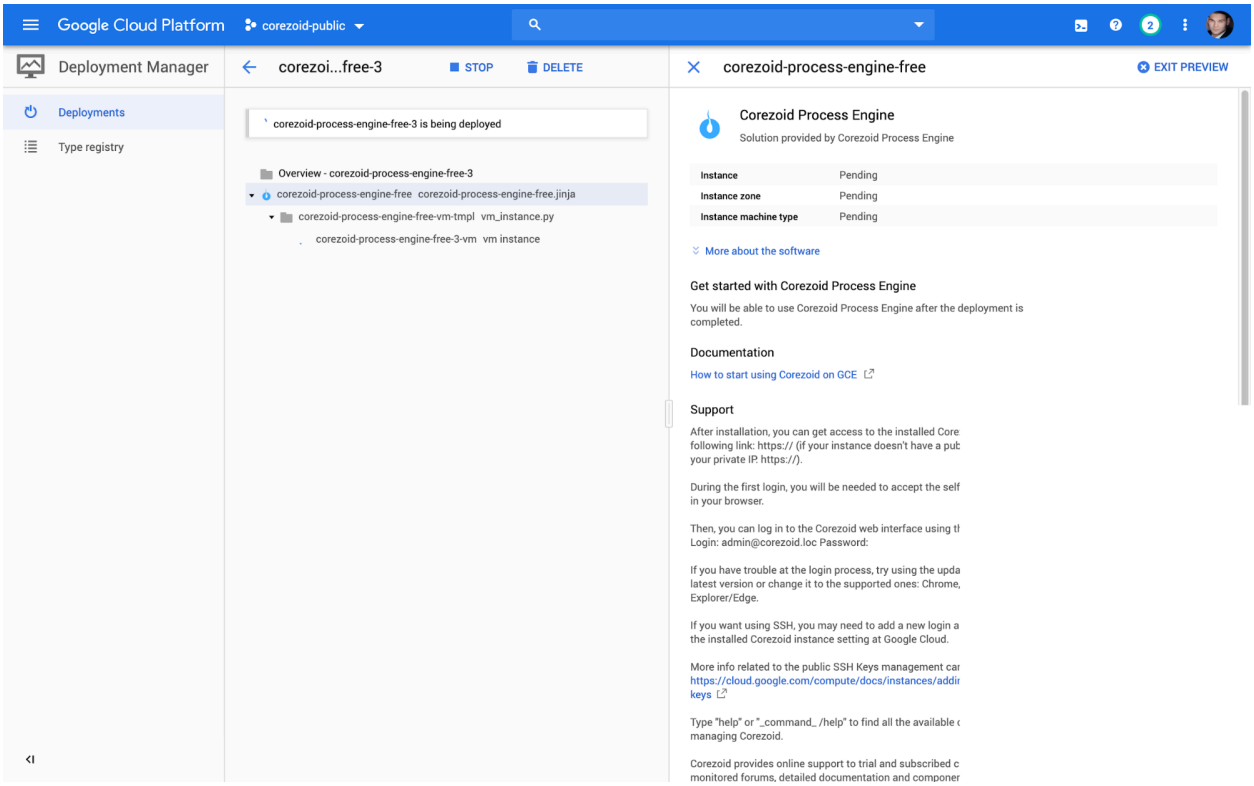
2. Configure deployment options and click **Deploy**.



Screenshot 311: **Deployment on GCE.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

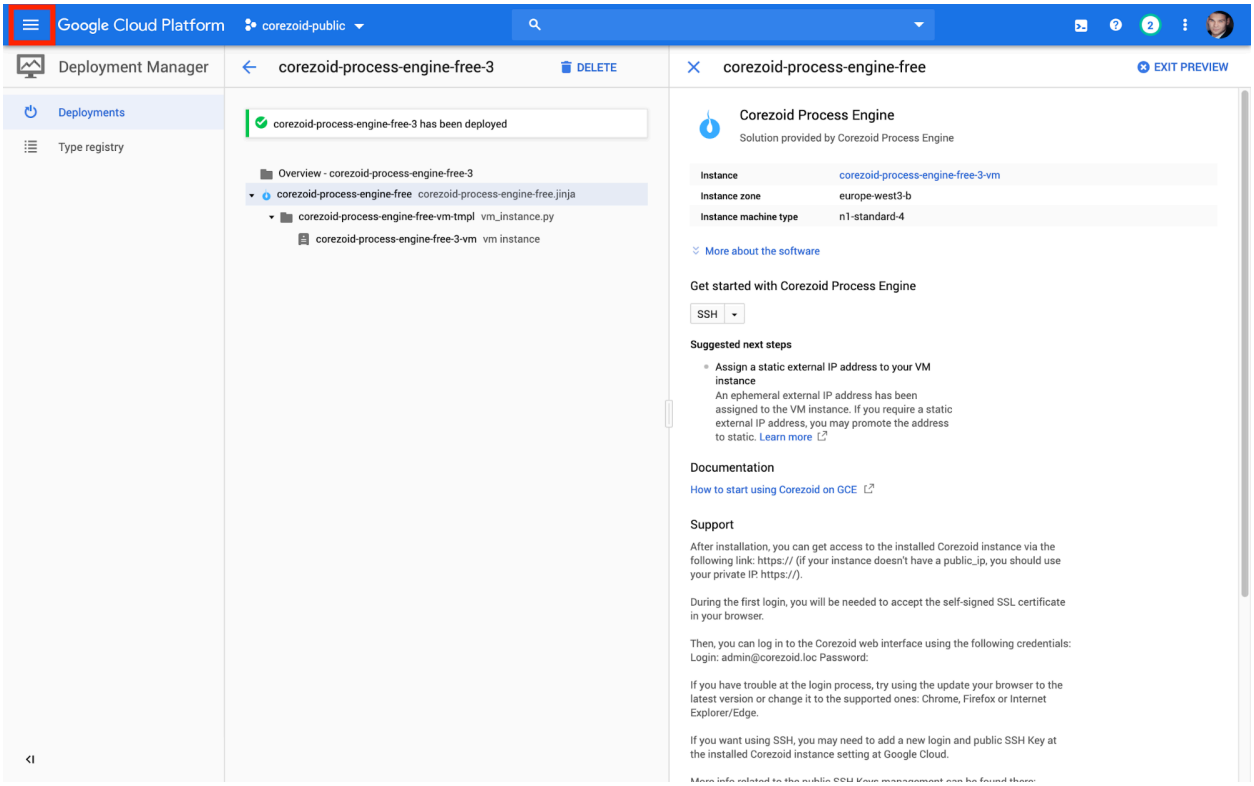
3. Wait until the deployment is complete.



Screenshot 312: **Deployment on GCE.**

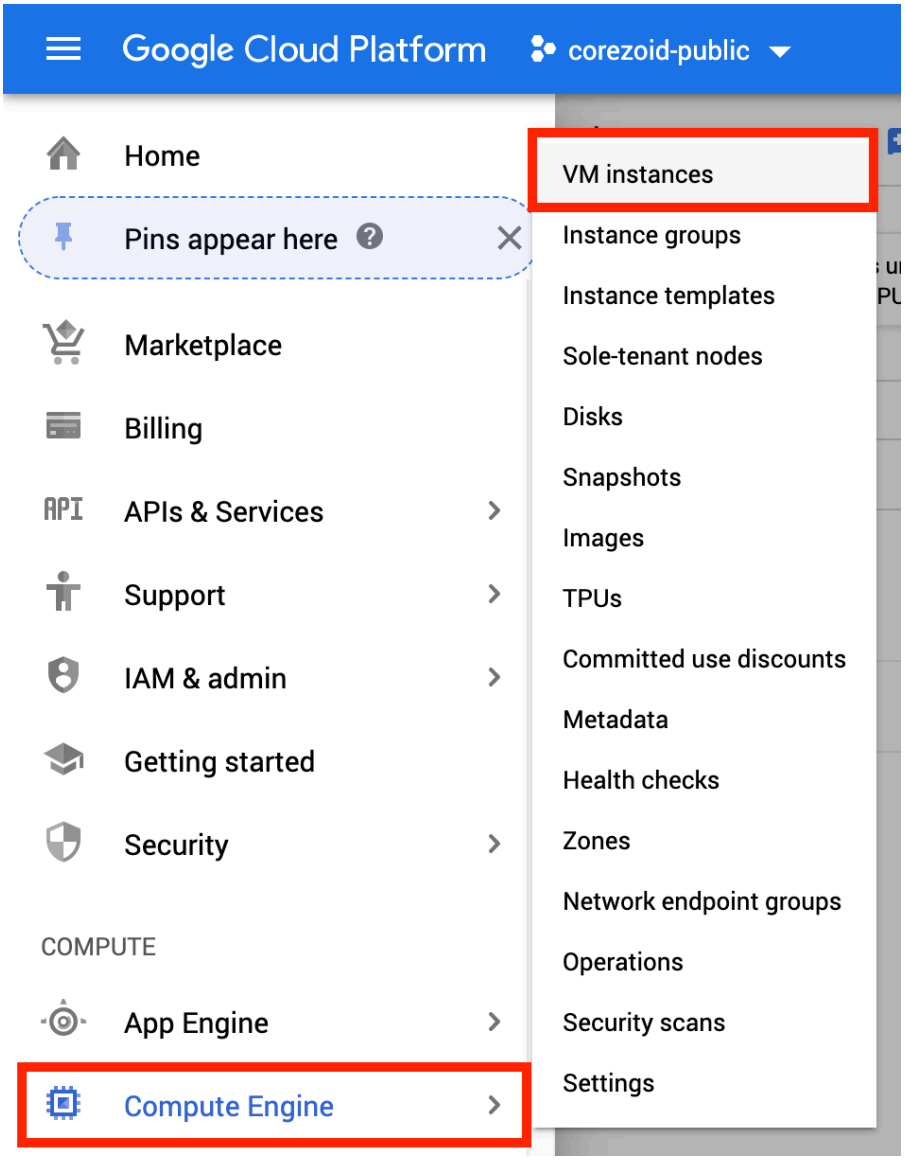
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

4. Click the **Menu** icon and select **Compute Engine -> VM instances**.



Screenshot 313: **Deployment on GCE.**

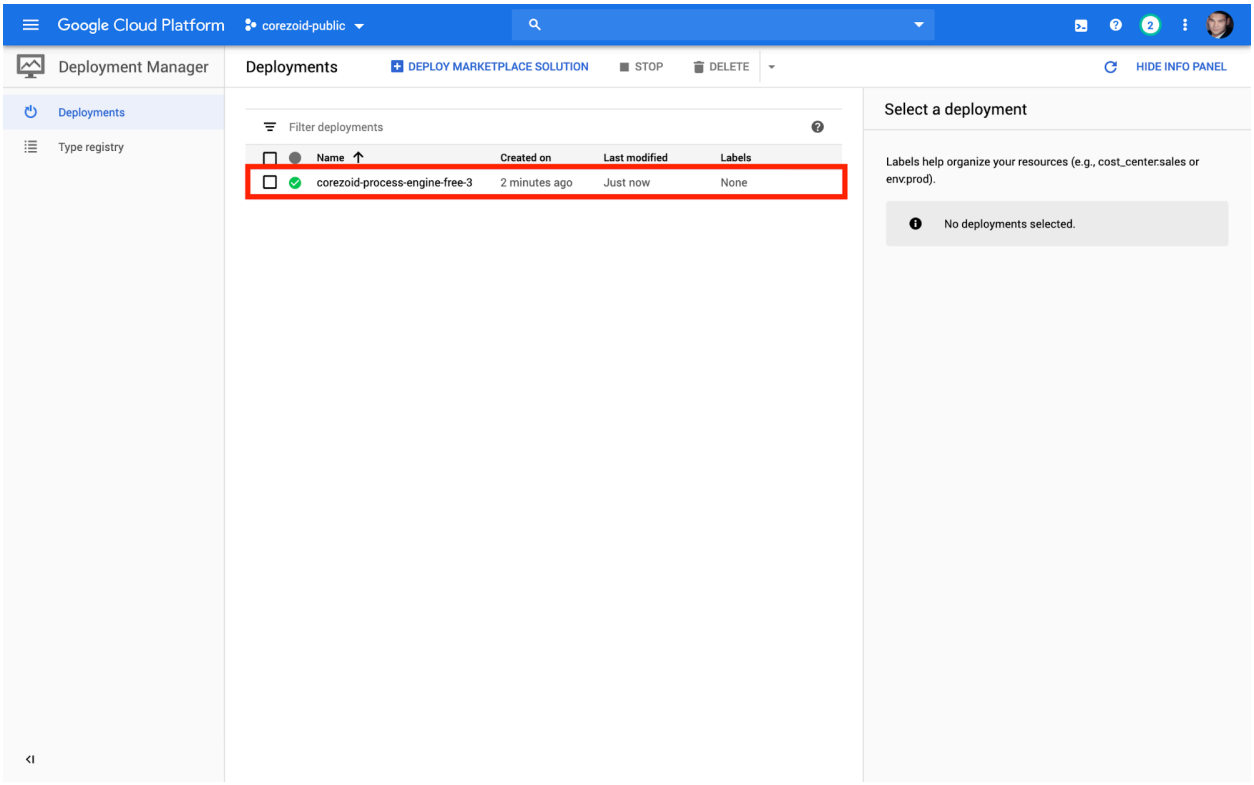
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 314: **Deployment on GCE.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

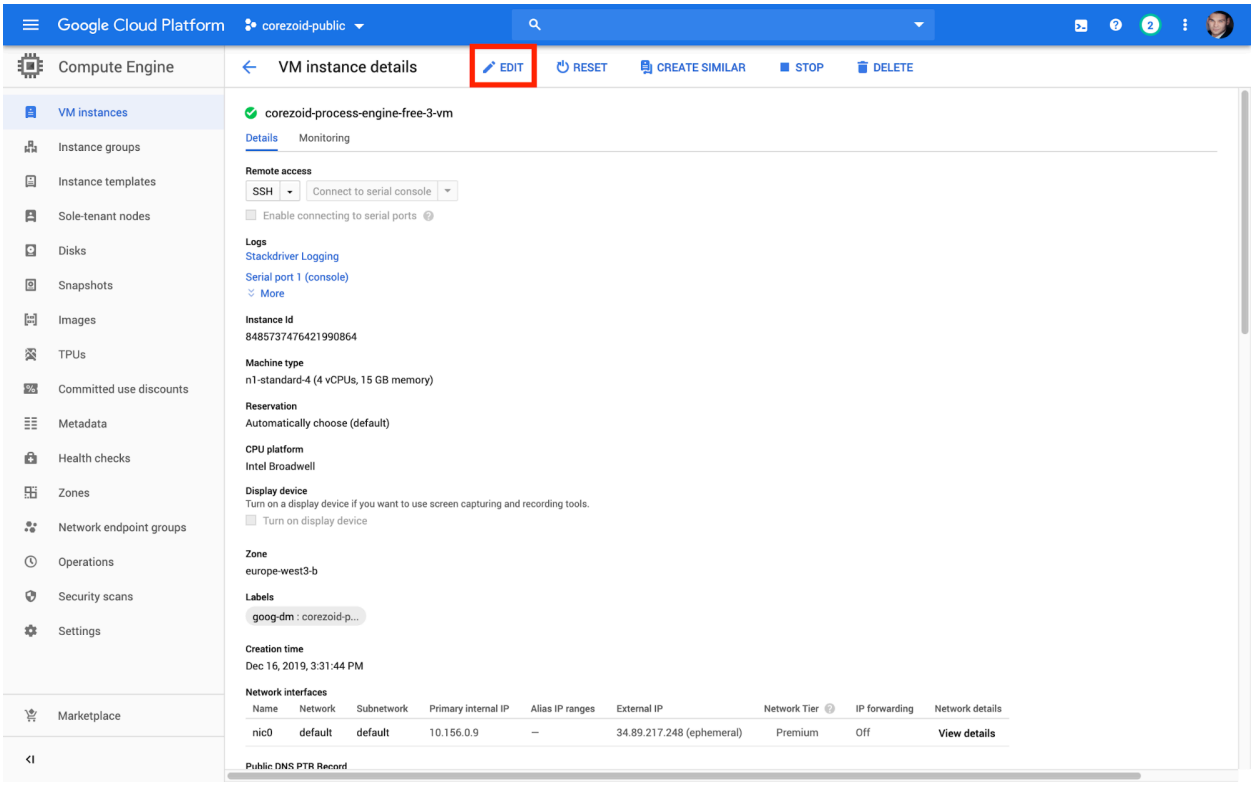
5. Select the deployed Corezoid instance in the list.



Screenshot 315: **Deployment on GCE.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

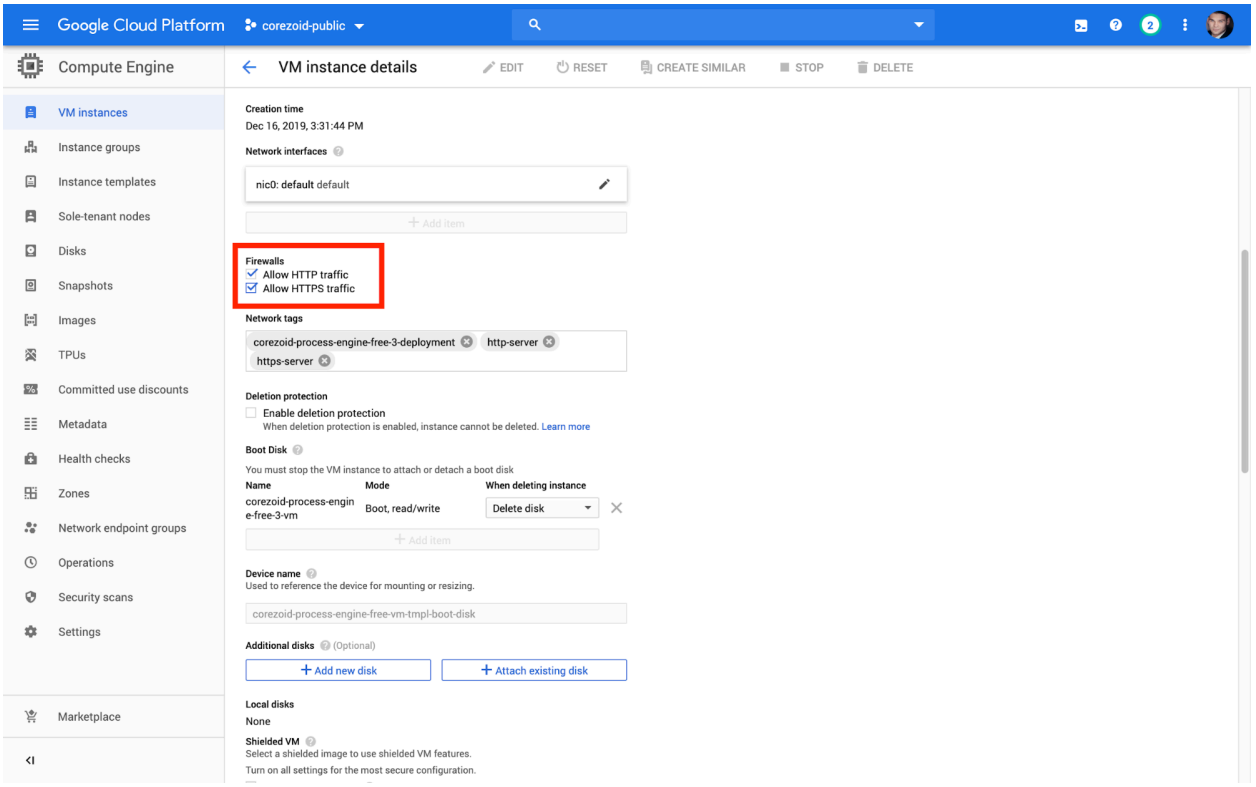
6. Click **Edit**.



Screenshot 316: **Deployment on GCE.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

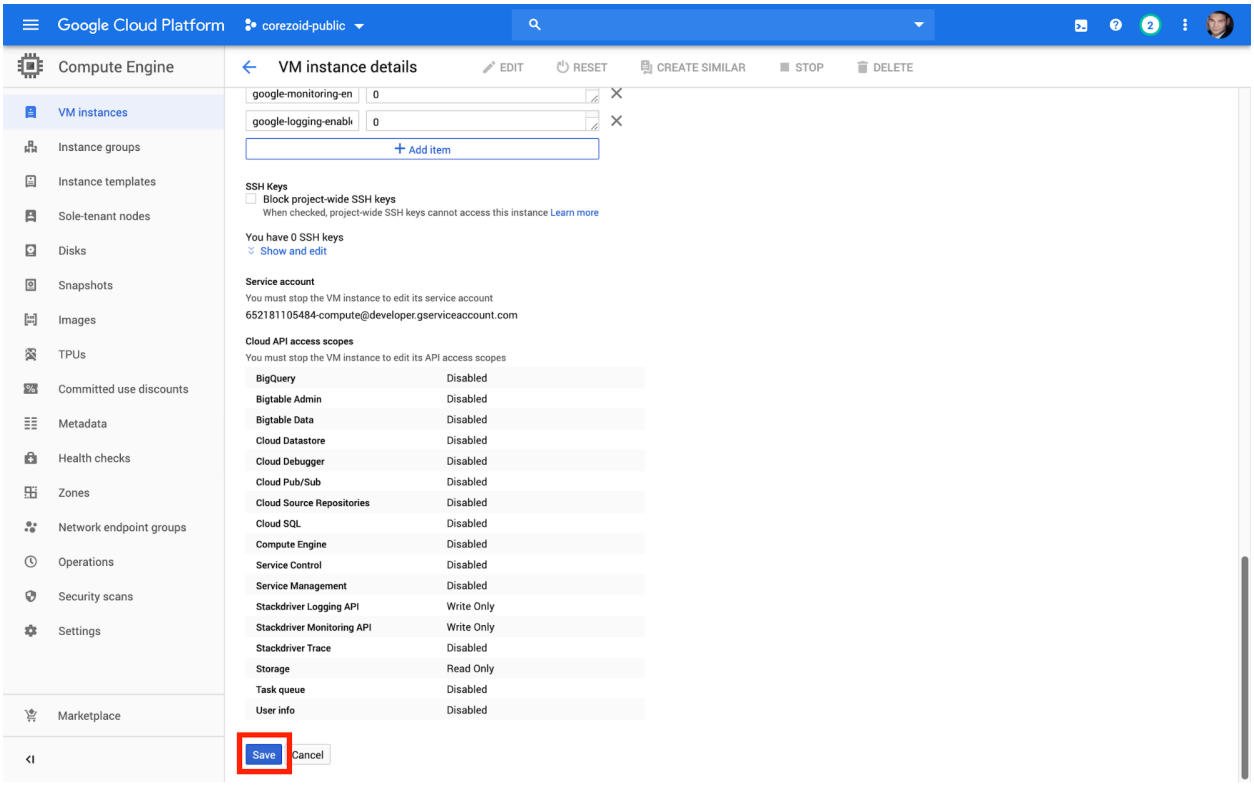
7. Scroll down to the **Firewall** section, then select the **Allow HTTP traffic** and **Allow HTTPS traffic** checkboxes.



Screenshot 317: **Deployment on GCE.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

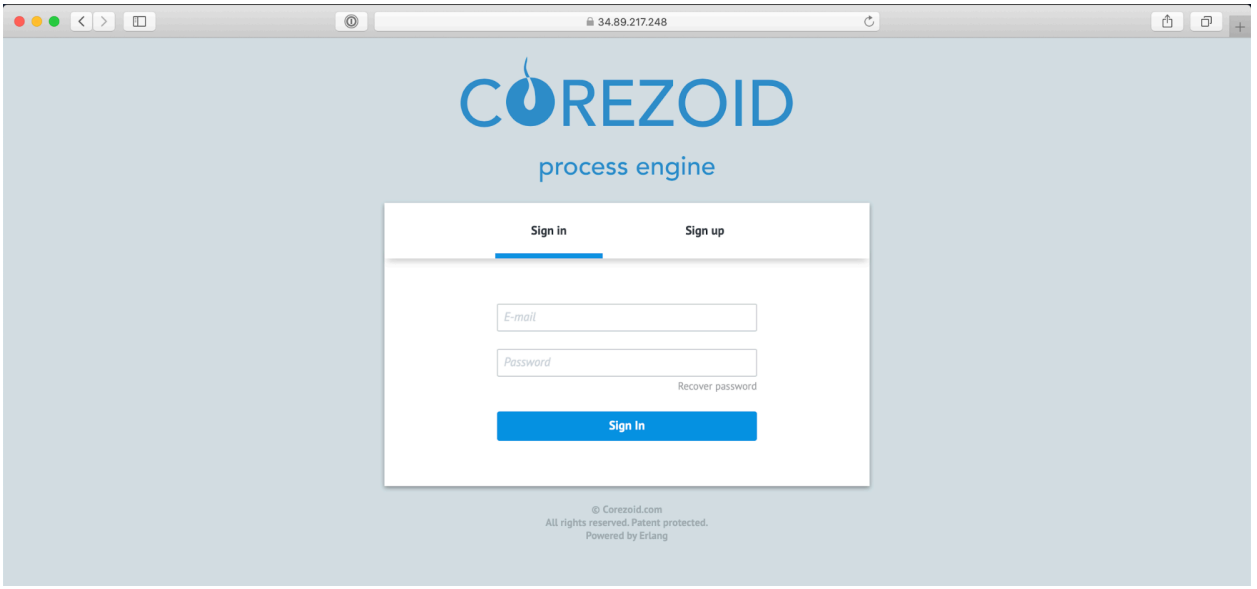
8. Scroll down and click **Save**.



Screenshot 318: **Deployment on GCE.**

9. After the installation, you can get access to the installed Corezoid instance via the following link: https://<public_ip> (if your instance doesn't have a public_ip, you need to use your private IP: https://<private_ip>). E.g. https://34.89.217.248

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

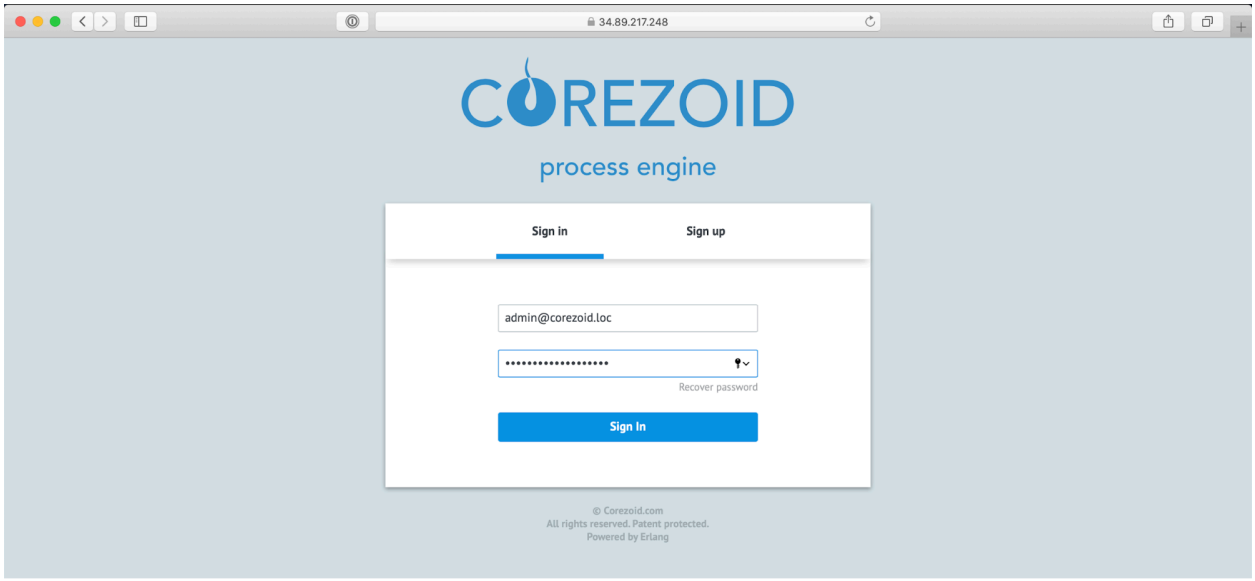


Screenshot 319: **Deployment on GCE.**

Note: When first time logging in, you need to accept the self-signed SSL certificate in your browser. Then, you can log in to the Corezoid web interface using the following credentials:

Login: **admin@corezoid.loc**

Password: **admin111**



Screenshot 320: **Deployment on GCE.**

Useful info:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

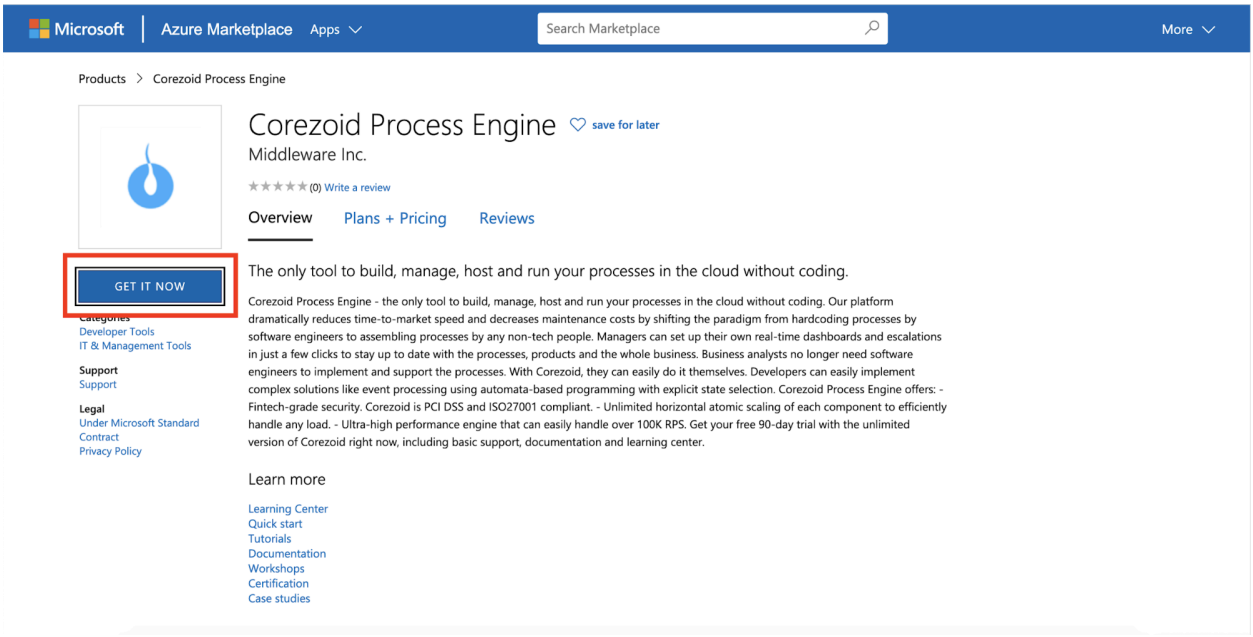
- If you have issues with logging in to the Process, try updating your browser to the latest version or switching to the supported browsers: Chrome, Firefox or Internet Explorer/Edge.
- If you want to use SSH, you may need to add a new login and public SSH Key in the settings of the Corezoid instance installed in the Google Cloud. For more information on SSH Keys management, go to this page.
- Type "help" or "*command* /help" to find all the available commands for managing Corezoid.
- If you have any questions, please do not hesitate to contact us: support@corezoid.com.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

4.4 Deploy Corezoid on Azure Marketplace

To deploy Corezoid on the Azure Marketplace:

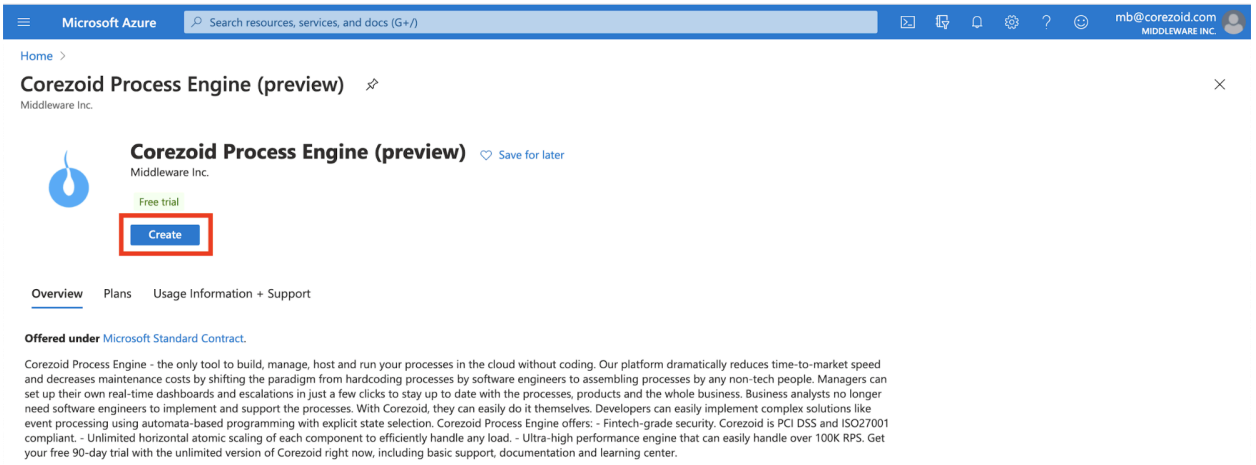
1. On the **Corezoid Process Engine** page, click **Get it now**.



Screenshot 321: **Deployment on Azure Marketplace.**

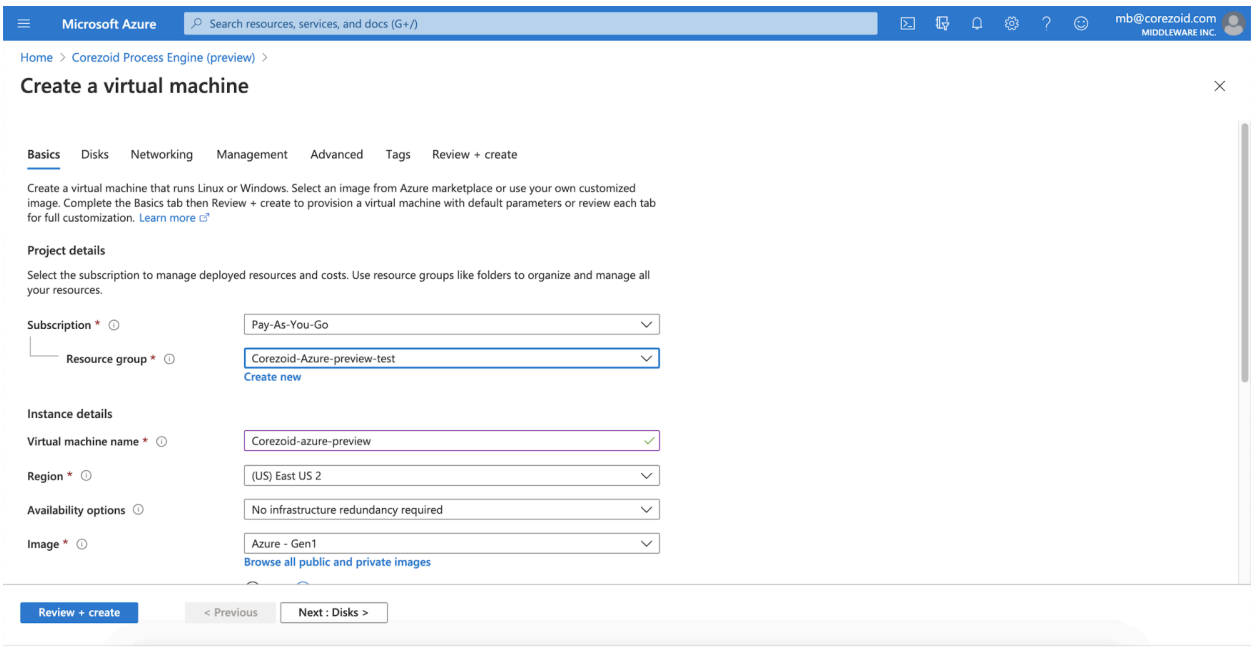
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

2. On the **Corezoid Process Engine (preview)** page, click the **Create**.



Screenshot 322: **Deployment on Azure Marketplace.**

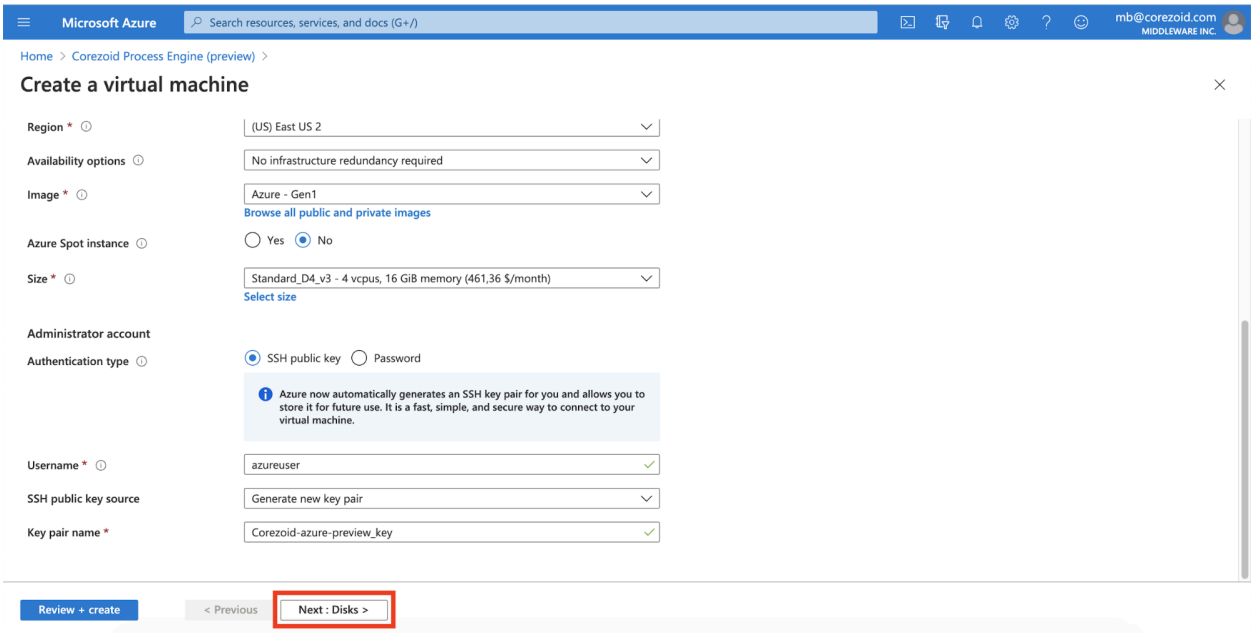
3. Select the existing Resource group or create a new one.



Screenshot 323: **Deployment on Azure Marketplace.**

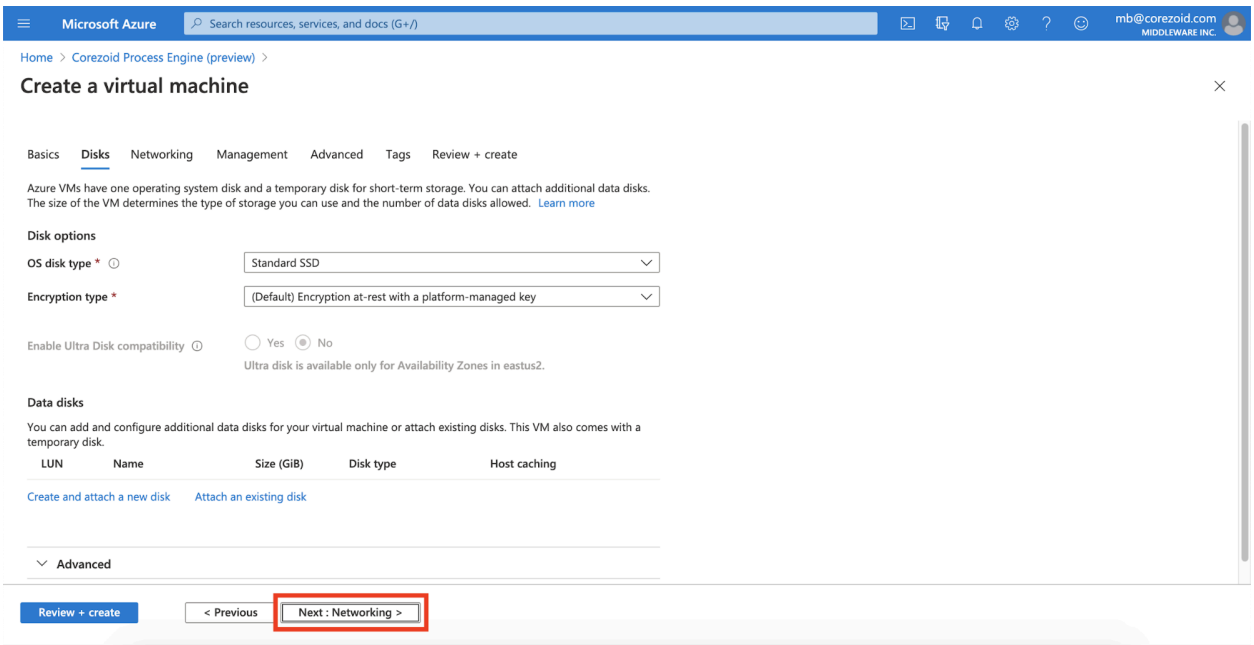
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

4. Create credentials for your SSH public key and click **Next: Disks**.



Screenshot 324: **Deployment on Azure Marketplace.**

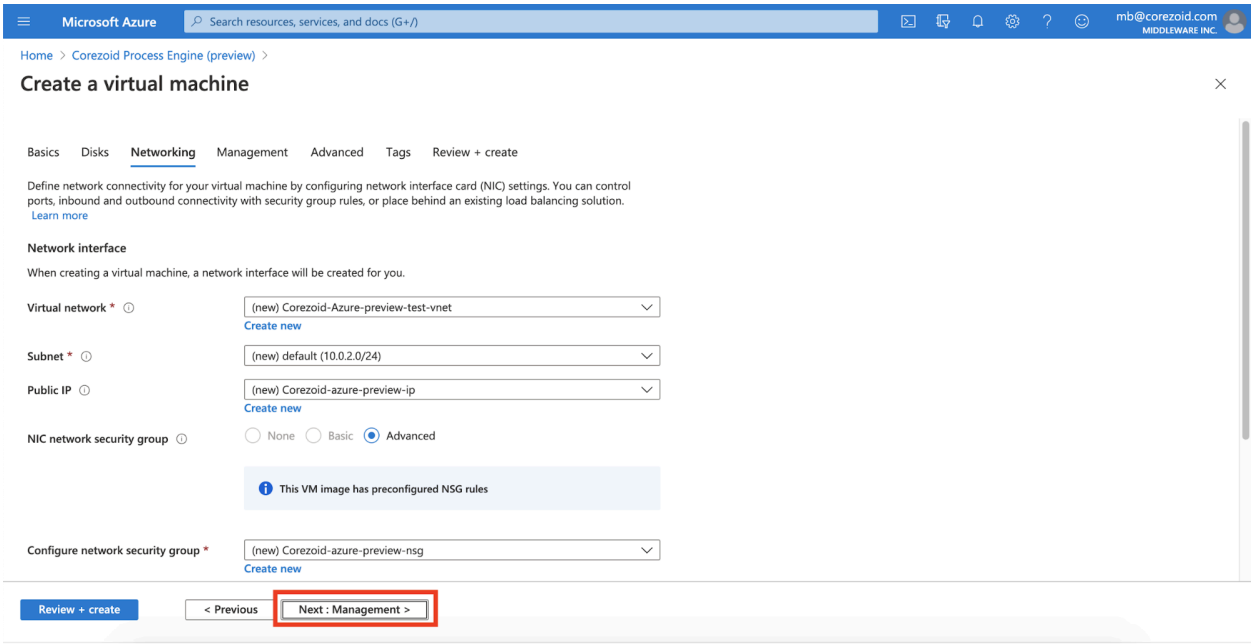
5. Select the required **Os disk type** and click **Next: Networking**.



Screenshot 325: **Deployment on Azure Marketplace.**

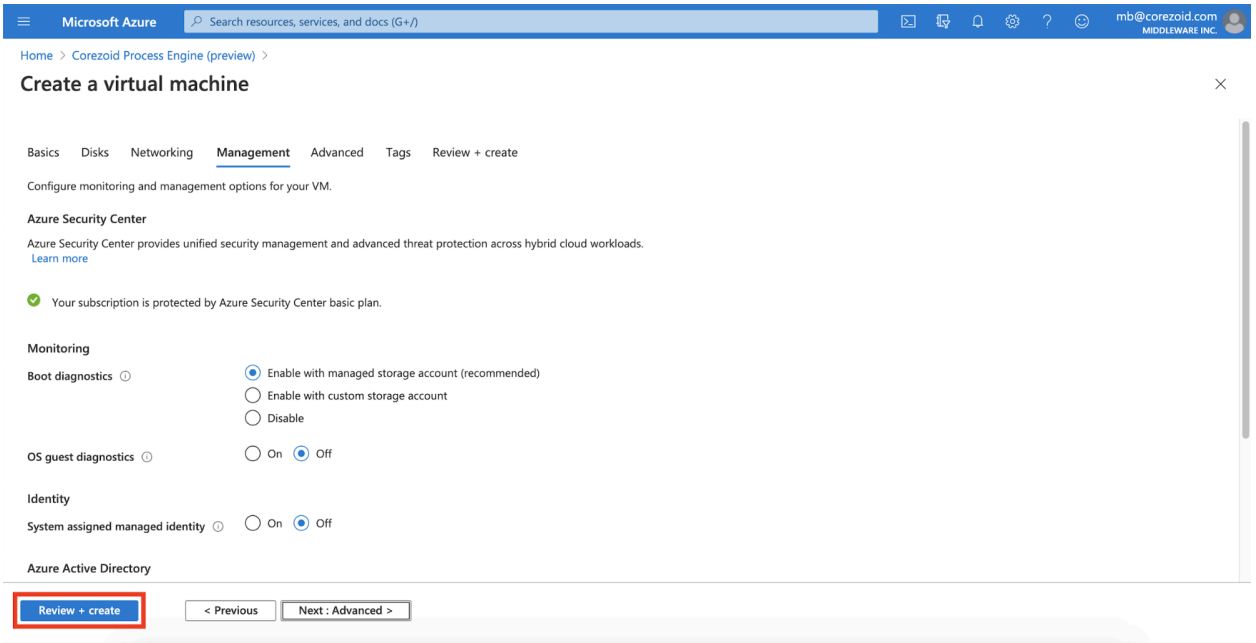
6. Select the existing Virtual Interface or create a new one, and then, click **Next: Management**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 326: **Deployment on Azure Marketplace.**

7. (Optional) Select all the necessary settings or leave them as is, and click **Review + create**.

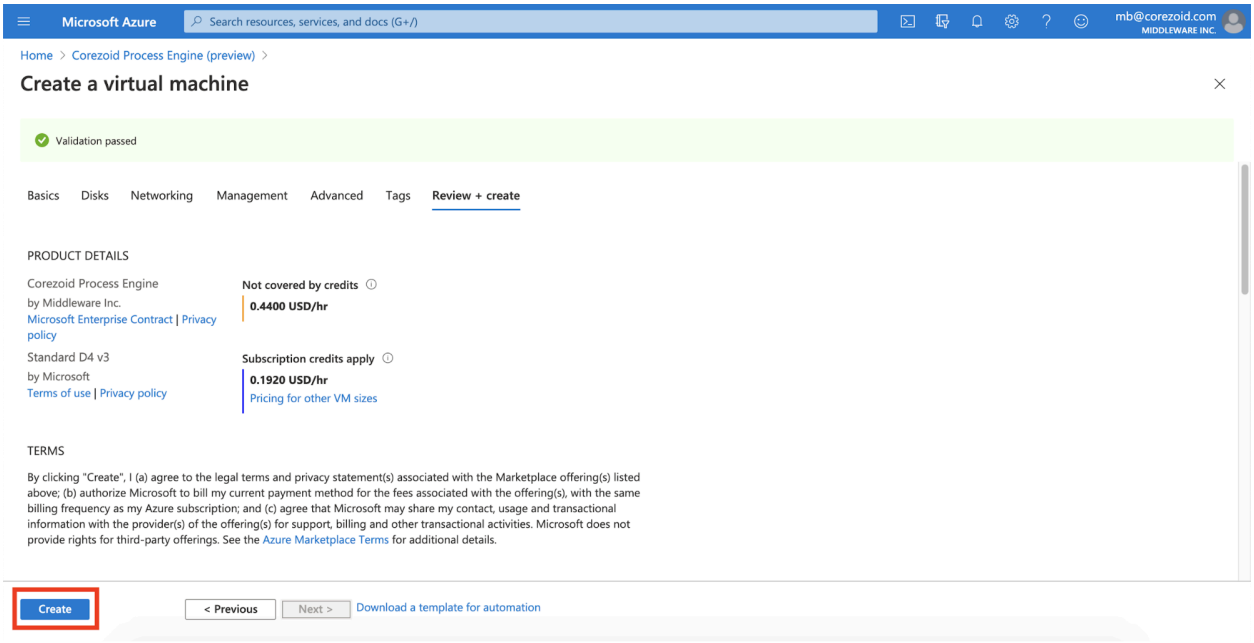


Screenshot 327: **Deployment on Azure Marketplace.**

Note: In the two optional tabs, **Advanced** and **Tags**, add settings if needed, and then click **Review + create**.

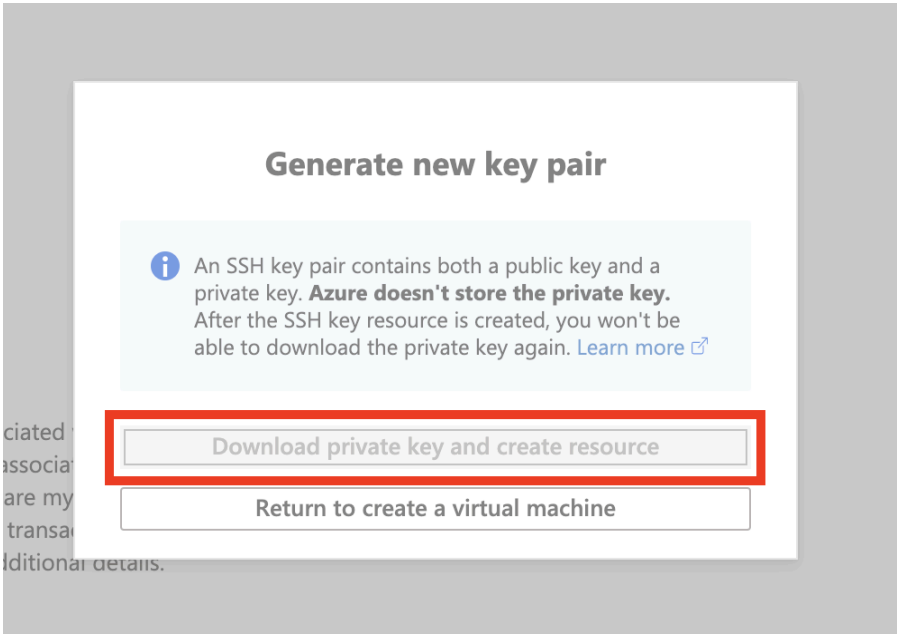
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

8. On the **Review + create** tab, check all the subscription details and click **Create**.



Screenshot 328: **Deployment on Azure Marketplace.**

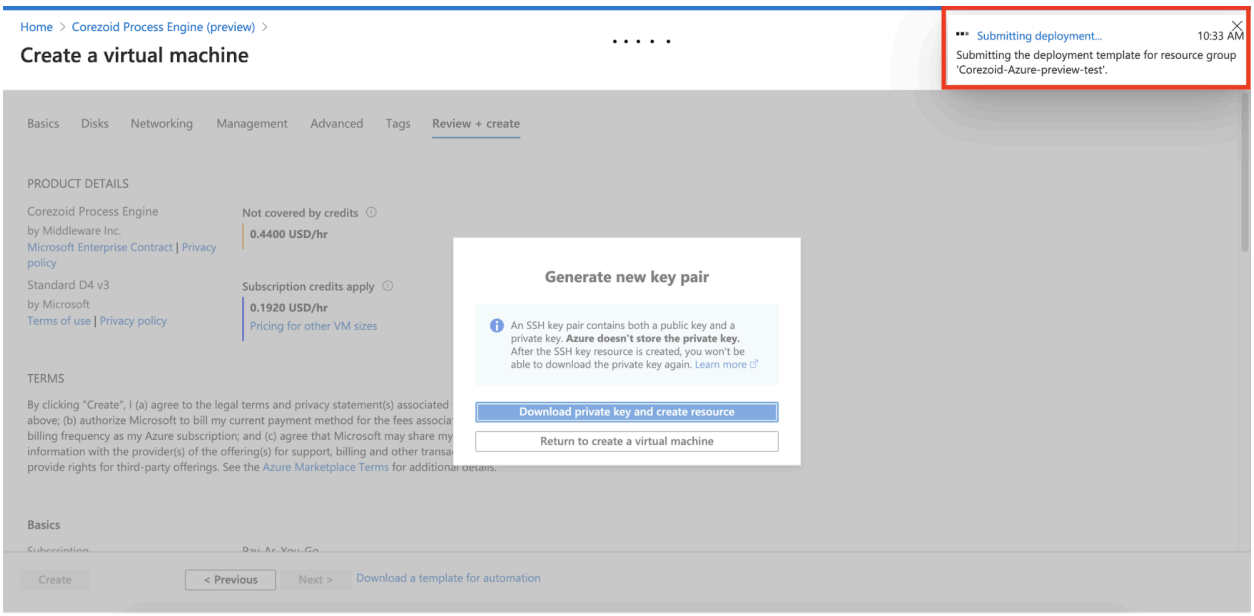
9. Download the generated new SSH key pair to enter your instance via SSH after the deployment.



Screenshot 329: **Deployment on Azure Marketplace.**

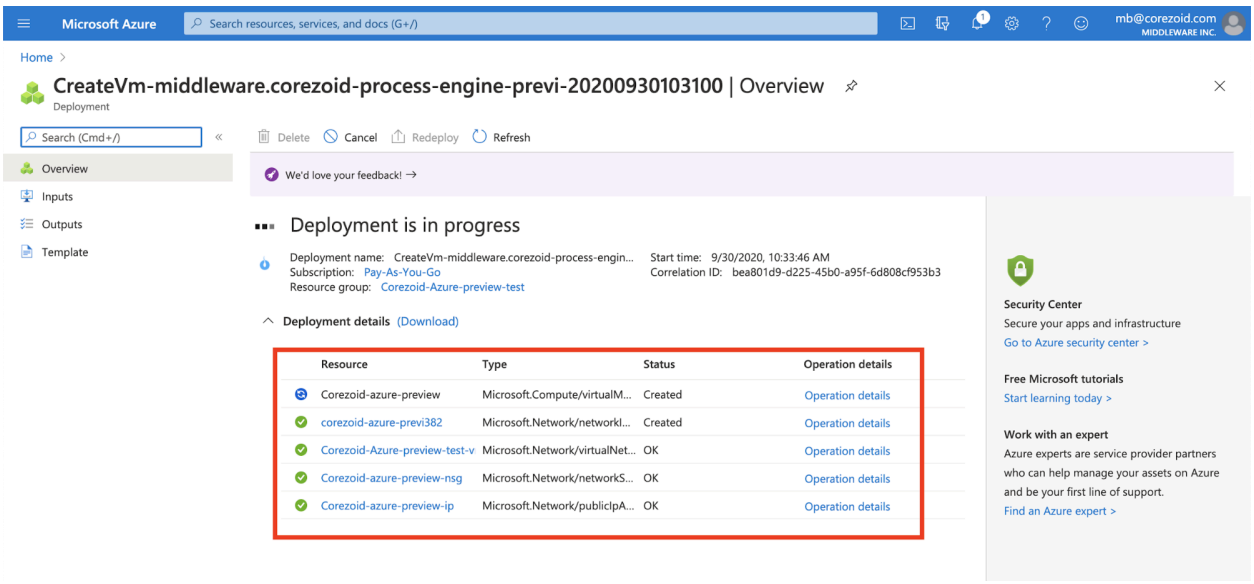
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

10. The deployment process will be started and will take a few minutes.



Screenshot 330: **Deployment on Azure Marketplace.**

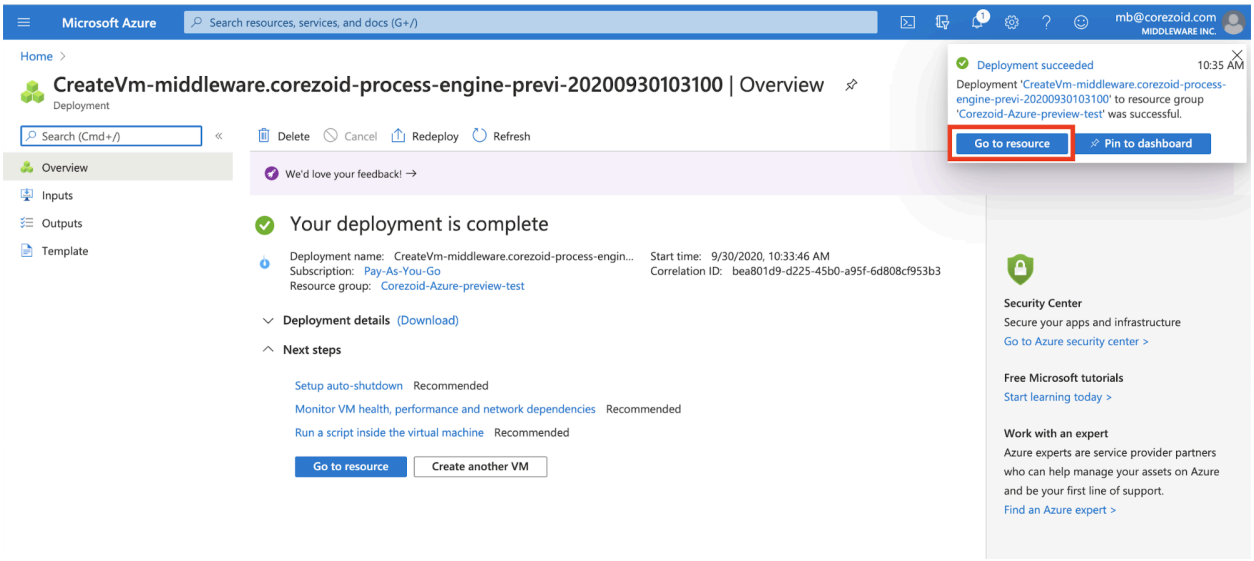
11. Review the deployment status on the screen.



Screenshot 331: **Deployment on Azure Marketplace.**

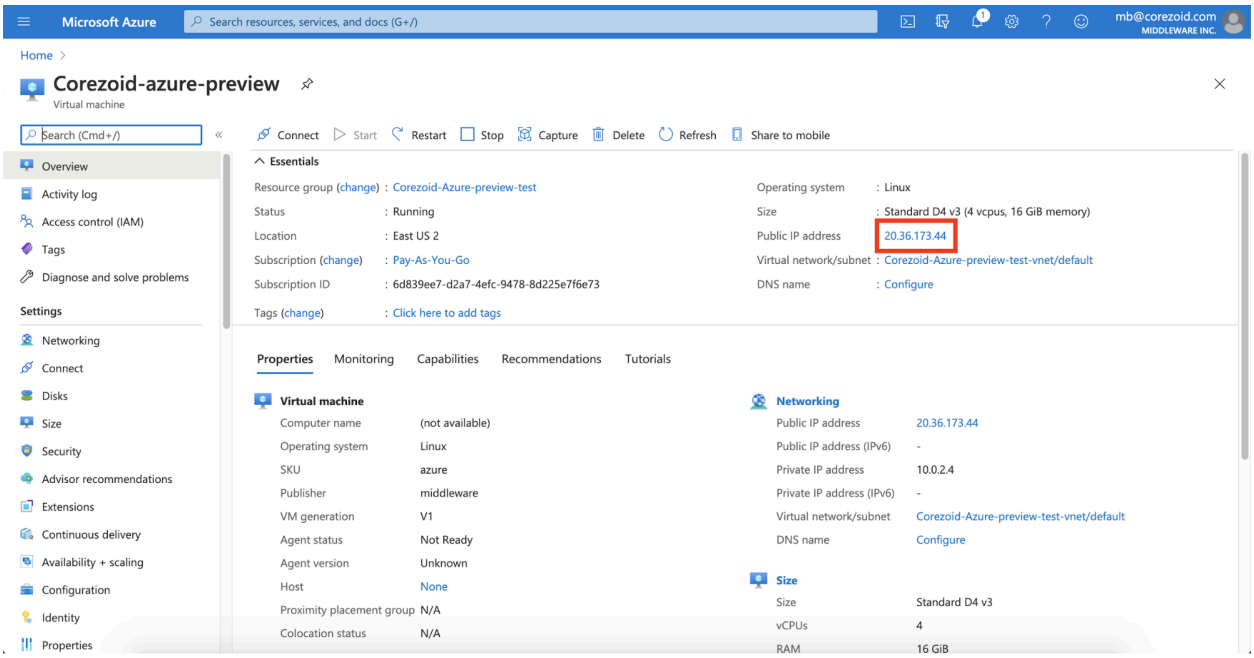
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

12. After the deployment completion, go to the created instance by clicking **Go to resource** in the upper-right corner of the page.



Screenshot 332: **Deployment on Azure Marketplace.**

13. On the resource screen, view all the Corezoid instance details and enter it via the public IP.



Screenshot 333: **Deployment on Azure Marketplace.**

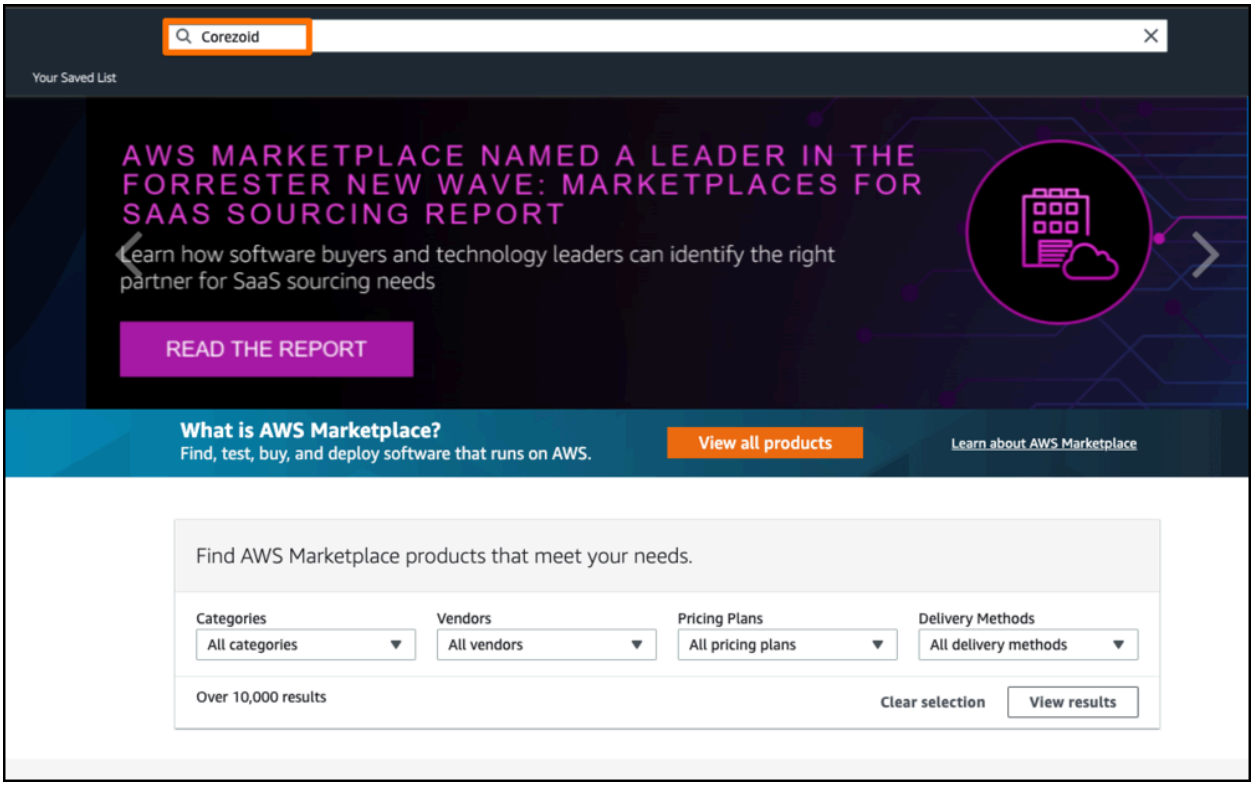
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

4.5 Deploy Corezoid on AWS Marketplace

To deploy Corezoid on the AWS Marketplace:

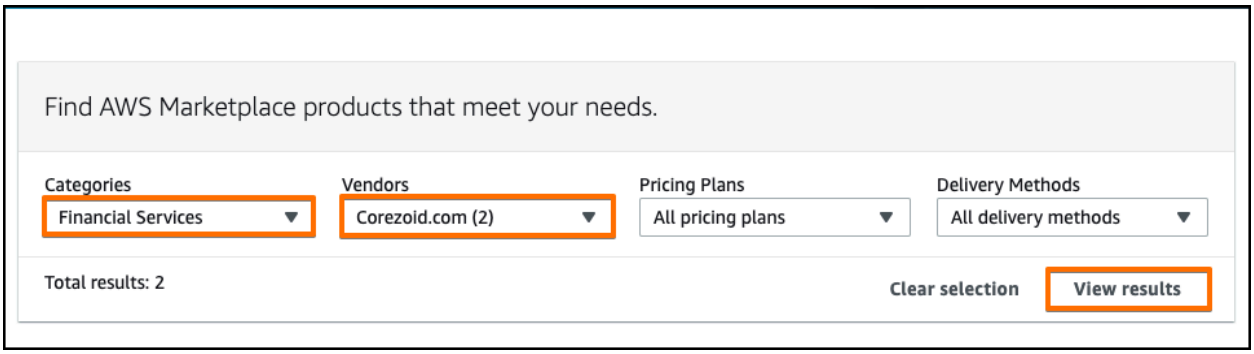
1. Log in to your AWS account and go to the AWS Marketplace.
2. Choose Corezoid to deploy on your instance: go to the aws.com page and type Corezoid in the **Search** field.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 334: **Deployment on AWS Marketplace.**

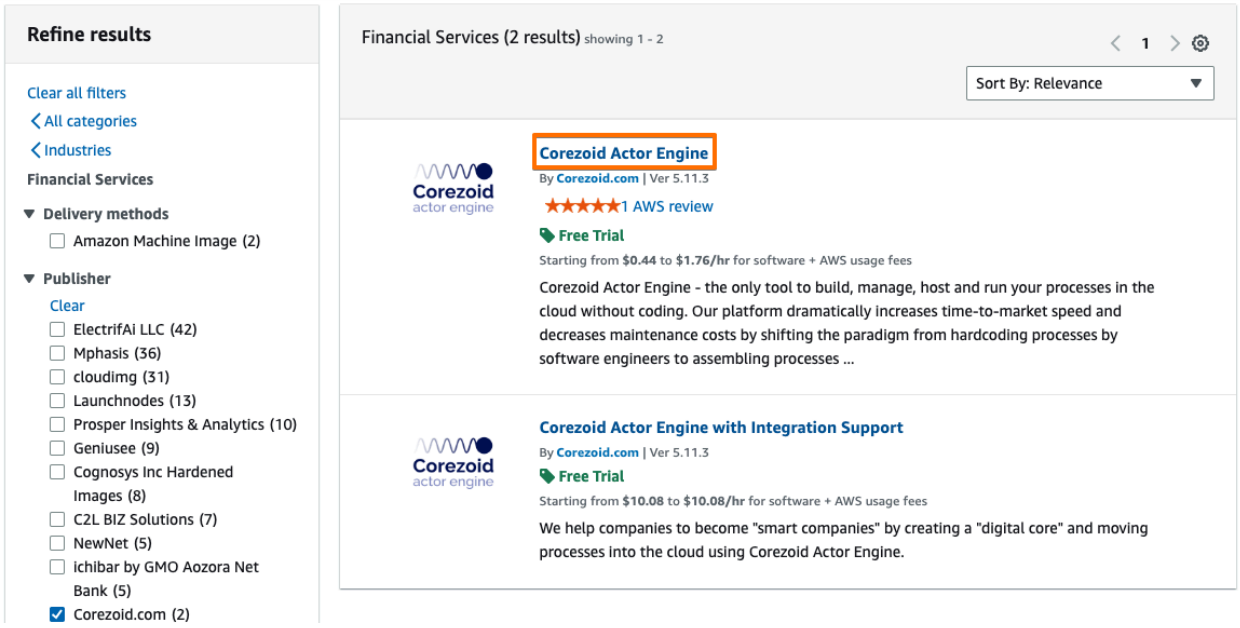
Note: you can browse through the categories (Categories->Financial Services, Vendors->Corezoid.com) to display the list of Corezoid options.



Screenshot 335: **Deployment on AWS Marketplace.**

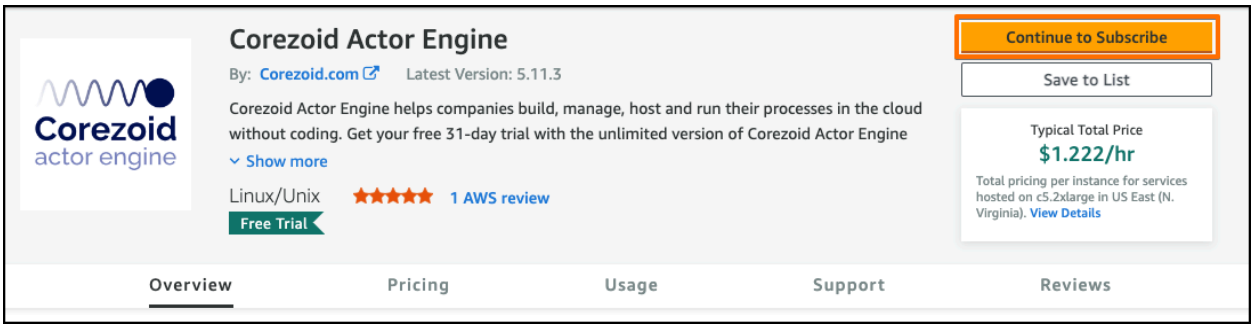
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3. Click **Corezoid Actor Engine**.



Screenshot 336: **Deployment on AWS Marketplace.**

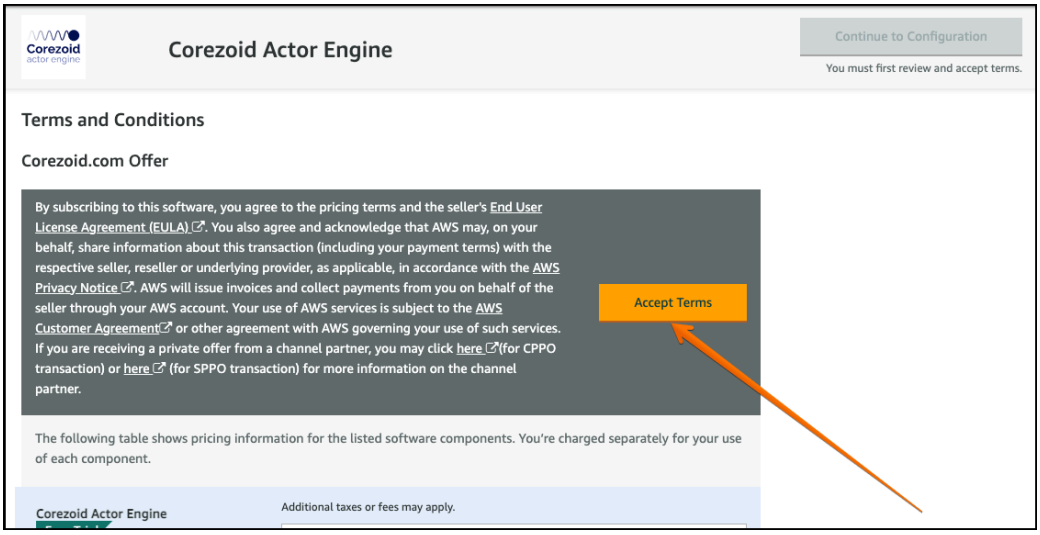
4. Click **Continue to Subscribe** to start the subscription process.



Screenshot 337: **Deployment on AWS Marketplace.**

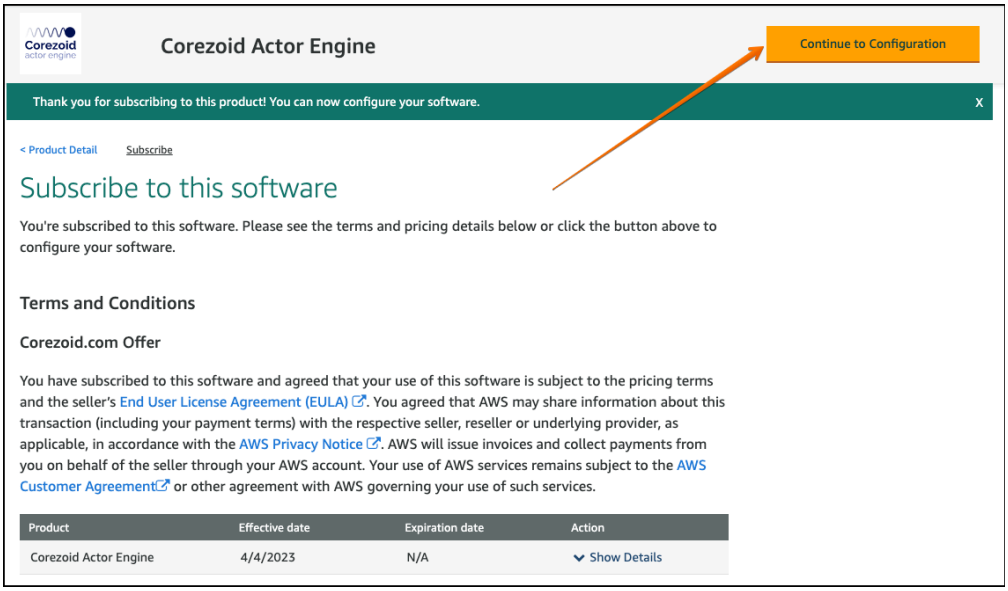
5. Review the terms and conditions of the subscription and click **Accept Terms** to proceed.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 338: **Deployment on AWS Marketplace.**

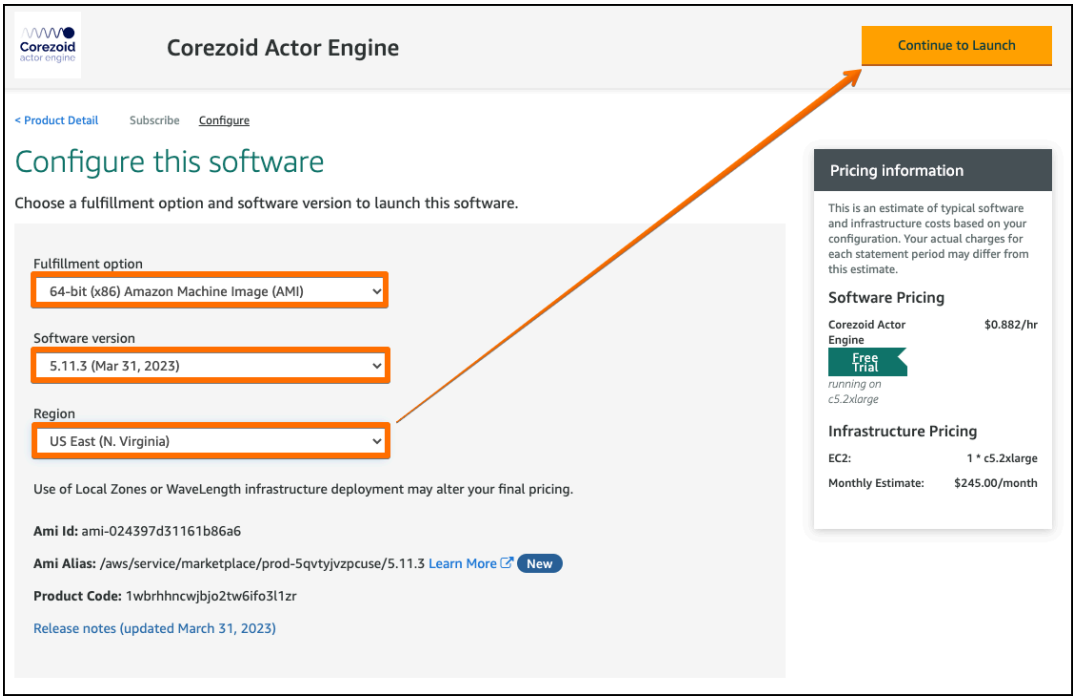
6. Review the terms and conditions of the offer (product, effective and expiration date), and then click **Continue to Configuration**.



Screenshot 339: **Deployment on AWS Marketplace.**

7. Configure the launch settings, such as the fulfillment option, software version, and the region where you want to deploy your instance, and then click **Continue to Launch**.

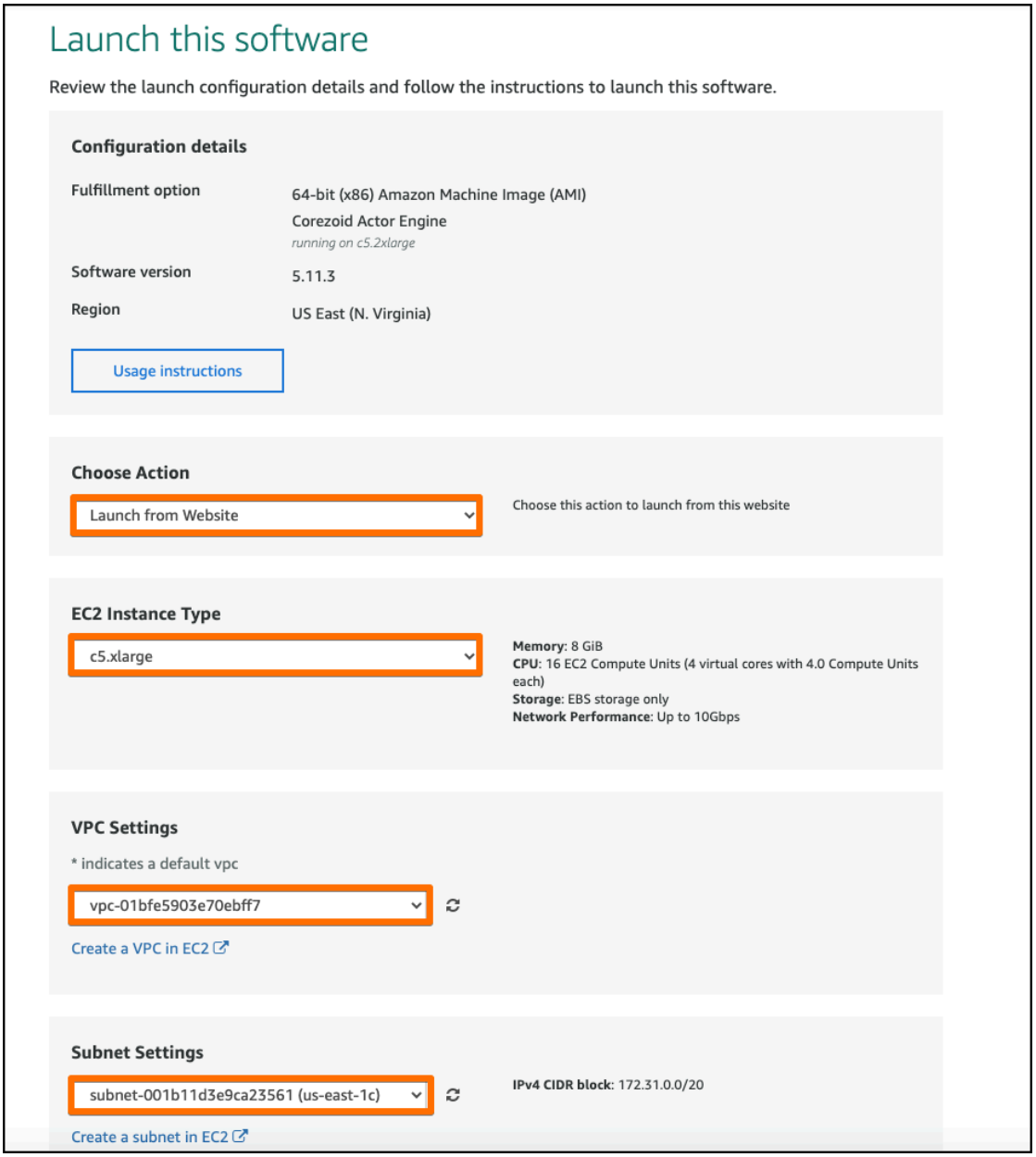
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 340: **Deployment on AWS Marketplace.**

8. Before launching your Corezoid instance, configure its settings: launch action type, EC2 instance type, VPC settings, subnet settings, and security group settings.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

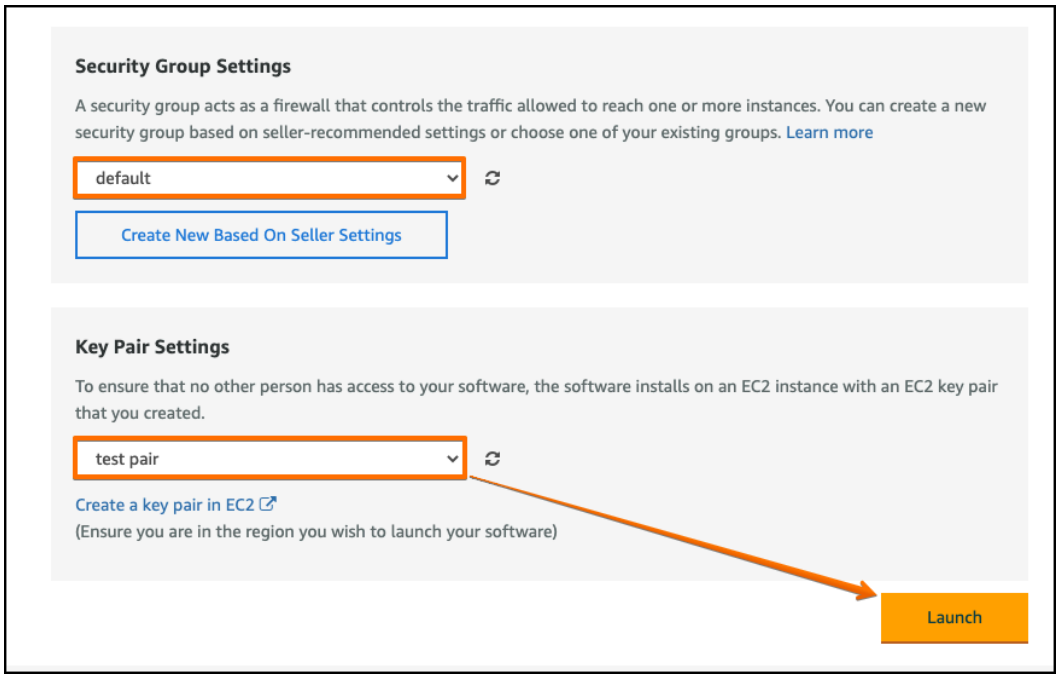


Screenshot 341: **Deployment on AWS Marketplace.**

9. In the **Security Group Settings** dialog:
    a. Leave the security group selected by default for newly created instances to configure it after the launch (see AWS documentation for more details).
    b. Select the key pair you want to use for accessing your instance.
    c. Click **Launch**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

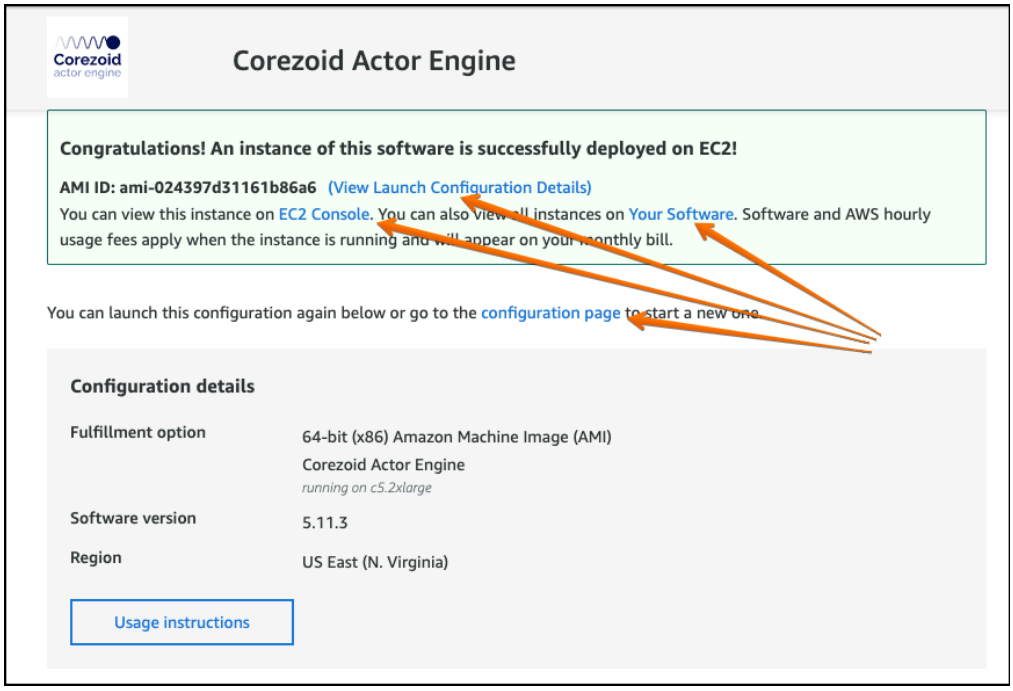
10. **Note:** There are no key pairs created by default for new instances on AWS. Thus, you need to create a key pair for your instance (see AWS documentation for more details).



Screenshot 342: **Deployment on AWS Marketplace.**

11. You've successfully deployed and launched your Corezoid instance on AWS. Before it's ready to be used you need to complete its configuration.
    You can customize Corezoid based on your requirements. On the **Corezoid Actor Engine** dialog, you can:
    a. View launch configuration details
    b. View and configure your instance in the EC2 console (click to proceed with the configuration of your instance)
    c. Launch the instance configuration again to start a new instance on AWS.

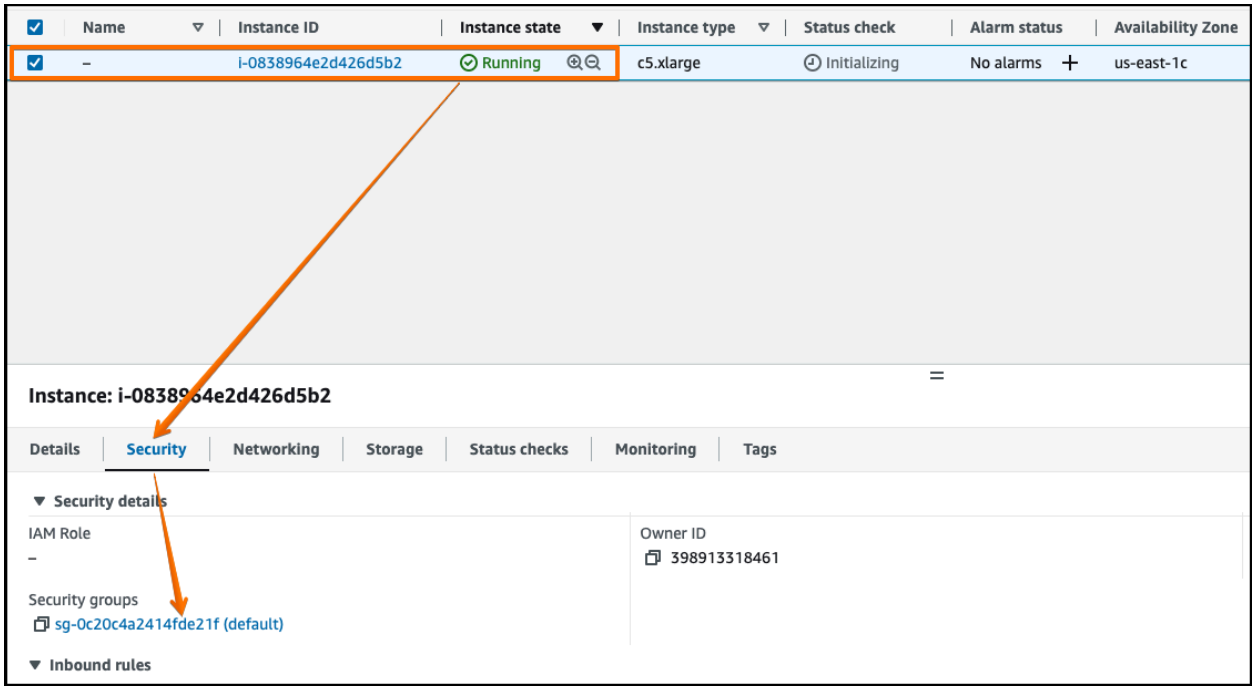
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 343: **Deployment on AWS Marketplace.**

12. To complete the instance configuration:
    a. Open your instance in the **AWS** console by clicking **EC2** console (or open AWS console-Instances).
    b. Select the needed instance and check whether its state is Running.
    c. Open the **Security** tab and click on the default security group to add the needed outbound security rules for the default group and open 22, 80, and 443 ports (For more information, go to AWS documentation).

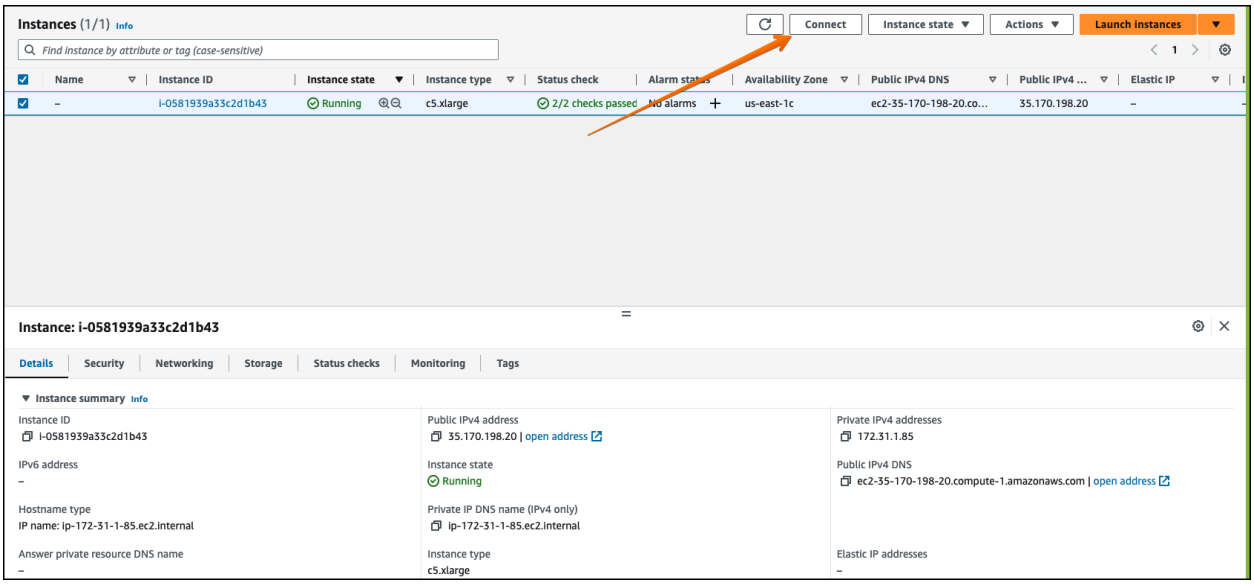
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 344: **Deployment on AWS Marketplace.**

13. To configure access to your Corezoid instance from the website, the upper-right corner, click **Connect**.

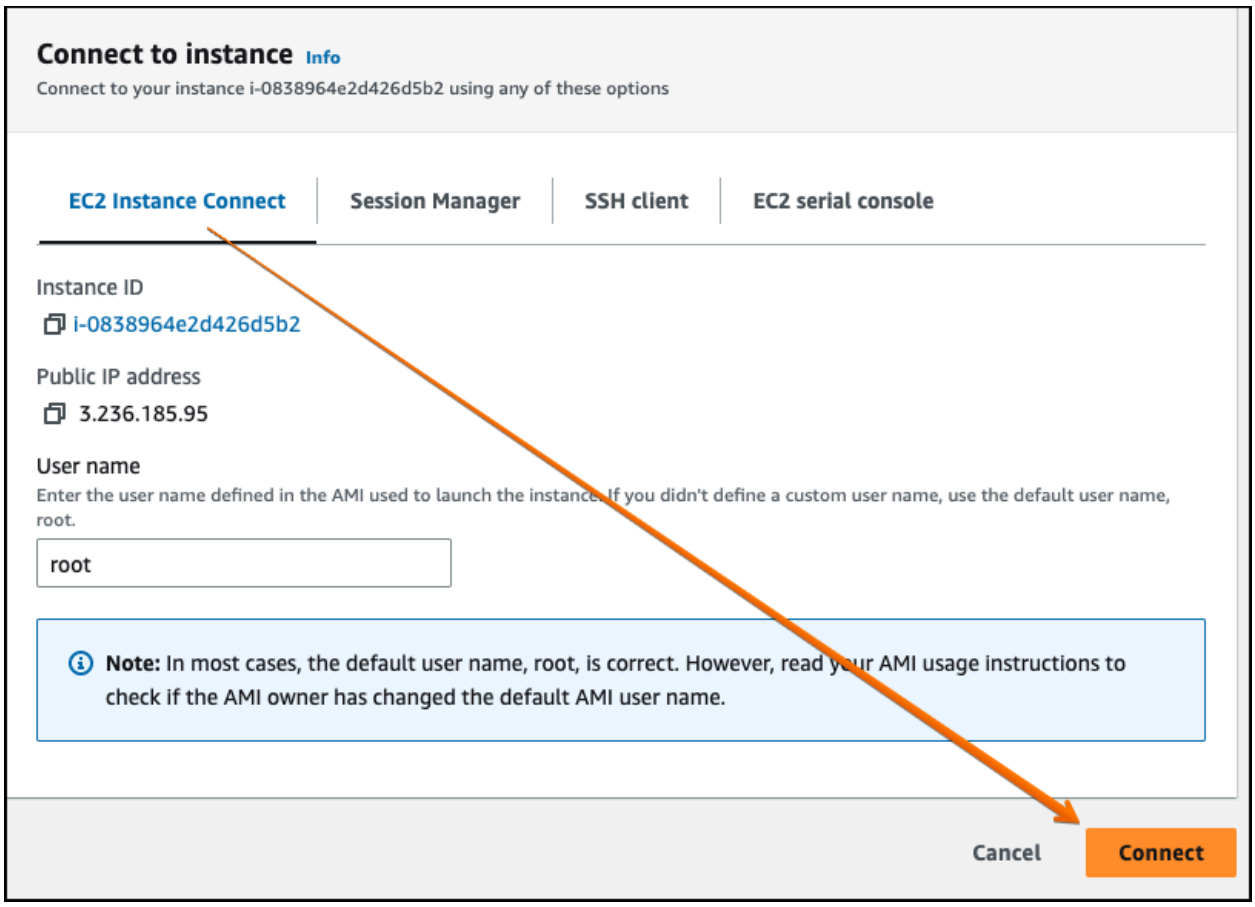
    Note: If you want to configure access to your instance from a local device, skip this step and proceed to the next one.



Screenshot 345: **Deployment on AWS Marketplace.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

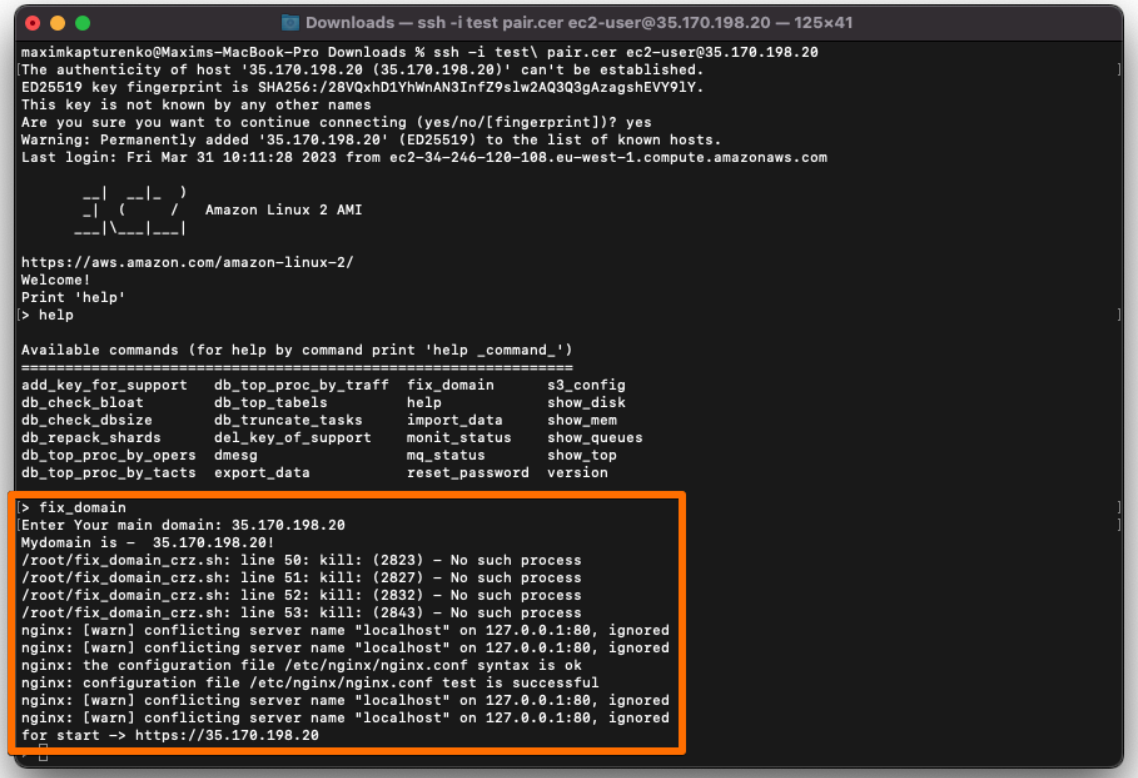
14. On the **EC2 Instance Connect** tab, click **Connect**.



Screenshot 346: **Deployment on AWS Marketplace.**

15. To configure access to your Corezoid instance from a local device, add your key pair (For more information, go to AWS documentation for more details).
16. To complete your instance configuration, have the **fix_domain** command completed successfully in your terminal. See the example of a successful command run result below.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 347: **Deployment on AWS Marketplace.**

17. Before logging in to Corezoid for the first time, accept the self-signed SSL certificate in your browser.
18. Log in to Corezoid using **admin@corezoid.loc** as login and **ami-id** of your Corezoid instance (For more information, go to Usage instructions). You can view and copy an ami-id for your instance in your AWS console.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Screenshot 348: **Deployment on AWS Marketplace.**

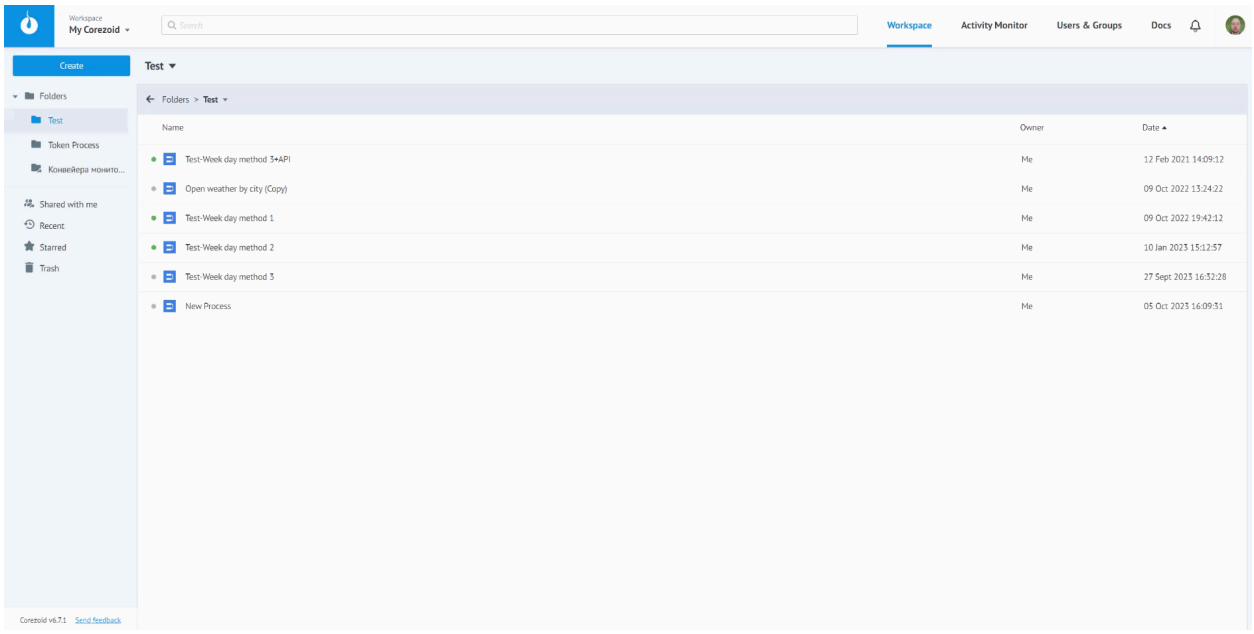
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

5. Corezoid Processes Guide

5.1 Building Processes

Corezoid Processes are constructed by adding and connecting nodes. To build a Process:

1. In Corezoid, open the **Workspace** tab.
2. Click Create and select Process.
3. Enter the Process name and click OK.
4. In the Process, add nodes by dragging them into the canvas from the toolbar.
5. Connect nodes by dragging lines between them to establish transitions.
6. Configure each node's properties (e.g., parameters, conditions, payload).

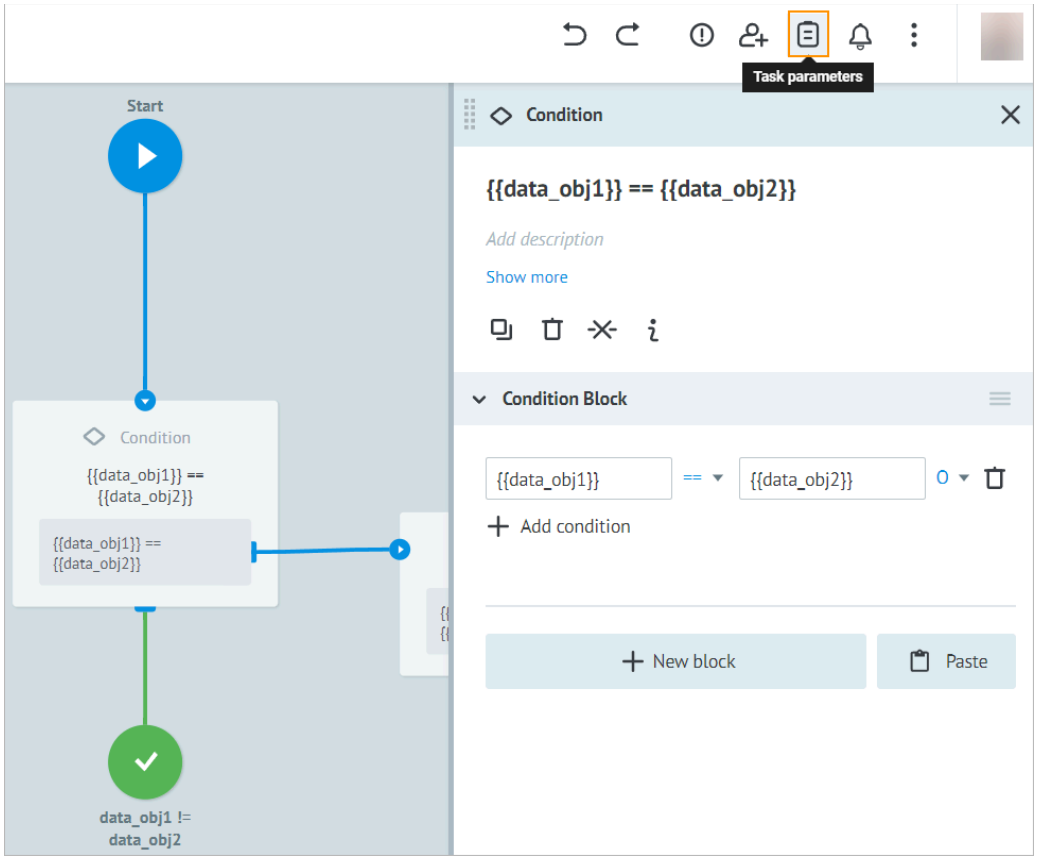


Screenshot 349: **Creating a Process.**

Working with Task Parameters

To access task parameters, open a Process and click the icon in the upper right corner:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



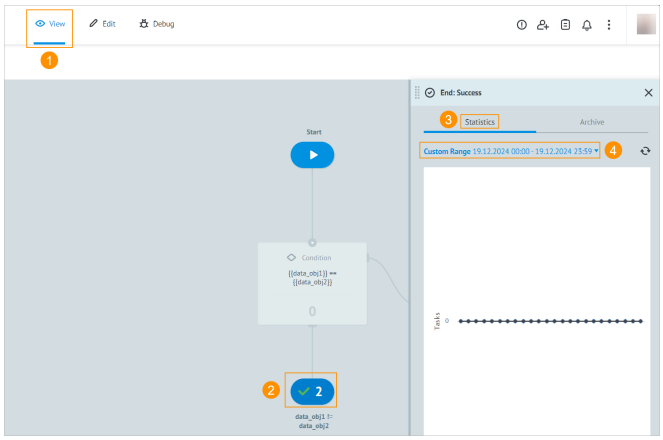
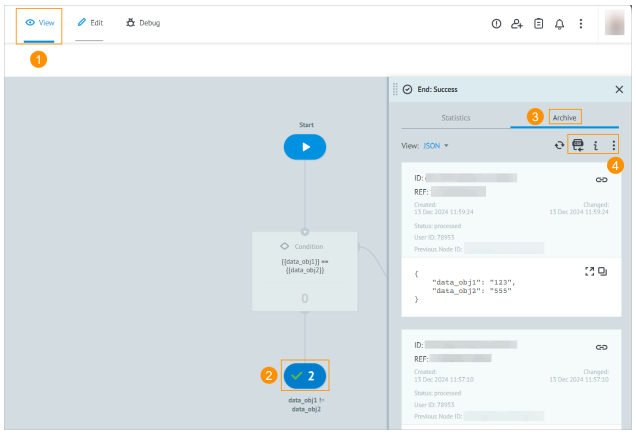
Screenshot 350: **Task parameters icon in a Process (Process Editor).**

Task Management Features

- **Task Viewing:** Allows you to monitor tasks as they flow through a Process, displayed in either a **JSON** or **table** format.
  How to find:

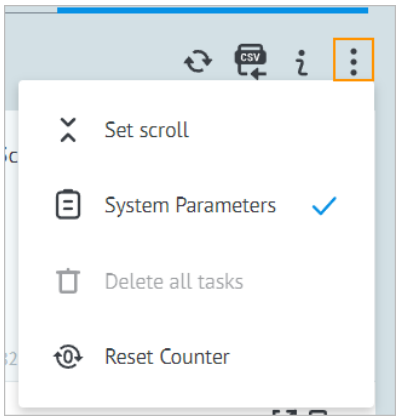
1. Open a Process in **Editor** and click the **View** tab.
2. Click the final node where you want to view a task.
3. Click the **Statistics** tab to view the task processing graph showing the total number of processed tasks, current queue size, and execution times, or the **Archive** tab to view the task content and parameters. Task info is updated in real-time.
4. Use the range selection on the **Statistics** tab to view the chart for a specific period. On the **Archive** tab, use the menu icons to export the task to a CSV file, view the selected node info, set scroll, make tasks view include or exclude System Parameters, and reset the node input tasks counter.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Task view-Statistics tab	Task view-Archive tab
	

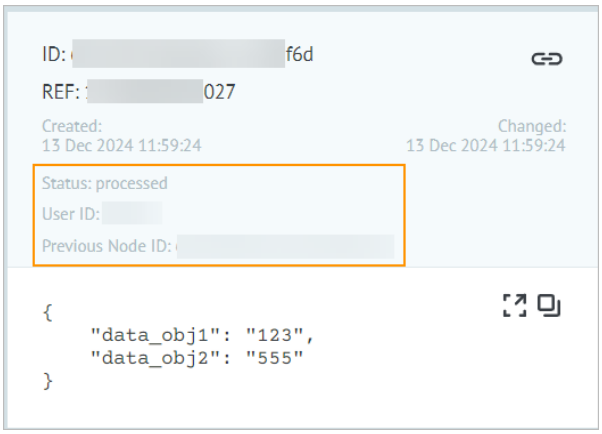
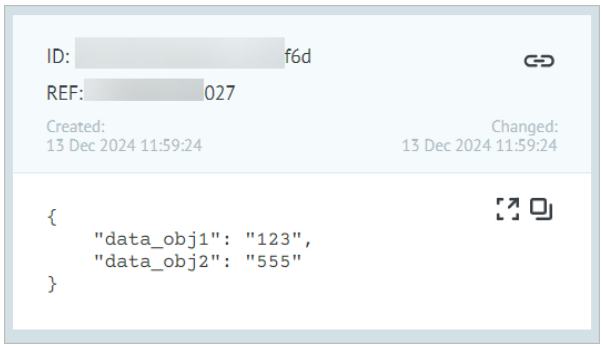
Screenshot 351: **Task parameters view: Statistics and Archive tabs.**

To include or exclude System Parameters (task processing status, user ID of the task owner, and previous node ID) in the task info, on the **Archive** tab, click the three-dot menu and select or deselect **System Parameters**.



Screenshot 352: **Archive tab three-dot menu.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

System Parameters on	System Parameters off
	

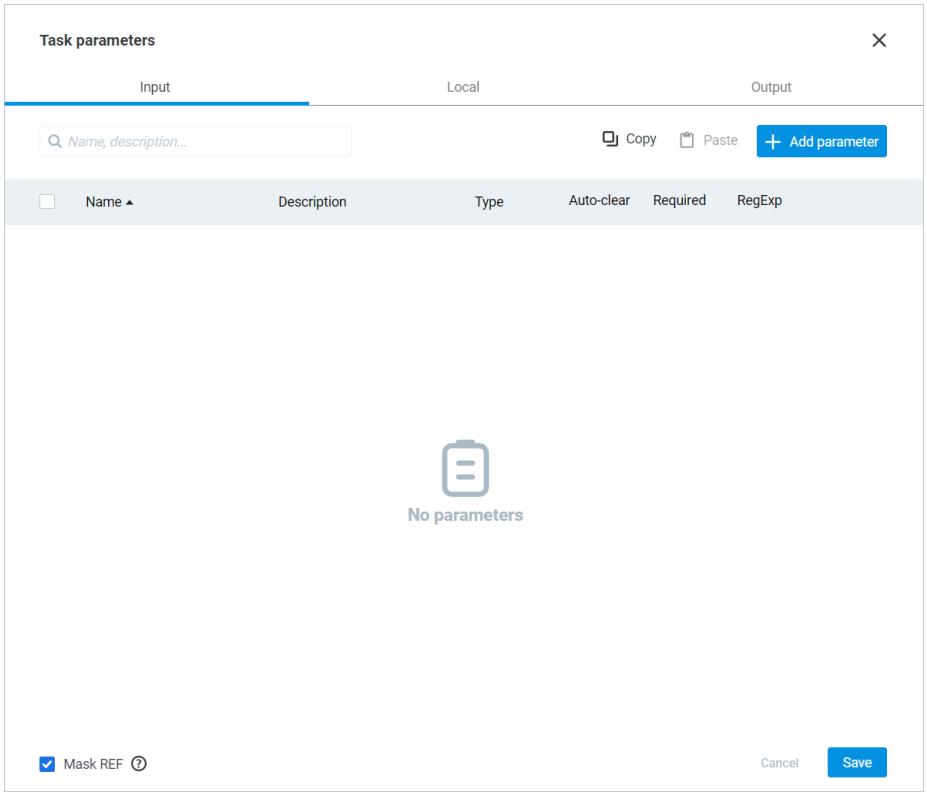
Screenshot 353: **System parameters.**

- **Debugging:** Helps identify and resolve issues by pinpointing tasks that failed at specific nodes.
  How to find:
  1. Open a Process and click the **View** tab.
  2. Click the final node where you want to view a failed task.
  3. Click the **Archive** tab to view the task error info.

For more details on debugging, see Debugging a Process.

You can configure task parameters in the window:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 354: **Input tab.**

Task parameters define the structure and content of data flowing through the Process:

1. **Input Parameters:**
    - Received by the **Start Node** at Process initiation.
    - Example: `{"user_id": 12345, "email": "user@example.com"}`.
2. **Local Parameters:**
    - Temporary variables used within the Process for calculations or intermediate logic.
3. **Output Parameters:**
    - Generated by the Process and available to external systems via the **Reply Node** or API.

Best Practices:

- Limit active tasks to 100,000 per Process for optimal performance.
- Use unique parameter names across all Processes to avoid conflicts.

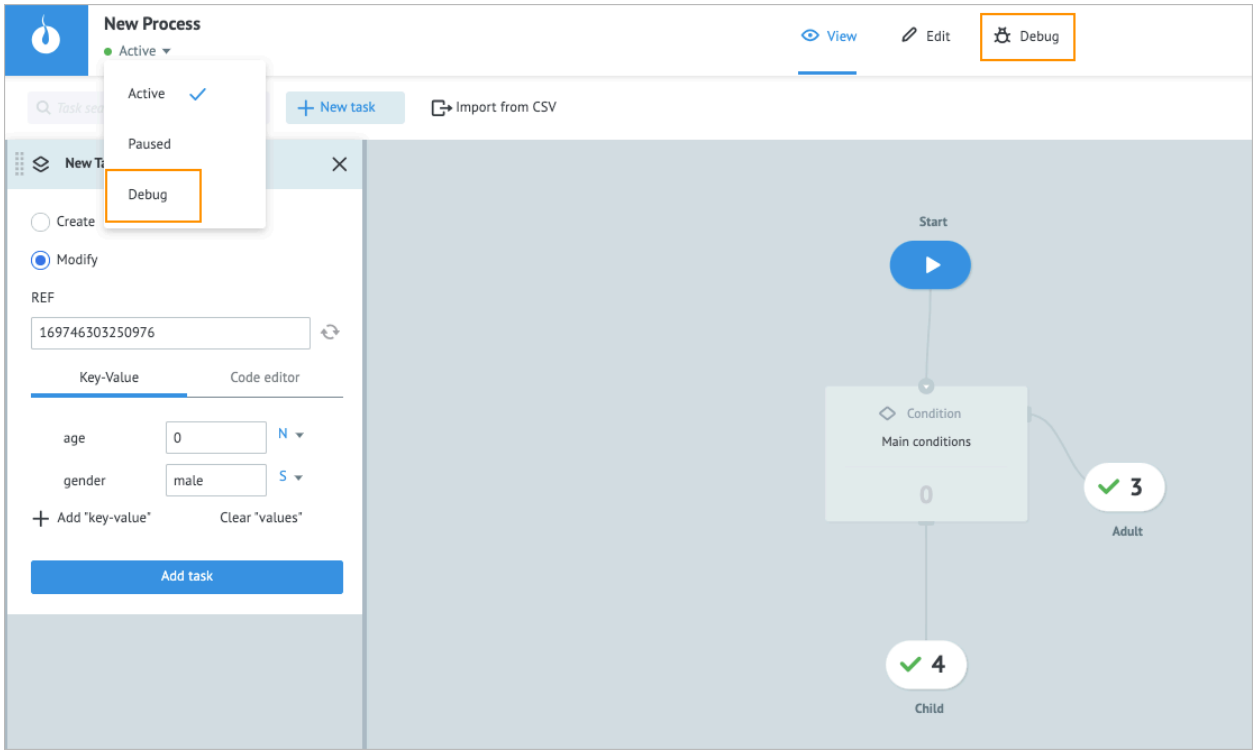
For more details, see Task parameters.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

5.2 Debugging a Process

You can debug a Process to see the details of each stage and to make sure all the internal stages work properly. To do so:

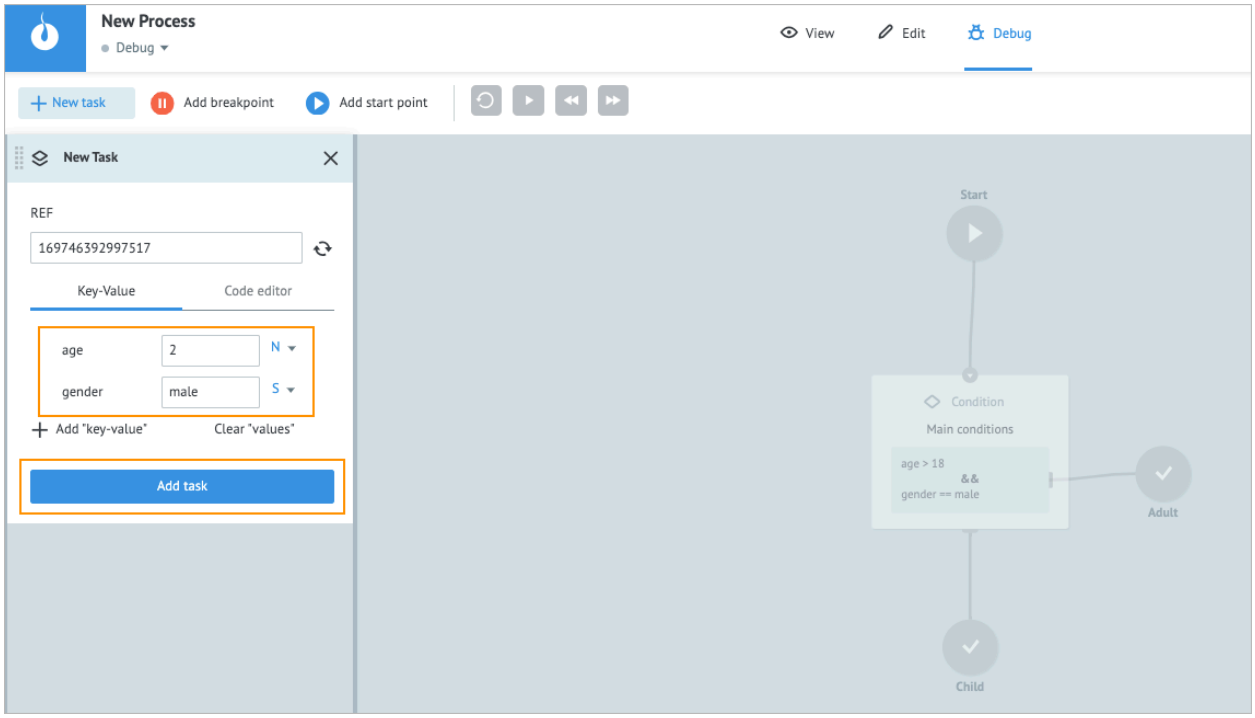
1. Open your Process, click the expand icon next to the status name, from the dropdown list, select **Debug**, and then on the upper panel, click **Debug**.



Screenshot 355: **Process debug.**

2. In the **New Task** dialog of the **Debug** mode, enter task parameters, and then click **Add task**.

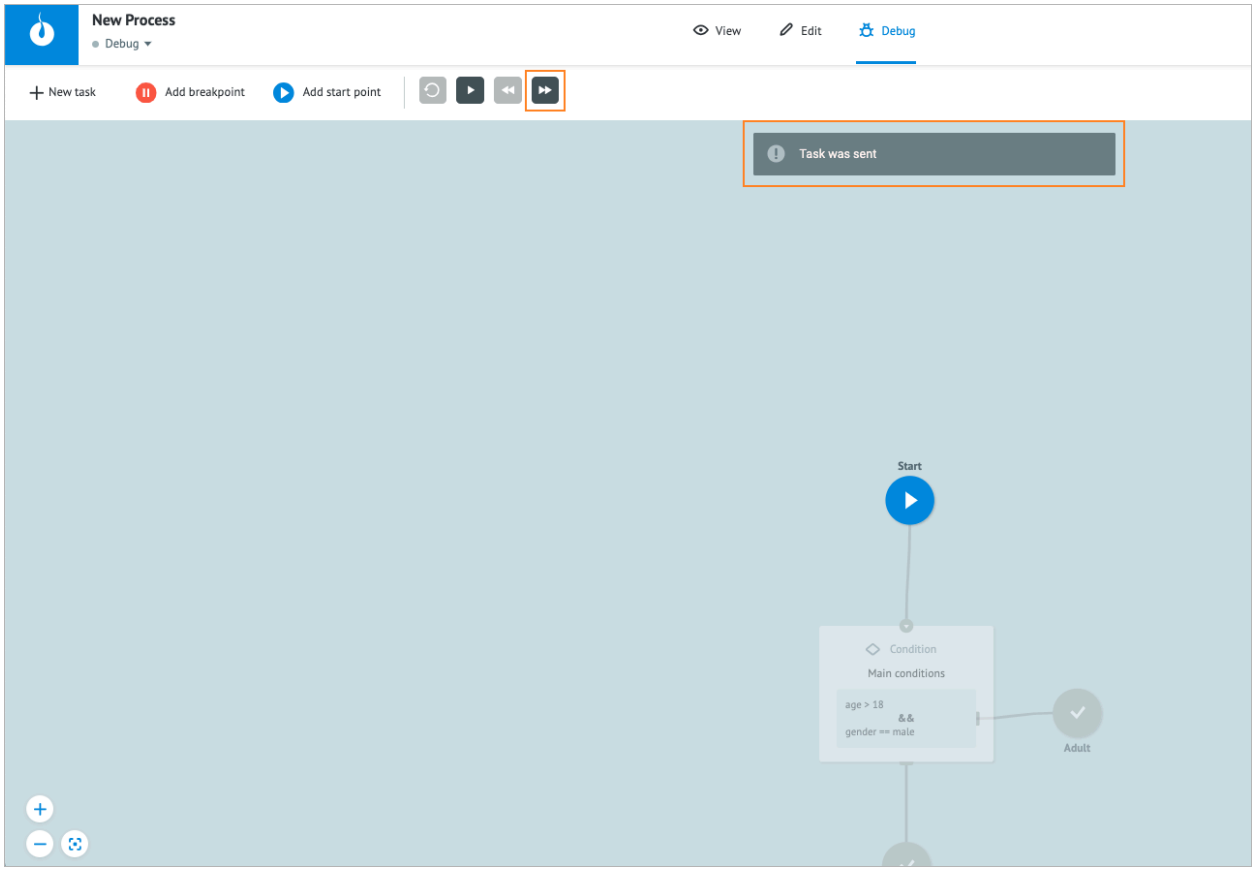
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 356: **Process debug-add task.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

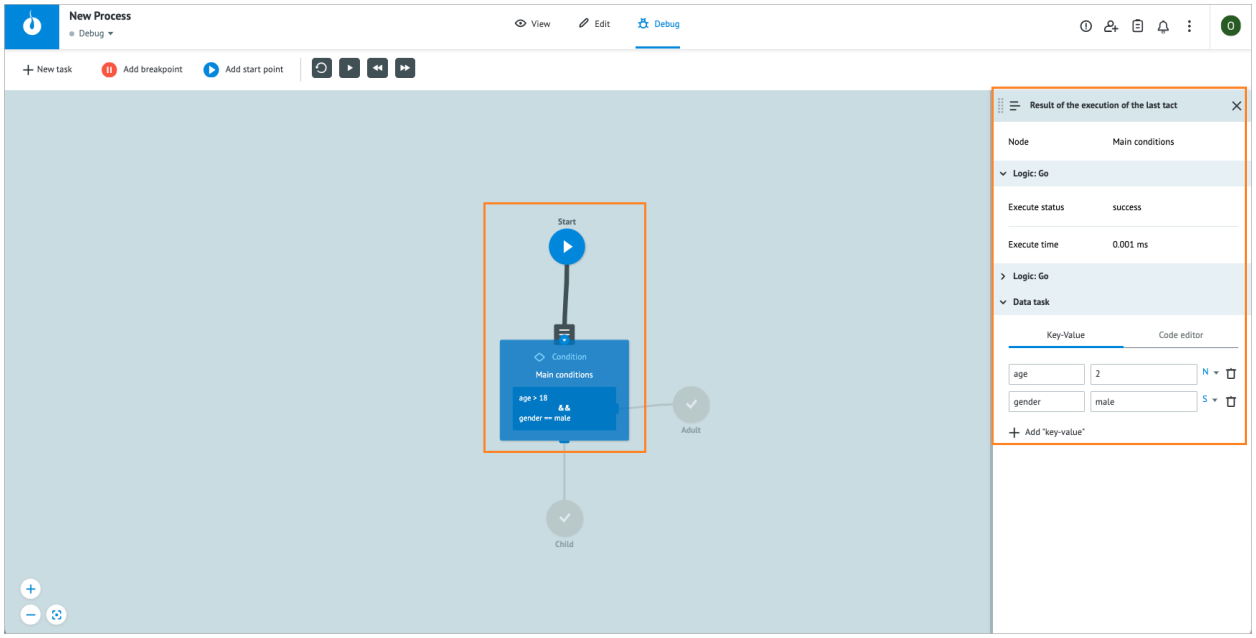
3. After the message “Task was sent” appears, click the **Forward** button .



Screenshot 357: **Task forward button.**

You can see the first stage of your task in the Process, and the details appear on the **Result of the execution of the last tact** panel.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 358: **Result of the execution of the last tact.**

Task parameters can be edited directly in the debugging process.

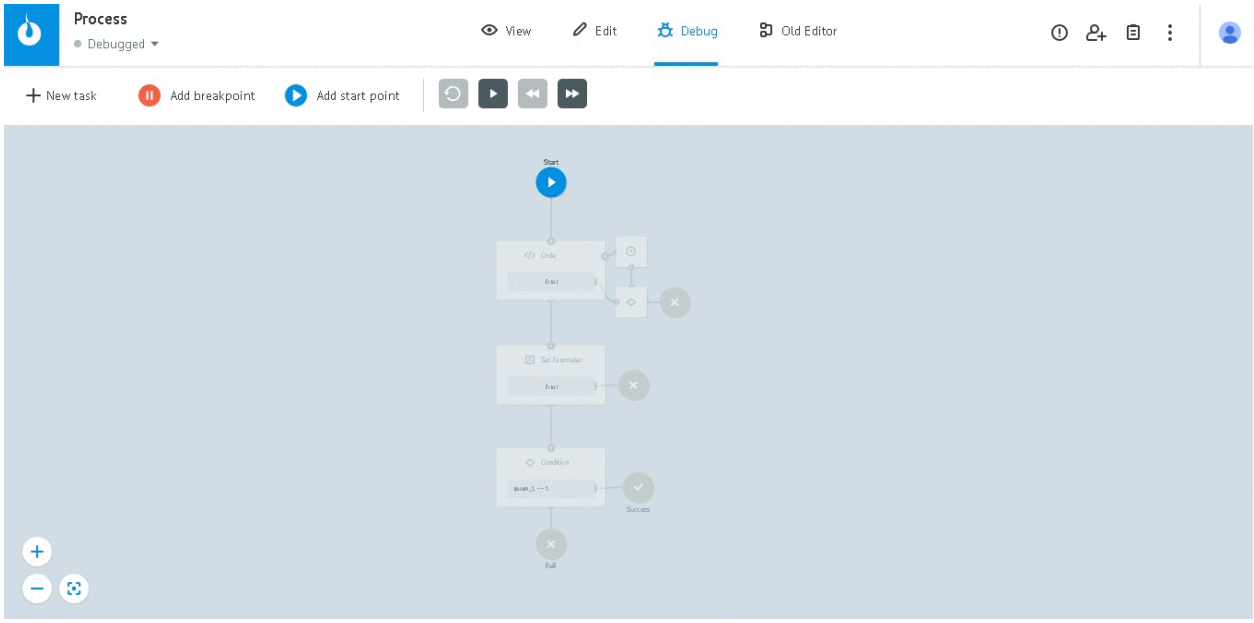
Every time you click the **Forward** button , the next stage details appear on the right panel, and you can check if it works properly step by step. You can also use other buttons on the upper panel:

- **Repeat** button: To repeat the needed stage.
- **Play** button: To play the whole Process.
- **Back** button: To go one step back.

You can modify the Process debugging by adding:

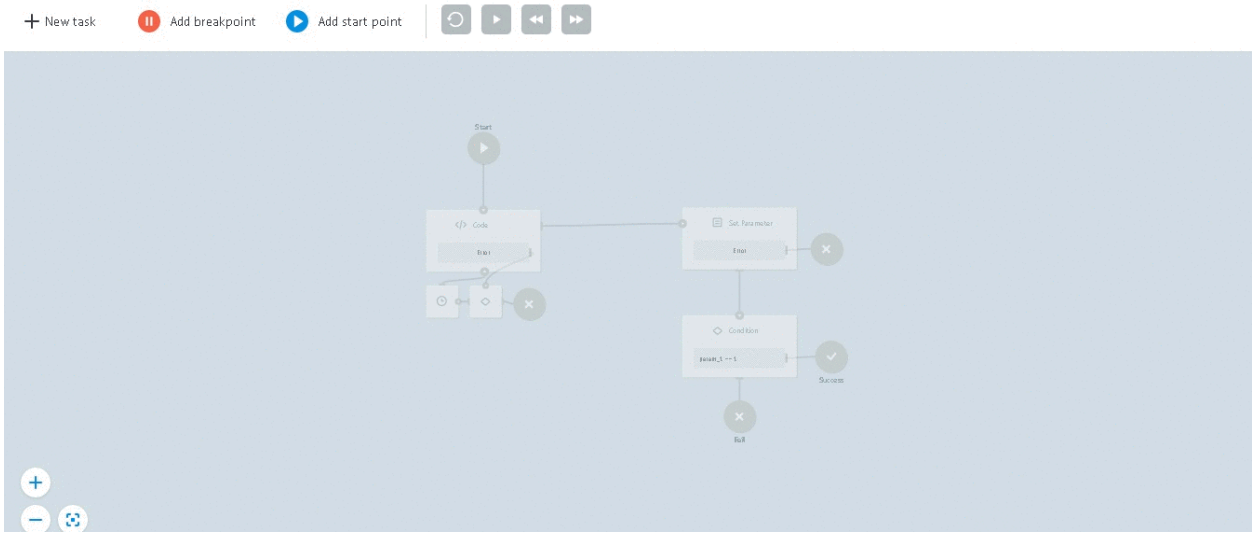
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- **Add breakpoint** button: To add the final point to the needed place in a Process.



Screenshot 359: **Add breakpoint button.**

- **Add start point** button: To add the start point to the needed place in a Process.

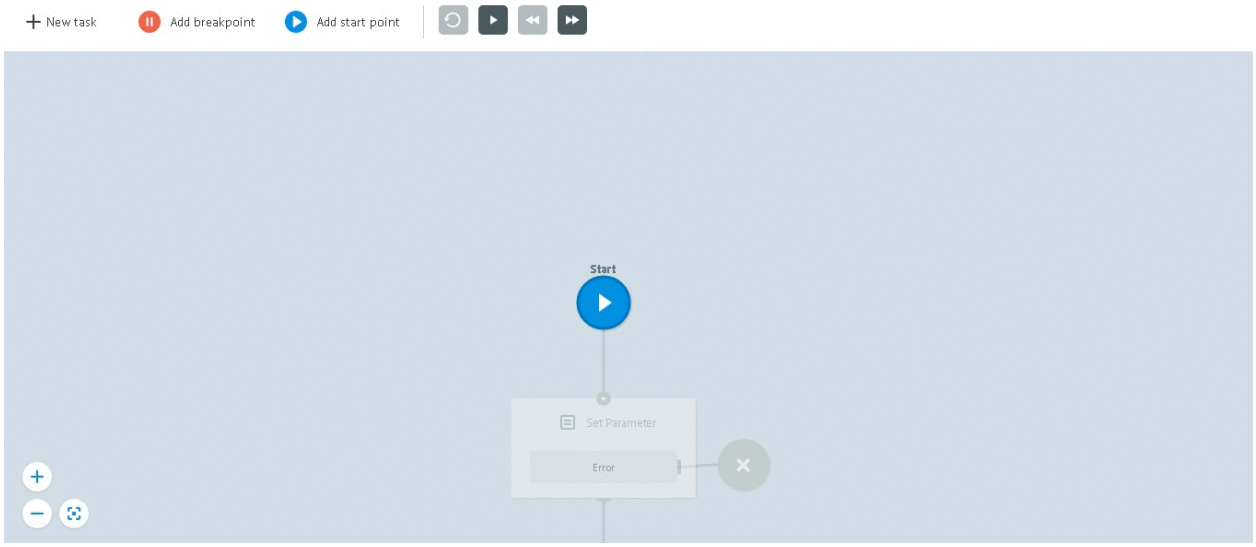


Screenshot 360: **Add start point button.**

To remove the **breakpoint** or **start point**, click it and press the **Delete** button on your keyboard.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Below, you can see an example of debugging a Process with delay.



Screenshot 361: **Debugging a Process with delay.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

6. Advanced Features

API Integration and Sync API

Corezoid supports API integration for real-time interactions. Key differences between **Corezoid API** and **Sync API**:

1. **Corezoid API:**
    - Used for asynchronous task submission.
    - Suited for bulk task processing and long-running workflows.
    - Responses are retrieved later using task IDs.
2. **Sync API:**
    - Enables immediate response for a submitted task.
    - Ideal for real-time use cases like chatbots or quick data retrieval.

API Gateway

The **API Gateway** provides advanced routing and task submission features:

- **Load Balancing:** Distribute requests across multiple Corezoid instances.
- **Security:** Add authentication layers (e.g., OAuth, API keys).
- **Transformation:** Pre-process incoming data (e.g., JSON-to-XML conversion).

Example Use:

- Use API Gateway to accept incoming webhooks from third-party services and route them into Corezoid workflows.

Customizing Notifications

- Configure multilingual templates for notifications.
- Add placeholders to dynamically include user-specific data.

- Email notification template editor with dynamic fields.

Real-Time Dashboard Creation

Dashboards in Corezoid display live metrics:

1. Open the **Dashboard** tab.
2. Drag widgets (e.g., charts, tables) onto the canvas.
3. Bind widgets to Process parameters or database queries.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- A dashboard showing live task metrics and API response times.

Advanced Automations and Conditional Logic

- Use nested **Condition Nodes** to implement complex decision trees.
- Automate workflows by integrating with external CRMs or marketing tools.

Security Features and Role-Based Access Control

- Assign roles and permissions to team members.
- Use **audit logs** to track changes in Processes.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

7. Communications Orchestrator

Overview

Communications Orchestrator is a set of out-of-the-box Corezoid Processes for creating and managing bots in different messengers. Corezoid enables you to create universal Processes for managing business logic in different messengers instead of developing the business logic individually for each messenger.

Communications Orchestrator supports the following messengers:

- Facebook Messenger
- Viber
- Telegram
- Apple Messages for Business
- Slack

Note: Whatsapp and Line are coming soon.

Orchestrator Deployment Issue in On-Premise Environments

Orchestrator deployment doesn't work in on-premise environments. This feature is designed for Simulator cloud where orchestrator creation is a mass operation.

Workaround for using Orchestrator in on-premise environments**:**

1. Create orchestrator in Corezoid cloud environment.
2. Manually transfer it (Project with processes) to your on-premise environment for further development

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

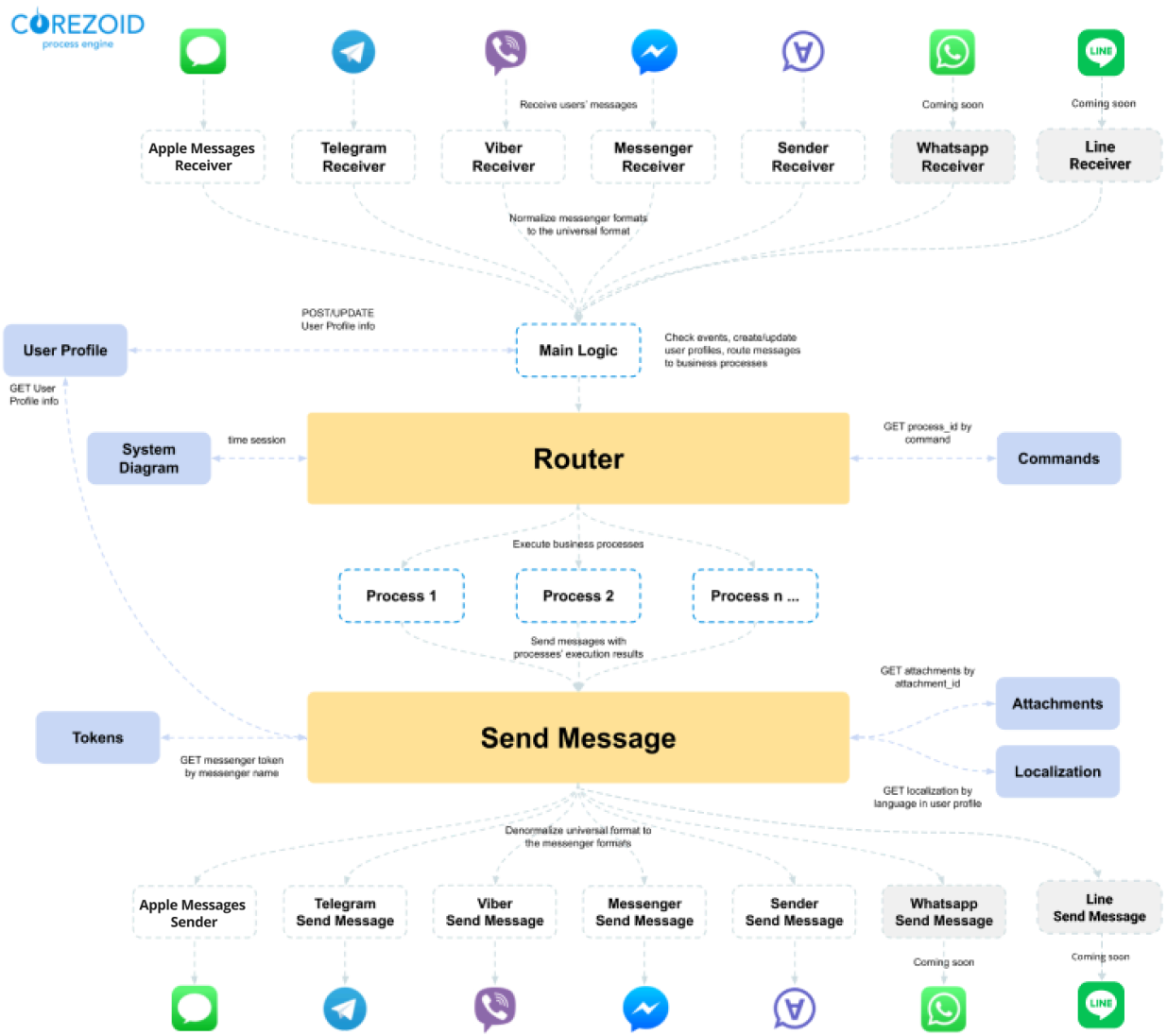


Diagram 25: **Communications Orchestrator architecture**.

Here's the sequential flow of component interactions in Communications Orchestrator, from user input to response:

1. User sends a message via one of the messenger channels (Telegram, Viber, Facebook Messenger, Apple Messages)
2. Message Reception:
    - The corresponding Receiver Process captures the incoming message
    - Converts messenger-specific format into unified standard format with fields: channel, chat_id, event, message
3. Main Logic Processing:
    - Processes standardized message data

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

    - Checks events and user profiles
    - Creates/updates user profiles when needed
    - Routes messages to appropriate business processes
4. Router Operation:
    - Receives commands and messages from Main Logic
    - Identifies target Process based on command type
    - Routes requests to appropriate Process (Process 1, Process 2, Process n)
    - Controls and monitors business Process execution
5. Business Process Execution:
    - Specific Process handles the user request
    - Generates response data
    - Prepares data for sending back to user
6. Send Message Processing:
    - Receives output from business processes
    - Converts unified message format to messenger-specific format
    - Manages message formatting and attachments
    - Applies localization if needed
    - Sends formatted messages back to the appropriate messenger channel
7. User Receives Response:
    - Message is delivered through the original messenger channel
    - Displayed to user in the messenger interface

Supporting Components Throughout Flow:

- User Profile: Stores and manages user data
- System Diagram: Maintains system configuration
- Commands: Manages available bot commands
- Attachments: Handles file and media processing
- Localization: Provides multi-language support
- Tokens: Manages messenger authentication tokens

Creating Bots in Messengers and Connecting Them to Your Communications Orchestrator

A bot in Corezoid represents a separate project, which is created from scratch following a simple procedure:

- Step 1. Create bots in messengers:
    - Telegram
    - Viber

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

    - Facebook Messenger
    - Apple Messages for Business
- Step 2. Connnect bots to Communications Orchestrator in Corezoid

You can connect Communications Orchestrator to new or earlier created messenger bots.

Step 1. Create bots in messengers

Communications Orchestrator accesses bots in the messengers using HTTP API with token identification. A token is a key for accessing your bot. Anyone who possesses this key can control your bot. The token storage and access to it must comply with security requirements for sensitive data.

To connect an existing bot to Communications Orchestrator, go to step 2. Connect bots to Communications Orchestrator in Corezoid.

Telegram

1. Begin a chat with BotFather and click **Start**.
2. Send the `/newbot` command to BotFather to create a new bot.
3. Follow the BotFather's instructions:
    - Name your bot.
    - Give your bot a username to be displayed in contacts, chats, and addresses. **Note:** A username must have the bot ending, for example, **TetrisBot** or **tetris_bot**. The name and username of a bot must be unique in Telegram.
4. Save the provided access token to your bot.

Viber

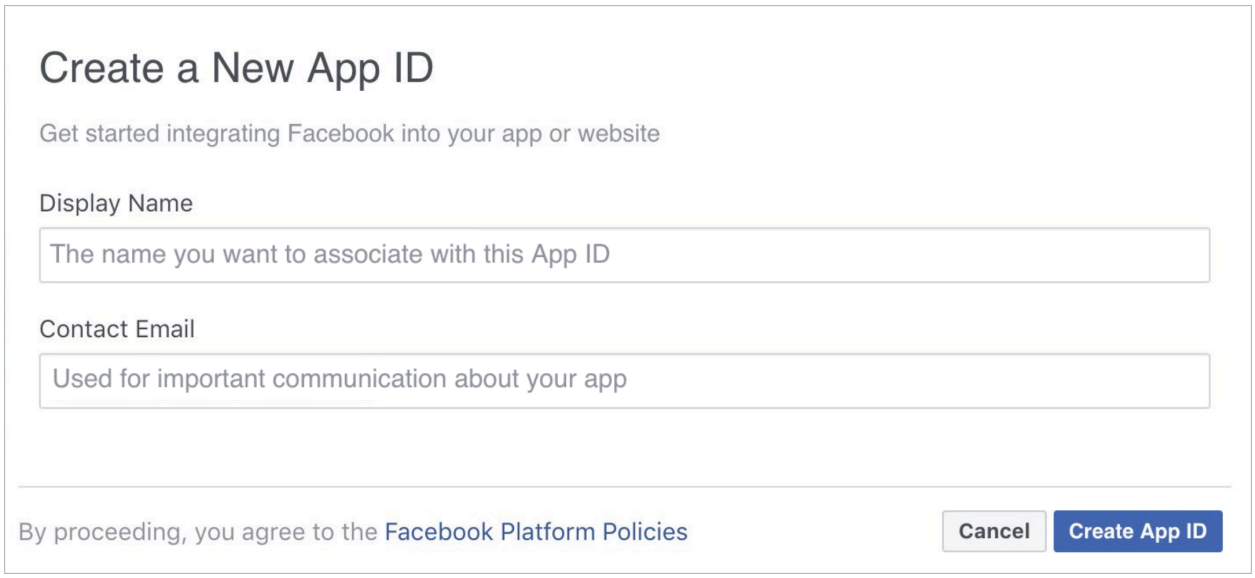
Follow the contact form link to submit your application for a Viber bot registration. Our Support team will guide you through the procedure and provide all the necessary information.

Facebook Messenger

1. Log in to your Facebook account.
2. Create a new page that your bot will be connected to by following the instructions. **Note:** You must connect each Facebook bot to a Facebook page.
3. Go to Facebook for Developers. **Note:** If you are new here, register as a developer.

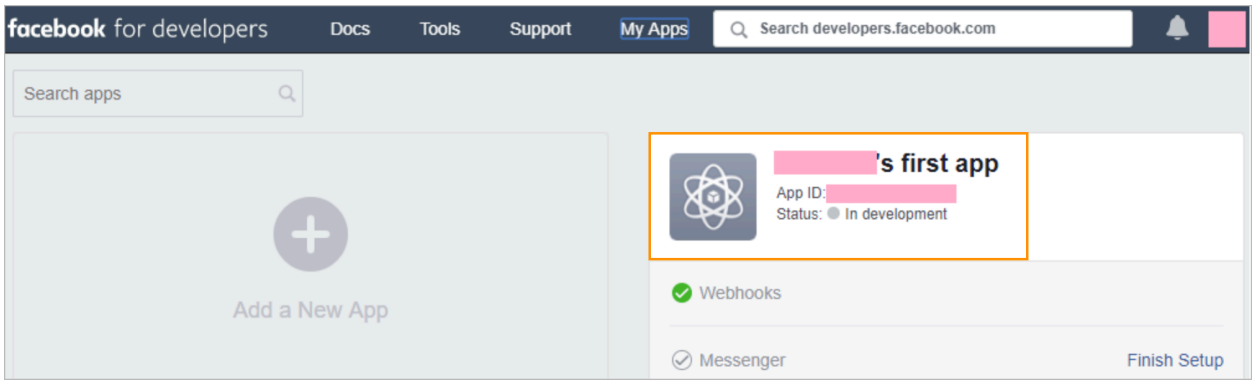
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

4. Hover over **My Apps**, and then click **Create New App**.
5. In the **Create a New App ID** dialog, fill in all the fields and click **Create App ID**.



Screenshot 362: **Create a Facebook messenger bot.**

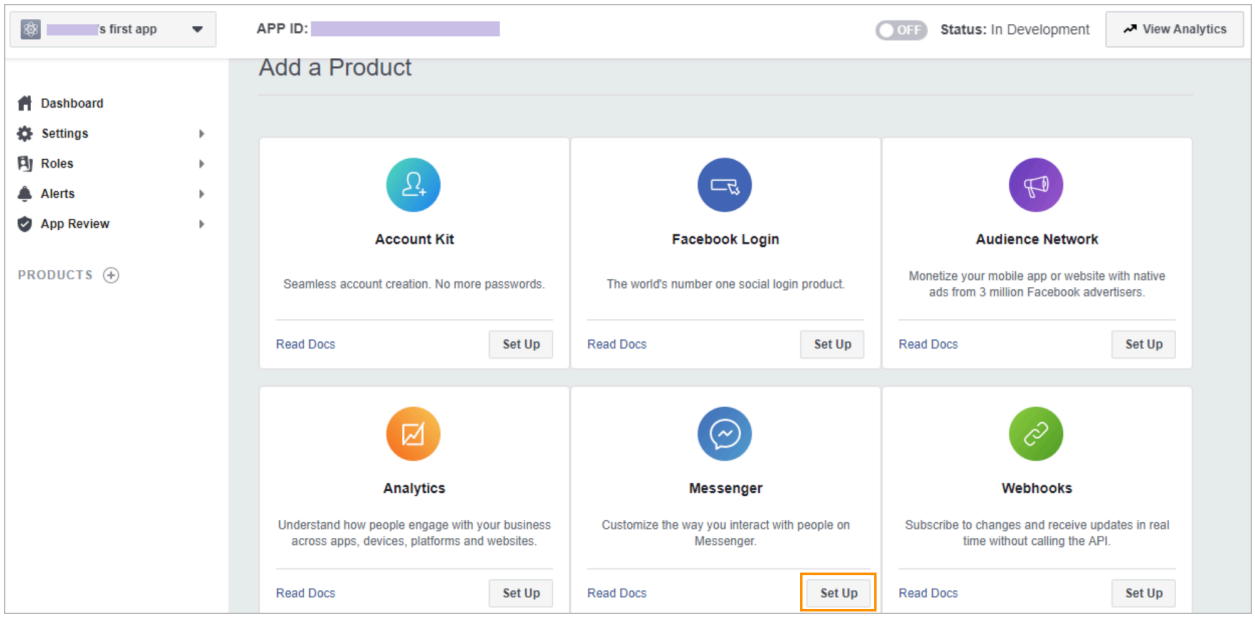
6. Click the app creation icon.



Screenshot 363: **Create a Facebook messenger bot.**

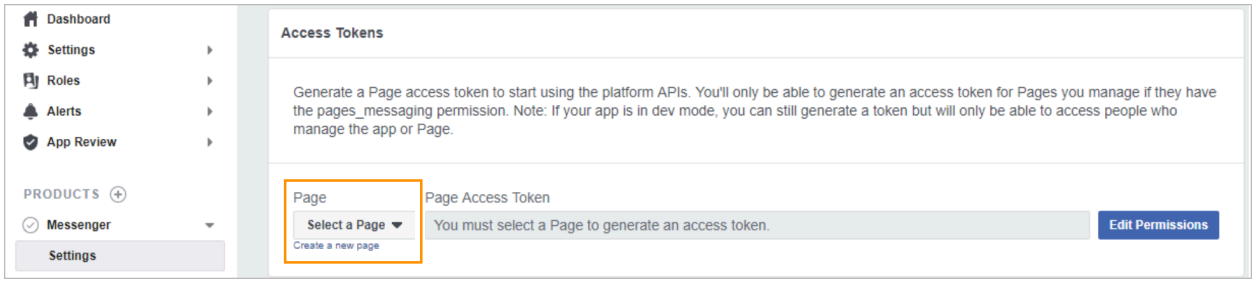
7. On the left navigation panel, click **Dashboard**, and in the **Add a Product** list, find **Messenger** and click **Set Up** below.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 364: **Create a Facebook messenger bot.**

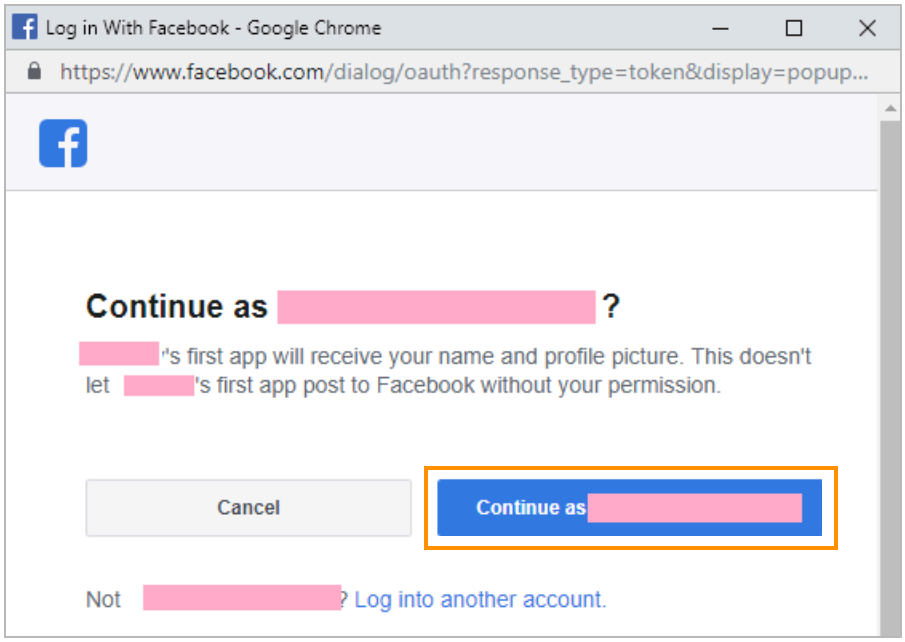
8. In the **Page** dropdown list of **Access Tokens** on the Messenger **Settings** page, select **Edit Permission**.
   Note: This is the page you created in step **2**.



Screenshot 365: **Create a Facebook messenger bot.**

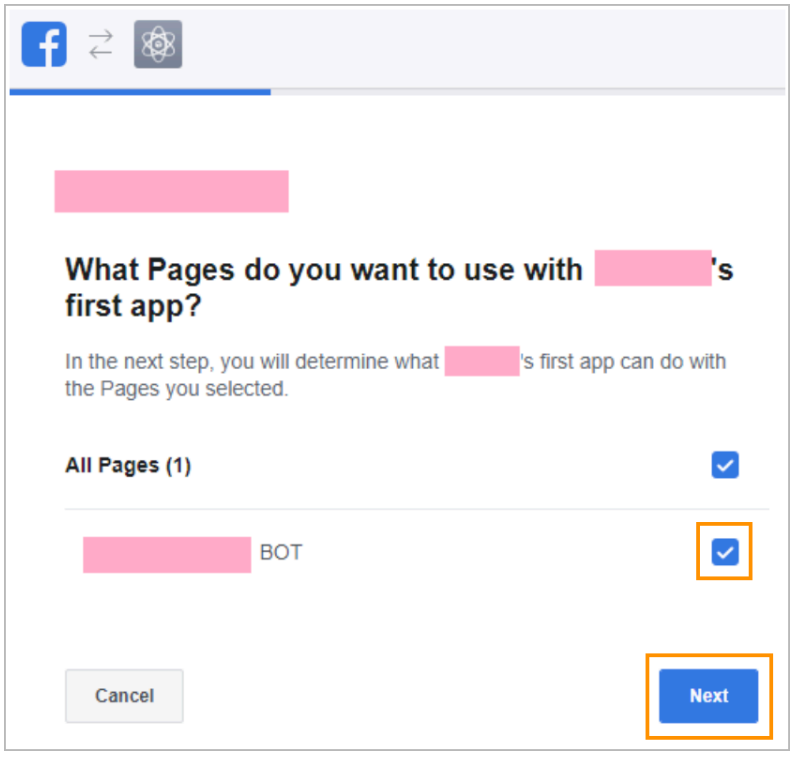
9. In the new browser dialog, click **Continue as [Your Profile Name]** to confirm assigning your bot to your Facebook account.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 366: **Create a Facebook messenger bot.**

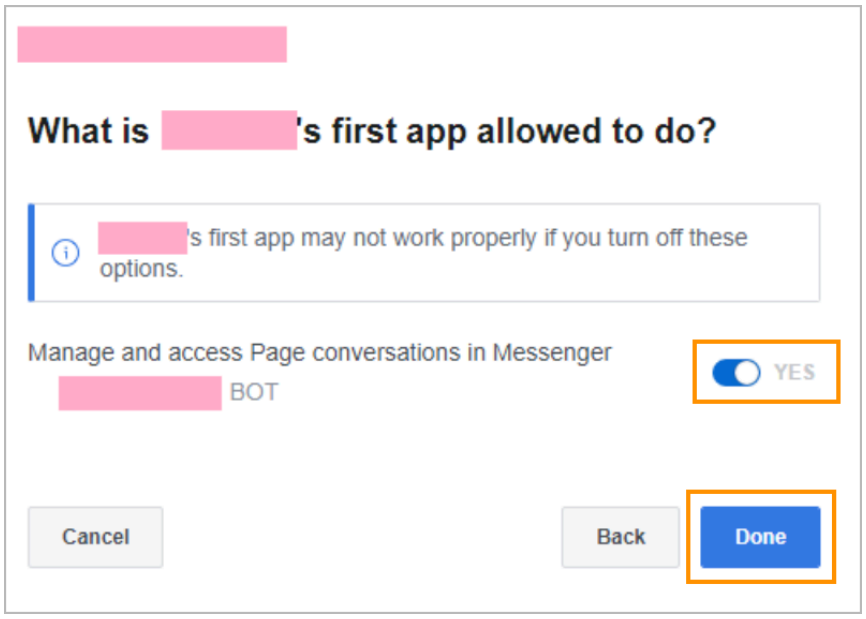
10. In the next dialog, select the checkbox of the page your bot will be attached to and click **Next**.
    Note: This is the page you created in step **2**.



Screenshot 367: **Create a Facebook messenger bot.**

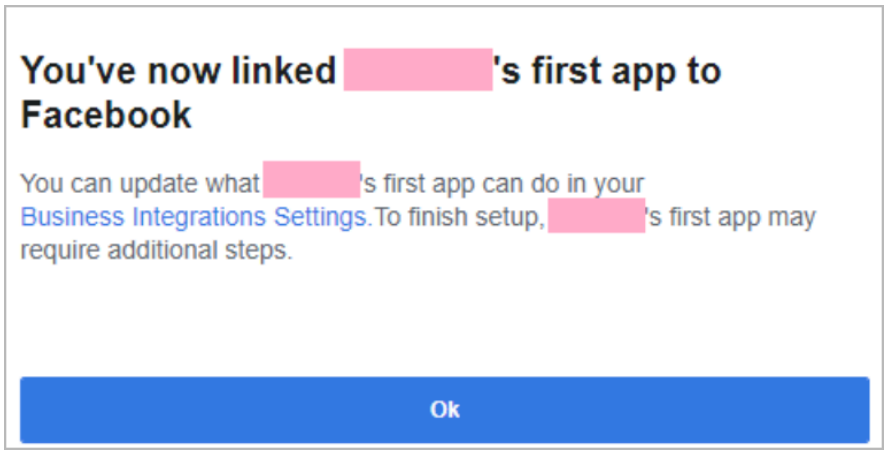
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

11. In the next dialog, set the toggle for your page to **YES** to allow your bot to participate in the P2P chats in the messenger and click **Done**.



Screenshot 368: **Create a Facebook messenger bot.**

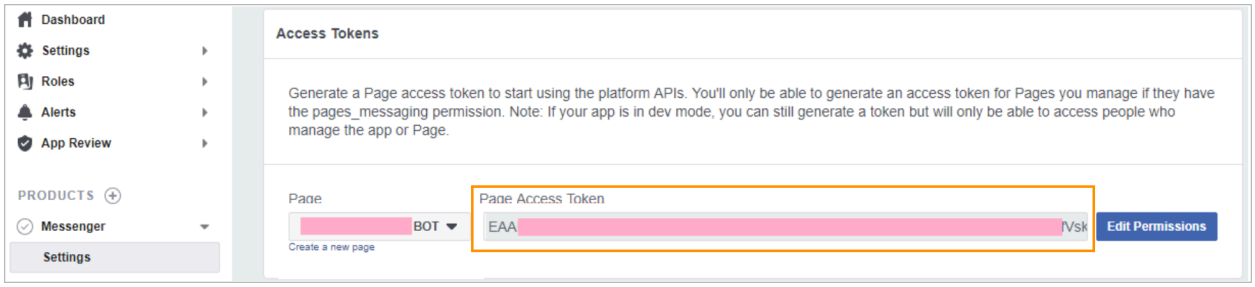
12. In the next dialog, click **Ok**.



Screenshot 369: **Create a Facebook messenger bot.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

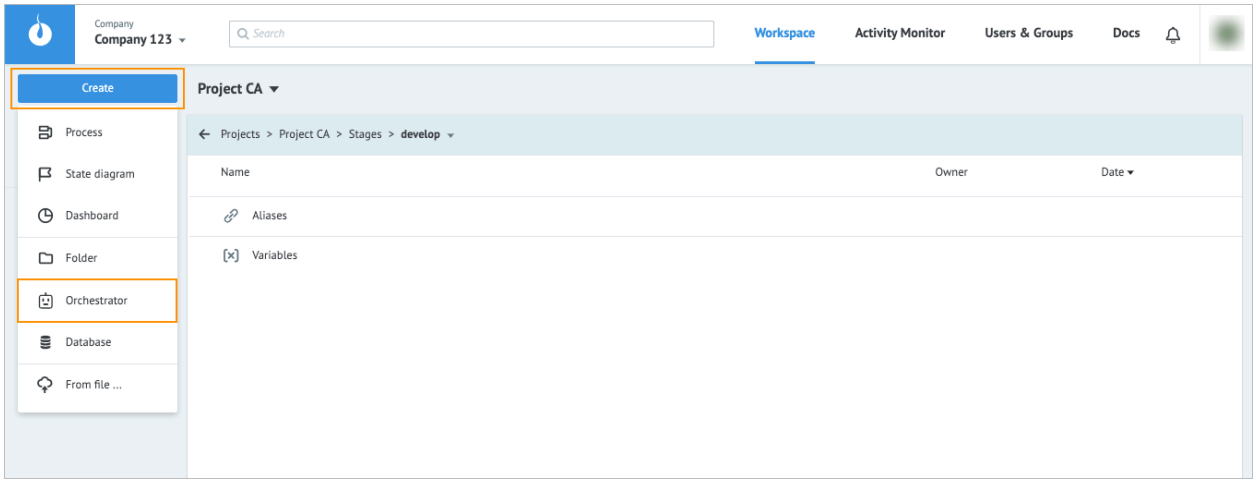
13. In the **Page Access Token** field, copy and save the access token.



Screenshot 370: **Create a Facebook messenger bot.**

Step 2. Connect bots to Communications Orchestrator in Corezoid

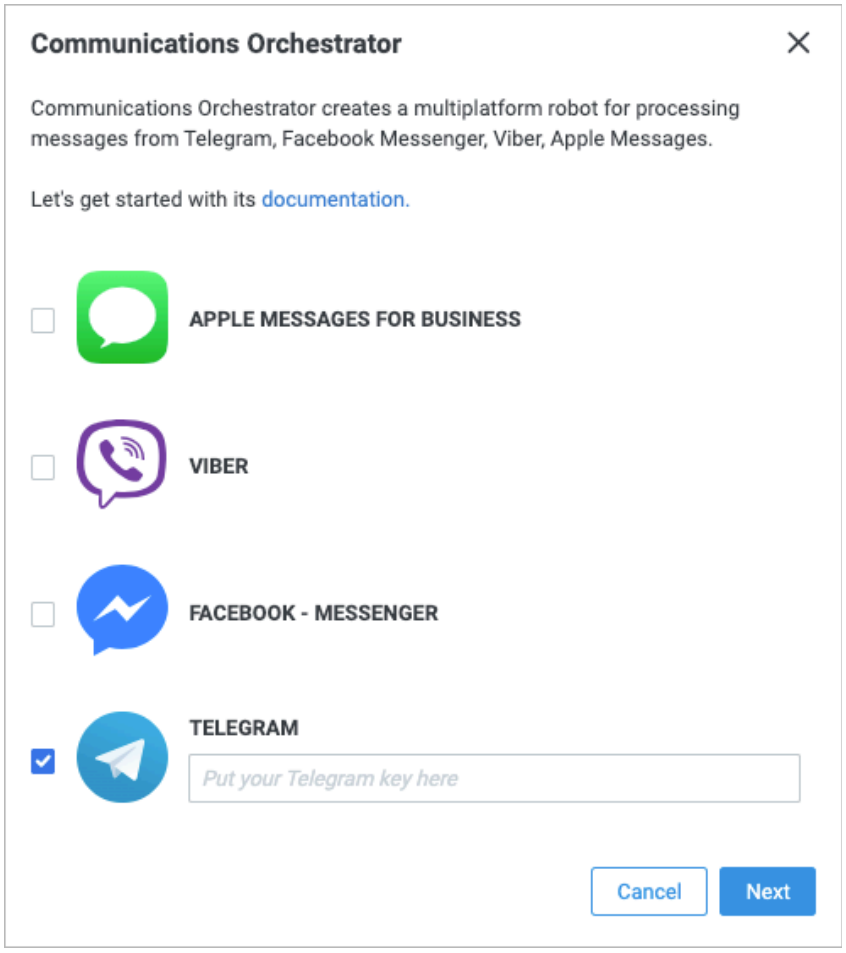
1. Go to admin.corezoid.com, open the **Workspace** tab, and click the **Projects** folder.
2. Select the needed stage, create a new project, or open an existing one.
   Note: To create a Communications Orchestrator project, choose a non-immutable stage.
3. In the upper-left corner, click **Create** and from the dropdown list, select **Orchestrator**.



Screenshot 371: **Create Communications Orchestrator.**

4. In the **Communications Orchestrator** dialog:
   a. Select the messengers you want to integrate.
   b. Paste the access token you copied when creating the bot
   c. Click **Next**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 372: **Messenger selection dialog.**

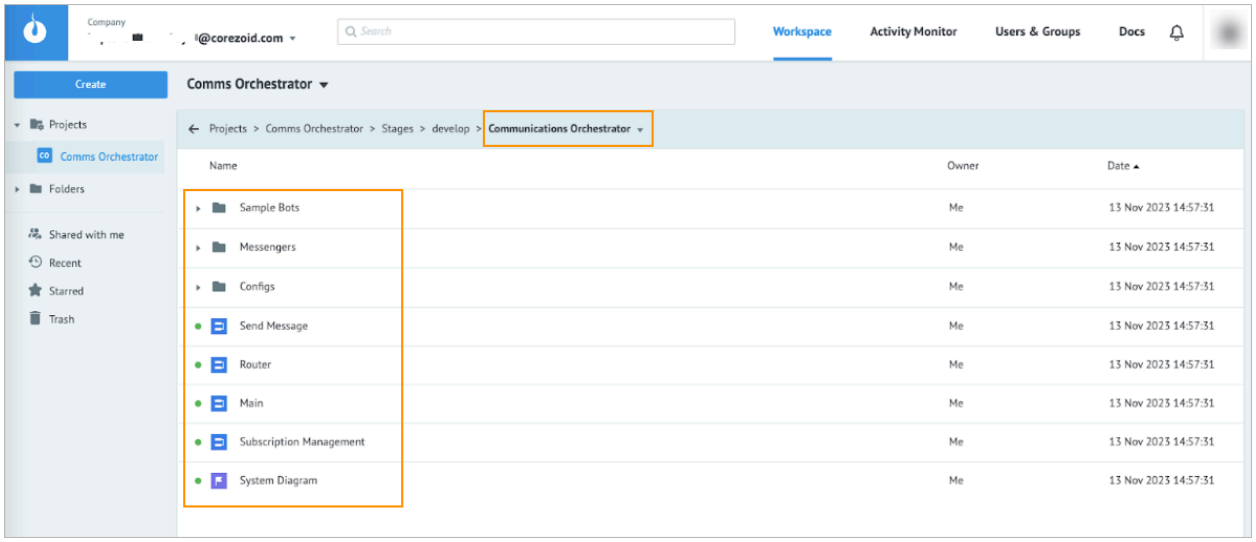
Note:

- If the connection procedure didn't succeed, check each token to be valid.
- If you connect Facebook Messenger, you need to subscribe to Facebook Webhooks in the Facebook admin panel. For more information, go to Subscribe to Facebook Webhook.

6. In the dialog, click **Open the created Bot** to open the **Communications Orchestrator** folder.

The bot is connected to the Communications Orchestrator folder, and you can start working in the folder. For more information, go to Components.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

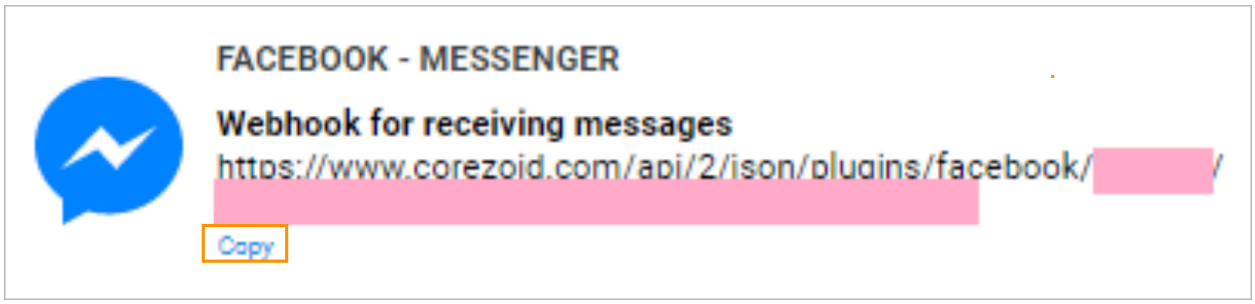


Screenshot 373: **Communications Orchestrator folder.**

Subscribe to Facebook Webhook

When you select other messengers, these steps are done automatically.

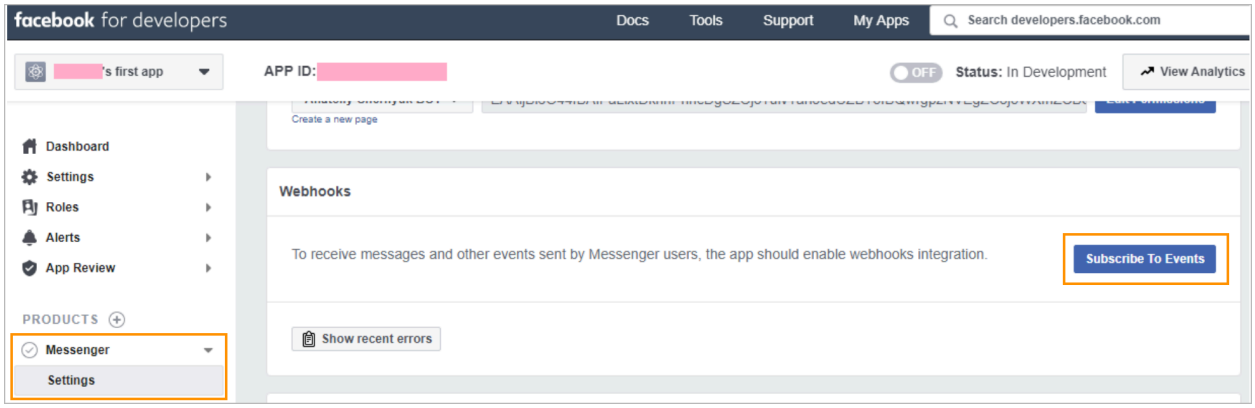
1. Go to your Facebook app's page for developers.
2. Copy your access token created above Facebook Messenger, step **14**).
3. In the **Communications Orchestrator** dialog of Corezoid, click **Copy** under the Webhook URL for Facebook Messenger.



Screenshot 374: **Facebook Messenger token copy.**

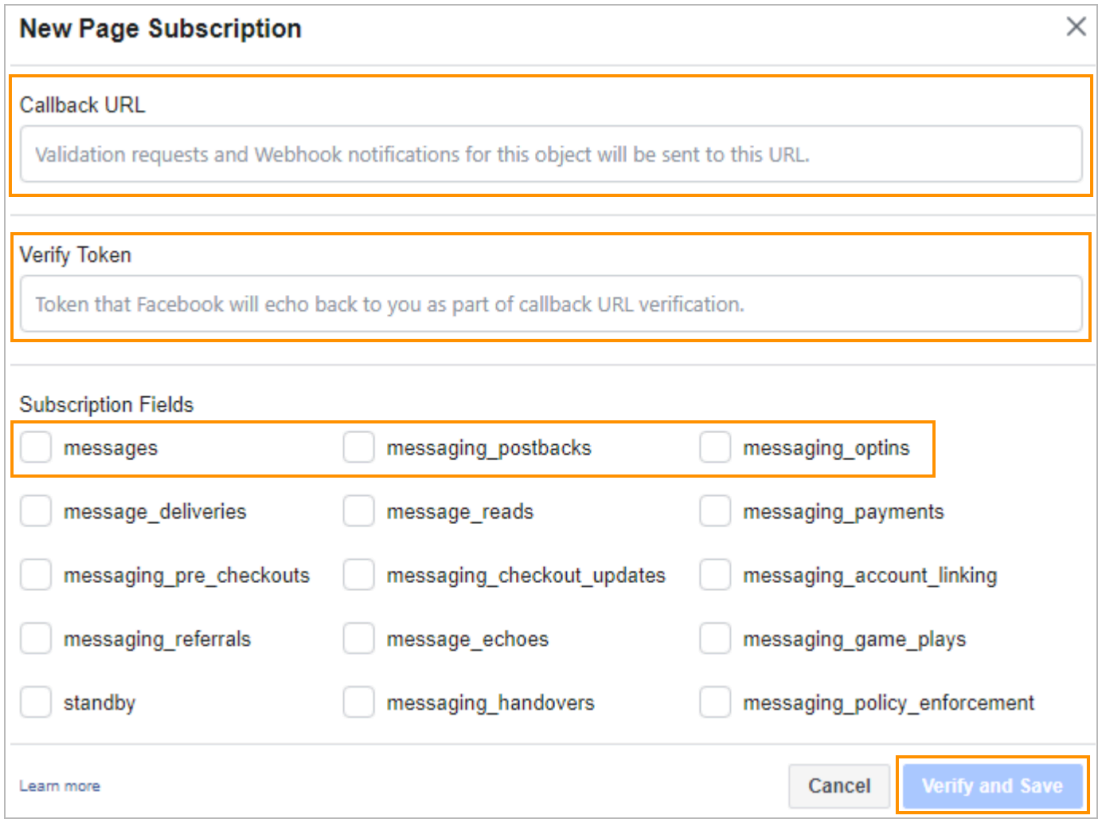
4. In the left navigation panel, go to **Messenger** > **Settings** and click **Subscribe to Events**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 375: **Facebook Messenger account settings.**

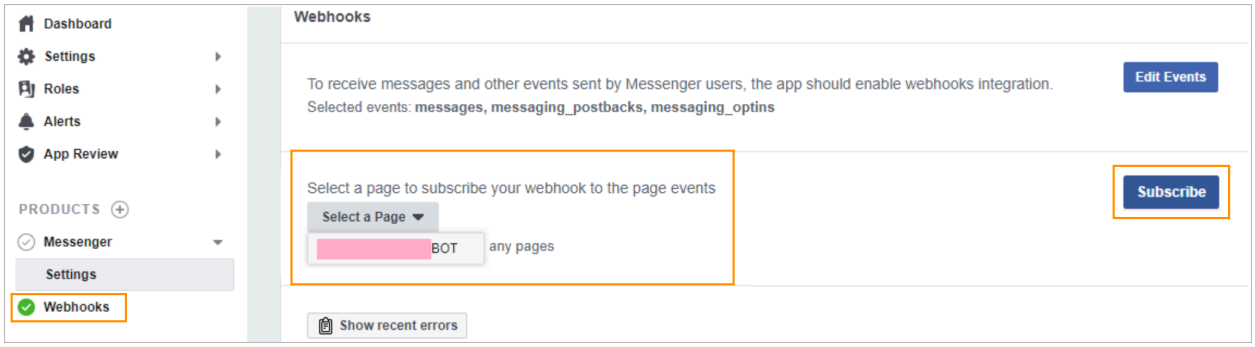
5. In the **New Page Subscription** dialog:
    a. Enter the copied Webhook URL in the **Callback URL** field.
    b. Enter the copied token in the **Verify Token** field.
    c. Select the **messages**, **messaging_postbacks**, and **messaging_optins** checkboxes.
    d. Click **Verify and Save**.



Screenshot 376: **Facebook Messenger subscription settings.**

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

6. In the **Select a Page** dropdown list of the **Webhooks** page, select the page to which your page will be subscribed for events and click **Subscribe**.



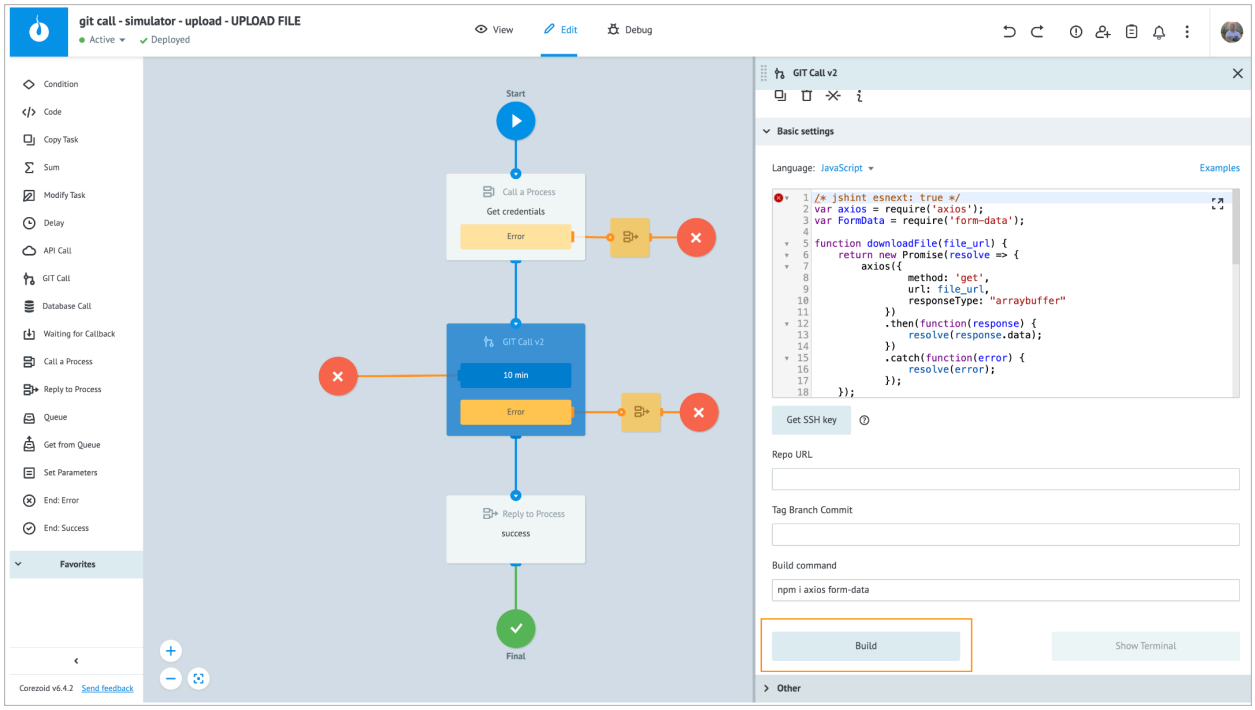
Screenshot 377: **Facebook Messenger Webhooks section.**

You've created your first Communications Orchestrator.

Create Communications Orchestrator

1. Create a bot for one or multiple messengers you plan to use with Communications Orchestrator and save the access token for that/those messenger(s).
2. Create a Communications Orchestrator project in Corezoid.
   Note: For more information on creating a project, go to Project.
3. Add the Git Call node to your account:
   a. Go to **Messengers** → **Simulator** → **API** → **upload** → **git call - simulator - upload - Upload file**.
   b. Click the Git Call node and click **Build**.
   c. In the upper-right corner, click **Deploy**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



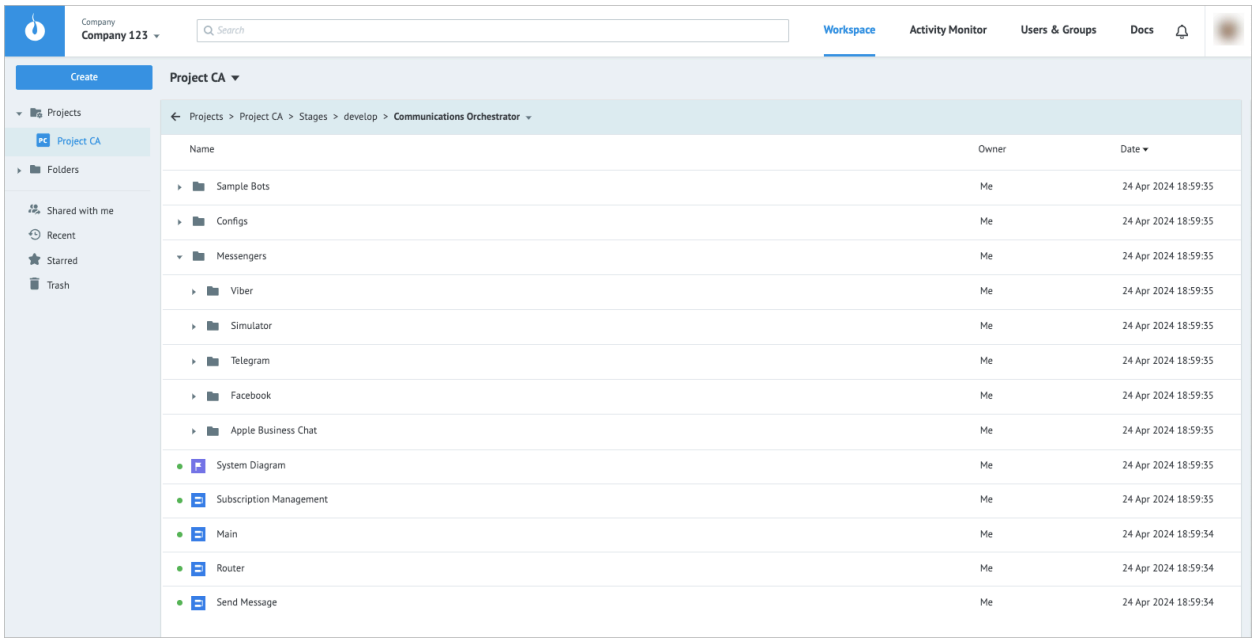
Screenshot 378: **Git Call predefined code build**.

The project folder with main components located in **Projects** > **Stages** > **develop** >

Communications Orchestrator stores the following required basic Processes and functionality:

- Viber/Telegram/Facebook Receiver Process
- Main Process
- Event types
- Router Process
- Send Message Process
- Configs folder
- Messengers > Simulator
- Account components

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

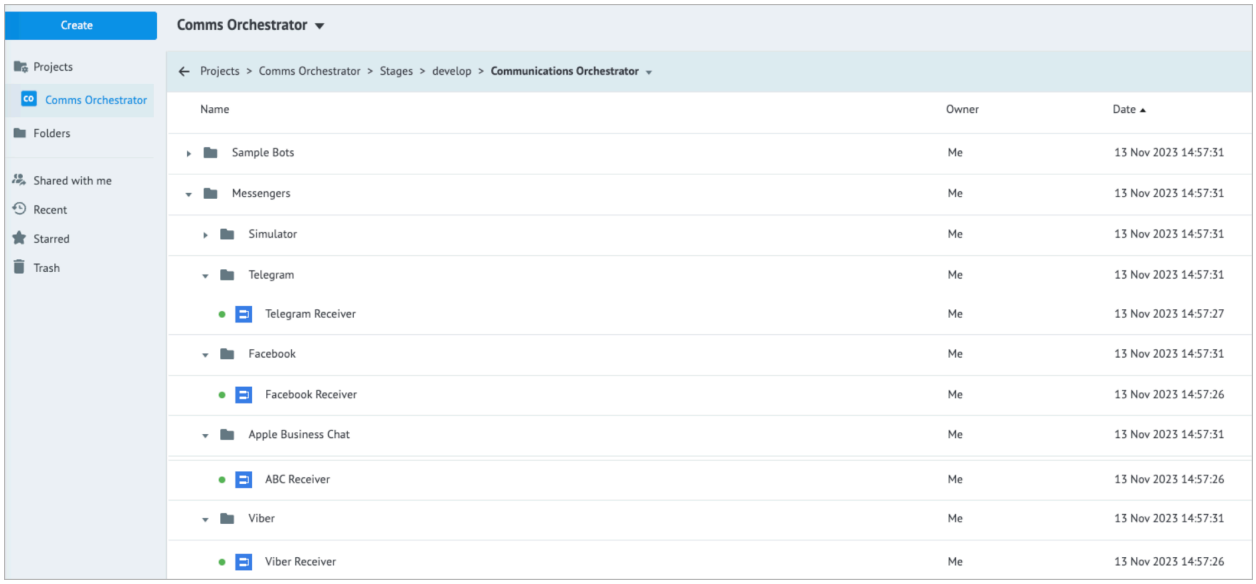


Screenshot 379: **Contents of the created Communications Orchestrator folder**.

Receiver Process

The Viber Receiver, Telegram Receiver, Facebook Receiver, ABC Receiver (Apple Business Chat), Slack Receiver Processes are located in the corresponding messenger folders: **Projects** > **Stages** > **Communications Orchestrator** > **Messengers** > [**Messenger Name**].

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 380: **Communications Orchestrator Messengers subfolders**.

The Receiver Processes are the work starting points of each bot: a [messenger name] Webhook connects to the [same-messenger-name] Receiver Process during Communications Orchestrator creation.

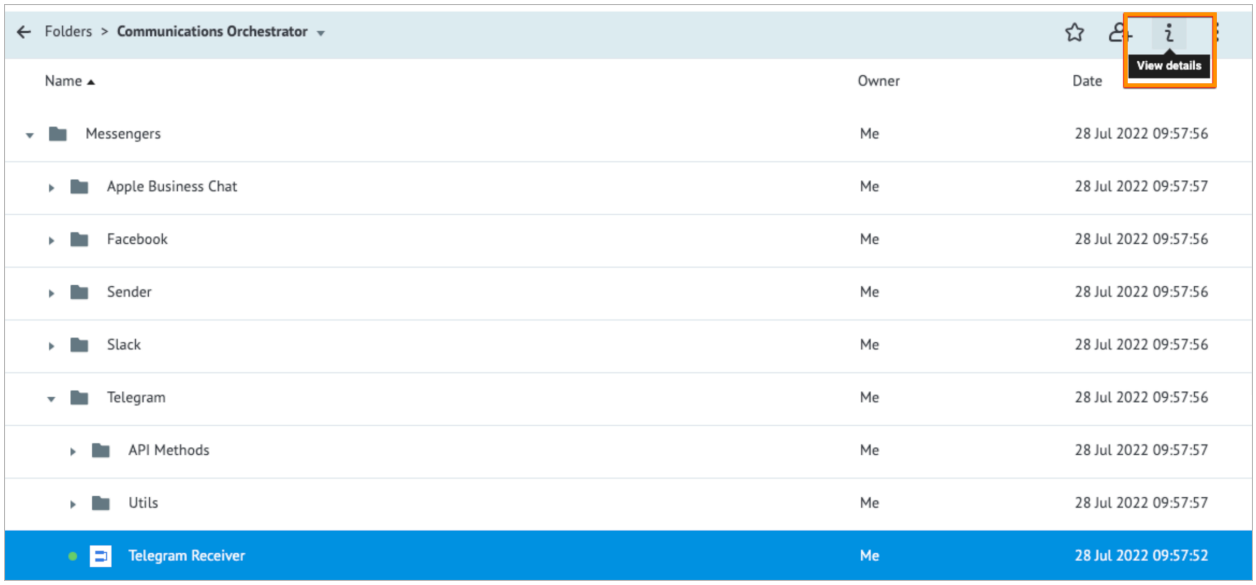
The Receiver Processes perform the following actions:

- Receive events (messages and files sent, buttons clicked, links followed, and so on) from users.
- Convert received data into the unified standard data structure:
    - `channel` – the message channel. Available options: `telegram`, `facebook`, `viber`, and `abc`.
    - `chat_id` – identifier of a chat, user, and so on.
    - `event` – event type.
    - `message` – message object.
- Forward the converted data to the Main Process.

To attach another bot to an existing project in Communications Orchestrator, connect a webhook of this bot to the corresponding Receiver Process manually:

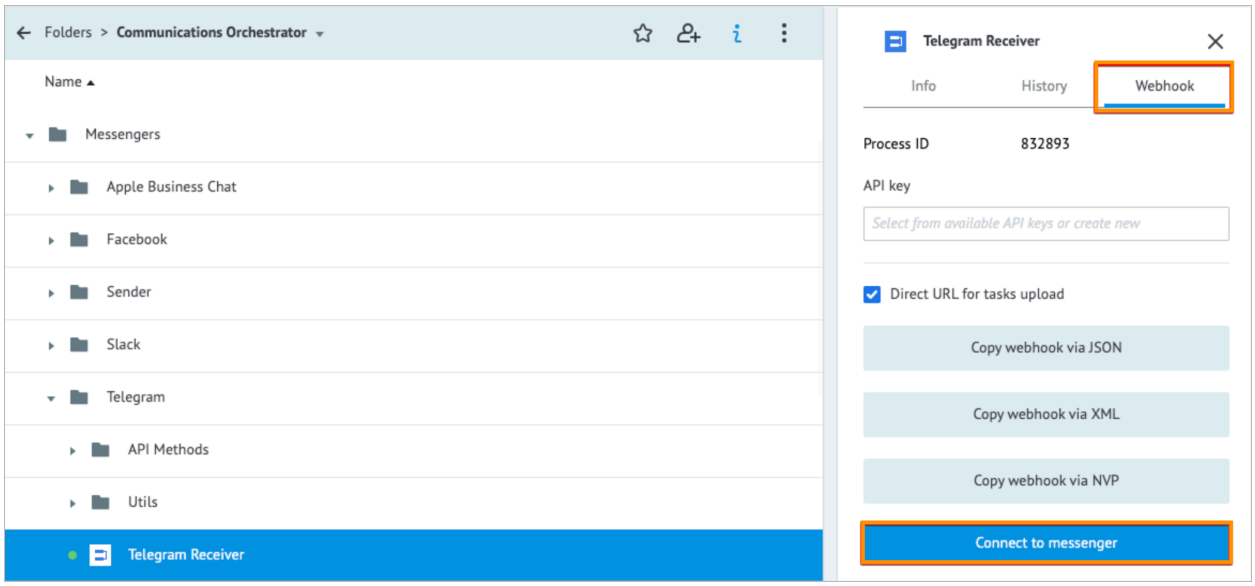
1. Click the corresponding [Messenger name] Receiver Process and in the upper-right corner, click **View details**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 381: **Telegram Receiver Process**.

2. On the **Webhook** tab of the selected [Messenger name] Receiver Process details panel, click **Connect to messenger**.

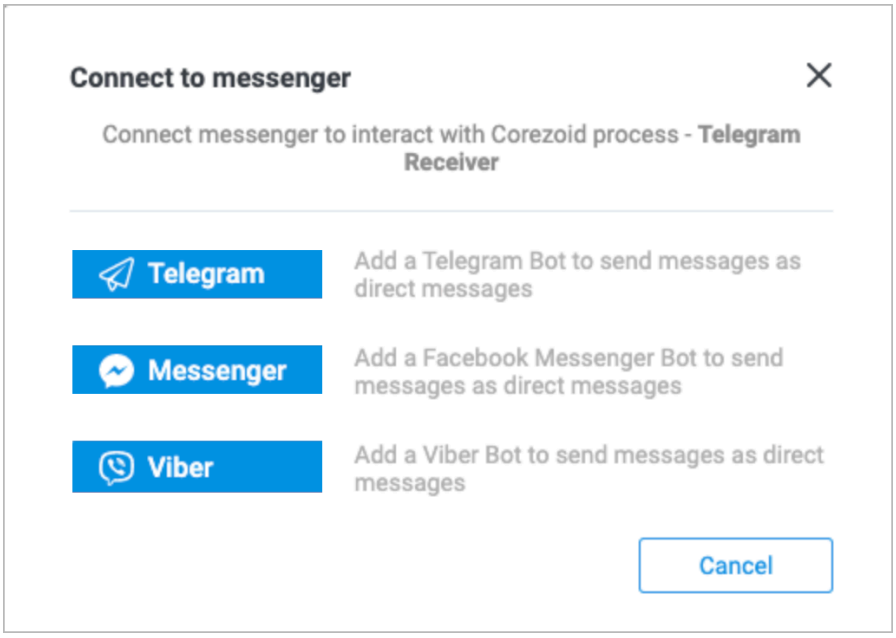


Screenshot 382: **Telegram Receiver Process webhook**.

3. In the Connect to messenger dialog:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

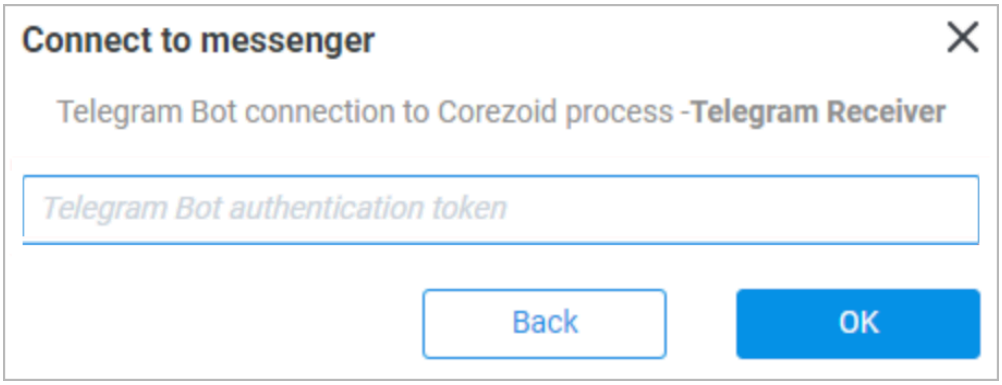
a. Select the needed messenger.



Screenshot 383: **Connect to messenger**.

Note: you can connect multiple bots in all the supported messengers to your Communications Orchestrator anytime by changing the settings in your Communications Orchestrator project.

b. Enter an access token to the new bot in the text box field and click **OK**.



Screenshot 384: **Connect to messenger token input**.

4. Update the token in the Tokens state diagram.

Note: The Task **REF** should have the **token** name.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

The Webhook is connected to the Receiver Process, and the bot is attached to the selected Process in Communications Orchestrator.

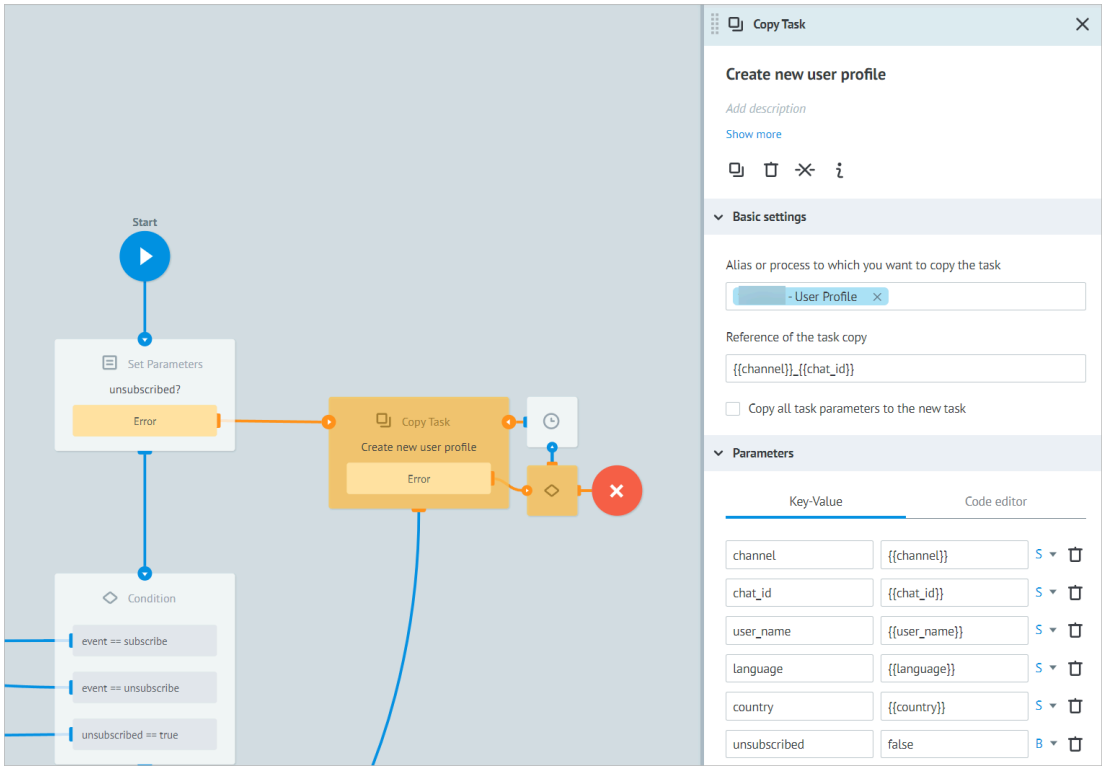
Main Process

The Main Process receives the converted data from the messengers and runs the initial business logic of the bot before routing tasks to sub-Processes.
Input JSON example


```
{
    "channel": "viber",
    "chat_id": "12312321",
    "event": "message",
    "message": {
    "type": "text",
    "text": "Lorem ipsum..."
            }
}
```


Incoming data parameters to the Main Process

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 385: **Incoming data parameters to the Main Process**.

Parameter	Type	Required	Description
channel	string	+	Name of a messenger from which a user wrote
chat_id	string	+	Unique identifier of a user
event	string	+	Event type. Available options: `start`, `context`, `message`, `command`.
message	object	+	Message object.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

first_name	string	-	User’s first name.
last_name	string	-	User’s last name.
gender	string	-	User’s gender.
user_picture	string	-	Link to a user’s profile picture.
country	string	-	Country where a user is currently in and detected by a messenger automatically.
language	string	-	Language set on the user’s device.
timezone	string	-	Time zone.
context	string	-	Data of the `context` event.

Event types

Type of event	Description
start	It is initiated when a user is connected to a bot for the first time; in Viber – each time a chat dialog is opened.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

context	It is initiated if the incoming message has a context. A context is an additional parameter that can be added to a link leading to a bot, like a command launching, reference link, and so on.
message	It is initiated when a user sends a message.
command	It is initiated when a user sent a message of the `/commandName` type.

Examples of data transmission using **context**:

- **Viber**: `viber://pa?chatURI={{public_account_name}}&context={{params}}`
- **Facebook Messenger**: `http://m.me/{{bot_name}}?ref={{params}}`
- **Telegram**: `https://telegram.me/{{bot_name}}?start={{params}}`

Event Types Overview

The system processes four main types of events:

- `start` - Triggered when:
  - User connects to the bot for the first time
  - In Viber - each time chat dialog is opened
  - Used for initial user setup and welcome messages
- `context` - Triggered when:
  - Incoming message has additional context parameters
  - Used with deep links and reference parameters
  - Can be used for campaign tracking or specific scenario initialization
- `message` - Triggered when:
  - User sends any type of message
  - Main event type for regular user interactions
- `command` - Triggered when:
  - User sends a message in `/commandName` format
  - Used for specific bot function activation

2. **Context Data Transmission Examples:** Each messenger has its specific format for passing context:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Viber:

viber://pa?chatURI={{public_account_name}}&context={{params}}

Facebook Messenger:

http://m.me/{{bot_name}}?ref={{params}}

Telegram:

https://telegram.me/{{bot_name}}?start={{params}}

3. **Typical Processing Logic:**

For `start` events:

- User profile creation
- Welcome message sending
- Initial bot menu presentation
- Setting up user preferences

For `context` events:

- Parameter extraction and validation
- Specific scenario activation
- Campaign tracking
- Deep link processing
- Special offer activation

For `message` events:

- Message type identification
- Content processing
- Response generation
- User state management
- Dialog flow control

For `command` events:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- Command validation
- Permission checking
- Function execution
- Response formatting
- State updates

4. Common Use Cases:

- Marketing Campaigns:
  - Using context parameters for tracking sources
  - Activating special offers
  - Campaign-specific flows
- User Authorization:
  - Initial user registration
  - Session management
  - Access control
- Multi-step Processes:
  - State management
  - Progress tracking
  - Process continuation
- Feature Access:
  - Command-based feature activation
  - Permission validation
  - Feature-specific flows

5. Event Processing Pattern:

```
{
  "channel": "viber", // messenger type
  "chat_id": "12312321", // unique user identifier
  "event": "message", // event type
  "message": {
    "type": "text",
    "text": "..."  // event content

  }
}
```

6. Integration Points:

- With User Profile:
  - Profile creation and updates

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

    - User preferences storage
    - State management
- With Router:
    - Event routing to appropriate handlers
    - Process selection
    - Flow control
- With Main Process:
    - Business logic execution
    - Response generation
    - State transitions

Router Process

The Router Process implements the main logic of processing Communications Orchestrator events. All the event types can be processed here.

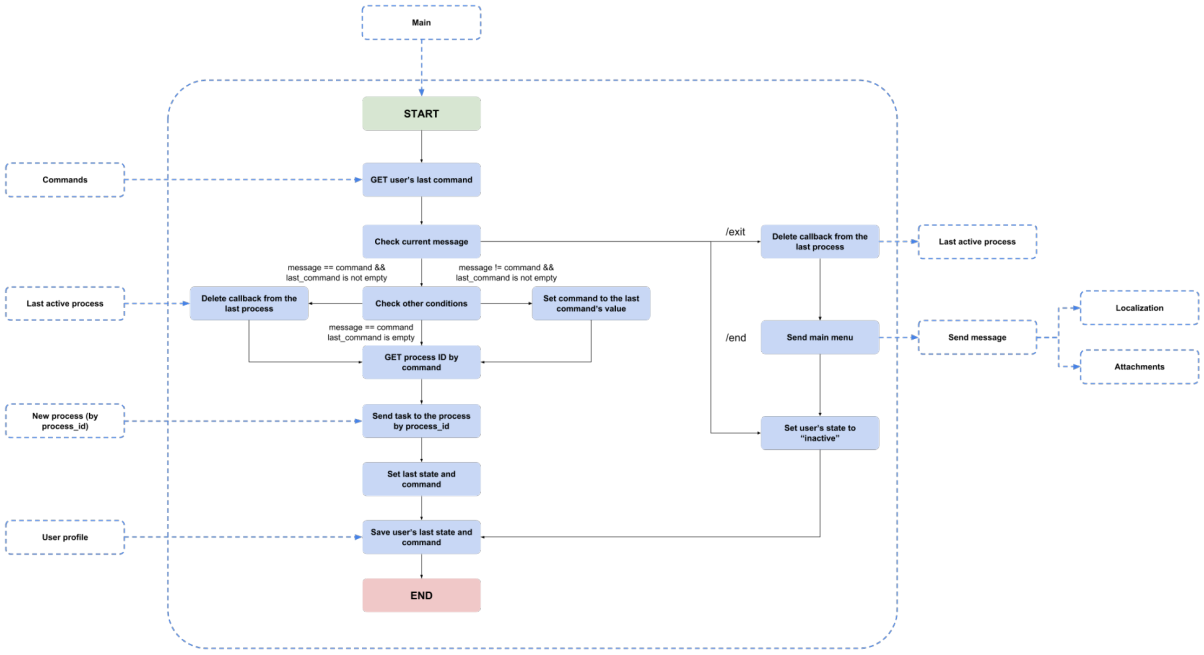


Diagram 26: **Router Process flow**.

Router Process Overview

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

1. Router Process Core Function:

- Serves as the main logic implementation for processing Communications Orchestrator events
- Handles all event types centrally
- Coordinates message flow and command execution

2. Main Process Flow:

- Starts with getting user's last command
- Proceeds to current message check
- Has two key conditions branches based on message type:
    - Message equals command with non-empty last command
    - Message equals command with empty last command

3. Core Decision Points:

- Check Current Message
    - Validates incoming message content
    - Determines message type and format
    - Routes based on message characteristics
- Check Other Conditions
    - Evaluates additional routing criteria
    - Handles special cases
    - Manages Process flow control

4. Command Processing:

- When command detected:
    - Gets Process ID by command
    - Sends task to the Process by process_id
    - Sets last state and command
    - Saves user's last state and command
- When no command:
    - Routes to appropriate handler
    - Maintains Process continuity
    - Manages state transitions

5. Exit Handling:

- Special `/exit` command processing
- Deletes callback from last Process
- Provides clean exit from current Process
- Returns to main menu state

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

6. End States:
- Two main termination paths:
    - `/exit` path: Cleanup and return to main menu
    - `/end` path: Complete current Process and send main menu

7. Integration Points:
- With User Profile:
    - User state management
    - Preference tracking
    - Session handling
- With Commands:
    - Command validation
    - Process ID lookup
    - Command execution
- With Last Active Process:
    - Process state management
    - Callback handling
    - Process transitions

8. Output Handling:
- Manages message sending to users
- Handles localization requirements
- Manages attachments
- Coordinates with appropriate messenger APIs

Send Message Process

The Send Message Process is responsible for sending messages to users. It receives text templates, attachments, and argument values to compose them together dynamically into a single ready-for-sending message and does message localization.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

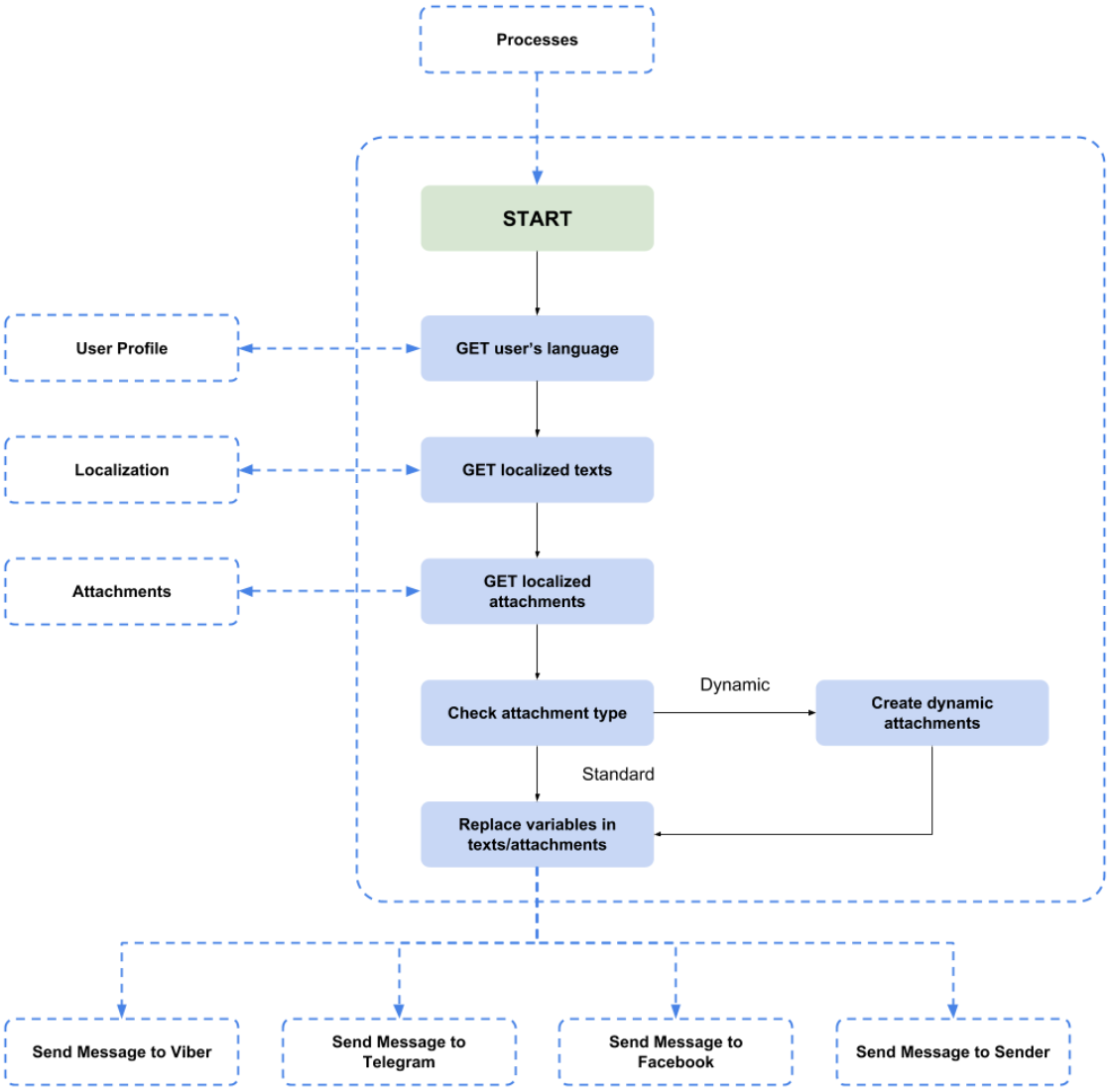


Diagram 27: **Send Message Process flow**.

Send Message Process Overview

1. Process Overview:
- Responsible for sending messages to users across different messaging platforms
- Handles message localization and attachment processing
- Manages both standard and dynamic message attachments
- Distributes messages to specific platform endpoints (Viber, Telegram, Facebook, etc.)
2. Core Process Flow:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

A. Initial Setup:

- Starts with gathering user information
- GET user's language from User Profile
- Retrieves localized texts from Localization store
- Obtains localized attachments from Attachments store

B. Content Processing:

- GET localized texts - pulls appropriate language versions
- GET localized attachments - retrieves platform-specific attachment formats
- Check attachment type - determines if Standard or Dynamic

C. Attachment Handling:

- Two distinct paths based on attachment type:
    - Standard: Direct replacement of variables
    - Dynamic: Creates dynamic attachments with additional processing
- Merges back into main flow after processing

3. Key Components Integration:

- User Profile Integration:
    - Language preferences
    - User settings
    - Personalization data
- Localization System:
    - Text content in multiple languages
    - Message templates
    - Error messages
- Attachments Management:
    - Button templates
    - Rich media content
    - Interactive elements

4. Platform-Specific Distribution:

- Separate handlers for each platform:
    - Send Message to Viber
    - Send Message to Telegram
    - Send Message to Facebook
    - Send Message to Sender (generic)

5. Variable Processing:

- Replace variables in texts/attachments:
    - User-specific data
    - Dynamic content
    - System variables
    - Contextual information

6. Dynamic Attachment Creation:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- Handles complex attachment types
- Processes templates
- Generates interactive elements
- Manages carousel and list formats

7. Key Features:

- Multi-language support
- Platform-specific formatting
- Dynamic content generation
- Template processing
- Variable substitution
- Error handling

8. Message Delivery:

- Platform-specific API integration
- Delivery confirmation
- Error handling and retries
- Rate limiting compliance

Parameter	Type	Required	Description
channel	string	+	Name of a messenger from which a user wrote.
chat_id	string	+	Unique identifier of a user in a messenger.
text_id	string	+	Name of a text object to be sent to a user. The Localization state diagram stores the texts.
attachment_id	string	+	Name of an object which describes an attachment to a text message. The Attachments state diagram stores the attachments

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

items	array	Only for Dynamic attachment	Array of objects for dynamic attachment. The objects are inserted as parameter arguments into a message attachment to be shown as a carousel or keyboard to a user.
currentPage	integer	Only for Dynamic attachment	Each messenger limits the maximum quantity of items shown in a carousel or keyboard. If an `items` array has more objects than allowed, this array is divided into pages (pagination). By default, the initial page number equals `1`.

Table 48: Send message parameters.

Input JSON Example

```
{
	"channel": "viber",
	"chat_id": "12345...",
	"text_id": "main_text",
	"attachment_id": "main_keyboard"
}
```

Input JSON Example (Dynamic attachment)

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

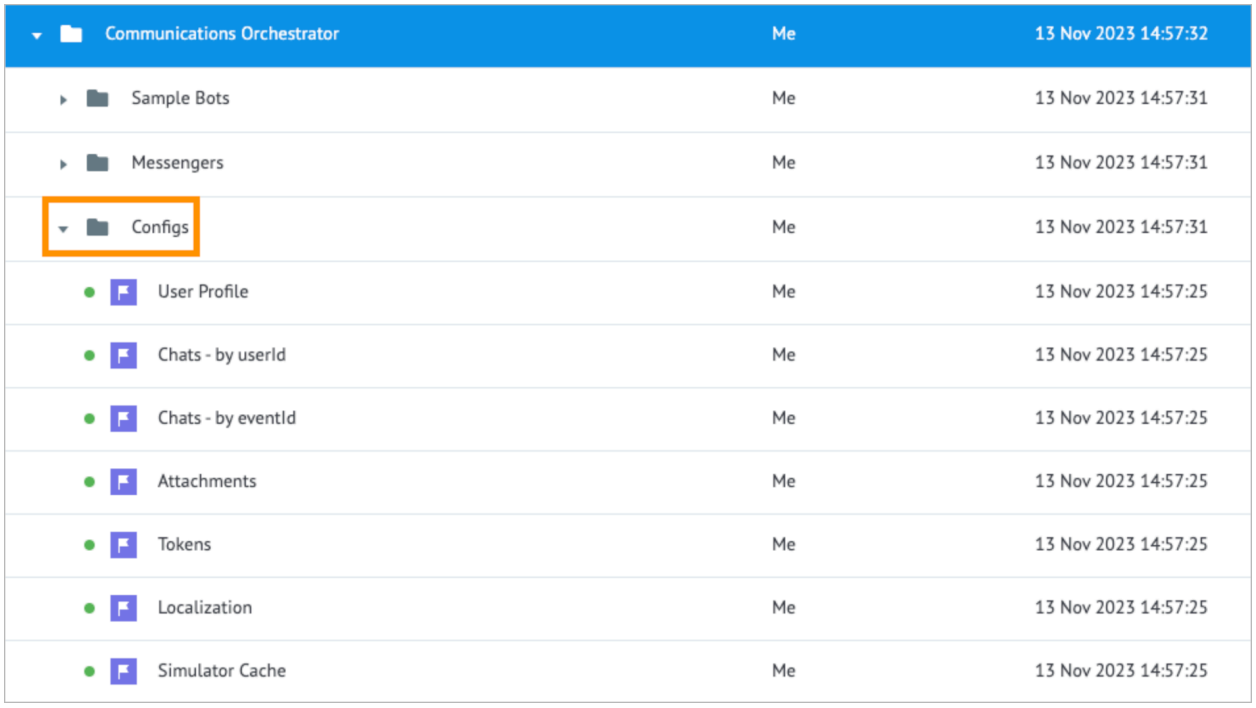
```
{
    "channel": "viber",
    "chat_id": "12345...",
    "text_id": "exchange_rates",
    "attachment_id": "carousel_pattern",
    "items": [{
            "name": "AED",
            "value": 4.283634
        },
        {
            "name": "AFN",
            "value": 85.711652
        },
        {
            "name": "ALL",
            "value": 126.118624
        },
        ...
        "currentPage": 1
    }
```


2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Configs folder

To store data, Communications Orchestrator has the following State Diagrams in the **Configs** folder:

- **User Profile**: Users' profiles received from bots.
- **Chats - by userId**: Saved dialogs with userId reference.
- **Chats - by eventId**: Saved dialogs with eventId reference.
- **Attachments**:Templates of message attachments are stored in JSON.
- **Tokens**: Access tokens to bots in messengers and credentials for integration with the Simulator.
- **Localization**: Texts for content localization.
- **Simulator Cache**: Cache of the main Simulator entities for quick access.



Screenshot 386: **Configs folder**.

The **Communications Orchestrator** Processes interact with the State Diagrams using the dynamic construction for data receiving.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

User profile

This **User Profile** State Diagram stores user data. When a user refers to the bot for the first time, a new user profile is created; then the profile is updated in future activities. A task with user data is created and edited in the **Main** Process using a reference of the strict template:

`{{channel}}_{{chat_id}}`,
where:

- `channel` is the name of a messenger a user referred from.
- `chat_id` is a unique user identifier in the messenger.

Example: `viber_2yOPxC85DSpJCJHpYzjqTw=`

As the `channel` and `chat_id` parameters are required in all the **Communications Orchestrator** Processes, the user data can be received and edited at any step.

Attachments

Messengers support various attachments to messages, such as buttons, keyboards, carousels, and so on. The **Attachments** State Diagram stores templates in JSON format for various attachment types, such as buttons, keyboards, carousels, and lists. These templates are used to dynamically construct and send interactive messages to users.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

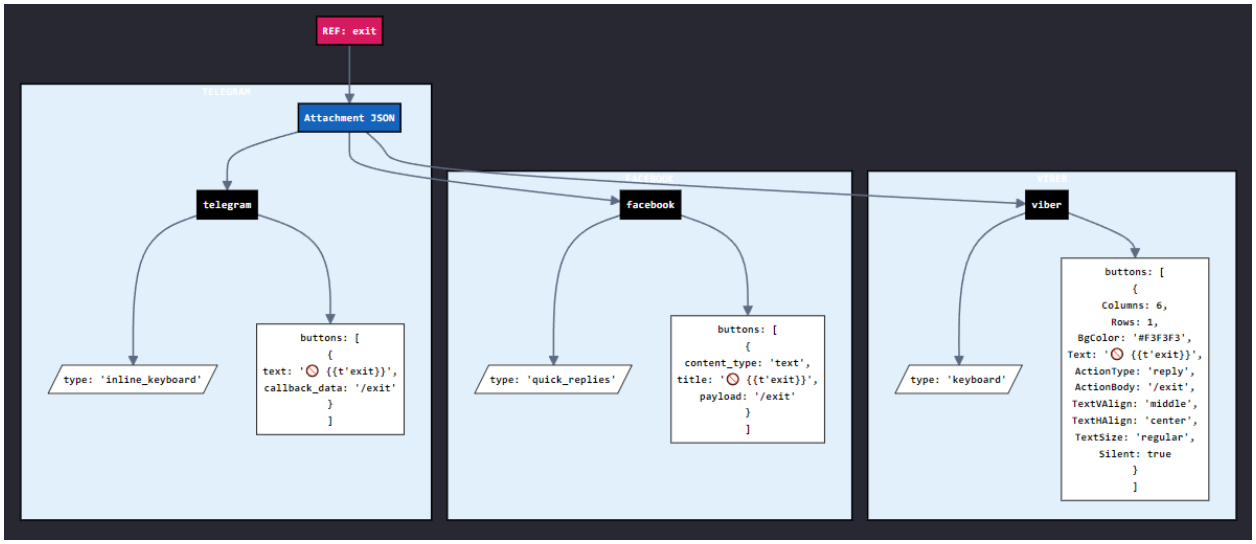


Diagram 28: **Attachments scheme.**

When adding an attachment to the State Diagram, it is recommended to specify the name of the Task Reference according to the target action for which this attachment will be used. For example:

- `mainMenu` for displaying the Main menu
- `exit` for a button to exit the current Process

This value is given in the `attachment_id` parameter when sending a message to a user.

To send a message with an attachment to a user, a bot uses the following logic:

1. The **Send Message** Process uses the dynamic construction `{{conv[{{attachment_state_diagram_id}}].ref[{{attachment_id}}]}}` to receive an object that describes all attachments for all messengers.
2. The attachment parameter gets an attachment that matches the channel parameter.
3. The attachment is localized. Texts from Localization are put instead of all the `{{t'<key>}}` variables.
4. Values are dynamically inserted into the attachment. The values of parameters incoming to the Send Message Process with a task are put instead of all the **Send Message** Process with a Task are put instead of all the `{{param}}` parameters used in the attachment description.

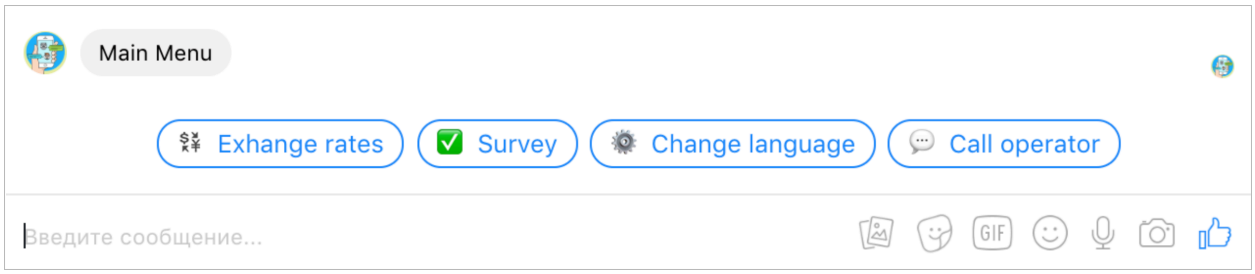
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

5. The composed message is sent to a user.

Examples

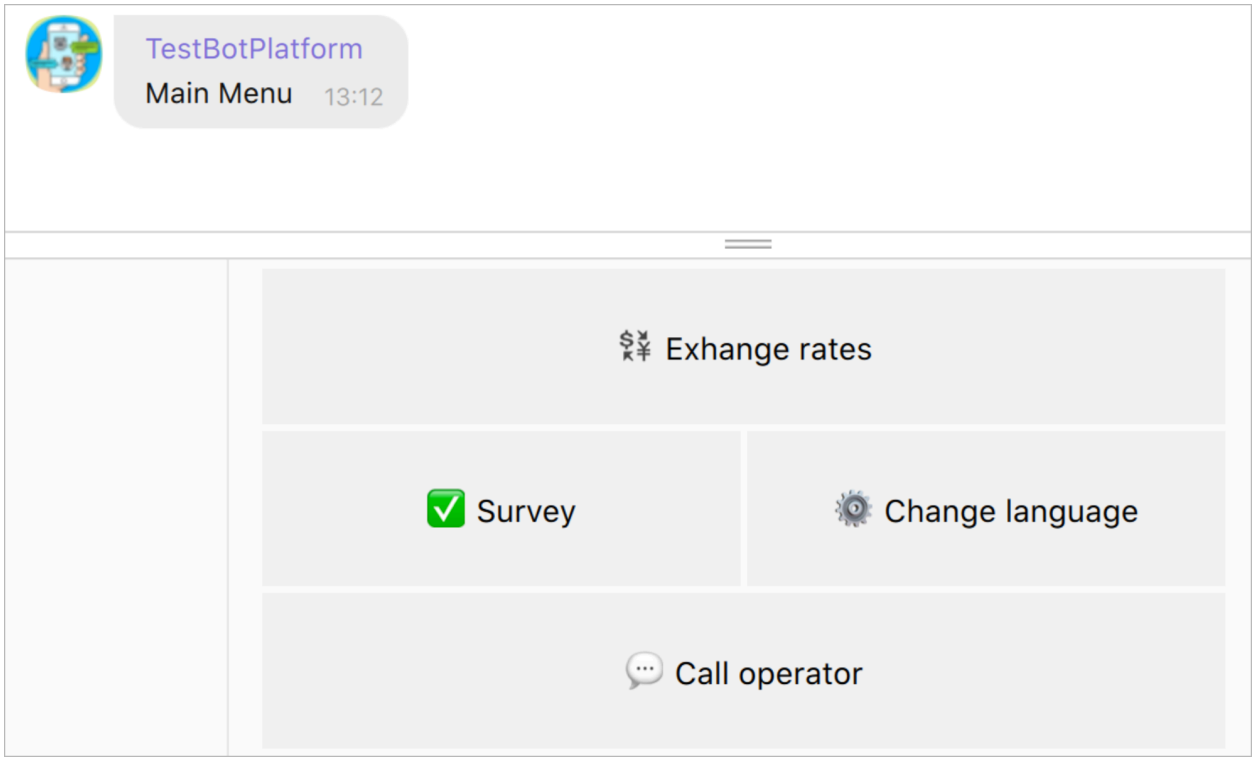
By default, the **Attachments** State Diagram contains ready-made attachments, for example, `mainKeyboard` – the main menu keys:

- Facebook Messenger



Screenshot 387: **Main menu**.

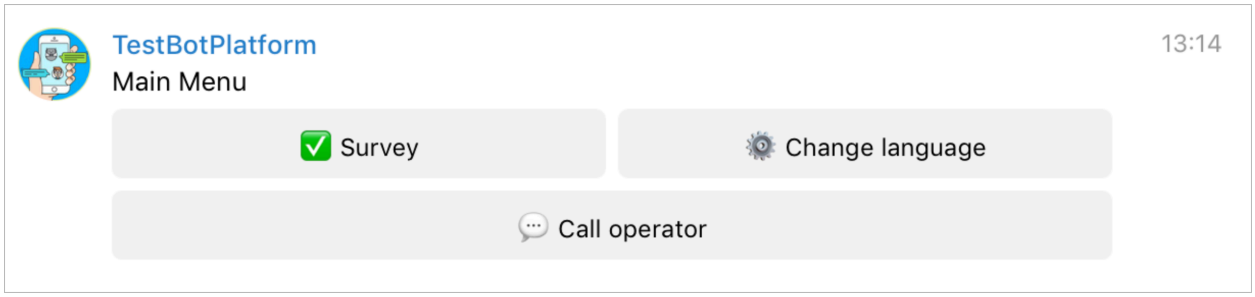
- Viber



Screenshot 388: **Main menu bot options**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

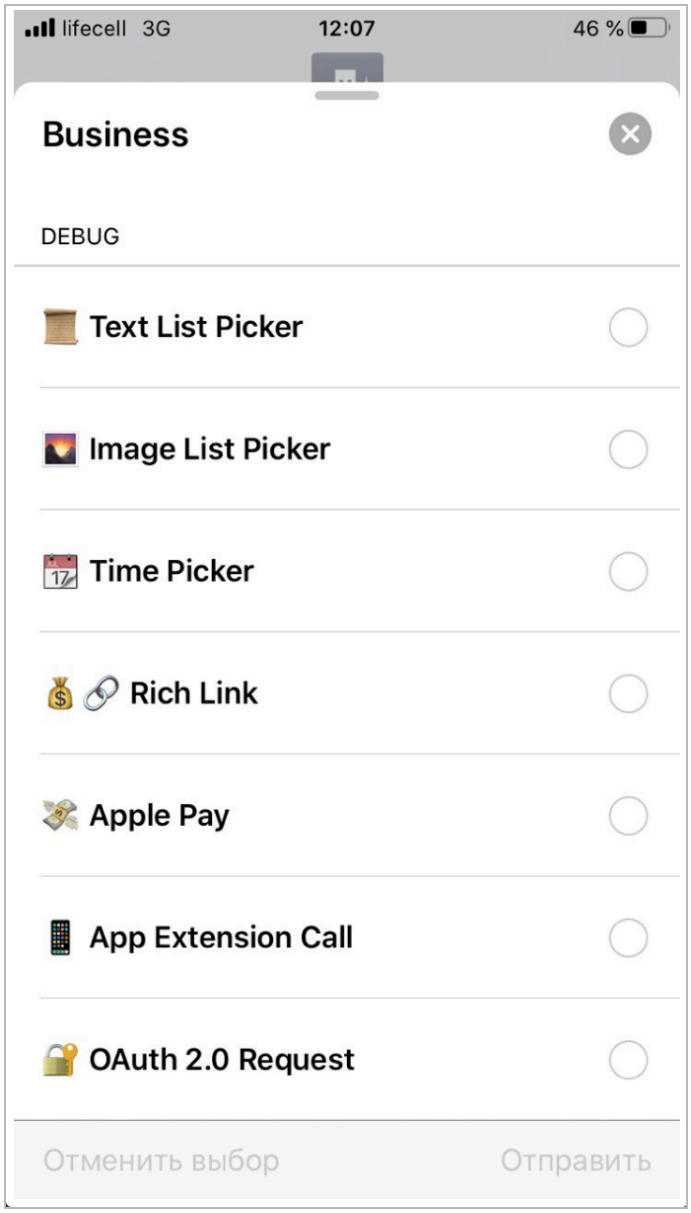
- Telegram



Screenshot 389: **Main menu bot options**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- Apple Messages for Business



Screenshot 390: **Apple messages for business**.

For more information on attachment types, go to the API documentation of each messenger:

- Viber
- Facebook Messenger
- Telegram
- Apple Messages for Business

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Attachment JSON Example:

REF: exit

```
{
    "telegram": {
        "type": "inline_keyboard",
        "buttons": [
            [{
                "text": "🚪 {{t'exit}}",
                "callback_data": "/exit"
            }]
        ]
    },
    "viber": {
        "type": "keyboard",
        "buttons": [{
            "Columns": 6,
            "Rows": 1,
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
            "BgColor": "#F3F3F3",
            "Text": "🚪 {{t'exit}}",
            "ActionType": "reply",
            "ActionBody": "/exit",
            "TextVAlign": "middle",
            "TextHAlign": "center",
            "TextSize": "regular",
            "Silent": true
        }]
    },
    "facebook": {
        "type": "quick_replies",
        "buttons": [{
            "content_type": "text",
            "title": "🚪 {{t'exit}}",
            "payload": "/exit"
        }]
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
    }

}
```

Dynamic Attachment and template

When developing a bot, there is often a need to display data with a homogeneous structure, like a catalog of products, a cart with selected goods, a list of current special offers, and so on which is considered a dynamic attachment. The **Send Message** Process supports the creation of attachments using a template for a varying number of items.
In the example below, you can see how to add a template for displaying exchange rates using an included-in-attachment template.

JSON Example (Dynamic attachment)

```
REF: carousel_pattern

{

    "attachment": {

        "facebook": {

            "type": "carousel",

            "items": [

                {

                    "title": "{{value}} {{name}}",

                    "subtitle": "1 USD"

                }
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
            ]
        },
        "viber": {
            "type": "carousel",
            "carouselRows": "1",
            "carouselColumns": "6",
            "items": [
                {
                    "Columns": 6,
                    "Rows": 1,
                    "ActionType": "reply",
                    "ActionBody": "none",
                    "Text": "1 USD = {{value}} {{name}}",
                    "TextSize": "small",
                    "TextVAlign": "middle",
                    "TextHAlign": "middle",
                    "Silent": true,
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
                    "BgColor": "#FFFFFF"
                }
            ]
        },
        "abc": {
            "type": "text_list_picker",
            "maxItemsCountToShow": 10,
            "items": [
                {
                    "identifier": "{{name}}",
                    "order": 0,
                    "style": "small",
                    "subtitle": "per 1 EUR",
                    "title": "{{value}} {{name}}"
                }
            ],
            "order": 0,
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
            "title": "💱 {{t'/exchangeRates}}",
            "multipleSelection": false,
            "receivedMessage": {
                "style": "small",
                "subtitle": "",
                "title": "💱 {{t'/exchangeRates}}"
            },
            "replyMessage": {
                "style": "small",
                "title": "💱 {{t'/exchangeRates}}",
                "subtitle": ""
            }
        }
    }
}
```

There is no template for Telegram because Telegram does not support this kind of message.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Items Sourcing Array

For creating a carousel, an array is needed for providing values to the template:

```
"items": [

    {

        "value": "4.183966 AED",

        "name": "1 USD"

    },

    {

        "value": "85.890287 AFN",

       "name": "1 USD"

    }

]
```

Parameters for Send Message Process

Calling the **Send Message** Process requires passing the following parameters:

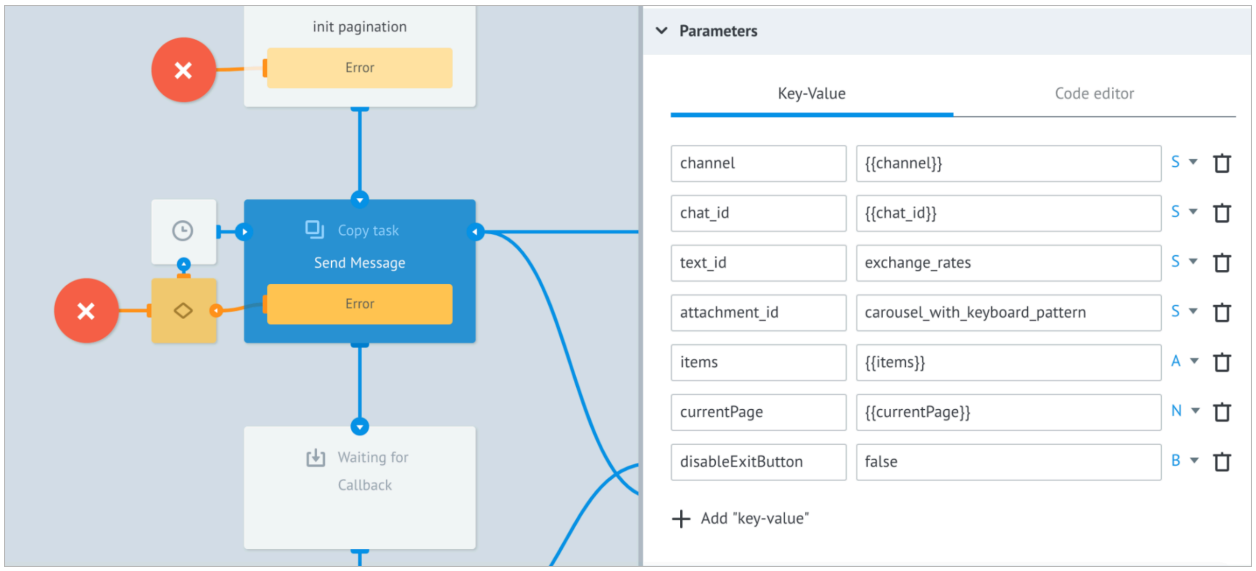
```
{

        "attachment_id": "carousel_pattern"

        "items": "items"

        "currentPage": 1
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

`"disableExitButton": false|true (a flag to show the Exit button)`

- `}`

A Code node named **createDynamicAttachment** performs all other necessary actions over an object.



Screenshot 391: **The createDynamicAttachment Code node**.

The **Send Message** Process considers messenger limitations and does corresponding pagination automatically: it breaks the array into pages and adds the navigation buttons. The **Exit** button is added if `disableExitButton = false`.

Converted JSON:

```
"message": {

    "quick_replies": [{

        "content_type": "text",

        "title": "🚪 Exit",
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

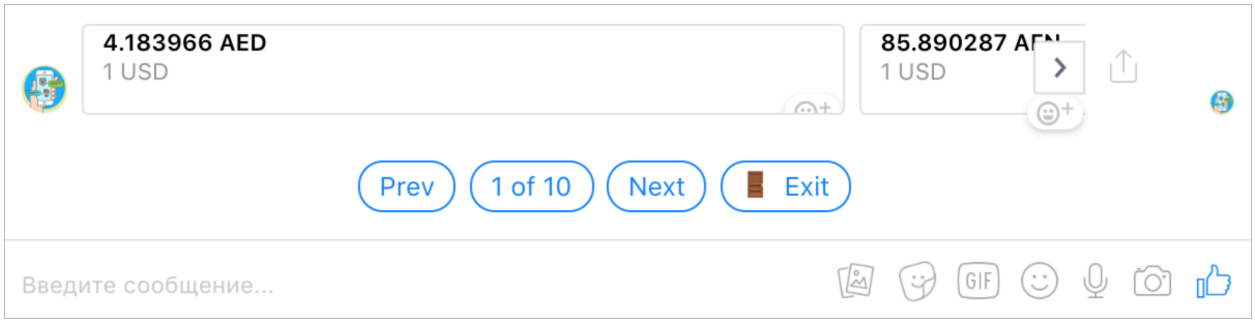
```
        "payload": "/exit"
    }],
    "attachment": {
          "type": "template",
          "payload": {
              "template_type": "generic",
              "elements": [
                 {
                     "title": "4.183966 AED",
                     "subtitle": "1 USD"
                 },
                {
                   "title": "85.890287 AFN",
                    "subtitle": "1 USD"
              },
              ...
           ]
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
        }

    }

        }
```

Examples of Displaying:

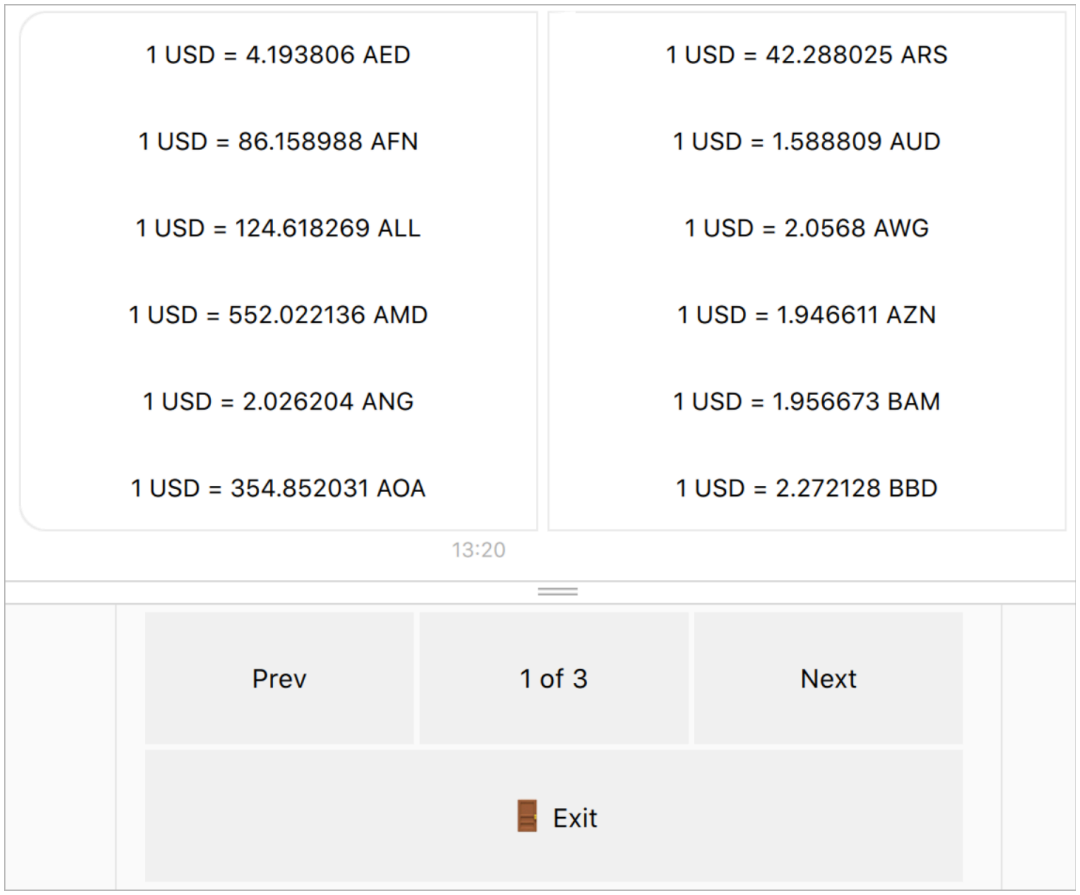
- **Facebook Messenger**



Screenshot 392: **Facebook Messenger**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

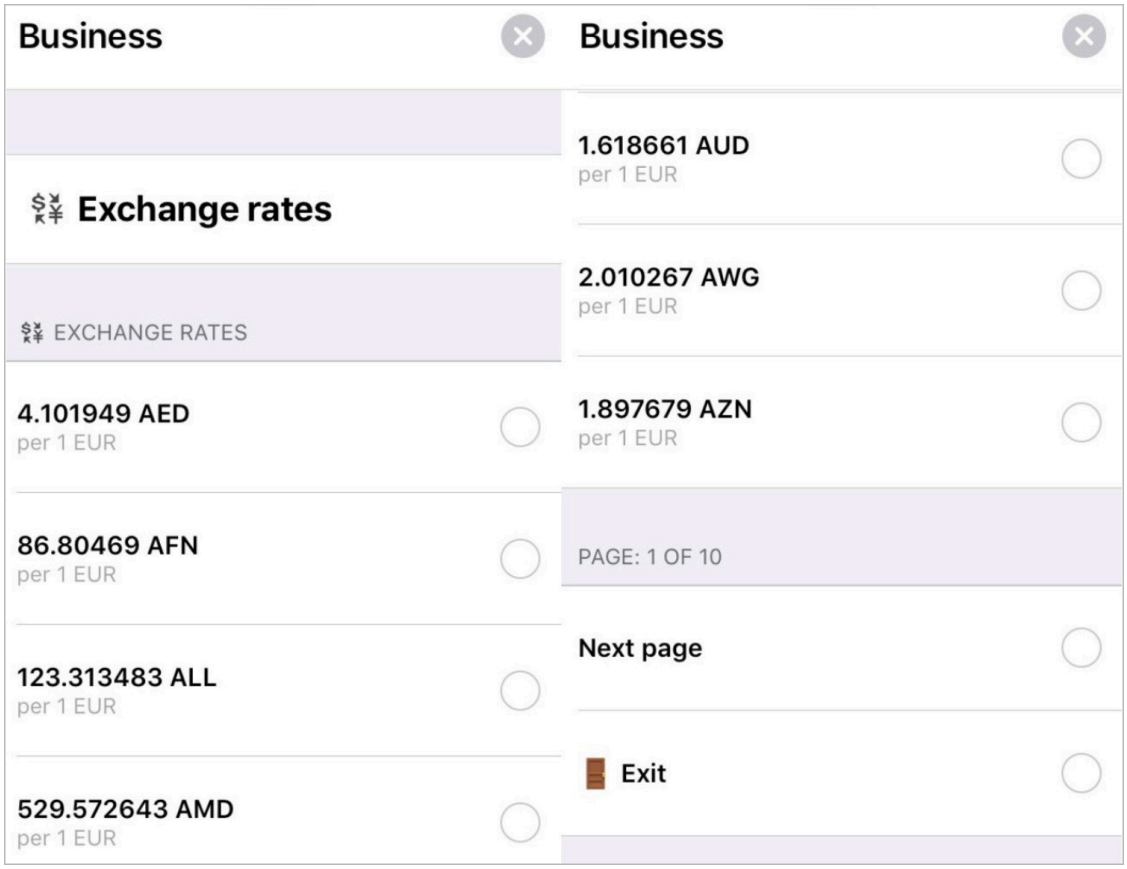
- **Viber**



Screenshot 393: **Viber exchange rate menu**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- **Apple Business Chat**



Screenshot 394: **Apple business chat exchange rate menu**.

Tokens

The **Tokens** State Diagram stores bots access tokens and credentials for integration with Simulator.

JSON Example:

```
REF: token

{

"abc": "",

"facebook": "",
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
"simulator": {
    "token": "",
    "developersGroupId": "",
    "id": "",
    "accId": "",
    "baseUrl": "https://api.simulator.company/v/1.0"
},
"slack": "",
"telegram": "",
"viber": ""
}
```

Localization

The **Localization** state diagram stores all texts of all messages and attachments in one Task:

Task example:

```
REF: localization
{
    "/exit": {
        "en": "Exit",
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
        "ru": "Выход",
        "uk": "Вихід"
    },
    "mainMenu": {
        "en": "Main Menu",
        "ru": "Главное меню",
        "uk": "Головне меню"
    },
    "no": {
        "en": "No",
        "ru": "Нет",
        "uk": "Ні"
    },
    "yes": {
        "en": "Yes",
        "ru": "Да",
        "uk": "Так"
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
    },

    ...

}
```

The approach allows you to:

1. Manage all bot texts from one place.
2. Send messages by only specifying a **`key`** of the necessary text in the **`localization`** object.
3. Localize bot interface. By default, the texts are in the **`en`**, **`ru`** and **`ua`**, languages, but any language can be added.

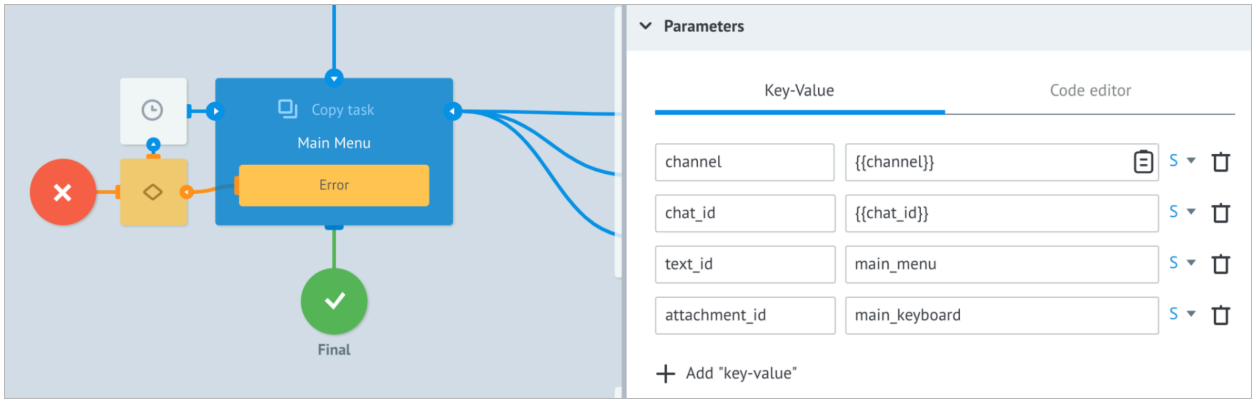
To send a text message, specify the key name `key` from the `localization` task as a value of the `text_id` parameter:
1. Use the `text_id` parameter and set its value to a key name that exists in the localization task/object. This allows the system to look up the appropriate localized text.

2. The actual message sending requires providing several parameters to the Send Message Process (see the example below).
3. The system will automatically:

- Look up the text in the localization object using the provided key
- Send the message in the user's preferred language (stored in User Profile State Diagram)
- Default to English ('en') if no language preference is set
- The language can be changed using the Set Parameters node named "SET default language == EN"

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Example. To send the Main menu to a bot, you need to provide the **Send Message** Process with the following parameters and values:



Screenshot 395: **Send Message Process**.

```
{
    "attachment_id": "mainKeyboard",
    "channel": "viber",
    "chat_id": "...",
    "text_id": "mainMenu"
}
```

For localization, attachments should have their parameters of UI texts (button names, labels, etc.) to meet the `{{t'<key>}}` template, where `<key>` is the key of a necessary text in the `localization` object.

Example. Localization of the **Exit** button:

```
{
    "content_type": "text",
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
    "title": "🚪 {{t'/exit}}",

    "payload": "/exit"

}
```

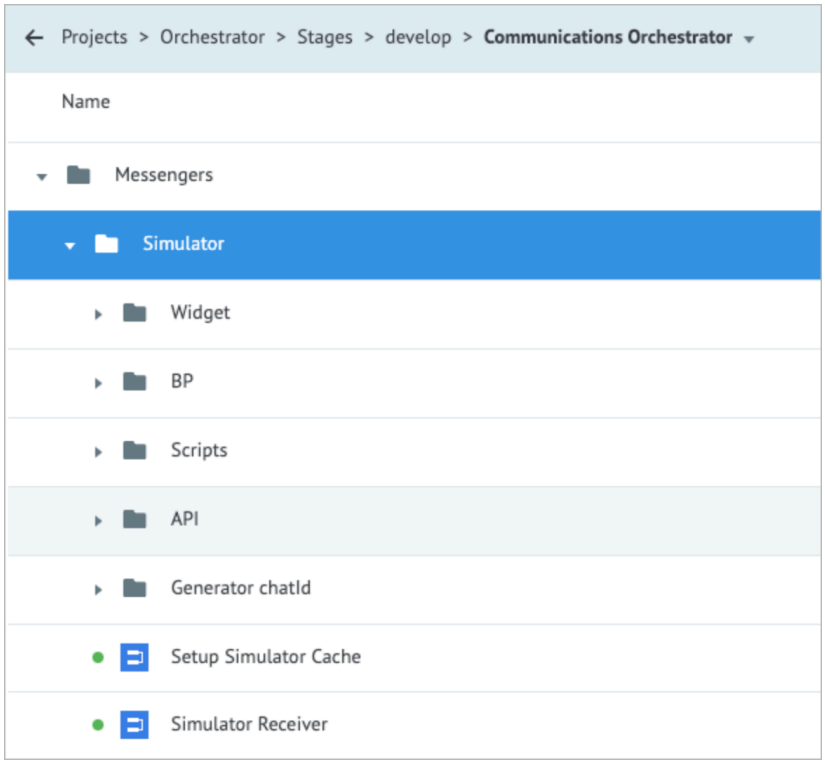
The **Send Message** Process sends a message in the same language as the one stored for a user in the **User Profile** State Diagram. The default language is `en` and can be changed in the **Set Parameters** node named ***SET default language == EN*** of the **Send Message** Process.

Simulator folder

Overview

The **Simulator** folder path:
Stages → Stage name → Communications Orchestrator → Messengers → Simulator folder.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 396: **Communication Orchestrator Simulator folder**.

A solution for organizing live chat between a client and a human operator is built on the Simulator within the Communications Orchestrator. The **Simulator** folder stores:

- Processes for the operator's workspace to handle dialogs
- Chat history, client profiles, analytics, and dashboards
- **BP:** business processes that you can reconfigure:
    - **bp > chats > freeze chat**: The Process that handles the freeze event in dialogs = dialog closure. In this Process, transactions against customer and operator accounts are made for analytics collection and the change of dialog connections with streams.
    - **bp > chats > create employee > create dashboards**: The Process starts with the **Creation of the new employees actor** event and creation of new dashboards for a new operator.
    - **bp > chats > send file from operator to client**: In this Process, sending files from an operator to a customer is performed.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

    - **bp > chats > update agent status**: If the **Status** field is altered, the Process starts when the **Update actor employees** event occurs. In this Process, transactions against operator accounts are made to track the time the operator was in each status.
    - **bp > chats > done chat**: In this Process, the **Done** reaction in the operator chat is handled.
    - **bp > chats > get available chatId**: The Process retrieves a free new chatId from a queue.
- **Scripts**: Stores the Processes for each script, which handles all requests from the Smart Forms.
- **Widget**: Widget Processes.
- **Generator chatId**: Processes for generating a unique **chatId**.
- **Setup Simulator Cache**: The process starts automatically when the Communications Orchestrator is created. It goes through all the entities created in Simulator and unites them into a single cache object. This object is stored in the Simulator Cache State Diagram and is used in most processes for quick access to Simulator objects.

Simulator entities in folder

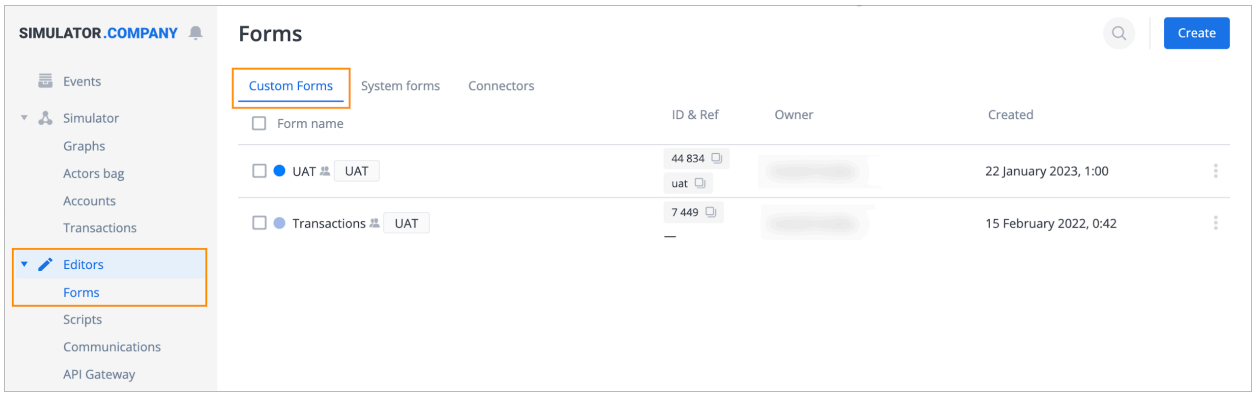
After creating a Communications Orchestrator in Corezoid, a set of necessary forms, actors, and accounts for orchestrating messages between the client and the operator, as well as for collecting analytics, is imported into your workspace in Simulator:

- Forms
- Actors
- Events
- Dialog components
- Dashboards
- Widgets
- Scripts

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Forms

To access forms, go to **Editors** → **Forms** → **Custom Forms**.



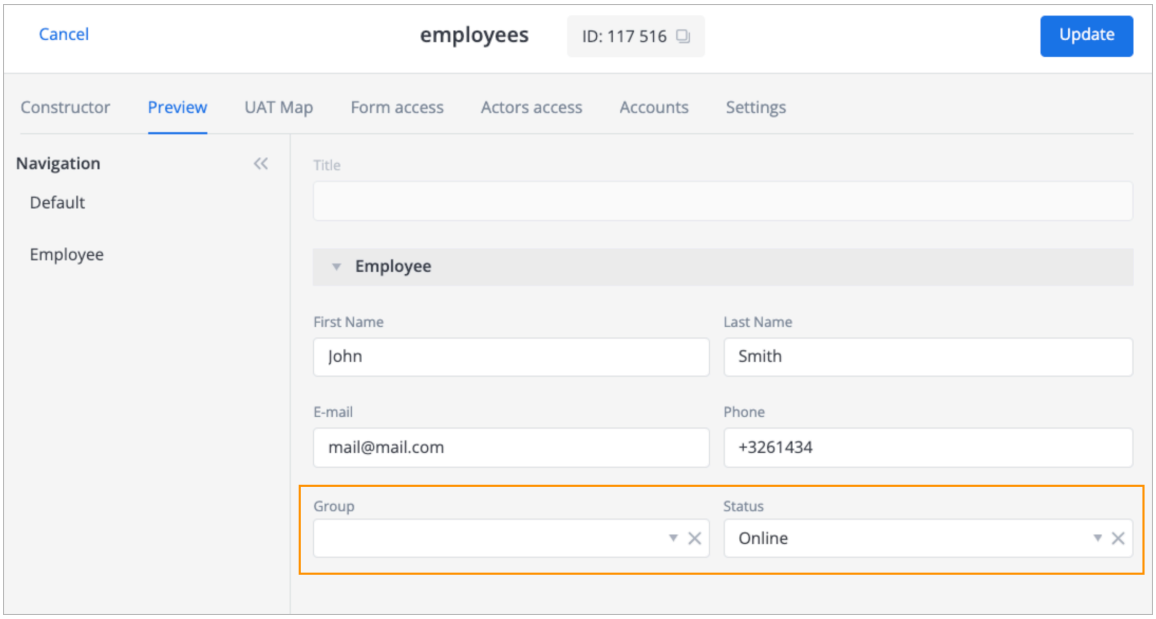
Screenshot 397: **Forms in Communications Orchestrator**.

With forms, you can describe the data structure for uniform objects (actors). Forms are very flexible, and their model can be changed or extended as needed. Any attributes described in the form can be used to build filters or selections of actors. The following form types were created:

- **employees**: Describes the data structure of employees (operators, supervisors, and so on):
    - The Group field refers to the filter for all actor groups and describes the structure of groups. Groups can be used to combine several employees and apply any mass actions to the group.
      Note: Groups are added as an example and are not used by default in the logic of Communications Orchestrator.

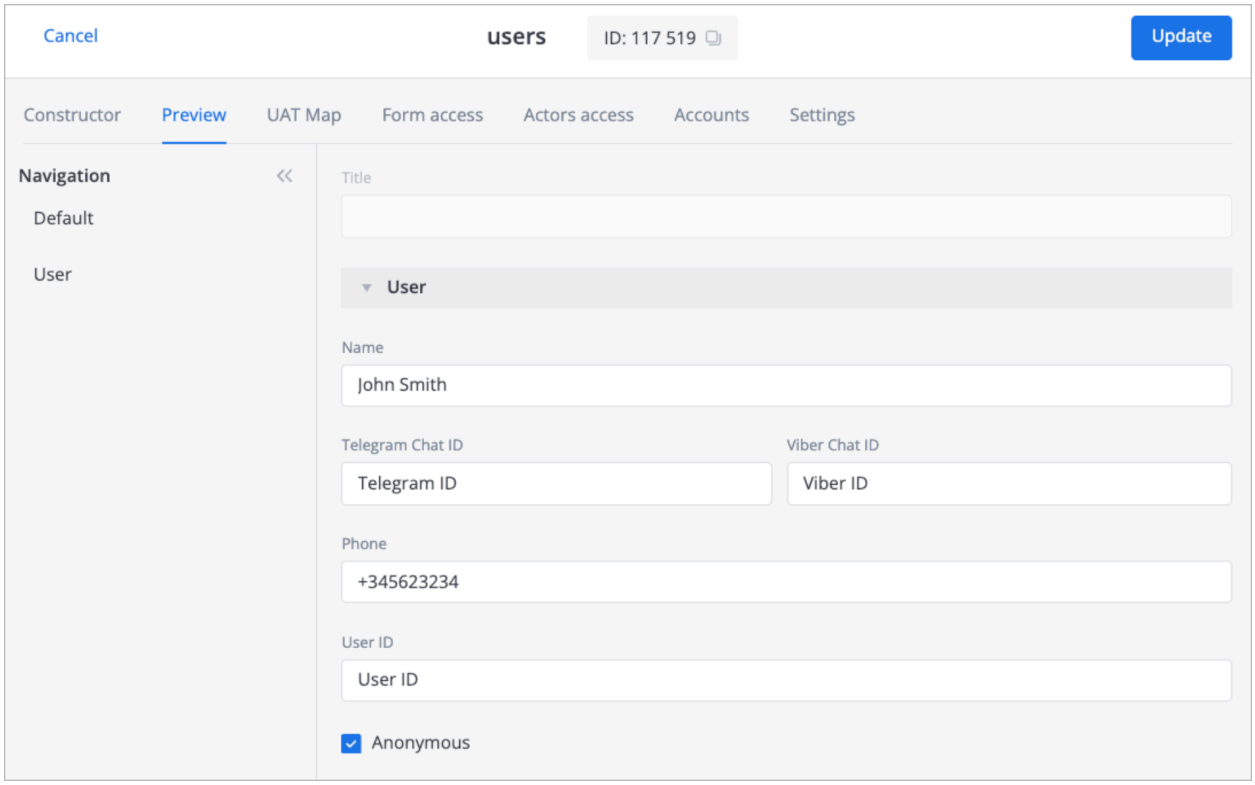
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- The **Status** field refers to the filter for all operator statuses.



Screenshot 398: **Status field in the Form**.

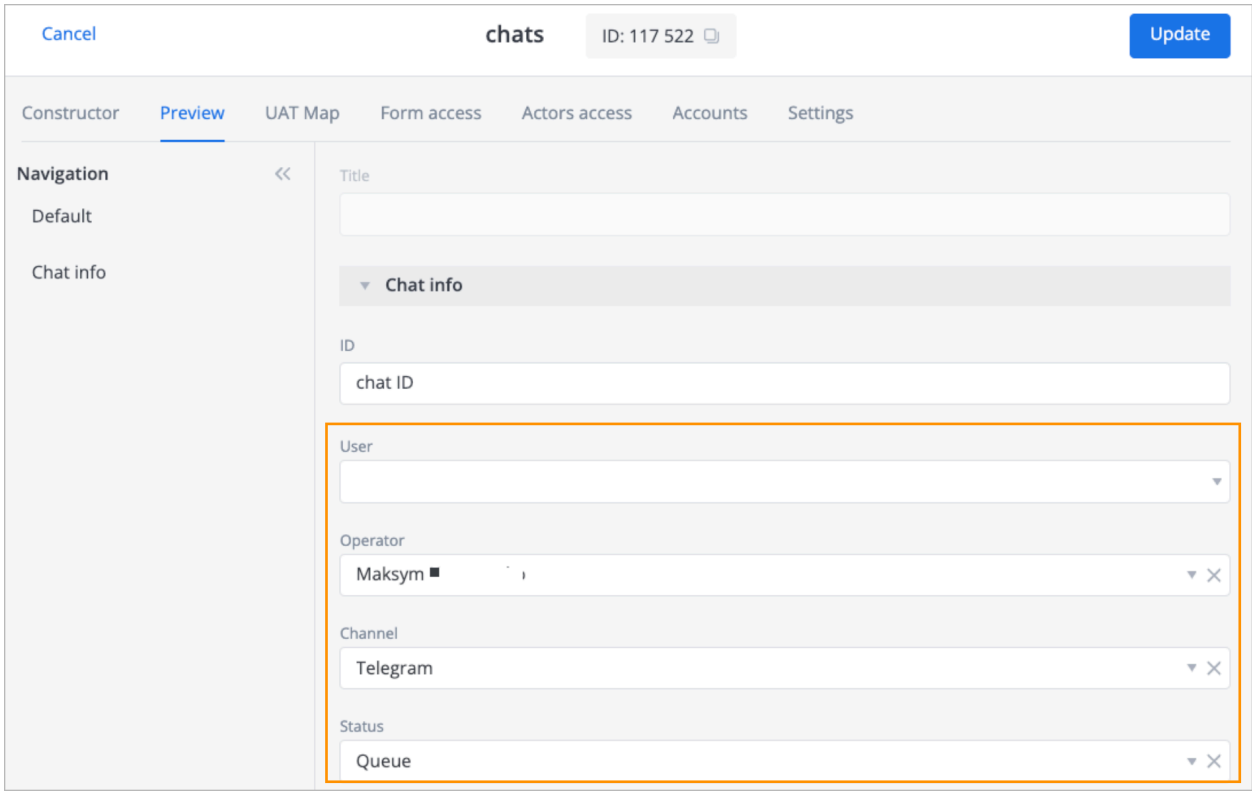
- **users**: Describes the structure of clients' data from various channels.



Screenshot 399: **User data from various channels**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- **chats**: Describes the structure of chats data:
    - The **User** field refers to the filter by all user actors.
    - The **Operator** field refers to the filter by all employee actors.
    - The **Channel** field refers to the filter by all channel actors.
    - The **Status** field refers to the filter by all chatStatuses actors.



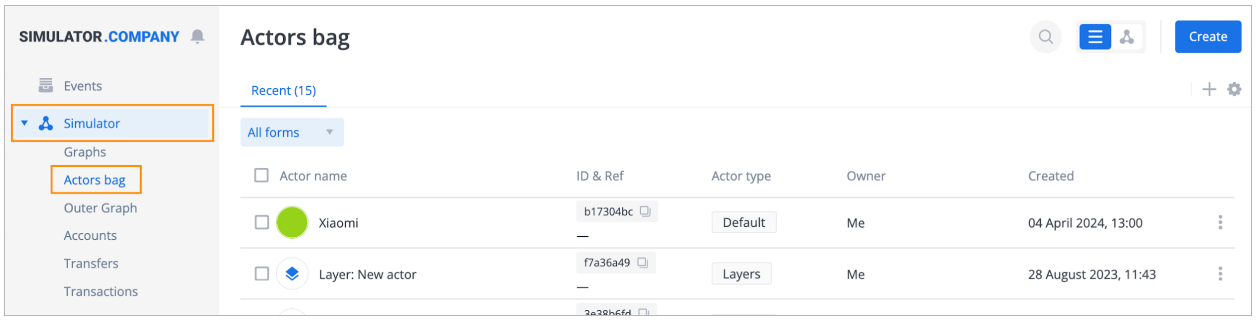
Screenshot 400: **Chat fields in the Form**.

- The **channels**, **chatStatuses**, and **operatorStatuses** forms serve as dictionaries. You can create as many dictionaries as needed and use them in your Processes.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Actors

To access actors, go to **Simulator** → **Actors bag**.



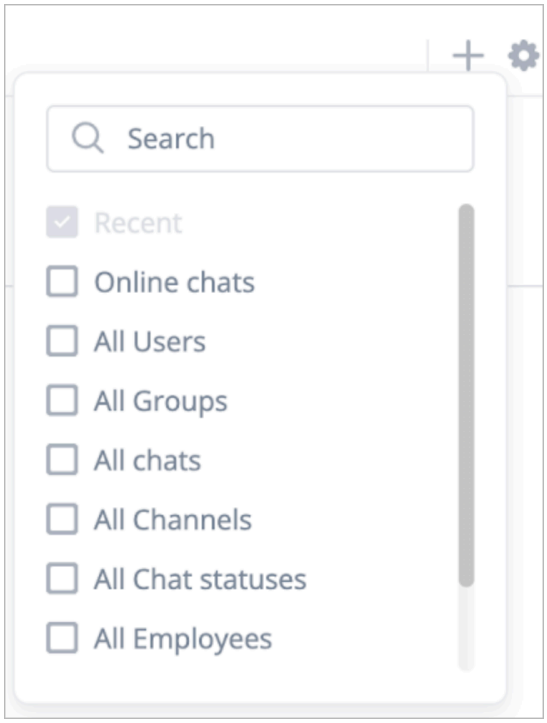
Screenshot 401: **Actors bag page**.

Actors bag is a place where we work with the actual actors. An actor can be a specific operator, a specific client, a specific chat, and so on.

On the **Actors bag** page, you can do the following:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- Apply filters or configure the needed ones.



Screenshot 402: **Applying filters**.

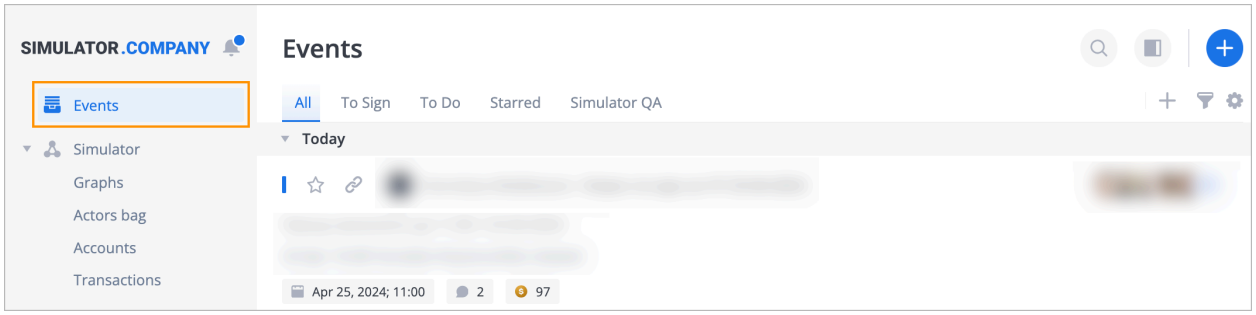
- Create new actors, view actor details, edit an actor, and so on.
  Note: To create an actor of the employee type in the form, turn the Optional fields toggle on and choose the corresponding user in the Single account user field, who is registered in the system. Doing this establishes a connection between the actor and the actual user in the system. A user can only be associated with one actor in the workspace at a time. This action is necessary for the logic of distributing dialogs between operators, which you can configure in the /startChat Process. When creating a Communications Orchestrator, one employee actor is created per the user who created the Communications Orchestrator.
- View actor's accounts. Accounts can be used to track any action, event, or other changes. After creating Communications Orchestrator, several accounts were created for clients and operators, where the following are accounted for:
  - The number of chats per operator (active / total)
  - Ratings given by clients to the operator
  - The time that the operator spent in different statuses.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Correspondingly, in Corezoid, Processes are built that respond to triggers to create transactions and update account balances. Account balances can also be used in business logic. For example, the activeChats account on actor-employees is involved in the logic of distributing dialogs between operators. If you need to start keeping track of anything else, simply open an account for the corresponding type of actors, specify the currency - what to track, and build Processes to create transactions. For more information, go to the /startChat Process.

Events

To access events, go to **Simulator** → **Events**.



Screenshot 403: **Events page**.

The **Events** page is the main working space for agents handling dialogs.
There are two modes for viewing the list of events: **Regular** and **Split**. For working with multiple dialogs simultaneously, we recommend using the **Split mode**. It allows you to switch between conversations with one click while seeing the full picture of your dialogs: the appearance of new dialogs, new reactions in current dialogs, and so on.
You can distribute events (dialogs) to different stream tabs. Stream tabs are a feature on the Events page that allow users to organize and view events in different streams. Key Features:

- Users can create custom stream tabs to organize events into different categories
- Stream tabs appear on the right side of the existing system tabs
- There are default system tabs that cannot be reordered: All, To Sign, To Do, and Starred
- Users can reorder custom stream tabs by dragging and dropping them

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

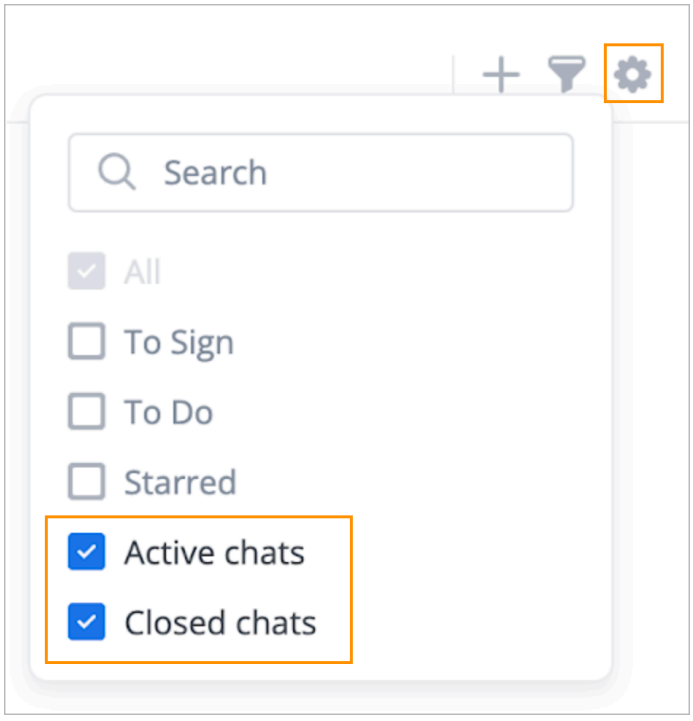
- The selected order of stream tabs is saved

Adding an event to a stream tab equals creating a connection (edge) between event actors and stream actors. There are three system stream tabs: **To Do**, **To Sign**, and **Starred**.

To stop displaying system stream tabs and enable the display of the ones you need, click the settings icon and select the checkboxes for the stream tabs you want to display.

After creating the Communications Orchestrator, two new stream tabs are added:

- **Active chats**: New dialogs get listed in this stream and remain there until they are closed. The logic for adding a new dialog to this stream is configured in the **/startChat** Corezoid Process.
- **Closed chats**: These are dialogs moved from the **Active chats** stream tab immediately after the chat status changes to **Closed**.



Screenshot 404: **Active and Closed chats streams**.

The same dialog can be maintained in multiple stream tabs simultaneously. You can create new stream tabs and configure the needed logic for adding dialogs to them, for example:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- Streams based on the themes of inquiries or based on the channels of inquiries
- Streams based on the duration of the dialog

Dialog components

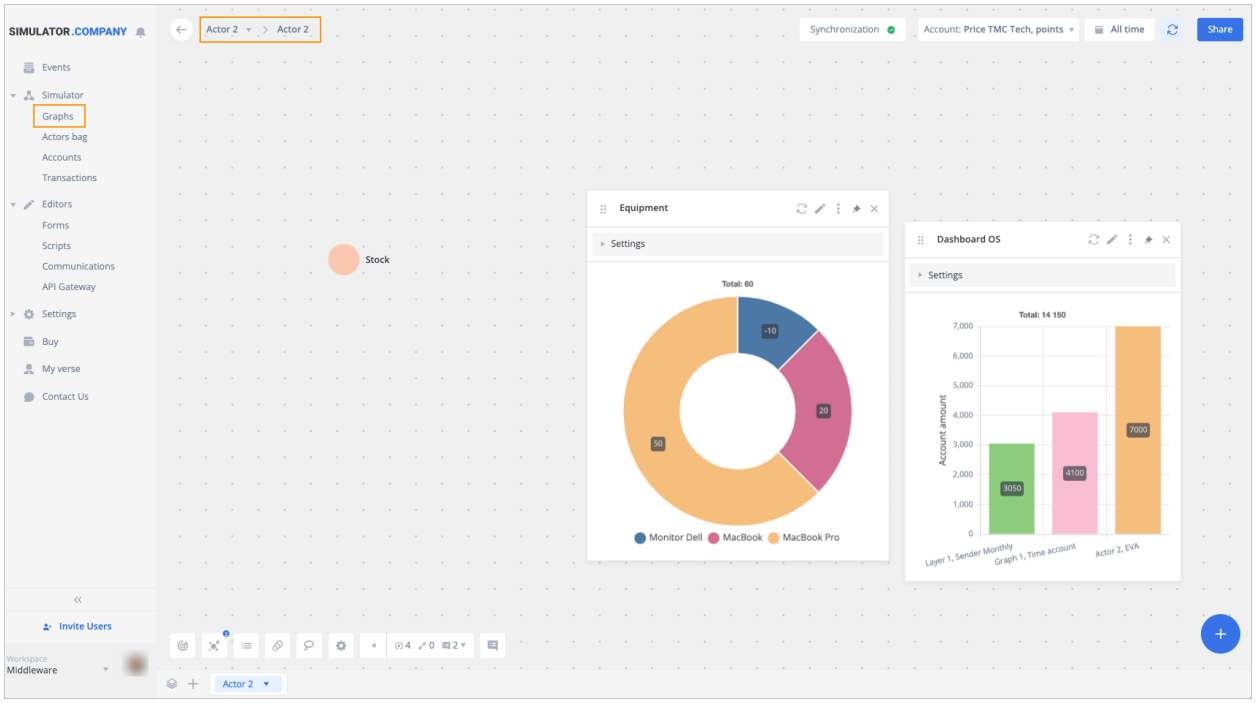
The dialog has the following components (actors, parameters, Smart Forms, snippets and Process logic):

1. **Dialog name**: A dialog can have any name and has the following dynamic structure: **{channel} - {chatId} - {userName}**
   Where:
   - **channel**: The channel of the inquiry.
   - **chatId**: A unique chat ID generated in the **chatId** generator Process. The algorithm can be modified.
   - **userName**: Username retrieved from a user actor.
2. **Card of an actor**: A card of a chats actor, which characterizes the dialog itself, is added to the dialog.
3. **Shared with**: indicates who has been added to the dialog and with what roles. The operator can manually share the dialog with another operator or a group.
4. **Linked actors**: A list of actors associated with the dialog. Linking the dialog to actors is necessary for displaying the selected dialog in the history of events for these actors. Currently, history is linked to actors in:
   - Stream tabs: **activeChats** and **closedChats**.
   - Employees: The actor of the operator who is handling the dialog.
   - Users: The actor of the client who contacted support.
5. **Done**: The action is linked to the logic of completing the dialog from the operator's side. The **Simulator Receiver** Corezoid Process tracks the dialog completion event and triggers the **/NPS** Process.
   Note: You can modify this logic.
6. Scripts: Are run within the dialog with the option to specify the recipients.
7. Snippets: Allow the fast use of predefined phrases.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Dashboards

To access dashboards, go to **Graphs** → **Dashboards**.



Screenshot 405: **Dashboards view on the Graphs page**.

Using analytics with dialogs

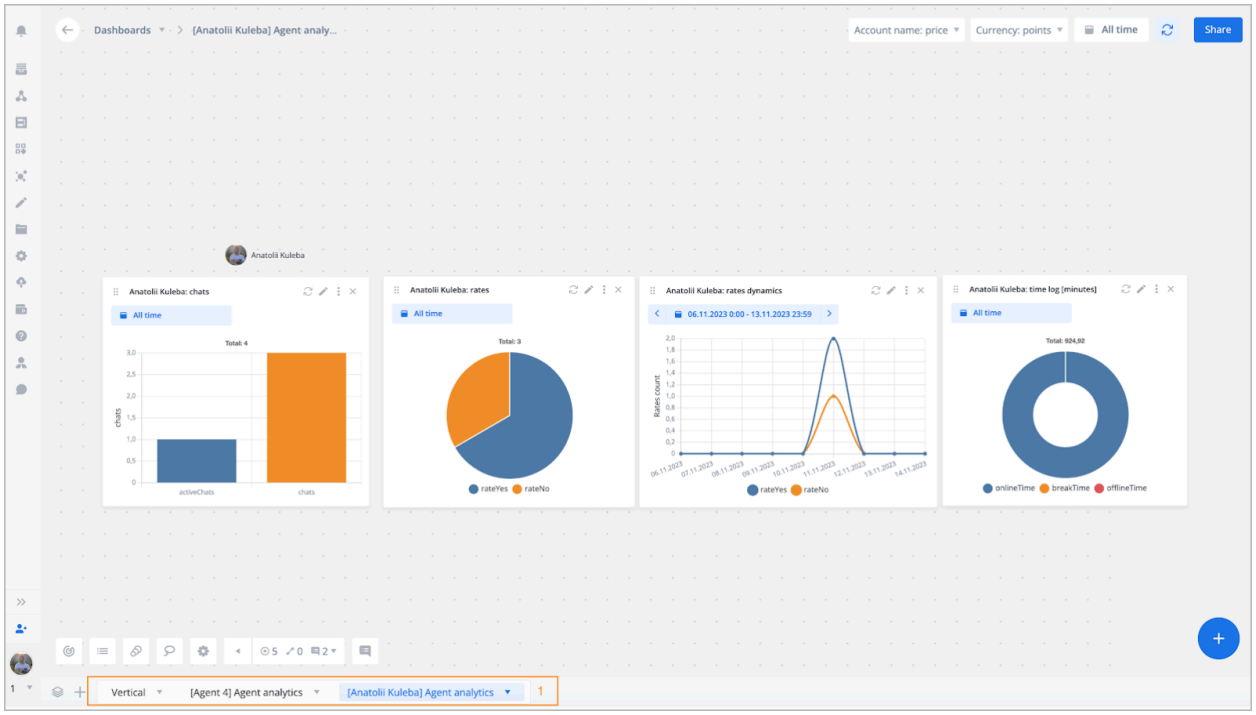
When account balances are tracked for actors such as operators or users, on the dashboards, you can visualize the balances or transaction dynamics.

Configure Dashboards

The architecture of Communications Orchestrator inherently includes the logic for automatically creating four dashboards as soon as an employee actor is created. A separate layer is created for each employee with metrics specific to them. This logic is built in the **bp** → **chats** → **create employee** → **create dashboards** Process and can be changed or expanded.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

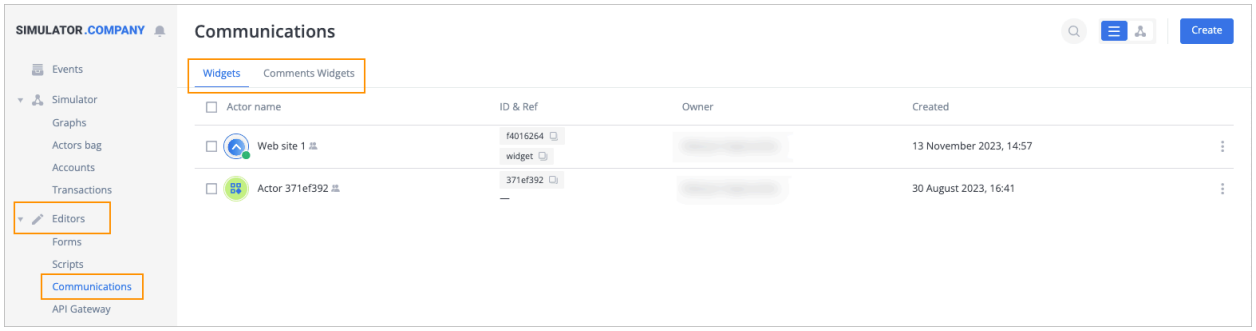
To switch between the layers, use the bottom panel.



Screenshot 406: **Switching between layers**.

Widgets

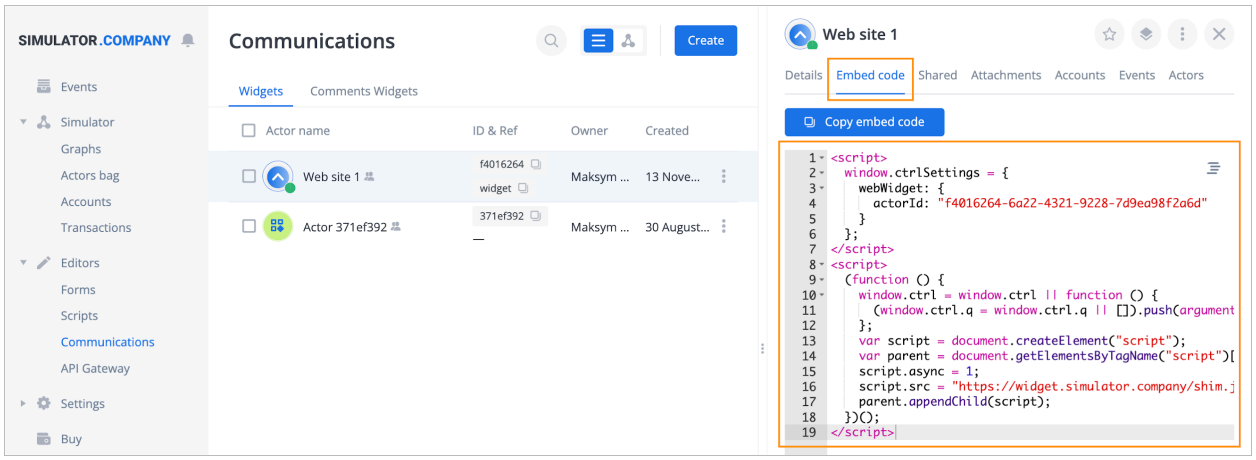
To access widgets, go to **Editors** → **Communications** → **Widgets** or **Comments Widgets**.



Screenshot 407: **Widgets**.

After the Communications Orchestrator has been created, a widget actor is created, which you can access on the **Communications** page. Click the widget, and on the **Embed code** tab, you can obtain the embed code for the widget and add it to your website or any test environment.

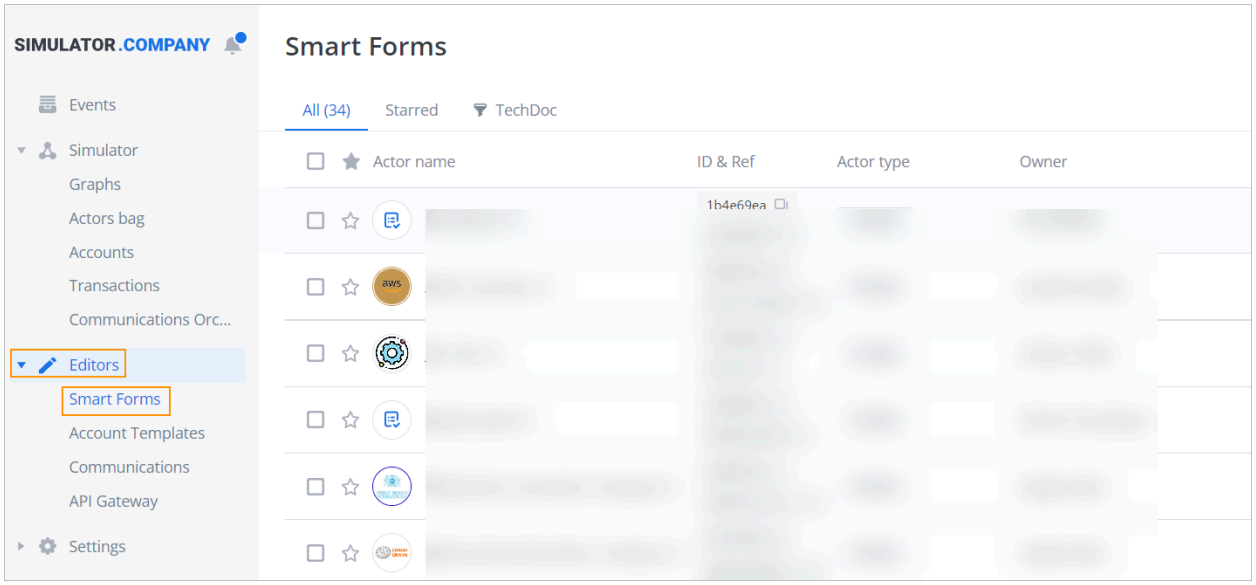
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 408: **Embed code**.

Scripts

To access Smart Forms (former Scripts), go to **Editors** → **Smart Forms**.

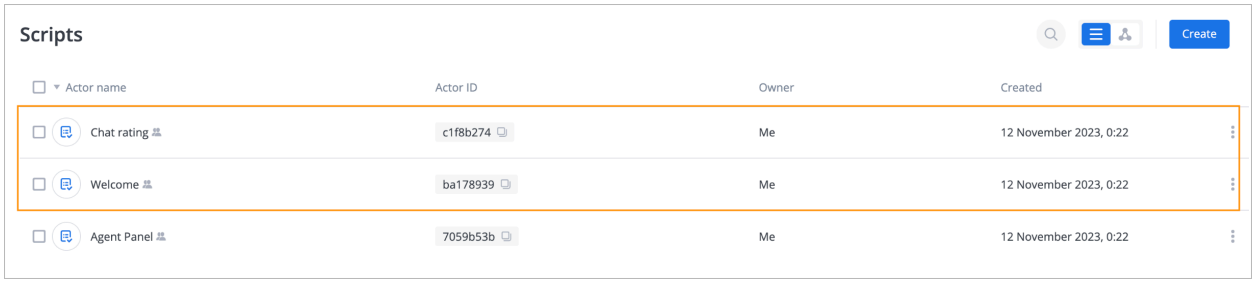


Screenshot 409: **Smart Forms (former Scripts)**.

The widget has the following Smart Forms:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- Together with the widget, the **Welcome** and **Chat rating** Smart Forms were created.



Screenshot 410: **Welcome and Chat rating Smart Forms.**

- **Start Script**: Executes when the widget is launched.
- **Final Script**: Executes when the user clicks the close button (X) in the widget.

The configuration of the Smart Forms and their execution logic can be modified.

In Corezoid, Processes for interacting with Smart Forms are located in the **Communications Orchestrator** → **Messengers** → **Simulator** → **Scripts** folder.

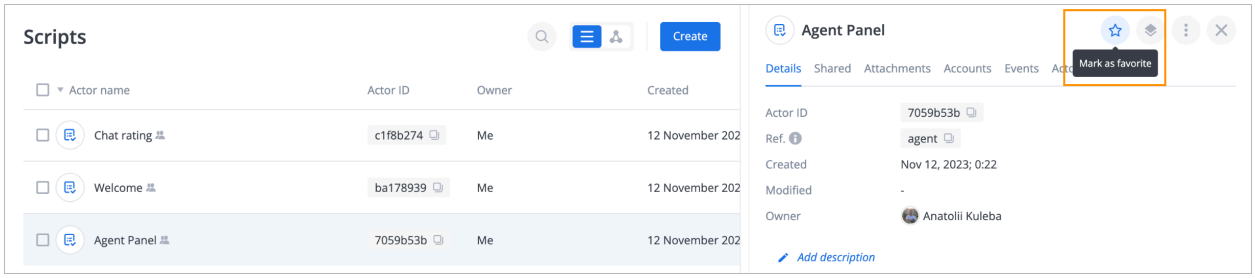
To use Smart Forms:

1. After creating Orchestrator, replace Corezoid credentials in the script settings with your own.
2. Create an API Key in Corezoid and grant Task management access to this API Key for each main receiver script (Process “script - {scriptName} - main receiver”).
3. Fill in the corresponding parameters in the settings of each script: API login, API secret, Company ID, Dev process ID, and Prod process ID.
   Note: You can specify the same Dev and Prod Processes if needed.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Agent panel Smart Form

- Is used for changing the operator's status to avoid giving operators direct access to actors-employees.
- You can add the script to favorites to add it to the side panel (it appears on the panel after refreshing the page).



Screenshot 411: **Welcome and Chat rating Smart Forms.**

- Every change in the operator's status is logged on the respective accounts and displayed on dashboards.
- The logic is configured in the Process **bp** → **chats** → **update agent** status.
- When adding new statuses, modify this Process accordingly.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

8. Tutorials

Google OAuth

Google OAuth 2.0 is an authorization protocol that allows a service (application) to be granted access rights to user resources on another service. The protocol can prevent an application to be entrusted with login and password data and grant only a restricted set of rights rather than all of them at once.

Before setting up Google OAuth 2.0, review Using OAuth 2.0 to Access Google APIs for essential background.

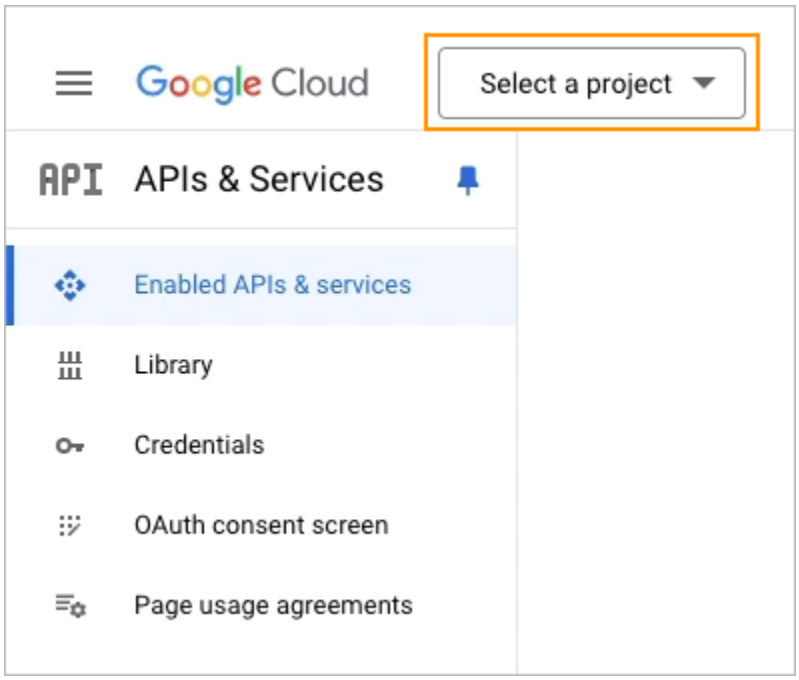
The general flow for setting up Google OAuth and Corezoid is the following:

1. Create a project on the Google Cloud Console.
2. Enable the Google OAuth API for your project.
3. Learn the requirements and flowchart for queries to Google APIs.
4. Create the OAuth 2.0 project folder in Corezoid.
5. Create Google credentials.
6. Obtain the Google code using your credentials.
7. Create the Corezoid process for generating the Google API access token.
8. Create and configure an API Call node in the token generation process.
9. Generate a Google API access token.
10. Create the Refresh token process for refreshing the Google API access token.
11. Configure token refresh.
12. Create and configure a state diagram for access token storage.

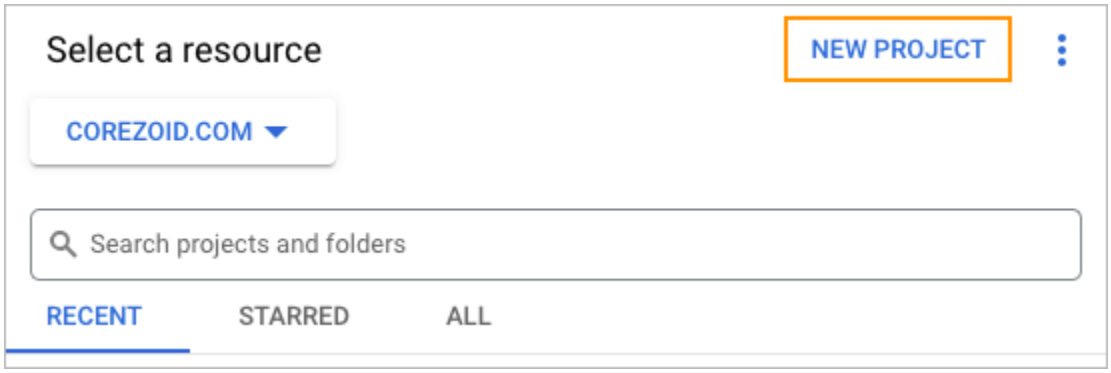
1. Create project on the Google Cloud Console

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

1. Go to console.developers.google.com and in the upper-left corner, click **Select a project**.

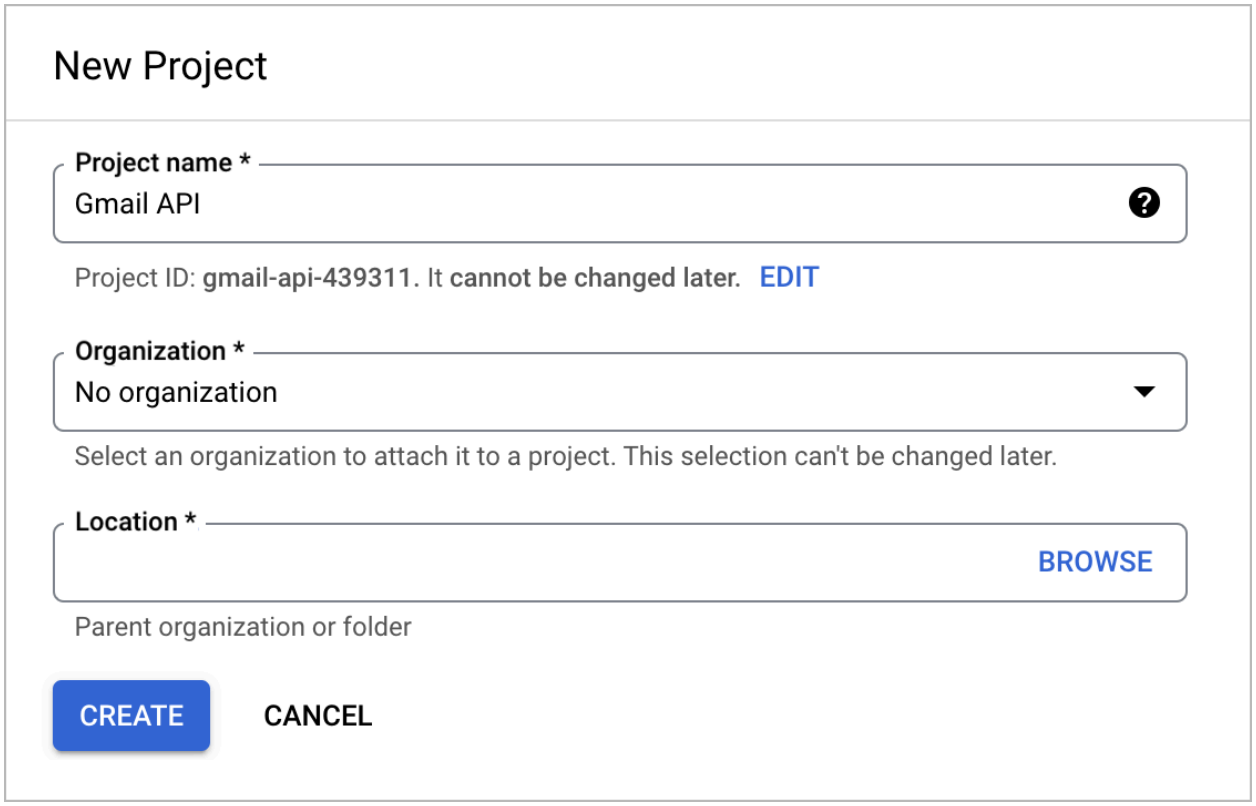


2. In the **Select a project** dialog, click **New project**.



3. In the **New Project** dialog:
    a. Enter your project name in the **Project name** field.
    b. Select your project organization in the **Organization** dropdown list.
    c. Select your project location in the **Location** dropdown list.
    d. Click **Create**.

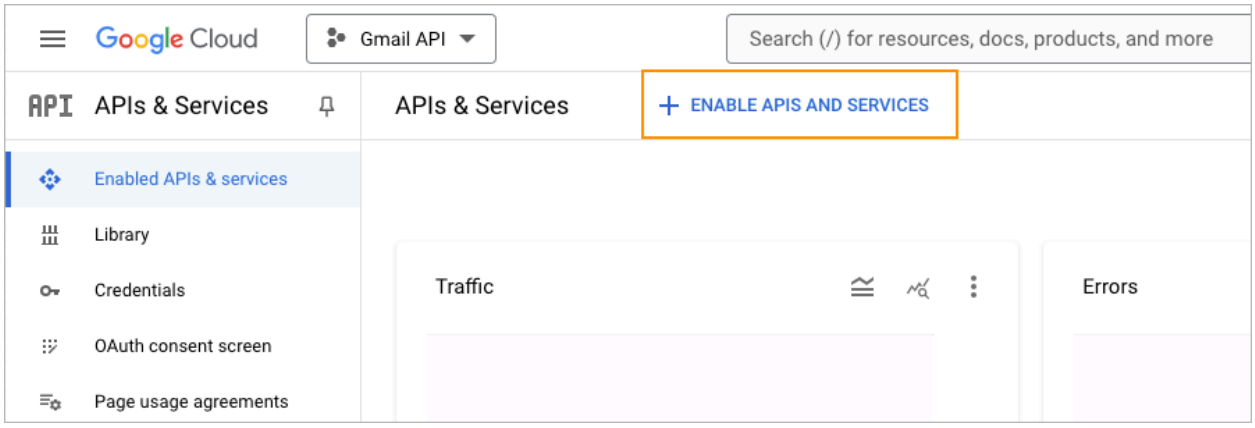
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Your project has been created on the Google Cloud Console.

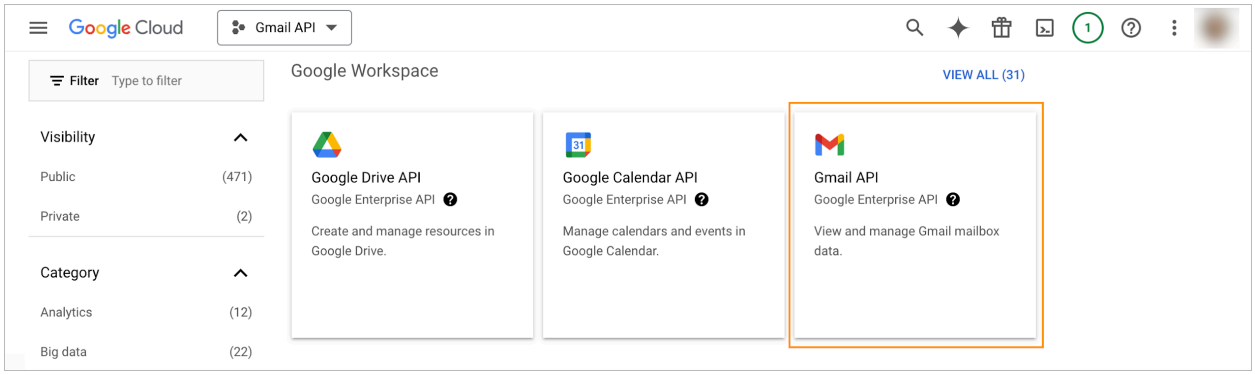
2. Enable Google OAuth API

1. On the **APIs & Services** page, click **+ Enable APIs and services**.

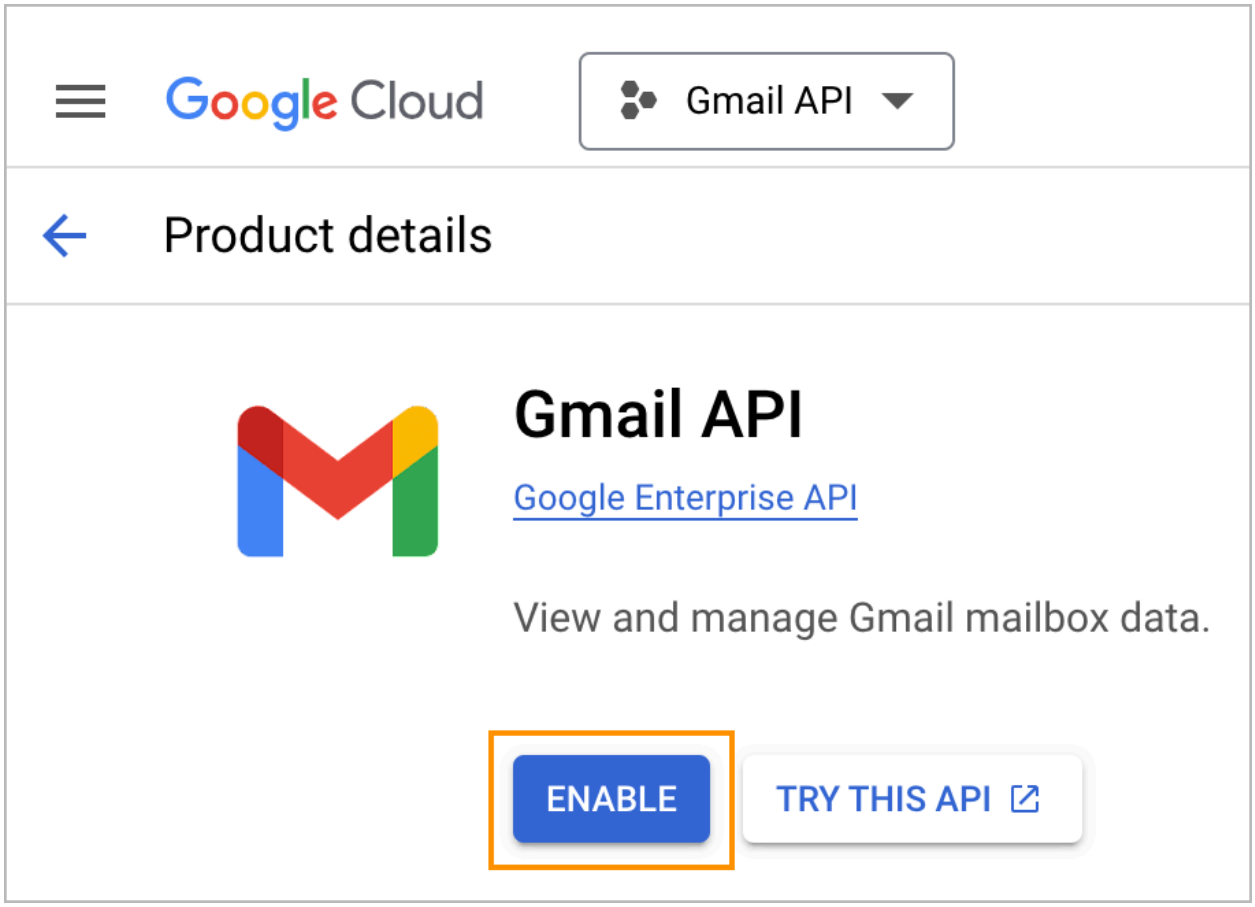


2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

2. In the list of APIs, select **Gmail API**.



3. In the **Gmail API** dialog, click **Enable**.

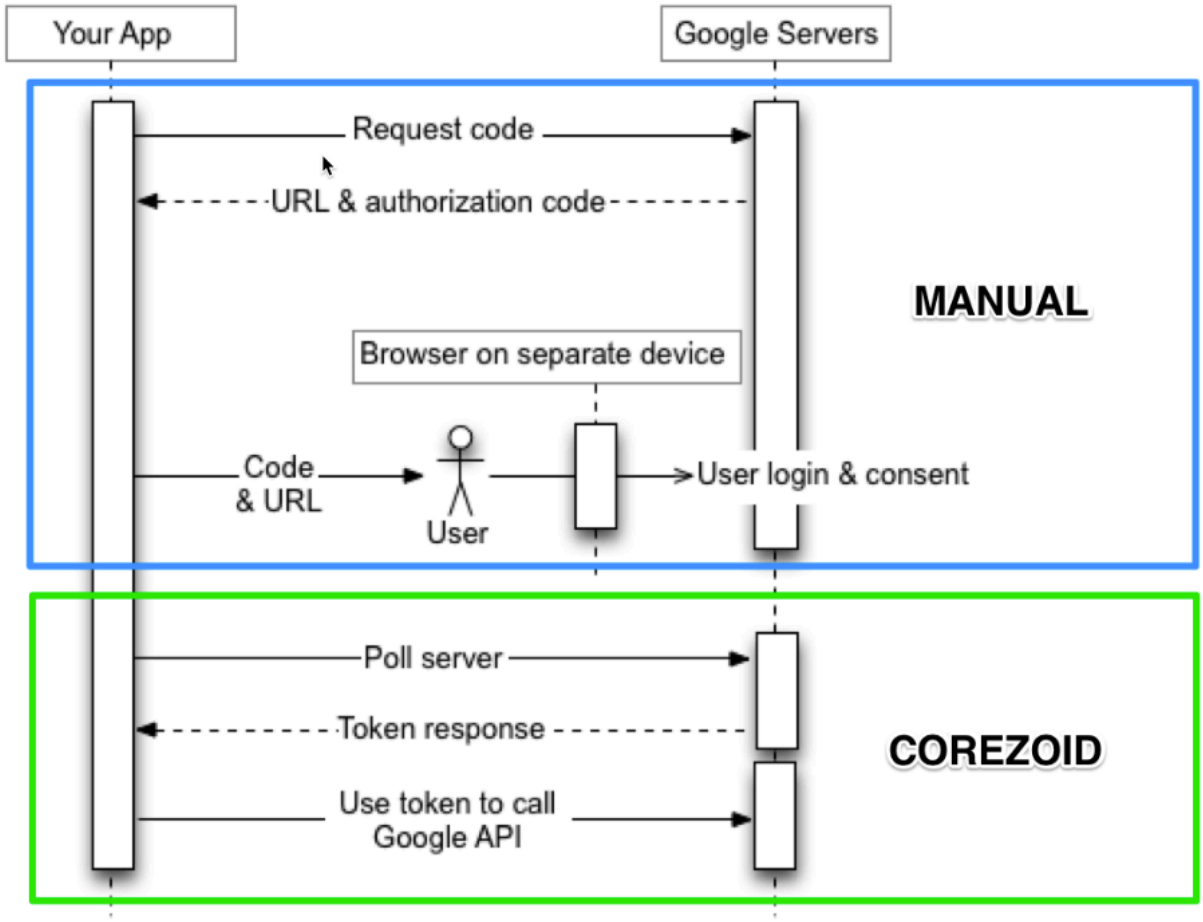


Google OAuth API has been enabled.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3. Learn requirements and flowchart for queries to Google APIs

All queries to Google APIs require authorization via the OAuth 2.0 protocol.
The flowchart below shows how Google services interact using OAuth 2.0.



Each request to API must contain the following parameters in the **Header**:
`: Bearer {{ACCESS_TOKEN}}`
where:
ACCESS_TOKEN authorizes queries and stores user details (such as user_id and user_role). It verifies your account before accessing the Google API.
Note: To reuse ACCESS_TOKEN across projects, use a unified storage process.

You can obtain **ACCESS_TOKEN** by adding the Set Parameters node in the following structure
`{{conv[process_id].ref[reference_of_request].name_of_the_parameter_the_ACCESS_TOKEN_is_located_in}}`

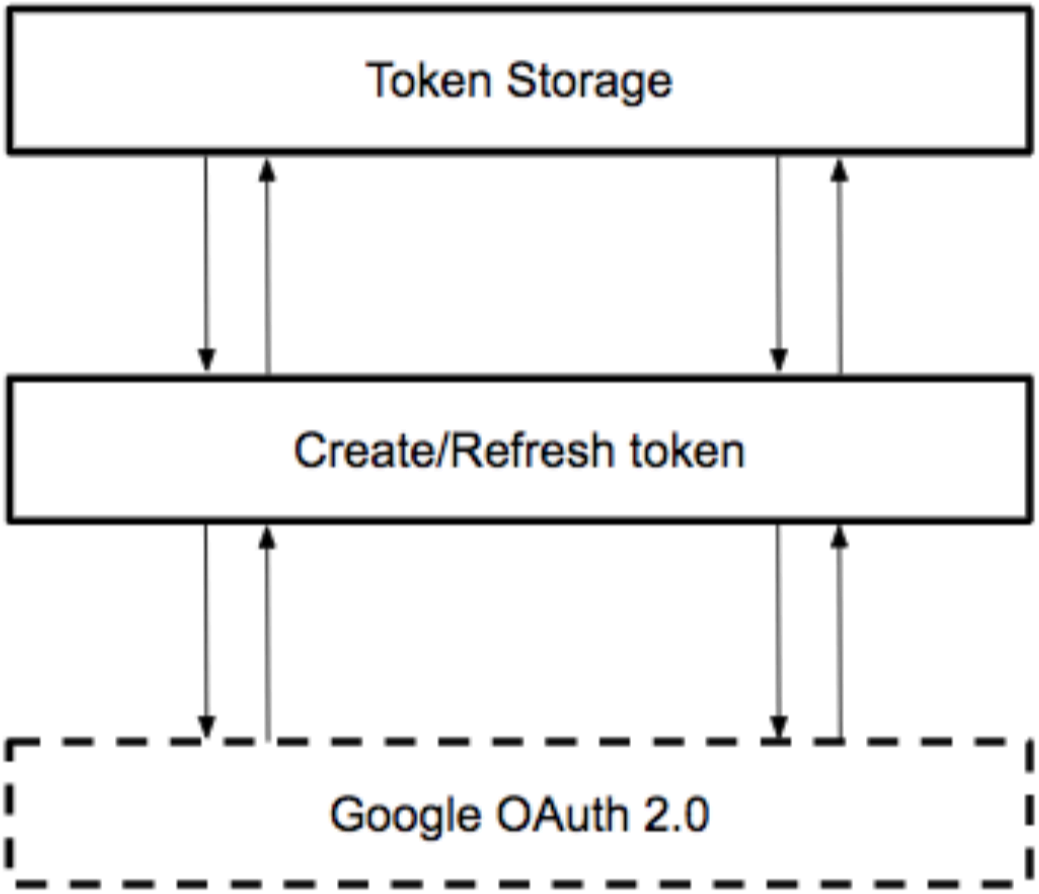
For example: `{{conv[4].ref[gmail].ACCESS_TOKEN}}`
You need to create three unified processes in Corezoid to manage tokens:

- **Create Token**: A process for generating the ACCESS_TOKEN via API calls.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- **Refresh Token**: A process for refreshing the ACCESS_TOKEN.
- **Token Storage**: A state diagram for storing and refreshing the ACCESS_TOKEN.

These processes will interact with Google OAuth 2.0, as shown in the figure below.

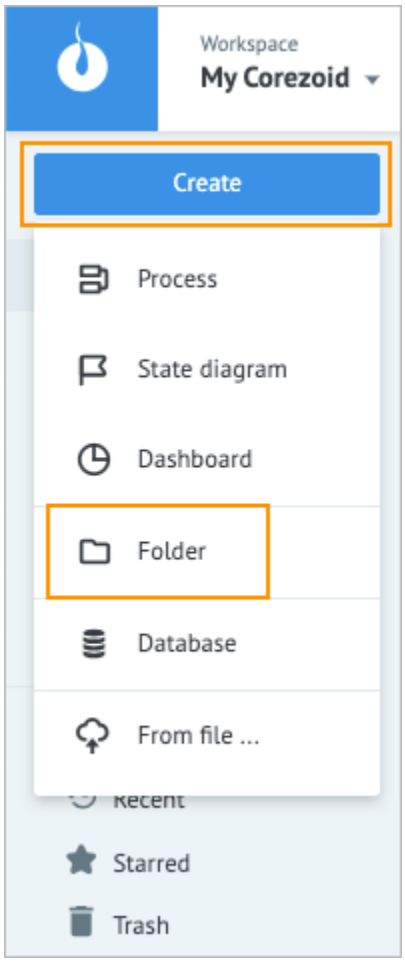


At his point, you have completed the Google project creation stage.

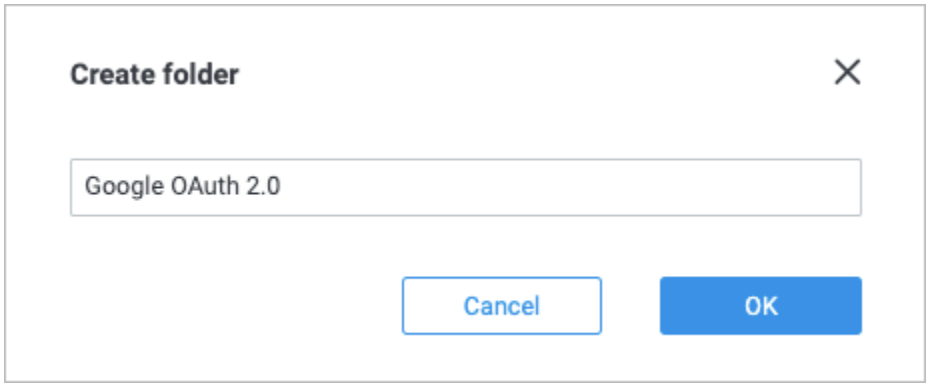
4. Create OAuth 2.0 project folder in Corezoid

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

1. In the upper-left corner of your workspace page, click **Create** and in the dropdown list, select **Folder**.



2. In the **Create folder** dialog, enter the **Google OAuth 2.0** folder name in the **Title** field and click **Ok**.

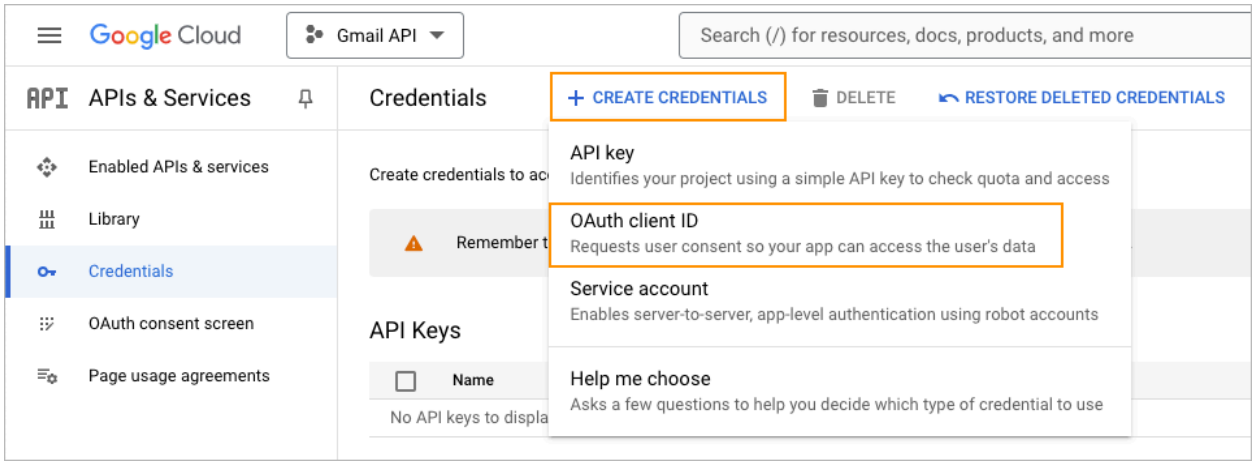


The **Google OAuth 2.0** folder has been created.

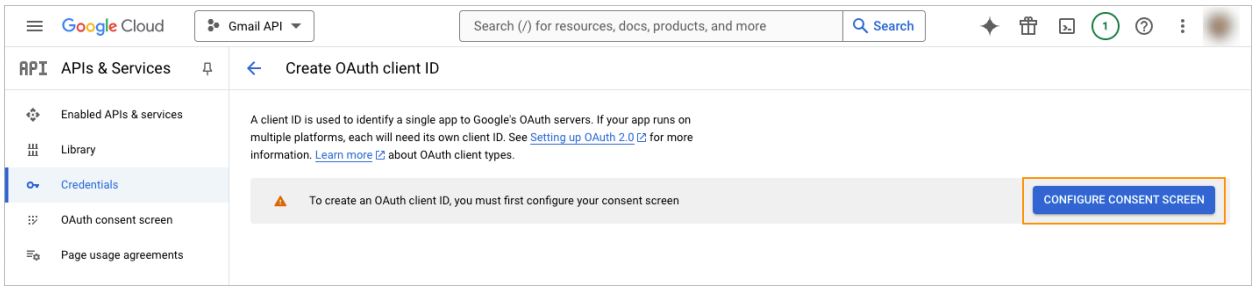
5. Create Google credentials

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

1. On the **Credentials** tab of the **APIs & Services** page, click **+ Create Credentials** and in the dropdown list, select **OAuth client ID**.

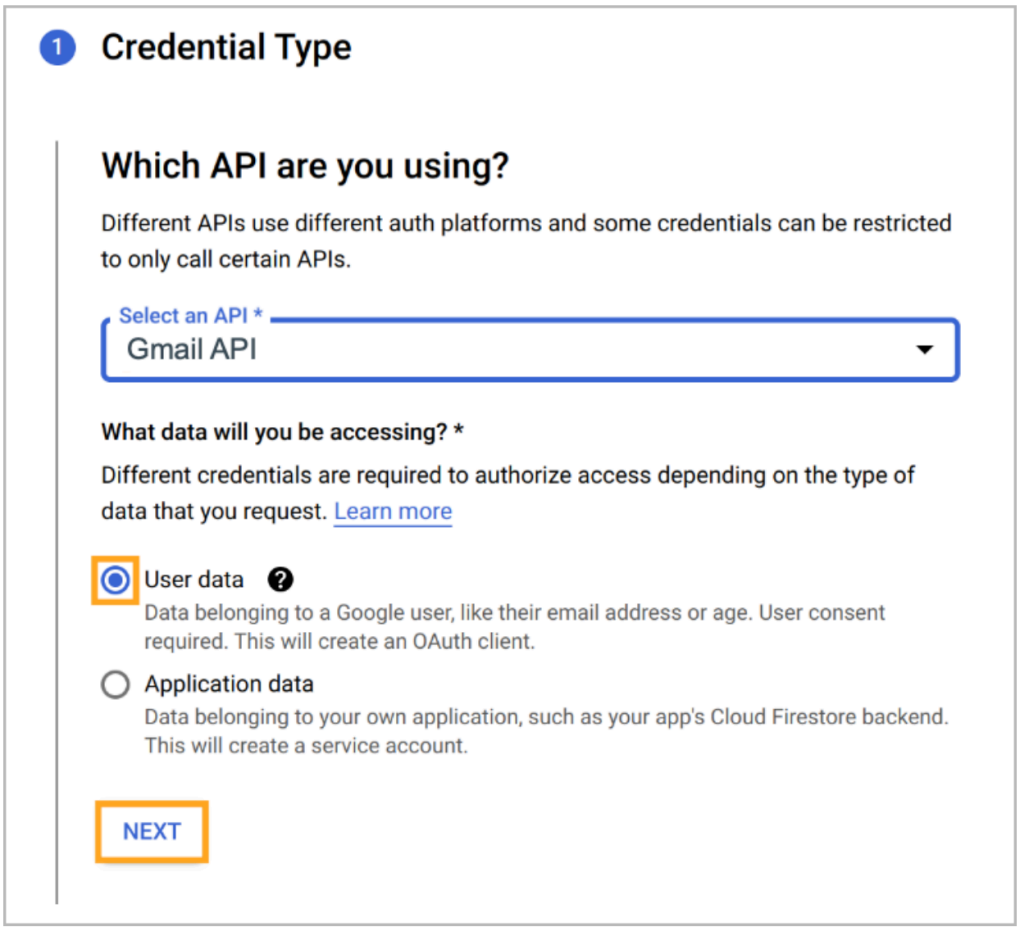


2. On the **Auth client ID** page, click **Configure consent screen**.



2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

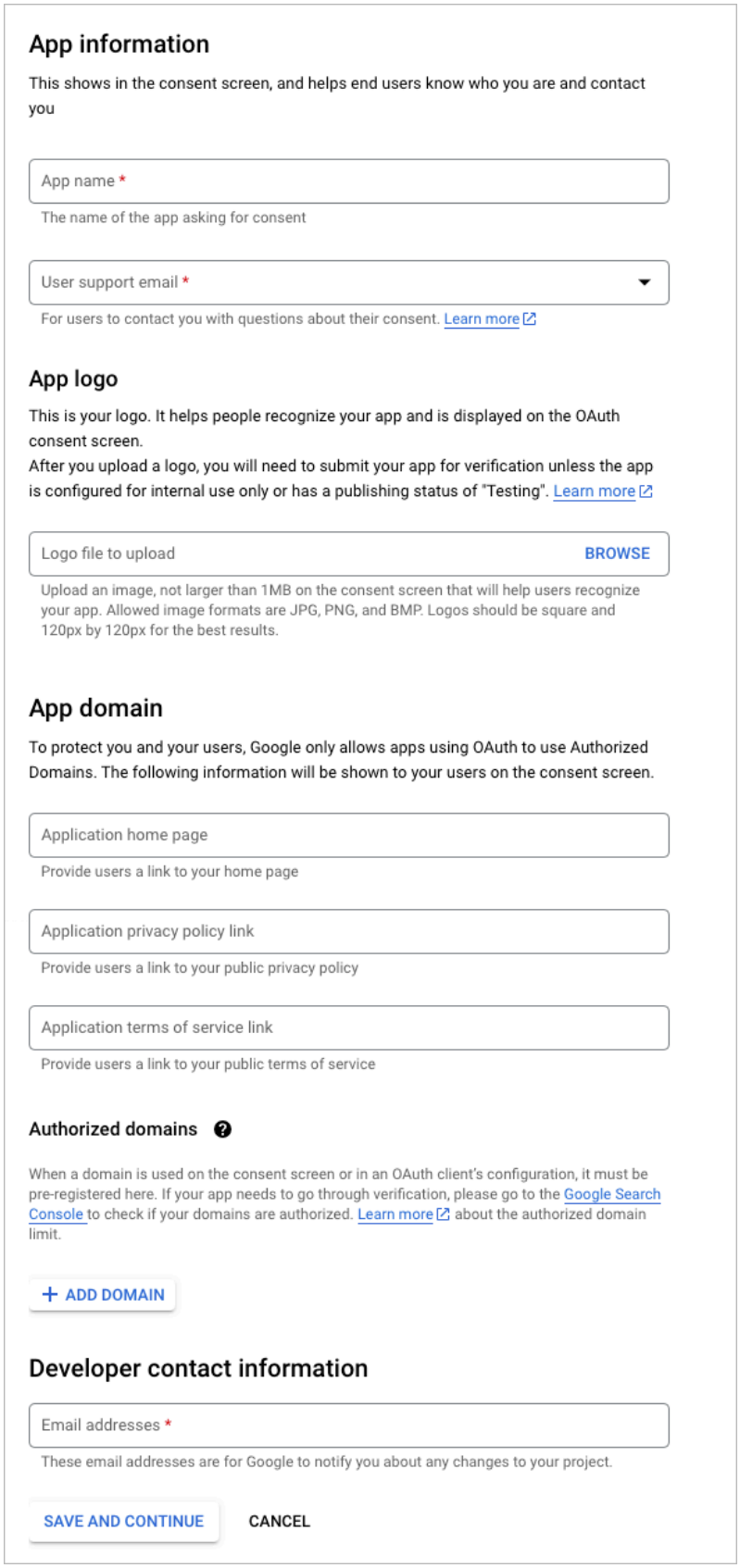
3. On the **Credential Type** page, select **Gmail API** in the **Select an API** dropdown list, select **User data**, and click **Next**.



2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

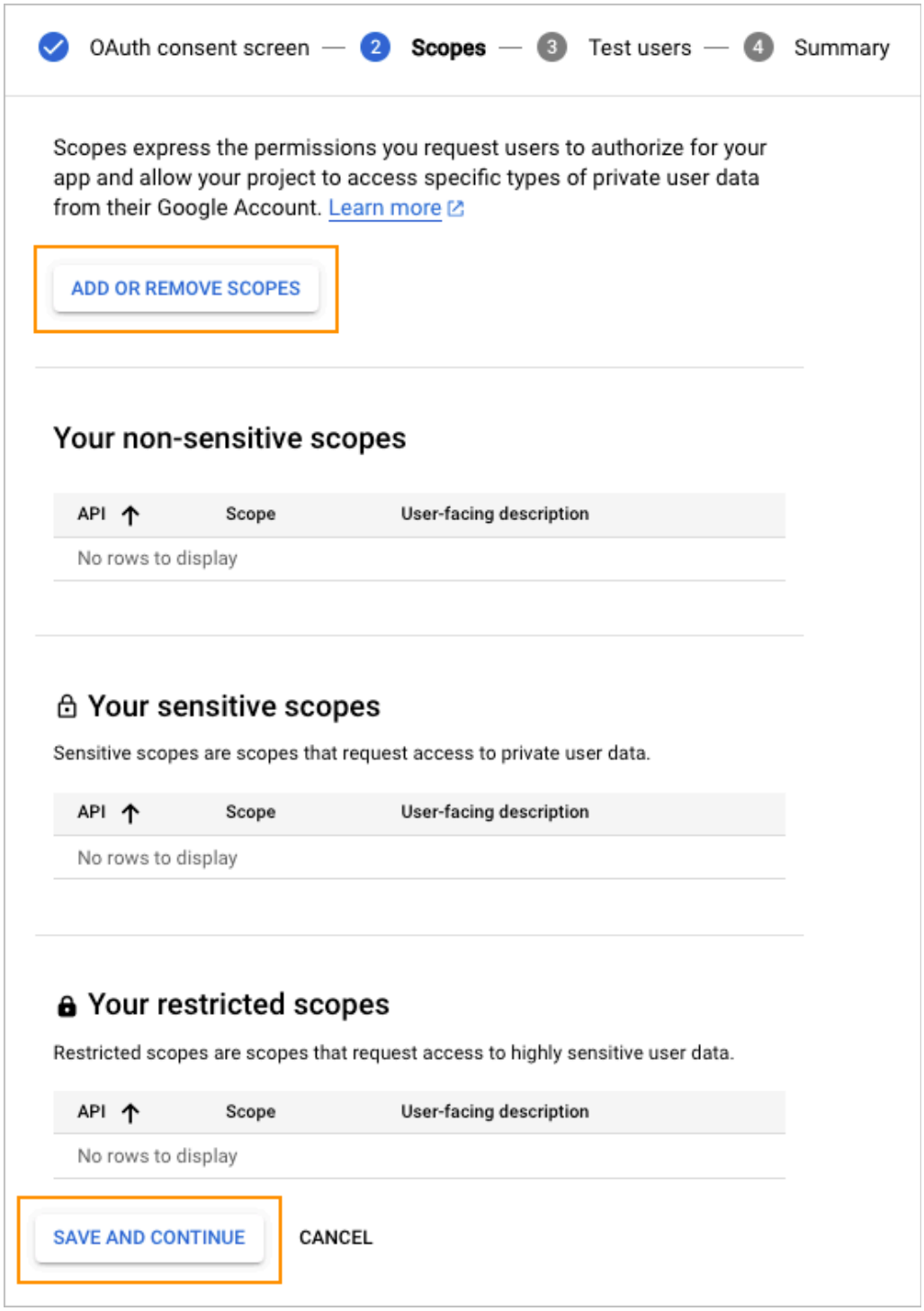
4. On the **App information** page, fill in all the required fields and click **Save and continue**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



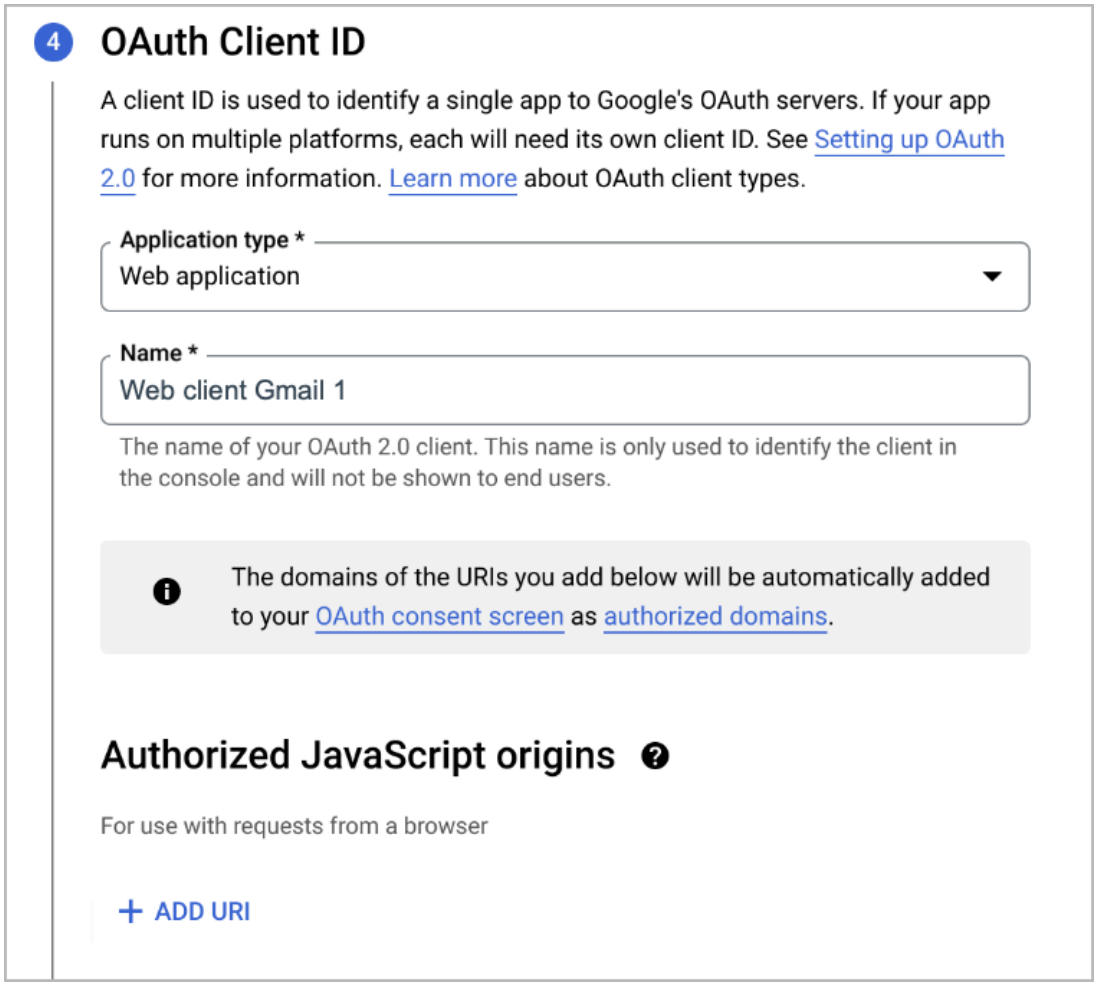
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

5. (Optional) On the **Scopes** page, add scopes by clicking **Add or remove scopes** and click **Save and continue**.



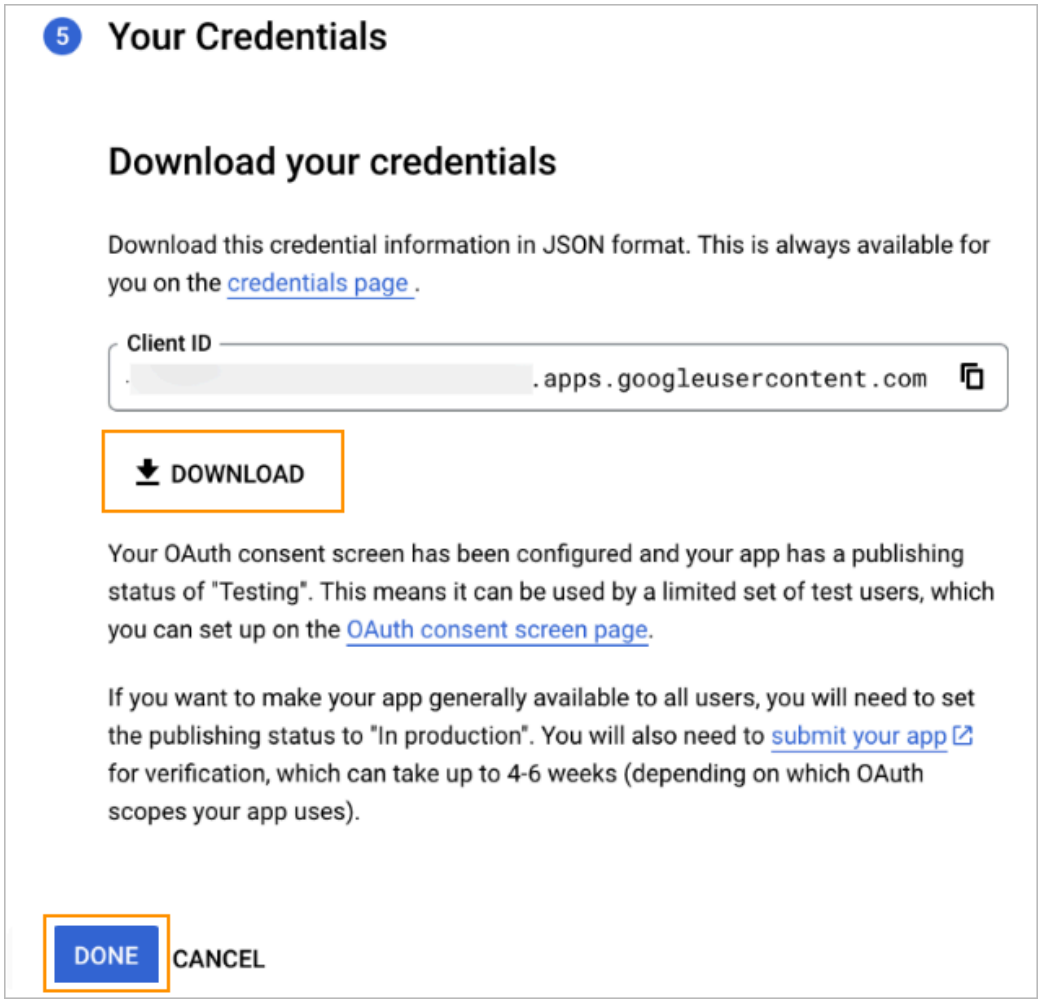
6. On the **OAuth Client ID** page:
    a. Select application type in the Application type dropdown list.
    b. Enter an application name in the **Name** field.
    c. Enter the redirect URI by clicking **+ Add URI**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

7. On the **Your Credentials** page, download the credentials you've created and click **Done**.



Your Google credentials have been created and downloaded.

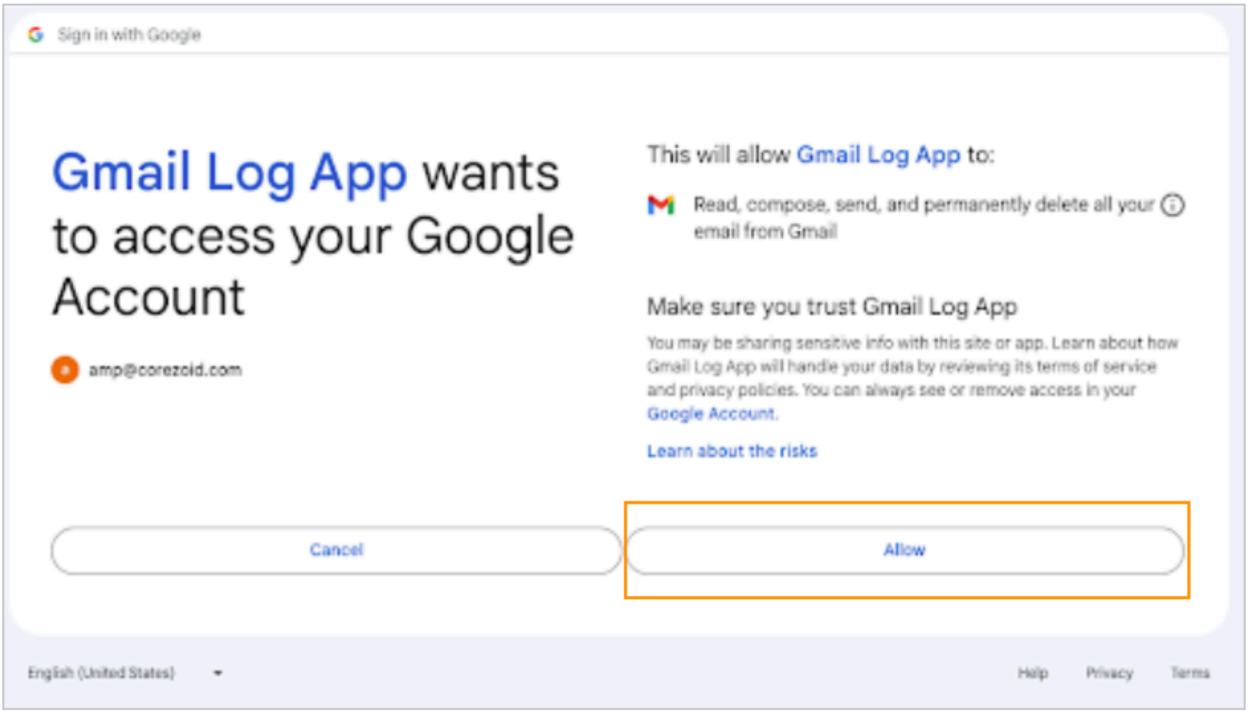
6. Obtain Google Code

1. In your web browser, enter the following URL:
   `https://accounts.google.com/o/oauth2/v2/auth?redirect_uri=http://localhost:3000/auth/google/callback&response_type=code&access_type=offline&scope=https://mail.google.com&client_id=client_id`
   where:
   - **redirect_uri**: Is the redirect URI that you've entered at step 7 when creating credentials http://localhost:3000/auth/google/callback.
   - **response_type**: Is the code to obtain the code value (step 3).
   - **access_type**: Is used offline.

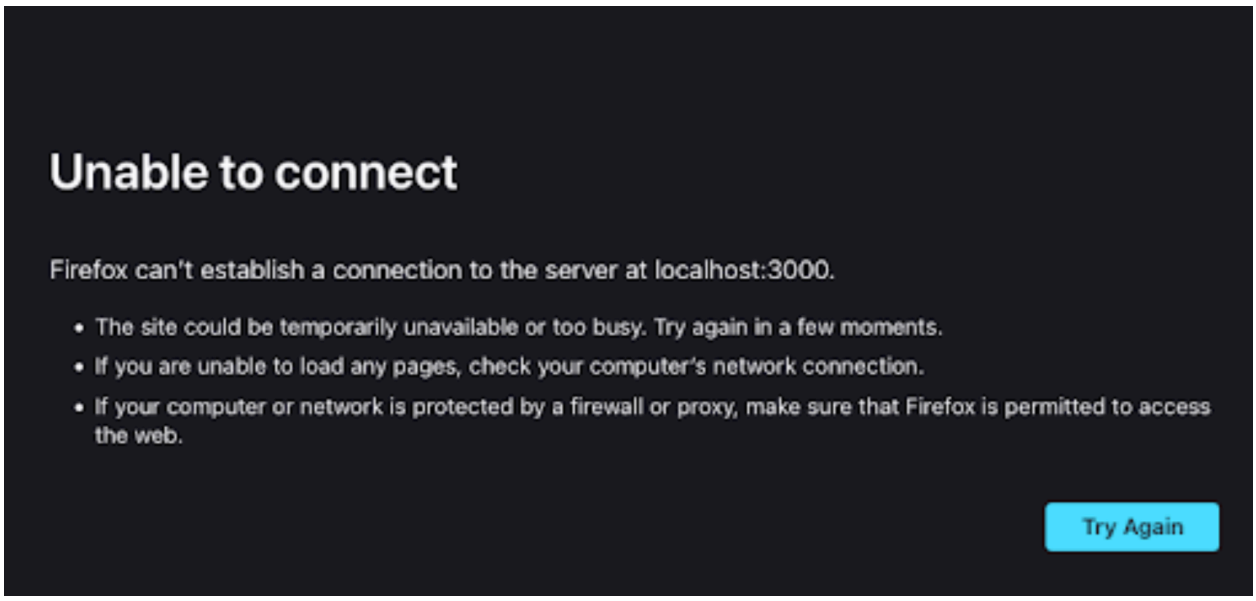
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- **scope**: Is the Google API you want to gain access to. As we use the Gmail API in this tutorial, the value is https://mail.google.com.
  Note: You can find the list of available Google API scopes here.
- **client_id**: Is the OAuth Client ID you've obtained when creating your Google credentials (step 8).

2. On the Google Account access confirmation page, click **Allow**.



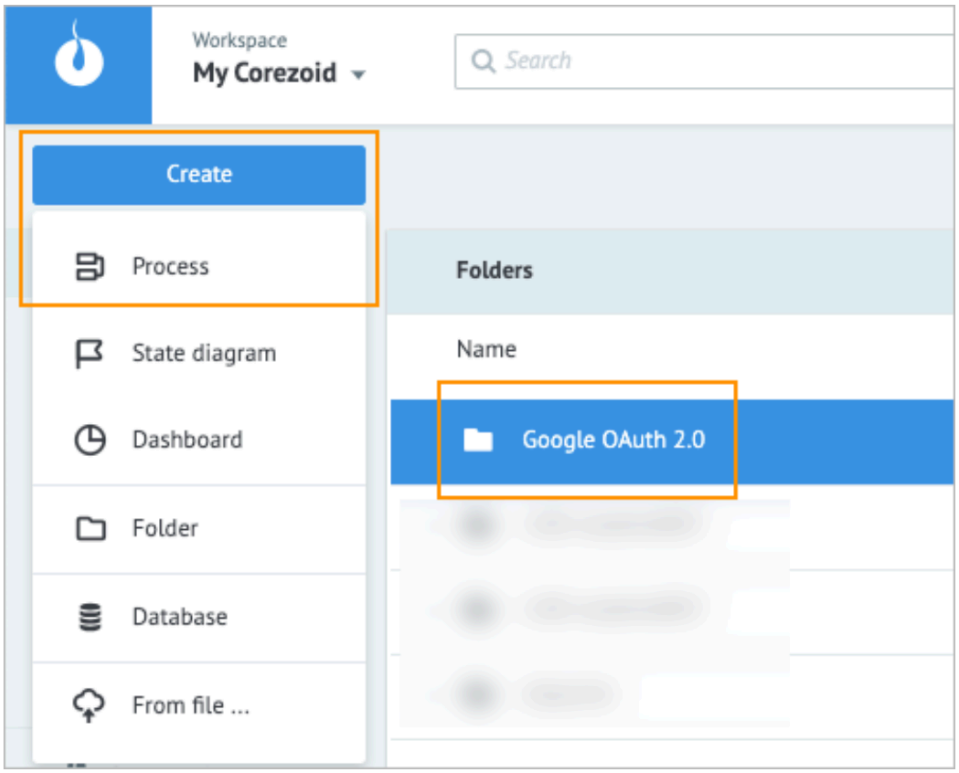
3. On the **Unable to connect** page, copy and save the page URL to get the **OAuth code**.



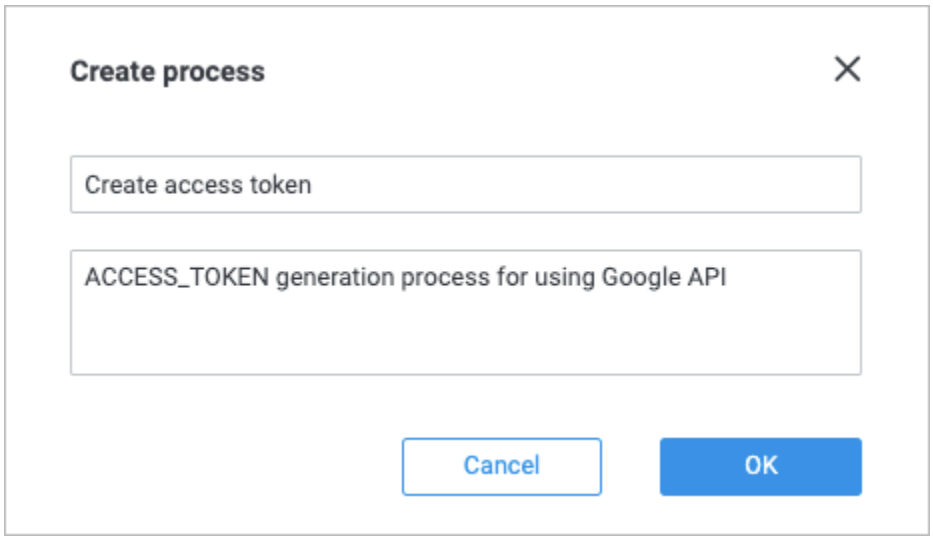
7. Create Process for Access Token Generation

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

1. On https://admin.corezoid.com/, open the OAuth 2.0 folder (that you've created) click **Create**, and from the dropdown list, select **Process**.



2. In the **New process** dialog:
    a. Enter **Create access token** in the **Title** field.
    b. Enter **ACCESS_TOKEN generation process for using Google API** in the **Description** field.
    c. Click **Ok**.

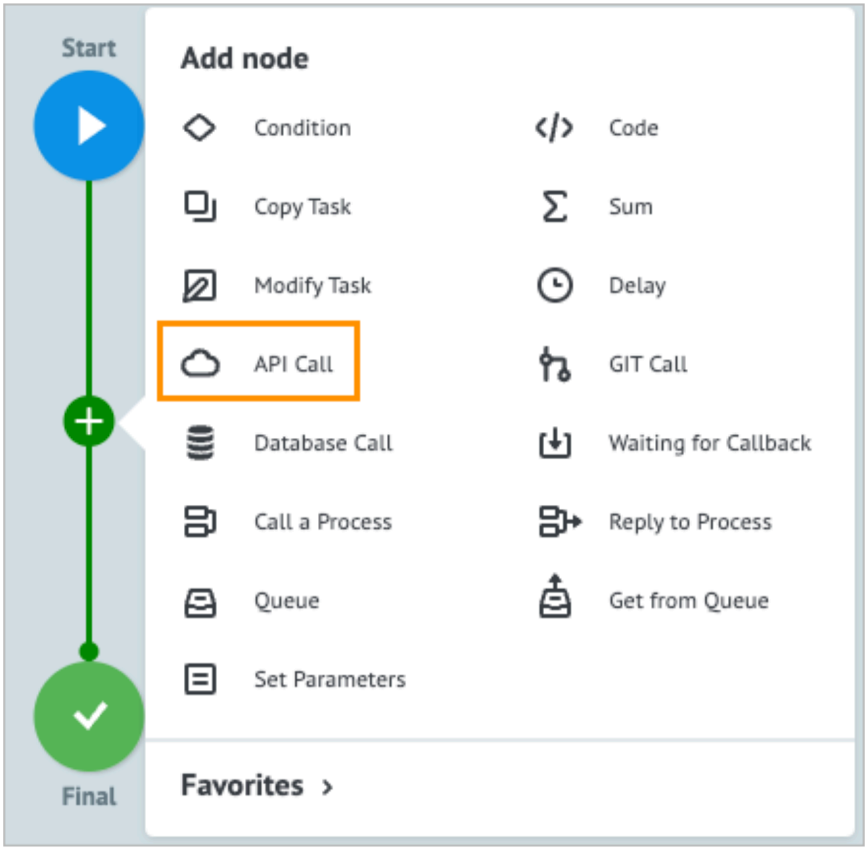


2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

The **Create access token** process has been created.

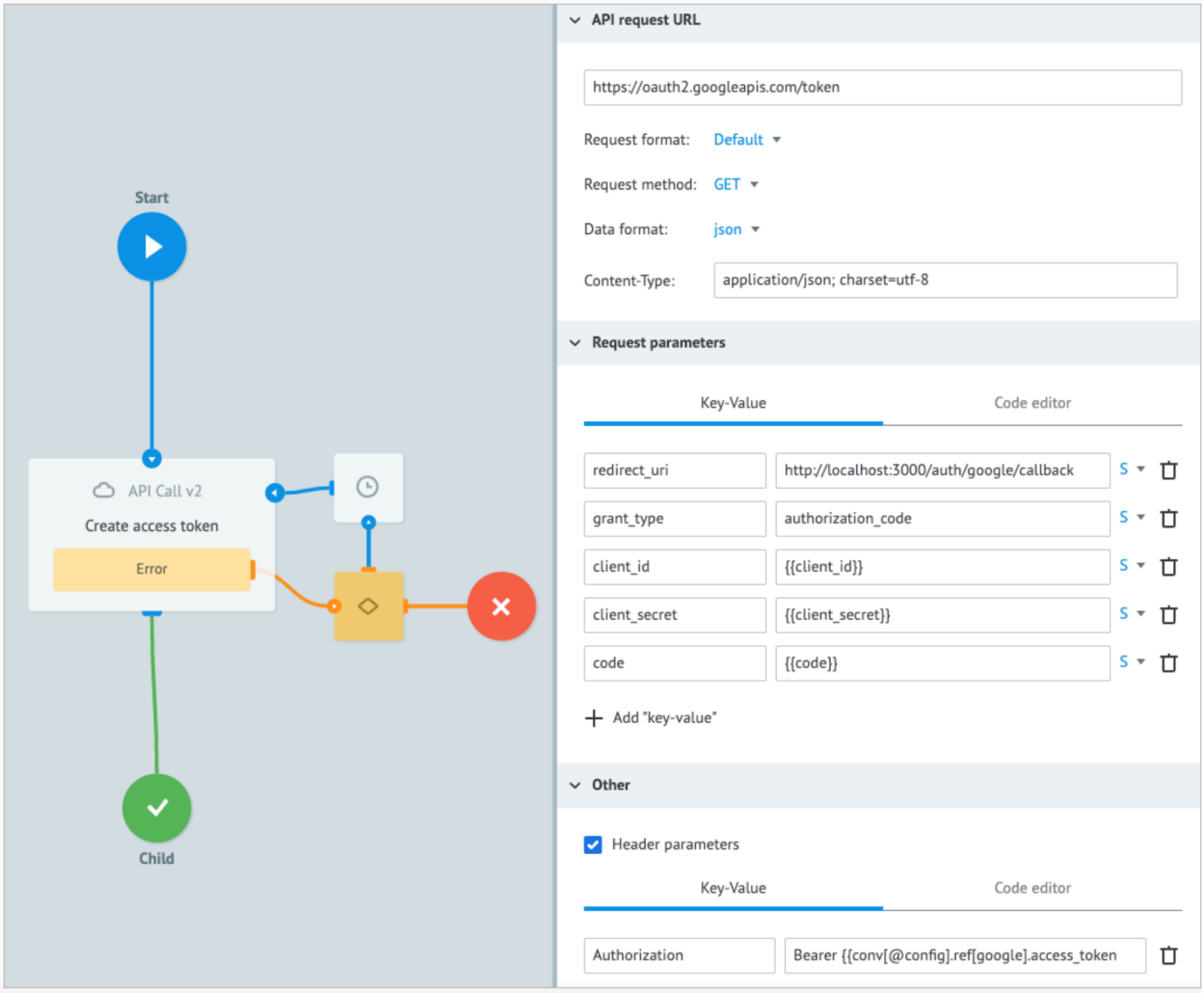
8. Create and configure API Call node in token generation process

1. In the **Create access token** process, add the **API Call** node, which will call Google OAuth 2.0 API for generating **ACCESS_TOKEN**.



2. In the **API Call** node details panel:
    a. Enter **"https://oauth2.googleapis.com/token"** in the **URL** field.
    b. Select **Default** in the **Request format** dropdown list.
    c. Select **GET** in the **Request method** dropdown list.
    d. Select **json** in the **Data format** dropdown list.
    e. Enter **"application/json; charset=utf-8"** in the **Content-Type** field.
    f. Enter **redirect_uri**, **grant_type**, **client_id**, **client_secret**, and **code** in the **Key** fields.
    g. Enter **http://localhost:3000/auth/google/callback**, **authorization_code**, **{{client ID}}**, **{{client secret}}**, and **{{code}}** in the **Value** fields.
    h. In **Other**, select the Header parameters checkbox and add **Authorization** and **Bearer {{conv[@config].ref[google].access_token}}** in the **Key** and **Value** fields correspondingly.

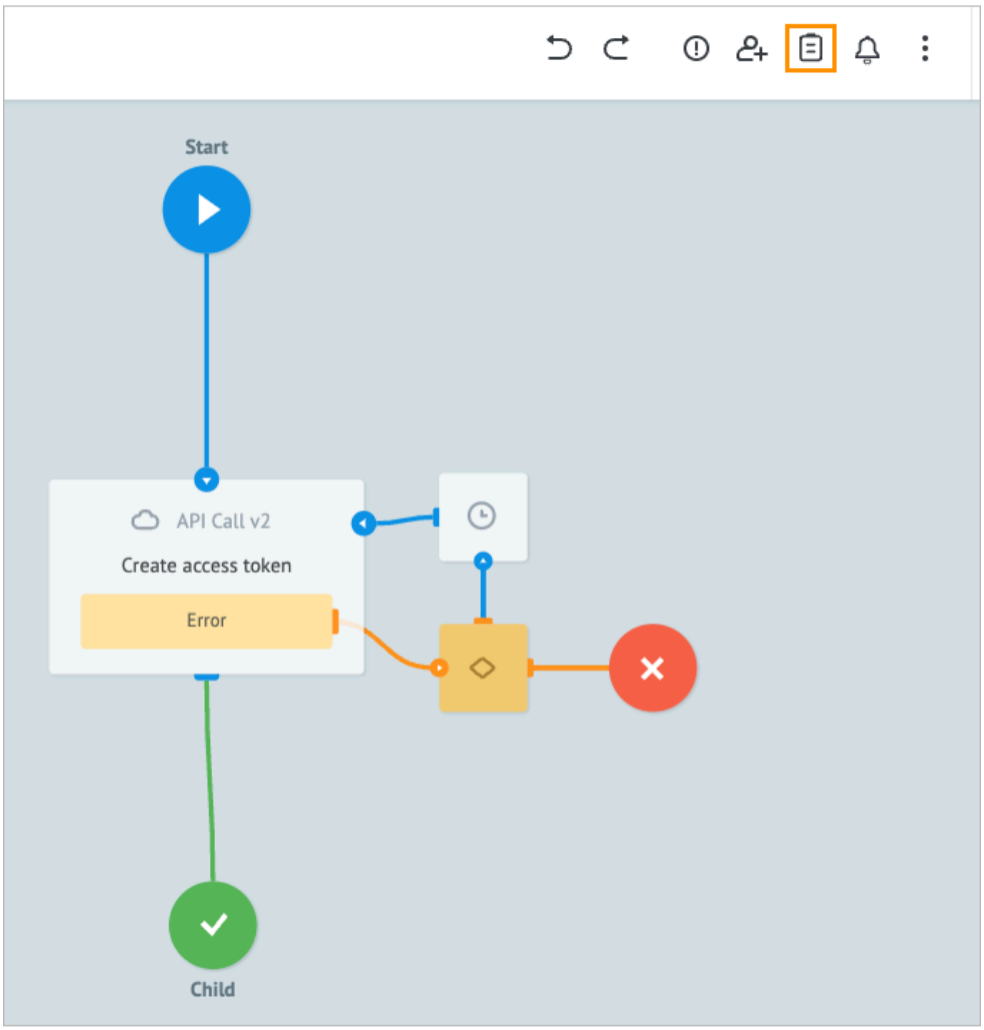
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



3. To ensure automatic parameter entry upon manual request for **ACCESS_TOKEN** generation, add the credentials parameters to the process tasks:

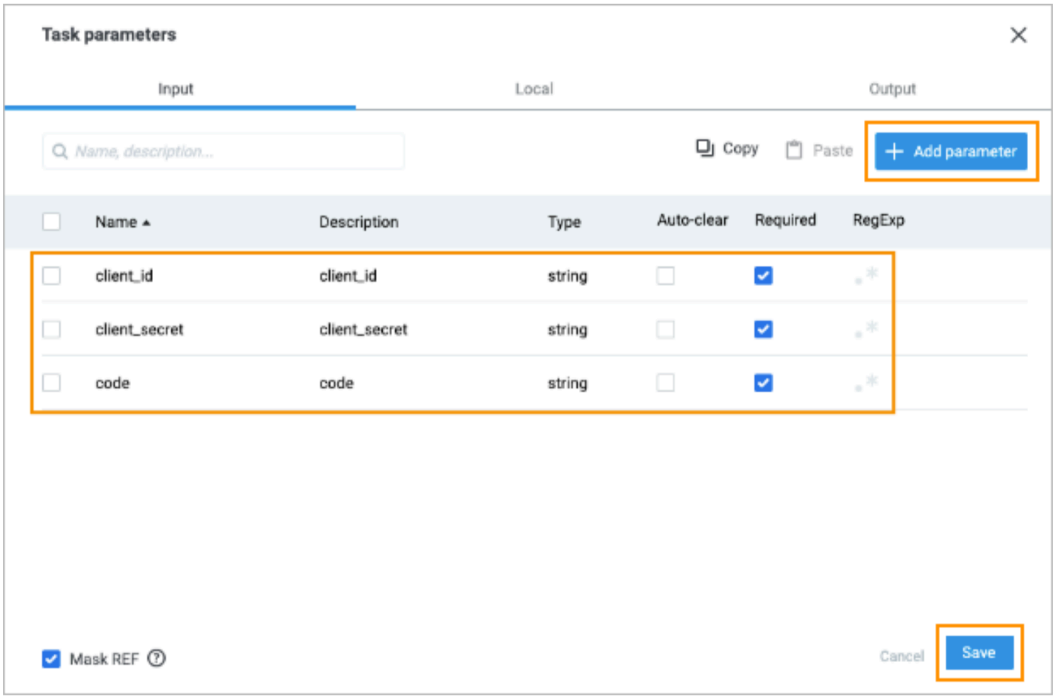
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

a. In the upper-right corner of the process page, click the task parameters icon.



2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

b. In the **Task parameters** dialog, click **Add parameter**, add the **client_id**, **code**, **client_secret** parameters and click **Save**.



The **API Call** you've configured is ready to request a call which will result in obtaining **ACCESS_TOKEN** from Google OAuth 2.0 API.

9. Generate Google Access Token

In the configured **Create access token** process, send a new task: If **ACCESS_TOKEN** is successfully created, your request appears in the Final node. Click the node to view the request details, where the **ACCESS_TOKEN** is included as one of the parameters.

10. Create Refresh token process

In your process folder, create a new process called **Refresh token** to configure the Google access token refresh.

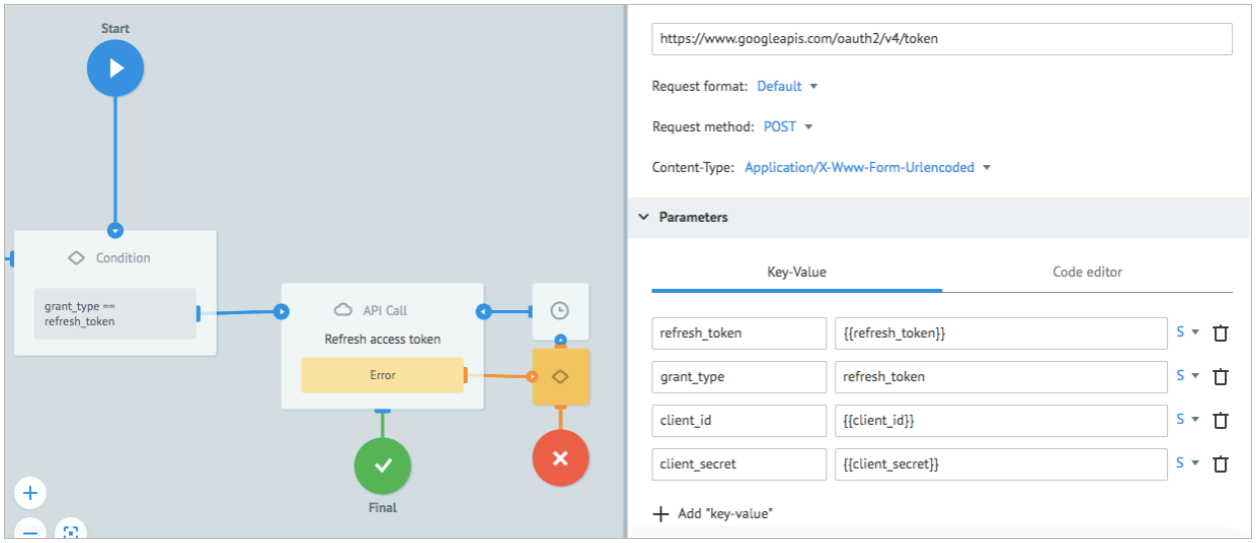
11. Configure token refresh

According to the **Google OAuth** protocol, the token expiration time is one hour, thus, you need to configure a Google API call for the token refresh.

1. In the **Refresh Token** process, add the **Condition** and **API Call** nodes.
   Note:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

    a. The **Condition** node is required to forward a request to the **API Call** node for the token to be refreshed under the **grant_type == refresh_token** condition.
    b. The **API Call** node is required to call Google API for a token refresh after this condition.
2. In the **Condition** node, add `refresh_token` receipt verification to the **grant_type** parameter.
3. In the **API Call** node details panel:
    a. Enter https://oauth2.googleapis.com/token in the **URL** field.
    b. Select **Default** in the **Request format** dropdown list.
    c. Select **POST** in the **Request method** dropdown list.
    d. Select **json** in the **Data format** dropdown list.
    e. Enter **"Application/X-Www-Form-Urlencoded"** in the **Content-Type** field.
    f. Enter **refresh_token**, **grant_type**, **client_id**, and **client_secret** in the **Key** fields.
    g. Enter **{{refresh_token}}**, **refresh_token**, **{{client_id}}**, and **{{client secret}}** in the **Value** fields.



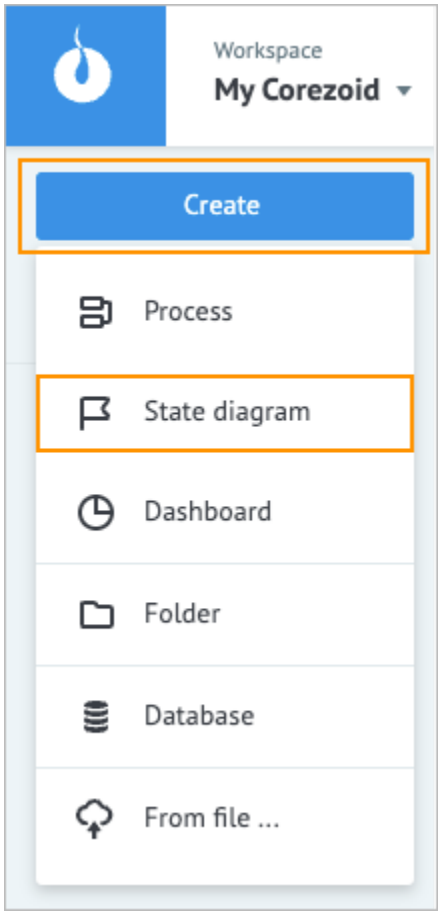
The process for **ACCESS_TOKEN** refresh has been created.

12. Create and configure state diagram for access token storage

To avoid repeatedly calling the **Create Token** and **Refresh Token** processes for each Google API request, implement a mechanism to store and manage **ACCESS_TOKEN**. This approach will help optimize API calls and reduce latency. You can use a state diagram to illustrate the token lifecycle and management strategy.

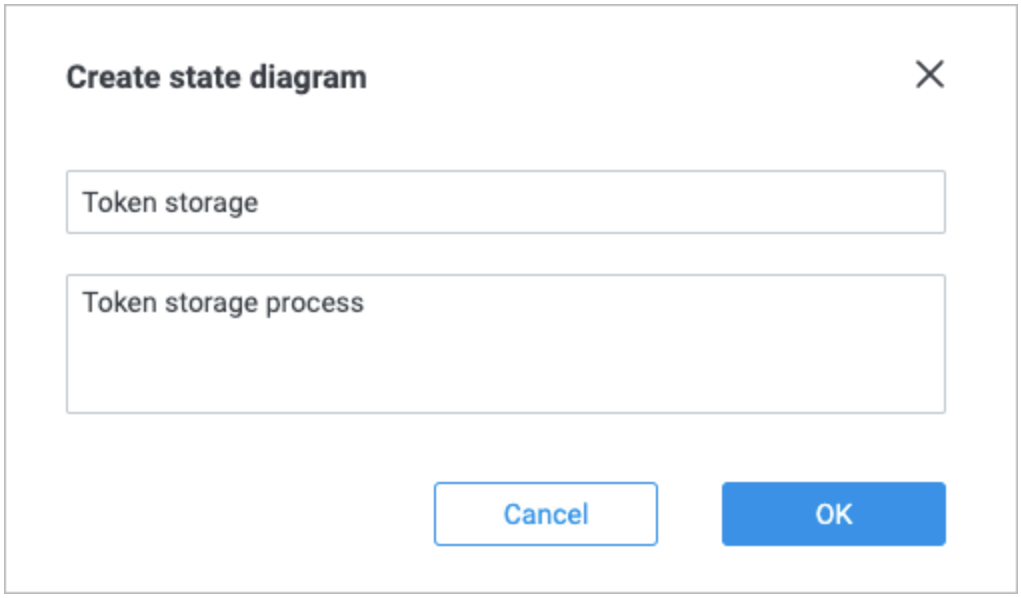
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

1. On https://admin.corezoid.com/, open the OAuth 2.0 folder (that you've created above), click **Create**, and from the dropdown list, select **State diagram**.



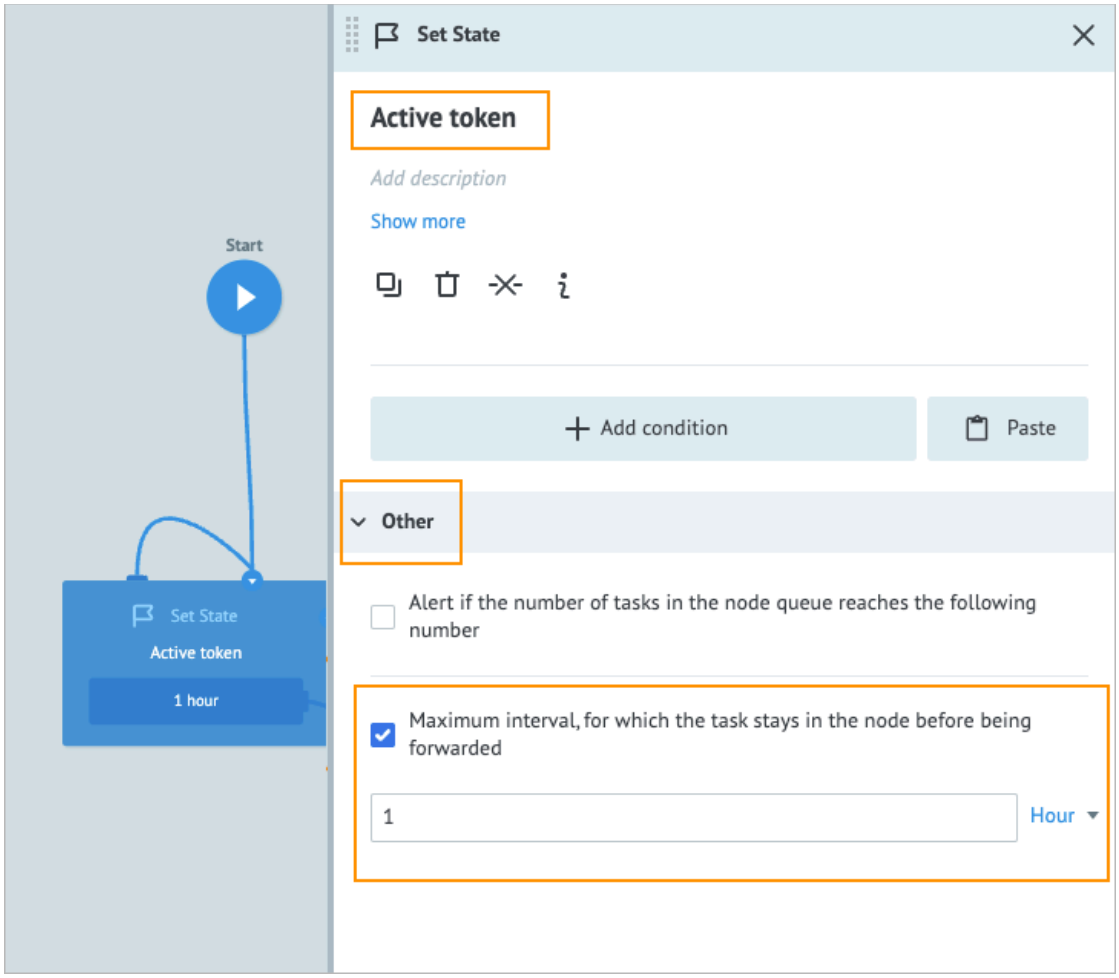
2. In the **Create state diagram** dialog:
    a. Enter **Token storage** in the **Title** field.
    b. Enter **Token storage process** in the **Description** field.
    c. Click **Ok**.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



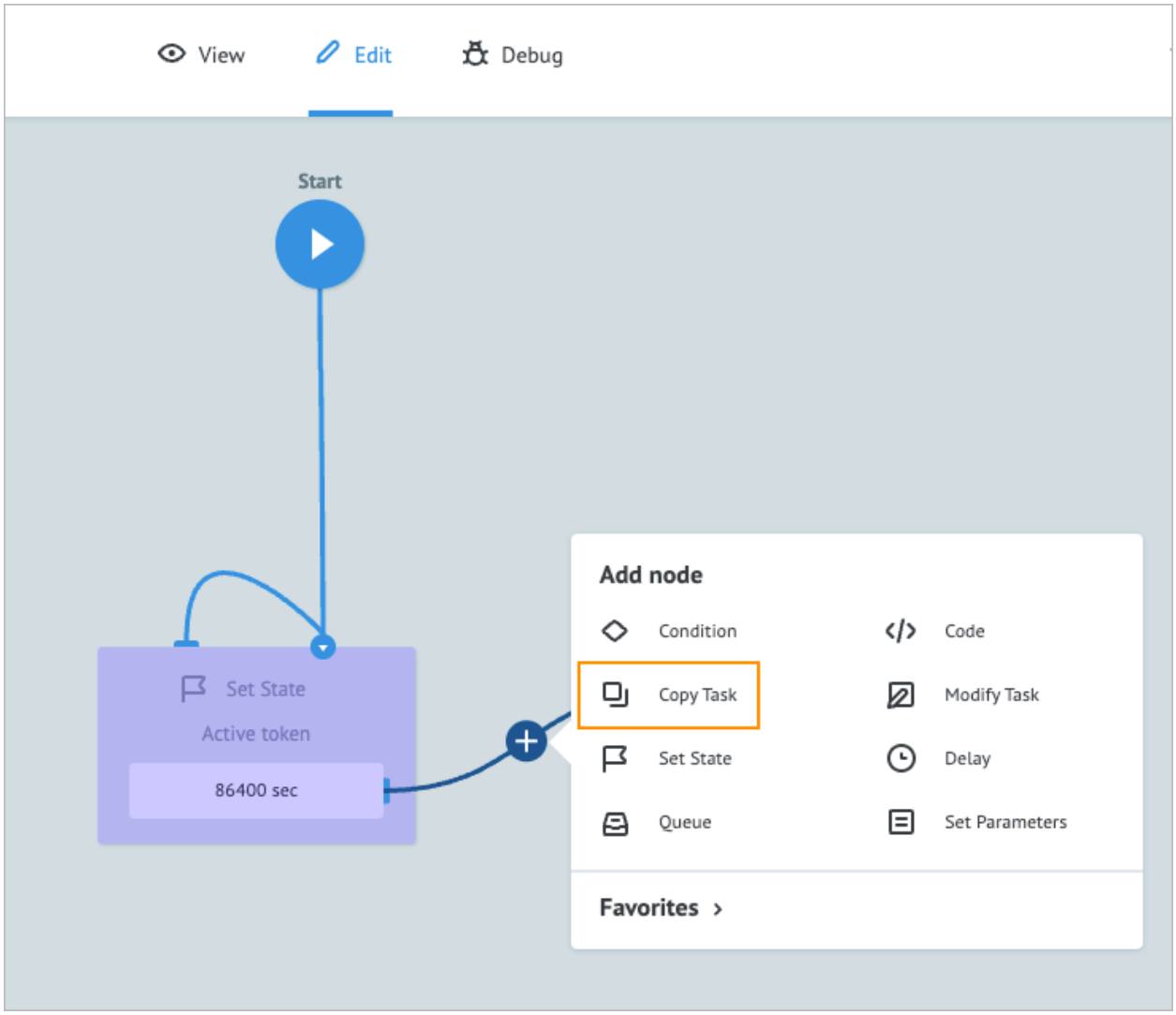
3. In the first **Set State** node details panel:
    a. Change the default name to **Active token**.
    b. Click **Other**, select the **Maximum interval, for which the task stays in the node before being forwarded** checkbox, enter **1** in the field below, and select **Hour**.
    Note: According to the Google OAuth protocol, token expiration time is one hour.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



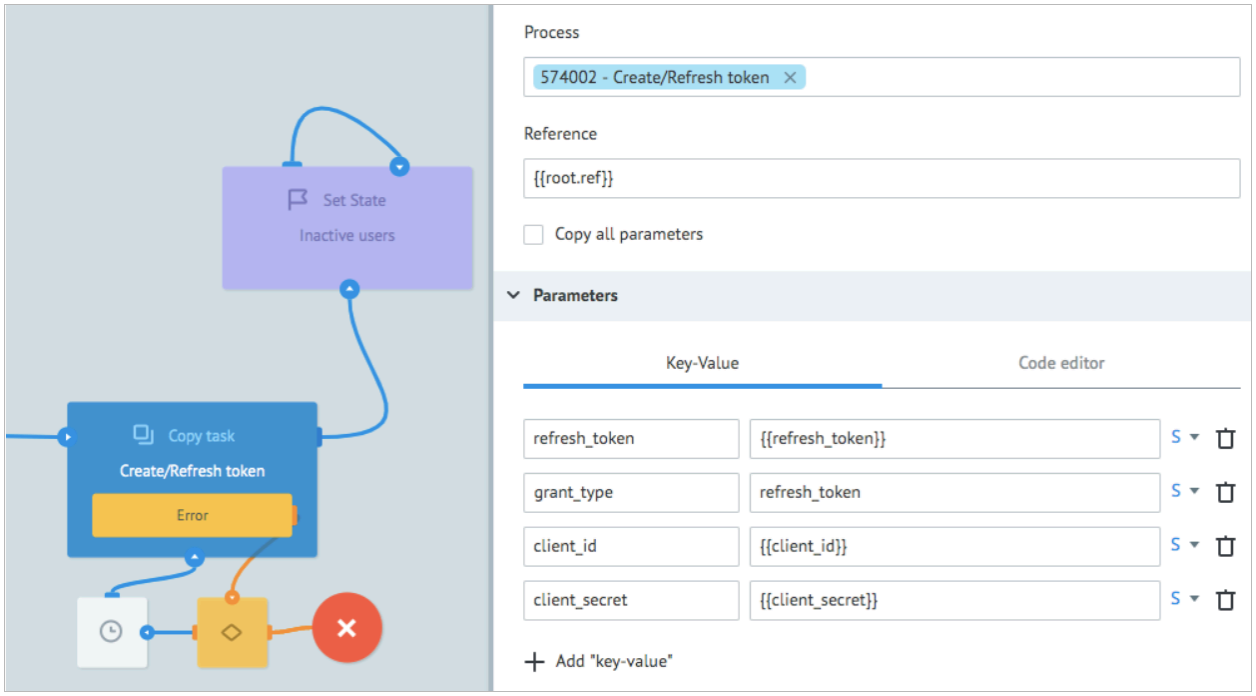
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

4. Add the **Copy Task** node.

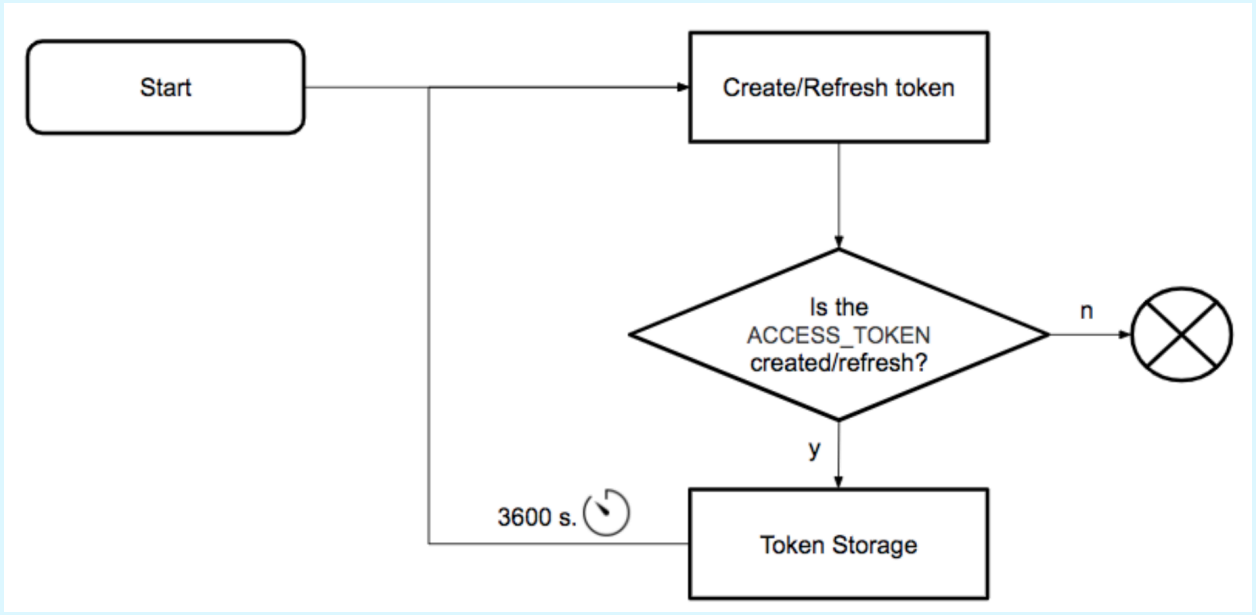


5. In the **Copy Task** node details panel:
    a. Enter the **Create/Refresh token** name in the **Add title** field.
    b. Enter the **Create/Refresh token** process in the **Process** field.
    c. Enter **{{root.ref}}** in the **Reference of the task copy** field.
    d. Select **Default** in the **Request format** dropdown list.
    e. Enter **refresh_token**, **grant_type**, **client_id**, **client_secret**, and **code** in the **Key** fields.
    f. Enter **{{refresh_token}}**, **refresh_token**, **{{client_id}}**, **{{client secret}}**, and **{{code}}** in the **Value** fields.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



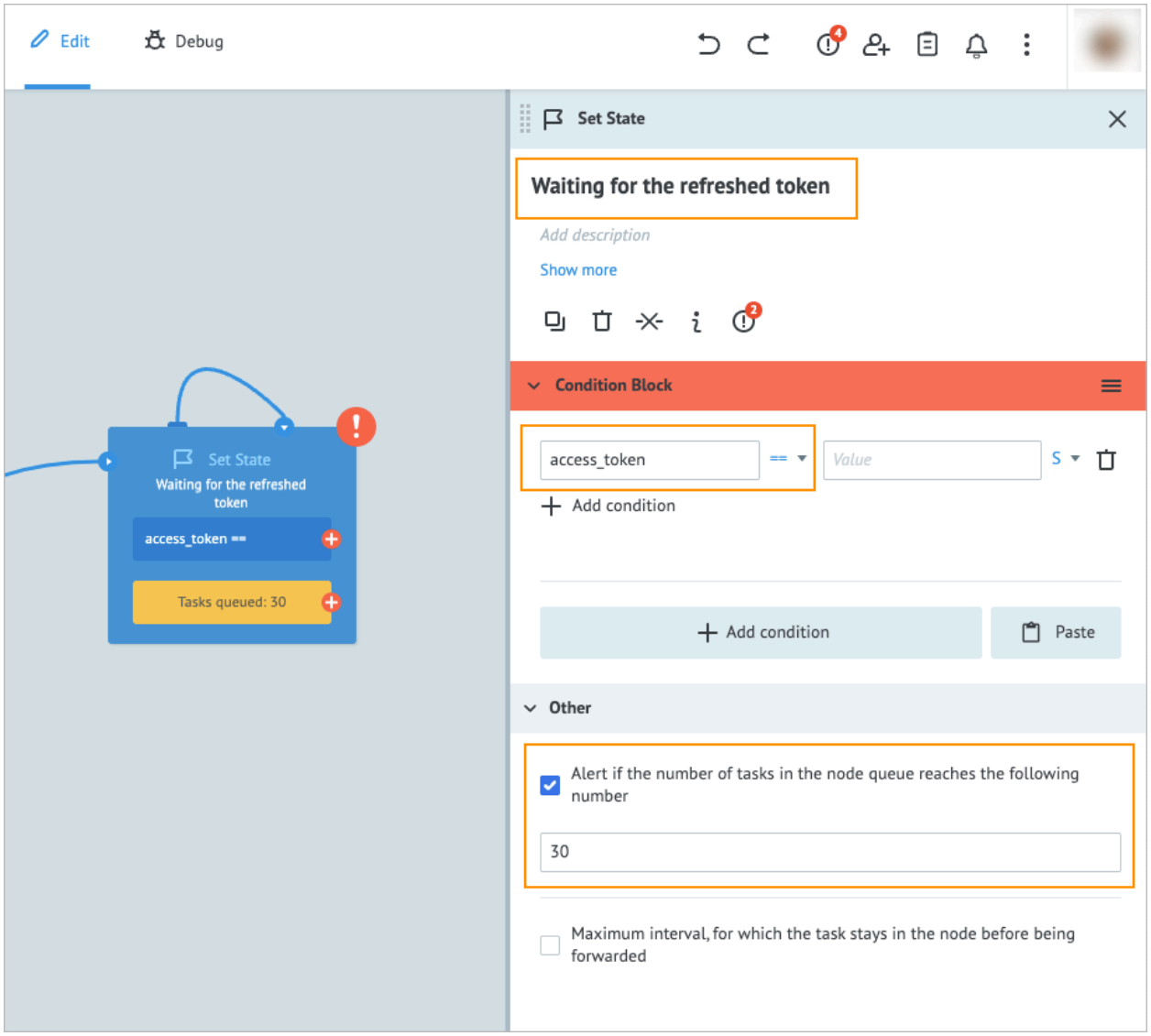
You create a request for **ACCESS_TOKEN** generation in the **Create/Refresh Token** process, and a copy of this request is sent to the **Token storage** state diagram. When it is time to refresh **ACCESS_TOKEN** during the Token storage process, the request will be passed through the **Copy Task** node for **ACCESS_TOKEN** refresh and wait for modification in the next **Set State** node.



6. In the **Set state** node details panel:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

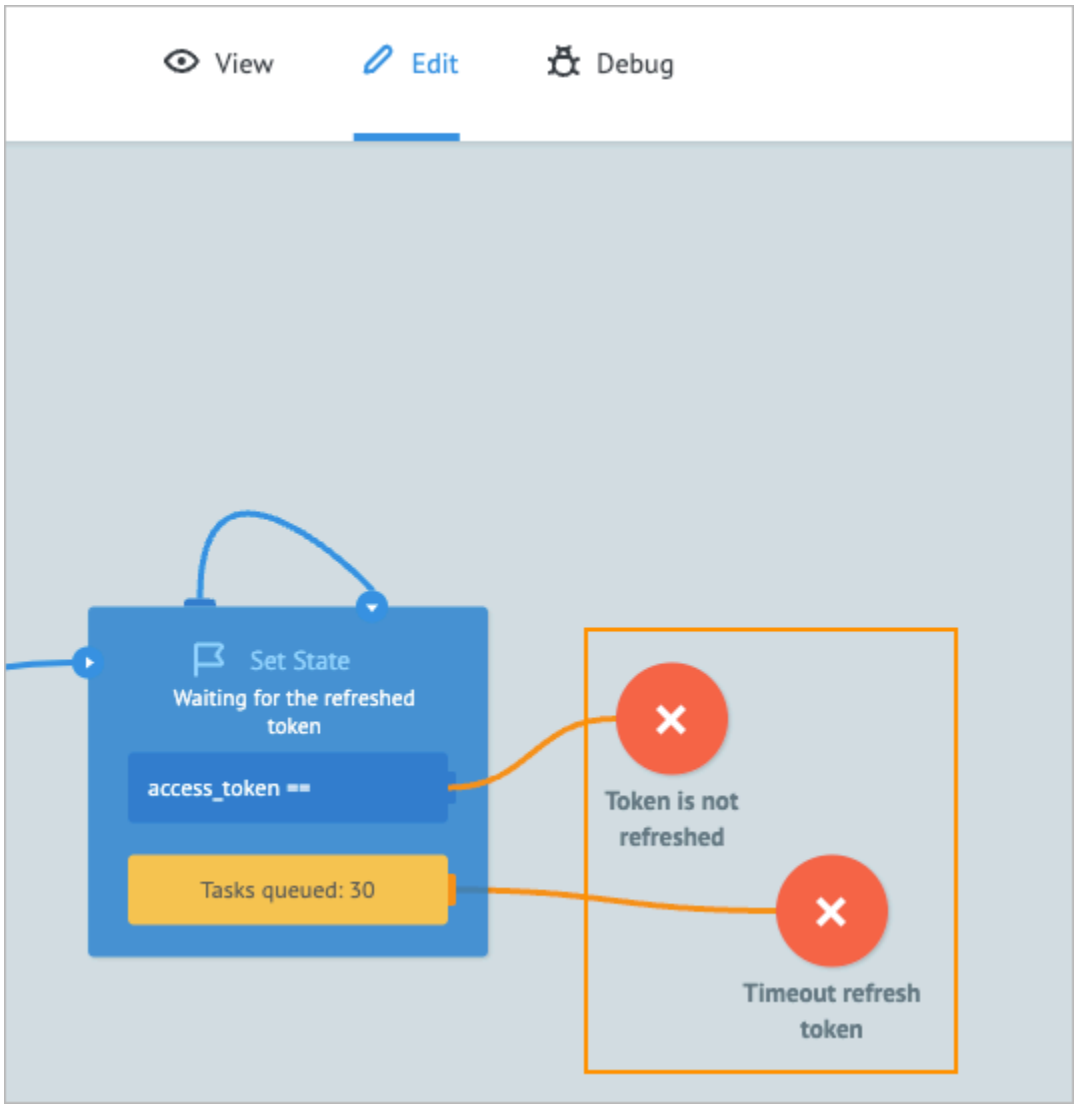
a. Change the default name to **Waiting for the refreshed token**.
b. Click **+ Condition** and enter **access_token ==** to check the presence of the access token parameter after the request modification.
c. Click **Other**, select the **Alert if the number of tasks in the node queue reaches the following number** checkbox, and enter **30** in the field below.



7. On the **Set State** node:
   a. Click the plus icon next to **access_token**, add the Final node and rename it to **Token is not refreshed**.
   Note: If an empty parameter is received, the request will be forwarded to the **Token is not refreshed** node.
   b. Click the plus icon next to **Tasks queued:30**, add the Final node and rename it to **Timeout refresh token**.

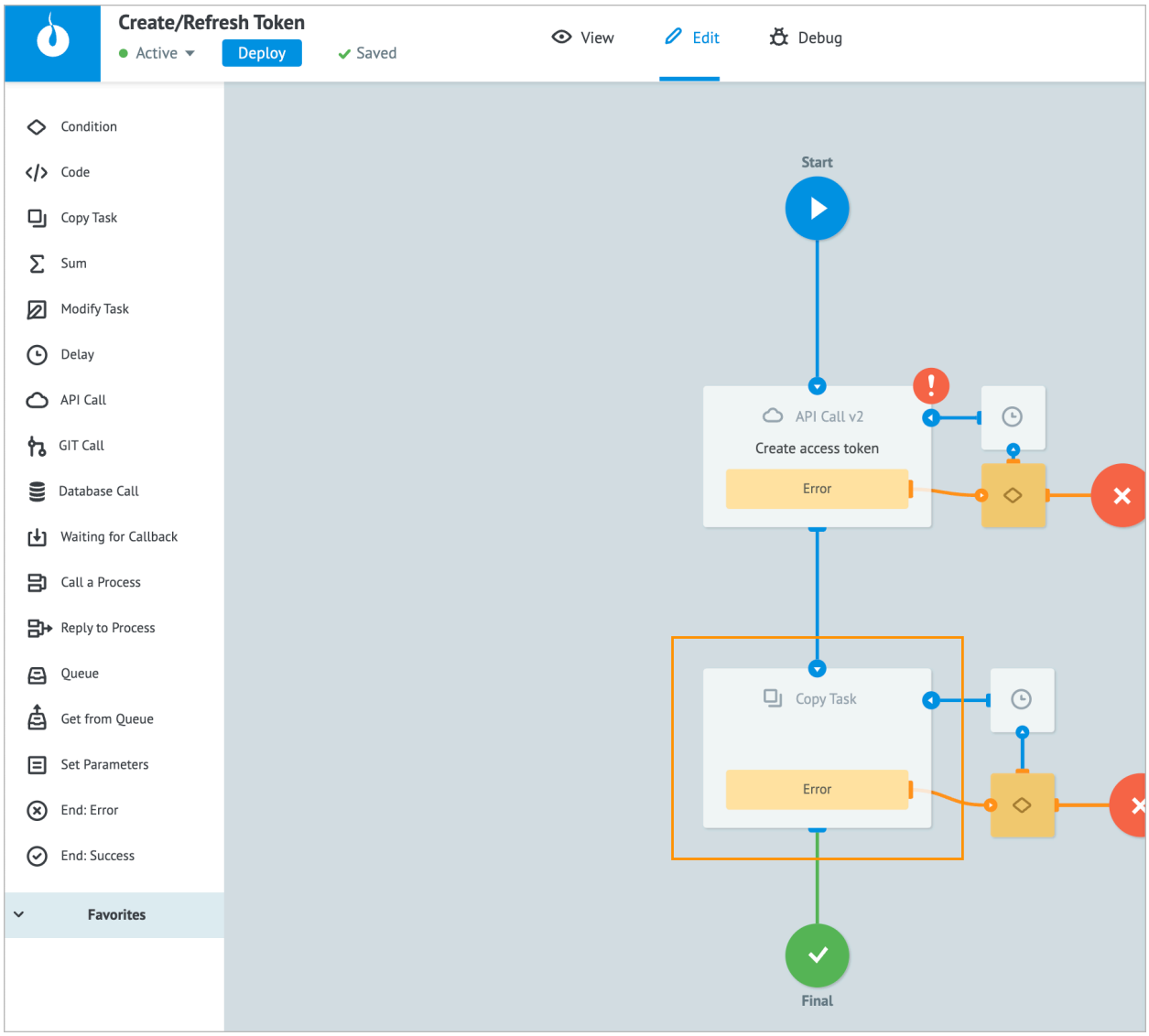
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Note: If a response is not received within 30 seconds, the request will be directed to the **Timeout refresh token** node.



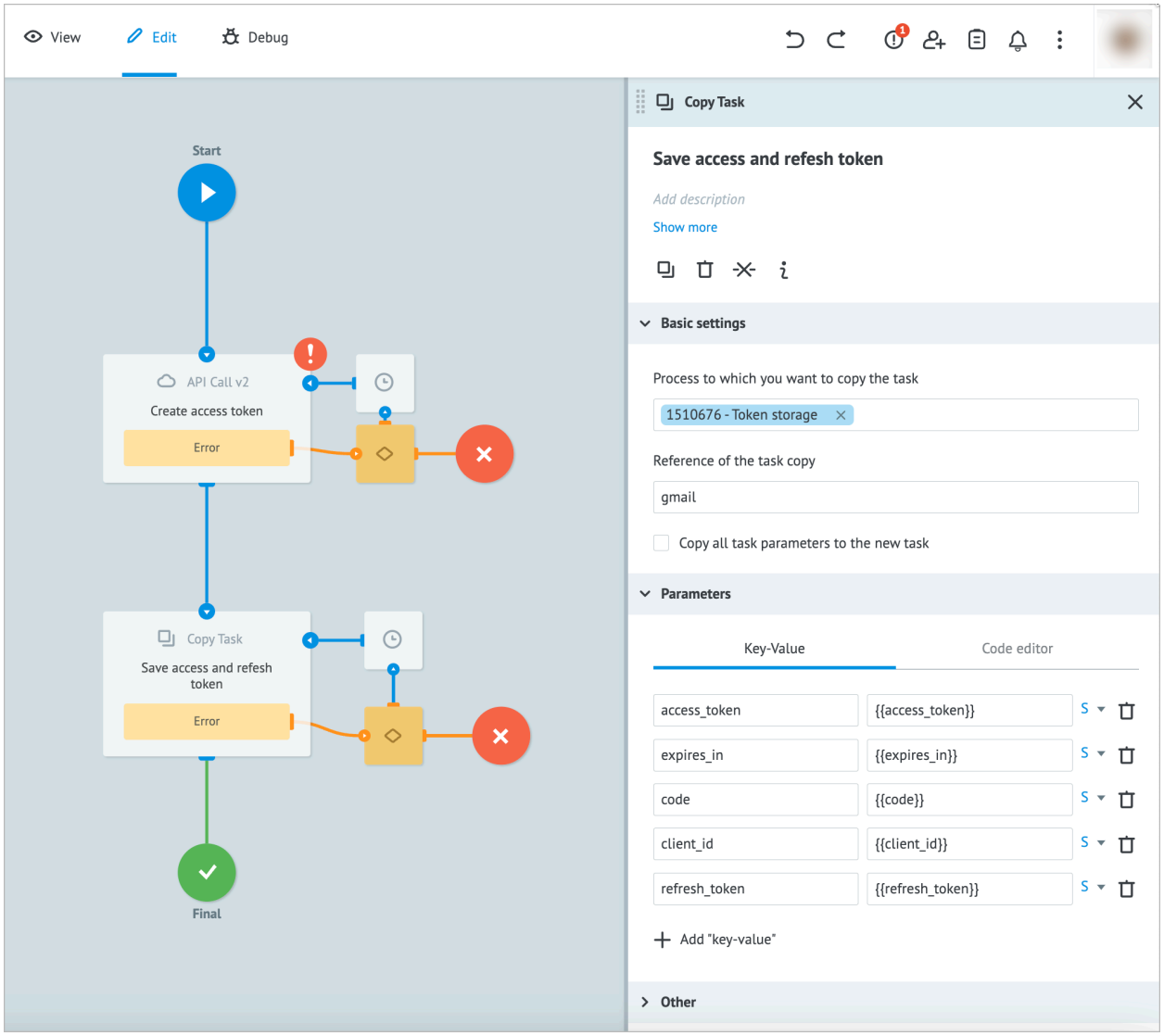
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

8. Go to the **Create/Refresh Token** process in the **Token storage** state diagram and add the **Copy Task** node after the **Create access token** node.



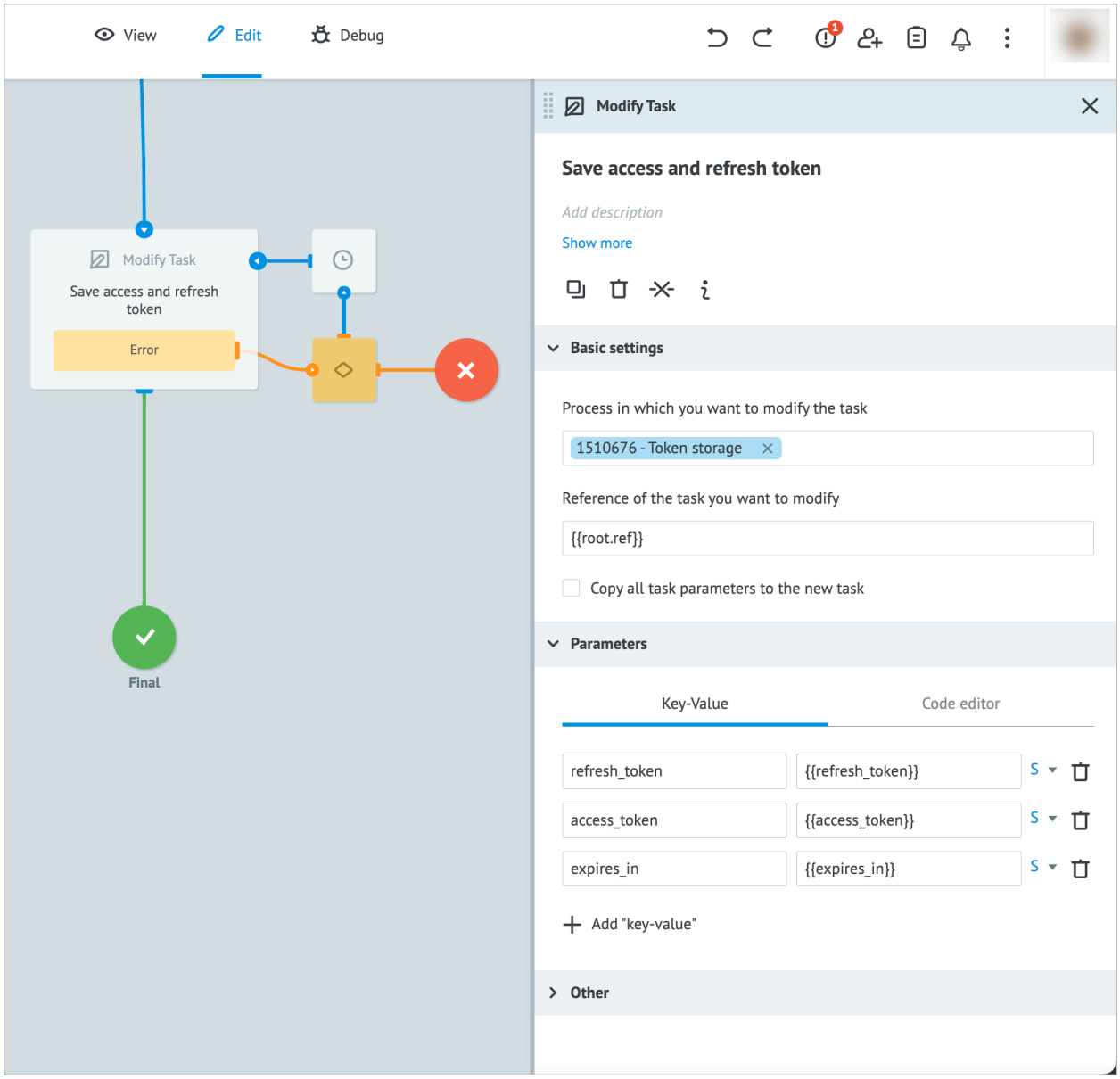
9. In the **Copy task** node details panel:
    a. Enter **Save access and refresh token** in the **Add title** field.
    b. Add the **Token storage** diagram in the **Process to which you want to copy the task** field.
    c. Enter **gmail** in the **Reference of the task copy** field.
    d. Enter **access_token**, **expires_in**, **code**, **client_id**, and **refresh_token** in the **Key** fields.
    e. Enter **{{access_token}}**, **{{expires_in}}**, **{{code}}**, **{{client_id}}**, and **{{refresh_token}}** in the **Value** fields.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



10. In the **Create/Refresh Token** process add the **Modify Task** node after the **Refresh access token** node.
11. In the **Modify task** node details panel:
    a. Enter **Save access and refresh token** in the **Add title** field.
    b. Add the **Token storage** diagram in the **Process to which you want to copy the task** field.
    c. Enter **{{root.ref}}** in the **Reference of the task copy** field.
    d. Enter **refresh_token**, **access_token**, and **expires_in** in the **Key** fields.
    e. Enter **{{refresh_token}}**, **{{access_token}}**, and **{{expires_in}}** in the **Value** fields.
    Note:
    - `refresh_token` is a token to refresh ACCESS_TOKEN.
    - `ACCESS_TOKEN` is a token to access Google API.
    - `expires_in` is ACCESS_TOKEN expiration time.

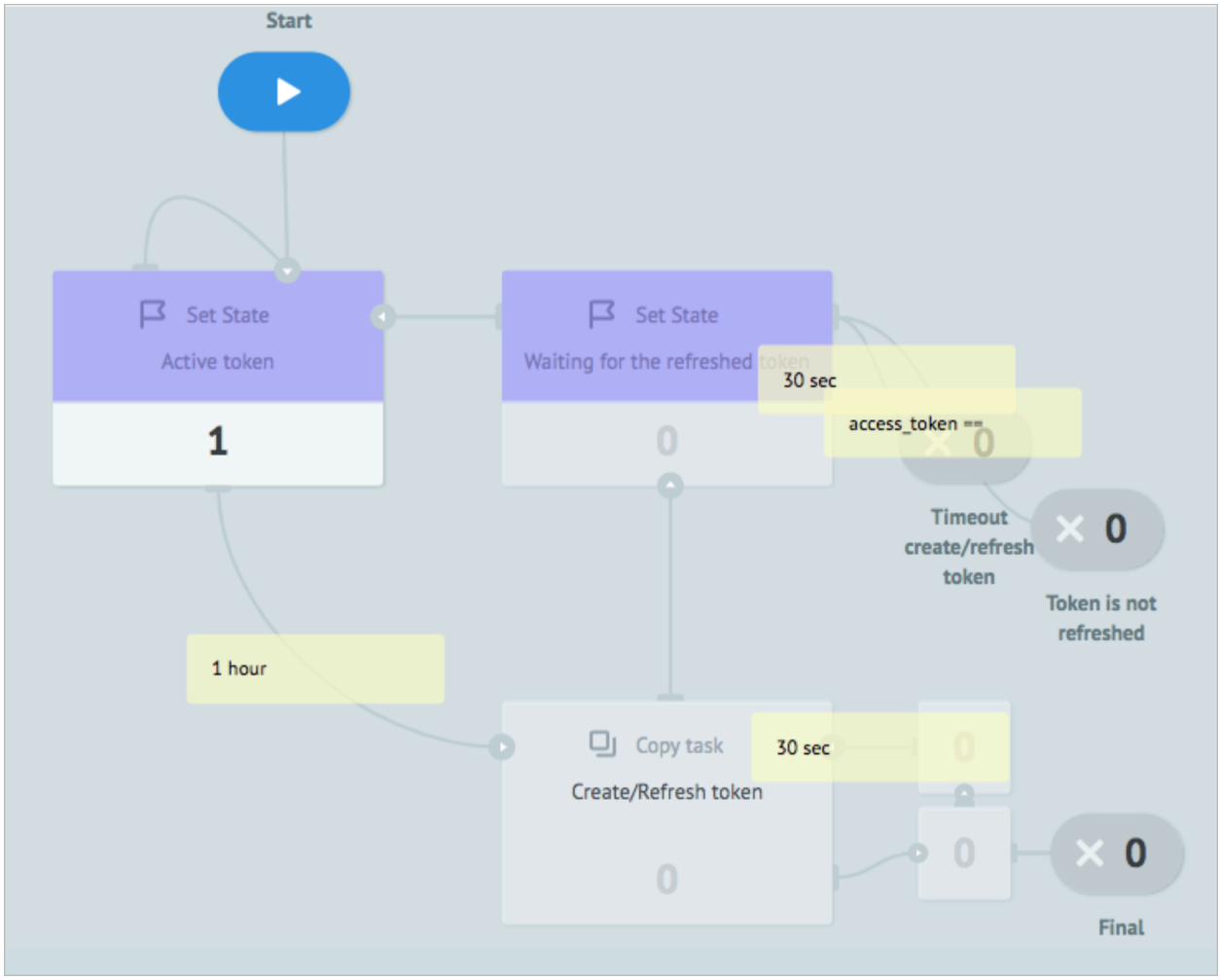
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



12. Repeat the point of getting **CODE** and create a new request for **ACCESS_TOKEN** generation in the **Create/Refresh Token** process, thereby launching a process of the token storage and refreshing in the **Token Storage** state diagram.
Note: If **ACCESS_TOKEN** is created successfully, your request will be in the **Active token** state. Click it to review the request that contains `access_token` as one of its

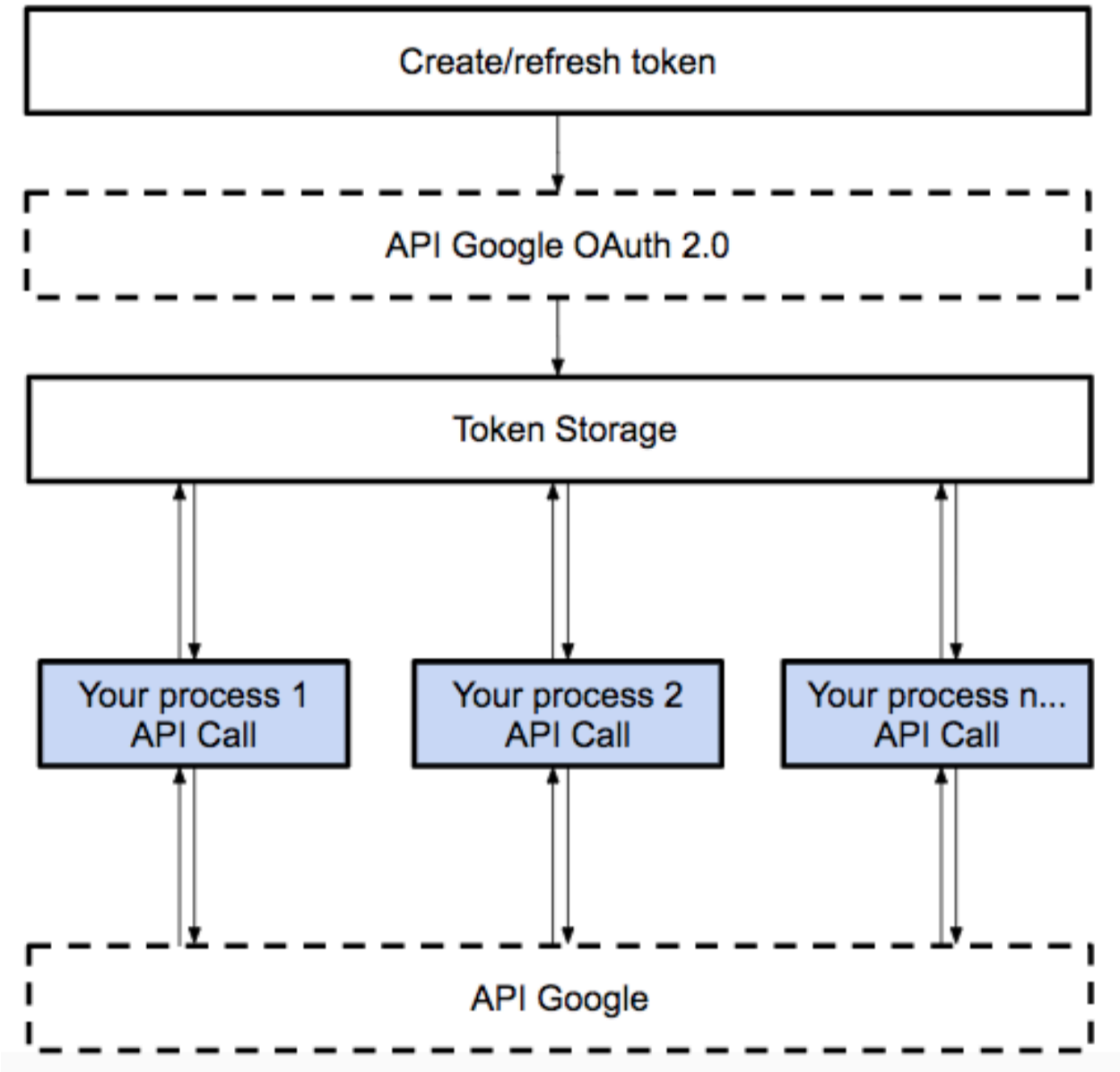
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

parameters.



2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Below, you can see the interaction of the Corezoid processes with the unified **Token Storage** state diagram and Google APIs.

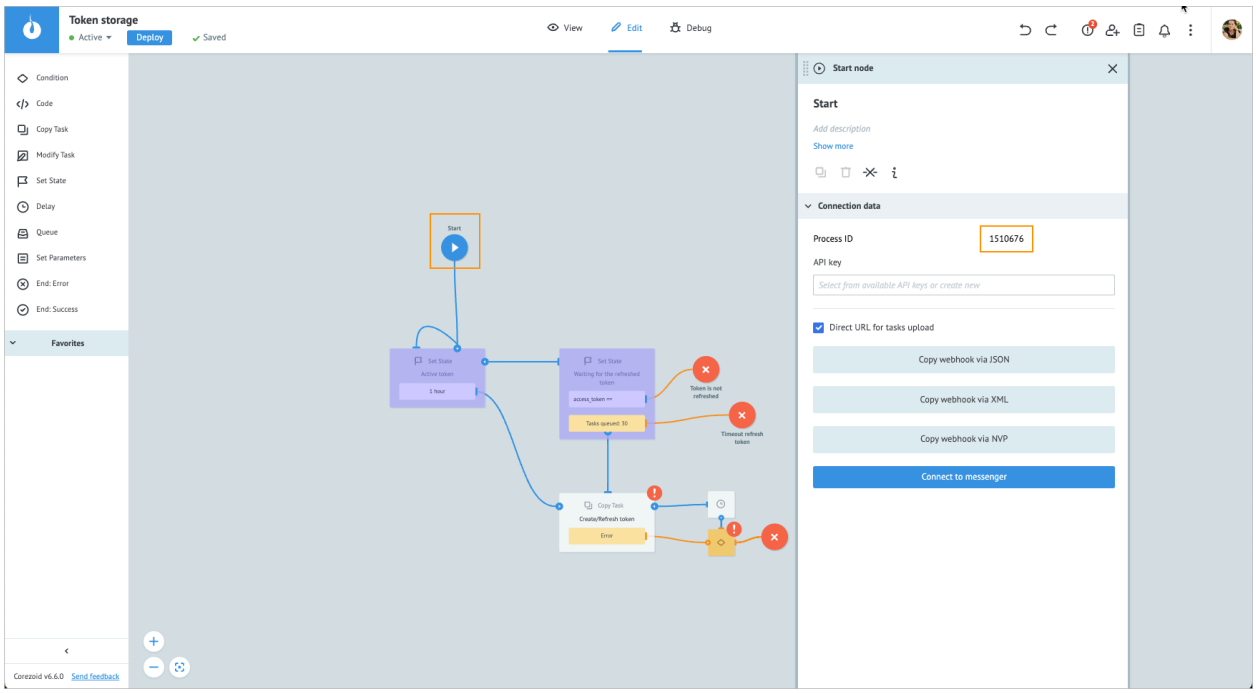


You can use the **ACCESS_TOKEN** obtained in all the processes of your company to work with any Google APIs by adding it to the Header as follows:
`Authorization: Bearer {{conv[DIAGRAM_ID].ref[REFERENCE].access_token}}`,
where:
`DIAGRAM_ID` is the ID of the **Token storage** state diagram.

To get the DIAGRAM_ID:

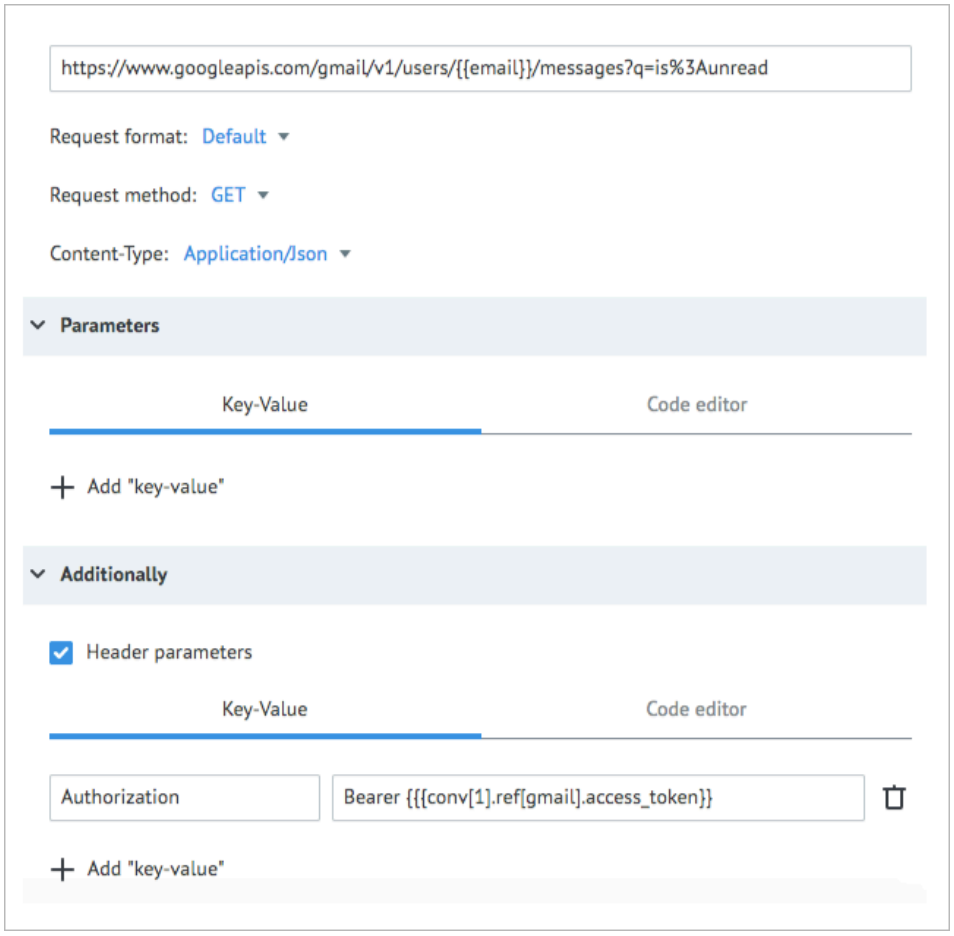
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

1. In the **Token Storage** state diagram, click the **Edit** mode.
2. Click the **Start** node of the state diagram and copy the number in the **ID process** value.



2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Example of Google API call via the **API Call** node using Google OAuth 2.0.



2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

9. References

9.1 Simulator documentation

9.2 Customer Agreement

Last update: June 28, 2022

BY ACCEPTING THIS AGREEMENT, YOU AGREE TO FOLLOW ITS TERMS AND CONDITIONS. IF YOU ENTER THIS AGREEMENT ON BEHALF OF A LEGAL ENTITY, YOU REPRESENT THAT YOU HAVE THE AUTHORITY TO BIND SUCH ENTITY TO THE TERMS AND CONDITIONS OF THIS AGREEMENT. BY CLICKING “I ACCEPT”, YOU ACKNOWLEDGE AND AGREE THAT YOUR SUBSCRIPTION WILL AUTOMATICALLY RENEW FOR SUCCESSIVE TERMS OF THE SAME DURATION AS YOUR INITIAL SUBSCRIPTION, AND WE WILL CONTINUE TO CHARGE YOU USING YOUR MOST RECENT PAYMENT METHOD ON FILE. YOU MAY CANCEL YOUR SUBSCRIPTION AT ANY TIME BY LOGGING INTO YOUR ACCOUNT AND CLICKING THE “CANCEL MY SUBSCRIPTION” IN YOUR ACCOUNT SETTINGS, AFTER WHICH NO ADDITIONAL CHARGES WILL BE MADE TO YOUR ACCOUNT. YOU MAY CONTACT US AT SUPPORT@COREZOID.COM WITH ANY QUESTIONS ABOUT CANCELING YOUR SUBSCRIPTION.

A. Definitions

- **Account**: Your account on the Corezoid Site associated with a valid e-mail address.
- **Agreement**: Is Corezoid customer agreement.
- **Corezoid Product**: Is the Services (including, without limitation, the Landing).
- **Confidential information**: All non-public information disclosed by Us, Our affiliates, business partners, or Our or their respective employees, contractors or agents that is designated as confidential or that, given the nature of the information or circumstances surrounding its disclosure, reasonably should be understood to be confidential. Confidential information includes:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

1. Non-public information relating to Our or Our affiliates' or business partners' technology, customers, business plans, promotional and marketing activities, finances, and other business affairs.
2. Third-party information that We are obligated to keep confidential.
3. The nature, content, and existence of any discussions or negotiations between You and Us. Confidential information does not include any information that:
    - Is or becomes publicly available without breach of this Agreement.
    - Can be shown by documentation to have been known to You at the time of Your receipt from Us.
    - Is received from a third party who did not acquire or disclose the same by a wrongful or tortious act.
    - Can be shown by documentation to have been independently developed by You without reference to the Confidential information.

- **Corezoid Site**: https://corezoid.com/ and any successor or related site designated by Us.
- **Customer Data**: All data, including all text, sound, video, or image files, and software, that are provided by You through the use of the Services.
- **End User**: Any person You permit to access Customer Data hosted in the Services or otherwise to use the Services.
- **Services**: The web services made available by Us via the Corezoid Site.
- **Landing**: Corezoid software We provide for use as part of Your Subscription to enable the functionality of the Services, including any updates or new releases made available in connection therewith.
- **Subscription**: An enrollment for the Services for a defined term as specified on the Corezoid Site.
- **Term**: The duration of a Subscription (e.g. 1 month or 12 months) as specified on the Corezoid Site.
- **You, Your**: Refers to the individual or the entity who has ordered the Services from Us.
- **Your Demo**: Content that You or any End User transfers for processing, storage, or hosting by the Services in connection with Your Account and any computational results that You or any End User derive from the foregoing through the use of the Services.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- **We**,**Our**, **Ours**, and **Us**: refers to Corezoid, Inc., a legal entity duly established, registered, and existing under valid laws of state Delaware (USA), registration number 5577397, registration address: 251 Little Falls Drive, Wilmington, New Castle Country, 19808, and all and any affiliates and/or subcontractors of Corezoid, Inc.

B. Rights granted

1. **Right to use**. Subject to the terms set forth in this Agreement, We grant You a nonexclusive, non-transferable, non-sublicensable, revocable, limited right to access and use the Services. It is strictly prohibited to download and/or install the Landing on Your own company servers, third-party servers, or third-party cloud servers without Our prior written consent or a separate written license agreement between Us and You. We reserve all other rights not expressly granted herein.
2. **Your Account**. To access the Services, You must create an Account. You are responsible for all activities that occur under Your Account. You are responsible for maintaining the confidentiality of any non-public authentication credentials associated with Your use of the Services. We are not responsible for unauthorized access to Your Account.
3. **Acceptable use**. You may use the Services and Landing only in accordance with this Agreement. You have no rights to reverse engineer, decompile, disassemble, or work around technical limitations in the Corezoid Product. You have no right to rent, lease, lend, resell, transfer, or host the Corezoid Product, or any portion thereof, to or for third parties.
4. **Customer Data**. You are solely responsible for the content of all Customer Data. You will secure and maintain all rights in the Customer Data necessary for Us to provide the Services to You without violating the rights of any third party or otherwise obligating Us to You or to any third party. You grant Us a worldwide, royalty-free, nonexclusive license to host and use any Customer Data provided through your use of the Services. We do not and will not assume any obligations with respect to Customer Data or to Your use of the Corezoid Product other than as expressly set forth in this Agreement.
5. **End Users**. You control access by End Users, and You are responsible for their use of the Corezoid Product in accordance with this Agreement.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

6. **Subcontractors**. We reserve the right to render the Services directly or through the use of contractors or subcontractors.

C. Ownership and restrictions

Ownership

You retain intellectual property rights and ownership in and to Customer Data. We retain intellectual property rights and ownership of the Services and Landing. You retain all ownership and intellectual property rights to materials resulting from the Services under this Agreement, except to the extent such materials incorporate the Corezoid Product in any manner.

Restrictions

You may not:
• Modify or remove any program marking or any notice of Our proprietary rights.
• Sell, distribute, license, lease, rent, assign, transfer, display, outsource, disclose, or make available the Corezoid Product to any third party other than as expressly permitted under the terms of this Agreement.
• Make the results of the Services available in any manner to any third party for use in such third party's business operations.
• Make derivative works of, modify, reverse compile, dissemble and/or reverse engineer any part of the Corezoid Product, or use or access the Corezoid Product in order to build or support, and/or assist a third party in supporting or building, services, products, or software competitive to the Corezoid Product.

D. Fees, taxes, and payment

1. **Fees**. You agree to pay for Services ordered, renewed by You, or renewed automatically pursuant to the applicable purchase, support, and other terms specified on the Corezoid Site. All prices are subject to change at the beginning of any Subscription renewal. If Your Subscription is automatically renewed, applicable fees for the Services will be charged from Your payment card. It is Your responsibility to maintain current and accurate credit card information on Your Account with Us.
2. **Late Fees**. We reserve the right to charge You interest at the rate of 1.5% per month (or the highest rate permitted by law, if less) on all late payments.
3. **Taxes**. You agree to pay any sales, value-added, or other similar taxes imposed by applicable law or similar amounts and duties that We are owed

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

under this Agreement and which We are permitted to collect from You under applicable law or that We must pay based on the Services ordered, except taxes based on Our income. Fees for Services specified on the Corezoid Site are exclusive of taxes and expenses.

4. **Payment.** All fees for the Services shall be paid in advance. You will pay Us the applicable fees and charges for the use of the Services as described on the Corezoid Site using one of the payment methods We support. All amounts payable under this Agreement will be made without setoff or counterclaim, and without any deduction or withholding, except as expressly permitted by this Agreement. Fees and charges for any new Services or new feature of the Services will be effective when We post updated fees and charges on the Corezoid Site unless We expressly state otherwise in a notice. We may increase or add new fees and charges for any existing Services by giving You at least 14 days advance notice. If You are subscribing to the virtual private cloud Services, You agree to pay for such Services according to the terms and conditions specified on the AWS marketplace or other marketplace designated by Us on the Corezoid Site.

E. Disclaimers

THE SERVICES ARE PROVIDED “AS IS.” WE MAKE NO REPRESENTATIONS OR WARRANTIES OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY OR OTHERWISE REGARDING THE SERVICES OR ANY THIRD-PARTY CONTENT ASSOCIATED WITH THE SERVICES, INCLUDING ANY WARRANTY THAT THE SERVICES OR ANY SUCH THIRD-PARTY CONTENT WILL BE UNINTERRUPTED, ERROR-FREE OR FREE OF HARMFUL COMPONENTS, OR THAT ANY CONTENT, INCLUDING YOUR CONTENT OR ANY THIRD-PARTY CONTENT, WILL BE SECURE OR NOT OTHERWISE LOST OR DAMAGED. EXCEPT TO THE EXTENT PROHIBITED BY LAW, WE DISCLAIM ALL WARRANTIES, INCLUDING ANY IMPLIED WARRANTIES OF MERCHANTABILITY, SATISFACTORY QUALITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE, NON-INFRINGEMENT, OR QUIET ENJOYMENT, AND ANY WARRANTIES ARISING OUT OF ANY COURSE OF DEALING OR USAGE OF TRADE.

F. Indemnification

1. **General**. You will defend, indemnify, and hold harmless Us, Our affiliates and licensors, and each of their respective employees, officers, directors, and representatives from and against any claims, damages, losses,

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

liabilities, costs, and expenses (including reasonable attorneys' fees) arising out of or relating to any third party claim concerning:

a. Your or any End Users' use of the Services (including any activities under Your Account and use by Your employees and personnel).
b. Breach of this Agreement or violation of applicable law by You or any End User.
c. Your Demo or the combination of Your Demo with other applications, content or processes, including any claim involving alleged infringement or misappropriation of third-party rights by Your Demo or by the use, development, design, production, advertising or marketing of Your Demo.
d. A dispute between You and any End User. If We or Our affiliates are obligated to respond to a third-party subpoena or other compulsory legal order or process described above, You will also reimburse Us for attorneys' fees, as well as Our employees' and contractors' time and materials spent responding to the third party subpoena or other compulsory legal order or process at Our then-current hourly rates.

2. **Process**. We will promptly notify You of any claim subject to this Section F, but Our failure to promptly notify You will only affect Your obligations under Section F to the extent that Our failure prejudices Your ability to defend the claim. You may:
    a. Use counsel of Your own choosing (subject to Our written consent) to defend against any claim.
    b. Settle the claim only with Our prior written consent, which we may withhold in our sole discretion. We may also assume control of the defense and settlement of the claim at any time.

G. Limitation of liability

WE WILL NOT BE LIABLE TO YOU FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, CONSEQUENTIAL, PUNITIVE, OR EXEMPLARY DAMAGES (INCLUDING DAMAGES FOR LOSS OF PROFITS, GOODWILL, USE, OR DATA), EVEN IF WE HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES. FURTHER, WE WILL NOT BE RESPONSIBLE FOR ANY COMPENSATION, REIMBURSEMENT, OR DAMAGES ARISING IN CONNECTION WITH:

1. YOUR INABILITY TO USE THE SERVICES, INCLUDING AS A RESULT OF ANY (I) TERMINATION OR SUSPENSION OF THIS AGREEMENT

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

OR YOUR USE OF OR ACCESS TO THE SERVICES, (II) OUR DISCONTINUATION OF ANY OR ALL OF THE SERVICES, OR, (III) ANY UNANTICIPATED OR UNSCHEDULED DOWNTIME OF ALL OR A PORTION OF THE SERVICES FOR ANY REASON, INCLUDING AS A RESULT OF POWER OUTAGES, SYSTEM FAILURES OR OTHER INTERRUPTIONS;

2. THE COST OF PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES;
3. ANY INVESTMENTS, EXPENDITURES, OR COMMITMENTS BY YOU IN CONNECTION WITH THIS AGREEMENT OR YOUR USE OF OR ACCESS TO THE SERVICES; OR
4. ANY UNAUTHORIZED ACCESS TO, ALTERATION OF, OR THE DELETION, DESTRUCTION, DAMAGE, LOSS OR FAILURE TO STORE ANY OF YOUR CONTENT OR OTHER DATA. IN ANY CASE, OUR AGGREGATE LIABILITY UNDER THIS AGREEMENT WILL BE LIMITED TO THE AMOUNT YOU ACTUALLY PAY US UNDER THIS AGREEMENT FOR THE SERVICES THAT GAVE RISE TO THE CLAIM DURING THE 12 MONTHS PRECEDING THE CLAIM.

H. Term, Termination, and Suspension

1. **Agreement Term and Termination**. This Agreement will remain in effect until the expiration of the Term or termination of Your Subscription, whichever is earliest. Upon expiration or termination, You must immediately stop using the Services and delete or destroy all copies of the Landing in your possession. Unless You notify us in writing that you wish to terminate Your Subscription at the end of the Term, Your Subscription will be automatically renewed for an additional Term of the same duration as the initial Term upon the end of then-current Term.
2. **Renewal**. Upon renewal of Your Subscription, this Agreement will terminate, and Your Subscription will thereafter be governed, by the terms and conditions set forth on the Corezoid Site on the date on which Your Subscription is renewed (the “Renewal Terms”). If you do not agree to any Renewal Terms, You may decline to renew Your Subscription.
3. **Subscription termination**. You may terminate Your Subscription at any time during the Term; provided, however, that no refunds of any prepaid amounts will be provided to You in connection with any such termination

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

prior to the expiration of the Term. Renewal Subscription will be terminated automatically if applicable fees are not credited to Our bank account for any reason within 3 (three) business days from the first day of the respective renewal Term.

4. **Suspension**. We may suspend Your use of the Services if: (a) We determine that it is reasonably needed to prevent unauthorized access to Customer Data or the proprietary information of any person or entity; (b) We determine that your continued use of the Services may infringe the intellectual property or other rights of Us or any third party; or (c) You do not pay any amounts due under this Agreement.
5. **Effect of Termination**. If Your Subscription is terminated or expired and is not renewed within 30 consecutive days from the last day of the paid Term, We will have the right to delete or destroy all Your data, Customer Data, and customized processes provided by You through the use of the Services, as well as materials resulting from the Services.

I. Miscellaneous

1. **Confidentiality and Publicity**. You may use Our Confidential information only in connection with Your use of the Services as permitted under this Agreement during the Term. You will not disclose Our Confidential Information to any third party during the Term or at any time thereafter. You will take all reasonable measures to avoid disclosure, dissemination, or unauthorized use of Our Confidential Information, including, at a minimum, those measures You take to protect Your own confidential information of a similar nature. You will not misrepresent or embellish the relationship between Us and You (including by expressing or implying that we support, sponsor, endorse, or contribute to You or Your business endeavors), or express or imply any relationship or affiliation between Us and You or any other person or entity except as expressly permitted by this Agreement.
2. **Force Majeure**. We will not be liable for any delay or failure to perform any obligation under this Agreement where the delay or failure results from any cause beyond Our reasonable control, including acts of God, labor disputes or other industrial disturbances, systemic electrical, telecommunications, or other utility failures, earthquake, storms or other elements of nature, blockages, embargoes, riots, acts or orders of government, acts of terrorism, or war.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3. **Independent Contractors; Non-Exclusive Rights**. We and You are independent contractors, and neither party nor any of their respective affiliates is an agent of the other for any purpose or has the authority to bind the other.
4. **No Third-Party Beneficiaries**. This Agreement does not create any third-party beneficiary rights in any individual or entity that is not a party to this Agreement.
5. **Import and Export Compliance**. In connection with this Agreement, each party will comply with all applicable import, re-import, export, and re-export control laws and regulations, including the Export Administration Regulations, the International Traffic in Arms Regulations, and country-specific economic sanctions programs implemented by the Office of Foreign Assets Control. For clarity, You are solely responsible for compliance related to the manner in which You choose to use the Services, including Your transfer and processing of Your Demo, the provision of Your Demo to End Users, and the jurisdiction in which any of the foregoing occur. You represent and warrant that neither You nor any of Your End Users are:
    a. Located in a country that is subject to a U.S. Government embargo, or that has been designated by the U.S. Government as a “terrorist supporting” country.
    b. Listed on any U.S. Government list of prohibited or restricted parties.
6. **Assignment**. You will not assign this Agreement, or delegate or sublicense any of Your rights under this Agreement, without Our prior written consent. Subject to the foregoing, this Agreement will be binding upon, and inure to the benefit of the parties and their respective successors and assigns.
7. **No Waivers**. The failure by Us to enforce any provision of this Agreement will not constitute a present or future waiver of such provision nor limit Our right to enforce such provision at a later time. All waivers by Us must be in writing to be effective.
8. **Severability**. In the event that any of the provisions of this Agreement shall be held by a court or other tribunal of competent jurisdiction to be unenforceable, such provision will be enforced to the maximum extent permissible and the remaining portions of this Agreement shall remain in full force and effect. In the event of any conflict between any provision hereof and any applicable laws to the contrary, the latter shall prevail, but

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

this Agreement shall be deemed modified only to the extent necessary to remove such conflicts.

9. **Governing Law**. This Agreement shall be construed, interpreted, and enforced in accordance with and shall be governed by the laws of the State of Delaware.
10. **Disputes**. Any dispute arising out of or relating to this Agreement, which is not possible to resolve by means of friendly negotiations shall be settled by the state courts located in San Mateo County, California, or the U.S. District Court for the Northern District of California, if applicable. YOU AGREE THAT ANY AND ALL DISPUTES UNDER THIS AGREEMENT MUST BE BROUGHT IN YOUR INDIVIDUAL CAPACITY AND NOT AS A PLAINTIFF OR CLASS MEMBER IN ANY PURPORTED CLASS OR REPRESENTATIVE PROCEEDING. YOU HEREBY WAIVE THE RIGHT TO PARTICIPATE IN A CLASS ACTION OR LITIGATE ON A CLASS-WIDE BASIS. YOU AGREE THAT YOU HAVE EXPRESSLY AND KNOWINGLY WAIVED THESE RIGHTS.
11. **Entire Agreement; English Language**. This Agreement is the entire agreement between You and Us regarding the subject matter of this Agreement. This Agreement supersedes all prior or contemporaneous representations, understandings, agreements, or communications between You and Us, whether written or verbal, regarding the subject matter of this Agreement. We will not be bound by, and specifically object to, any term, condition, or other provision which is different from or in addition to the provisions of this Agreement (whether or not it would materially alter this Agreement) and which is submitted by You in any order, receipt, acceptance, confirmation, correspondence or other document. If We provide a translation of the English language version of this Agreement, the English language version of the Agreement will control if there is any conflict.
12. **Notice**:
    a. **To You**. We may provide any notice to You under this Agreement by: (i) posting a notice on the Site; or (ii) sending a message to the email address then associated with Your Account. Notices We provide by posting on the Site will be effective upon posting and notices We provide by email will be effective when We send the email. It is Your responsibility to keep Your email address current. You will be deemed to have received any email sent to the email address then

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

associated with Your Account when We send the email, whether or not You actually receive the email.

b. **To Us**. All and any notices to Us shall be addressed by electronic mail to: support@corezoid.com.(c) Language. All communications and notices to be made or given pursuant to this Agreement must be in the English language.

13. **Support Statement**. We will support customers who run Corezoid Actor Engine products on supported Operating Systems, irrespective of whether they are running in VMware environments or not. We support Operating Systems, not specific hardware configurations. Accordingly, VMware operates as a hardware abstraction layer. VMware supports a set of certified Operating Systems and Hardware, and the customer and VMware will be responsible for any interactions or issues that arise at the Hardware or Operating System layer as a result of their use of VMware. We will not require You to recreate and troubleshoot every issue in a non-VMware environment; however, We do reserve the right to request You to diagnose certain issues in a native certified Operating System environment, operating without the virtual environment. We will only make this request when there is reason to believe that the virtual environment is a contributing factor to the issue.

Any time spent on investigation of problems that may, in our sole opinion, be related to VMware, will be handled in the following fashion:

1. We will provide standard support to all Corezoid Actor Engine products.
2. If a problem is encountered while Corezoid Actor Engine is/are running in a VMware environment, You may be required to recreate the problem on a non-VMware server unit, at which time we will provide regular support.
3. You can authorize Us to investigate the VMware-related items at normal time and materials rates. If such investigation shows that the problem is VMware related, You may contract Us to provide a software change to resolve the issue if such a resolution is possible.
4. Regardless of the problem type or source, if the problem is determined to be a non-VMware-related issue - time spent on investigation and resolution will be covered as part of regular maintenance, and support will be provided as usual.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Amazon Linux 2 is a supported OS for Corezoid Actor Engine. You will receive the same support on Amazon Linux 2 as they would on any other certified operating system.

9.3 Privacy Policy

Last update: June 28, 2022

Corezoid, Inc. and its affiliates (“Corezoid”, “we” or “us”) are committed to respecting your online privacy and keeping your information secure and confidential.

This Privacy Policy (“Privacy Policy”) explains how Corezoid collects, uses, discloses, and secures the Personal Information you (“customer”, “you” or “your”) provide during your online and offline interactions with us in connection with the website located at corezoid.com and any subdomains, as applicable (the “Site”) in order to use Corezoid (“Product”).

As used in this Privacy Policy, ”Personal Information“ means any information that can be used to individually identify a person, and may include, but is not limited to, name, email address, postal or other physical address, only 4 last digits of your credit or debit card number, title, and other personally identifiable information.

If you reside in the European Union (“EU”), United Kingdom, Lichtenstein, Norway, Iceland, or Switzerland, you may have additional rights with respect to your Personal Information, as further outlined below. These rights may include rights under the EU’s General Data Protection Regulation (“GDPR”).

Corezoid will be the controller of your Personal Information that is provided, collected, and/or processed pursuant to this Privacy Policy, except where otherwise expressly stated below. If you have any questions about whether any of the foregoing applies to you, please contact us using the information set forth in the “Contact Us”.

This Privacy Policy may be updated from time to time for reasons such as operational practices or regulatory changes, so we recommend that you review our Privacy Policy when returning to our Site.

Information collected

This Privacy Policy applies to Personal Information we collect from you, but does not apply to any services or products offered by third parties and made available to you through our Site. We obtain Personal Information in the following forms: (i) information provided by you as part of the

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

sign-up process, and (ii) information that we obtain or collect automatically from your use of the Site.

Personal Information provided by you: You may be required to register in order to access certain features of the Site. To become a registered user, you will be asked to sign up to create an account for the Site, and will be asked to provide:

- Your name
- Password
- E-mail address
- Home, work, or mobile telephone number
- 4 last digits of your credit or debit card number
- Google+ profile information (when you use Google+ to log in to our Site or to create an account on our Site)
- Facebook profile information, namely the email used for your Facebook account (when you use Facebook to log in to our Site or to create an account on our Site)
- GitHub profile information, namely the email used for your GitHub account (when you use GitHub to log in to our Site or to create an account on our Site)
- IP addresses and other information are collected passively, as further detailed in the “Information that we obtain or collect automatically from your use of the Site”

Information that we obtain or collect automatically from your use of the Site: We may automatically collect certain information from your use of the Product, such as usage data, preferences, or any other data that includes Information concerning how you use the Product.

In particular, our servers automatically record certain information, and/or we may use cookies (small text files placed on a device’s hard disk by a web service) or similar tracking technologies to collect the following types of information:

- Your browser type, language, plug-ins, Internet domain, and operating system
- Your Internet Protocol (IP) address
- The website(s) you visited before visiting the Site and the website(s) you visited after visiting the Site
- Unique identifiers, including mobile device identification numbers, that may identify the physical location of such devices

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- Data related to your interactions with content on the Site, such as use patterns and the type of content viewed or favorited on the Site

In addition, by using our Product, network information may be transmitted back to us such as Product usage information. This information is transmitted back to us so we can determine how users are interacting with our Products, to assist us with improving our Products, and to correct any problems that may occur.

Geolocation

We collect your location-based information for the purpose of optimizing your connection to our data center. We will only share this information with our data center provider for the sole purpose of providing you with this service.

If you do not wish to allow us to share your information in this manner you may opt-out of location-based services at any time by editing the setting at the device level or by contacting us using the information in the “Contact Us” section of this Privacy Policy.

Tracking technologies

We and our partners may use cookies or similar technologies to analyze trends, administer the website, track users’ movements around the website, and gather demographic information about our user base as a whole. Users can control the use of cookies at the individual browser level.

Please note that you may set most browsers to notify you if you receive a cookie or you may choose to block cookies with your browser, but if you choose to erase or block your cookies, you will need to re-enter your original user ID and password to gain access to certain parts of the Site.

Choice

Please note that you may choose not to create an account. If you so choose, you will still have access to many of the Site’s web pages, however, you may be unable to access certain options and services that we offer only to registered users. You can choose whether to provide Personal Information to us, but note that you may be unable to access certain options, offers, and services if they require Personal Information that you have not provided. You can sign up, and therefore consent, to receive email or newsletter communications from us. If you would like to discontinue receiving these communications, you can update your preferences by using the “Unsubscribe” link found in such emails or by contacting us using the information in the “Contact Us” section of this Privacy Policy.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Use and disclosure of information

Corezoid uses your Personal Information to:

- Operate and improve the Site and/or Product
- Customize your experience on the Site and/or Product
- Send you communications that may be of interest to you, either electronically or otherwise
- Enforce this Privacy Policy
- Enforce Corezoid customer agreement
- Conduct market analysis, traffic flow analysis, and related reporting to third parties
- Provide reports based on information collected from the use of our Product and Site
- Keep you up to date on the latest Product announcements, software updates, software upgrades, system enhancements, special offers, and other information, provided that you can edit your marketing preferences
- Provide support and assistance for our Product and Site
- Provide the ability to create personal profile areas and view protected content
- Provide the ability to contact you, and provide you with shipping and billing information
- Provide customer feedback and support
- Conduct questionnaires and surveys in order to provide better products and services to our customers and end users. Your completion of any questionnaires is voluntary
- Understand more about you so that we can personalize newsletters and websites to your preferences. For example, allows you the opportunity to request specific information on products and services that may be of interest

For any other purposes disclosed to you at the time, we collect your Personal Information

Sharing your personal information

We may share certain Personal Information with third-party vendors or subcontractors who provide software applications, web hosting, and other technologies or services for the Site. We will only provide these third parties with access to Personal Information that is reasonably

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

necessary to perform their work or comply with law. Those third parties will never use such Personal Information for any other purpose except to provide services in connection with the Site.

We will not share or sell any of your Personal Information to any third party except as otherwise stated in this Privacy Policy and in the following circumstances: (a) in response to subpoenas, court orders or legal process, to the extent permitted and as restricted by law; (b) when disclosure is required to maintain the security and integrity of the Site, or to protect any user's security or the security of other persons, consistent with applicable laws; (c) when disclosure is directed or consented to by the user who has input the Personal Information; (d) in the event that we go through a business transition, such as a merger, divestiture, acquisition, liquidation or sale of all or a portion of its assets, your Personal Information will, in most instances, be part of the assets transferred; or (e) in limited circumstances, we may disclose your email address in order to comply with laws and regulations, including the Controlling the Assault of Non-Solicited Pornography and Marketing Act (CAN-SPAM Act of 2003).

Please note that we may retain certain Personal Information after your account has been terminated. We reserve the right to use your Personal Information in any aggregated data collection after you have terminated your account with us, however, we will ensure that the use of such information or data will not identify you personally. For the avoidance of doubt, payment information will be deleted once your account is terminated.

Security of information

Personal Information you provide to us is protected by the password you create when registering to use the Site. Please understand that you can help prevent the unauthorized disclosure of information by choosing and protecting your password appropriately. You can also help prevent unauthorized disclosure by not sharing your password and preventing others from using your computer or mobile device.

We have implemented reasonable administrative, technical, and physical security measures to protect your personal information against unauthorized access, destruction, or alteration. For example:

- Databases encryption
- Passwords encryption
- SSL encryption (https) is everywhere where we deal with Personal Information
- Data is kept on secure servers, located in the EU

We use commercially reasonable procedures to protect all collected Personal Information. Please understand that no security system is perfect and, as such, we do not guarantee the Site's security, or that your Personal Information won't be intercepted while being transmitted to

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

us. If we learn of a security systems breach, then we may either post a notice, or attempt to notify you by email, and we will take reasonable steps to remedy the breach.

Data subject rights

You have certain rights with respect to your Personal Information as set forth below. Please note that in some circumstances, we may not be able to fully comply with your requests, or we may ask you to provide us with additional information in connection with your request, which may be Personal Information, for example, if we need to verify your identity or the nature of your request. In such situations, however, we will still respond to let you know of our decision.

To make any of the following requests, contact us using the contact details referred to in the “Contact Us” section of this Privacy Policy.

- **Access**: You can request more information about the Personal Information we hold about you. You can also request a copy of the Personal Information.
- **Rectification**: If you believe that any Personal Information we are holding about you is incorrect or incomplete, you can request that we correct or supplement such data. You can also correct some of this information directly by logging into your service account. Please contact us as soon as possible upon noticing any such inaccuracy or incompleteness.
- **Objection**: You can contact us to let us know that you object to the collection or use of your Personal Information for certain purposes.
- **Erasure**: You can request that we erase some or all of your Personal Information from our systems.
- **Restriction of Processing**: You can ask us to restrict further processing of your Personal Information.
- **Portability**: You have the right to ask for a copy of your Personal Information in a machine-readable format. You can also request that we transmit the data to another entity where technically feasible.
- **Withdrawal of Consent**: If we are processing your Personal Information based on your consent (as indicated at the time of collection of such data), you have the right to withdraw your consent at any time. Please note, however, that if you exercise this right, you may have to then provide express consent on a case-by-case basis for the use or disclosure of certain of your Personal Information, if such use or disclosure is necessary to enable you to utilize some or all of our Products.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

- **Right to File Complaint**: You have the right to lodge a complaint about our practices with respect to your Personal Information with the supervisory authority of your country or EU Member State.

Your California privacy rights

California residents who have an established business relationship with us may make a written request to Corezoid about whether we have disclosed any Personal Information to any third parties for the third parties' direct marketing purposes during the prior calendar year. To make such a request, please contact us using the contact details referred to in the "Contact Us" section of this Privacy Policy.

Coppa and access by minors

Corezoid does not knowingly collect or store any personal information from or about children under the age of 16.

If you believe a child under the age of 16 has under any circumstances provided us with personal information and data, a parent or legal guardian can contact us using the contact details referred to in the "Contact Us" section of this Privacy Policy to request that their children's information be deleted from our records.

Changes to this privacy policy

We will occasionally update this Privacy Policy to reflect customer feedback and changes in our Services. When we post changes to a statement, we will revise the "last updated" date at the top of the statement. If there are material changes to the statement or in how Corezoid will use Services information, we will notify you either by posting a notice of such changes before they take effect or by directly sending you a notification.

User consent

By using Product and/or Site, you consent to the terms of this Privacy Policy and to our processing of Personal Information for the purposes given above as well as those explained where Corezoid collects Personal Information.

We may transfer your Personal Information to the United States, to any Corezoid affiliate worldwide, or to third parties acting on our behalf for the purposes of processing or storage, and by providing any Personal Information you fully understand and unambiguously consent to such transfer, processing and storage of such information.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Contact us

If you have any inquiries or complaints, you may contact us by:

- Email: support@corezoid.com.
- Post: 2093 Philadelphia Pike #2235 Claymont 19703 DE, US
- Phone: +1 (908) 955-4333

9.4 Cookie Policy

Last update: June 28, 2022

Corezoid, Inc. (“Corezoid”) and our partners use cookies or similar technologies to analyze trends, administer and track users’ movements during your visit to our website located at corezoid.com and any subdomains, as applicable (the “Site”) or use of Corezoid (“Product”), and gather information about you, where you access our Site or Products and how you use our Product and Site.

What are cookies and does Corezoid use them?

Cookies are small text files that are placed on your computer by websites and services that you visit or access. They are widely used to make websites and services work and function with greater efficiency and to provide information about our users' experience during the use of, or interaction with, our websites, Products, services, and advertisements. Some cookies last only for the duration of your web session and expire when you exit your browser; other cookies may last for longer than your web session, including after you exit your browser, for example by remembering you when you return to our website. The table below explains the cookies that Corezoid and our third-party partners use and why.

Cookie	Purpose	Practice

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Functional Cookies	**Preferences and Settings**	These cookies are used to record a user's choice and settings that enable our websites and Products to operate correctly or that maintain your preferences over time and may be stored on your device. For example, Corezoid saves preferences, such as language, browser, and multimedia player settings, which enables the browser to remember these settings each time you return to the site. **Note:** *If you opt out of interest-based advertising, we store your opt-out preference in a cookie on your device*.
Advertising Cookies	**Interest-Based Advertising**	Corezoid uses cookies to collect data about your online activity and identify your interests so that we can provide advertising that is most relevant to you. You can opt out of receiving interest-based advertising from Corezoid as described in the How to Control Cookies section of this cookie policy and in our Privacy Policy.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Universal Analytics	**Analytics**	To provide our products and improve your user experience on our websites and with our Products, Corezoid uses cookies and other identifiers to gather usage and performance data. For example, we use cookies to count the number of unique visitors to a web page or service or to our blog and to develop other statistics about the operations of our Products. This includes cookies from Corezoid and from third-party analytics providers. We use the information to compile reports and to help us improve our websites and Products.
Service Cookies	**Performance**	Corezoid uses performance cookies for load balancing to ensure that websites and Products remain up and operating.

How to control cookies

Most web browsers automatically accept cookies but provide controls that allow you to block or delete them. For example, in most modern browsers, you can block or delete cookies by clicking **Settings > Privacy > Cookies**. Instructions for blocking or deleting cookies in other browsers may be available in each browser's privacy or help documentation. To find out more about cookies, including how to see what cookies have been set and how to manage and delete them, visit www.aboutcookies.org.

Certain features of the Product and services depend on cookies. Please be aware that if you choose to block cookies, you may not be able to sign in or use those features, and preferences that are dependent on cookies may be lost. If you choose to delete cookies, settings, and preferences controlled by those cookies, including advertising preferences, will be deleted and may need to be recreated.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Changes to this privacy policy

We will occasionally update this Cookie Policy to reflect customer feedback and changes in our Product. When we post changes to a statement, we will revise the “last updated” date at the top of the statement. If there are material changes to the statement or in how Corezoid will use Product information, we will notify you either by posting a notice of such changes before they take effect or by directly sending you a notification.

Contact us

If you have any inquiries or complaints, you may contact us by:
• Email: support@corezoid.com.
• Post: 2093 Philadelphia Pike #2235 Claymont 19703 DE, US
• Phone: +1 (908) 955-4333

9.5 Send Notification Emails

List of Corezoid events for which notification emails are sent

There are seven types of system messages in Corezoid:

- Password recovery
- Registration confirmation
- Invite a user to a company
- Object sharing
- Changing a user's password
- Object download
- Task download
- Task upload error

After installing the system, you need to create and configure a process that sends these system messages or only some of them.

Depending on the action, an object with different parameters is sent to the process, among them there will always be two mandatory parameters:

- **type** - email type
- **recipients** : [{email}] - users’ emails for mailing

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Optional parameter:

lang - user language - depending on it, you can prepare email templates in different languages. If a user has not selected a language, the default value will be lang = en.

Message	Task parameters
Password recovery	● **type** = password_recovery ● **recipients** = [{email}] ● **url** - link for a password recovery
Registration confirmation	● **type** = confirm_registration ● **recipients** = [{email}] ● **url** - link for registration confirmation ● **nick** - username ● **lang** - user language
Invite a user to a company	● **type** = invite ● **recipients** = [{email}] ● **company_name** - if a user was invited to a specific company (not to My Corezoid), then the name of this company will be among the parameters ● **url** - link for confirming an invitation ● **lang** - user language ● **login** - inviter login

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Object sharing	• **type** = share • **recipients** = [{email}] • **company_name** - if a shared object was created in a specific company, then the name of this company will be among the parameters • **url** - link to a shared object • **lang** - user language • **login** - inviter login • **obj_type** - shared object type (possible values: *conv, folder, dashboard, instance, config*) • **obj_name**, **object_name** - shared object name • **obj_id** - shared object ID
Changing a user password	• **type** = expire_password_notify • **recipients** = [{email}] • **reset_password_url** - link to password reset • **secs_to_expire** - seconds until a password expires

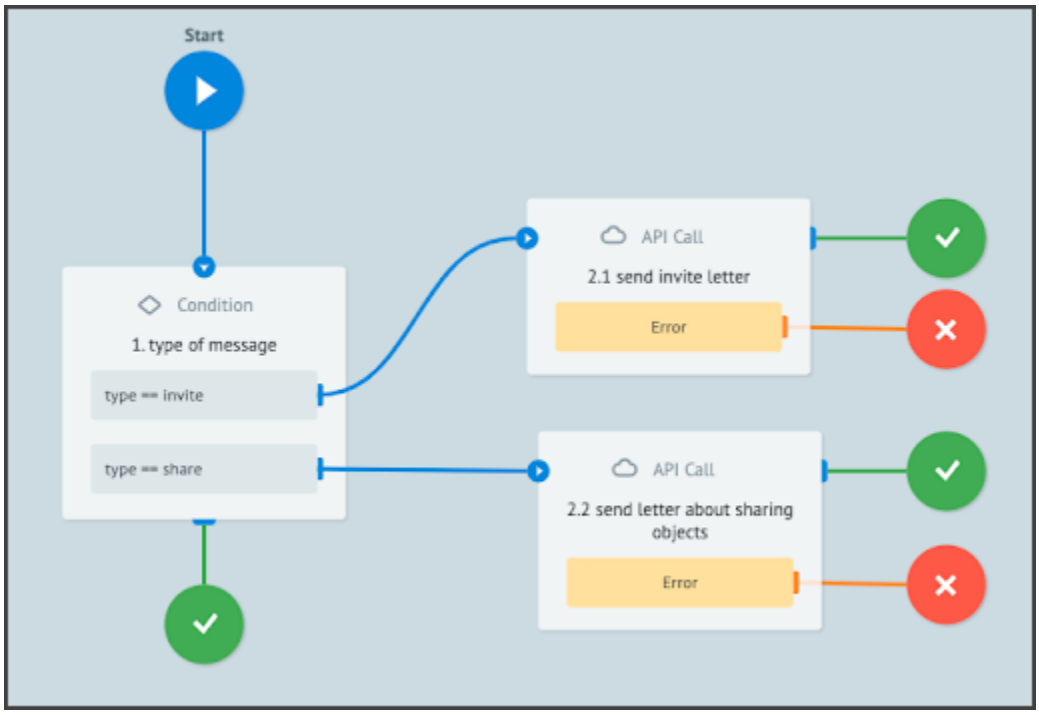
2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Object download	• **type** = download_complete • **recipients** = [{email}] • **action_type** = download (default) • **obj_type** = downloaded object type (possible values: *folder, conv, dashboard, instance, config, alias, project, stage, version, api_gateway*) • **subject** - default email subject • **body** - default email text • **obj_id** - downloaded object identifier • **title** - downloaded object name • **create_time** - download link creation date • **url** - download link
Task download	• **type** = download_complete • **recipients** = [{email}] • **action_type** = download (default) • **obj_type** = file_download (default) • **subject** - default email subject • **body** - default email text • **obj_id** - ID of a process from which the task was loaded • **title** - process name • **create_time** - download link creation date • **url** - download link • **proc** = ok - system parameter

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Task upload error	• **type** = upload_complete • **recipients** = [{email}] • **action_type** = upload (default) • **obj_type** = task (default) • **subject** - default email subject • **body** - default email text • **conv_id** - ID of a process to which the task failed to upload • **name** - process name • **create_time** - creation date of the link for downloading the error file • **url** - link for downloading the error file • **proc** = ok - system parameter

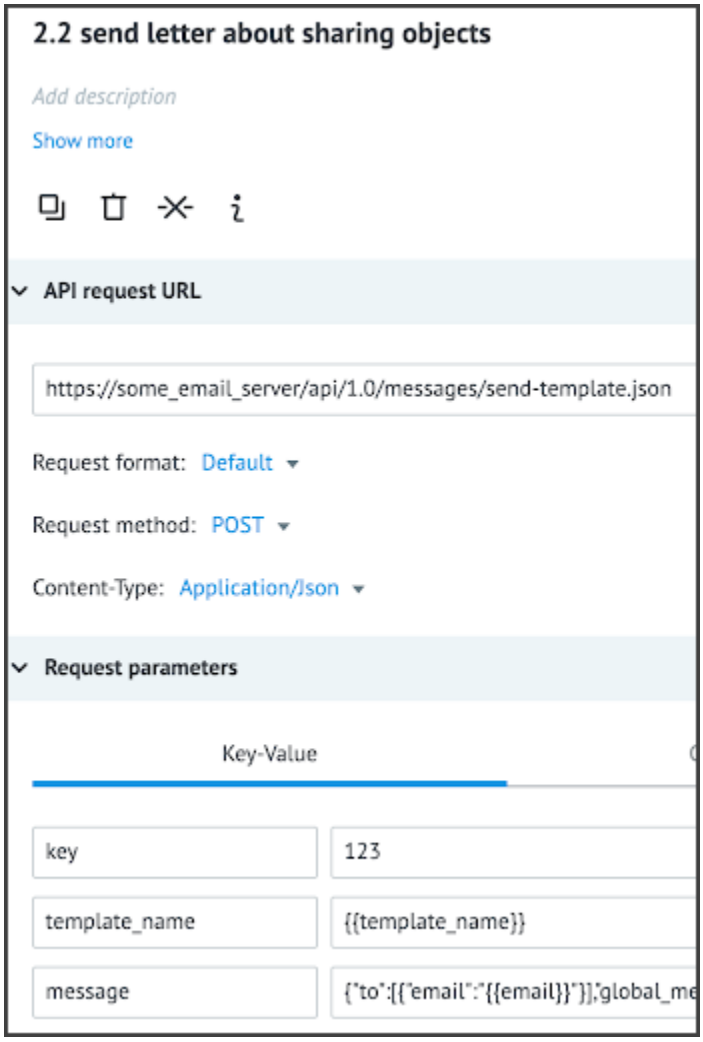
Process example



1. Add the **Condition** node after the **Start** node.
2. In the **Condition** node, configure branching conditions for sending two types of messages: Invite a user to a company (type = invite) and Object sharing (type = share).

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

3. Add the **API Call** node for every message type.
   Example of API Call node configuration.



4. Add an array processing sequence if an email has to be sent to multiple users:

After the **Condition** node, add the **Code** node that will process the array of emails and decrease it each time a task is passed through this node. Paste the following code into the code editor (language: JavaScript):

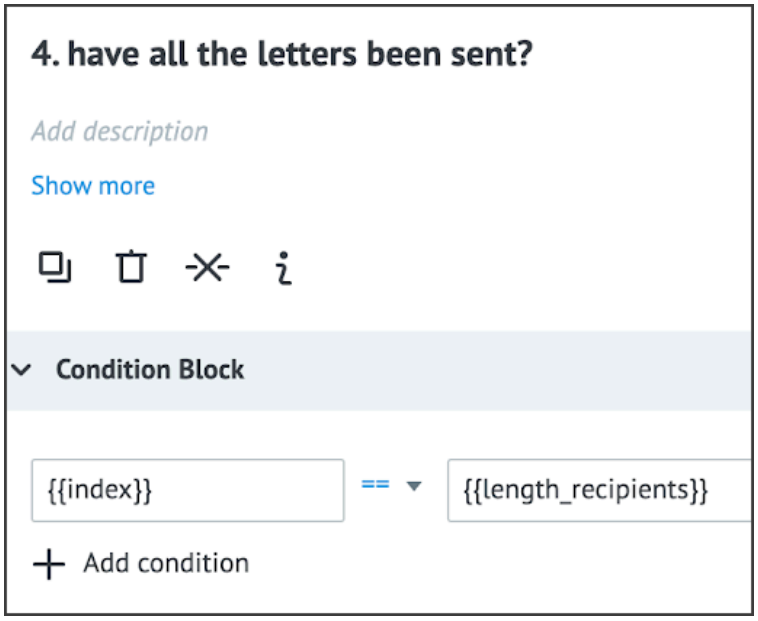
```
if (data.index == undefined) {

data.index = 0;

} else {

data.index = data.index + 1;

}
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

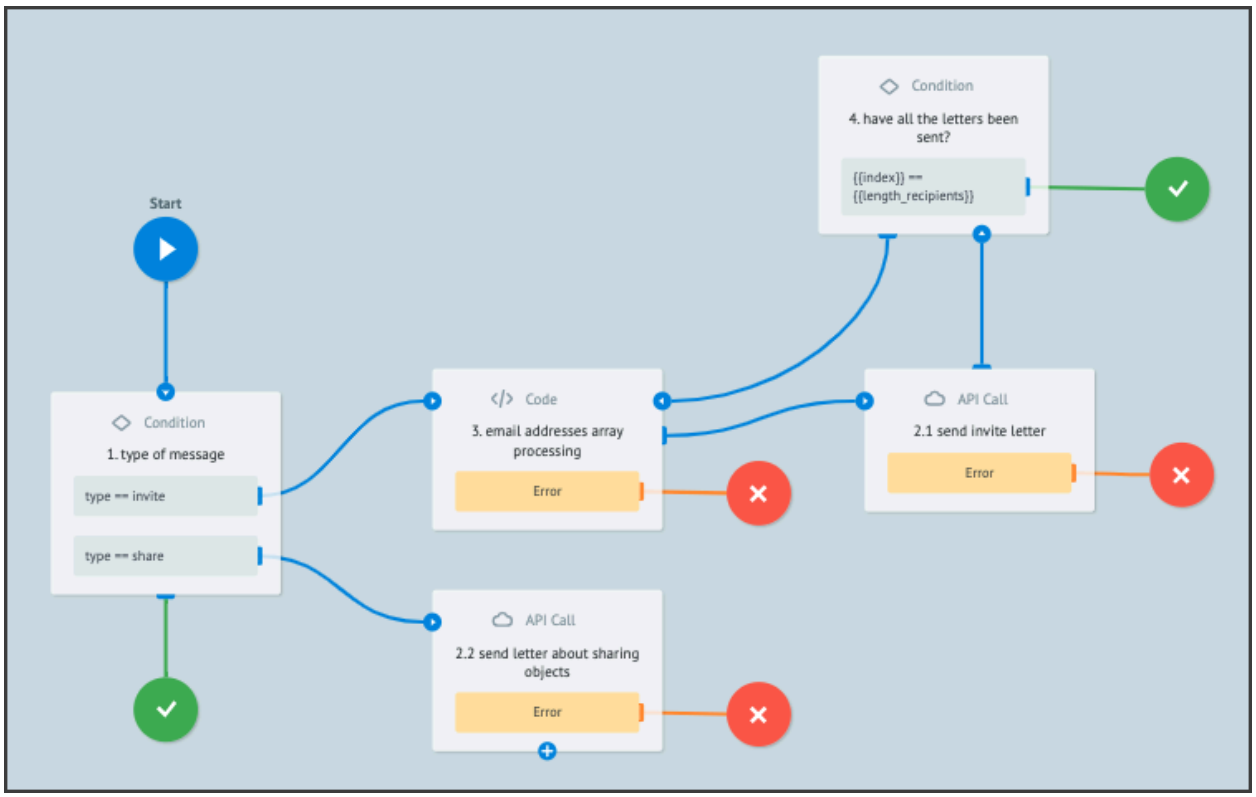
```
data.length_recipients = data.recipients.length - 1;
data.email = data.recipients[data.index].email;
```

a. Connect the **Code** node to the **API Call** node.
b. After the **API Call** node, add another **Condition** node to check the length of the array.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



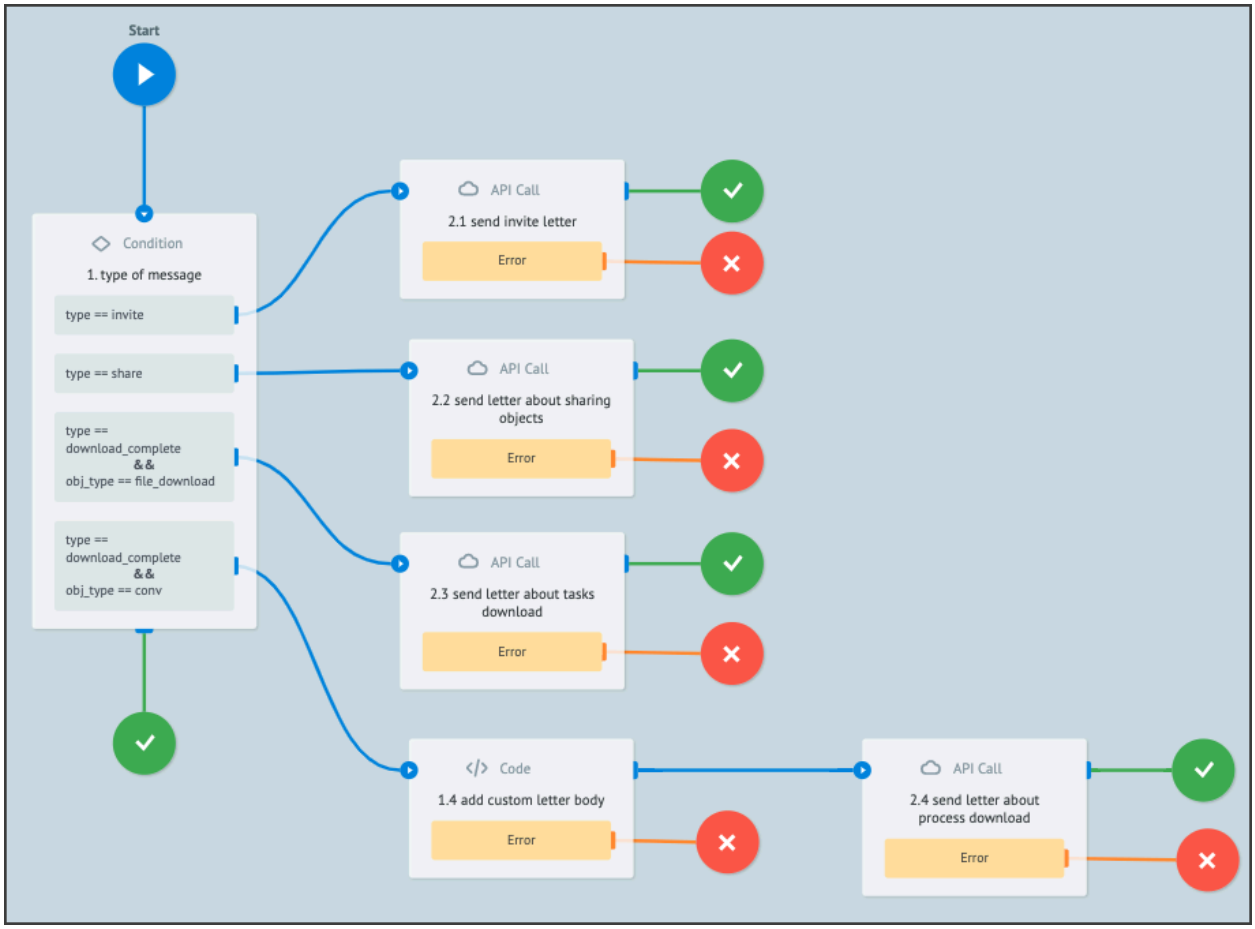
d. Connect the API Call node (2.2) to the Condition node (4).



5. Separate the sequence of sending emails that notify on downloading various objects and make object-specific texts for them: Add additional constituent conditions to node 1, and

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

add the following nodes to the process: 2.3, 1.4, 2.4:



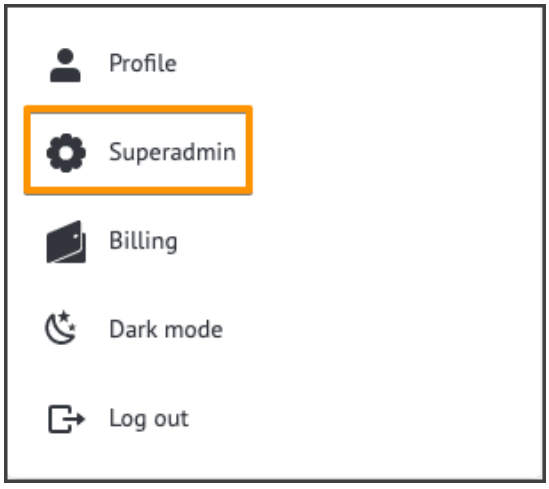
You can supplement the process with individual sequence of nodes necessary for sending emails.

Adding Process ID to settings

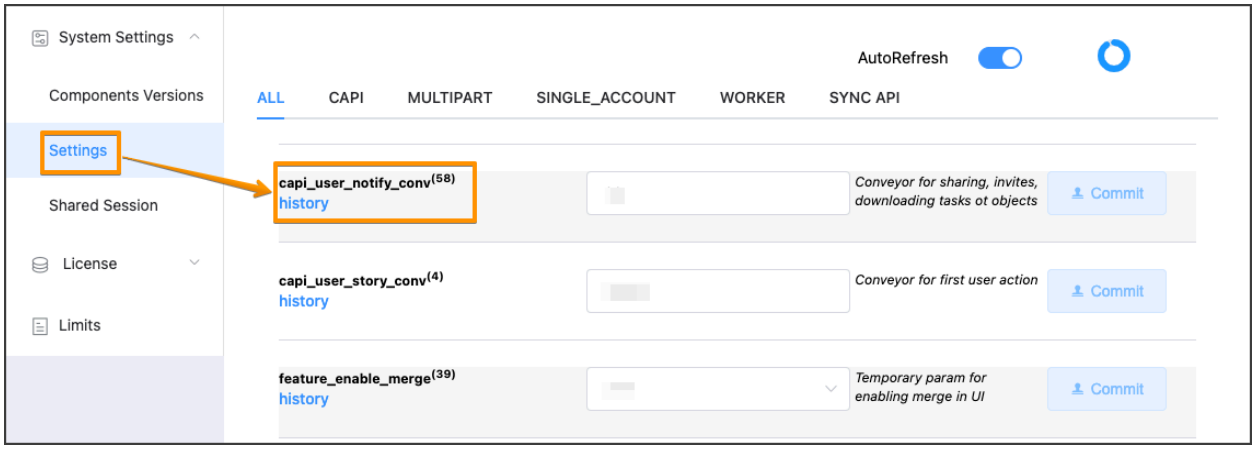
After completing the process configuration, add the process ID to the **capi_user_notify_conv** setting of the **Superadministrator** menu, **Settings** section. To do that:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

1. In the upper-right corner of the page, click your profile image, and in the dropdown list, select **Superadmin**.



2. On the **Settings** tab, enter the process ID in the **capi_user_notify_conv** field.



2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

9.6 Notifying users on process blockin/deblocking

Overview

After installing the Corezoid, you need to create and configure a process that sends notifications on processes blocking/unblocking

Block/unblock users

The feature to block/unblock users is available only for super admins. A notification can be sent to an affected Process owner or another person.

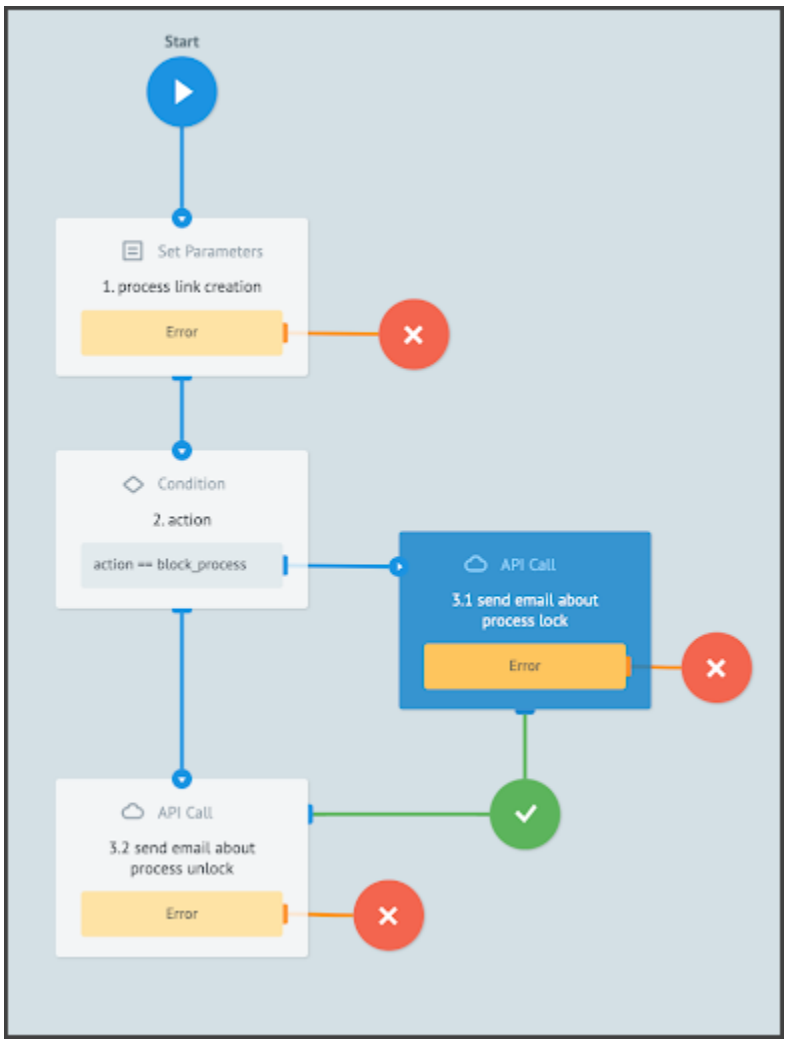
An object with the following parameters is sent to the Process:

- blocked_reason - reason for blocking specified by a superadmin
- action - action type, possible values:
    - block_process
    - unblock_process
- user_id - identifier of a superadmin which blocked/unblocked the process
- login - login of a superadmin which blocked/unblocked the process
- obj = conv (default)
- obj_id - process identifier
- process_owner - process owner login (email)
- process_owner_id - process owner identifier
- lang - process owner language

Use these parameters for creating a notification email body, subject, recipient.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

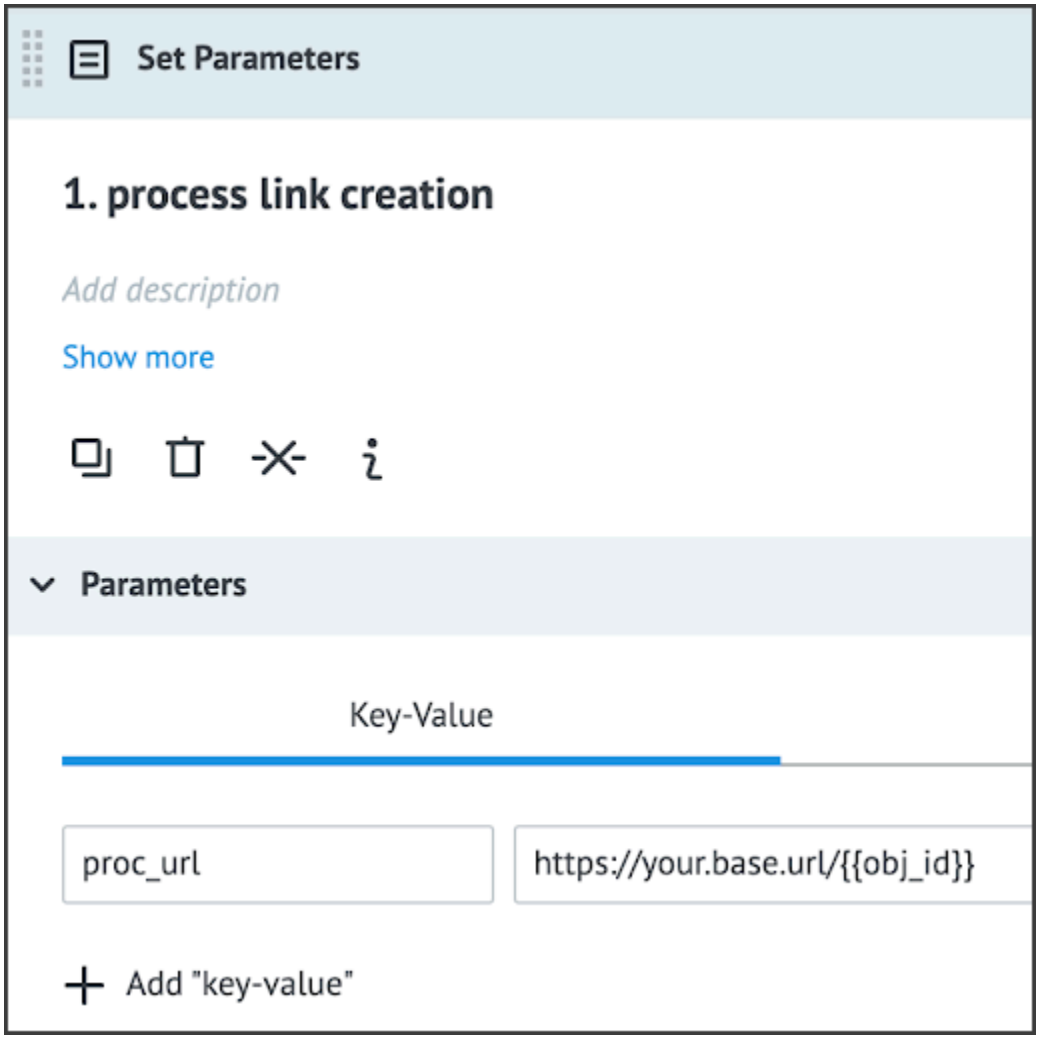
Process example



1. Add the **Set Parameters** node after the **Start** node. This step allows a notification email to open the process directly using this link.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

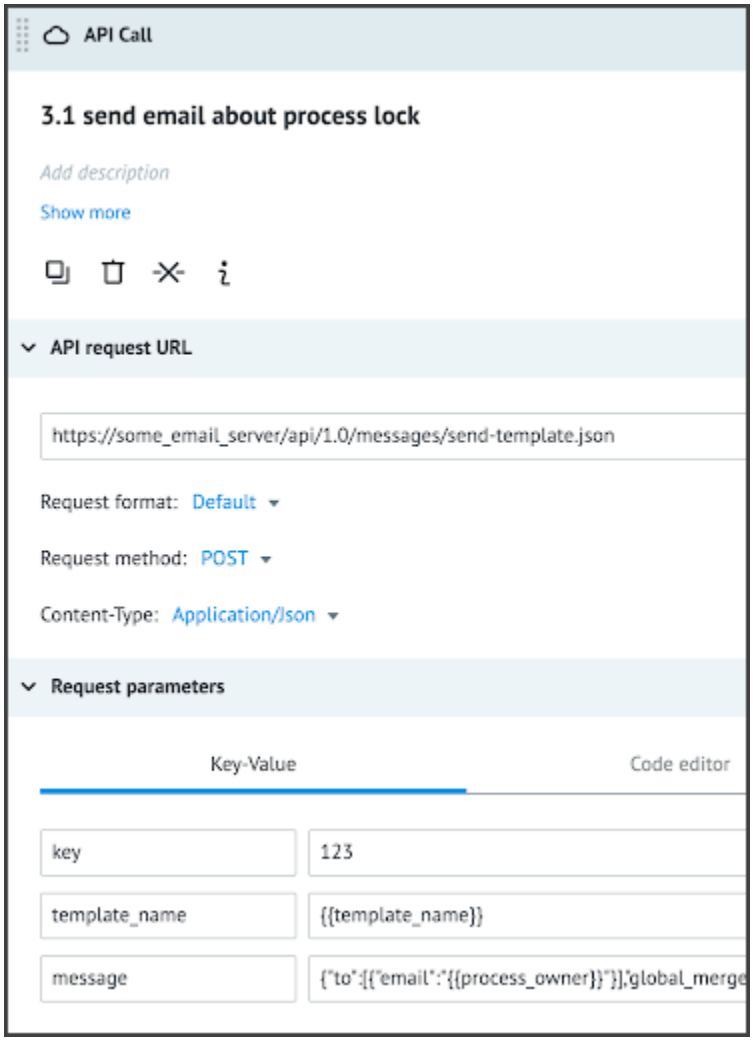
Note: For creating a process link this step is optional.



2. After the **Set Parameters** node, add the **Condition** node where configure branching conditions for sending two types of messages: process block and unblock.
3. Add **API Call** nodes for each message type and connect the **API Call** nodes to the **Condition** node:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Example of the **API Call** node configuration.



You can add an individual sequence of nodes for sending other emails to the process.

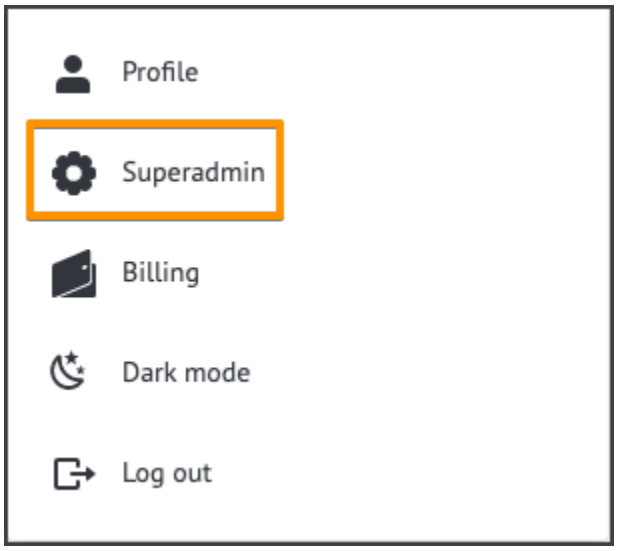
Adding Process ID to settings

After completing the process configuration, add the process ID to the **capi_block_unblock_notify_conv** setting of the **Superadministrator** menu, **Settings** section.

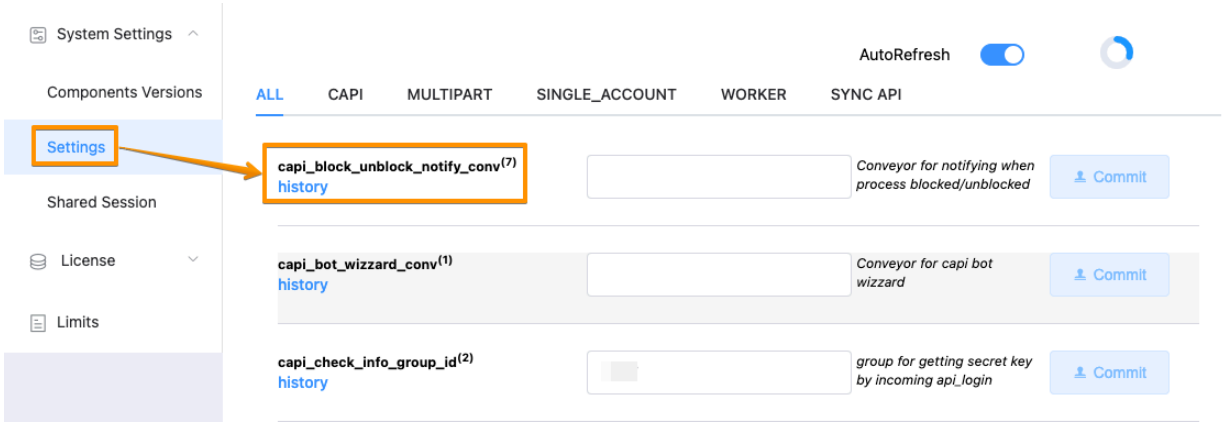
To do that:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

1. In the upper-right corner of the page, click your profile image, and in the dropdown list, select **Superadmin**.



2. On the **Settings** tab, enter the process ID in the **capi_block_unblock_notify_conv** field.



10. Additional resources

[API Gateway]

[Simulator site]

[Corezoid site]

[Simulator API]

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

11. Release Notes

12. Appendices

12.1 Corezoid Glossary of Terms

A

1. **Account:** A space where you can see users of your Workspace and invited ones, created groups and groups you are invited to join, created API keys, and settings.
2. **API Key**: A unique identifier used to authenticate requests made to the Corezoid API.
3. **Alias** : A human-readable identifier replacing numeric IDs for Processes, Dashboards, or state diagrams->An **Alias** is a symbolic name replacing numeric object identifiers, used to interact with Processes and State Diagrams via nodes or the API. Created at the company or stage level, aliases link to specific objects and can be managed only by their owner for secure integration.

B

4. **Bot Wizard**: A tool within Corezoid for creating and managing bots that interact with various messaging platforms.

C

5. **Callback Node**: A node that pauses the Process until a specific external response or action is received.
6. **Chart**: A visual representation of Process data on Dashboards.
7. **Commit**: A saved version of a Process or diagram, used for tracking changes and deploying updates.
8. **Communications Orchestrator:** A set of ready-to-use Processes for creating and managing bots in different messengers.
9. **Company ID**: A unique identifier for organizations using Corezoid.
10. **Condition Node**: The node that creates a condition using the specified parameters and routes tasks in your workflow based on the condition.
11. **Conveyor**: A collection of tasks waiting to be processed in Corezoid. It is used in nodes to track the status of task processing.
12. **Counter:** represents values without any associated actions; it can be system-level or user-defined.
13. **Counters**: Provide statistics on tasks that entered a final (End) node.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

14. **Corezoid API** is the interface that allows developers to interact with Corezoid using a request-response scheme. With the Corezoid API, you can integrate Corezoid's capabilities into your applications, platforms, or systems, adding new features.

D

15. **Dashboard**: A customizable interface for monitoring and visualizing Processes and metrics.
16. **Database:** In Corezoid, it is a connection between the actual database and a Process where you use the Database Call node.
17. **Delay Node**: A node that pauses task execution for a specified amount of time.
18. **Direct URL**: A unique link for sending tasks directly to Processes or State Diagrams.

E

19. **End Node**: A terminal node in a Process that signifies task completion.

F

20. **Folder**: A structure within Corezoid for organizing Processes, Dashboards, and diagrams.
21. **Function:** An operation that needs to be performed on the object from the queue in a node. Functions can be added as an API, program code, another Process, and so on.

G

22. **Git Call Node**: A node that integrates with version control systems like Git for managing Process code.
23. **Group:** A group of users that have similar roles, permissions, or characteristics. You can create and use groups to facilitate the management of users and their access to projects or other objects within Corezoid.

I

24. **Infinite loop:** infinite loop occurs when a task continuously cycles through a sequence of nodes without reaching an End node.
25. **Invite**: A feature for adding new users to a company or workspace within Corezoid.

L

26. **Logic:** represents tools for managing logic within a node. It can be:
    System logic T that specifies the maximum time an object can stay in a node.
    System logic N that specifies the maximum number of objects that can be in the queue in a node.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

M

27. **Merge:** An operation available for Stages and Versions in Projects. It adds data from a stage, version, or file (exported stage or version) to another stage. Thus, you can have a stage or version or a stage/version file as a source and an existing stage as a target.
28. **Modify Task Node**: A node used to alter or update task parameters dynamically within a Process.

N

29. **Node**: A building block of Processes in Corezoid, performing specific actions like API calls, delays, or condition checks.

P

30. **Parameter**: Data input or configuration used by nodes in a Process.
31. **Process**: A sequence of nodes executing defined tasks or workflows.
32. **Profile:** Contains all user personal data including username, email, account ID, personal information, and billing details.
33. **Project**: is a storage that you can create in a Corezoid Workspace to store Processes, State Diagrams, Dashboards, API keys, aliases, folders, and files. Projects are designed for easier management of your objects and are created with preconfigured environments, Stages, by default.

Q

34. **Queue**: A system for managing and storing tasks awaiting execution in Corezoid.
35. **Queue Node**: A node used to enqueue tasks for processing or storage.

R

36. **Request Per Second** (RPS): Maximum number of tasks that can be processed across all Processes per second.

S

37. **Set Parameters Node**: A node type used to assign or modify parameters within a task.
38. **Signature**: A cryptographic element used for secure API authentication.
39. **Stage** is an environment inside a Project where you can store and run Corezoid objects independently of other Stages and Projects. Stages can be modifiable (mutable) and unmodifiable (immutable).
40. **Start Node**: The entry point for tasks entering a Process.
41. **State Change:** A state change occurs when a task transitions between nodes**.**
42. **State Diagram**: State Diagram is the type of Process that stores object states and data. Owing to its design and the fact that there are no restrictions on use in Processes, State

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Diagrams are best for tracking object states, accounting, and storing large amounts of data.

43. **Sum Node**: A node that calculates and returns the sum of specified parameters or values.
44. **Sync API:** An API provided by Corezoid that enables synchronous communication between external systems and Corezoid Processes.

T

45. **Task**: A single unit of work processed through a Corezoid workflow.

U

46. **User ID**: A unique identifier assigned to each user in Corezoid.

V

47. **Variable:** A stage object where data can be stored and then used in specified nodes. Variables store data in the RAW and JSON formats.
48. **Version:** A snapshot of a Stage containing all its objects but not containing its tasks.

W

49. **Webhook**: A user-defined HTTP callback triggered by specific events in Corezoid.
50. **Waiting for Callback Node**: A node that holds a task until a callback is received, allowing asynchronous operations.
51. **Workspace:** The main environment where you work, create projects, send tasks to Processes and State Diagrams, use dashboards to analyze data, and so on.

Notes: This glossary includes commonly used terms in Corezoid workflows and documentation. For more detailed explanations, refer to the Corezoid user guide or API reference.

12.2 Configuration Examples

- See Deployment options.
- Helmchart

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

12.3 API Reference

Corezoid public Open API lets you automate everything you can do in the Corezoid UI — create, modify and query tasks, processes, dashboards, folders, users and other objects. It is versioned (/api/1/… and the newer /api/2/…) and JSON-based. Each call is a single POST that carries an ops[] array of operations (for example type:"create", obj:"task", conv_id:"1234"), allowing you to batch many actions in one request.

Authentication is signature-based rather than token-based: you combine timestamp + secret + payload (SHA-1/MD5 in v1, SHA-256 in v2) to produce the SIGNATURE, and build the URL:

https://api.corezoid.com/api/{version}/json/{api_login}/{timestamp}/{signature}

which the server validates before executing your ops.

The spec has more than 70 endpoints grouped into logical resources such as task, conv (Process), folder, dashboard, user, history, stat, file upload/download/compare/merge, and admin functions. The Open API 2.0 page lists every path, its input schema and sample 200/4xx responses.

The v2 namespace adds multipart upload, object versioning and richer responses while deprecating some v1 fields (e.g., project_id removed from list:history).

#	API Endpoint	Method	Description	Parameters	Example Usage
1	`/api/2/json/{API_LOGIN}/{GMT_UNIXTIME}/{SIGNATURE}`	POST	General API entry point for various operations	`API_LOGIN`, `GMT_UNIXTIME`, `SIGNATURE`	Create aliases, modify settings
2	`/api/2/json/{API_LOGIN}/{GMT_UNIXTIME_1}/{SIGNATURE}`	POST	Alias management	`type`, `obj_id`, `company_id`	Create, modify, delete aliases
3	`/api/2/json/{API_LOGIN}/{GMT_UNIXTIME_4}/{`	POST	Compile code node settings	`conv_id`, `node_id`, `lang`, `src`	Load or get node code

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

	SIGNATURE}				
4	/api/2/json/{API_LOGIN}/{GMT_UNIXTIME_7}/{SIGNATURE}	POST	Create and manage direct URLs	alias_id, conv_id, company_id	Create a Direct URL for uploads
5	/api/2/json/{API_LOGIN}/{GMT_UNIXTIME_8}/{SIGNATURE}	POST	Token/channel management	env, conv_id, company_id, token	Create a new token for a webhook
6	/api/2/json/{API_LOGIN}/{GMT_UNIXTIME_10}/{SIGNATURE}	POST	Chart management	dashboard_id, obj_type, series	Create or modify charts
7	/api/2/json/{API_LOGIN}/{GMT_UNIXTIME_12}/{SIGNATURE}	POST	List commits	conv_id, version, company_id	Get commit history
8	/api/2/json/{API_LOGIN}/{GMT_UNIXTIME_13}/{SIGNATURE}	POST	API key access	type, obj_id, api_key	Show or modify API keys
9	/api/2/json/{API_LOGIN}/{GMT_UNIXTIME_14}/{SIGNATURE}	POST	Company groups and user management	group, filter, company_id	List users, manage groups
10	/api/2/json/{API_LOGIN}/{GMT_UNIXTIME_17}/{SIGNATURE}	POST	Retrieve node settings	API_LOGIN, GMT_UNIXTIME, SIGNATURE	Access node configurations
11	/api/2/json/{API_LOGIN}/{GMT_UNIXTIME_22}/{SIGNATURE}	POST	List Processes and diagrams	ops	Search Processes by pattern
12	/api/2/json/{API_LOGIN}/{GMT_UNIXTIME_38}/	POST	List nodes	API_LOGIN, GMT_UNIXTIME,	Retrieve all nodes in a workspace

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

	{SIGNATURE}			SIGNATURE	
13	/api/2/json/{API_LOGIN}/{GMT_UNIXTIME_54}/{SIGNATURE}	POST	Retrieve system statistics	API_LOGIN, GMT_UNIXTIME, SIGNATURE	Access usage stats for Processes
14	/api/2/json/{API_LOGIN}/{GMT_UNIXTIME_64}/{SIGNATURE}	POST	Stop upload operations	ops	Interrupt ongoing uploads
15	/api/2/json/{API_LOGIN}/{GMT_UNIXTIME_15}/{SIGNATURE}	POST	Create or list companies	ops	Manage company objects
16	/api/2/json/{API_LOGIN}/{GMT_UNIXTIME_29}/{SIGNATURE}	POST	Global counters management	ops	Retrieve counters like users, conveyors
17	/api/2/json/{API_LOGIN}/{GMT_UNIXTIME_32}/{SIGNATURE}	POST	View object and operation history	ops	Retrieve logs for objects or users
18	/api/2/json/{API_LOGIN}/{GMT_UNIXTIME_35}/{SIGNATURE}	POST	Invite user management	ops	Manage user invitations

Table 49: API call examples.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

1. Create task (by conv_id)

Request


```
{
  "ops": [
    {
      "type": "create",
      "conv_id": 123456,
      "obj": "task",
      "company_id": "i111111222",
      "data": {
        "foo": "bar"
      },
      "ref": "123498765678"
    }
  ]
}
```


Response


```
{
  "request_proc": "ok",
  "ops": [
    {
```


**2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.**

```
      "id": "",
      "proc": "ok",
      "obj": "task",
      "ref": "123498765678",
      "obj_id": "1123xe556ger645t245cg"
    }
  ]
}
```


2. Modify task

Request


```
{
  "ops": [
    {
      "type": "modify",
      "conv_id": 123098,
      "obj": "task",
      "company_id": "i543378975",
      "data": {
        "foo": "bar"
      },
      "ref": "15989649323554"
```


2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
    }
  ]
}
```

Response

```
{
  "request_proc": "ok",
  "ops": [
    {
      "id": "",
      "proc": "ok",
      "obj": "task",
      "ref": "15989649323554",
      "obj_id": "5f4e44c682ba9635bf66e1d7"
    }
  ]
}
```

3. Delete task

Request

```
{
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

```
  "ops": [
    {
      "type": "delete",
      "obj": "task",
      "conv_id": 234234,
      "company_id": "i260999999",
      "node_id": "g4342ij44goer09409w0e0мd",
      "obj_id": "6304dd15094bab211e00042c"
    }
  ]
}
```

Response

```
{
 "request_proc": "ok",
 "ops": [
   {
     "id": "",
     "proc": "ok",
     "obj": "task",
     "obj_id": "6304dd15094bab211e00042c"
   }
 ]
}
```

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

12.4 Error Codes and Troubleshooting

Git Call Node Errors

Type: software or hardware	Tag	Description	Cause	Solution
software	git_call_return_format_error	dynamic description	Incorrect response format from GIT Call node.	Check Git Call parameters
software	git_call_executing_error	dynamic description	Incorrect code returned from GIT Call node.	Check GIT Call code-related parameters
software	git_call_is_not_supported	Git Call v1 no longer supported	Using unsupported version 1 of GIT Call node.	Use GIT Call v2
hardware	git_call_fatal_error	dynamic description	Hardware error.	

Table 50: **Node errors.**

API Call Node Errors

Type: software	Tag	Description	Cause	Solution

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

or hardware				
hardware	api_expire_query	Expire task to API, please check your api for correct answer, or increase max_threads for connections	Your API query has expired: The API has taken too long to respond. The number you set in the Limit the number of simultaneous requests to the API field has been exceeded.	
hardware	api_bind_error	Error bind params to http query	The request parameters you defined in the Key and Value fields could not be added to the URL to form a request.	
software	api_bad_answer	dynamic description	The node did not receive a response from API or failed to establish a connection with it after 60 seconds.	
software	api_wrong_convert_param	dynamic description: Param: ~ts, Value: ~ts, Try convert to: ~ts	Parameter value failed to convert with the specified parameter type.	

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

software	api_size_overflow_limit	Your task size: ~p bytes...	The size of the task data exceeds the specified limit set for your environment.	Reduce the task data volume
software	api_no_valid_json	dynamic description	The request header or body is formatted incorrectly.	
software	api_no_request_proc	Bad request_proc tag	Incorrect tag "request_proc" is set.	
software	api_no_ops	No or bad ops	The API response does not contain an ops block or contains an incorrect ops block.	
hardware	api_fatal_error	Error running api	There was an internal process error while executing the API request.	
software	api_bad_certificate	Check your certificate file		

Table 51: **Node errors.**

Copy Task Node Errors

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Type: software or hardware	Tag	Description	Cause	Solution
hardware	copy_task_fatal_error	Error running copy task	The Copy Task node error.	
hardware	copy_task_timeout	timeout for executing copy_task request	Timeout when performing task copying.	
software	not_unical_ref	not unical reference	The task reference is not unique.	
software	access_denied	It's not allowed to copy task between stages	You are trying to send or receive tasks from another stage.	
software	not_found_task	not found task	No task with specified reference exists in the Process.	
software	duplicate_callback	The last one callback is still running	You are sending the next task with same reference while first one still in Callback node.	Wait until the first task leaves the Callback node.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

software	copy_task_size_overflow_limit	Your task size exceeds limit	The size of the task exceeds the limit	

Table 52: **Node errors.**

RPC Node Errors

Type: software or hardware	Tag	Description	Cause	Solution
software	not_unical_ref	not unical reference	Task reference is not unique.	
software	access_denied	Not allowed to call Process between stages	You are trying to send or receive tasks from another stage.	
software	conveyor_is_not_active	conveyor is not active	The target Process is not active.	
software	not_found_main_node	not found start node	The Start node is missing in Process you're replying to.	
hardware	crash_api	Error rpc request	Error sending task to Process with Reply to Process node.	

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

hardware	rpc_task_fatal_error	Error running rpc task	Error sending task to	

Table 53: **Node errors.**

Code Node Errors

Type: software or hardware	Tag	Description	Cause	Solution
hardware	code_timeout	timeout for executing code	Code has been running for too long.	
hardware	code_fatal_error	Error running code	There is a hardware error.	
software	code_executing_error	dynamic description	Your code contains an error.	
software	code_return_format_error	dynamic description	Code has return statement not in function or returns non-object value.	
software	code_return_size_overflow	Task size exceeds limit	The size of your code exceeds the limit.	Reduce the code size

Table 54: **Node errors.**

Git Call Node Errors

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Type: software or hardware	Tag	Description	Cause	Solution
software	git_call_return_format_error	dynamic description	Incorrect response format from GIT Call node.	Check Git Call response format settings
software	git_call_executing_error	dynamic description	Incorrect code returned from GIT Call node.	Check the Git Call code execution
software	git_call_is_not_supported	Git Call v1 no longer supported	Using unsupported version 1 of GIT Call node.	Use Git Call v2
hardware	git_call_fatal_error	dynamic description	Hardware error.	

Table 55: **Node errors.**

Database Call Node Errors

Type: software or hardware	Tag	Description	Cause	Solution

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

software	db_call_return_format_error	dynamic description	Incorrect response format from Database Call node.	Check the Database Call format settings
software	access_denied	Not allowed to run SQL between stages	Trying to connect to database from another stage.	
software	db_call_executing_error	dynamic description	Incorrect code returned from Database Call node.	Check the Database Call
hardware	db_call_fatal_error	dynamic description	Hardware error.	

Table 56: **Node errors.**

Queue Node Errors

Type: software or hardware	Tag	Description	Cause	Solution
software	queue_wrong_convert_param	Parameter conversion error	Parameter value failed to convert.	
software	access_denied	Access denied	No access to Process.	

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

software	queue_wrong_logic	No queue logic	Queue node missing in Process.	Add a Queue node.

Table 57: **Node errors.**

Modify Task Node Errors

Type: software or hardware	Tag	Description	Cause	Solution
software	modify_task_size_overflow_limit	Size limit exceeded	Task data exceeds environment limit.	
software	access_denied	Access denied	No access to Process.	
software	not_found_task	Task not found	Task doesn't exist in Process.	

Table 58: **Node errors.**

Get Task Node Errors

Type: software or hardware	Tag	Description	Cause	Solution
hardware	get_task_fatal_error	Error running get task	Hardware error.	

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

software	conveyor_is_not_active	Process not active	Target Process inactive.	
software	get_task_wrong_convert_param	Parameter conversion error	Parameter value failed to convert.	
hardware	crash_api	Error get_task request	Error retrieving task.	

Table 59: **Node errors.**

12.5 Integration Guides

See Tutorials

12.6 Security Best Practices

- Logging policy
- Authentication

12.7 Configuration Notes

Corezoid Superadmin

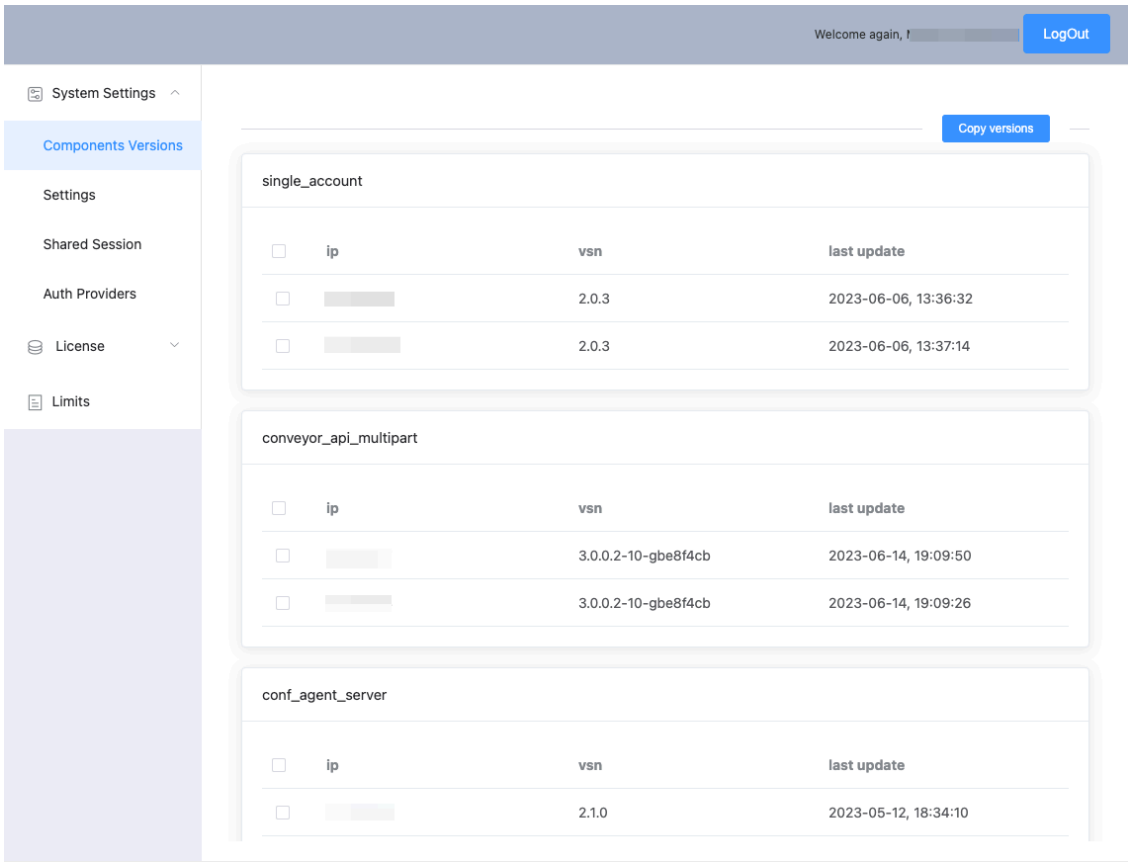
General Info

Corezoid Superadmin is an application designed for Corezoid instance administrators. It allows configuring certain system parameters through the interface instead of the configuration file. With Corezoid Superadmin, administrators can view the current versions of application components and their change history. Additionally, it enables administrators to log in as another user and manage the license file.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

System Settings section

On the **Components Versions** page, you can see a list of installed components in the Corezoid application along with their version history, including the current version:



Screenshot 412: **Superadmin panel.**

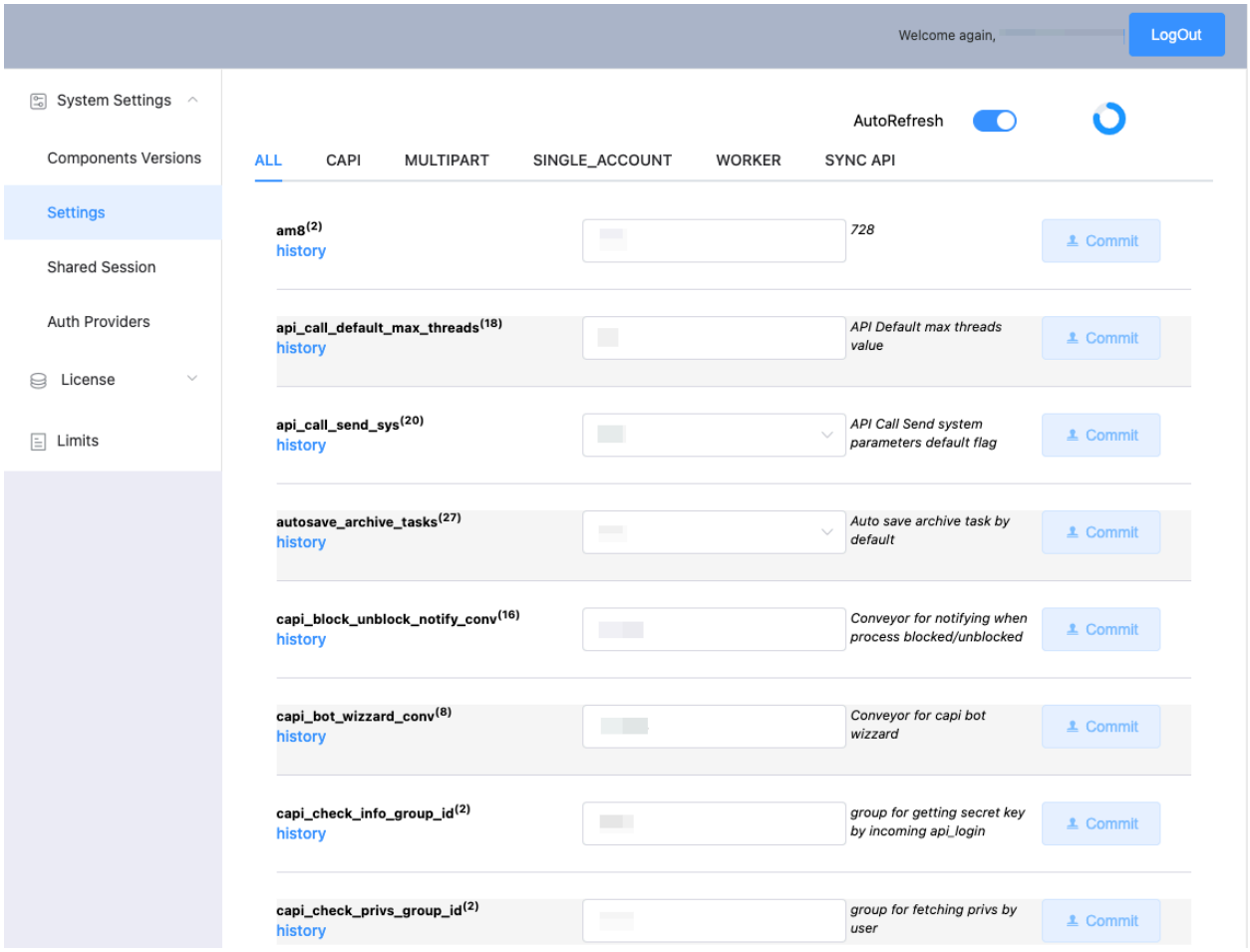
The **Copy versions** button allows you to copy the entire version history. This data is necessary for the Middleware team when analyzing bug reports and incidents.
You can also delete old component version history records by following these steps:

1. Select the records you want to delete;
2. Click the **Delete** button that appears at the top of the table, next to the **Copy versions** button;
3. In the pop-up window, click the **I understand all risks** button.

Settings

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

On the **System Settings** page, you will find a list of system parameters that can be modified on the fly without updating the configuration file or reloading the application.



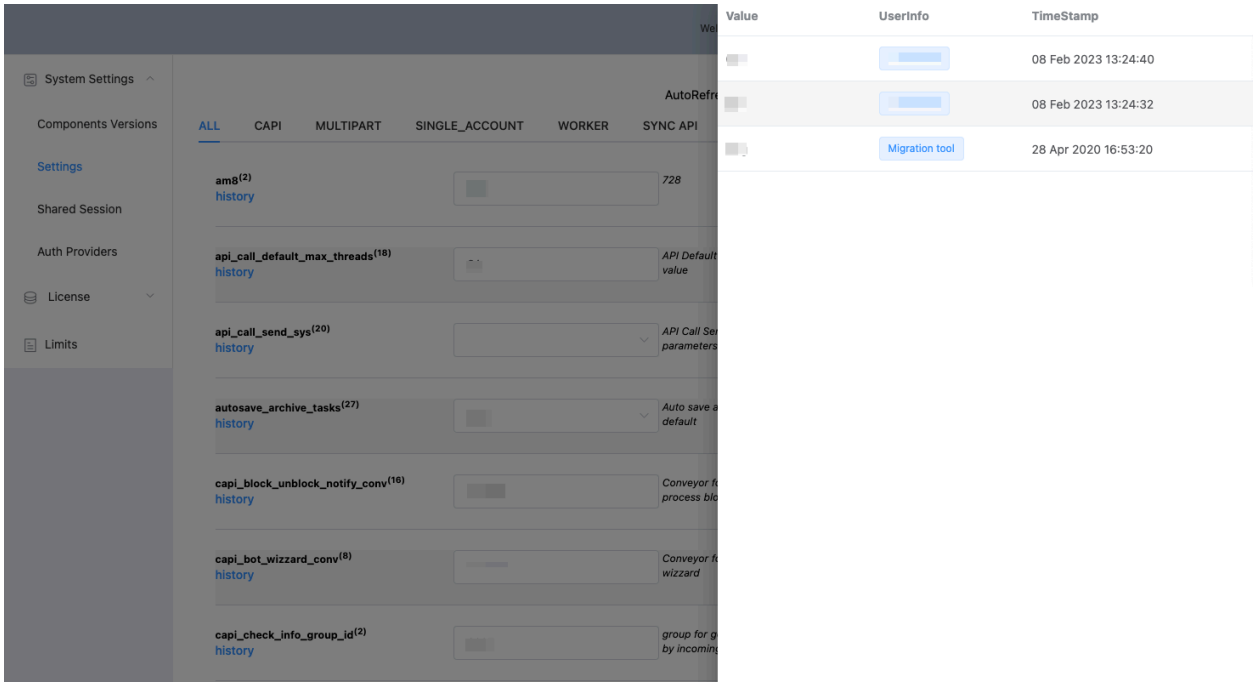
Screenshot 413: **Superadmin panel: Settings.**

To apply a new value to a parameter:

1. Find the desired parameter in the list;
2. Modify the parameter value in the corresponding field;
3. Click the **Commit** button to the right**.**

You can also view the history of parameter value changes, including who (UserInfo), when (TimeStamp), and how (Value) the changes were made. The change history can be accessed in the side menu by clicking on the **history** button under each parameter name:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.



Screenshot 414: **Superadmin panel: history.**

The list of the system parameters, which you can change via the UI without updating the config file and reloading the application. For each parameter, the changes history is saved.

Parameter	Description
api_call_default_max_threads number	The default maximum number of threads to an external API. Used in the API Call node (Other - Limit the number of simultaneous requests to the API)
api_call_send_sys bool	The default checkbox value for the API Call node (Other - Send system parameters). When checked, system parameters are added to the request URL ('conv_signature', 'conv_time, conv_id')

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

autosave_archive_tasks bool	The default checkbox value for saving the archive task in the End node (Options - Save task archive)
billing_process_control atom	The parameter that manages Processes based on the 'billing_manager' settings. When set to 'corezoid_internal', autoblocking will not be enabled. The blocking feature is activated only when the following three parameters are set to 'single_space': 'companies_manager', 'billing_manager', and 'billing_process_control'. For other values, the feature is disabled, meaning autoblocking and auto-unblocking are turned off. **Allowed Values**: corezoid_internal (default), single_space
capi_share_api_keys_in_company bool	The flag that allows all company users to access and use API Keys and API Secret capi_share_api_keys_in_company = true company users can view, delete, or copy any API key capi_share_api_keys_in_company = false company users can view, or copy only the keys created by the user and they can not delete any keys Company owners or superadmins can view, delete, or copy any keys, regardless of the value of the 'capi_share_api_keys_in_company' parameter.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

capi_site_box_solution bool	ON-PREMISES delivery option flag Enables/disables certain functionality If flag = true: a) elastic search is disabled, search will be performed directly in the database instead b) companies can't be created c) users can't be invited d) each user can see other users
capi_site_auth_corezoid bool	Enables/disables Corezoid authorization
capi_site_auth_single_account bool	Enables/disables Single Account authorization
capi_site_oauth_google bool	Enables/disables Google authorization
capi_site_oauth_ldap bool	Enables/disables LDAP authorization

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

capi_site_oauth_pb bool	Enables/disables Privatbank authorization
feature_enable_merge bool	Temporary param for enabling merge-related UI
worker_write_history_task bool	Enables/disables the feature of saving the history of a task movement through a Process to the database.
capi_check_info_group_id number	Group ID for getting API secret key by API user login. Users that are added to this group are applications. The group is used for the correct operation of system components.
capi_check_privs_group_id number	Group ID for checking user rights. The users added to this group are applications. The group is used for maintaining the operation of system components.
capi_service_desk_group_id number	User group ID with the right to view all Corezoid objects
capi_super_admin_group_id number	User group ID (superadmins) with full rights to any operations on Corezoid objects

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

capi_sa_client_id string	Identifier of a client, which uses the Single Account application
capi_sa_client_secret string	Secret key of a client, which uses the Single Account application
capi_send_sms_conv string	Irrelevant setting. Will be removed in future versions
mult_key_login_id number	API user login for the Multipart component (in conjunction with 'mult_key_secret' setting) operation. When the system is being deployed, this setting is configured automatically.
mult_key_secret string	API secret key for the Multipart component (in conjunction with 'mult_key_login' setting) operation. When the system is being deployed, this setting is configured automatically.
sa_key_login_id number	API user login for the Single Account application (in conjunction with 'sa_key_secret' setting) operation. When the system is being deployed, this setting is configured automatically.
sa_key_secret string	API secret key for the Single Account application (in conjunction with 'sa_key_login_id' setting) operation. When the system is being deployed, this setting is configured automatically.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

syncapi_key_login_id number	API user login for the Sync API component (in conjunction with 'syncapi_key_secret' setting) operation. When the system is being deployed, this setting is configured automatically.
syncapi_key_secret string	API secret key for the Sync API component (in conjunction with 'syncapi_key_login_id' setting) operation. When the system is being deployed, this setting is configured automatically.
capi_block_unblock_notify_conv number	Identifier of the Process, which notifies users of blocking/unblocking other Processes. Blocked/unblocked status change occurs in the workspace of a specific Process, and setting these statuses is available only for a superadmin user. A Process owner receives an email notification on blocking or unblocking the Process. If the Process ID is not specified, locking/unlocking is still can be done. The Process is shipped to customer companies and is configured individually using the customer's mail server.
capi_user_activity_conv number	Identifier of the Process, which logs user operations on Corezoid objects. The Process is used to track user actions (blocking, sharing, creating a new Process, etc) for corezoid.com. Not shipped to customers
capi_user_notify_conv number	Identifier of the Process, which sends emails when an events occur. The Process is shipped to customer companies and is configured individually using the customer's mail server. The events that trigger email sending: inviting a new user, sharing an object with a user, registration confirmation, objects downloading, tasks importing from csv files, and password recovery.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

capi_telegram_conv number	Identifier of the system Process (SP) that creates a link between a bot webhook and a Process. The SP is shipped to customer companies. The SP receives a request (create channel) for linking another Process as a webhook to instant messengers. Having received the request, SP sends a request to the selected messenger API, which contains a direct_url for receiving events from the messenger. When an event occurs, the messenger sends a request to the direct_url.
capi_payment_logs_conv number	Identifier of the Process, which handles user plan purchase or upgrade events. The Process is used only for corezoid.com. Not shipped to customers.
worker_billing_conv_id number	Identifier of the Process, which handles consumption of tasks state changes (tacts)/tasks movement between Process nodes. The Process is used only for corezoid.com. Not shipped to customers.
capi_register_event_conv number	Identifier of the Process, which handles user registration. The Process sends a registration notification email to the user and sets a limit of tacts. Used for corezoid.com marketing cases. Not shipped to customers.
capi_user_story_conv number	Identifier of the Process that registers a user's very first action in Corezoid (create, modify, delete objects, etc.). The Process is used only for corezoid.com. Not shipped to customers.
capi_bot_wizzard_conv number	Identifier of the Process, which creates a Bot Platform. The Process is shipped to customer companies optionally (at the company's request).

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

mapi_register_event_conv number	Identifier of the Process, which creates a company in Corezoid, if the company was created in Sender.
capi_notify_change_owner_conv number	Identifier of the Process, which notifies of a Process owner change. If the Process ID is not specified, the change of ownership is still possible.
capi_cookie_expr_time time (ms)	CAPI cookie expiration time (ms). If the user has not performed any action in the Corezoid UI within the specified time, the user is logged out of the system. Used only for the internal optional authorization module (contained in the CAPI component).
capi_cookie_secret string	CAPI requests signature salt
capi_max_stored_versions_per_project number	Max number of versions stored for a project. This number refers to Processes (workspace/project) versions and not Corezoid instance versions. The default value is 10.
capi_password_block_lifetime_secs	Password lifetime, after the expiration of which the user is blocked. Only for plain email auth users.
capi_password_left_start_notifications_secs	Time of sending notifications to users on their password expiration (send N seconds before expiration).

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

capi_password_salt string	Salt for hashing user password in the DB.
capi_sa_redirect_uri string	Redirect uri is a URL, to which a user browser is redirected after a single account authorization.
capi_site_auth_corezoid_lock_time_min number (minutes)	Time of login blocking for a user after the max failed login attempts was reached. Active only for capi_site_auth_corezoid_max_login_attempts_strategy = lock, set in minutes.
capi_site_auth_corezoid_max_login_attempts_strategy enum	Type of protection from incorrect password entry: captcha display or lock login
capi_site_first_day_of_week enum	First day of the week according to a country standard, which is used in time intervals in dashboard charts.
mult_key_api_throughput_tasks number	Max tasks count for a one-time download in csv file (from a node).
worker_float_decimal_count	Parameter, which allows configuring the number of digits after the comma

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

number	

Table 60: **Superadmin table parameters.**

- Superadmin panel
- Admin configuration
- System maintenance

12.8 Performance Tips

- Monitoring.

12.9 User Agreements and Policies

- Customer Agreement
- Privacy Policy
- Cookie Policy

12.10 Code Samples

- Git Call
- DB Call
- Corezoid Repo

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

12.11 Corezoid SDK

The Corezoid SDK is a lightweight, open-source client library (available in PHP, Java, Python, Go, Erlang) that wraps Corezoid's REST endpoints so developers can queue, add, modify or query "tasks" in a Corezoid process with just a few method calls; it automatically builds the signed URLs required by the platform, handles JSON serialization, retries and error parsing, and exposes convenience helpers (ping, bulk upload, token refresh, log download) while staying dependency-free and compatible with everything from legacy PHP 5.3 to modern PHP 8, making it the fastest way to embed Corezoid's event-driven workflow engine into any backend or microservice.

You can find Corezoid SDKs in different programming languages and the associated documentation on the Corezoid Git repository.

Corezoid PHP SDK

`Corezoid.php` is the **single class that implements the entire Corezoid PHP SDK**.
It wraps Corezoid's REST API (upload/modify tasks, ping, sign-check) and hides the details of URL signing, cURL calls and JSON handling so that you can push data to Corezoid with a few lines of code.

corezoid.php file content:

Field	Value	Meaning
`name`	`corezoid/corezoid`	Package ID on Packagist (`vendor/package`)
`type`	`library`	Marks it as a reusable library

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

`description`	`SDK for Corezoid`	Short description
`keywords`	`["corezoid"]`	Search tags
`homepage`	GitHub repo	Source URL
`version`	`1.0.0`	Current version (hard-coded)
`license`	`OSL 3.0`	Open Software License v3
`authors`	Corezoid, support@…	Author info for Packagist
`require`	`"php": ">=5.3.0"`	Minimum PHP version
`autoload.classmap`	`["./Corezoid.php"]`	Tells Composer to load *Corezoid.php*

Table 61: **corezoid.php file content.**

The *composer.json* file in **corezoid/sdk-php** is the “passport” of the **Corezoid SDK** PHP library. Composer reads it to know how to install the package, which PHP versions it supports, and how to autoload the *Corezoid.php* class.

composer.json file content:

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Field	Value	Meaning
`name`	`corezoid/corezoid`	Package ID on Packagist (`vendor/package`)
`type`	`library`	Marks it as a reusable library
`description`	`SDK for Corezoid`	Short description
`keywords`	`["corezoid"]`	Search tags
`homepage`	GitHub repo	Source URL
`version`	`1.0.0`	Current version (hard-coded)
`license`	`OSL 3.0`	Open Software License v3
`authors`	Corezoid, support@…	Author info for Packagist
`require`	`"php": ">=5.3.0"`	Minimum PHP version
`autoload.classmap`	`["./Corezoid.php"]`	Tells Composer to load *Corezoid.php*

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.

Table 62: **composer.json file content.**

Visit the Corezoid PHP SDK git for more details.

2025 © Corezoid Inc.,
2093 Philadelphia Pike #2235 Claymont 19703 DE, US.
All rights reserved.